Geely Automotive Research Institute (Ningbo)Co.Ltd	Document Release Status: **RELEASED**	
	Release Date: **2018-12-19**	Version: **(2)**
	Document No: **1234567890**	Revision: **002**
Geely Automotive Research Institute	Volume No: **01**	Page No: 1 (8 7 9)
Document Type: **Note-SWRS**	Reference No:	

Document Name

Base Technology SWRS

GEEA2.0

xxx

Issued by	Approved by
Yu Jiaqi	**Zhu Zhenglong**

Base Tech Software Requirement Specification
IEM

Contents

0 REQUIREMENT LOG 18

0.1 List of requirements 18

0.2 Requirements change log 44

0.2.1 New requirements in this issue 44

0.2.2 Updated requirements in this issue 45

0.2.3 Modifications in the updated requirements in this issue 45

0.2.4 Deleted requirements in this issue 45

1 LIST OF REQUIREMENTS WITH DEVIATIONS 46

2 INTRODUCTION 46

2.1 What is the purpose of this document? 46

2.2 Who should read this document? 46

2.3 How is this document organized? 46

2.4 How do you receive more information? 46

2.5 References to Volvo Cars 46

3 REVISION HISTORY 46

4 ECU SOFTWARE REQUIREMENTS 47

4.1 Car Configuration 48

4.1.1 Central Car Configuration 48

4.1.1.1 Car Configuration Slave 48

4.1.1.1.1 Central Car Configuration Requirements 48
4.1.1.1.1.1 Car Configuration Parameter Data 49
4.1.1.1.1.1.1 Parameter Creation 50
4.1.1.1.1.1.2 Parameter Properties 50
4.1.1.1.1.1.3 Information in CCdB 50
4.1.1.1.1.1.4 Parameter Subscription 50
4.1.1.1.1.2 System Design 51
4.1.1.1.1.3 Car Configuration Subscriber Requirements 51
4.1.1.1.1.4 Signals provided by the system 56

4.1.1.1.2 Appendix 58
4.1.1.1.2.1 Guideline subscriber states 58
4.1.1.1.2.1.1 Guideline ECU Bulk State 58
4.1.1.1.2.1.2 Guideline ECU Valid Config State 59
4.1.1.1.2.2 Table VCP signal mapping 60
4.1.1.1.2.3 Table Car Configuration DTC DID record for CC Subscriber ECUs61

4.1.1.1.3 Definitions and Abbreviations 62

4.1.1.1.4 Requisite documents 63

4.1.1.1.5 Revision History 63
4.1.2 Overview of Vehicle Configurations and Customer Settings 64
4.1.2.1 Overview of Vehicle Configurations and Customer Settings 64
4.1.2.1.1 Vehicle Configuration 64
4.1.2.1.1.1 Configuration Using Global Parameters 64
4.1.2.1.1.2 Configuration Using Local Parameters 65
4.1.2.1.1.3 Special cases of the Local parameters 65
4.1.2.1.2 Customer Settings 65
4.1.2.1.2.1 Types of Customer Settings 66
4.1.2.1.3 Summary of Configuration Types 66
4.1.2.1.4 Introduction 66
4.1.2.1.4.1 Document Purpose 66
4.1.2.1.5 Definitions and Abbreviations 66
4.1.2.1.6 Background 67
4.2 Diag & ECU Platform - proj reqs 68
4.2.1 SRD Diagnostics and ECU Platform 68
4.2.1.1 Diagnostics and ECU Platform 68
4.2.1.1.1 Introduction 68
4.2.1.1.1.1 Definitions & Abbreviations 69
4.2.1.1.1.2 Requisite documents 69
4.2.1.1.1.3 Requirement Overview 70
4.2.1.2 Hardware Platform 74
4.2.1.2.1 Requirements 74
4.2.1.3 ECU Addresses 75
4.2.1.3.1 Requirements 75
4.2.1.4 CAN Network 84
4.2.1.4.1 Requirements 84
4.2.1.5 Network Management 87
4.2.1.5.1 Requirements 87

4.2.1.6 Software Download 90
4.2.1.6.1 Requirements 90
4.2.1.7 AUTOSAR 99
4.2.1.7.1 Requirements 99
4.2.1.8 Startup Time 100
4.2.1.8.1 Requirements 100
4.3 Communication 101
4.3.1 CAN Data Link Layer 101
4.3.1.1 CAN Data Link Layer 101
4.3.1.1.1 Requirements 101
4.3.1.1.1.1 ECU Level 101
4.3.1.1.1.1.1 Resynchronisation and bit sampling 103
4.3.1.1.1.1.2 Speed tolerance definition 104
4.3.1.1.1.1.3 Network Transmission Speed - 500kbps 104
4.3.1.1.1.1.4 CAN FD bit register settings for 2 Mbps 105
4.3.1.1.2 Requisite Documents 106
4.3.1.1.3 Definitions and Abbreviations 106
4.3.2 Guideline for quality factor signals 106
4.3.2.1 Guideline for quality factor signals 106
4.3.2.1.1 Software Component Recommendations 106
4.3.2.1.1.1 QF values definition and use 106
4.3.2.1.1.1.1 Value 3 - 11b OK 106
4.3.2.1.1.1.2 Value 2 - 10b Not within specification 107
4.3.2.1.1.1.3 Value 1 - 01b No data exists 107
4.3.2.1.1.1.4 Value 0 - 00b Faulty 107
4.3.2.1.1.2 QF values definition and use summary table 108
4.3.2.1.1.3 Supporting information and clarifications from lessons learned108
4.3.2.1.1.3.1 DTCs 108
4.3.2.1.1.3.2 Temporary errors and filtering 109
4.3.2.1.1.3.3 Engine cranking 109
4.3.2.1.1.3.4 Detecting missing data 109
4.3.2.1.1.3.5 Multiple variants of a node 109
4.3.2.1.1.3.6 Data value 109
4.3.2.1.1.3.7 Philosophy and development of the QF concept 110
4.3.2.1.1.3.8 History of QF 1 110
4.3.2.1.1.4 Recomendations for creation of QF:s 111
4.3.2.1.2 Introduction 111
4.3.2.1.2.1 What is the purpose of this document 111
4.3.2.1.2.2 Risks 111

4.3.2.1.2.3 Who should read this document 112
4.3.2.1.2.4 Definitions and Abbreviations 112

4.3.3 DoCAN 112

4.3.3.1 DoCAN 112

4.3.3.1.1 Transport layer protocol 112
4.3.3.1.1.1 Protocol control information specification 113
4.3.3.1.1.1.1 BlockSize (BS) parameter definition 113
4.3.3.1.1.1.2 Separation (STmin) parameter definition 115
4.3.3.1.1.1.3 Separation time between single frames 117
4.3.3.1.1.2 Maximum number of FC. Wait frame transmission (N_WFTmax)118
4.3.3.1.1.3 Network layer timing 119
4.3.3.1.1.3.1 Timing parameters 119
4.3.3.1.1.3.2 Unexpected arrival of N_PDU 125

4.3.3.1.2 Data link layer usage 126
4.3.3.1.2.1 Adressing formats 126
4.3.3.1.2.2 CAN frame data padding 126
4.3.3.1.2.3 Use data frame by frame 128
4.3.3.1.2.3.1 Example A 130
4.3.3.1.2.3.2 Example B 131

4.3.3.1.3 Definitions and Abbreviations 131

4.3.3.1.4 Introduction 132

4.3.3.1.5 Requisite documents 133

4.3.4 Guideline for Safety Related Communication 134

4.3.4.1 Guideline for Safety Related Communication 134

4.3.4.1.1 Appendix C - Transient Filter 134

4.3.4.1.2 Appendix B - Examples of CAN Signals Reception 135

4.3.4.1.3 Appendix D - Handling of Difference in Generation and Consumption Rates 136

4.3.4.1.4 Appendix F - Example of Transmission Manager on Sender ECU ... 137

4.3.4.1.5 Appendix E - Proposal for Distributed Run-Time Fault Injection ... 138

4.3.4.1.6 Appendix A - Examples of Monitored Signal Generation 138
4.3.4.1.6.1 Case (A) - System With Two Identical CPUs and SW 140
4.3.4.1.6.2 Case (B) - System With Two Different CPU and-or SW, Alternative 1 142
4.3.4.1.6.3 Case (B) - System with Two Different CPU and-or SW, alternative 2143
4.3.4.1.6.4 Case (C), asynchronous system, alternative 1 144
4.3.4.1.6.5 Case (D), asynchronous system, alternative 2 145

4.3.4.1.7 Definitions and Abbreviations 145

4.3.4.1.8 Introduction 146
4.3.4.1.8.1 What is the purpose of this document 146
4.3.4.1.8.2 Motivation 146
4.3.4.1.8.3 Who should read this document 146
4.3.4.1.8.4 How do you receive more information 146
4.3.4.1.8.5 Requisite Documents 146

4.3.4.1.9 How To Use This Guideline 146
4.3.4.1.9.1 Limitations of this Guideline 147
4.3.4.1.9.2 Relevance of This Guideline When Developing a Technical Safety Concept 147

4.3.4.1.10 Communication Related Safety Mechanisms 147
4.3.4.1.10.1 Lower Level Network Protocol Safety Mechanisms 147
4.3.4.1.10.1.1 CAN 147
4.3.4.1.10.1.2 LIN 148
4.3.4.1.10.1.3 Flexray 148
4.3.4.1.10.2 AUTOSAR Platform Communication Safety Mechanisms 148
4.3.4.1.10.2.1 Generic Failure Modes and Detection Mechanisms in AUTOSAR 148
4.3.4.1.10.2.2 E2E Library 148
4.3.4.1.10.2.3 CRC Library 149
4.3.4.1.10.3 Application Level Safety Mechanisms 149
4.3.4.1.10.3.1 Quality Factor 149
4.3.4.1.10.3.2 Counter 149
4.3.4.1.10.3.3 Boolean Signals 149
4.3.4.1.10.3.4 Oversampling 150
4.3.4.1.10.3.5 Handshake Protocol 150
4.3.4.1.10.3.6 Range Check and Plausibility Check 150
4.3.4.1.10.4 System Level Safety Mechanisms 150
4.3.4.1.10.4.1 Redundancy (Only relevant for ASIL C and D) 150
4.3.4.1.10.5 Other Mechanisms 150
4.3.4.1.10.5.1 Self-detected CPU HW failures 150
4.3.4.1.10.5.2 HW SW Independence 150
4.3.4.1.10.5.3 Monitor 150
4.3.4.1.10.5.4 Run-time Fault Injection 151

4.3.4.1.11 Method 151

4.4 Data Security 153

4.4.1 Random number generation 153

4.4.1.1 Random number generation 153

4.4.1.1.1 Requirements 153

4.4.1.1.2 Introduction 154

4.4.2 Security Access algorithm 155

4.4.2.1 Security Access algorithm 155

4.4.2.1.1 Configuration of the Level of Protection 155
4.4.2.1.2 Random Number Generation 157
4.4.2.1.3 Security Levels (Areas) 157
4.4.2.1.4 Introduction 158
4.4.2.1.5 Algorithm Description 159
4.4.2.1.6 Security Constant 159
4.4.3 Software Authentication 160
4.4.3.1 General Software Authentication 160
4.4.3.1.1 System Description 160
4.4.3.1.1.1 Create Software Part 161
4.4.3.1.1.1.1 Verification Block Table 162
4.4.3.1.1.2 Sign Software Part 168
4.4.3.1.1.2.1 Sign and verify unprocessed data 173
4.4.3.1.1.2.2 Sign a Delta Software Part 174
4.4.3.1.1.3 Download a (signed) software part 175
4.4.3.1.1.4 Verify a signed software part 176
4.4.3.1.1.5 Key Management for Test and production ECUs 177
4.4.3.1.2 Signing Methods 182
4.4.3.1.2.1 Signing Method - Variability control 182
4.4.3.1.2.2 Signing Method - Algorithm Definitions 184
4.4.3.1.2.3 Signing Method - Key balancing 185
4.4.3.1.2.4 Software Settings File 186
4.4.3.1.3 Appendix 188
4.4.3.1.3.1 Example - Data format identifiers and signing (sw_part_based)188
4.4.3.1.3.2 Example - Software Part evolution 188
4.4.3.1.3.3 Example - Signing function when the processing method is unknown 189
4.4.3.1.3.4 Example - PublicKeyCheckSum 190
4.4.3.1.4 Definitions and Abbreviations 191
4.4.3.1.5 Requisite documents 191
4.4.3.1.6 Introduction 194
4.4.4 Privacy 199
4.4.4.1 Privacy in all ECUs 199
4.4.4.1.1 Requirements 199

4.4.4.1.2 Introduction 200
4.5 Diagnostics 200
4.5.1 UDS Data 200
4.5.1.1 UDS Data 200
4.5.1.1.1 Requirements 200
4.5.1.1.1.1 Requirements from section Diagnostic Data 200
4.5.1.1.1.2 Requirements from section Record Data - General 202
4.5.1.1.1.3 Requirements from section ECU external signals 209
4.5.1.1.1.4 Requirements from section ECU internal signals 221
4.5.1.1.1.5 Requirements from section ECU_Vehicle status 226
4.5.1.1.1.6 Requirements from section ECU_Vehicle identification 234
4.5.1.1.1.7 Requirements from section DTC information - General 249
4.5.1.1.1.8 Requirements from section Calibration of DTCs 253
4.5.1.1.1.9 Requirements from section Detection of faults 256
4.5.1.1.1.10 Requirements from section Detection of faults in external electrical circuits 259
4.5.1.1.1.11 Requirements from section Detection of low supply voltage 281
4.5.1.1.1.12 Requirements from section Detection of faults in internal electrical circuits 284
4.5.1.1.1.13 Requirements from section Detection of faults inferred from system behavior 294
4.5.1.1.1.14 Requirements from section Detection of faults due to configuration errors 302
4.5.1.1.1.15 Requirements from section Detection of faults caused by received signals 326
4.5.1.1.1.16 Requirements from section Detection of faults caused by the ECU environment 335
4.5.1.1.1.17 Requirements from section Detection of events associated with driver indications 342
4.5.1.1.1.18 Requirements from section DTC status bits 342
4.5.1.1.1.19 Requirements from section DTC snapshot data identification . 345
4.5.1.1.1.20 Requirements from section DTC snapshot sampling criteria ... 345
4.5.1.1.1.21 Requirements from section Global DTC snapshot data 348
4.5.1.1.1.22 Requirements from section Local DTC snapshot data 351
4.5.1.1.1.23 Requirements from section DTC operation cycle counters 351
4.5.1.1.1.24 Requirements from section DTC fault detection counters 357
4.5.1.1.1.25 Requirements from section DTC time stamp 359
4.5.1.1.1.26 Requirements from section DTC status indicators 360
4.5.1.1.1.27 Requirements from section Store DTC information 364
4.5.1.1.1.28 Requirements from section Transfer of DTC information to long term memory 367
4.5.1.1.1.29 Requirements from section Clear DTC information 368
4.5.1.1.1.30 Requirements from section Routines - General 370
4.5.1.1.1.31 Requirements from section Supported routines 372
4.5.1.1.1.32 Requirements from section Response times 378
4.5.1.1.2 APPENDIX 380
4.5.1.1.2.1 Appendix A 380
4.5.1.1.3 Overview 381
4.5.1.1.3.1 Definitions and Abbreviations 381
4.5.1.1.3.1.1 Abbreviations 381
4.5.1.1.3.1.2 Definitions 382
4.5.1.1.3.2 External Publications 384

4.5.1.1.3.3 Requisite Documents 384

4.5.1.1.4 Diagnostic Data 385
4.5.1.1.4.1 Record data 386
4.5.1.1.4.1.1 General 386
4.5.1.1.4.1.2 ECU external signals 388
4.5.1.1.4.1.3 ECU internal signals 390
4.5.1.1.4.1.4 ECU_Vehicle status 391
4.5.1.1.4.1.5 Vehicle network data 394
4.5.1.1.4.1.6 ECU_Vehicle identification 394
4.5.1.1.4.1.7 DTC information 399
4.5.1.1.4.1.7.1 General 399
4.5.1.1.4.1.7.2 Detection of faults 402
4.5.1.1.4.1.7.2.1 Detection of faults in external electrical circuits 405
4.5.1.1.4.1.7.2.2 Detection of low supply voltage 409
4.5.1.1.4.1.7.2.3 Detection of faults in internal electrical circuits 409
4.5.1.1.4.1.7.2.4 Detection of faults due to configuration errors 412
4.5.1.1.4.1.7.2.5 Calibration of configuration DTCs 415
4.5.1.1.4.1.7.2.6 Detection of faults inferred from system behaviour 417
4.5.1.1.4.1.7.2.7 Detection of faults caused by received signals 419
4.5.1.1.4.1.7.2.8 Detection of faults caused by the ECU environment 421
4.5.1.1.4.1.7.2.9 Detection of events associated with driver indications 423
4.5.1.1.4.1.7.2.10 Detection of faults caused by too high quiescent current 423
4.5.1.1.4.1.7.3 DTC status bits 424
4.5.1.1.4.1.7.4 DTC snapshot data 425
4.5.1.1.4.1.7.4.1 DTC snapshot data identification 425
4.5.1.1.4.1.7.4.2 DTC snapshot sampling criteria 427
4.5.1.1.4.1.7.4.3 Global DTC snapshot data 430
4.5.1.1.4.1.7.4.4 Local DTC snapshot data 432
4.5.1.1.4.1.7.5 DTC extended data 432
4.5.1.1.4.1.7.5.1 DTC operation cycle counters 432
4.5.1.1.4.1.7.5.2 DTC fault detection counters 438
4.5.1.1.4.1.7.5.3 DTC time stamp 440
4.5.1.1.4.1.7.5.4 DTC status indicators 440
4.5.1.1.4.1.7.6 Store DTC information 444
4.5.1.1.4.1.7.6.1 Transfer of DTC information to long term memory 445
4.5.1.1.4.1.7.7 Clear DTC information 446
4.5.1.1.4.2 Routines 448
4.5.1.1.4.2.1 General 448
4.5.1.1.4.2.2 Supported routines 449
4.5.1.1.4.2.3 Response times 451

4.5.2 UDS Services 452

4.5.2.1 UDS Services 452

4.5.2.1.1 Diagnostic service requirements 452
4.5.2.1.1.1 Supported services 453
4.5.2.1.1.1.1 Diagnostic and Communication Management Functional Unit454
4.5.2.1.1.1.2 Data Transmission Functional Unit 461
4.5.2.1.1.1.3 Stored Data Transmission Functional Unit 474
4.5.2.1.1.1.4 InputOutput Control Functional Unit 488
4.5.2.1.1.1.5 Remote Activation of Routine Functional Unit 493
4.5.2.1.1.1.6 Upload download functional unit 496
4.5.2.1.1.1.7 Additional routine requirements 508
4.5.2.1.1.1.8 Additional session requirements 515
4.5.2.1.1.1.8.1 General 515
4.5.2.1.1.1.8.2 Response message data parameter sessionParameterRecord516
4.5.2.1.1.2 Response times 517
4.5.2.1.1.3 Effect on the ECU operation 525
4.5.2.1.1.4 Entry conditions 531

4.5.2.1.1.5 Exit conditions 539
4.5.2.1.1.6 Security access 540
4.5.2.1.1.7 Negative response 547

4.5.2.1.2 Appendix 548
4.5.2.1.2.1 Appendix A- Suggestion code for transferResponseParameterRecord calculation 548
4.5.2.1.2.1.1 CRC16-CITT C-code example 1 (fast) 548
4.5.2.1.2.1.2 CRC16-CITT C-code example 2 (slow) 549
4.5.2.1.2.1.3 CRC32 C-code example 550
4.5.2.1.2.2 Appendix B- Additional ReadGenericInformation requirements553
4.5.2.1.2.2.1 Request message definition 553
4.5.2.1.2.2.2 Request message data parameter definition 553
4.5.2.1.2.2.3 Positive response message definition 553
4.5.2.1.2.2.4 Positive response message data parameter definition 556
4.5.2.1.2.2.5 Supported negative response codes (NRC_) 556

4.5.2.1.3 Introduction 557
4.5.2.1.3.1 Requisite documents 557
4.5.2.1.3.2 Definitions and Abbreviations 558

4.5.2.1.4 Diagnostic message addressing 560
4.5.2.1.4.1 Logical address format for diagnostic request and response 561
4.5.2.1.4.2 Physical logical address range 562
4.5.2.1.4.3 Functional logical address range 564
4.5.2.1.4.4 Logical address range for periodic response messages 565
4.5.2.1.4.5 Logical address range for external diagnostic tester 566
4.5.2.1.4.6 ECU ability to receive requests 567

4.5.3 UDS Session 569

4.5.3.1 UDS Session generic reqs 569

4.5.3.1.1 Message timing definition 569

4.5.3.1.2 Overview 570
4.5.3.1.2.1 Definitions and Abbrevations 572
4.5.3.1.2.2 Requisite documents 572

4.5.3.2 UDS Session ECU reqs 572

4.5.3.2.1 Appendix 572
4.5.3.2.1.1 Example, queuing of requests in a server 572
4.5.3.2.1.2 Example, queuing of requests versus full duplex 574

4.5.3.2.2 Timing handling during communication 576
4.5.3.2.2.1 Queued request messages to one server 577
4.5.3.2.2.2 P4 definition 578

4.5.3.2.3 Timing parameters 580
4.5.3.2.3.1 Message timing definition 580
4.5.3.2.3.1.1 P2Server 580
4.5.3.2.3.1.2 P2(star)Server 580

4.5.4 UDSonCAN 581

4.5.4.1 UDSonCAN - Generic 581
4.5.4.1.1 Transport_Network Layer 581
4.5.4.1.2 Data Link Layer 582
4.5.4.1.3 Physical Layer 582
4.5.4.1.4 Introduction 582
4.5.4.1.4.1 Requisite documents 583
4.5.4.1.4.2 Definitions and Abbreviations 584
4.5.4.1.4.3 Overview 585
4.5.4.1.5 Application Layer 586
4.5.4.1.6 Session Layer 587
4.5.4.1.6.1 Addressing USDT frames 587
4.5.4.2 UDSonCAN - Public CAN ECU 588
4.5.4.2.1 Diagnostic Data 588
4.5.4.2.1.1 Record Data 589
4.5.4.2.1.2 Diagnostic trouble code information 589
4.5.4.2.2 Diagnostic Services 624
4.6 Software Download 625
4.6.1 General Software Download 625
4.6.1.1 Generic SWDL 625
4.6.1.1.1 Requisite documents 625
4.6.1.1.1.1 Referenced document, structure and relations 625
4.6.1.1.2 Definitions and Abbreviations 627
4.6.1.1.3 Overview 628
4.6.1.1.4 Introduction 628
4.6.1.1.5 What is the purpose of this document 628
4.6.1.2 General Bootloader requirements 629
4.6.1.2.1 Software Authentication 629
4.6.1.2.1.1 Software Authentication - Introduction 629
4.6.1.2.1.2 Software Structure 631
4.6.1.2.1.2.1 ECU Software Structure (ESS) 633
4.6.1.2.1.2.2 ECU Software Structure - Declaration 636

4.6.1.2.1.2.3 Root ECU Software Structure 638
4.6.1.2.1.3 Erase Status Information 638
4.6.1.2.1.4 Verification Block Table 639
4.6.1.2.1.5 Validation of ECU software(s) - Authenticity Verification 641
4.6.1.2.1.5.1 Authenticity Verification - Overview 641
4.6.1.2.1.5.2 Validation of Logical Blocks 642
4.6.1.2.1.5.3 Validity Status Information 645
4.6.1.2.1.5.4 Public Key Storage 646
4.6.1.2.1.6 Programming Sequence - Authenticity Verification 646
4.6.1.2.1.6.1 Memory Erasure 646
4.6.1.2.1.6.2 Request Download 647
4.6.1.2.1.6.3 Check Memory 649
4.6.1.2.1.6.4 Complete and Compatible 651
4.6.1.2.1.6.5 ECU Reset - Application start 651
4.6.1.2.1.6.6 Download and activate the SBL 652
4.6.1.2.1.6.7 Request Upload 653
4.6.1.2.1.7 Additional requirements for ECUs having multiple processors 654
4.6.1.2.1.7.1 Multiple processors - Secondary Bootloader 655
4.6.1.2.1.8 Additional requirements for ECUs implementing delta encoding656
4.6.1.2.1.9 Signing method 658
4.6.1.2.1.10 Software Authentication - Generic security requirements 659

4.6.1.2.2 Validation of ECU software(s) - Complete and Compatible 661
4.6.1.2.2.1 Complete and Compatible - Complete check 664
4.6.1.2.2.2 Complete and Compatible - Compatible check 666
4.6.1.2.2.3 CompleteCompatibleFunction() 667

4.6.1.2.3 Time window for PROG detection 668

4.6.1.2.4 ECU program mode 669
4.6.1.2.4.1 Simplified bootloader state and flow chart diagram 670

4.6.1.2.5 Enter the bootloader (PBL) from the application 672

4.6.1.2.6 Pre-calculated hash 674

4.6.1.2.7 Appendix 675
4.6.1.2.7.1 Examples of different scenarios for the Complete & Compatible function 675
4.6.1.2.7.2 Examples of different Compatible functions 676
4.6.1.2.7.2.1 Bootloader Compatbile check 676
4.6.1.2.7.2.2 Application Complete & Compatible block 676

4.6.1.2.8 Writing of received data to non-volatile memory 679

4.6.1.2.9 File download order 680

4.6.1.2.10 Separate memory sectors 680

4.6.1.2.11 Delta encoding 681

4.6.1.2.12 Handling of distribution relays 682

4.6.1.2.13 Calibration and customer data 682
4.6.1.2.14 Internal data transfer speed 683
4.6.1.2.15 Target 683
4.6.1.2.16 Power consumption 683
4.6.1.2.17 Additional bootloader requirements for private ECU 684
4.6.1.2.18 Data compression and encryption 685
4.6.1.2.19 Additional bootloader requirements for multiple networks 686
4.6.1.2.20 Two level bootloader 686
4.6.1.2.20.1 Primary Bootloader (PBL) 687
4.6.1.2.20.1.1 Primary bootloader diagnostic data 689
4.6.1.2.20.1.2 Primary bootloader diagnostic services 689
4.6.1.2.20.2 Secondary Bootloader (SBL) 689
4.6.1.2.20.2.1 Secondary bootloader diagnostic data 691
4.6.1.2.20.2.2 Secondary bootloader diagnostic services 691
4.6.1.2.21 Additional bootloader requirements for secondary processors 691
4.6.2 Data Compression and Encryption 692
4.6.2.1 Data Compression and Encryption 692
4.6.2.1.1 Software Component Requirements 692
4.6.2.1.1.1 Compression Method Selection 692
4.6.2.1.1.1.1 Method Selection 692
4.6.2.1.1.2 Compression Method #1 Requirements 693
4.6.2.1.1.2.1 Data Compression 693
4.6.2.1.1.3 Compression Method #2 Requirements 694
4.6.2.1.1.3.1 Data Compression 694
4.6.2.1.1.4 Compression Method #3 Requirements 695
4.6.2.1.1.4.1 Data Compression 695
4.6.2.1.2 Appendix 696
4.6.2.1.2.1 Compression Method #1 Examples 696
4.6.2.1.2.1.1 LZSS Compression C-code Example 696
4.6.2.1.2.1.2 LZSS Decompression C-code Example 706
4.6.2.1.2.1.3 Implementation Advice Regarding Decompression 710
4.6.2.1.3 Data Encryption 711
4.6.2.1.4 Overview 711
4.6.2.1.4.1 Definitions and Abbreviations 712
4.6.2.1.4.2 External Publications 712
4.6.2.1.5 Software Compression 712

4.6.2.1.5.1 Compression Method #1 712
4.6.2.1.5.1.1 Data Compression 712
4.6.2.1.5.2 Compression Method #2 713
4.6.2.1.5.2.1 Data Compression 713
4.6.2.1.5.3 Compression Method #3 714
4.6.2.1.5.3.1 Data Compression 714
4.6.2.1.5.3.2 Data Decompression 715

4.6.3 Delta Encoding 715

4.6.3.1 Delta Encoding Introduction 715

4.6.3.1.1 Logical System Components 715

4.6.3.1.2 Requisite Documents 715

4.6.3.1.3 Definitions & Abbreviations 716

4.6.3.2 Delta generation 716

4.6.3.2.1 Requirements 716

4.6.3.3 Delta installation 722

4.6.3.3.1 Requirements 722
4.6.3.3.1.1 Error Handling 726

4.6.3.4 Extended information 729

4.6.3.4.1 Delta installation - Sequence and memory usage 729

4.6.4 Bootloader - UDSonCAN 729

4.6.4.1 Bootloader - UDSonCAN 729

4.6.4.1.1 Transport-Network Layer 729

4.6.4.1.2 Data Link Layer 730
4.6.4.1.2.1 Addressing USDT frames 730

4.6.4.1.3 Session Layer 731

4.6.4.1.4 ECU Program Mode 731
4.6.4.1.4.1 Bootloader state diagram 732
4.6.4.1.4.2 Bootloader flow chart diagram - Overview 733
4.6.4.1.4.2.1 Bootloader flow chart diagram (Init) 735
4.6.4.1.4.2.2 Bootloader flow chart diagram (ProgSession) 740
4.6.4.1.4.2.3 Flow chart diagram (Application) 742

4.6.4.1.5 Physical Layer 743

4.6.4.1.6 Application Layer 744
4.6.4.1.7 Requisite Documents 744
4.6.4.1.8 Introduction 744
4.6.4.1.9 Definitions and Abbreviations 745
4.6.4.1.10 Unified Diagnostic Services Implementation 746
4.6.4.1.10.1 Diagnostic Services 746
4.6.4.1.10.1.1 RequestDownload (34 hex) Service 746
4.6.4.1.11 Overview 746
4.6.5 Versatile Binary Format Specification 748
4.6.5.1 Versatile Binary Format Specification 748
4.6.5.1.1 Overview of the VBF file 748
4.6.5.1.1.1 Version Section 748
4.6.5.1.1.2 Header Section 748
4.6.5.1.1.2.1 description 750
4.6.5.1.1.2.2 sw_part_number 751
4.6.5.1.1.2.3 sw_current_part_number 752
4.6.5.1.1.2.4 sw_version 753
4.6.5.1.1.2.5 sw_current_version 754
4.6.5.1.1.2.6 sw_part_type 755
4.6.5.1.1.2.7 data_format_identifier 756
4.6.5.1.1.2.8 ecu_address 757
4.6.5.1.1.2.9 erase 758
4.6.5.1.1.2.10 call 759
4.6.5.1.1.2.11 verification_block_start 759
4.6.5.1.1.2.12 verification_block_length 760
4.6.5.1.1.2.13 verification_block_root_hash 761
4.6.5.1.1.2.14 sw_signature 761
4.6.5.1.1.2.15 sw_signature_dev 762
4.6.5.1.1.2.16 file_checksum 762
4.6.5.1.1.2.17 parameter_settings 763
4.6.5.1.1.2.18 Header section example #1 765
4.6.5.1.1.2.19 Header section example #2 766
4.6.5.1.1.2.20 Header section example #3 767
4.6.5.1.1.2.21 Header section example #4 769
4.6.5.1.1.3 Data Section 770
4.6.5.1.1.3.1 Checksum calculation 771
4.6.5.1.2 Appendix A 772
4.6.5.1.2.1 CRC16-CITT C-code example 772
4.6.5.1.3 Appendix B 773
4.6.5.1.3.1 CRC32 C-code example 773
4.6.5.1.4 Overview 775
4.6.5.1.4.1 Requisite documents 775
4.6.5.1.4.2 Definitions and Abbreviations 775

4.6.5.1.5 Versatile Binary Format (VBF) 776
4.6.5.1.5.1 VBF version 776
4.6.5.1.5.2 Introduction 776
4.6.5.1.5.3 File Naming 777
4.6.5.1.5.4 VBF file generation 778

4.6.5.1.6 Notation and lexical elements 778
4.6.5.1.6.1 Notation 778
4.6.5.1.6.2 General structure 778
4.6.5.1.6.3 Identifiers, reserved words and expressions 779
4.6.5.1.6.4 Integers 780
4.6.5.1.6.5 Comments 780
4.6.5.1.6.6 White Space Characters 781
4.6.5.1.6.7 Non-printing Characters 781

4.7 Software Platform 782

4.7.1 Autosar Configuration 782

4.7.1.1 Autosar Configuration 782

4.7.1.1.1 E2E PROTECTION CONCEPT 782
4.7.1.1.1.1 E2E Protection Introduction 782
4.7.1.1.1.2 E2E Protection Requirements 782
4.7.1.1.1.2.1 Profile VCC Requirements 783
4.7.1.1.1.3 Profiles vs Transformers and Wrappers 786

4.7.1.1.2 TIME SYNCHRONIZATION CONCEPT 786
4.7.1.1.2.1 Time Synchronization Introduction 786
4.7.1.1.2.2 Time Synchronization Requirements 787

4.7.1.1.3 AUTOSAR CONFIGURATION REQUIREMENTS 787

4.7.1.1.4 Documents 792

4.7.1.1.5 Definitions and Abbreviations 792

4.7.2 Autosar Network Management - Additional Requirements 792

4.7.2.1 AutoSar NM - Generic reqs 792

4.7.2.1.1 Introduction 792

4.7.2.1.2 Overview 794
4.7.2.1.2.1 Definitions and Abbreviations 795
4.7.2.1.2.2 Requisite documents 796
4.7.2.1.2.3 OEM specific concepts 797

4.7.2.2 AutoSar NM - CAN reqs 797

4.7.2.2.1 Diagnostics 797

4.7.2.2.2 Timing requirements 799

4.7.2.3 AutoSar NM - CAN & FlexRay additional reqs 801

4.7.2.3.1 Requirements 801

4.7.2.4 AutoSar NM - CAN wake up reqs 803

4.7.2.4.1 Requirements 803
4.7.2.4.1.1 Microcontroller is powered in sleep 804
4.7.2.4.1.2 Microcontroller is unpowered in sleep 805

4.7.3 Autosar BSW - Tier1 to Tier2 Requirements 806

4.7.3.1 Autosar 4.0.3 BSW - Tier1 to Tier2 Requirements 806

4.7.3.1.1 Requirements 806
5 DETAILED CHANGE INFO 807

0 REQUIREMENT LOG

0.1 List of requirements

298	Clear ProgSignature in RAM	1
299	Complete & Compatible function	1
300	S3Server timer timeout in SBL	1
301	General bootloader requirements	1
302	Check ProgSignature in RAM	1
355	CC Subscriber ECU configuration states	1
356	CC Subscriber ECU in Bulk (unconfigured) state	1
357	Routines for receiving VCP signal (CC Domain Master excluded)	1
358	Invalid or unrecognized parameter values	1
359	Configuration of ECU with updated CCP data	1
360	Selection of CCPs in VCP signal	1
361	Car Configuration data storage	1
362	Car Configuration parameter definition	1
363	Handling of parameter data	1
364	ECU configuration after boot-up	1
365	CC Subscriber ECU DID support	1
392	Standards for CAN interface	1
393	The priority between standard documents and this document	1
394	Communication protocol	1
395	Behaviour outside normal supply voltage range	1
396	CAN bus level during reset	1
397	Bus fault reporting	1
398	Bit time set up for 500 kbps	1
399	Resychronisation strategy	1
400	Sample mode	1
401	PLL bypass	1
402	PLL locking	1
1198	Transport and Network layer	1
1199	Precedence of requirements	1
1200	BlockSize parameter non-programming session server side	1
1201	BlockSize parameter non-programming session client side	1
1202	BlockSize parameter programming session server side	1
1203	BlockSize parameter programming session client side	1
1204	Separation time (STmin) non-programming session server side	1
1205	Separation time (STmin) non-programming session client side	1
1206	Separation time (STmin) programming session server side	1
1207	Separation time (STmin) programming session client side	1
1208	Separation time between single frames - programming session	1
1209	N_WFTmax value for server side	1
1210	N_WFTmax value for client side	1
1211	Flowcontrol Wait usage	1
1212	N_As timeout in non-programming session	1
1213	N_Ar timeout in non-programming session	1
1214	N_Bs timeout in non-programming session	1
1215	N_Br Performance requirement in non-programming session for FC.CTS and FC.OVFLW	1
1216	N_Br Performance requirement in non-programming session for FC.Wait	1
1217	N_Cs Performance requirement in non-programming session	1
1218	N_Cr timeout in non-programming session	1
1219	N_As timeout in programming session	1
1220	N_Ar timeout in programming session	1
1221	N_Bs timeout in programming session	1
1222	N_Br performance requirement in programming session for FC.CTS and FC.OVFLW	1
1223	N_Br Performance requirement in programming session for	1

	FC.Wait	
1224	N_Cs Performance requirement in programming session	1
1225	N_Cr timeout in programming session	1
1226	Duplex communication	1
1227	Addressing format	1
1228	CAN frame identifier length	1
1229	Supporting functional requests	1
1230	Length of Classic CAN frames	1
1231	Handling of CAN requests in regards of DLC	1
1232	Unused data in the CAN frame	1
1233	Forward N_Data from each N_PDU to upper layer	1
1576	Entropy for random numbers	1
1577	Random number generator algorithms	1
1586	Default values - Delay timers and counters for false attempts	1
1587	SecurityAccess (27) - time delay after false securityAccess attempts	1
1588	Seed - Random Number Generation	1
1589	Distribution of the Fixed Bytes	1
1590	Fixed Bytes default value	1
1591	Key Storage	1
1592	Security Level usage	1
1594	Signing Method SHA256_RSA2048 - Hash function	1
1595	Signing Method SHA256_RSA2048 - Key Size	1
1596	Signing Method SHA256_RSA2048 - Public Key Algorithm	1
1597	Software authentication security classes	1
1598	Default Signing Method	1
1599	Signing Method - Variability control	1
1600	Format of the Verification Block Table	1
1601	Verification Block Root Hash	1
1602	Verification Block Table - Input order to Hash function per data block	1
1603	Verification Block Table Format Identifier - Definition	1
1604	Verification Block Table Header Identifiers	1
1605	Verification Block Table on unprocessed data	1
1606	Verification Block Table per Software Part	1
1607	Production Key programming	1
1608	Public Key Types	1
1609	Software Authentication Test and Development Key	1
1610	Append signature to software part	1
1611	Sign Software Part - Private Key syntax	1
1612	Signing function - Generate signature	1
1613	Signing function - Integrity Check	1
1614	Sign a delta file	1
1615	Sign and verify unprocessed data	1
1625	Spare Capacity	1
1627	Multiple CPU Design	1
1628	Domain Master Address	1
1629	ECU Addresses	1
1630	Component Selection - CAN Physical Interfaces	1
1631	Missing frame master - CAN	1
1632	No priority inversion	1
1657	Network management CAN frame identifiers	1
1662	Bootloader CAN busoff handling	1
1663	Bootloader routing - Spare networks	1
1664	Deployment of the J2534 requirements	1
1665	SWDL time in aftermarket	1
1666	SWDL time in factory	1
1667	Separate software files	1
1669	Communication Gateway Processing	1
1670	Communication Stack Processing Period	1
1671	Communication Stack Processing Period Jitter	1
1672	AUTOSAR version	1

1673	Startup Time	1
1804	Calibration of agedDTCLimit	1
1805	Calibration of confirmedDTCLimit	1
1806	Calibration of DTC test sample failed	1
1807	Calibration of DTC test sample passed	1
1808	Calibration of DTCTestFailedLimit	1
1809	Calibration of DTCTestPassedLimit	1
1810	Calibration of unconfirmedDTCLimit	1
1811	Clear DTC information by diagnostic service	1
1812	Clear DTC information when a DTC is aged	1
1813	Clear DTC information when an aged DTC reoccurs	1
1814	Clear DTC information when the long term memory overflows	1
1815	Detected of events associated with driver indications	1
1816	DTC values	1
1817	Failure type value for DTCs	1
1818	Generic Test Run Criteria - car mode transition inhibit criteria	1
1819	Generic Test Run Criteria - car modes inhibit criteria	1
1820	Generic Test Run Criteria - enable criteria	1
1821	Generic Test Run Criteria - low power inhibit criteria	1
1822	Generic Test Run Criteria - low voltage inhibit criteria	1
1823	Generic Test Run Criteria - usage mode transition inhibit criteria	1
1824	Test Run Criteria - Updating the DTC Fault Detection Counter	1
1825	Base DTC value for detection of corrupt data received from other ECU by end-to-end checksum	1
1826	Base DTC value for detection of corrupt data received from other ECU by rolling counter	1
1827	Base DTC value for detection of corrupt data received from other ECU by up-date-bit	1
1828	Base DTC value for detection of other ECU not operating normally	1
1829	Base DTC value for detection of other ECU sending data tagged as invalid	1
1830	Base DTC value for detection of other ECU sending faulty data	1
1831	Detection of corrupt data received from other ECU by end-to-end checksum	1
1832	Detection of corrupt data received from other ECU by rolling counter	1
1833	Detection of corrupt data received from other ECU by up-date-bit	1
1834	Detection of other ECU not operating normally	1
1835	Detection of other ECU sending data tagged as invalid	1
1836	Detection of other ECU sending faulty data	1
1837	Failure type value for detection of corrupt data received from other ECU by end-to-end checksum	1
1838	Failure type value for detection of corrupt data received from other ECU by rolling counter	1
1839	Failure type value for detection of corrupt data received from other ECU by up-date bit	1
1840	Failure type value for detection of other ECU not operating normally	1
1841	Failure type value for detection of other ECU sending data tagged as inv...	1
1842	Failure type value for detection of other ECU sending faulty data	1
1843	Root DTC for ECU not operating normally	1
1844	Root DTC for ECU sending data tagged as invalid	1
1845	Test Run Criteria for detection of corrupt data received from other ECU by end-to-end checksum	1
1846	Test Run Criteria for detection of corrupt data received from other ECU by rolling counter	1
1847	Test Run Criteria for detection of corrupt data received from	1

	other ECU by up-date bit	
1848	Test Run Criteria for detection of other ECU not operating normally	1
1849	Test Run Criteria for detection of other ECU sending data tagged as invalid	1
1850	Test Run Criteria for detection of other ECU sending faulty data	1
1851	Base DTC value for detection of events caused by customer operating vehi...	1
1852	Base DTC value for detection of events caused by ECU supply voltage is t...	1
1853	Base DTC value for detection of events caused by ECU supply voltage is t...	1
1854	Base DTC value for detection of events caused by other condition that is out of range for full opera	1
1855	Base DTC value for detection of faults caused by ambient teperature too ...	1
1856	Base DTC value for detection of faults caused by ambient teperature too low	1
1857	Base DTC value for detection of other faults inferred from system behaviour	1
1858	Detection of events caused by customer operating vehicle or vehicle equi...	1
1859	Detection of events caused by ECU supply voltage is too high for full op...	1
1860	Detection of events caused by ECU supply voltage is too low for full ope...	1
1861	Detection of events caused by other condition that is out of range for f...	1
1862	Failure type value for detection of events caused by customer operating ...	1
1863	Failure type value for detection of events caused by ECU supply voltage ...	1
1864	Failure type value for detection of events caused by ECU supply voltage ...	1
1865	Failure type value for detection of events caused by other condition tha...	1
1866	Failure type value for detection of faults caused by ambient teperature	1
1867	Test period time for detection of events caused by customer operating ve...	1
1868	Test period time for detection of events caused by ECU supply voltage is...	1
1869	Test period time for detection of events caused by ECU supply voltage is...	1
1870	Test period time for detection of events caused by other condition that is out of range for full ope	1
1871	Test period time for detection of faults caused by ambient teperature	1
1872	Test Run Criteria for detection of faults caused by the ECU operating environment or customer action	1
1873	agedDTCLimit for inconsistent or incompatible vehicle configuration data	1
1874	agedDTCLimit for invalid vehicle configuration data	1
1875	agedDTCLimit for missing Car Configuration Logic (CCL)	1
1876	agedDTCLimit for missing vehicle configuration data	1
1877	agedDTCLimit for missing Vehicle Configuration Data (VCD)	1
1878	agedDTCLimit for new configuration not learned	1
1879	agedDTCLimit for VCD and CCL data inconsistency	1
1880	Base DTC value for detection of software incompatibility with another ECU	1
1881	Clearing of DTC for detection of suppler software installed in	1

	ECU.	
1882	ConfirmedDTCLimit for inconsistent or incompatible vehicle configuration...	1
1883	ConfirmedDTCLimit for invalid vehicle configuration data	1
1884	ConfirmedDTCLimit for missing Car Configuration Logic (CCL)	1
1885	ConfirmedDTCLimit for missing vehicle configuration data	1
1886	ConfirmedDTCLimit for missing Vehicle Configuration Data (VCD)	1
1887	ConfirmedDTCLimit for new configuration not learned	1
1888	ConfirmedDTCLimit for VCD and CCL data inconsistency	1
1889	Detection of data integrity fault in configuration data memory	1
1890	Detection of incompatible VIN	1
1891	Detection of inconsistent or incompatible vehicle configuration data	1
1892	Detection of invalid local configuration data	1
1893	Detection of invalid vehicle configuration data	1
1894	Detection of missing Car Configuration Logic (CCL)	1
1895	Detection of missing vehicle configuration data (ECU never configured)	1
1896	Detection of missing Vehicle Configuration Data (VCD)	1
1897	Detection of new configuration not learned	1
1898	Detection of software incompatibility with another ECU	1
1899	Detection of suppler specific software installed in ECU	1
1900	Detection of VCD and CCL data inconsistency	1
1901	DTC value for detection of CCL part no inconsistency	1
1902	DTC value for detection of data integrity fault in configuration data me...	1
1903	DTC value for detection of incompatible VIN	1
1904	DTC value for detection of inconsistent or incompatible vehicle configur...	1
1905	DTC value for detection of invalid local configuration data	1
1906	DTC value for detection of invalid vehicle configuration data	1
1907	DTC value for detection of missing Car Configuration Logic data	1
1908	DTC value for detection of missing vehicle configuration data	1
1909	DTC value for detection of missing Vehicle Configuration Data (VCD)	1
1910	DTC value for detection of new configuration not learned	1
1911	Failure type value for detection of CCL part no inconsistency	1
1912	Failure type value for detection of data integrity fault in configuratio...	1
1913	Failure type value for detection of inconsistent or incompatible vehicle...	1
1914	Failure type value for detection of invalid local configuration data	1
1915	Failure type value for detection of invalid vehicle configuration data	1
1916	Failure type value for detection of missing Car Configuration Logic data	1
1917	Failure type value for detection of missing Vehicle Configur. Data (VCD)	1
1918	Failure type value for detection of missing vehicle configuration data	1
1919	Failure type value for detection of software incompatibility with anothe...	1
1920	Failure type value for new configuration not learned	1
1921	FDC 10 max value for inconsistent or incompatible vehicle configuration ...	1
1922	FDC 10 max value for invalid vehicle configuration data	1
1923	FDC 10 max value for missing Car Configuration Logic (CCL)	1
1924	FDC 10 max value for missing vehicle configuration data	1
1925	FDC 10 max value for missing Vehicle Configuration Data (VCD)	1

1926	FDC 10 max value for new configuration not learned	1
1927	FDC 10 max value for VCD and CCL data inconsistency	1
1928	Test Run Criteria for detection of CCL part no inconsistency	1
1929	Test Run Criteria for detection of data integrity fault in configuration...	1
1930	Test Run Criteria for detection of incompatible VIN	1
1931	Test Run Criteria for detection of inconsistent or incompatible vehicle ...	1
1932	Test Run Criteria for detection of invalid local configuration data	1
1933	Test Run Criteria for detection of missing Car Configuration Logic data	1
1934	Test Run Criteria for detection of missing vehicle configuration data	1
1935	Test Run Criteria for detection of missing Vehicle Configuration Data (VCD)	1
1936	Test Run Criteria for invalid vehicle configuration data	1
1937	Test Run Criteria for new configuration not learned	1
1938	Test sample failed criteria for inconsistent or incompatible vehicle con...	1
1939	Test sample failed criteria for invalid vehicle configuration data	1
1940	Test sample failed criteria for missing Car Configuration Logic (CCL)	1
1941	Test sample failed criteria for missing vehicle configuration data	1
1942	Test sample failed criteria for missing Vehicle Configuration Data (VCD)	1
1943	Test sample failed criteria for new configuration not learned	1
1944	Test sample failed criteria for VCD and CCL data inconsistency	1
1945	Test sample passed criteria for inconsistent or incompatible vehicle configuration data	1
1946	Test sample passed criteria for invalid vehicle configuration data	1
1947	Test sample passed criteria for missing vehicle configuration data	1
1948	Test sample passed criteria for missing Vehicle Configuration Data (VCD)	1
1949	Test sample passed criteria for new configuration not learned	1
1950	Test sample passed criteria for No Car Configuration Logic (CCL) is present in CCL memory.	1
1951	Test sample passed criteria for VCD and CCL data inconsistency	1
1952	UnconfirmedDTCLimit for inconsistent or incompatible vehicle configurati...	1
1953	UnconfirmedDTCLimit for invalid vehicle configuration data	1
1954	UnconfirmedDTCLimit for missing Car Configuration Logic (CCL)	1
1955	UnconfirmedDTCLimit for missing vehicle configuration data	1
1956	UnconfirmedDTCLimit for missing Vehicle Configuration Data (VCD)	1
1957	UnconfirmedDTCLimit for new configuration not learned	1
1958	UnconfirmedDTCLimit for VCD and CCL data inconsistency	1
1959	Base DTC value for detection of low supply voltage	1
1960	Base DTC value for faults in high side driver outputs	1
1961	Base DTC value for low side driver outputs	1
1962	Base DTC value for short circuit and open circuit faults in FM, PCM, PWM...	1
1963	Base DTC value for short circuit to battery or open circuit faults in an...	1
1964	Base DTC value for switch inputs	1
1965	Detection of CS- or CRC-error on private ECU network	1
1966	Detection of data not updated by transmitter on private ECU network	1
1967	Detection of faults for switch inputs	1

1968	Detection of incorrect timing and format on private ECU network	1
1969	Detection of low supply voltage	1
1970	Detection of missed deadline for message on private ECU network	1
1971	Detection of other external electrical circuit faults	1
1972	Detection of other faults for analog inputs	1
1973	Detection of other faults for FM, PCM, PWM etc. inputs	1
1974	Detection of other faults for high side driver outputs	1
1975	Detection of other faults for low side driver outputs	1
1976	Detection of other faults on private ECU network	1
1977	Detection of physical layer faults on private ECU network	1
1978	Detection of short circuit and open circuit faults in analog inputs	1
1979	Detection of short circuit and open circuit faults in FM, PCM, PWM etc. ...	1
1980	Detection of short circuit and open circuit faults in high side driver o...	1
1981	Detection of short circuit and open circuit faults in low side driver ou...	1
1982	Detection of transmit error on private ECU network	1
1983	Detection of unexpected transmissions on private ECU network	1
1984	DTC value for other external electrical circuit faults	1
1985	Failure type value for CS- or CRC-error on private ECU network	1
1986	Failure type value for detection of low supply voltage	1
1987	Failure type value for incorrect timing and format on private ECU network	1
1988	Failure type value for missed deadline for message on private ECU network	1
1989	Failure type value for physical layer faults on private ECU network	1
1990	Failure type value for short circuit and open circuit faults in FM, PCM,...	1
1991	Failure type value for short circuit to battery faults in low side drive...	1
1992	Failure type value for short circuit to battery or open circuit faults i...	1
1993	Failure type value for short circuit to battery or open circuit faults in high side driver outputs	1
1994	Failure type value for short circuit to ground faults in analog inputs	1
1995	Failure type value for short circuit to ground faults in high side drive...	1
1996	Failure type value for short circuit to ground or open circuit faults in...	1
1997	Failure type value for stuck at faults for switch inputs	1
1998	Failure type value for transmit error on private ECU network	1
1999	Failure type value for unexpected transmissions on private ECU network	1
2000	Test period time for detection of other external electrical circuit faults	1
2001	Test period time for detection of other faults on private ECU network	1
2002	Test period time for detection of unexpected transmissions on private EC...	1
2003	Test period time for other faults for analog inputs	1
2004	Test period time for other faults for FM, PCM, PWM etc. inputs	1
2005	Test period time for other faults for high side driver outputs	1
2006	Test period time for other faults for low side driver outputs	1
2007	Test period time for other faults for switch inputs	1
2008	Test period time for physical layer faults on private ECU	1

	network	
2009	Test period time for short circuit and open circuit faults in analog inputs	1
2010	Test period time for short circuit and open circuit faults in FM, PCM, P...	1
2011	Test period time for short circuit and open circuit faults in high side ...	1
2012	Test period time for short circuit and open circuit faults in low side d...	1
2013	Test period time for stuck at faults for switch inputs	1
2014	Test Run Criteria for CS- or CRC-error on private ECU network	1
2015	Test Run Criteria for data not updated by transmitter on private ECU net...	1
2016	Test Run Criteria for detection of transmit error on private ECU network	1
2017	Test Run Criteria for fault detection in external circuits	1
2018	Test Run Criteria for incorrect timing and format on private ECU network	1
2019	Test Run Criteria for missed deadline for message on private ECU network	1
2020	Test Run Criteria for short circuit to battery faults in low side driver...	1
2021	Test Run Criteria for short circuit to battery or open circuit faults in high side driver outputs	1
2022	Test Run Criteria for short circuit to ground faults in high side driver...	1
2023	Test Run Criteria for short circuit to ground or open circuit faults in ...	1
2024	type value for data not updated by transmitter on private ECU network	1
2025	Base DTC value for detection of data integrity fault in non volatile memory	1
2026	Base DTC value for detection of data integrity fault in volatile memory	1
2027	Base DTC value for detection of faults in other internal parts	1
2028	Base DTC value for detection of internally connected actuator fault	1
2029	Base DTC value for detection of internally connected sensor fault	1
2030	Detection of data integrity fault in non volatile memory	1
2031	Detection of data integrity fault in volatile memory	1
2032	Detection of faults in other internal parts	1
2033	Detection of internal device communication error	1
2034	Detection of internal faults not on the main circuit board	1
2035	Detection of internally connected actuator fault	1
2036	Detection of internally connected sensor fault	1
2037	Detection of programme execution fault	1
2038	DTC value for detection of internal device communication error	1
2039	DTC value for detection of programme execution fault	1
2040	Failure type value for detection of data integrity fault in non volatile...	1
2041	Failure type value for detection of data integrity fault in volatile memory	1
2042	Failure type value for detection of faults in other internal parts	1
2043	Failure type value for detection of internal device communication error	1
2044	Failure type value for detection of internally connected actuator fault	1
2045	Failure type value for detection of internally connected sensor fault	1
2046	Failure type value for detection of programme execution fault	1

2047	Test period time for detection of faults in other internal parts	1
2048	Test period time for detection of internally connected actuator fault	1
2049	Test period time for detection of internally connected sensor fault	1
2050	Test Run Criteria and Test Period time for detection of data integrity f...	1
2051	Test Run Criteria and Test Period time for detection of data integrity f...	1
2052	Test Run Criteria and Test period time for detection of programme execution fault	1
2053	Test Run Criteria for detection of internal device communication error	1
2054	Test Run Criteria for fault detection in internal circuitsk	1
2055	Base DTC value for comparison of system behaviour with predictive model	1
2056	Base DTC value for detection of closed loop control system out of control	1
2057	Base DTC value for detection of failure to react after event	1
2058	Base DTC value for detection of other faults inferred from system behaviour	1
2059	Base DTC value for detection of system adaptation out of range	1
2060	Comparison of system behaviour with predictive model	1
2061	Detection of closed loop control system out of control	1
2062	Detection of failure to react after event	1
2063	Detection of other faults inferred from system behaviour	1
2064	Detection of system adaptation out of range	1
2065	Failure type value for comparison of system behaviour with predictive model	1
2066	Failure type value for detection of closed loop control system out of co...	1
2067	Failure type value for detection of failure to react after event	1
2068	Failure type value for detection of other faults inferred from system be...	1
2069	Failure type value for detection of system adaptation out of range	1
2070	Test period time for comparison of system behaviour with predictive model	1
2071	Test period time for detection of closed loop control system out of control	1
2072	Test period time for detection of failure to react after event	1
2073	Test period time for detection of other faults inferred from system beha...	1
2074	Test Run Criteria for comparison of system behaviour with predictive model	1
2075	Test Run Criteria for detection of closed loop control system out of con...	1
2076	Test Run Criteria for detection of failure to react after event	1
2077	Test Run Criteria for detection of faults inferred from system behaviour	1
2078	Test Run Criteria for detection of system adaptation out of range	1
2079	Accuracy of vehicle battery voltage measurement	1
2080	AgedDTCLimit or low supply voltage fault detection	1
2081	Averaging of ECU supply voltage measurement	1
2082	Averaging of vehicle battery voltage measurement	1
2083	ConfirmedDTCLimit for low supply voltage fault detection	1
2084	Distribution of the vehicle battery voltage value	1
2085	ECU supply voltage measurement	1
2086	Sample time for ECU supply voltage measurement	1
2087	Sample time for vehicle battery voltage measurement	1
2088	Test Run Criteria for low supply voltage fault detection	1

2089	Test sample failed criteria for low supply voltage fault detection	1
2090	Test sample passed criteria for low supply voltage fault detection	1
2091	UnconfirmedDTCLimit for low supply voltage fault detection	1
2092	Vehicle battery voltage measurement	1
2093	Compliance to ISO14229-1	1
2094	Compliance to ISO26021-1	1
2095	Compliance to ISO26262	1
2096	Diagnostic data for private ECUs	1
2097	Support for Fault Detection Counter #10	1
2098	Support for Fault Detection Counter #11	1
2099	Support for Fault Detection Counter #12	1
2100	DTC Failure Type Byte	1
2101	DTCs defined by GMRDB	1
2102	DTCs for faults caused by the ECU operating environment or by customer	1
2103	DTCs for secondary faults caused by data from other ECU indicated as invalid	1
2104	DTCs for secondary faults caused by data from other ECU not indicated as invalid	1
2105	DTCs that are needed only during the development of the ECU	1
2106	DTCs that are needed only during the development of the ECU - implementer specified	1
2107	ECU operation cycle	1
2108	Emissions related DTCs defined by SAEJ2012	1
2109	Non emissions related DTCs defined by SAEJ2012	1
2110	Reporting of DTC information	1
2111	Support for Operation Cycle Counter #1	1
2112	Support for Operation Cycle Counter #2	1
2113	Support for Operation Cycle Counter #3	1
2114	Support for Operation Cycle Counter #4	1
2115	Support for Operation Cycle Counter #5	1
2116	Support for Operation Cycle Counter #6	1
2117	Support for Operation Cycle Counter #7	1
2118	Data records not included in local snapshot data	1
2119	Order of multple samples of data records in global and local snapshot data records	1
2120	Sampling criteria for DTCSnapshotRecordNumber 0x20, single sample	1
2121	Sampling criteria for DTCSnapshotRecordNumber 0x21, single sample	1
2122	Sampling criteria for DTCSnapshotRecordNumber 0x30 to 0x3F, multiple samples	1
2123	Sampling criteria for DTCSnapshotRecordNumber 0x40 to 0x4F, single sample	1
2124	Sampling criteria for DTCSnapshotRecordNumber 0x50 to 0x5F, other samples	1
2125	Support for DTC status bit confirmedDTC	1
2126	Support for DTC status bit pendingDTC	1
2127	Support for DTC status bit testFailed	1
2128	Support for DTC status bit testFailedSinceLastClear	1
2129	Support for DTC status bit testFailedThisOperationCycle	1
2130	Support for DTC status bit testNotCompletedSinceLast-Clear	1
2131	Support for DTC status bit testNotCompletedThis-OperationCycle	1
2132	Support for DTC status bit warningIndicatorRequested	1
2133	Support for DTC status indicator #30	1
2134	Support for DTC status indicator #30, AgedDTC.	1
2135	Support for DTC status indicator #30, EmissionRelatedDTC	1
2136	Support for DTC status indicator #30, SymptomSinceLastClear	1
2137	Support for DTC status indicator #30, testFailedSinceLastClear/Aged	1

2138	Support for DTC status indicator #30, UnconfirmedDTC	1
2139	Support for DTC status indicator #30, UnconfirmedDTCSinceLastClear	1
2140	Support for DTC status indicator #30, UnconfirmedDTCThisOperationCyle	1
2141	Support for DTC status indicator #30, WarningIndicatorRequestedSinceLastClear	1
2142	Support for DTC time stamp #20	1
2143	Support for DTC time stamp #21	1
2144	Access to external analog input signal data records	1
2145	Access to external FM, PCM, PWM input signal data records	1
2146	Access to external output signal data records	1
2147	Access to external switch input signal data records	1
2148	Access to network signal data records - signals received by public ECU	1
2149	Access to network signal data records - signals transmitted by public ECU	1
2150	Access to other external signal data records defined by the implementer	1
2151	Access to power supply input signal data records	1
2152	Control of network signal data records - periodic signals transmitted by...	1
2153	External analog input signals update time	1
2154	External FM, PCM, PWM input signals update time	1
2155	External output signals update time - actual output signal	1
2156	External switch input signals update time	1
2157	for external analog input signal data records - application signal	1
2158	Individual control of external output signals - application signal	1
2159	Individual control of network signal data records - signals transmitted ...	1
2160	Individual control of other external signal data records defined by the ...	1
2161	Power supply input signals update time	1
2162	Read and control of the network signal data record 0xDD02 transmitted by public ECU	1
2163	Reporting external analog input signal data records while the input parameter value is under tester	1
2164	Reporting external FM, PCM, PWM input signal data records while the inpu...	1
2165	Reporting external switch input signal data records while the input para...	1
2166	Reporting other external signal data records defined by the implementer ...	1
2167	Reporting power supply input signal data records while the input paramet...	1
2168	Reporting received network signal data records while the input parameter...	1
2169	Support for external analog input signal data records - actual input signal	1
2170	Support for external FM, PCM, PWM input signal data records - actual inp...	1
2171	Support for external FM, PCM, PWM input signal data records - engineering units	1
2172	Support for external output signal data records - actual output signal	1
2173	Support for external output signal data records - application signal	1
2174	Support for external switch input signal data records actual input signal	1
2175	Support for external switch input signal data records - application signal	1

2176	Support for network signal data records - signals received by public ECU	1
2177	Support for network signal data records - signals transmitted by public ECU	1
2178	Support for other external signal data records defined by the implementer	1
2179	Support for power supply data record	1
2180	Access to input from internal HMI signal data records	1
2181	Access to other internal signal data records defined by the implementer	1
2182	Access to output to internal HMI signal data records	1
2183	Individual control of other ECU internal signal data records defined by ...	1
2184	Individual control of output to internal HMI signals	1
2185	Reporting input from internal HMI signal data records while the input pa...	1
2186	Reporting other ECU internal signal data records defined by the implemen...	1
2187	Support for input from internal HMI signal data records - actual input signal	1
2188	Support for input from internal HMI signal data records - application si...	1
2189	Support for other internal signal data records defined by the implementer	1
2190	Support for output to internal HMI signal data records	1
2191	Update time for input from internal HMI signal data records	1
2192	Access to Application Diagnostic Database Part Number data record	1
2193	Access to Complete ECU PartSerial Number data record	1
2194	Access to Diagnostic Autosar version number	1
2195	Access to ECU Core Assembly Part Number data record	1
2196	Access to ECU Delivery Assembly Part Number data record	1
2197	Access to ECU Serial Number data record	1
2198	Access to ECU Software Extended Part Numbers data record	1
2199	Access to ECU Software Part Numbers data record	1
2200	Access to Primary Bootloader Diagnostic Database Part Number	1
2201	Access to Primary Bootloader Software Part Number data record	1
2202	Access to Private ECU(s) or component(s) Serial Number data record	1
2203	Access to Secondary Bootloader Diagnostic Database Part Number	1
2204	Access to Secondary Bootloader Software Version Number data record	1
2205	Access to Vehicle Configuration Parameters data record	1
2206	Access to Vehicle Identification Number data record	1
2207	Access to Vehicle Information Section data record	1
2208	Application Diagnostic Database Part Number data record	1
2209	Autosar BSW cluster versions	1
2210	Complete ECU PartSerial Number data record	1
2211	ECU Core Assembly Part Number data record	1
2212	ECU Delivery Assembly Part Number data record	1
2213	ECU Serial Number data record	1
2214	ECU Software Extended Part Numbers data record	1
2215	ECU Software Part Number data record	1
2216	ECU Software Structure Part Number data record	1
2217	No read of the Security Constant data record	1
2218	One time write of the Security Constant data record	1
2219	Part number data records coding	1
2220	Primary Bootloader Diagnostic Database Part Number	1
2221	Primary Bootloader Software Part Number data record	1
2222	Private ECU(s) or component(s) Serial Number data record	1

2223	Secondary Bootloader Diagnostic Database Part Number	1
2224	Secondary Bootloader Software Version Number data record	1
2225	Secondary Bootloader Software Version Number data record coding	1
2226	Vehicle Configuration Parameters data record	1
2227	Vehicle Identification Number data record	1
2228	Vehicle Information Section data record	1
2229	Write of the Public Key data record	1
2230	Active diagnostic session data record	1
2231	Build List data record	1
2232	Car Configuration Parameter Faults data record	1
2233	Car Mode data record	1
2234	Control of Usage Mode data record	1
2235	Control or write of Car Mode data record	1
2236	Customer setting parameters data record	1
2237	Diagnostic Read out #1 to #4 data record	1
2238	ECU List data record	1
2239	Partial Network Cluster (PNC) data record	1
2240	Read Access to Active diagnostic session data record	1
2241	Read Access to Build List data record	1
2242	Read access to Car Configuration Parameter Faults data record	1
2243	Read access to Car Mode data record	1
2244	Read access to Customer setting parameters data record	1
2245	Read Access to Diagnostic Read out #1 to #4 data record	1
2246	Read Access to ECU List data record	1
2247	Read access to Partial Network Cluster (PNC) data record	1
2248	Read access to RVDC data record	1
2249	Read access to System adaption data record	1
2250	Read access to Usage Mode data record	1
2251	Read Access to Vehicle state of health data record	1
2252	Remote Vehicle Data Collection (RVDC) data record	1
2253	System adaption data record	1
2254	Usage Mode data record	1
2255	Vehicle state of health data record	1
2256	Write access to Customer setting parameters data record	1
2257	Write access to System adaption data record	1
2258	Definition of global real time data record	1
2259	Definition of total distance data record	1
2260	Definition of vehicle battery voltage data record	1
2261	Distribution of global DTC snapshot data	1
2262	Maintaining global real time after power loss	1
2263	Storage of global DTC snapshot data when not available	1
2264	Supported data records in global DTC snapshot data	1
2265	Storage of local DTC snapshot data when not available	1
2266	Support for local DTC snapshot data	1
2267	Access development specific data records	1
2268	Access Development specific data records - implementer specified	1
2269	Access Options-System Supplier Specific data records	1
2270	Access System Supplier Specific data records	1
2271	Access to DTC Global Snapshot Data records	1
2272	Access to ECU Identification and Configuration data records	1
2273	Access to Identification Option - ISO specified data records	1
2274	Access to Identification Option - Vehicle Manufacturer Specific data records	1
2275	Access to Vehicle manufacturer specific data records	1
2276	Data types and classifications	1
2277	Development specific data records	1
2278	Development specific data records - implementer specified	1
2279	DTC Global Snapshot Data records defined in GMRDB	1
2280	Dynamically Defined Data Identifiers in ascending order	1
2281	Dynamically defined periodic Data Identifiers in ascending	1

	order	
2282	ECU Identification and Configuration data records	1
2283	Identification Option - ISO specified data records	1
2284	Identification Option - Vehicle Manufacturer Specific data records	1
2285	Identification Options-System Supplier Specific data records	1
2286	Specification of supported services for each data record	1
2287	Supported services for Dynamically Defined Data Identifiers	1
2288	Supported services for Dynamically defined periodic Data Identifiers	1
2289	System Supplier Specific data records	1
2290	Vehicle manufacturer specific data records defined in GMRDB	1
2291	Check Complete & Compatible response time	1
2292	Check Memory response time	1
2293	Check Programming Pre-conditions response time	1
2294	Erase Memory response time	1
2295	Extended Erase Memory response time	1
2296	Extended Erase Memory response time - ECU Software Structure	1
2297	Software Installation response time	1
2298	Access to Vehicle manufacturer specific routines	1
2299	Development Specific routines	1
2300	ISO specified routines	1
2301	System Supplier Specific routines	1
2302	Vehicle manufacturer specific routines defined in GMRDB	1
2303	DTC storage time	1
2304	Storage of DTC fault detection counters 10 and 11 in short term memory	1
2305	Storage of DTC fault detection counters 12 in long term memory	1
2306	Storage of DTC information in short term memory	1
2307	Storage of DTC operation cycle counter 5 in long term memory	1
2308	Storage of DTC operation cycle counters 1, 2, 3, 4, 6 and 7 in long term memory	1
2309	Storage of DTC snapshot data in long term memory	1
2310	Storage of DTC status bits no 0, 2, 3, 4 and 5 in long term memory	1
2311	Storage of DTC status bits no 1, 6 and 7 in short term memory	1
2312	Storage of DTC status indicator bit no 1 in short term memory	1
2313	Storage of DTC status indicator bits no 0, 2-7 in long term memory	1
2314	Storage of DTC time stamp in long term memory	1
2315	Access to ActivateSBL routine	1
2316	Access to Check Complete & Compatible routine	1
2317	Access to Check Memory routine	1
2318	Access to Check Programming Pre-conditions routine	1
2319	Access to Erase Memory routine	1
2320	Access to Software Installation routine	1
2321	Access to Software Installation routine	1
2322	ActivateSBL routine	1
2323	ActivateSBL routine response	1
2324	ActivateSBL routine type	1
2325	Check Complete & Compatible routine	1
2326	Check Complete & Compatible routine type	1
2327	Check Memory routine	1
2328	Check Memory routine type	1
2329	Check Programming Pre-conditions routine	1
2330	Check Programming Pre-conditions routine type	1
2331	Erase Memory routine	1
2332	Erase Memory routine "already erased" detection	1
2333	Erase Memory routine type	1
2334	Software Installation routine	1
2335	Transfer of DTC information to long term memory	1

2336	Unique ECU address	1
2337	Routine result availability	1
2338	Routine Type 1- Short routine, definition	1
2339	Routine Type 1- Short routine, positive response on startRoutine	1
2340	RoutineInfo byte	1
2341	RoutineStatus, RoutineAborted	1
2342	RoutineStatus, routineCompleted	1
2343	RoutineStatus, RoutineExcecutes	1
2344	Type 1- Short routine, sub-function requestRoutineResults	1
2345	Type 2- Long routine, definition	1
2346	Type 2- Long routine, positive response on startRoutine	1
2347	Type 2- Long routine, sub-function RequestRoutineResults	1
2348	Type 2- Long routine, sub-function stopRoutine	1
2349	Type 3 - Continuous routine, definition	1
2350	Type 3 - Continuous routine, positive response on startRoutine	1
2351	Type 3 - Continuous routine, sub-function RequestRoutineResults	1
2352	Type 3 - Continuous routine, sub-function stopRoutine	1
2353	Maintain active diagnostic functionality at session transitions	1
2354	Reset active diagnostic functionality at session transitions	1
2355	SessionParameterRecord definition	1
2356	Allowed diagnostic services	1
2357	Diagnostic service specification	1
2358	Safety requirements	1
2359	ECU ability to receive a complete message	1
2360	ECU ability to receive requests	1
2361	ECU operation cycle	1
2362	ECU start up time	1
2363	Service clearDiagnosticInformation (14) affecting ECU functionality	1
2364	Service diagnosticSessionControl (10) - all other session transissions than from and to programmingS	1
2365	Service diagnosticSessionControl (10) - transition from and to programmi...	1
2366	Service dynamicallyDefineDataIdentifier (2C) affecting ECU functionality	1
2367	Service ECUReset (11) affecting ECU functionality	1
2368	Service inputOutputControlByIdentifier (2F) affecting ECU functionality	1
2369	Service readDataByIdentifier (22) affecting ECU functionality	1
2370	Service readDataByPeriodicIdentifier (2A) affecting ECU functionality	1
2371	Service ReadDTCInformation (19) affecting ECU functionality	1
2372	Service readGenericInformation (AF) affecting ECU functionality	1
2373	Service readMemoryByAddress (23) affecting ECU functionality	1
2374	Service routineControl (31) affecting ECU functionality	1
2375	Service securityAccess (27) affecting ECU functionality	1
2376	Service testerPresent (3E) affecting ECU functionality	1
2377	Service writeDataByIdentifier (2E) affecting ECU functionality	1
2378	Service writeMemoryByAddress (3D) affecting ECU functionality	1
2379	Entry conditions for diagnostic service ClearDiagnosticInformation (14)	1
2380	Entry conditions for diagnostic service DynamicallyDefineDataIdentifier (2C)	1
2381	Entry conditions for diagnostic service InputOutputControlByIdentifier (2F)	1
2382	Entry conditions for diagnostic service ReadDataByIdentifier (22)	1
2383	Entry conditions for diagnostic service	1

	ReadDataByPeriodicIdentifier (2A)	
2384	Entry conditions for diagnostic service ReadDTCInformation (19)	1
2385	Entry conditions for diagnostic service ReadGenericInformation (AF)	1
2386	Entry conditions for diagnostic service ReadMemoryByAddress (23)	1
2387	Entry conditions for diagnostic service RequestDownload (34)	1
2388	Entry conditions for diagnostic service RequestFileTransfer (38)	1
2389	Entry conditions for diagnostic service RequestTransferExit (37)	1
2390	Entry conditions for diagnostic service RequestUpload (35)	1
2391	Entry conditions for diagnostic service RoutineControl (31)	1
2392	Entry conditions for diagnostic service SecurityAccess (27)	1
2393	Entry conditions for diagnostic service TesterPresent (3E)	1
2394	Entry conditions for diagnostic service TransferData (36)	1
2395	Entry conditions for diagnostic service WriteDataByIdentifier (2E)	1
2396	Entry conditions for diagnostic service WriteMemoryByAddress (3D)	1
2397	Entry conditions for service ECUReset (11)	1
2398	Entry conditions for the service diagnosticSessionConrol (10), changing to programmingSession (02)	1
2399	Entry conditions for the service diagnosticSessionConrol (10), other ses...	1
2400	Exit conditions during diagnostics services	1
2401	Functional addressing specific networks	1
2402	Functional request shall be unsegmented on all networks	1
2403	Logical address for external diagnostic tester	1
2404	Logical address range for periodic response messages	1
2405	Negative response code busyRepeatRequest (21)	1
2406	Negative response code generalReject (10)	1
2407	Negative response code other than generalReject (10) and busyRepeatRequest (21).	1
2408	Allocating ECU addresses	1
2409	Assigning domain ID address	1
2410	Assigning ECU ID address	1
2411	Assigning network ID address	1
2412	If a diagnostic service makes a reset, the response shall be sent before...	1
2413	Non volatile data shall be written before responding on the diagnostic request	1
2414	P4Server_max for services not supported by the ECU	1
2415	P4Server_max response time for clearDiagnosticInformation (14)	1
2416	P4Server_max response time for diagnosticSessionControl (10)	1
2417	P4Server_max response time for dynamicallyDefineDataIdentifier (2C)	1
2418	P4Server_max response time for ECUReset (11)	1
2419	P4Server_max response time for inputOutputControlByIdentifier (2F)	1
2420	P4Server_max response time for readDataByIdentifier (22)	1
2421	P4Server_max response time for readDataByPeriodicIdentifier (2A)	1
2422	P4Server_max response time for readDTCInformation (19)	1
2423	P4Server_max response time for readGenericInformation (AF)	1
2424	P4Server_max response time for readMemoryByAddress (23)	1
2425	P4Server_max response time for requestDownload (34)	1
2426	P4Server_max response time for requestFileTransfer (38)	1
2427	P4Server_max response time for requestTransferExit (37)	1
2428	P4Server_max response time for requestUpload (35)	1
2429	P4Server_max response time for routineControl (31) except for startRoutine (1), routineType = 1	1

2430	P4Server_max response time for routineControl (31) startRoutine (1), rou...	1
2431	P4Server_max response time for securityAccess (27)	1
2432	P4Server_max response time for testerPresent (3E)	1
2433	P4Server_max response time for transferData (36)	1
2434	P4Server_max response time for writeDataByIdentifier (2E)	1
2435	P4Server_max response time for writeMemoryByAddress (3D)	1
2436	Response time on the diagnostic service requests	1
2437	Security access for service diagnosticSessionControl (10)	1
2438	Security access protection for service clearDiagnosticInformation (14)	1
2439	Security access protection for service dynamicallyDefineDataIdentifier (2C)	1
2440	Security access protection for service ECUReset (11)	1
2441	Security access protection for service inputOutputControlByIdentifier (2F)	1
2442	Security access protection for service readDataByIdentifier (22), other than system supplier specifi	1
2443	Security access protection for service readDataByIdentifier (22), system supplier specific dataIdent	1
2444	Security access protection for service readDataByPeriodicIdentifier (2A)	1
2445	Security access protection for service readDTCInformation (19)	1
2446	Security access protection for service readGenericInformation (AF)	1
2447	Security access protection for service readMemoryByAddress (23)	1
2448	Security access protection for service requestDownload (34)	1
2449	Security access protection for service requestFileTransfer (38)	1
2450	Security access protection for service requestTransferExit (37)	1
2451	Security access protection for service requestUpload (35)	1
2452	Security access protection for service routineControl (31)	1
2453	Security access protection for service testerPresent (3E)	1
2454	Security access protection for service transferData (36)	1
2455	Security access protection for service writeDataByIdentifier (2E)	1
2456	Security access protection for service writeMemoryByAddress (3D)	1
2457	DynamicallyDefineDataIdentifier (2C)	1
2458	DynamicallyDefineDataIdentifier (2C) - clear of dynamicallyDefineDataIdentifier	1
2459	DynamicallyDefineDataIdentifier (2C) - clearDynamicallyDefinedDataIdenti...	1
2460	DynamicallyDefineDataIdentifier (2C) - clearDynamicallyDefinedDataIdenti...	1
2461	DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01, 81)	1
2462	DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - dynamic...	1
2463	DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - memorySize	1
2464	DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - positio...	1
2465	DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - sourceD...	1
2466	DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - addressAndLengthFormatIdentifier	1
2467	DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - addressAndLengthFormatIdentifier	1
2468	DynamicallyDefineDataIdentifier (2C) -	1

	defineByMemoryAddress (02) - dynamicallyDefinedDataIdentifier	
2469	DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - memo...	1
2470	DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - memo...	1
2471	DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02, 82)	1
2472	ReadDataByIdentifier (22) - dataIdentifier(-s)	1
2473	ReadDataByIdentifier (22) - multiple identifiers with one request	1
2474	ReadDataByPeriodicIdentifier (2A)	1
2475	ReadDataByPeriodicIdentifier (2A) - periodicDataIdentifier	1
2476	ReadDataByPeriodicIdentifier (2A) - response message type	1
2477	ReadDataByPeriodicIdentifier (2A) - transmissionMode fast (03)	1
2478	ReadDataByPeriodicIdentifier (2A) - transmissionMode medium (02)	1
2479	ReadDataByPeriodicIdentifier (2A) - transmissionMode slow (01)	1
2480	ReadDataByPeriodicIdentifier (2A) - transmissionMode stop (04)	1
2481	ReadMemoryByAddress (23)	1
2482	ReadMemoryByAddress (23) - addressAndLengthFormatIdentifier (14)	1
2483	ReadMemoryByAddress (23) - addressAndLengthFormatIdentifier (24)	1
2484	ReadMemoryByAddress (23) - memoryAddress	1
2485	ReadMemoryByAddress (23) - memorySize	1
2486	WriteDataByIdentifier (2E)	1
2487	WriteDataByIdentifier (2E) - dataIdentifier	1
2488	WriteDataByIdentifier (2E) - dataRecord	1
2489	WriteMemoryByAddress (3D)	1
2490	WriteMemoryByAddress (3D) - addressAndLengthFormatIdentifier (14)	1
2491	WriteMemoryByAddress (3D) - addressAndLengthFormatIdentifier (24)	1
2492	WriteMemoryByAddress (3D) - dataRecord	1
2493	WriteMemoryByAddress (3D) - memoryAddress	1
2494	WriteMemoryByAddress (3D) - memorySize	1
2495	DiagnosticSessionControl (10)	1
2496	DiagnosticSessionControl (10) defaultSession (01, 81)	1
2497	DiagnosticSessionControl (10) extendedDiagnosticSession (03, 83)	1
2498	DiagnosticSessionControl (10) programmingSession (02, 82)	1
2499	DiagnosticSessionControl (10) - sessionParameterRecord	1
2500	ECUReset (11)	1
2501	ECUReset (11) - hardReset (01, 81)	1
2502	SecurityAccess (27)	1
2503	SecurityAccess (27) - requestSeed (01, 81)	1
2504	SecurityAccess (27) - requestSeed (03-1F, 83-9F)	1
2505	SecurityAccess (27) - requestSeed (21-3F, A1-BF)	1
2506	SecurityAccess (27) - securityAccess algorithm and constant in the lower requestSeed range	1
2507	SecurityAccess (27) - securityAccess algorithm and constant in the higher requestSeed range	1
2508	SecurityAccess (27) - sendKey (02, 82)	1
2509	SecurityAccess (27) - sendKey (04-20, 84-A0)	1
2510	SecurityAccess (27) - sendKey (22-40, A2-C0)	1
2511	SecurityAccess (27) - valid seed	1
2512	TesterPresent (3E) - zeroSubFunction (00, 80)	1

2513	InputOutputControlByIdentifier (2F)	1
2514	InputOutputControlByIdentifier (2F) - Consecutive shortTermAdjustment re...	1
2515	InputOutputControlByIdentifier (2F) - controlMask	1
2516	InputOutputControlByIdentifier (2F) - ControlMask bit values of returnCo...	1
2517	InputOutputControlByIdentifier (2F) - controlState data record	1
2518	InputOutputControlByIdentifier (2F) - controlState parameter of the posi...	1
2519	InputOutputControlByIdentifier (2F) - dataIdentifier	1
2520	InputOutputControlByIdentifier (2F) - inputOutputControlParameter freezeCurrentState (02)	1
2521	InputOutputControlByIdentifier (2F) - inputOutputControlParameter returnControlToECU (00)	1
2522	InputOutputControlByIdentifier (2F) - inputOutputControlParameter shortTermAdjustment (03)	1
2523	InputOutputControlByIdentifier (2F) - Static substituted values	1
2524	InputOutputControlByIdentifier (2F) - Temporarily controlling	1
2525	RoutineControl (31)	1
2526	RoutineControl (31) - RoutineControlType requestRoutineResults (03)	1
2527	RoutineControl (31) - RoutineControlType requestRoutineResults (03) - ro...	1
2528	RoutineControl (31) - RoutineControlType startRoutine - routineControlOptionRecord	1
2529	RoutineControl (31) - RoutineControlType startRoutine (01)	1
2530	RoutineControl (31) - RoutineControlType startRoutine (01) - routineIden...	1
2531	RoutineControl (31) - RoutineControlType stopRoutine (02)	1
2532	RoutineControl (31) - RoutineControlType stopRoutine (02) - routineControlOptionRecord	1
2533	RoutineControl (31) - RoutineControlType stopRoutine (02) - routineIdent...	1
2534	ClearDiagnosticInformation (14)	1
2535	ClearDiagnosticInformation (14) - groupOfDTC All DTCs in emission related ECU	1
2536	ClearDiagnosticInformation (14) - groupOfDTC All Groups	1
2537	ClearDiagnosticInformation (14) - groupOfDTCSpecific DTC	1
2538	Data to be reported in ReadGenericInformation positive repsonse	1
2539	Definition of ReadGenericInformation positive response message data para...	1
2540	Definitions of ReadGenericInformation sub-function and data parameters	1
2541	Format of ReadGenericInformation request message	1
2542	ReadDTCInformation (19)	1
2543	ReadDTCInformation (19) - reportDTCByStatusMask (02, 82)	1
2544	ReadDTCInformation (19) - reportDTCByStatusMask (02, 82) - DTCStatusMask	1
2545	ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 84)	1
2546	ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 86...	1
2547	ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 86...	1
2548	ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 86...	1
2549	ReadDTCInformation (19) - reportDTCSnapshotIdentification 03, 83)	1
2550	ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84)	1

2551	ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84) - ...	1
2552	ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84) - ...	1
2553	ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84)- D...	1
2554	ReadDTCInformation (19) - reportDTCWithPermanentStatus (15)	1
2555	ReadDTCInformation (19) - reportSupportedDTC (0A)	1
2556	ReadGenericInformation (AF)	1
2557	ReadGenericInformation (AF) - reportDTCSnapshotByDTCNumber (04, 84)	1
2558	ReadGenericInformation (AF) - reportDTCSnapshotByDTCNumber (04, 84) - DT...	1
2559	ReadGenericInformation (AF) - reportDTCSnapshotByDTCNumber (04, 84) - DT...	1
2560	ReadGenericInformation (AF) - reportGenericExtendedDataByDTCNumber (06, ...	1
2561	ReadGenericInformation (AF) - reportGenericExtendedDataByDTCNumber (06, ...	1
2562	ReadGenericInformation (AF) - reportGenericExtendedDataByDTCNumber (06, 86)	1
2563	ReadGenericInformation negative response code conditionsNotCorrect	1
2564	ReadGenericInformation negative response code incorrectMessageLengthOrInvalidFormat	1
2565	ReadGenericInformation negative response code order of priority	1
2566	ReadGenericInformation negative response code requestCorrectlyReceived-ResponsePending	1
2567	ReadGenericInformation negative response code requestOutOfRange	1
2568	ReadGenericInformation negative response code subFunctionNotSupported	1
2569	ReadGenericInformation positive response message data parameters definie...	1
2570	Response message format for ReadGenericInformation - reportGenericExtend...	1
2571	Response message format for ReadGenericInformation - reportGenericSnapsh...	1
2572	Values of DTCExtendedDataRecordNumber	1
2573	Values of DTCMaskRecord	1
2574	Values of DTCMaskRecord	1
2575	Values of DTCSnapShotRecordNumber	1
2576	RequestDownload - addressAndLengthFormatIdentifier (44)	1
2577	RequestDownload - dataFormatIdentifier (00)	1
2578	RequestDownload - dataFormatIdentifier (01-0F and 11-7F and 81-FF)	1
2579	RequestDownload - dataFormatIdentifier (10)	1
2580	RequestDownload - dataFormatIdentifier (80)	1
2581	RequestDownload - memoryAdress	1
2582	RequestDownload - memorySize	1
2583	RequestDownload (34)	1
2584	RequestFileTransfer - dataFormatIdentifier (00)	1
2585	RequestFileTransfer - dataFormatIdentifier (01-0F and 11-7F and 81-FF)	1
2586	RequestFileTransfer - dataFormatIdentifier (10)	1
2587	RequestFileTransfer - dataFormatIdentifier (80)	1
2588	RequestFileTransfer - filePathAndName	1
2589	RequestFileTransfer - filePathAndNameLength	1
2590	RequestFileTransfer - fileSizeCompressed	1

2591	RequestFileTransfer - fileSizeParameterLenght	1
2592	RequestFileTransfer - fileSizeUnCompressed	1
2593	RequestFileTransfer - modeOfOperation (01-03)	1
2594	RequestFileTransfer - modeOfOperation (03 - ReplaceFile)	1
2595	RequestFileTransfer - modeOfOperation (04-05)	1
2596	RequestFileTransfer (38)	1
2597	RequestTransferExit (37)	1
2598	RequestTransferExit (37) transferResponseParameterRecord	1
2599	RequestTransferExit (37) transferResponseParameterRecord - algorithm	1
2600	RequestTransferExit (37) transferResponseParameterRecord - algorithm	1
2601	RequestTransferExit (37) transferResponseParameterRecord - use non-volatile memory	1
2602	RequestUpload - addressAndLengthFormatIdentifier (44)	1
2603	RequestUpload - dataFormatIdentifier	1
2604	RequestUpload - memoryAdress	1
2605	RequestUpload - memorySize	1
2606	RequestUpload (35)	1
2607	TransferData - blockSequenceCounter	1
2608	TransferData - transferRequestParameterRecord	1
2609	TransferData - transferResponseParameterRecord	1
2610	TransferData (36)	1
2611	TransferData (36) blockSequenceCounter handling	1
2612	TransferData (36) blockSequenceCounter received with the same value	1
2613	Session Layer	1
2614	P4Server_max definition	1
2615	P4Server_max equal to P2Server_max	1
2616	Order of responses in programming session	1
2617	Server behavior for additional client requests in programming session	1
2618	Server support of queued requests in programming session	1
2619	Start of P2Server for queued request in programming session	1
2620	P2(star)Server_max	1
2621	P2(star)Server_min	1
2622	P2Server_max - non programming session	1
2623	P2Server_max - ProgrammingSession	1
2624	P2server_min	1
2650	Addressing USDT frames	1
2651	Application bootloader requirement	1
2652	Application layer	1
2653	CAN data link layer	1
2654	CAN physical layer	1
2655	Diagnostic Session Layer	1
2656	Transport_Network layer	1
2657	Application diagnostic data	1
2658	Diagnostic services	1
2659	Bus-off	1
2660	Bus-off _ Decrease FDC10 steps	1
2661	Bus-off _ DTC ID	1
2662	Bus-off _ DTCAgedLimit	1
2663	Bus-off _ DTCConfirmedLimit	1
2664	Bus-off _ Failure type	1
2665	Bus-off _ FDC10 max value	1
2666	Bus-off _ Increase FDC10 steps	1
2667	Bus-off _ Test period	1
2668	Bus-off _ Test sample failed criteria	1
2669	Bus-off _ Test sample passed criteria	1
2670	Bus-off _ TRC	1
2671	Bus-off _ UnconfirmedDTCLimit	1
2672	CANH circuit short to battery voltage	1

2673	CANH circuit short to battery voltage _ Decrease FDC10 steps	1
2674	CANH circuit short to battery voltage _ DTC ID	1
2675	CANH circuit short to battery voltage _ DTCAgedLimit	1
2676	CANH circuit short to battery voltage _ DTCConfirmedLimit	1
2677	CANH circuit short to battery voltage _ Failure type	1
2678	CANH circuit short to battery voltage _ FDC10 max value	1
2679	CANH circuit short to battery voltage _ Increase FDC10 steps	1
2680	CANH circuit short to battery voltage _ Test period	1
2681	CANH circuit short to battery voltage _ Test sample failed criteria	1
2682	CANH circuit short to battery voltage _ Test sample passed criteria	1
2683	CANH circuit short to battery voltage _ TRC	1
2684	CANH circuit short to battery voltage _ UnconfirmedDTCLimit	1
2685	CANH circuit short to CANL circuit	1
2686	CANH circuit short to CANL circuit _ Decrease FDC10 steps	1
2687	CANH circuit short to CANL circuit _ DTC ID	1
2688	CANH circuit short to CANL circuit _ DTCAgedLimit	1
2689	CANH circuit short to CANL circuit _ DTCConfirmedLimit	1
2690	CANH circuit short to CANL circuit _ Failure type	1
2691	CANH circuit short to CANL circuit _ FDC10 max value	1
2692	CANH circuit short to CANL circuit _ Increase FDC10 steps	1
2693	CANH circuit short to CANL circuit _ Test period	1
2694	CANH circuit short to CANL circuit _ Test sample failed criteria	1
2695	CANH circuit short to CANL circuit _ Test sample passed criteria	1
2696	CANH circuit short to CANL circuit _ TRC	1
2697	CANH circuit short to CANL circuit _ UnconfirmedDTCLimit	1
2698	CANH circuit short to ground	1
2699	CANH circuit short to ground _ Decrease FDC10 steps	1
2700	CANH circuit short to ground _ DTC ID	1
2701	CANH circuit short to ground _ DTCAgedLimit	1
2702	CANH circuit short to ground _ DTCConfirmedLimit	1
2703	CANH circuit short to ground _ Failure type	1
2704	CANH circuit short to ground _ FDC10 max value	1
2705	CANH circuit short to ground _ Increase FDC10 steps	1
2706	CANH circuit short to ground _ Test period	1
2707	CANH circuit short to ground _ Test sample failed criteria	1
2708	CANH circuit short to ground _ Test sample passed criteria	1
2709	CANH circuit short to ground _ TRC	1
2710	CANH circuit short to ground _ UnconfirmedDTCLimit	1
2711	CANL circuit short to battery voltage	1
2712	CANL circuit short to battery voltage _ Decrease FDC steps	1
2713	CANL circuit short to battery voltage _ DTC ID	1
2714	CANL circuit short to battery voltage _ DTCAgedLimit	1
2715	CANL circuit short to battery voltage _ DTCConfirmedLimit	1
2716	CANL circuit short to battery voltage _ Failure type	1
2717	CANL circuit short to battery voltage _ FDC10 max value	1
2718	CANL circuit short to battery voltage _ Increase FDC10 steps	1
2719	CANL circuit short to battery voltage _ Test period	1
2720	CANL circuit short to battery voltage _ Test sample failed criteria	1
2721	CANL circuit short to battery voltage _ Test sample passed criteria	1
2722	CANL circuit short to battery voltage _ TRC	1
2723	CANL circuit short to battery voltage _ UnconfirmedDTCLimit	1
2724	CANL circuit short to ground	1
2725	CANL circuit short to ground _ Decrease FDC10 steps	1
2726	CANL circuit short to ground _ DTC ID	1
2727	CANL circuit short to ground _ DTCAgedLimit	1
2728	CANL circuit short to ground _ DTCConfirmedLimit	1
2729	CANL circuit short to ground _ Failure type	1

2730	CANL circuit short to ground _ FDC10 max value	1
2731	CANL circuit short to ground _ Increase FDC10 steps	1
2732	CANL circuit short to ground _ Test period	1
2733	CANL circuit short to ground _ Test sample failed criteria	1
2734	CANL circuit short to ground _ Test sample passed criteria	1
2735	CANL circuit short to ground _ TRC	1
2736	CANL circuit short to ground _ UnconfirmedDTCLimit	1
2737	Common CAN electrical fault	1
2738	Common CAN electrical fault _ Decrease FDC10 steps	1
2739	Common CAN electrical fault _ DTC ID	1
2740	Common CAN electrical fault _ DTCAgedLimit	1
2741	Common CAN electrical fault _ DTCConfirmedLimit	1
2742	Common CAN electrical fault _ Failure type	1
2743	Common CAN electrical fault _ FDC10 max value	1
2744	Common CAN electrical fault _ Increase FDC10 steps	1
2745	Common CAN electrical fault _ Test sample failed criteria	1
2746	Common CAN electrical fault _ Test sample passed criteria	1
2747	Common CAN electrical fault _ TRC	1
2748	Common CAN electrical fault _ UnconfirmedDTCLimit	1
2749	Common CAN electrical fault _Test period	1
2750	DTC Generic Test Run Criteria	1
2764	Permanent missing frame _ TRC 1	1
2765	Permanent missing frame _ TRC 2	1
2766	Permanent missing frame	1
2767	Permanent missing frame _ Test period missing frame master frame reception	1
2768	Permanent missing frame _ Test period missing frame slave	1
2769	Permanent missing frame _ Decrease FDC10 steps	1
2770	Permanent missing frame _ DTC ID	1
2771	Permanent missing frame _ DTCAgedLimit	1
2772	Permanent missing frame _ DTCConfirmedLimit	1
2773	Permanent missing frame _ Failure type	1
2774	Permanent missing frame _ FDC10 max value	1
2775	Permanent missing frame _ Increase FDC10 steps	1
2776	Permanent missing frame _ Test period missing frame master	1
2777	Permanent missing frame _ Test sample failed criteria	1
2778	Permanent missing frame _ TRC 10	1
2779	Permanent missing frame _ TRC 11	1
2780	Permanent missing frame _ TRC 3	1
2781	Permanent missing frame _ TRC 4	1
2782	Permanent missing frame _ TRC 5	1
2783	Permanent missing frame _ TRC 6	1
2784	Permanent missing frame _ TRC 7	1
2785	Permanent missing frame _ TRC 8	1
2786	Permanent missing frame _ TRC 9	1
2787	Permanent missing frame _ UnconfirmedDTCLimit	1
2788	Permanent missing frame _ Test sample passed criteria	1
2789	Separating type of electrical fault	1
2790	Network management history record data	1
2791	Network management state history buffer _ Identifier	1
3104	ECUs with multiple networks	1
3105	Gateway requirement	1
3106	Interrupted communication to private ECU	1
3107	Programming sequence for private ECU	1
3108	Read information from a private ECU	1
3109	Interrupted communication to the secondary processor	1
3110	Programming sequence for secondary processors	1
3111	Calibration and Customer data	1
3112	Support for (un)compressed data	1
3113	Delta encoding	1
3114	Disable monitoring and logging of DTCs	1
3115	Suspend non diagnostic communication	1

3116	ProgSignature - CLEAR	1
3117	ProgSignature - SET	1
3118	File download order	1
3119	Handling of distribution relays	1
3120	Internal data transfer speed	1
3121	Power Consumption	1
3122	Pre-calculated hash	1
3123	Separate memory sectors	1
3124	Multiple processors - verification principle	1
3125	Multiple processors - Main processor SBL verification	1
3126	Multiple processors - Secondary processor SBL verification	1
3127	Delta encoding - Verification	1
3128	Erase Status Information	1
3129	Check Memory - Return values	1
3130	Check Memory - verification of programmed data	1
3131	Secondary Bootloader - Condition for activation	1
3132	Secondary Bootloader Verification of unused data	1
3133	Check Memory - Application Start	1
3134	Erase Identifier - restrictions	1
3135	Request Download - restrictions on number of addressed logical blocks	1
3136	RequestUpload - Restricted use	1
3137	Software authentication signing method and algorithms	1
3138	Allocation of the Authenticity Verification function	1
3139	Authenticity verification at every start-up	1
3140	Authenticity verification at run time (main application running)	1
3141	Authenticity verification at software download (Program mode)	1
3142	Debug tools and ports	1
3143	Logical Block - Properties	1
3144	Logical Block Declaration	1
3145	Number of logical blocks	1
3146	Programming of Logical Blocks	1
3147	Software parts per logical block	1
3148	ECU Software Structure Declaration	1
3149	Erase memory when ECU Software Structure (ESS) is updated	1
3150	Immediately apply ESS update	1
3151	Reprogrammable ECU Software Structure (ESS)	1
3152	Restricted software download when no valid ESS stored	1
3153	Storage of the ECU Software Structure (ESS)	1
3154	Validation of the ECU Software Structure	1
3155	Root ECU Software Structure Declaration	1
3156	Public Key storage	1
3157	Re-programmable initial development key	1
3158	Condition for a logical block to be valid	1
3159	Conditions for a logical block to be invalid	1
3160	Validity Status Information - Properties	1
3161	Allocation of the Verification Block Table	1
3162	Verification Block Table - Reserve space at application build process	1
3163	Non-volatile memory for software storage	1
3164	Time window (Timeout_Prog)	1
3165	Two level bootloader	1
3166	PBL can only write data to volatile memory	1
3167	PBL in non-volatile memory	1
3168	Pre-condition state of the ECUs IO	1
3169	Protected boot sector	1
3170	Primary bootloader diagnostic data	1
3171	Primary bootloader diagnostic services	1
3172	Prevent the possibility to download the SBL when already activated	1
3173	SBL in volatile memory	1
3174	Support of erase and write data to non-volatile memory	1

3175	The SBL shall include all PBL functionality	1
3176	Secondary bootloader diagnostic data	1
3177	Secondary bootloader diagnostic services	1
3178	Complete and Compatible sequence diagram	1
3179	Validation of ECU Software(s)	1
3180	Complete and Compatible - Compatible function	1
3181	Complete&Compatible - Definition of being Compatible	1
3182	Complete and Compatible - Complete function	1
3183	Complete and Compatible - Definition of being Complete	1
3184	CompleteCompatibleFunction - error handling	1
3185	CompleteCompatibleFunction()	1
3186	CompleteCompatibleFunction() return value	1
3187	Maximum time to perform the Complete&Compatible validation	1
3188	Writing of received data	1
3240	Data compression Method 1 parameters	1
3241	Data compression Method 1 parameters	1
3242	Data compression Method 1 parameters	1
3243	Data compression method 1	1
3244	RequestDownload Service	1
3245	Application Layer	1
3246	Data Link Layer	1
3247	Supported CAN IDs	1
3248	Transport_Network Layer	1
3249	Diagnostic Session Layer	1
3250	Physical Layer	1
3251	Bootloader flow chart	1
3252	Complete hardware initiation when starting the ECU	1
3253	Cold or warm start detected	1
3254	Cold start detected	1
3255	Warm start detected	1
3256	ProgSignature is not SET	1
3257	ProgSignature is SET	1
3258	PROG received within Timeout_Prog	1
3259	No PROG received within Timeout_Prog	1
3260	Non complete or compatible software(s)	1
3261	Complete and compatible software(s)	1
3262	Receive and process diagnostic programming session requests	1
3263	S3Server timer timeout in PBL	1
3264	Reception of DiagnosticSessionControl(defaultSession)	1
3265	Reception of DiagSessionControl(programmingSession)	1
3266	Reception of the ECUReset service	1
3267	Enter bootloader from ECU application	1
3268	Write ProgSignature	1
3269	Trig a reset in ECU application mode	1
3476	Comments	1
3477	Header expression termination	1
3478	Header Identifiers	1
3479	IdentifierValue format	1
3480	Integers	1
3481	Non-printing characters	1
3482	White space characters	1
3483	Data section - start	1
3484	Data section - structure	1
3485	Software part version in the data section	1
3486	Data block checksum	1
3487	Header section	1
3488	Header section - Format	1
3489	call	1
3490	data_format_identifier	1
3491	description	1
3492	ecu_address	1
3493	erase	1
3494	file_checksum	1

3495	sw_current_part_number	1
3496	sw_current_version	1
3497	sw_part_number	1
3498	sw_part_type	1
3499	sw_signature	1
3500	sw_signature_dev	1
3501	sw_version	1
3502	verification_block_length	1
3503	verification_block_root_hash	1
3504	verification_block_start	1
3505	Version section	1
3506	File Naming	1
3507	VBF file generation	1
3508	Autosar configuration - required configuration values	1
3509	Autosar configuration - recommended values	1
3510	Simultaneous use of Profiles and Transformers	1
3511	Use of Profiles and Transformers	1
3512	E2E CRC Calculation	1
3513	E2E Data-Type Size	1
3514	E2E Protection Mechanisms	1
3515	E2E Protection Wrapper	1
3516	Hardware time stamping support	1
3517	Use of AUTOSAR BSW modules	1
3518	Definition of TCANPowerWakeUpToApp	1
3519	Definition of TCANResumeCom	1
3520	Timing at power up or reset initialization CAN	1
3521	Timing at resumed communication CAN - relay powered/ignition sensing node	1
3522	CAN network management history algorithm	1
3523	Diagnostic trouble code for CAN bus-off	1
3524	Definition of TCANEventWakeUpToApp	1
3525	Timing at network and hardware wake up CAN - microcontroller powered in sleep	1
3526	Timing at network and hardware wake up CAN - microcontroller unpowered in sleep	1
3537	Network management state history buffers	1
3538	Storage and clearing of network management state history buffers	1
3539	Minimum number of write events of history buffer	1
3558	Needed Tier1 requirements on Tier2 for AUTOSAR Diagnostic BSW	1
6286	Compatibility between versions	1
6287	Compression	1
6288	Data file type	1
6289	Delta file distribution	1
6290	Delta file name	1
6291	Delta file release information	1
6292	Delta file size	1
6293	Delta file structure	1
6294	Delta file version info	1
6295	Delta installation RAM buffer size	1
6296	Delta installation time	1
6297	New and obsolete data info	1
6298	Data decoding	1
6299	Data decompression	1
6300	Delta installation data	1
6301	Delta installation memory segment	1
6302	Delta installation RAM usage	1
6303	Delta memory segment size	1
6304	Delta SWDL in normal operational mode	1
6305	Delta SWDL in program mode	1
6306	Obsolete data removal	1

6307	Software consistency check	1
6308	Streamed installation	1
6309	Target sector erase check	1
6310	Consistency check failure	1
6311	Delta file corruption	1
6312	Error code mapping	1
6313	Full version install check	1
6314	Not enough memory	1
6315	Recover from interrupted Installation	1
6316	Subsequent reprogramming during recover	1
6317	Version mismatch	1
6411	In-Place Reconstruction of Delta Files	1
6429	Security Access Algorithm Description	1
6436	Privacy Regulations	1
6437	Privacy as Default	1
6438	Purpose and Consent	1
6443	Industry Standards	1
6446	Remove Debug Functionality	1
6448	System Hardening and Minimize Attack Surface	1
6453	Handling Anomalies	1
6455	Encryption keys and Algorithms	1
6456	Least Necessary Privilege	1
6459	Contact Persons	1
6461	Cyber Security Risk Assessment	1
6464	Incident and Patch Management	1
6465	Security Review Involvement	1
6466	Security Compliance Status review	1
6512	Event logging	1
6522	Password Protection	1
22265	CAN FD bit register settings for 2 Mbps	1
22281	CAN FD BlockSize parameter non-programming session server side	1
22292	Length of CAN FD frames	1
45485	Network management FR frame identifiers	1
96055	Terminating ECUs on CAN	1
123905	Data compression method 3	1
124148	Data compression Method 3 parameters	1
130849	Verification Block Table in a padded data block	1
130979	Software Settings - Definition	1
130980	Software Settings File - valid part numbers	1
132335	Data Compression Method Selection	1
132947	Data Compression Method 2	1
132949	Data Compression Method 2 Parameters	1
132953	Data Compression Method 2 Data Format	1
134130	file_checksum - no data blocks	1
134131	parameter_settings	1
134210	Bootloader CAN busoff handling	1

0.2 Requirements change log

0.2.1 New requirements in this issue

123905	Data compression method 3	1
124148	Data compression Method 3 parameters	1
130849	Verification Block Table in a padded data block	1
130979	Software Settings - Definition	1
130980	Software Settings File - valid part numbers	1
132335	Data Compression Method Selection	1
132947	Data Compression Method 2	1
132949	Data Compression Method 2 Parameters	1
132953	Data Compression Method 2 Data Format	1

134130	file_checksum - no data blocks	1
134131	parameter_settings	1
134210	Bootloader CAN busoff handling	1

0.2.2 Updated requirements in this issue

1597	Software authentication security classes	1
1599	Signing Method - Variability control	1
1600	Format of the Verification Block Table	1
1601	Verification Block Root Hash	1
1603	Verification Block Table Format Identifier - Definition	1
1604	Verification Block Table Header Identifiers	1
1605	Verification Block Table on unprocessed data	1
1608	Public Key Types	1
1610	Append signature to software part	1
1612	Signing function - Generate signature	1
1614	Sign a delta file	1
1629	ECU Addresses	1
1657	Network management CAN frame identifiers	1
1663	Bootloader routing - Spare networks	1
1664	Deployment of the J2534 requirements	1
1665	SWDL time in aftermarket	1
1666	SWDL time in factory	1
3112	Support for (un)compressed data	1
3113	Delta encoding	1
3135	Request Download - restrictions on number of addressed logical blocks	1
3136	RequestUpload - Restricted use	1
3139	Authenticity verification at every start-up	1
3156	Public Key storage	1
3157	Re-programmable initial development key	1
3160	Validity Status Information - Properties	1
3161	Allocation of the Verification Block Table	1
3240	Data compression Method 1 parameters	1
3241	Data compression Method 1 parameters	1
3242	Data compression Method 1 parameters	1
3243	Data compression method 1	1
3478	Header Identifiers	1
3483	Data section - start	1
3486	Data block checksum	1
3488	Header section - Format	1
3490	data_format_identifier	1
3498	sw_part_type	1
3499	sw_signature	1
3503	verification_block_root_hash	1
45485	Network management FR frame identifiers	1
96055	Terminating ECUs on CAN	1

0.2.3 Modifications in the updated requirements in this issue

For details, please look in the DETAILED CHANGE INFO

0.2.4 Deleted requirements in this issue

1 LIST OF REQUIREMENTS WITH DEVIATIONS

2 INTRODUCTION

2.1 What is the purpose of this document?

This document is a Base Technology SWRS. It contains the software requirements for implementation of the Base Technology concepts needed for an ECU to be integrated into the vehicle electrical system. This specification may contain generic requirements that are not applicable to specific ECUs.

2.2 Who should read this document?

This document is intended for:

- Developers of ECU software and hardware at the car OEM, ECU suppliers (Tier 1:s) or sub suppliers/contractors (Tier 2:s).
- Test engineers of electrical systems, ECUs or ECU software and hardware.

2.3 How is this document organized?

This document contains one or several Base Technology concept specifications. Each concept is located in a separate section under "ECU Software Requirements" and can be seen as a stand alone document. Each concept typically has its own introduction chapters containing reference lists, definition and abbreviation lists and similar information.

2.4 How do you receive more information?

More information can be found in the reference document lists. Questions and support issues may also be sent to the team within CEVT or Geely responsible for a specific ECU or to the central Base Technology team responsible for the specific project.

2.5 References to Volvo Cars

This document may contain references to Volvo Cars. Unless explicitly stated such references should be seen as a reference to the corresponding entity (document, tool, team, etc) at Geely or CEVT. Please refer to the ECU responsible team for further details.

There may also exist references to the requirement identifiers *"REQPROD"* that is used in Volvo Cars specifications. If available, they are listed as "Legacy ID" for the requirements of this specification.

3 REVISION HISTORY

Revision	Date	Description	Author
001	2017-XX-XX	Initial release of the Base Technology SWRS for the XXX ECU	<ECU SW responsible>
002	2018-12-19	Second release of the Base Technology SWRS for the IEM	Yu Jiaqi

4 ECU SOFTWARE REQUIREMENTS

ID	Allocated concept	Version	Status	Last Changed
x0400000000834AC3	AUTOSAR	(2)	CS Released	2018-04-25
x040000000083C952	Autosar 4.0.3 BSW - Tier1 to Tier2 Requirements	(2)	CS Released	2018-04-25
x040000000083E0AC	Autosar Configuration	(3)	CS Released	2018-04-25
x04000000000DE498	AutoSar NM - CAN & FlexRay additional reqs	(1)	CS Released	2017-03-10
x04000000000DE4A7	AutoSar NM - CAN reqs	(1)	CS Released	2017-03-10
x04000000000DE4AF	AutoSar NM - CAN wake up reqs	(1)	CS Released	2017-03-10
x04000000001E654B	AutoSar NM - Generic reqs	(2)	CS Released	2017-09-19
x0400000000114F233	Bootloader - UDSonCAN	(3)	CS Released	2018-10-02
x04000000006BC43D	CAN Data Link Layer	(4)	CS Released	2018-03-27
x0400000000EF16D5	CAN Network	(7)	CS Released	2018-08-13
x04000000001F377C	Car Configuration Slave	(2)	CS Released	2017-10-05
x04000000001D2FF3	Cyber sequrity requirements for all ECUs supporting SW download	(2)	CS Released	2017-08-30
x0400000000109E131	Data Compression and Encryption	(3)	CS Released	2018-09-27
x0400000000010D67E	Delta Encoding Introduction	(1)	CS Released	2017-03-10
x0400000000010D681	Delta generation	(1)	CS Released	2017-03-10
x0400000000012195D	Delta installation	(2)	CS Released	2017-03-14
x04000000011617B6	Diagnostics and ECU Platform	(2)	CS Released	2018-10-08
x04000000007C6C66	DoCAN	(4)	CS Released	2018-04-25
x04000000014E9611	ECU Addresses	(11)	CS Released	2018-11-26
x0400000000011C7AE	Extended information	(1)	CS Released	2017-03-10
x0400000000114DC20	General Bootloader requirements	(2)	CS Released	2018-10-05
x0400000001070429	General Software Authentication	(4)	CS Released	2018-09-27
x04000000010524E3	Generic SWDL	(2)	CS Released	2018-09-28
x0400000000011395D	Guideline for quality factor signals	(2)	CS Released	2017-03-10
x04000000000DE375	Guideline for Safety Related Communication	(1)	CS Released	2017-03-10
x04000000006A948E	Hardware Platform	(7)	CS Released	2018-03-22
x0400000000142C6A4	Network Management	(6)	CS Released	2018-11-16
x04000000000DD19B	Overview of Vehicle Configurations and Customer Settings	(1)	CS Released	2017-03-09
x04000000001D303F	Privacy in all ECUs	(2)	CS Released	2017-08-30
x04000000000DE46B	Random number generation	(1)	CS Released	2017-03-09
x0400000000837DC0	Security Access algorithm	(3)	CS Released	2018-04-25
x04000000014EC4A8	Software Download	(10)	CS Released	2018-11-26
x0400000000834ADF	Startup Time	(2)	CS Released	2018-04-25
x040000000085F9DE	UDS Data	(4)	CS Released	2018-04-25
x0400000000073CA44	UDS Services	(3)	CS Released	2018-04-25
x04000000000DE5F7	UDS Session ECU reqs	(1)	CS Released	2017-03-09
x04000000000DE5FA	UDS Session generic reqs	(1)	CS Released	2017-03-09
x040000000085A2CE	UDSonCAN - Generic	(3)	CS Released	2018-04-25
x040000000085A2D6	UDSonCAN - Public CAN ECU	(2)	CS Released	2018-04-25
x0400000000113B6AC	Versatile Binary Format Specification	(2)	CS Released	2018-09-27

4.1 Car Configuration

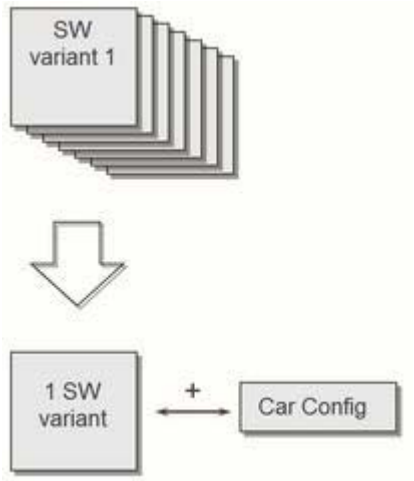
4.1.1 Central Car Configuration

4.1.1.1 Car Configuration Slave

4.1.1.1.1 Central Car Configuration Requirements

Central Car Configuration is a function to perform multiple ECU software configurations within a multiplexed network. Pre-defined car/functional parameters are generated by a Domain Master and distributed periodically to all other Domain Masters and sub-bus network ECUs within the platform architecture.

Main purpose of the CCC function is to minimize the necessary amount of ECU hardware and software variants with diversified functional behaviour (e.g. dependent on customer vehicle fitted parts and feature setup), i.e. connected/subscribing ECUs subscribe to CCC data in order to adapt to its current environment.



Car Configuration parameters are considered global parameters broadcasted within the vehicle network. CCPs are updated/modified through SWDL at EoL or aftersales. Customer settings/personalization, within an individual ECU SW, can interact with or override the global parameter setting for a specific function, but shall not alter the CCC data.

CCC ECU usage summary:

- Reduces the number of ECU software parts.
- Makes it possible to use the same software in an ECU - and configure the software “on-line”.
- Traditional software configuration can be combined with software configuration using Car Config.
- The software must be tested with all valid parameter settings - and not valid.
- Test work is not reduced.

Figures *CCC Function Solution* and *CCC Concept Overview* depicts further.

Figure: CCC Function Solution.

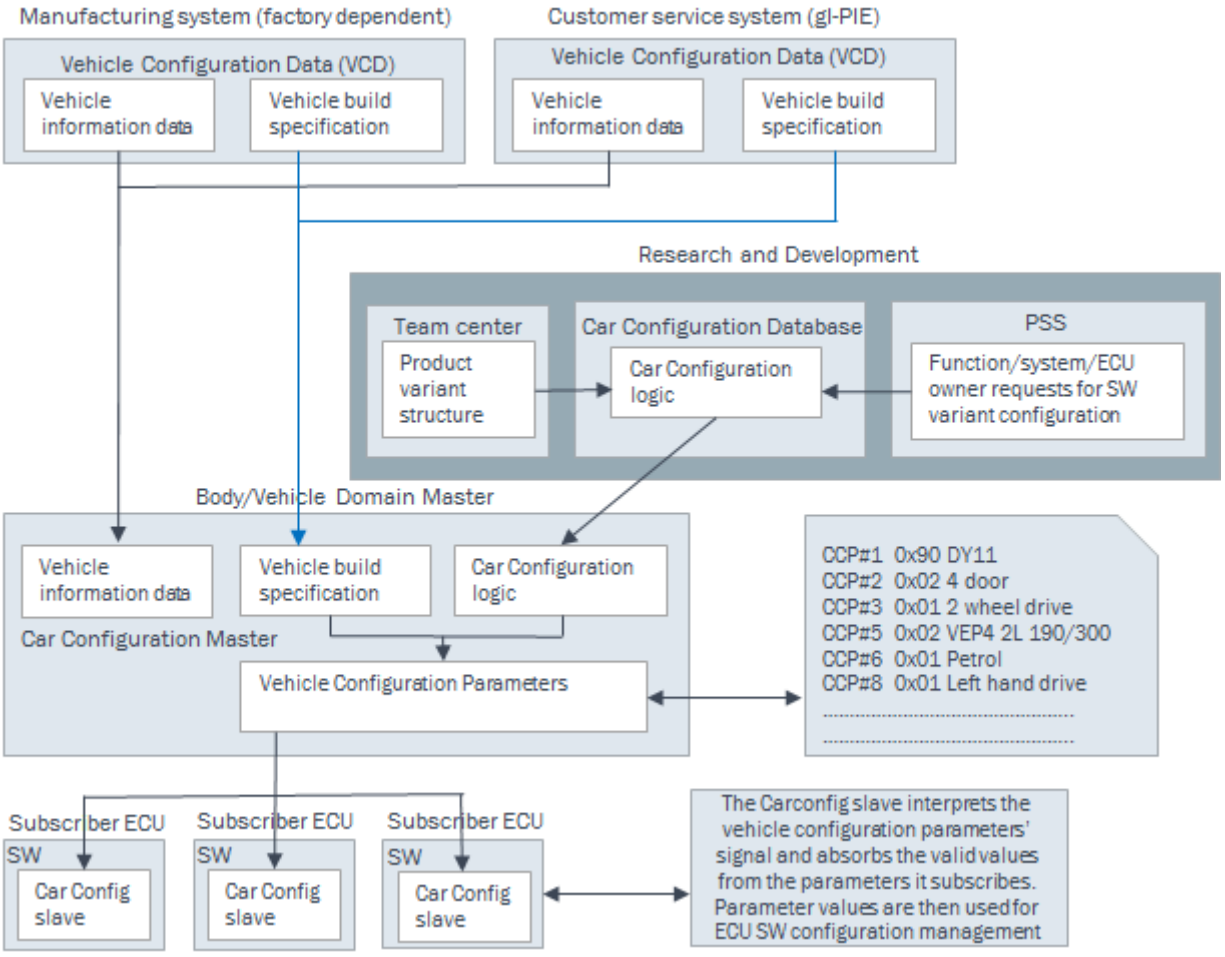
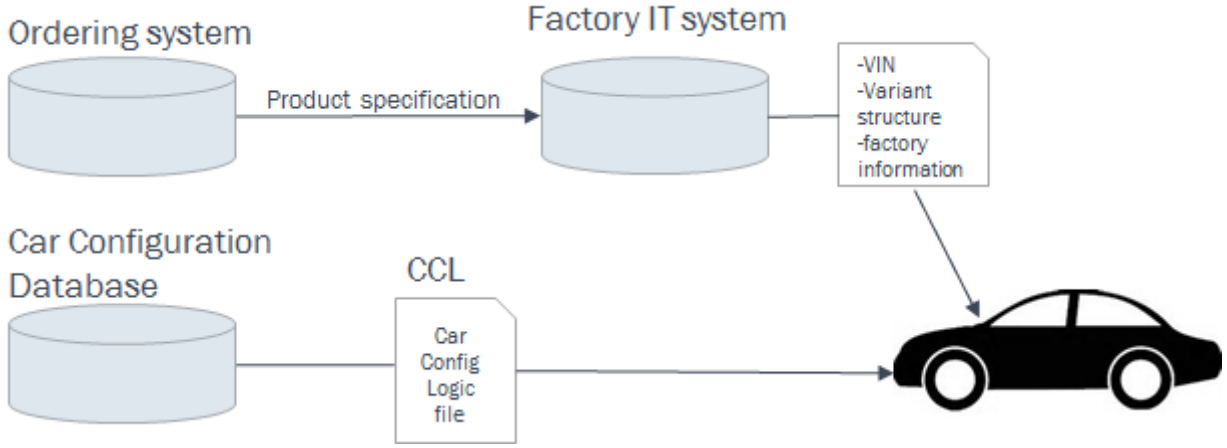


Figure: CCC Concept Overview.



4.1.1.1.1.1 Car Configuration Parameter Data

To support the working process some guidelines regarding administration and usage of CCPs are presented below. View figure *CCC Concept Overview* in chapter *Central Car Configuration* for concept overview and view chapter *Example of Car Config model* for usage example.

4.1.1.1.1.1.1 Parameter Creation

Function, subsystem or node developers request for parameter definitions and usage in order to configure specific functionality for the realizing system solution.

All CCP change requests shall be initiated according to the process stated for each project, which the CC team will monitor. If the configuration need can be achieved using the VCD and CCL, e.g. within a specific vehicle platform, the definition of the parameter (and its connected logical conditions) is handled in CCdB, by the CC team.

4.1.1.1.1.1.2 Parameter Properties

Each CCP shall have a defined PSS ownership, presumably a function, subsystem or node owner.
Each CCP shall also have a lifetime limit, in reality meaning that the parameter can be removed in case it is no longer used for configuration in any ECU SW.
Changes and additions to parameter definition shall be approved by the parameter owner and the CC team.
Each CCP size is 1 Byte, 254 possible hex values (excluding 0x00 & 0xFF).
View the table below for CCP example and associated data.

Table 1. Example of a CCP definition.

CCP#	CCP Name	CCP Description	CCP Value (Hex)	CCP Value Descriptions	VCP Block ID# (Hex)	VCP Byte Pos
13	Propulsion type	Level of vehicle electrification	01	Combustion engine only	2	7
			02	Hybrid vehicle		
			03	Plugin hybrid vehicle		

Note! '#'-sign is short for number in the above table. Also, every CCP shall have a defined serial number (CCP#), parameter description, value definitions and descriptions respectively. Each CCP# has a correspondent VCP signal position (BlockID# & Byte Position, view chapter "VCP signal mapping" for more information) for obtaining a set value when broadcasted within the network.

4.1.1.1.1.1.3 Information in CCdB

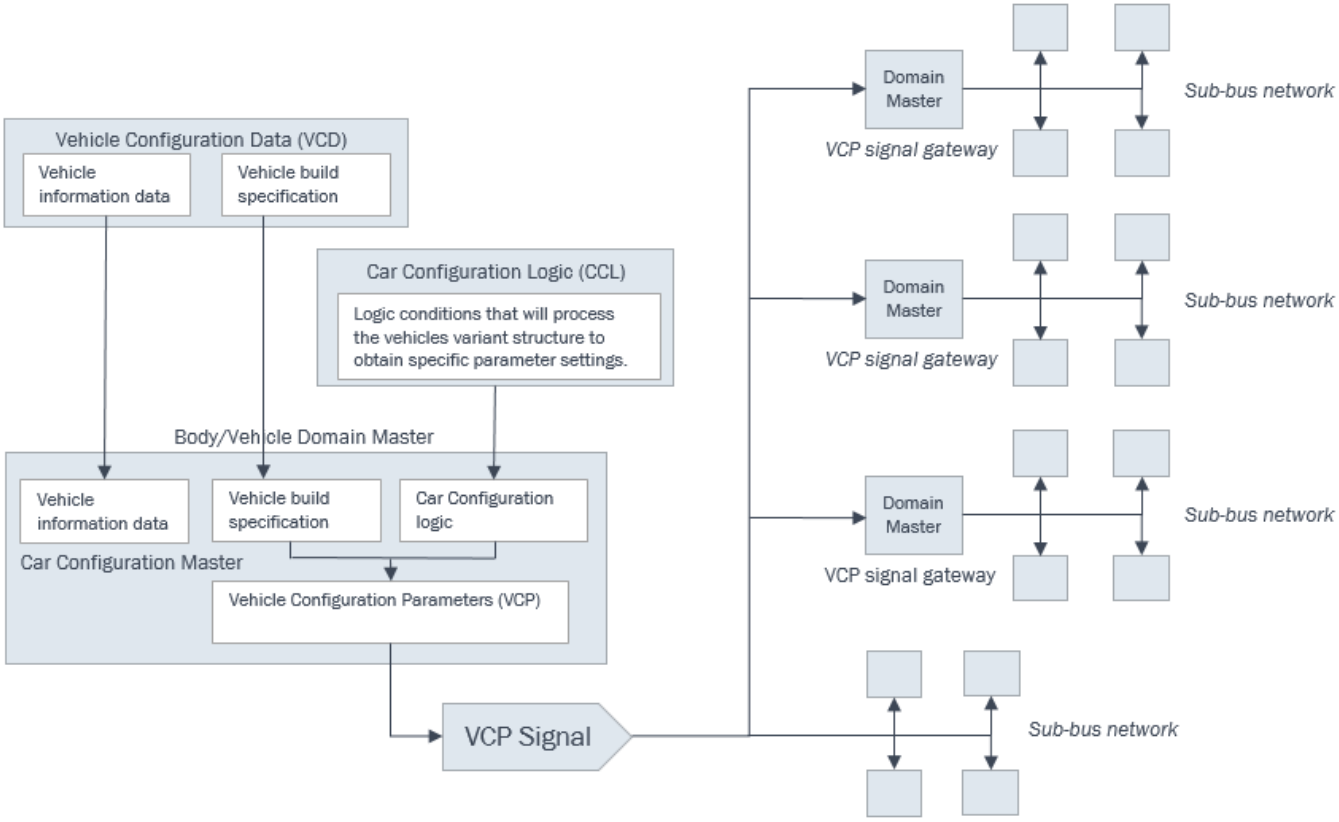
All information related to Car Configuration parameters shall exist in CCdB. This is information about parameter description & coding, generation logics, PSS ownership, which ECUs are allowed to subscribe to a parameter and which parameters a certain CC Subscriber ECU configures by.

4.1.1.1.1.1.4 Parameter Subscription

Each new ECU subscription to a CCP shall be requested to the CC team and approved by the parameter owner. It shall also be reported if a parameter is no longer used by a CC Subscriber ECU.

In addition, each new function/subsystem subscription to a CCP shall be approved by the parameter owner (see chapter *Parameter Properties*). I.e. even if a node is already using the parameter for one function a new application must be done if a new function in the node shall use the parameter.

4.1.1.1.1.2 System Design



The figure above gives a brief overview of the Car Configuration Management Sub-system design, details and requirements on actual implementation are given in *SRD Car Configuration Management System*, Requisite documents.

4.1.1.1.1.3 Car Configuration Subscriber Requirements

355 v7 **CC Subscriber ECU configuration states**

Version history:
(7):

Purpose: To define the possible states (including necessary handling) of CC Subscriber ECU configuration using allowed CCP data.

Source:

Verification Method: Test

Legacy ID: 65555v6

ASIL: QM - Not safety relevant

a) CC legacy general principles:

The ECU/system shall determine what appropriate action(s) are to be taken for a specific function/feature when utilizing a default configuration or validated CCP data. The ECU/system failure mode handling/operating strategy might contain additional routines that need to be handled and documented.

CC Subscriber ECUs shall implement default values for CCPs based on any required default operating condition so that the appropriate functionality can be achieved and where necessary failure mode management actions shall be active.

The default operating conditions are dependent on the strategic requirements of the ECU. The operation of the functions within the ECU strategy shall be in line with FMEA rules affecting safety of the vehicle and occupants and customer satisfaction in the absence of configuration information. Wherever possible the vehicle should operate normally. If the vehicle could be made dangerous or perform unsatisfactory for the customer/occupant, then failure mode management actions need to be executed to prevent or change this vehicle behaviour. The failure mode management actions shall be in respect of the ECU FMEA recommended actions.

b) Two CC Subscriber ECU states are defined as follows:

- **Bulk State:** In this state to receive required Car Config parameters for the first time and keep state until all have been evaluated valid:
  - The below applies to every CCP that the subscriber ECU expects to receive and evaluate:
    - Whenever a valid CCP value is available/received it shall be used.
    - For a received but not valid CCP value, use the ECU default value for that parameter instead.
- **Valid Configuration State:** In this state only to detect a possible change in car config parameter values:
  - The below applies to every CCP that the subscriber ECU expects to receive and evaluate:
    - For a received but not valid CCP value, consider using the previously stored valid value for that parameter instead.

CC Subscriber state guidelines provided for configuration using CCP data (distributed within the VCP signal) **in chapter *Guideline subscriber states*.**

c) Evaluation of a CCP value implies that (compared to whatever configuration the ECU/system currently utilizes) the ECU/system must determine if a received CCP value is valid, e.g. recognized, within valid range for acceptance, possible to transition to if ECU already has a Valid Configuration State etc.

356 v6 **CC Subscriber ECU in Bulk (unconfigured) state**

Version history:
(6):

Purpose: To identify an unconfigured/bulk state.
Source:
Verification Method: Test
Legacy ID: 65556v5
ASIL: QM - Not safety relevant

If the CC Subscriber ECU has not received or does not contain expected CCP data after 30 seconds from its application boot-up sequence while residing within ECU delivery default/Bulk State, the CC Subscriber ECU shall use *DTC "Car Configuration – Not Configured"* U2300-55, DTC test according to *[CCF_2] UDS Data*.

357 v9 **Routines for receiving VCP signal (CC Domain Master excluded)**

Version history:
(9):

Purpose: To assure flexibility of the CC Subscriber ECUs when receiving Car Configuration data.
Source:
Verification Method: Test
Legacy ID: 65561v7
ASIL: QM - Not safety relevant

a) The CC Subscriber ECU shall be able to read BlockID#, in the VCP signals, arriving in any order, in accordance with table *VCP signal mapping* in chapter *Record of tables.*

b) The CC Subscriber ECU shall be able to receive and process every arrival of a BlockID in the Vehicle configuration signals VehCfgPrm and VehCfgPrmExt.

358 v7 **Invalid or unrecognized parameter values**

Version history:
(7):

Purpose: To prevent any damage/security risk internally/externally in the CC Subscriber ECU associated with invalid or unrecognized parameter values.
Source:
Verification Method: Test
Legacy ID: 65582v6
ASIL: QM - Not safety relevant

If any CCP value(s) used for configuration (including values 00_H and FF_H) is received and evaluated invalid or unrecognized by the CC Subscriber ECU, the related function shall perform the following actions:

a) Enter a safe mode (depending on ECU strategy in line with FMEA rules) if there is an imminent risk of serious errors.

b) Use *DTC "Car Configuration –Invalid/Incompatible Configuration"* U2300-56, DTC test according to *[CCF_2] UDS Data*.

c) Minimize (avoid if possible) the impact on functions not related to parameter usage.

Note: For functions that are not safety nor security critical, whose behaviour/setting would be a similar or equal even if a received CCP value is not expected, a range of CCP values could be adopted as valid, e.g. CCP value range 0x01 to 0xFE is valid for configuration.

359 v7 **Configuration of ECU with updated CCP data**

Version history:
(7):

Purpose: To enable automatic configuration/re-configuration of ECU SW using updated CCP data.
Source:
Verification Method: Test
Legacy ID: 65583v6
ASIL: QM - Not safety relevant

a) The CC Subscriber ECU shall have two separate states of configuration relating to Central Car Configuration parameters. The "Bulk State" is when the ECU is delivered/built-in without pre-programming, i.e. factory settings. The "Valid Configuration State" is when all expected/allowed CCP values are in use and have been received and evaluated valid at least once.

b) The CC Subscriber ECU is required to transition from "Bulk State" -> "Valid Configuration State" after a complete set of expected CCPs have been evaluated valid and used for SW configuration.

360 v4 Selection of CCPs in VCP signal

Version history:
(4):

Purpose: To ensure that the correct CC Subscriber ECUs get the correct Car Configuration data, and
to avoid misuse of CCP data resulting in unnecessary SW-updates when changes on CCP definitions occur.
Source:
Verification Method: Test
Legacy ID: 65584v3
ASIL: QM - Not safety relevant

The CC Subscriber ECU shall use information in a parameter list or equivalent (released/extracted from CCdB) containing the CCP numbers that the CC Subscriber ECU has requested use for or is required to configure by.

The BlockID# (in the VCP signal), and byte position, in which a specific CCP can be obtained, shall be according to table *VCP signal mapping* in chapter *Record of tables.*

361 v7 Car Configuration data storage

Version history:
(7):

Purpose: The CC Subscriber ECU must be able to maintain parameter values between normal drive cycles, ECU resets, power cycles and low voltage due to engine cranking.
Source:
Verification Method: Test
Legacy ID: 65585v6
ASIL: QM - Not safety relevant

All CC Subscriber ECUs shall store their valid CCP data used for configuration in non-volatile memory (NVM). The format/interpretation of the parameter(s) to be stored shall be specified in the SWRS of each ECU subscribing to that parameter.

362 v3 Car Configuration parameter definition

Version history:
(3):

Purpose: To assure that each Car Configuration parameter is used according to its original purpose.
Source:
Verification Method: Analysis
Legacy ID: 65586v2
ASIL: QM - Not safety relevant

A Car Configuration parameter shall have the same definition in the CC Subscriber ECU as in CCdB.

363 v4 Handling of parameter data

Version history:
(4):

Purpose: General handling when reading CCPs within VCP signal and to avoid faulty ECU and vehicle behaviour respectively.
Source:
Verification Method: Test
Legacy ID: 65587v3
ASIL: QM - Not safety relevant

a) CCP value 00_h indicates that the CCP is not in use (not defined), e.g. not used within vehicle project/platform or spare/reserved CCP for future use.

b) CCP value FF_h indicates that the CCP has been generated erroneously.
c) CCP value 00_h and FF_h shall not be used for any ECU SW configuration.

364 v6 ECU configuration after boot-up

Version history:
(6):

Purpose: Enabling correct/updated CC Subscriber ECU configuration after ECU boot-up. Evaluation check of received parameters, after ECU boot-up, expected to be run only once per driving cycle.
Source:
Verification Method: Test
Legacy ID: 65588v5
ASIL: QM - Not safety relevant

After ECU application boot-up sequence the CC Subscriber ECU shall use the current set of stored CCP values (or default configuration if no previous data received/available).

Once, as soon as a new set of parameter values is received, the CC Subscriber ECU shall evaluate the new parameter values, dependent on which configuration state the CC Subscriber ECU currently resides in (see requirement *CC Subscriber ECU configuration states*).

365 v7 **CC Subscriber ECU DID support**

Version history:
(7):

Purpose: To identify how many, and which parameter number(s), that are faulty, up to 10 faulty parameters.
Source:
Verification Method: Test
Legacy ID: 65589v6
ASIL: QM - Not safety relevant

a) When receiving invalid or unrecognized parameter values (according to requirement *Invalid or unrecognized parameter values*), the CC Subscriber ECU shall store the faulty parameter and related value.

b) The CC Subscriber ECU shall support read out of *DID "Car Configuration - Faulty Parameters Received"* (0xE103 as specified in *[CCF_2] UDS Data*), format according to the table below. It shall be possible to read out up to 10 faulty CCPs identified by the CC Subscriber ECU, required only for parameter #/IDs that can be represented using 2 bytes hexadecimal code. If more then 10 faulty parameters are identified, 10 faulty parameters shall be able to read out but the number of invalid parameters shall represent the total amount of faulty parameters.

Table: DID Format, Invalid/Incompatible CCPs.

(Byte 1)	Number of invalid parameters
Parameter 1 (Byte 2)	Faulty parameter (MSB)
Parameter 1 (Byte 3)	Faulty parameter (LSB)
Parameter 1 (Byte 4)	Faulty parameter value
Parameter 2 (Byte 5)	Faulty parameter (MSB)
Parameter 2 (Byte 6)	Faulty parameter (LSB)
Parameter 2 (Byte 7)	Faulty parameter value
…	…

4.1.1.1.1.4 Signals provided by the system

Table: SDB signal extract.

Signal name
VehCfgPrm

Publishing FA	Publishing object	Subscribing FA	Subscribing object
-	-	-	-

Publishing component	Subscribing component	Publishing media	Subscribing media	Gateways	Connection
CC Domain Master	-	-	-	-	-

Size [bits]	Signal type	Info type	Gen. type	Initial value	Group name
64	Sporadic	-	-	-	-

Publisher latency [ms]	Write interval [ms]	Subscriber latency [ms]	Max read interval	Max age	Update bit	Constant
-	125	-	40ms	-		-

Description:	Signal used for the distribution of Vehicle Configuration Parameters (VCP) #1 - #504. At least two consecutive cycles of blockID:s 0x01 - 0x48 (dec 1 - 72) are distributed within 30 seconds from transition of UsageMode to 'Active' or 'Driving'. CC Subscribers shall be able to handle a reduction of the write interval.
Encoding type:	View table *VCP signal mapping* in chapter *Record of tables*.

Signal name
VehCfgPrmExt

Publishing FA	Publishing object	Subscribing FA	Subscribing object
-	-	-	-

Publishing component	Subscribing component	Publishing media	Subscribing media	Gateways	Connection
CC Domain Master	-	-	-	-	-

Size [bits]	Signal type	Info type	Gen. type	Initial value	Group name
64	Sporadic	-	-	-	-

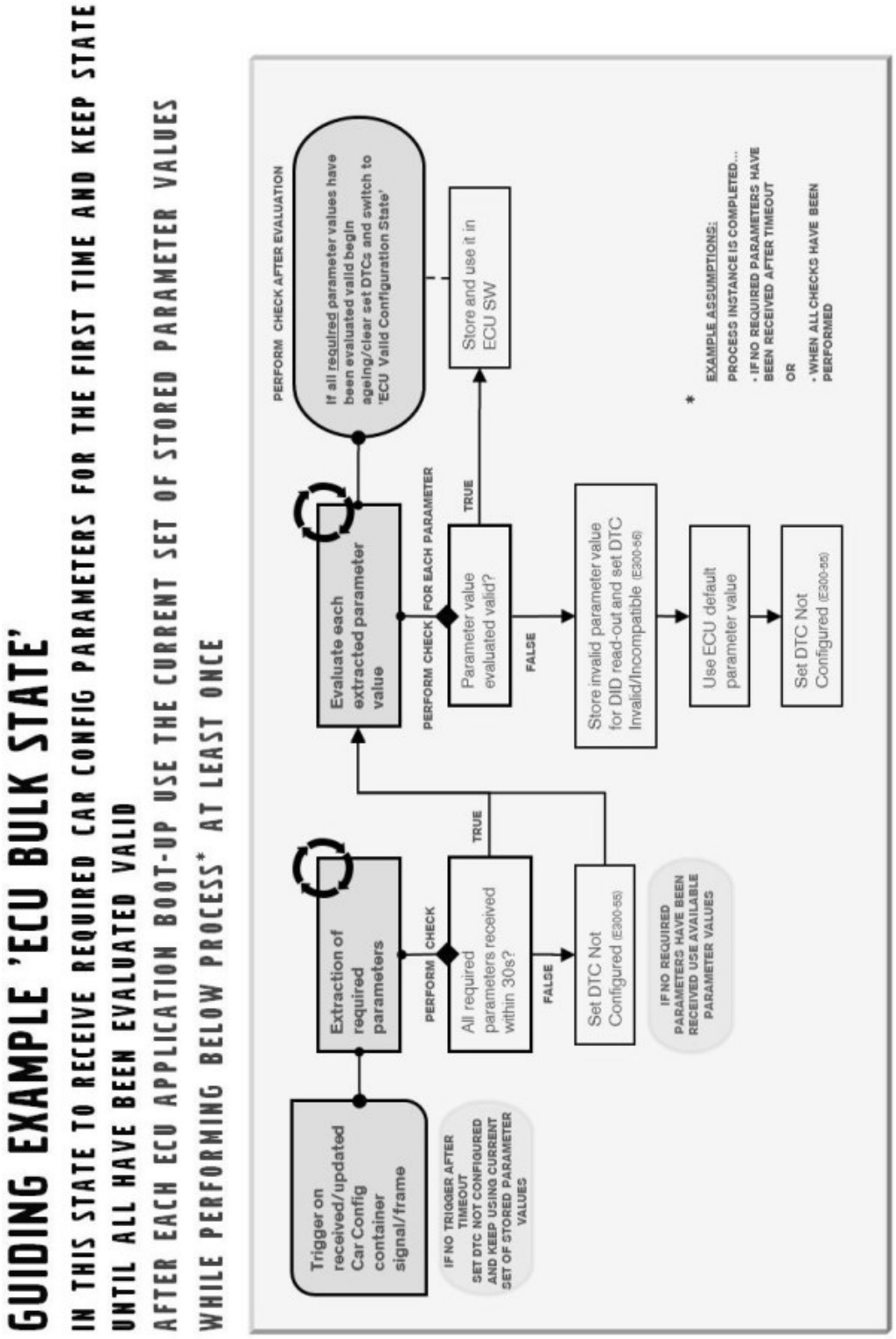
Publisher latency [ms]	Write interval [ms]	Subscriber latency [ms]	Max read interval	Max age	Update bit	Constant
-	125	-	40ms	-		-

Description:	Signal used for the distribution of Vehicle Configuration Parameters (VCP) #505 - #999. At least two consecutive cycles of blockID:s 0x01 - 0x48 (dec 1 - 72) are distributed within 30 seconds from transition of UsageMode to 'Active' or 'Driving'. CC Subscribers shall be able to handle a reduction of the write interval.
Encoding type:	View table *VCP signal mapping* in chapter *Record of tables*.

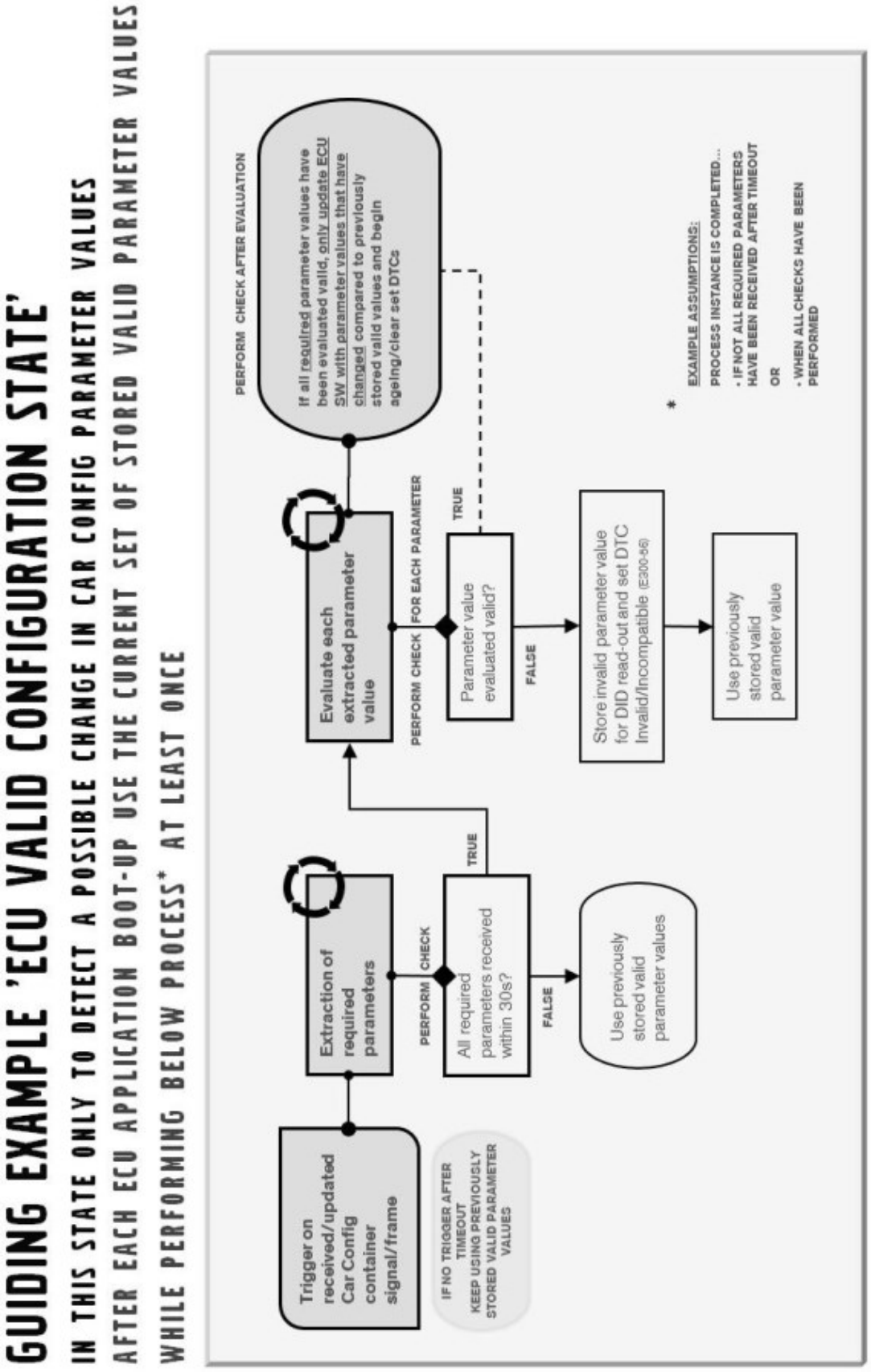
4.1.1.1.2 Appendix

4.1.1.1.2.1 Guideline subscriber states

4.1.1.1.2.1.1 Guideline ECU Bulk State



4.1.1.1.2.1.2 Guideline ECU Valid Config State



4.1.1.1.2.2 Table VCP signal mapping

Tables 'VCP signal mapping' below gives a CCP number placement if broadcasted within a packet message of 64-bit data (8 bytes per VCP signal frame). When transmitted, a CCP# byte placement will in fact contain one vehicle specific generated hexadecimal value for that CCP#.

Table: VCP signal mapping - VehCfgPrm

Signal name: VehCfgPrm							
Byte_Pos_1 (Bits 63-56)	**Byte_Pos_2 (Bits 55-48)**	**Byte_Pos_3 (Bits 47-40)**	**Byte_Pos_4 (Bits 39-32)**	**Byte_Pos_5 (Bits 31-24)**	**Byte_Pos_6 (Bits 23-16)**	**Byte_Pos_7 (Bits 15-8)**	**Byte_Pos_8 (Bits 7-0)**
BlockID_#1 (MSB, Hex)	**<Car_Configuration_parameter_#>**						
01 $_H$	<1>	<2>	<3>	<4>	<5>	<6>	<7>
02 $_H$	<8>	<9>	<10>	<11>	<12>	<13>	<14>
03 $_H$	<15>	<16>	<17>	<18>	<19>	<20>	<21>
04 $_H$	<22>	<23>	<24>	<25>	<26>	<27>	<28>
05 $_H$	<29>	<30>	<31>	<32>	<33>	<34>	<35>
06 $_H$	<36>	<37>	<38>	<39>	<40>	<41>	<42>
07 $_H$	<43>	<44>	<45>	<46>	<47>	<48>	<49>
08 $_H$	<50>	<51>	<52>	<53>	<54>	<55>	<56>
09 $_H$	<57>	<58>	<59>	<60>	<61>	<62>	<63>
0A $_H$	<64>	<65>	<66>	<67>	<68>	<69>	<70>
0B $_H$	<71>	<72>	<73>	<74>	<75>	<76>	<77>
0C $_H$	<78>	<79>	<80>	<81>	<82>	<83>	<84>
0D $_H$	<85>	<86>	<87>	<88>	<89>	<90>	<91>
0E $_H$	<92>	<93>	<94>	<95>	<96>	<97>	<98>
0F $_H$	<99>	<100>	<101>	<102>	<103>	<104>	<105>
10 $_H$	<106>	<107>	<108>	<109>	<110>	<111>	<112>
↓	...	...	...	...	...	...	...
48 $_H$	<498>	<499>	<500>	<501>	<502>	<503>	<504>

***E.g. if CCP 10 has been assigned hex value 05, after a VCD/CCL* à** VCP conversion operation, subscribing ECUs can obtain the parameter 10 assigned value 0x05 (hex), when a 64-bit frame arrives with 1 st byte set to 02 $_h$, at byte position 4.

The BlockID number and the actual byte position (parameter byte offset within frame) that a specific CCP number shall attain is calculated using the following formulas:

BlockID for CCPs 1-504:

$$BlockID\#_{10} = int\left(\frac{CCP\# + 6}{7}\right)$$

BytePos# = CCP# - (BlockID# $_{10}$ x 7 – 6) + 2

Table: VCP signal mapping - VehCfgPrmExt

Signal name: VehCfgPrmExt							
Byte_Pos_1 (Bits 63-56)	**Byte_Pos_2 (Bits 55-48)**	**Byte_Pos_3 (Bits 47-40)**	**Byte_Pos_4 (Bits 39-32)**	**Byte_Pos_5 (Bits 31-24)**	**Byte_Pos_6 (Bits 23-16)**	**Byte_Pos_7 (Bits 15-8)**	**Byte_Pos_8 (Bits 7-0)**
BlockID_#1 (MSB, Hex)	**<Car_Configuration_parameter_#>**						
01 $_{H}$	<505>	<506>	<507>	<508>	<509>	<510>	<511>
02 $_{H}$	<512>	<513>	<514>	<515>	<516>	<517>	<518>
03 $_{H}$	<519>	<520>	<521>	<522>	<523>	<524>	<525>
04 $_{H}$	<526>	<527>	<528>	<529>	<530>	<531>	<532>
05 $_{H}$	<533>	<534>	<535>	<536>	<537>	<538>	<539>
06 $_{H}$	<540>	<541>	<542>	<543>	<544>	<545>	<546>
07 $_{H}$	<547>	<548>	<549>	<550>	<551>	<552>	<553>
↓	...	...	...	...	...	...	...
47 $_{H}$	<995>	<996>	<997>	<998>	<999>	NA	NA
48 $_{H}$	NA	NA	NA	NA	NA	NA	NA

The BlockID number and the actual byte position (parameter byte offset within frame) that a specific CCP number shall attain is calculated using the following formulas:

BlockID for CCPs 505-999:

$$BlockID\#_{10} = \text{int}\frac{(CCP\# - 504 + 6)}{7}$$

BytePos# = CCP# - 504 - (BlockID# $_{10}$ x 7 – 6) + 2

4.1.1.1.2.3 Table Car Configuration DTC DID record for CC Subscriber ECUs

Car Configuration DTC/DID record	**UDS Data DTC/DID reference**
Car Configuration – Not Configured	E300-55 (U2300-55)
Car Configuration – Invalid/Incompatible Configuration	E300-56 (U2300-56)
Car Configuration - Faulty Parameters Received	E103

4.1.1.1.3 Definitions and Abbreviations

Definition	Description
0x	Indicating a hexadecimal value following the prefix '0x', e.g. 0x0A, 0x1F, 0x24. Same as $0A_{h}=10_{10}$, $1F_{h}=31_{10}$, $24_{h}=36_{10}$.
Architecture	The fundamental organisation of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution. (IEEE 1471-2000)
CC Subscriber ECU	Car Configuration Subscriber: A Domain Master ECU or an ECU part of a Domain Master sub-bus network, utilizing Car Configuration parameters for SW configuration.
System	A collection of components organised to accomplish a specific function or set of functions (IEEE Std 1471).
Component	An element that encapsulates behaviour. Could be either physical or logical.
System requirement	Input requirements to the system development process are referred to as System Requirements. These are requirements on a system as a whole and not divided onto components.
System design	A result from the part of the development process called system design. The main purpose of this process is to decide and describe how the system should be implemented in order to fulfil the system requirements (input requirements).
Function Area	A Function Area (FA) is an area of functional responsibility for a subset of the entire functionality of a car.

Abbreviation	Stands for	Description
CC	Car Configuration	Generally used as short for Central Car Configuration.
CCC	Central Car Configuration	A base technology for vehicle configuration.
CCdB	Car Configuration Database	A tool for handling all Central Car Configuration data and for generating all related logic files (CCL).
CCL	Car Configuration Logic	VBF-file containing all CCP logical conditions for generating the VCP data.
CCP	Car Configuration Parameter	Car Configuration parameters are considered global parameters broadcasted within the vehicle network.
CC Subscriber ECU	Car Configuration Subscriber	Car Configuration Subscriber: A Domain Master ECU or an ECU part of a Domain Master sub-bus network, utilizing Car Configuration parameters for SW configuration.
DID	Data ID	View *Ref[CCF_2]*.
DTC	Diagnostic Trouble Code	View *Ref[CCF_2]*.
ECC	Error Code Correction	Mechanism to maintain data integrity.
ECU	Electronic Control Unit	
EoL	End-of-Line	Final manufacturing stage for customer cars, all software is downloaded via factory tester/VCATS etc.
FAD	Function Area Description	The document describing the functional behaviour of a function area. It contains the functional input

		requirements to the system development.
FA	Function area	
MSC	Message Sequence Chart	A graphical representation exemplifying the information flows between objects in a function area as a result of the input from an actor.
NVM	Non-Volatile Memory	E.g. FLASH, EEPROM.
PSS	Product System Structure	
P/T	Powertrain	Vehicle propulsion unit; in this context consisting of an internal combustion engine, combined with clutch, transmission and/or electrical motor.
SRD	System Requirements and Description	
SW	Software	
SWDL	Software Download	
TBD	To Be Defined	
UML	Unified Modelling Language	A language for object oriented system analysis and design.
VBF	Versatile Binary Format	SWDL file format.
VCD	Vehicle Configuration Data	A factory system generated VBF file consisting of vehicle specific Information Data and P-Spec sections respectively.
VCP	Vehicle Configuration Parameters	The VCP is the actual generated parameter values prepared for broadcast within the ECU network architecture.
VM	Volatile Memory	E.g. RAM.

4.1.1.1.4 Requisite documents

Reference Num	Title	Logical Component name
[CCF_1]	SRD Car Configuration Management System	Car Configuration Master
[CCF_2]	Unified Diagnostic Services (UDS) Data	UDS Data
[CCF_3]	Overview of Vehicle Configurations & Customer Settings	Overview of Vehicle Configurations and Customer Settings

4.1.1.1.5 Revision History

Rev. (issue)	Vers.	Date	Description	Approved by	Responsible
001	1	2017.02.28	First issue (System Weaver version)	Magnus Nordberg, PSS333	Thea Välemark, PSS333
002	2	2017.10.03	Signal VehCfgPrmExt added	Magnus Nordberg, PSS333	Thea Välemark, PSS333

4.1.2 Overview of Vehicle Configurations and Customer Settings

4.1.2.1 Overview of Vehicle Configurations and Customer Settings

4.1.2.1.1 Vehicle Configuration

An efficient method for SW Configuration is achieved using parameters (constant reference values). There are two major types of configuration methods: Global and Local. Configuration parameters, often compiled within a SW file that may contain only one of those two distinct parameter types. Local parameters may either reside in a separate file or can be embedded in ECU's software.

Properties for SW Configuration files:

- Handled according to approved SW handling processes.
- Traceable in individual car.
- Downloaded at assembly plants or service shops.
- Stored in SW databases.

4.1.2.1.1.1 Configuration Using Global Parameters

Global parameters denote major systems, functions and features implemented in vehicle via ECU applications. Global parameters notion is often termed Central Car Configuration (or CarConfig in short), are used to configure ECU functions centrally over a distributed network during run-time.

The CarConfig file is generated when the vehicle is built or updated; therefore it is unique per vehicle. Parameter values are stored in the Central Electronics Module or equivalent, from where they are transmitted on all networks to the ECUs requiring them.
It is strongly recommended to choose each parameter as specific and function oriented as possible. This facilitates to reconfigure multiple ECUs just by generating & downloading new CarConfig file.

In general following guidelines should be fulfilled when applying global parameters changes:

- Each parameter value shall specifically point out the behaviour of the function using that parameter. A parameter with an indirect effect on the functional behaviour (e.g. market code or date) shall not be used, for two reasons:
  - If the relationship between the indirect parameter and the functional requirements is changed (e.g. the legislation on the market is changed) the function will no longer behave correctly.
  - If a new code for the indirect parameter is introduced (e.g. new market) the new parameter code cannot be used to configure the function without re-writing the software in the receiving ECU.
- It shall not be possible to add an accessory function by only altering a parameter value. Adding an accessory function (e.g. cruise control) must involve the download of additional software (i.e. a new software part number).
- A global parameter should not be unique to a specific ECU; i.e. other ECUs would preferably be able to use it. Thus it gets usage potential by multiple modules/features as desired by CarConfig notion; otherwise this should be solved locally.
- It's not allowed to re-define (name & content of) existing parameter values, if they have already been used in customer cars.

- A global Parameter can not be dynamic relying on a network signal from another module as to calculate the parameter value to be published over the network.
- The parameter would rather be generic and not for a specific platform usage solely, to facilitate carry over/back of nodes & functions.

Following are specifications of two Global parameters currently in use:

Parameter No.	Module Configurable Item	Value Comments	Variant Code
245	PEDESTRIAN PROTECTION	Without Pedestrian Protection	EZ01
		With Pedestrian Protection	EZ02
246	STOP/START LOGIC	Without Stop/Start	CP01
		SIN	CP02
		SIN/SIG	CP03
		SID	CP04
		SINS	CP05

4.1.2.1.1.2 Configuration Using Local Parameters

Local parameters consist of a set of values to configure ECU application, often packed in a binary file. They are kreate as a part of the ECU Software, stored in the SW database and are traceable via SW article numbers. They are downloaded or updated at assembly plants or at service shops. This type is used when the global parameters usage is not possible or appropriate to use; they are used in single ECU, not accessible to other ECUs. Local parameters can either exist in a separate file or be embedded in ECU software.

Example:
Calibration values for diesel injectors.
Calibration values for SRS.

4.1.2.1.1.3 Special cases of the Local parameters

Local calibration data are parameters temporarily applied in ECUs during development phases for tests and calibration/optimization purposes. These parameters are generated, applied and stored locally in concerned developing departments.

Example:
Calibration and optimization data for ECM software.

4.1.2.1.2 Customer Settings

Customer settings is a collective name referred to all those available options to select or adjust some pre-defined set of functions or features e.g. to enhance users' comfort or vehicle usability and performance (adaption to driving conditions etc). Settings are activated or modified through customer choice using available HMI (buttons, Center Stack, displays). The customer has the option to save own settings in available profiles, to re-load them for activation at next driving session.

Customer settings have following properties:

- Not traceable to individual cars by OEM databases

- Settings can be done by customer via HMI
- Default settings can be done via ECU applications
- Profiles are backed-up to be restored after a car service session

4.1.2.1.2.1 Types of Customer Settings

All customers setting will in next platform are stored and re-stored using Profile notion; however, Customer settings can roughly be divided in two types:

- Personal settings:

The customer can select, modify or adjust some pre-defined set of settings to improve vehicle comfort, usability etc.

Examples include adjustments for:
Side mirror, Seat (position, heat), Steering wheel (height, force level), Climate, and Infotainment system features.

- Driving settings:

These settings contain the functions to enhance vehicle usability and performance, e.g. adaption to changing driving conditions etc.

Examples are activation of:
ECO Mode, Extended EV mode, Sport Mode, PAS settings, DSTC settings, LDW settings.

4.1.2.1.3 Summary of Configuration Types

The table below summarizes major vehicle configuration types, theirs sources, download methods and storage area.

Configuration Type	Data Source	Realization	Storage area
Global Parameter	Team Center/Variant Structure	SWDL	gl-PIE
Local Parameter	Team Center/Variant Structure	SWDL or embedded in SW applications	gl-PIE
Local Calibration Data	Locally at concerned departments	SWDL or embedded in SW applications	Locally at concerned departments
Customer Settings	ECU software or similar files	Initially as default values in ECU application; later via customers.	In-vehicle back-up so that customer may restore them after service.

4.1.2.1.4 Introduction

4.1.2.1.4.1 Document Purpose

This document aims to guide users through the subject and terminology for different types of configuration parameters and customer settings.

4.1.2.1.5 Definitions and Abbreviations

Abbr.	Stands for	Description
CCdB	Car Configuration Database	Tool & Data Base for generating CarConfig data

DSTC	Dynamic Stabilization & Traction System	
EV	Electric Vehicle	
ECU	Electronic Control Unit	Microcontroller-based module
LDW	Lane Departure Warning	An active safety feature
PAS	Parking Assist System	An audio/video based assisting feature
P-spec	Product specification	Vehicle building data, unique for each car
SW	Soft Ware	

4.1.2.1.6 Background

There are different configuration methods for vehicles systems and functions. Following chapters make an effort to explain and establish common terminology within organisation.

Although to make any change in car functions often involves ECU application configured with new input values, here we reserve the term CONFIGURATION for major & permanent (i.e. by SWDL) configuration of ECU software, and the term SETTINGS for activating and adjustments in current implemented functions.

The configuration types and settings explained here are:

- Vehicle SW Configuration methods:
  - Central Car Configuration
  - Local Configuration
  - Temporary calibration/optimization data

- Customer Settings categories:
  - Personal Settings
  - Driving Settings

The purpose of configuration and settings could be to:

- Control, modify, or optimize the behaviour of system or device functions.
- Reduce the number of software or hardware variants.
- Input static calibration & optimization values for system or device functions.
- Share common data or information.
- Store shared data in common area.

The following configuration methods are in use:

- Software download, Central Car Configuration
- Software download, Local ECU Configuration
- Incorporating local parameters in SW applications
- Selecting the desired settings via HMI, by customer
- Having default setting values via ECU applications

4.2 Diag & ECU Platform - proj reqs

4.2.1 SRD Diagnostics and ECU Platform

4.2.1.1 Diagnostics and ECU Platform

4.2.1.1.1 Introduction

Statement of Work (SOW)
Diagnostics and ECU Platform Statement Of Work is the name of the SOW (Statement Of Work) from Diagnostics & ECU Platform and it is essential that it is part of the ESOW for each node that intend to use requirements from our area.

This SOW defines the additional required engineering tasks, responsibilities and deliverables for the design, engineering development, quality assurance and test of an Electronic Control Unit (ECU) / Subsystem that shall conform to the Vehicle Program requirements on Diagnostics and ECU Platform.

SW Platform
Issues with regards to the AUTOSAR platform and its development are described in the SOWs (mentioned above) with the Tier 1.

The main purpose of SWDL is:

- Reduced time for development
  - Avoided lead time for hardware changes
- Reduced number of hardware partnumbers
  - Variants are configured with software
  - Less delivery units
  - Less units in stock saves space (factory, aftersales and suppliers)
- The vehicle is after production possible to change
  - Updating of software “quality issues”
  - New functionality “replaces hardware changes”
  - Upgrade “increased/changed functionality”

A bootloader consists of a primary and secondary bootloader (xBL) that supports the software download protocol. The bootloader always gain program control after a reset, and passes the control over to the application when it has determined that no download is requested.

The software download concept is based on a primary bootloader that is fixed in the ECU and always executed at power on before the application is started and a secondary bootloader that is downloaded before downloading the applications and configuration datafiles.

Software download functionality is not a part of the AUTOSAR based ECU architecture. To support software download a bootloader shall be used (a stand alone application). Enter and exit of the bootloader is handled by a common API, known by the AUTOSAR based ECU architecture and the bootloader.

CarConfig

Car Configuration is used to organize car specific information and to minimize the software variants by sending out information about the car over the multiplexed network. This makes it possible to use the same software to perform functions that differ between the car models.

4.2.1.1.1.1 Definitions & Abbreviations

Abbr.	Stands for	Description
AUTOSAR	AUTomotive Open System ARchitecture	
CAN	Controller Area Network	
N/A	Not Applicable	
IP	Internet Protocol	The IP is the principal communications protocol used for relaying datagrams (packets) across an internetwork using the Internet Protocol Suite.
SW	SoftWare	
ECU	Electronic Control Unit	Electronic control unit with a computer/microcontroller.
TCP	Transmission Control Protocol	TCP is one of the core protocols of the Internet Protocol Suite. TCP is one of the two original components of the suite, complementing the IP, and therefore the entire suite is commonly referred to as TCP/IP.
SWC	SoftWare Component	
LIN	Local Interconnect Network	Low speed single wire network
MOST	Media Oriented Systems Transport	Optical Multimedia Network
RTE	RunTime Environment	
SRD	System Requirements Description	This document is an SRD.
SWE	Swedish	Used to give a Swedish equivalence.
Edge node		An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
GMRDB	Global Master Reference DataBase	Database used for keeping track of all diagnostic entities in the vehicle.
S-feed	Switched power supply	Switched power supply– S-feed –is typically controlled by an external switch, usually a relay, known as "power-switched".
C-feed	Continuos power supply	Continuous power supply– C-feed –is typically (but not necessary) directly connected to battery, known as "30-feed" or "battery-powered".
TBD	To Be Defined	

4.2.1.1.1.2 Requisite documents

Ref.	Source	Description
[1]	Internal	Autosar Network Management - additional requirements

[2]	Internal	Components for Physical Interfaces
[3]	Internal	LIN Datalink Layer
[4]	Internal	Diagnostic and Bootloader Gateway Specification
	Internal	UDS Services
	Internal	UDSonCAN
	Internal	FlexRay Data Link Layer
	External	CAN 2.0B Standard
	External	MOST Specification 3VE2
	External	RFC 1918

4.2.1.1.1.3 Requirement Overview

This section is intended to give an overview of all requirements from "Diagnostics & ECU Platform" except for SRD Car Configuration Management and SRD Diagnostics Management.

This document also contain project unique requirements from the "Diagnostics & ECU Platform" area.

Diagnostics & ECU Platform requirements is divided into eight conceptual areas:

- Diagnostics & ECU Platform
- Communication
- Software Platform
- Hardware Platform
- Car Configuration
- Diagnostics
- Software Download
- Data Security

Each of these area is divided into subgroups containing a number of base technology specifications.
The list below shall be regarded as an example:

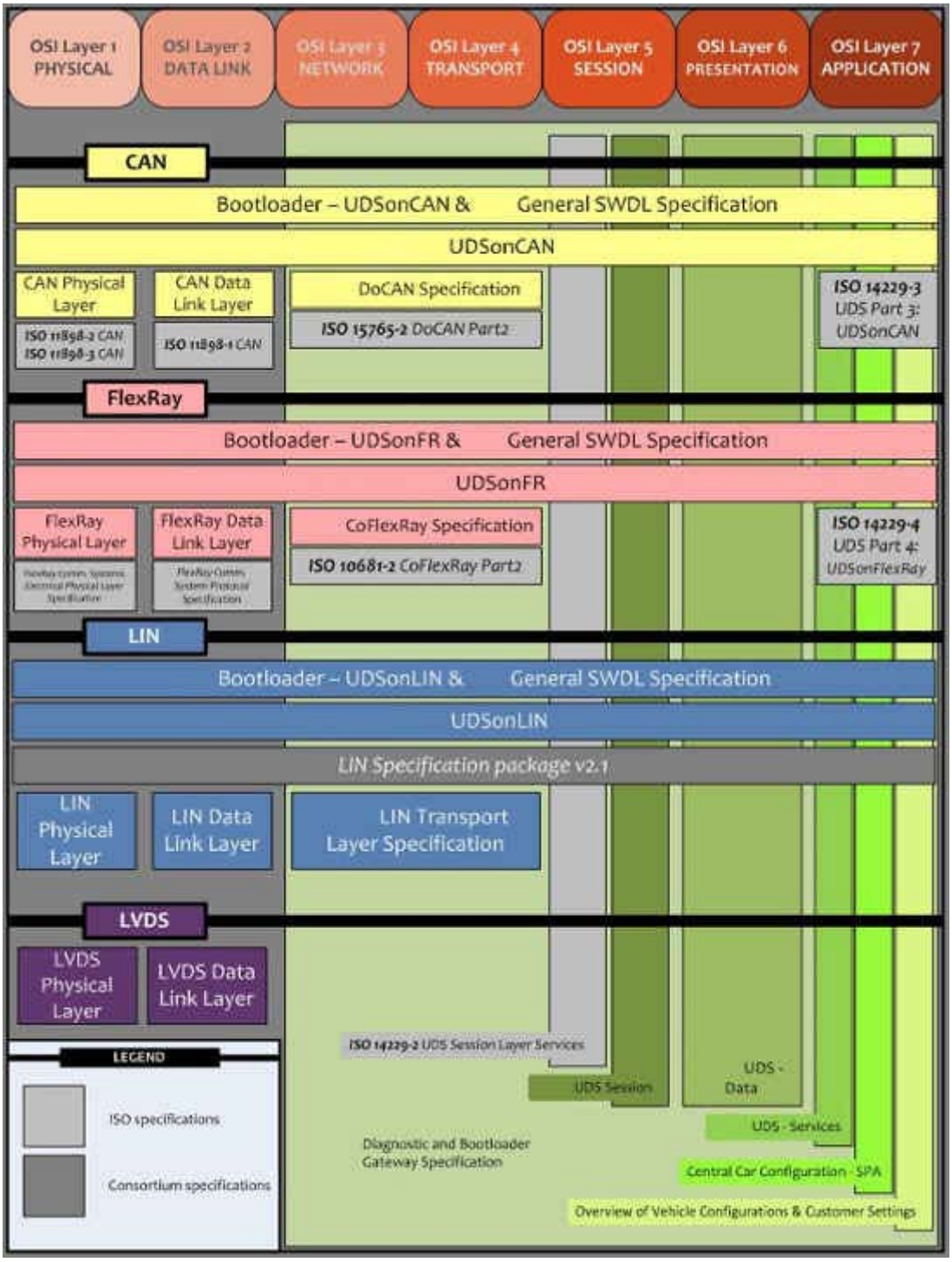
Table: Example of base technologies

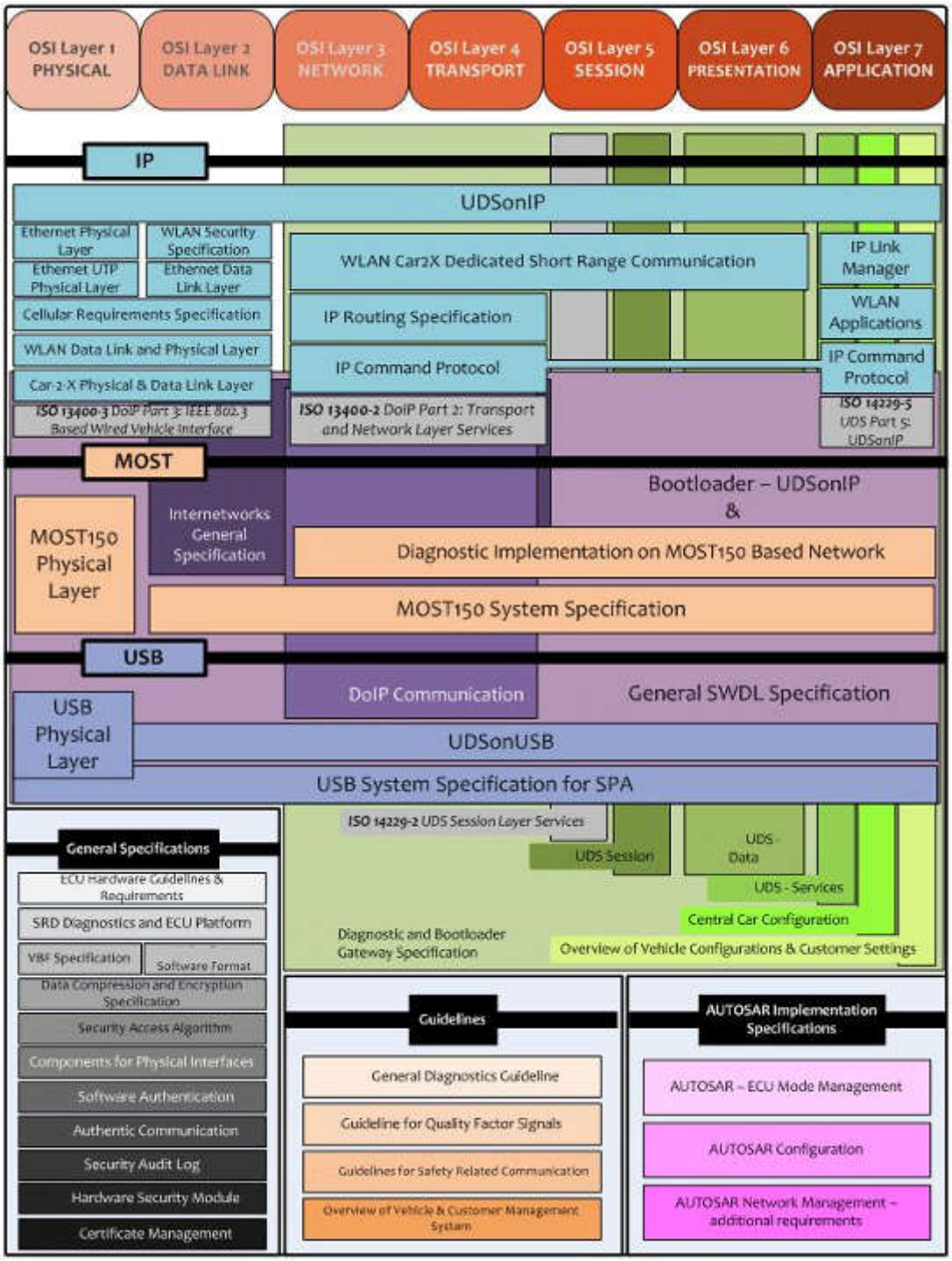
Diag & ECU Platform
SRD Diagnostics and ECU Platform
Car Configuration
Central Car Configuration
Overview of Vehicle Configurations & Customer Settings
Communication
CAN Data Link Layer
Ethernet Data Link Layer
FlexRay Data Link Layer
LIN Data Link Layer
LVDS Data Link Layer
CoFlexRay
DoCAN

DoIP Communication
Guidelines for Safety Related Communication
Internetworks General Specification
IP Command Protocol
IP Link Manager
IP Routing Specification
Guideline for quality factor signals
LIN Transport Layer Specification
LVDS Control Transport Layer Specification
MOST150 System Specification
USB System Specification
WLAN Application
WLAN Security Specification
Diagnostics
General Diagnostic Guideline
Diagnostic and Bootloader Gateway Specification
Unified Diagnostic Services on LIN, UDSonLIN
UDSonIP
UDSonMOST150
UDSonFR
UDSonCAN
UDSonUSB
UDSonLVDS
UDS Data
UDS Service
UDS Session
Software Download
Data Compression and Encryption Specification
Versatile Binary Format Specification 2.5
Software Format
General Software Download Specification
Bootloader - UDSonCAN
Bootloader - UDSonFR
Bootloader - UDSonIP
Bootloader - UDSonLIN
Bootloader - UDSonLVDS
Software Platform
Autosar Network Management - additional requirements
Autosar BSW - Tier1 to Tier2 Requiremens
Autosar Configuration
Autosar Diagnostic Guideline
Hardware Platform
FlexRay Physical Layer
Ethernet UTP Physical Layer
Ethernet Physical Layer
USB Physical Layer
LVDS - APIX2

MOST150 Physical layer
CAN Physical Layer
WLAN Physical & DataLink Layer
Cellular requirements specification
Car-2-X - Physical & DataLink Layer
LIN Physical Layer
ECU Hardware Guidelines & Requirements
Components for physical interfaces
LTE Physical and Datalink Layer
Wireless Co-existence
Data Security
Security Access algorithm
Software Authentication
Authentic communication
Certificate (client - in keystore) /credential/trust (root cert, in truststore) management
Hardware security module
Security Audit log

The relation of the different base technology specifications to each other, to networks, to standards such as ISO and their mapping to the different OSI layers are described in the following 2 pictures:


OSI Layer 1 PHYSICAL
OSI Layer 2 DATA LINK
OSI Layer 3 NETWORK
OSI Layer 4 TRANSPORT
OSI Layer 5 SESSION
OSI Layer 6 PRESENTATION
OSI Layer 7 APPLICATION
CAN
Bootloader – UDSonCAN & General SWDL Specification
UDSonCAN
CAN Physical Layer
ISO 11898-2 CAN
ISO 11898-3 CAN
CAN Data Link Layer
ISO 11898-1 CAN
DoCAN Specification
ISO 15765-2 DoCAN Part2
ISO 14229-3 UDS Part 3: UDSonCAN
FlexRay
Bootloader – UDSonFR & General SWDL Specification
UDSonFR
FlexRay Physical Layer
FlexRay Data Link Layer
CoFlexRay Specification
ISO 10681-2 CoFlexRay Part2
ISO 14229-4 UDS Part 4: UDSonFlexRay
LIN
Bootloader – UDSonLIN & General SWDL Specification
UDSonLIN
LIN Specification package v2.1
LIN Physical Layer
LIN Data Link Layer
LIN Transport Layer Specification
LVDS
LVDS Physical Layer
LVDS Data Link Layer
LEGEND
ISO specifications
Consortium specifications
ISO 14229-2 UDS Session Layer Services
UDS Session
UDS - Data
UDS - Services
Diagnostic and Bootloader Gateway Specification
Central Car Configuration - SPA
Overview of Vehicle Configurations & Customer Settings



4.2.1.2 Hardware Platform

4.2.1.2.1 Requirements

1625 v3 **Spare Capacity**

Version history:
(3):

Purpose: To provide for functional growth.

Source:
Verification Method: Analysis
Legacy ID: 77743v2

All new ECU's shall have a spare capacity of 30% at introduction time (Job1). The requirement is valid for all memory types (Flash, RAM and NV RAM) and CPU load.

Note: This requirement is valid for all new ECU's.

1627 v3 Multiple CPU Design

Version history:
(3): Change the Verification Method to Analysis

Purpose: To clarify that requirements are applicable for the complete ECU and always considered as one entity.
Source:
Verification Method: Analysis
Legacy ID:

An ECU is always considered as one entity and is not allowed to create bottlenecks due to connection dependencies between multiple CPUs.

Note: This requirement is valid for any ECU which uses multiple CPUs, e.g. one AUTOSAR based microcontroller for vehicle communication, and one microprocessor running an operating system such as Linux.

4.2.1.3 ECU Addresses

4.2.1.3.1 Requirements

1628 v2 Domain Master Address

Version history:
(2): Change the Verification Method to Analysis

Purpose: Clarification of the addresses of the domain masters.
Source:
Verification Method: Analysis
Legacy ID: 342168v0

All Domain Masters shall use the address 0x1Y01 where the two last nibbles, 01 states that the ECU is a Domain Master.

1629 v43 ECU Addresses

Version history:
(41): Add MVEM node in Propulsion CAN.
(42):
(43):

Purpose: To deploy valid ECU addresses to each ECU.

Source: -
Verification Method: Analysis
Legacy ID: 76545v32

FLEXRAY
ECUs with Flexray interface
FlexRay ECUs shall use FlexRay Frame ID's in the static segment【 to address application communication and Frame ID's in the dynamic segment to address diagnostic and SWDL communication. The Flexray Frame ID is included in the ECU extract.

CAN
ECUs with CAN interface
All CAN ECUs shall have a CAN ID according to the table below. The CAN ID is a static address calculated from the ECU address and is used to address CAN ECUs for SWDL and Diagnostics. The CAN ID is included in the ECU extract.

LIN
LIN slave ECUs, using SWDL functionality (bootloader)
All LIN ECUs using a bootloader shall use a complete ECU address according to the table below. The ECU address is a static address that is used to address LIN ECUs for SWDL and Diagnostics. The ECU address is included in the LDF file.

LIN slave ECUs without SWDL, but using LIN diagnostic frames
A LIN slave ECU (not using a bootloader) shall use a NAD address according to the table below. Otherwise the LIN master will not be able to address diagnostic requests to the LIN slave. The NAD address is the last byte from the 2 byte ECU address and is used for diagnostics. The NAD address is included in the LDF file.

LIN slave ECUs without SWDL and without LIN diagnostic frames
If there is no need for a LIN master to address diagnostic requests to a LIN slave ECU, then a NAD address is NOT needed. In this case the LIN slave will be assigned a node address default value. The default value will be 0x7D. In case of multiple LIN Slaves on the same network the addresses will be set in decreasing order starting from 0x7D. The LIN address to be used by the LIN slave is included in the LDF file.

LIN masters
LIN masters shall use the NAD according to table ECU Addresses below in order to address diagnostic requests to LIN slaves.

MOST
ECUs with MOST150 interfaces
MOST Logical Node Address (RxTxLog) is a static address predefined as according to the table below and is used by MOST applications when using the control channel on MOST. The MOST Logical Node Address must be implemented into the MOST ECU by supplier.

IP
ECUs with IP-based interfaces
The internal MAC addresses and internal IP addresses of IP-ECUs shall be assigned as according to table ECU Addresses. The IP address must be implemented into the IP ECU by

supplier. The following IP address space has been chosen: 198.18.0.0/16 for the vehicle network. From this address space, the vehicle IP network is using one subnet according to below:

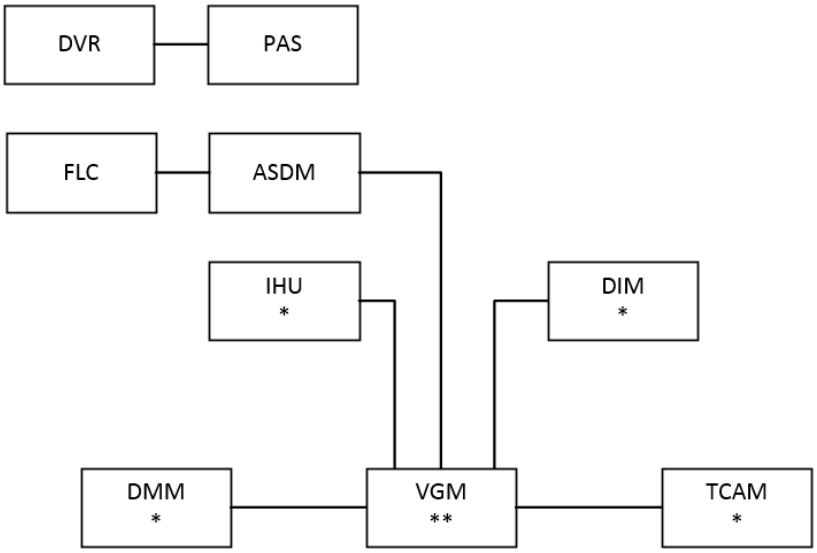


Figure:IP subnets of the GEEA 2.0 network

NOTE 1: In *Figure IP subnets of the vehicle network*, (the Ethernet capability of APIX2 link is not used for diagnostics. Instead, FlexRay is used for diagnostics.

NOTE 2: In *Figure IP subnets of the vehicle network* and in the *table "ECU Addresses"*, some IP addresses are marked with a * or a **.

* == Internal ECU-IP address used for Diagnostic (ISO13400) TCP clients (Diagnostic Server) to connect to the Edge Node (VGM).

** == Internal ECU-IP address used by Edge Node (VGM) acting as TCP Server (Diagnostic Client) offering Diagnostic Service

Overview of IP-address space :

The vehicle IP network shall use one subnet according to table below.

Network Friendly Name:	**VEHICLE IP NETWORK**
Subnet ID:	198.18.0.0 / 16
Usable IP addresses:	198.18.0.1 – 198.18.255.254
Netmask:	255.255.0.0
Broadcast address:	198.18.255.255
Default GW	VGM – Ethernet interface according to,

address:	***Table:*** *ECU addresses – Showing IP-based ECU's*

IP address space allocation depending on physical network

The IP address range of 198.18.0.0 / 16 has been divided into different address ranges to be able to identify which physical communication carrier is used for a specific ECU. The tables below are used in conjunction with REQPROD 342169.

Example: ECU's which are connected to Ethernet will have an IP address range of: 198.18.32.1-198.18.47.255.

Network Friendly Name:	**MOST150**
Network ID:	198.18. **0**.0
IP addresses:	198.18.0.1 – 198.18.15.255

Network Friendly Name:	**APIX/LVDS**
Network ID:	198.18. **16**.0
IP addresses:	198.18.16.1 – 198.18.31.255

Network Friendly Name:	**Ethernet**
Network ID:	198.18. **32**.0
IP addresses:	198.18.32.1 – 198.18.47.255

Network Friendly Name:	**USB**
Network ID:	198.18. **48**.0
IP addresses:	198.18.48.1 – 198.18.63.255

Relationship between ECU addresses and IP addresses
The ECU address consist of 16 bits and are divided into 4 nibbles as follows: [Address range]. [Domain id].[Network id].[Ecu id]

NOTE: The addressing format for diagnostic request and response, source address (SA) and target address (TA), using Domain ID, Network ID and ECU ID is defined in [*UDS Services*].

The IP address v4 consist of 32 bits and are divided into 4 bytes. It is required for diagnostic purposes, that it shall be possible to calculate the ECU address from the IP address of a specific ECU. The IP addresses have been divided as follows:

198.18.{ssssdddd}.{nnnneeee}.

ssss = Consists of 4 bits, defines the start of the Network ID.
dddd = Consists of 4 bits, defines the Domain id
nnnn = Consists of 4 bits, defines the Network id
eeee = Consists of 4 bits, defines the ECU id

The ECU addresses are always represented in hexadecimal form and the IP addresses are written in decimal form. This means when calculating the ECU address from the IP address there is a need for conversion from hexadecimal to decimal.

Example:
ECU Address 0x1212, would mean IP address 198.18.2.18.

198.18.[S] [D].[N][E]

[S]=0x0 (Network Friendly Name = “MOST150”, Network ID: 198.18. **0**.0)
[D] = 0x2
[N] = 0x1
[E] = 0x2

Table: IP Address calculation

IP address calculation				
	198	**18**	**{ssssdddd}**	**{nnnneeee}**
Binary	1100 0110	0001 0010	0000 0010	0001 0010
Hex	0xC6	0x12	0x02	0x12
Decimal	198	18	2	18

Table: ECU addresses

ECU Name	ECU Alias	Diagnostic Network = Source network for SWDL	ECU Address	CAN ID	NAD (LIN)	Internal MAC Addresses	Internal IP Addresses
Air Condition Control Module	ACCM	ECM_LIN1	N/A	N/A	0x7D	N/A	N/A
Air Fragrance Unit	AFU	CCM_LIN2	N/A	N/A	0x7C	N/A	N/A
Active Grill Shutter Module	AGM	ECM_LIN4	N/A	N/A	0x7A	N/A	N/A
Ambience Lights Module	ALM1	CEM_LIN5	N/A	N/A	0x7D	N/A	N/A
Ambience Lights Module	ALM2	CEM_LIN8	N/A	N/A	0x7D	N/A	N/A
Air Particulate Matter Sensor	APMS	CCM_LIN2	N/A	N/A	0x7B	N/A	N/A
Active Safety Domain Master	ASDM	Backbone FlexRay	0x1401	0x701	N/A	Ethernet: 02:00:00:00:14:01	Ethernet: 198.18.36.1
Active Spoiler Shutter Module	ASSM	ECM_LIN4	N/A	N/A	0x7C	N/A	N/A
Adjustable Steering Wheel Module	ASWM	CEM_LIN4	N/A	N/A	0x7D	N/A	N/A
Audio Module	AUD	Private Info CAN	0x1212	0x712	N/A	N/A	Ethernet: 198.18.34.1 (Note 1)
Brake Booster Module	BBM	Backbone FlexRay	0x1602	0x702	N/A	N/A	N/A
Battery Backed-up Sounder	BBS	CEM_LIN2	N/A	N/A	0x7C	N/A	N/A
Battery Circuit Four way Valve	BCFV	ECM_LIN1	N/A	N/A	0x7B	N/A	N/A
Battery Cooling Pump Front module	BCPF	ECM_LIN3	N/A	N/A	0x7D	N/A	N/A
Battery Circuit Three way Valve	BCTV	ECM_LIN1	N/A	N/A	0x7A	N/A	N/A

Battery Energy Control Module	BECM	Propulsion CAN	0x1635	0x735	N/A	N/A	N/A
Battery Electric Expansion Valve	BEXV	CCM_LIN2	N/A	N/A	0x77	N/A	N/A
Battery Monitoring Sensor	BMS	CEM_LIN6	0x1B61	N/A	0x61	N/A	N/A
Bluetooth NFC CommunicationModule	BNCM(NFC)	TCAM_LIN1	N/A	N/A	0x7C	N/A	N/A
Bluetooth Short Range Module	BSRM	TCAM_LIN1	N/A	N/A	0x7D	N/A	N/A
Climate Control Module	CCM	Body CAN	0x1A11	0x711	N/A	N/A	N/A
Center Console Switch Module	CCSM	IHU_LIN2	N/A	N/A	0x7D	N/A	N/A
Charger and DCDC Module	CDD	Propulsion CAN	0x1650	0x750	N/A	N/A	N/A
Carbon Dioxide Sensor	CDS	CCM_LIN2	N/A	N/A	0x7D	N/A	N/A
Central Electronic Module	CEM	Backbone FlexRay	0x1A01	0x701	N/A	N/A	N/A
Condenser Electric Expansion Valve	CEXV	CCM_LIN2	N/A	N/A	0x73	N/A	N/A
Centre Stack Display	CSD	GMSL	0x1241	N/A	0x41	N/A	Ethernet: 198.18.34.1 (Note 1)
Cloud Service Provider	CSP	N/A	N/A	N/A	N/A	N/A	N/A
Drivetrain Coolant Pump	DCP	ECM_LIN3	N/A	N/A	0x7B	N/A	N/A
Driver Door Module	DDM	Body CAN	0x1A12	0x712	N/A	N/A	N/A
Driver Door Switches	DDS	DDM_LIN1	N/A	N/A	0x7D	N/A	N/A
Differential Electronic Module	DEM	Propulsion CAN	0x1638	0x738	N/A	N/A	N/A
Display Head Unit	DHU	Ethernet	0x1201	0x701	N/A	Ethernet: 02:00:00:02:12:01	Ethernet: 198.18.34.1* (Note 4)
Display Head Unit	DHU	Ethernet	0x120F	N/A	N/A	Ethernet: 02:00:00:02:12:15	Ethernet: 198.18.34.15 (Note 4)
Driver Information Module	DIM	Backbone FlexRay	0x1801	N/A	N/A	GPU SWDL via Ethernet (Note 2)	GPU SWDL via Ethernet (Note 2)
Driver Information Module	DIM	DIM Ethernet	0x1013	N/A	N/A	Ethernet: 02:00:00:00:10:13	Ethernet: 198.18.32.19
Driver Information Screen	DIS	DIM_LIN1	0x1253	N/A	DIM_LIN1: 0x53	N/A	Ethernet: 198.18.34.1 (Note 1)
Driver Information Screen	DIS	DIM_LIN1	0x1853	N/A	DIM_LIN1: 0x53	N/A	N/A
Driver Monitor Module	DMM	DMM Ethernet	0x1012	0x712	N/A	Ethernet: 02:00:00:00:10:12	Ethernet: 198.18.32.18 *
Drive Mode Switch Module	DMSM	IHU_LIN2	N/A	N/A	0x7C	N/A	N/A
Passenger Door Module	PDM	Body CAN	0x1A13	0x713	N/A	N/A	N/A
Power Operated Tailgate	POT	Body CAN	0x1A15	0x715	N/A	N/A	N/A
Dedicated Short Range Communication	DSRC	Connectivity CAN HS	0x1021	0x721	N/A	N/A	N/A
Digital Video Recorder	DVR	Passive Safety CAN	0x1514	0x714	N/A	Ethernet: 02:00:00:00:15:14	198.18.37.20
Engine Control Module	ECM	Propulsion CAN	0x1630	0x730	0x30	N/A	N/A

Engine Coolant Pump Module	ECPM	ECM_LIN3	N/A	N/A	0x06	N/A	N/A
Electrical motor Cooling Regulator Valve module	ECRV	ECM_LIN1	N/A	N/A	0x10	N/A	N/A
Electric Drivetrain Coolant Pump	EDCP	ECM_LIN3	N/A	N/A	0x5A	N/A	N/A
Evaporator Electric Expansion Valve	EEXV	CCM_LIN2	N/A	N/A	0x78	N/A	N/A
Electronic Gear Selector Module	EGSM	Propulsion CAN	0x1633	0x733	0x33	N/A	N/A
Exterior Sound Module	ESM	Propulsion CAN	0x163A	0x73A	N/A	N/A	N/A
Electric Vehicle Communication Controller	EVCC	Private CAN	N/A	N/A	N/A	N/A	N/A
Fast Charger Socket Indicator	FCSI	CEM_LIN7	N/A	N/A	0x7D	N/A	N/A
Forward Loooking Camera	FLC	FlexRay	0x1411	0x711	N/A	N/A	N/A
Forward Looking Radar	FLR	CAN FD 2	0x1444	0x744	0x7B	N/A	N/A
Foot Movement Detection Module	FMDM	CEM_LIN6	N/A	N/A	0x62	N/A	N/A
Front Side Radar Left	FSRL	CAN FD 1	0x1431	0x731	0x78	N/A	N/A
Front Side Radar Right	FSRR	CAN FD 1	0x1430	0x730	N/A	N/A	N/A
HVAC Blower Module Front	HBMF	CCM_LIN1	N/A	N/A	0x7D	N/A	N/A
Headlamp Control Module Left	HCML	Body Exposed CAN	0x1BB3	0x7B3	N/A	N/A	N/A
Headlamp Control Module Right	HCMR	Body Exposed CAN	0x1BB4	0x7B4	N/A	N/A	N/A
Heater Circuit Three way Valve	HCTV	ECM_LIN1	N/A	N/A	0x11	N/A	N/A
HVAC Flap Actuators	HFA	CCM_LIN4	N/A	N/A	0x7D	N/A	N/A
Hands Off Detection	HOD	DIM_LIN1	N/A	N/A	0x7C	N/A	N/A
Head Up Display	HUD	DIM_LIN1	0x1251	N/A	DIM_LIN1: 0x51	N/A	Ethernet: 198.18.34.1 (Note 1)
Head Up Display	HUD	DIM_LIN1	0x1851	N/A	DIM_LIN1: 0x51	N/A	N/A
High Voltage Coolant Heater	HVCH	CCM_LIN2	N/A	N/A	0x7A	N/A	N/A
High Voltage Secondary Conversionmodule	HVSC	Chassis CAN 1	0x1673	0x773	N/A	N/A	N/A
Inverter ERAD Module	IEM	Propulsion CAN	0x1637	0x737	N/A	N/A	N/A
Inverter Generator Module	IGM	Propulsion CAN	0x1636	0x736	N/A	N/A	N/A
Infotainment Head Unit	IHU	Ethernet	0x1201	0x701	N/A	Ethernet: 02:00:00:02:12:01	Ethernet: 198.18.34.1* (Note3)
Infotainment Head Unit	IHU	Ethernet	0x120F	N/A	N/A	Ethernet: 02:00:00:02:12:15	Ethernet: 198.18.34.15 (Note3)
Trailer Module	TRM	Body CAN	0x1A17	0x717	N/A	N/A	N/A
Interior Movement Sensor	IMS	CEM_LIN2	N/A	N/A	0x7B	N/A	N/A
Interia Monitor Unit	IMU	Private CAN	N/A	N/A	N/A	N/A	N/A

Interior Rear view Mirror Module	IRMM	CEM_LIN1	N/A	N/A	0x13	N/A	N/A
Integrated Start Generate Module	ISGM	Propulsion CAN	0x163E	0x73E	N/A	N/A	N/A
Rear Left Door Module	RLDM	Body CAN	0x1A21	0x721	N/A	N/A	N/A
Motor Generator Module	MGM	Propulsion CAN	0x1631	0x731	N/A	N/A	N/A
Driver Massage Module	MMD	SMD_LIN1	0x1A32	0x732	0x32	N/A	N/A
Massage Module Passenger	MMP	SMP_LIN1	0x1A33	0x733	0x33	N/A	N/A
Medium Voltage Battery Module	MVBM	Propulsion CAN	0x1652	0x752	N/A	N/A	N/A
Middle Voltage Converter Module	MVCM	Propulsion CAN	0x163D	0x73D	N/A	N/A	N/A
Middle Voltage Electrical Motor	MVEM	Propulsion CAN	0x1651	0x751	N/A	N/A	N/A
On Board Charger	OBC	Propulsion CAN	0x1634	0x734	N/A	N/A	N/A
Overhead Console	OHC	CEM_LIN3	N/A	N/A	0x7D	N/A	N/A
Occupant Weight Sensor	OWS	SRS_LIN1	N/A	N/A	0x42	N/A	N/A
Parking Assist System	PAS	CAN FD 4	0x1460	0x760	N/A	02:00:00:00:14:60	198.18.36.96
Rear Right Door Module	RRDM	Body CAN	0x1A22	0x722	N/A	N/A	N/A
Air Particulate Matter Sensor Indoor	PMSI	CCM_LIN2	N/A	N/A	0x76	N/A	N/A
Air Particulate Matter Sensor Outdoor	PMSO	CCM_LIN2	N/A	N/A	0x75	N/A	N/A
Driver Power Operated Door	DPOD	DRM_LIN1	N/A	N/A	0x7D	N/A	N/A
Passenger Power Operated Door	PPOD	DRM_LIN1	N/A	N/A	0x7C	N/A	N/A
Power Steering Control Module 1	PSCM1	Chassis CAN 1	0x1670	0x770	N/A	N/A	N/A
Power Steering Control Module 2	PSCM2	Chassis CAN 1	0x1671	0x771	N/A	N/A	N/A
Passenger Smart Surface Module	PSSM	IHU_LIN2	N/A	N/A	0x7B	N/A	N/A
Rear Light Control Module Left	RCML	Body Exposed CAN	0x1BB5	0x7B5	N/A	N/A	N/A
Rear lamp Control Module Middle	RCMM	Body Exposed CAN	0x1BB7	0x7B7	N/A	N/A	N/A
Rear Light Control Module Right	RCMR	Body Exposed CAN	0x1BB6	0x7B6	N/A	N/A	N/A
Rear Console Switch Module	RCSM	IHU_LIN2	N/A	N/A	0x7A	N/A	N/A
Left Power Operated Door	LPOD	DRM_LIN1	N/A	N/A	0x7B	N/A	N/A
Rain and Light Sensor Module	RLSM	CEM_LIN1	N/A	N/A	0x7B	N/A	N/A
Rear View Mirror Display	RMD	Passive Safety CAN	0x1513	0x713	N/A	N/A	N/A
Restraint Module Left	RML	Passive Safety CAN	0x1510	0x710	N/A	N/A	N/A
Restraint Module Right	RMR	Passive Safety CAN	0x1511	0x711	N/A	N/A	N/A
Rear Power Operated Door	RPOD	DRM_LIN1	N/A	N/A	0x7A	N/A	N/A
Seat Module Driver	SMD	Body CAN	0x1A27	0x727	N/A	N/A	N/A
Rear Seat Display Module	RSDM	IHU_LIN2	N/A	N/A	0x78	N/A	N/A
Rear Seat Entertainment Module	RSEM	GMSL	N/A	N/A	0x7D	N/A	N/A
Steering Angle Sensor	SAS	Chassis CAN 1	0x1672	0x772	N/A	N/A	N/A
Second Battery Monitoring Sensor	SBMS	ECM_LIN3	N/A	N/A	0x76	N/A	N/A

Steering Column Lock	SCL	Chassis CAN 2	0x1615	0x715	N/A	N/A	N/A
Sun Curtain Module	SCM	CEM_LIN3	N/A	N/A	0x7B	N/A	N/A
Sunroof Intelligent Change Colour Glass Module	SIG	CEM_LIN3	N/A	N/A	0x79	N/A	N/A
Seat Module Passenger	SMP	Body CAN	0x1A28	0x728	N/A	N/A	N/A
Seat Module Back	SMB	Body CAN	0x1A29	0x729	N/A	N/A	N/A
Interior Lighting Control Module	ILCM	Body CAN	0x1A30	0x730	N/A	N/A	N/A
Side Obstacle Detection Left	SODL	CAN FD 3	0x1450	0x750	N/A	N/A	N/A
Side Obstacle Detection Right	SODR	CAN FD 3	0x1451	0x751	N/A	N/A	N/A
Sun Roof Module	SRM	CEM_LIN3	N/A	N/A	0x78	N/A	N/A
Supplementary Restraint System	SRS	Backbone FlexRay	0x1C01	0x701	N/A	N/A	N/A
Suspension Module	SUM	Chassis CAN 2	0x1614	0x714	N/A	N/A	N/A
Steering Wheel Module	SWM	CEM_LIN4	0x1B72	N/A	0x72	N/A	N/A
Steering Wheel Switch Module	SWSM	DIM_LIN1	N/A	N/A	0x7D	N/A	N/A
TBD	SWTL	Private DHU CAN	N/A	N/A	N/A	N/A	N/A
TBD	SWTR	Private DHU CAN	N/A	N/A	N/A	N/A	N/A
Transmission Actuator Control Module	TACM	Propulsion CAN	0x163C	0x73C	0x3C	N/A	N/A
Telematics & Connectivity Module	TCAM	TCAM Ethernet	0x1011	0x711	N/A	Ethernet: 02:00:00:00:10:11	Ethernet: 198.18.32.17 *
Thermal Cup Holder /	TCH	IHU_LIN2	N/A	N/A	0x79	N/A	N/A
Transmission Control Module	TCM	Propulsion CAN	0x1632	0x732	N/A	N/A	N/A
Door Radar Module	DRM	Body CAN	0x1A31	0x731	0x31	N/A	N/A
VAC Blower Module Rear	VBMR	CCM_LIN2	N/A	N/A	0x74	N/A	N/A
Vehicle Dynamics Domain Master	VDDM	Backbone FlexRay	0x1601	0x701	N/A	N/A	N/A
Vent Grill Actuator	VGA	CCM_LIN3	N/A	N/A	0x7D	N/A	N/A
Vehicle Gateway Module	VGM	Connectivity Ethernet	0x1001	0x701	N/A	Ethernet: 02:00:00:00:10:01	Ethernet: 198.18.32.1 **
Wiper Motor Module	WMM	CEM_LIN1	N/A	N/A	0x12	N/A	N/A
Wireless Phone Charger/Near-field Communication	WPC/NFC	IHU_LIN1	N/A	N/A	0x7D	N/A	N/A
Wireless Power Transfer	WPT	Propulsion CAN	0x163B	0x73B	N/A	N/A	N/A

Note: This requirement is valid for all ECU's.
Note 1: The VCM/VGM shall route diagnostic traffic to infotainment subnodes via the IHU. Therefore, the subnodes need to have the same IP address as the IHU on the Ethernet interface.
Note 2: The VCM/VGM shall route diagnostic traffic with destination to DIM GPU (0x1802) via APIX2 using the stated IP address.
Note 3: The IHU is using 198.18.34.1 for the IHU VP and 198.18.34.15 for the IHU AP.
Note 4: The DHU is using 198.18.34.1 for the DHU QNX and 198.18.34.15 for the DHU Android.

Table: OBD addresses

ECU name according to SAE J1979	ECU name in GEEA2.0	29 bit address Req/Resp	11 bit address for Request/Response
ECM EngineControl	ECM	0x18DA10F1/0x18DAF110	7E0/7E8
TCM TransmisCtrl	TCM	0x18DA18F1/0x18DAF118	7E1/7E9
BECM B+EnergyCtrl	BECM	0x18DACBF1/0x18DAF1CB	7E2/7EA
BECM B+EnergyCtrl	MVBM	0x18DACBF1/0x18DAF1CB	7E2/7EA
DMCM DriveMotorCtrl	IEM	0x18DA11F1/0x18DAF111	7E3/7EB
DMCM DriveMotorCtrl	MVEM	0x18DA11F1/0x18DAF111	7E3/7EB
SGCM Start/GenCtrl	IGM	0x18DA14F1/0x18DAF114	7E4/7EC
SGCM Start/GenCtrl	MGM	0x18DA14F1/0x18DAF114	7E4/7EC
SGCM Start/GenCtrl	ISGM	0x18DA14F1/0x18DAF114	7E4/7EC
BCCM B+ChargerCtrl	OBC	0x18DACAF1/0x18DAF1CA	7E5/7ED
BCCM B+ChargerCtrl	CDD	0x18DACAF1/0x18DAF1CA	7E5/7ED
BSCM BrakeSystem	VDDM	0x18DA28F1/0x18DAF128	7E6/7EE
EBBC ElecBrkBstCtrl	BBM	0x18DA29F1/0x18DAF129	7E7/7EF

4.2.1.4 CAN Network

4.2.1.4.1 Requirements

1630 v3 **Component Selection - CAN Physical Interfaces**

Version history:
(3):

Purpose: This is to prevent extra DC load on the network.
Source: This is necessary for maintaining the CAN signal voltage level specified in partially powered network
Verification Method: Analysis
Legacy ID: 76513v1

For power-switched ECUs an “ideal passive” transceiver shall be used. Transceivers shall not leak more than 5µA from an active CAN bus, in power-off mode.

For battery-powered ECUs a transceiver with sleep functionality must be used.

The list of approved components is available in *Components For Physical Interfaces,* for reference see Diagnostics & ECU Platform - Requisite documents.

Note: This requirement is valid for all CAN ECU's.

1631 v5 Missing frame master - CAN

Version history:
(5):

Purpose: The missing frame concept are used for pinpointing network errors on the bus.
Source: UDSonCAN
Verification Method: Test
Legacy ID: 76550v2

The ECU routing diagnostic requests to and from a CAN network shall be missing frame master for that network. Note that an ECU can be a slave on FlexRay and a master on CAN and vice versa. The missing frame master ECU's are listed below.

ECU	Network									
	Infotainment CAN	Passive Safety CAN	Propulsion CAN	Chassi CAN 1	Chassi CAN 2	Body Exposed CAN	Body CAN	Connectivity CAN	CAN FD1	CAN FD2
VGM	-	-	-	-	-	-	-	Master		
IHU	Master	-	-	-	-	-	-	-		
VDDM	-	-	Master	Master	Master	-	-	-		
CEM	-	-	-	-	-	Master	Master	-		
ASDM	-	Master	-	-	-	-	-	-	Master	Master

Note: This requirement is valid for the ECUs in the table above.

1632 v2 No priority inversion

Version history:
(2):

Purpose: The concept of guaranteed worst case latency offers several advantages to the system designer as well as to the node designer and node supplier. The SDB team will use guaranteed worst case latency to design a set of frames which will fulfil the signalling requirements given in the specifications defining vehicle functions. It will also be possible to guarantee that no signals will be overwritten, even when they have been transferred over gateways. Guaranteed worst case latency also permits busload to be significantly higher than the 30% often seen as the practical limit for reliable communication on CAN buses.

The communication driver shall be designed to be free of priority inversion when frames are transmitted on the CAN bus by AUTOSAR based ECU's. If all CAN stacks are designed according to the priority inversion avoidance (PIA) concept it becomes possible to analyse the bus behaviour a priori. This

analysis makes it possible to calculate and guarantee the worst case latency of all frames.

Source: EESE Strategic Decision

Verification Method: Analysis

Legacy ID: 77275v1

The communication driver shall be designed to be free of priority inversion when frames are transmitted on the CAN bus.

96055 v2 Terminating ECUs on CAN

Version history:
(2): Add the Terminal in the DHU Private CAN1

Purpose:

Source: ECU topology

Verification Method: Analysis

Legacy ID:

List of terminating ECUs for each CAN network

Table: Terminating ECUs

CAN Network	Terminating ECU1	Terminating ECU2
Propulsion CAN	ECM	VGM
Chassis CAN1	ECM	SAS
Chassis CAN2	ECM	VDDM
Passive Safety CAN	ASDM	SRS
AD Redundancy CAN	PSCM	FLC
Safety CAN FD1	FSRL	FSRR
Safety CAN FD2	ASDM	FLR
Safety CAN FD3	SODL	SODR
Safety CAN FD4	ASDM	PAS
Body Exposed CAN	HCML	HCMR
Body CAN	CEM	CCM
DHU Private CAN1	DHU	SWTL
Connectivity CAN HS	VGM	TCAM
Private CAN1	FLC	IMU
Private CAN2	BECM	EVCC
Private Info CAN	IHU/DHU	AUD
Diagnostics CAN	VGM	Tester

4.2.1.5 Network Management

4.2.1.5.1 Requirements

1657 v37 **Network management CAN frame identifiers**

Version history:
(37):

Purpose: To ensure network management functionality.
Source: -
Verification Method: Analysis
Legacy ID: 130742v31

An ECU using AUTOSAR CAN Network Management shall be able to receive network management frames with CAN identifiers ranging from 0x500 to 0x53F.

The CAN frame identifiers in the table below shall be used for transmission of network management frames containing the NM-PDU.

ECU Alias	NM CAN Network	NM CAN Frame ID	VFCVector ID
PSCM	AD Redundancy CAN	0x501	0x540
IMU	AD Redundancy CAN	0x502	0x541
FLC	AD Redundancy CAN	0x503	0x542
BBM	AD Redundancy CAN	0x504	0x543
CEM	Body CAN	0x501	0x541
CCM	Body CAN	0x502	0x542
DDM	Body CAN	0x503	0x543
PDM	Body CAN	0x504	0x544
DRM	Body CAN	0x505	0x545
POT	Body CAN	0x507	0x547
RLDM	Body CAN	0x509	0x549
TRM	Body CAN	0x50A	0x555
RRDM	Body CAN	0x510	0x550
SMD	Body CAN	0x512	0x552
SMP	Body CAN	0x513	0x553
SMB	Body CAN	0x514	0x554
ILCM	Body CAN	0x515	0x555
CEM	Body Exposed CAN	0x52A	0x540
HCML	Body Exposed CAN	0x531	0x541
HCMR	Body Exposed CAN	0x532	0x542
RCML	Body Exposed CAN	0x533	0x543
RCMR	Body Exposed CAN	0x534	0x544
RCMM	Body Exposed CAN	0x535	0x545
ASDM	CAN FD 1	0x501	0x540
FSRR	CAN FD 1	0x502	0x541

FSRL	CAN FD 1	0x503	0x542
ASDM	CAN FD 3	0x501	0x540
SODL	CAN FD 3	0x502	0x541
SODR	CAN FD 3	0x503	0x542
ASDM	CAN FD 4	0x501	0x540
PAS	CAN FD 4	0x502	0x541
ASDM	CAN FD 2	0x501	0x540
FLR	CAN FD 2	0x502	0x541
FLC	CAN FD 2	0x503	0x542
ECM	Chassis CAN 1	0x510	0x540
VDDM	Chassis CAN 1	0x521	0x545
PSCM1	Chassis CAN 1	0x523	0x541
SAS	Chassis CAN 1	0x527	0x544
PAS	Chassis CAN 1	0x528	0x546
ASDM	Chassis CAN 1	0x529	0x547
TCM	Chassis CAN 1	0x530	0x548
HVSC	Chassis CAN 1	0x531	0x549
MVEM	Chassis CAN 1	0x532	0x550
ECM	Chassis CAN 2	0x51F	0x546
VDDM	Chassis CAN 2	0x522	0x549
SUM	Chassis CAN 2	0x525	0x548
SCL	Chassis CAN 2	0x526	0x547
BBM	Chassis CAN 2	0x527	0x550
DSRC	Connectivity CAN	0x508	0x548
TCAM	Connectivity CAN	0x509	0x549
VGM	Connectivity CAN	0x533	0x543
AUD	Infotainment CAN	0x53A	0x54A
IHU	Infotainment CAN	0x53B	0x54B
SRS	Passive Safety CAN	0x50B	0x54B
RMR	Passive Safety CAN	0x50D	0x54D
RML	Passive Safety CAN	0x50E	0x54E
ASDM	Passive Safety CAN	0x50F	0x54F
DVR	Passive Safety CAN	0x511	0x551
DMM	Passive Safety CAN	0x512	0x552
RMD	Passive Safety CAN	0x513	0x553
EGSM	Propulsion CAN	0x516	0x545
TCM	Propulsion CAN	0x517	0x555
OBC	Propulsion CAN	0x518	0x552
BECM	Propulsion CAN	0x519	0x541
SRS	Propulsion CAN	0x51A	0x544
IGM	Propulsion CAN	0x51B	0x548
IEM	Propulsion CAN	0x51C	0x547
DEM	Propulsion CAN	0x51E	0x542
ECM	Propulsion CAN	0x520	0x543

ECS	Propulsion CAN	0x521	0x544
MVBM	Propulsion CAN	0x522	0x550
MVCM	Propulsion CAN	0x523	0x551
ISGM	Propulsion CAN	0x524	0x549
WPT	Propulsion CAN	0x525	0x556
VDDM	Propulsion CAN	0x526	0x554
ESM	Propulsion CAN	0x528	0x546
TACM	Propulsion CAN	0x52B	0x553
VGM	Propulsion CAN	0x52C	0x557
MGM	Propulsion CAN	0x52D	0x558
CDD	Propulsion CAN	0x52F	0x559

I

Note 1:
NM CAN Frames use IDs in range 0x501-0x53F, all unlisted in this range shall be considered spare IDs.

Note 2:
This requirement applies to all CAN ECUs where AUTOSAR Network Management is used.

45485 v4 **Network management FR frame identifiers**

Version history:
(4):

Purpose: To ensure network management functionality.
Source:
Verification Method: Analysis
Legacy ID:

An ECU using AUTOSAR FR Network Management shall be able to receive network management frames with FR identifiers.

The FR frame identifiers in the table below shall be used for transmission of network management frames containing the NM-PDU.

Node on Backbone FlexRay	FR NM Node ID
DIM	0x51
VDDM	0x52
CEM	0x53
ASDM	0x54
SRS	0x55
IHU	0x56

VGM	0x57
BBM	0x59

Node on FLC FlexRay	FR NM Node ID
ASDM	0x54
FLC	0x55

4.2.1.6 Software Download

4.2.1.6.1 Requirements

1662 v2 **Bootloader CAN busoff handling**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To increase availablility of the system. Avoid that an ECU gets stuck in bus-off state, or exits programming session due to network errors.
Source:
Verification Method: Analysis
Legacy ID: 392111v0

If hardware support automatic CAN busoff recovery, that recovery mechanism shall be enabled.
If hardware doesn't support automatic bus-off recovery the bootloader shall check all active CAN controllers for bus-off status at maximum 10 ms intervals and if bus-off is detected start the bus-off recovery sequence within 10ms.
This requirement applies to Primary Boot Loader (PBL) and Secondary Boot Loader (SBL).

Note: If a permanent busoff condition is detected that can not be recovered, and a complete and compatible application exist, the S3server timer shall still run and will finally elapse, causing and exit of the programming session.

1663 v7 **Bootloader routing - Spare networks**

Version history:
(6):
(7):

Purpose: To be able to handle future growth of networks.
Source: EESE strategical decision
Verification Method: Analysis
Legacy ID: 316251v2

The ECU shall prepare the bootloader for adding new networks according to below.

1 : D : N : E
| | | |

| | | --- ECU ID
| | --- Network ID
| --- Domain ID
--- Always 1

VGM

0x1010-0x101F--> Connectivity Ethernet
0x1100-0x11FF--> Connectivity Ethernet
0x1200-0x12FF--> Connectivity Ethernet
0x1300-0x13FF--> Connectivity Ethernet
0x1400-0x14FF--> Flexray
0x1500-0x15FF-> Flexray
0x1600-0x16FF--> Flexray
0x1800-0x18FF--> Flexray
0x1A00-0x1AFF--> Flexray
0x1B00-0x1BFF--> Flexray
0x1C00-0x1CFF--> Flexray
0x1D00-0x1DFF--> Flexray
0x1E00-0x1EFF--> Flexray

VDDM

0x161x -> Chassis CAN 2
0x162x -> Chassis CAN 2 (Spare)
0x163x -> Propulsion CAN
0x164x -> Propulsion CAN (Spare)
0x165x -> Propulsion CAN
0x166x -> Propulsion CAN
0x167x -> Chassis CAN 1
0x168x -> Chassis CAN 1 (Spare)

CEM

0x1A02-0x1AFF--> Body CAN
0x1B10-0x1B1F--> LIN1
0x1B20-0x1B2F--> LIN3
0x1B30-0x1B3F--> LIN0/K-Line
0x1B40-0x1B4F--> LIN5
0x1B50-0x1B5F--> LIN2
0x1B60-0x1B6F--> LIN6
0x1B70-0x1B73--> LIN4
0x1B74-0x1B79--> LIN7
0x1B7A-0x1B7F--> LIN8
0x1BB0-0x1BBF--> Body CAN Exposed

ASDM

0x1410-0x141F--> Flexray
0x1420-0x142F--> Safety CAN FD (Spare)
0x1430-0x143F--> Safety CAN FD 1
0x1440-0x144F--> Safety CAN FD 2
0x1450-0x145F--> Safety CAN FD 3

0x1460-0x146F--> Safety CAN FD 4
0x1470-0x147F--> Safety CAN FD (Spare)
0x1510-0x151F--> Passive Safety CAN

Note: This requirement is applicable for ASDM, CEM, VGM and VDDM.

1664 v7 Deployment of the J2534 requirements

Version history:
(7):

Purpose: To define which ECU that shall implement the J2534 requirements.
Source: Diagnostic and Bootloader Gateway Specification
Verification Method: Analysis
Legacy ID: 76536v1

The VGM shall implement the J2534 requirements according to *Diagnostic and Bootloader Gateway Specification* for reference see Diagnostics & ECU Platform - Requisite documents.

Target ECU	ECU addr	J2534 Request	J2534 Response
VDDM	0x1601	0x1DD01601	0x1EC02E80
BBM	0x1602	0x1DD01602	0x1EC04E80
ECM	0x1630	0x1DD01630	0x1EC60E80
MGM	0x1631	0x1DD01631	0x1EC62E80
TCM	0x1632	0x1DD01632	0x1EC64E80
OBC	0x1634	0x1DD01634	0x1EC68E80
BECM	0x1635	0x1DD01635	0x1EC6AE80
IGM	0x1636	0x1DD01636	0x1EC6CE80
IEM	0x1637	0x1DD01637	0x1EC6EE80
ISGM	0x163E	0x1DD0163E	0x1EC7CE80
CDD	0x1650	0x1DD01650	0x1ECA0E80
MVEM	0x1651	0x1DD01651	0x1ECA2E80
MVBM	0x1652	0x1DD01652	0x1ECA4E80
Functional addr.	0x1FFF	0x1DD01FFF	N/A

Note: This requirement is valid for the VGM.

1665 v30 SWDL time in aftermarket

Version history:
(29):
(30):

Purpose: To optimize SWDL in aftermarket.
Source: REQ - 017049 in Teamcenter: The maximum total time for purchase, deliverance and flashing new software to a complete vehicle is 0.8h (48 min). This time includes: - Connection/disconnection of the communication

interface, e.g. WLAN or Ethernet-cable, to VIDA. - Purchase and deliverance of the software from PIE to VIDA. The maximum total time for purchase and datatransfer from PIE to VIDA is 0.3h (18 min). - Flashing the software into ECUs in the vehicle using VIDA. The maximum total time for flashing new software to a complete vehicle is 0.5h (30 min). The maximum total time for flashing new software to a single ECU is 0.2h (12 min).

Verification Method: Test

Legacy ID: 316711v22

The figures below includes maximum time per ECU for erase software and download software. This requirement is valid on ECU level, assuming an ideal downloading tool. Supplier should do their best to make the SWDL time as short as possible. ECU's that do not appear in the list have no SWDL function until a new agreement is made with the base technology team.

Table: SWDL Time in aftermarket

ECU Name	ECU Alias	SWDL time in aftermarket
Active Safety Domain Master 1	ASDM1	4.5min
Active Safety Domain Master 2	ASDM2	18min
Active Safety Domain Master 3	ASDM3	26min
Audio Module	AUD	TBD
Brake Booster Module	BBM	80s
Battery Energy Control Module	BECM	TBD
Battery Monitoring Sensor	BMS	3-5min
Climate Control Module	CCM	150s
Charger and DCDC module	CDD	TBD
Central Electronic Module	CEM	55s
Centre Stack Display	CSD	TBD
Driver Door Module	DDM	Software file: 30s Parameter file: 10s
Differential Electronic Module	DEM	30s
Driver Information Module	DIM(GC)	APP:250s Calibration：1800s
Driver Information Module	DIM(AC)	APP:60s Calibration：20s
Driver Information Screen	DIS	TBD
Driver Monitor Module	DMM1	4min
Driver Power Operated Door	DPOD	TBD
Driver Radar Module	DRM	TBD
Dedicated Short Range Communication	DSRC	TBD
Digital Video Recorder	DVR	9min
Engine Control Module1	ECM1	195s
Engine Control Module2	ECM2	200s
Engine Control Module3	ECM3	247s

Electronic Gear Selector Module	EGSM	TBD
Exterior Sound Module	ESM	60s
Forward Loooking Camera1	FLC1	7-8.5min
Forward Loooking Camera2	FLC2	7-8.5min
Forward Looking Radar	FLR	80s
Front Side Radar Left	FSRL	120s
Front Side Radar Right	FSRR	120s
Headlamp Control Module Left1	HCML1	TBD
Headlamp Control Module Left2	HCML2	TBD
Headlamp Control Module Right1	HCMR1	TBD
Headlamp Control Module Right2	HCMR2	TBD
Head Up Display	HUD	App:650s Calibration:25s
High Voltage Secondary Conversion Module	HVSC	TBD
Inverter ERAD Module	IEM	3min
Inverter Generator Module	IGM	3min
Infotainment Head Unit	IHU	20min
Display Head Unit	DHU	<45min
Interior Lighting Control Module	ILCM	TBD
Integrated Start Generate Module	ISGM	5min
Left Power Operated Door	LPOD	TBD
Motor Generator Module	MGM	3min
Driver Massage Module	MMD	TBD
Massage Module Passenger	MMP	TBD
Medium Voltage Battery Module	MVBM	3-5min
Medium Voltage Converter Module	MVCM	3min
Medium Voltage Converter Module2	MVCM2	TBD
Middle Voltage Electrical Motor	MVEM	3min
On Board Charger	OBC	TBD
Parking Assist System1	PAS1	2-3min
Parking Assist System2	PAS2	2-3min
Parking Assist System3	PAS3	20min
Passenger Door Module	PDM	Software file: 30s Parameter file: 10s
Power Operated Tailgate	POT	30s
Passenger Power Operated Door	PPOD	TBD
Power Steering Control Module 1	PSCM1	60s
Power Steering Control Module 2	PSCM2	60s

Rear Light Control Module Left	RCML	TBD
Rear light Control Middle Module	RCMM	TBD
Rear Light Control Module Right	RCMR	TBD
Rear Left Door Module	RLDM	Software file: 30s Parameter file: 10s
Rear View Mirror Display	RMD	TBD
Restraint Module Left	RML	TBD
Restraint Module Right	RMR	TBD
Rear Power Operated Door	RPOD	TBD
Rear Right Door Module	RRDM	Software file: 30s Parameter file: 10s
Steering Angle Sensor	SAS	TBD
Steering Column Lock	SCL	20s
Seat Module Back	SMB	120s
Seat Module Driver	SMD	120s
Seat Module Passenger	SMP	120s
Side Obstacle Detection Left	SODL	120s
Side Obstacle Detection Right	SODR	120s
Supplementary Restraint System	SRS	50s
Suspension Module	SUM2	120s
Steering Wheel Module	SWM	180s
Transmission Actuator Control Module	TACM	60s
Telematics & Connectivity Module	TCAM	10min
Transmission Control Module1	TCM1	180s
Transmission Control Module2	TCM2	160s
Trailer Module	TRM	TBD
Vehicle Dynamics Domain Master	VDDM	4min
Vehicle Gateway Module	VGM	30s
Wireless Power Transfer	WPT	TBD

Notes:
If project or cluster is not specifically mentioned for an ECU then the given download time is valid for all project&/clusters that ECU is defined in.

1666 v33 **SWDL time in factory**

Version history:
(32):
(33):

Purpose: To optimize SWDL in factory.

Source: REQ - 017646 in Teamcenter: The total available time for SWDL in factory EOL is 6.5 minutes at 60 cars/h (total SWDL time for all ECU:s/files on the same bus).

Verification Method: Test

Legacy ID: 76551v25

The figures below includes maximum time per ECU for erase software and download software. This requirement is valid on ECU level, assuming an ideal downloading tool.Supplier should do their best to make the SWDL time as short as possible. ECU's that do not appear in the list have no SWDL function until a new agreement is made with the base technology team.

Table: SWDL Time in factory

ECU Name	ECU Alias	SWDL time in factory
Active Safety Domain Master 1	ASDM1	0
Active Safety Domain Master 2	ASDM2	30s
Active Safety Domain Master 3	ASDM3	30s
Audio Module	AUD	0
Brake Booster Module	BBM	80s
Battery Energy Control Module	BECM	0
Battery Monitoring Sensor	BMS	0
Climate Control Module	CCM	0
Charger and DCDC module	CDD	0
Central Electronic Module	CEM	55s
Centre Stack Display	CSD	0
Driver Door Module	DDM	10s
Differential Electronic Module	DEM	0
Driver Information Module	DIM(GC)	0
Driver Information Module	DIM(AC)	30s
Driver Information Screen	DIS	0
Driver Monitor Module1	DMM1	120s
Driver Power Operated Door	DPOD	0
Driver Radar Module	DRM	0
Dedicated Short Range Communication	DSRC	120s
Digital Video Recorder	DVR	0
Engine Control Module1	ECM1	195s
Engine Control Module2	ECM2	200s
Engine Control Module3	ECM3	247s
Electronic Gear Selector Module	EGSM	0
Exterior Sound Module	ESM	0
Forward Loooking Camera1	FLC1	0

Forward Loooking Camera2	FLC2	0
Forward Looking Radar	FLR	0
Front Side Radar Left	FSRL	0
Front Side Radar Right	FSRR	0
Headlamp Control Module Left1	HCML1	0
Headlamp Control Module Left2	HCML2	0
Headlamp Control Module Right1	HCMR1	0
Headlamp Control Module Right2	HCMR2	0
Head Up Display	HUD	0
High Voltage Secondary Conversion Module	HVSC	0
Inverter ERAD Module	IEM	0
Inverter Generator Module	IGM	0
Infotainment Head Unit	IHU	1s
Display Head Unit	DHU	90s
Display Head Unit	DHU	90s
Interior Lighting Control Module	ILCM	0
Integrated Start Generate Module	ISGM	0
Left Power Operated Door	LPOD	0
Motor Generator Module	MGM	0
Driver Massage Module	MMD	0
Massage Module Passenger	MMP	0
Medium Voltage Battery Module	MVBM	0
Medium Voltage Converter Module	MVCM	0
Medium Voltage Converter Module2	MVCM2	0
Middle Voltage Electrical Motor	MVEM	0
On Board Charger	OBC	0
Parking Assist System1	PAS1	0
Parking Assist System2	PAS2	0
Parking Assist System3	PAS3	0
Passenger Door Module	PDM	10s
Power Operated Tailgate	POT	20s
Passenger Power Operated Door	PPOD	0
Power Steering Control Module 1	PSCM1	60s
Power Steering Control Module 2	PSCM2	60s
Rear Light Control Module Left	RCML	0
Rear light Control Middle Module	RCMM	0
Rear Light Control Module Right	RCMR	0
Rear Left Door Module	RLDM	10s
Rear View Mirror Display	RMD	0
Restraint Module Left	RML	0

Restraint Module Right	RMR	0
Rear Power Operated Door	RPOD	0
Rear Right Door Module	RRDM	10s
Steering Angle Sensor	SAS	0
Steering Column Lock	SCL	5s
Seat Module Back	SMB	15s
Seat Module Driver	SMD	15s
Seat Module Passenger	SMP	15s
Side Obstacle Detection Left	SODL	0
Side Obstacle Detection Right	SODR	0
Supplementary Restraint System	SRS	6s
Suspension Module	SUM2	60s
Suspension Module	SUM2	60s
Steering Wheel Module	SWM	0
Transmission Actuator Control Module	TACM	30s
Telematics & Connectivity Module	TCAM	20s
Transmission Control Module1	TCM1	88s
Transmission Control Module2	TCM2	60s
Trailer Module	TRM	0
Vehicle Dynamics Domain Master	VDDM	240s
Vehicle Gateway Module	VGM	0
Wireless Power Transfer	WPT	0

Notes:
SWDL time equal to "0" second means that the ECU is not allowed to download data in the factory. In this case the software shall be pre-loaded before delivery to the factory.
If project or cluster is not specifically mentioned for an ECU then the given download time is valid for all project&/clusters that ECU is defined in.

1667 v3 **Separate software files**

Version history:
(3):

Purpose: To reduce the amount of non-diffable content in software files and to enable the possibility of flexible ECU updates.

Source:

Verification Method: Analysis

Legacy ID:

Purpose:

In order to increase software flexibility and reduce the amount of non-diffable content in vbf-files, all graphics, fonts, and speech files used by an ECU must be delivered separately without

any software dependencies.

Note: This requirement is applicable for ECUs which stores vehicle HMI, fonts, or speech related files.

6411 v1 **In-Place Reconstruction of Delta Files**

Version history:
(1):

Purpose: To reduce the time spent on the programming phase and the time spent in transfer over data communication networks.
Source: General Bootloader requirements - Delta encoding
Verification Method: Analysis
Legacy ID:

The ECU shall support in-place reconstruction of software delta files according to the [*Delta Encoding*] specification.

This requirement is applicable for ECUs which software size exceeds 10MB in size or exceeds 10min in programming time.

Note: The programming time is calculated from the first EraseMemory/RequestDownload request to the last positively confirmed RequestTransferExit/CheckMemory response (SBL is excluded from this time calculation).

4.2.1.7 AUTOSAR

4.2.1.7.1 Requirements

1669 v2 **Communication Gateway Processing**

Version history:
(2):

Purpose: To guarantee message timing.
Source: Communication Gateway
Verification Method: Analysis
Legacy ID: 76543v1

The gateway functionality (which is a part of the communication layer) shall run every Communication SW processing period.

Note: This requirement is valid for all ECU having more than one bus interface.

1670 v3 **Communication Stack Processing Period**

Version history:
(3):

Purpose: To be able to optimize network performance.

Source: -
Verification Method: Analysis
Legacy ID: 76539v2

The Communication stack processing period shall be 5,0 ms.

Note: This requirement is valid for all ECUs with CAN, FlexRay, or LIN Master bus interfaces.

1671 v4 **Communication Stack Processing Period Jitter**

Version history:
(4):

Purpose: To be able to optimize network performance.
Source: -
Verification Method: Test
Legacy ID: 76542v2

The jitter of the Communication stack processing period shall be less than 1 ms.

Note: This requirement is valid for all ECUs with CAN, FlexRay, or LIN Master bus interfaces.

1672 v1 **AUTOSAR version**

Version history:
(1):

Purpose: Specify AUTOSAR version.
Source:
Verification Method: Analysis
Legacy ID: 353876v0

AUTOSAR version 4.2.2 shall be used.

Note: Other revisions such as 4.0.3 may be used after agreement with the base technology team.

4.2.1.8 Startup Time

4.2.1.8.1 Requirements

1673 v2 **Startup Time**

Version history:
(2): Change the Verification Method to Test

Purpose: In order to conform with tools used during manufacturing, and reduce the time spent in workshops and over-the-air diagnostics/SWDL use cases.
Source:
Verification Method: Test
Legacy ID:

Purpose:

The ECU shall startup within 2,5 seconds and have full diagnostic and software download capabilities within this time. The time shall be measured from a state where the power to the ECU is completely turned off until it is capable of receiving diagnostic requests from both an internal tester in the vehicle, and an external tester connected through OBD.

Note: This requirement is valid for all ECUs with diagnostics and/or software download capabilities, and is applicable for both AUTOSAR based microcontrollers and microcontrollers running an operating system such as Linux.

4.3 Communication

4.3.1 CAN Data Link Layer

4.3.1.1 CAN Data Link Layer

4.3.1.1.1 Requirements

392 v4 **Standards for CAN interface**

Version history:
(4):

Purpose: Specification of CAN protocol
Source: N.A.
Verification Method: Test
Legacy ID: 53881v2

The interface shall be specified according to CAN standard using bit rate specified in section Network Transmission Speed 500 kbps. References can be found in [Ext CAN DL 3], [Ext CAN DL 4] and [Ext CAN DL 5].

393 v4 **The priority between standard documents and this document**

Version history:
(4):

Purpose: To optimise the CAN system solution
Source: N.A.
Verification Method: Analysis
Legacy ID: 53850v2

In case of discrepancy between the standard documents and this specification, this specification has precedence.

4.3.1.1.1.1 ECU Level

The power supply voltage level at the node shall be measured continuously. HW and SW inaccuracy in the monitoring of the supply voltage level must be taken into account when the

ECU is designed. The ECU must meet this operating voltage range requirement also in a worst case condition.
Network communication outside the mandatory voltage range is encouraged on the condition that the ECU can correctly read and write messages and cause no disturbance to the network. There must be margins in both ends of the operating voltage range in regards to the voltage level where communication becomes unreliable or is no longer possible due hardware constraints.

394 v3 **Communication protocol**

Version history:
(3):

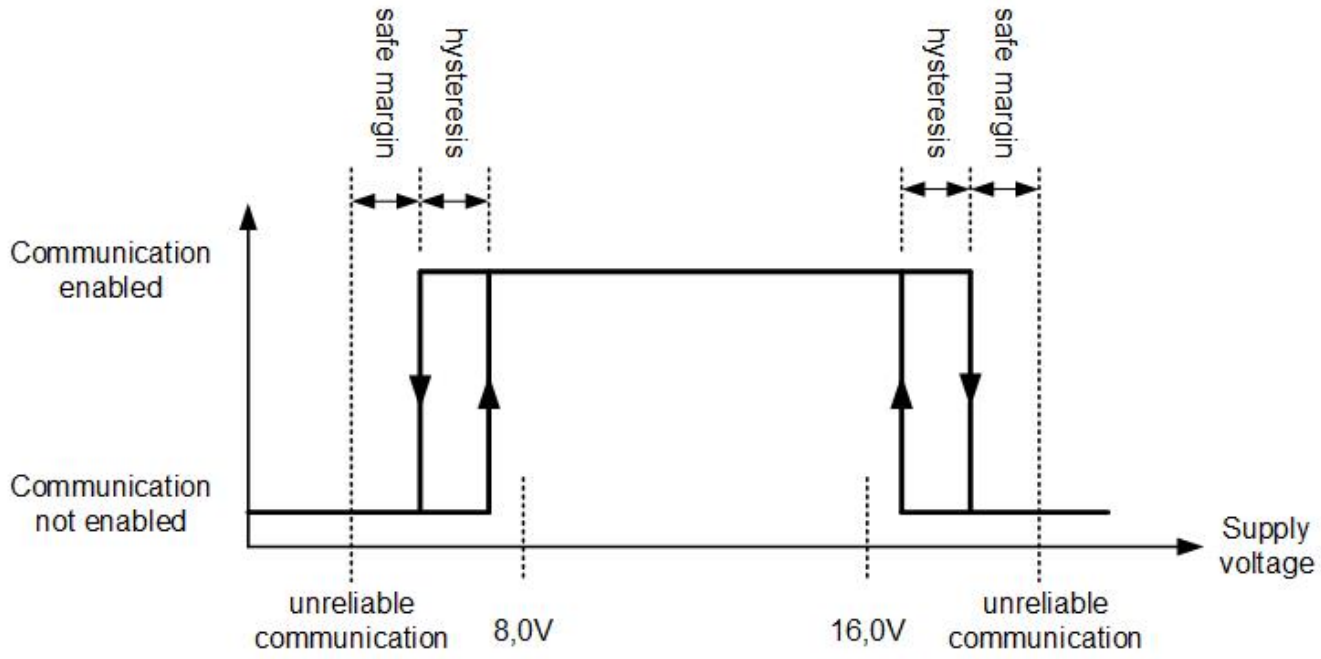
Purpose: Specification of CAN protocol used
Source: N.A.
Verification Method: Analysis
Legacy ID: 53848v2

Controller Area Network (CAN) in accordance with CAN V2.0 [Ext CAN DL 2] .

395 v3 **Behaviour outside normal supply voltage range**

Version history:
(3):

Purpose: Ensure stability at high and low supply voltage
Source: N.A.
Verification Method: Test
Legacy ID: 53883v2



Hysteresis shall be used to ensure stable behaviour under low and high supply voltage conditions. Hysteresis shall be minimum 100mV.
Note: This requirement addresses bus communication only and is not aligned with the operating voltage range requirement for the specific ECU.

396 v3 **CAN bus level during reset**

Version history:
(3):

Purpose: Prevent the ECU from disturbing the network during a local reset.
Source: N.A.
Verification Method: Test
Legacy ID: 53884v2

Modules shall not drive the CAN bus dominant during module reset.

397 v4 **Bus fault reporting**

Version history:
(4): Change the Verification Method to Analysis

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 53885v2

At least one module on each bus must be able to detect and report faults on the physical bus. The faults and their respective fault codes are specified in [CAN DL 6] . The SRD Diagnostics and ECU Platform for the specific vehicle program specifies which module should perform this fault monitoring.

4.3.1.1.1.1.1 Resynchronisation and bit sampling

399 v3 **Resychronisation strategy**

Version history:
(3):

Purpose: Increase communication robustnes
Source: N.A.
Verification Method: Analysis
Legacy ID: 53862v2

Resynchronisation on recessive to dominant transitions only.

400 v3 **Sample mode**

Version history:
(3):

Purpose: Single sample mode is the preferred operating mode at 500 kbit/s.
Source: N.A.
Verification Method: Analysis
Legacy ID: 53863v2

Only single sample mode shall be used.

4.3.1.1.1.1.2 Speed tolerance definition

401 v3 **PLL bypass**

Version history:
(3):

Purpose: Minimise communication jitter
Source: N.A.
Verification Method: Analysis
Legacy ID: 53853v2

If a PLL is being used by, and if supported by the chosen microcontroller, the CAN physical layer shall be clocked directly from the base oscillator, i.e. PLL bypass should be used.
Note: there are various ways to achieve this, e.g. control register in the CAN controller, microcontroller central clock module, etc.

402 v3 **PLL locking**

Version history:
(3):

Purpose: Minimise the number of communication error events
Source: N.A.
Verification Method: Analysis
Legacy ID: 53861v2

When a PLL is being used to clock the CAN physical layer, transmission and reception are only allowed when the PLL is locked.

4.3.1.1.1.1.3 Network Transmission Speed - 500kbps

398 v3 **Bit time set up for 500 kbps**

Version history:
(3):

Purpose: Specify accepted sampling point range
Source: N.A.
Verification Method: Test + Analysis
Legacy ID: 53864v2

Bit timing set up for nodes with maximum transmission speed tolerance ± 0.4% .
For a maximum transmission speed tolerance of ± 0.4%, the bit time set up for nodes shall be:

BIT NtQ	tSJWmin Num tQ	tSEG2min Num tQ	tSEG2max Num tQ	Sample point %	SJWmin %
22	3	3	5	77 – 86	13.6
21	3	3	5	76 – 86	14.3
20	3	3	5	75 – 85	15.0
19	3	3	4	79 – 84	15.8
18	2	2	4	78 – 89	11.1
17	2	2	4	76 – 88	11.8
16	2	2	3	81 – 88	12.5
15	2	2	3	80 – 87	13.3
14	2	2	3	79 – 86	14.3
13	2	2	2	85	15.4
12	2	2	2	83	16.7
11	2	2	2	82	18.2
10	2	2	2	80	20.0

Note 1: Calculation of the Tseg1 value
NTq = Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2, see [Ext CAN DL 3]
Sync_Seg = 1
Tseg1 = Prop_Seg + Phase_Seg1
Tseg2 = Phase_Seg2

Note 2: to achieve 10 or more tQ at 500kbps, the input clock to the CAN controller must have a minimum frequency of 5MHz.

4.3.1.1.1.1.4 CAN FD bit register settings for 2 Mbps

22265 v2 **CAN FD bit register settings for 2 Mbps**

Version history:
(2):

Purpose: Support CAN FD with 2 Mbps data rate
Source:
Verification Method: Inspection
Legacy ID:

One of the following standardized bit rate settings shall be used.

Alternative 1:

Frame part	Number of time quantas	Quanta time	Sjw number of time quantas	tSEG2 number of time quantas
Arbitration part	40	50ns	8	8
Data part	10	50ns	2	2

Alternative 2:

Frame part	Number of time quantas	Quanta time	Sjw number of time quantas	tSEG2 number of time quantas
Arbitration part	80	25ns	16	16

Data part	20	25ns	4	4

Other settings may only be used after agreement with the OEM project Base Technology responsible.

4.3.1.1.2 Requisite Documents

These are requirements only if explicitly referred to as that.

Reference	Document
[Ext CAN DL 1]	ISO 7498-1: Open Systems Interconnection - basic reference model
[Ext CAN DL 2]	CAN specification V2.0 part B (BOSCH)
[Ext CAN DL 3]	ISO 11898-1:2015(E): Road Vehicles - Controller Area Network (CAN) – Part 1: Data Link Layer and Physical Signalling
[Ext CAN DL 4]	ISO 11898-2:2016(E): Road Vehicles - Controller Area Network (CAN) – Part 2: High-Speed Medium Access Unit
[Ext CAN DL 5]	SAE J2284 – 4: High Speed CAN for Vehicle Applications at 500 KBPS with CAN FD Data at 2 Mbps, issued 2016-06
[CAN DL 6]	UDSonCAN

4.3.1.1.3 Definitions and Abbreviations

Abbreviation	Description
CAN	Controller Area Network
ECU	Electronic Control Unit
ISO	International Standards Organisation
PLL	Phase Locked Loop
SAE	Society of Automotive Engineers

4.3.2 Guideline for quality factor signals

4.3.2.1 Guideline for quality factor signals

4.3.2.1.1 Software Component Recommendations

In a distributed system where data is sent between different nodes over a network, normally the node generating/producing the data is the only node knowing the quality of the data (through a diagnostic function). To be able to inform the node using the data about the data quality, a ‘quality factor’ value (Q-factor) can be linked to the signal containing the data and sent over the network at the same time. The ‘quality factor’ indicates some status information about the data signal, e.g. if the value is inaccurate because of a faulty sensor, and can be used to make decision about replacement values/reconfiguration.

4.3.2.1.1.1 QF values definition and use

4.3.2.1.1.1.1 Value 3 - 11b OK

This value indicates that the signal is accurate and that the data it contains is truly representative of the real state.

The accuracy of the transmitted signal under this normal operating condition should be stated in the signal definition in the SWRS.

4.3.2.1.1.1.2 Value 2 - 10b Not within specification

This value indicates that although the signal still contains data that is representative of the real state of the vehicle, a fault has been detected which indicates that the accuracy has been affected to an extent which may affect the performance or durability of exceptional subscribers. Which nodes that are typical subscribers, exceptional subscribers and the tolerance for the value 2 must be agreed in advance and the expected behaviour of subscribers must be well known and documented. The reduction in accuracy from that achieved with normal operation must be well documented. It is expected that only a small minority of subscribers to QFs will react to the value 2.

Example: Engine torque calculation - if the primary input of mass air flow fails it is possible to determine engine torque using manifold pressure, but with a notable reduction in accuracy. Most users of the Torque signal will not be affected but the reduced accuracy may not be adequate to allow the optimum shift quality / durability compromise to be maintained in the transmission. In this case the transmission can change to a slightly jerkier, but still functional, shift behaviour with minimal effect on the customer

4.3.2.1.1.1.3 Value 1 - 01b No data exists

This value indicates that the signal is set to its initial value because it has not yet been possible to measure or calculate real data. This situation may persist until the node power up sequence is fully completed (all applications have been executed at least once). The expected delay until real data is provided should be specified by the publishing node and/or function owner.
The QF must change to a new appropriate value as soon as the application program has started to write fully valid information (this means that the real measurements and/or calculations are complete and enough time has elapsed for any filter function to have a valid output value) to the NOS.
The receiving function must be designed to function correctly with no data for up to the maximum time defined above. After this time it should treat a QF value of 1 as identically equal to a QF value of 3 unless there is a clear and documented customer benefit for it to do otherwise. (This is to support legacy and minimise the customer impact)
There must be no undocumented assumptions or interpretations of the QF! Example: receiving functions should not assume that they will always receive the value of 1 and should NOT attempt to use it as a reset marker. Some transmitting functions may be able to transmit valid data fast enough to catch the first transmission by the NOS.
If there is a need for a more extensive information exchange for a specific feature, then this must be solved with a unique defined signal between the concerned functions, and shall not be solved by a undocumented interpretation of the standard QF.

4.3.2.1.1.1.4 Value 0 - 00b Faulty

This value indicates that there is a serious fault which prevents the signal being correctly generated by the publisher. The publisher may store a DTC to indicate this.
Where there is ANY useful data potentially left in the signal under these conditions, this should be stated in the SWRS to allow subscribers to make an optimum judgement as to whether customer functionality can be preserved.
Subscribers which can no longer provide safe and/or reliable customer function must default to a safe condition and if required store a DTC indicating what has happened. Note, as with the publisher, the subscriber must give careful consideration to whether it is actually necessary to

remove customer function, particularly when the received signal is a secondary input to the function. It is not generally recommended that subscribers use the data when QF = 0 and extra care should be taken if the option is used.

4.3.2.1.1.2 QF values definition and use summary table

The following table summarises the specified behaviour of publishers and subscribers, but it is essential that the detailed definitions above are studied in addition.

QF	Publisher		Typical Subscriber		Exceptional subscriber	
	Behaviour	**DTC**	**Behaviour**	**DTC**	**Behaviour**	**DTC**
3	OK". The corresponding data signal is calculated within the stated accuracy.	No	Function works normally no reaction.	No	Function works normally no reaction.	No
2	"Not within Specification". Signal subscribers should assume the corresponding data signal is calculated within a stated additional percentage of the stated accuracy.	MC*			Gracefully degraded performance specifically agreed and documented	MC*
1	"No data exists" No data has yet been written to the NOS.	No	Function works normally no reaction. Normal operation must include any special measures needed to cope with initial delay to receive valid data			No
0	"Faulty". The accuracy of the corresponding data signal is undefined.	MC*	Degraded or no performance.			MC*

Table: Summary of possible publisher/subscriber behaviour

*Invalid data DTC:s are conditionally mandatory. The Base Technology Specification *UDS Data* specifies during what conditions DTC:s shall be set based on invalid data.

4.3.2.1.1.3 Supporting information and clarifications from lessons learned

4.3.2.1.1.3.1 DTCs

In the above notes DTCs are mentioned a number of times. These recommendations do not override the diagnostics requirements for the vehicle (typically specified by the UDS Data Specification) for example that a DTC should only be set in the subscriber where there is a noticeable customer symptom, which might not occur if a QF <> 3 value only persisted for a short time. The publisher shall set a DTC if it has transmitted even a single value of 2 or 0. The reason is that the publisher cannot know how fast the subscribers will set DTCs and it is vital that if any subscriber has an DTC there is a corresponding publisher DTC to allow the root cause to be identified and rectified.

Accuracy (after the initial period when QF might be 1 has expired)

Define: True value of data: d_t (i.e. as would be measured by lab instrumentation)
Published value of data: d_p (i.e. the value which appears on the bus)
Normal error: Δ_n (i.e. the range of error possible with the publisher working correctly)
Additional error: Δ_a (i.e. the error added by the faults which have been detected)

if $(d_t - \Delta_n \leq d_p \leq d_t + \Delta_n)$ then QF = 3
elseif $(d_t - (\Delta_n + \Delta_a) \leq d_p \leq d_t + (\Delta_n + \Delta_a))$ then QF = 2
else QF = 0

Note : Δ_n and Δ_a to be defined and documented to the fullest extent possible

4.3.2.1.1.3.2 Temporary errors and filtering

It is beyond the scope of this guideline to define how short duration faults should be handled in all cases. The basic question is "what if a disturbance of typically short duration occurs such as a voltage spike?". In this case the publisher may be temporarily unable to provide accurate data. If it is determined that the module must report a problem with the data due to low or high supply voltage, an appropriate QF value must be chosen depending on the accuracy of the data as for any other fault.

4.3.2.1.1.3.3 Engine cranking

A very important special case of a temporary inability to transmit correct data is during engine cranking, or more generally during changes of vehicle power states. There is a generally agreed principle that ECUs should detect cranking, and not react to problems of any sort, including unexpected QF values, or set DTCs until a suitable time has elapsed after the engine has started to be sure that full function of the electrical system has been restored. Of course the exception to this is ECU functions which are needed to enable the cranking process. Unfortunately there is not yet an agreed standard for all brands and vehicles as to how this should work and it also depends on the vehicle architecture.

4.3.2.1.1.3.4 Detecting missing data

The QF is just like any other signal as far as its transmission around the network is concerned. It is NOT modified in any way by passing through a gateway, nor changed if missing frames are detected. Other mechanisms exist to deal with these issues, and QFs should NOT be used for this purpose. The QF shall not and can not be used by the subscriber, to get information on whether or not the publisher function is reporting any data to the network transport layer. This shall be taken care of by an UB (UpdateBit) or other mechanism such as a message counter.

4.3.2.1.1.3.5 Multiple variants of a node

In case different variants of one node exist (e.g. Engine Control Modules, Brake Control Modules…), these nodes shall ensure that they apply the same rules for generating QFs they send out across their variants. In this case, the receiver can always react in the same way on a specific QF signal received from different node variants, i.e. has a common interface to the variants.

4.3.2.1.1.3.6 Data value

The corresponding data signal to the QF signal shall always contain the actual measured/calculated value for the transmitted data. This is valid for all possible QF values. Only during initialization, this data signal may contain the init value defined in the signal SWRS.

4.3.2.1.1.3.7 Philosophy and development of the QF concept

This section is not formally part of the definition, but attempts to explain the logic for WHY the guideline has been written as it has, and the history of problems which have lead to this logic.
It is vital to understand that the QF is a mechanism to assist the mitigation of faults for the benefit of the vehicle user only. It only has meaning as part of the design of a distributed function in which sharing of data contributes to delivering a customer feature. In the past the generation and use of quality factors have been done in isolation which has lead to publishing nodes setting low quality factor values too readily, and subscribing nodes having needlessly severe reactions. This combination produces a situation where the symptoms experienced by the customer in the vehicle may be far more severe than is actually justified by the fault which has occurred.
The QF guideline can't cover all possible needs of data quality information, but is rather a generalized standard method that will fit the majority of needs. If there are exceptions that can't be handled within the framework defined in this spec, then the subscriber and publisher must agree upon a specific solution for their specific need.
It is essential that each user of a QF checks that their reaction is appropriate to the behaviour of the publisher. It is expected that the FMEA of the function should indicate the effect on that function of an error in its incoming signals. This should inform whether a QF is needed at all, and what the correct reaction should be.
In theory of course it may be that different users of a signal might have different accuracy thresholds where reactions were required. For this reason there may be cases where quality indication mechanisms not covered by this framework are needed. This spec does not forbid these. So far no case has been found where defining more than two different states of degradation produce additional benefit.

4.3.2.1.1.3.8 History of QF 1

The QF value of 1 requires some specific explanation, as it has caused confusion and debate in the past. This value was used in earlier versions of this specification as the initial value AND to indicate that although no fault had been detected, some diagnostic evaluation was still ongoing. The classic example of this is the coolant temperature. OBD legislation requires that this is monitored to make sure the engine is genuinely warming up correctly. This evaluation inevitably takes many minutes, and there may indeed be usage patterns of the vehicle where the evaluation is never completed.
There is no conceivable customer function on the vehicle which could not operate for several minutes after start, and in some usage patterns not operate at all. Therefore there was no useful way in which a subscriber could react in any other way to the value of 1 than to treat it exactly the same as 3. Unfortunately many subscribers followed a wrong but understandable logic that because 2 was worse than 3 and 0 was worse than 2; then 1 must also be worse than 2. This created fundamentally wrong behaviour in the vehicle which required software changes to correct it.
The definition of QF 1 now should eliminate this doubt, but subscribing functions should be designed where possible to work correctly with publishing functions with the earlier behaviour to avoid the cost and risk of changing the publishing function.
If there is a need for a more extensive information exchange for a specific feature, then this must be solved with a unique defined signal between the concerned functions, and shall not be solved by a undocumented use of the standard QF.
An example of this has been raised in the late stages of development of this version of the specification. This occurs in vehicles where the steering angle sensor can only detect movement of the steering wheel and not its absolute position. An absolute position is

established by learning the straight ahead position by correlating with wheel speeds and yaw rate. This takes some time and clearly can only happen with the vehicle moving. The straight ahead position is remembered when the power is off, but if the steering wheel is moved with power off, there will be an error until a new straight ahead position is learned. None of the defined QF values in this spec allow for this, so a further signal indicating completion of the learning process had to be added to the vehicle

4.3.2.1.1.4 Recomendations for creation of QF:s

- The Quality Factor (QF) should handle and carry the quality number (value) of the data transmitted on the vehicle network.
- The Quality Factor should be a 2-bit signal.
- The Quality Factor signal should be transmitted in a signal group with the corresponding data signal. Note: The Quality Factor should be written by the application at the same time as the corresponding data signal without any interruption in between.
- The Quality Factor signal value should reflect the status of its corresponding data by using the following values, these are universally defined in section *QF values definition and use*:
  - **11b** "OK"
  - **10b** "Not within Specification"
  - **01b** "No data exists"
  - **00b** "Faulty"
- The initial value of the Quality Factor signal should be 01b.
- The conditions for the Quality Factor states and state transitions must be specified and documented (through state-transition diagrams and/or text in the SWRS doument). It is the responsibility of the person owning the specification of how a particular data signal is calculated to document a complete and unambiguous specification of how the associated QF is to be calculated.
- It is not allowed to use QF data as the cause of setting an OBD (and thus ERC classified) DTC.

4.3.2.1.2 Introduction

4.3.2.1.2.1 What is the purpose of this document

The purpose of this guideline is to inform about a mechanism for providing subscribers to a signal with information about failures which will reduce the accuracy of the signal. This mechanism can be used by subscribers to mitigate customer symptoms and/or improve the ability to diagnose and rectify the vehicle. This guide line defines a method for publishers to define a Quality Factor, and subscribers to reacts to it to ensure that the QF provides the intended benefits.

4.3.2.1.2.2 Risks

In the past, misunderstanding and therefore inappropriate use of this and similar mechanisms has sometimes had the opposite effect i.e. it has aggravated symptoms and made it harder to diagnose and rectify the vehicle with in some cases some severe costs as a consequence of the errenous hanling. Therefore it is of absolute importance that all affected participants agree upon and document when and how to behave on the different levels of qulity factor values. There have been many problems caused by the functions in publishing ECUs calculating QFs wrongly and the functions in subscribing ECUs reacting wrongly to QFs even where they have been calculated correctly. It is vital therefore that designers of subscribing functions carefully check the behaviour of the publishing function and do NOT rely on them meeting this guide line.

The level of accuracy or toleranses of a signal value is not of interest in this document and must be documented in another document, see section *Who should read this document?*.

4.3.2.1.2.3 Who should read this document

This document should be read by the system integrators, the component developers, the test engineers, and the function owners which are using or will use quality factor signals. They together must agree upon a strategy how to handle different levels of quality factor values and manually document it properly in the SWRS so all future users (senders and receivers) of a quality factor signal always know how a certain qulity factor value must be treated.

4.3.2.1.2.4 Definitions and Abbreviations

Definition/Abbreviation	Description
ECU	Electronic Control Unit
DTC	Diagnostic Trouble Code
ERC	Emission Related Component,
FMEA	Failure Modes and effects analysis
NOS	Network Operating Software. The standard software package which includes a component to handle network communications.
Operating Cycle	The Operating Cycle consists of the time between Power Up (alt. Wake Up) and Power Down (alt. Enter Sleep Mode) as specified by each node specification. E.g. the engine Operating (Driving) Cycle consists of engine start-up, and engine shutoff ('Engine start' is defined as the point at which normal, synchronized spark and fuel control is obtained or when the engine reaches a speed of 150 rpm below the normal, warmed-up idle speed (as determined in the drive position for vehicles equipped with an automatic transmission))
QF (or Q-factor)	Quality Factor
SWRS	Software Requirement Specification

4.3.3 DoCAN

4.3.3.1 DoCAN

4.3.3.1.1 Transport layer protocol

The transport/network layer required by this document shall be compliant to *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services*, with the restrictions/additions as defined by the subsequent sub-sections.

This document specifies OEM specific requirements. If there are other kinds or requirements like legislative requirements (for example OBD requirements), these requirements may have higher priority than the requirements in this specicifation. However if the other requirements have performance requirements lower than the performance requirements specified in this document, the performance requirements in this document shall be followed. This is valid as long as the performance requirement in this document does not contradict timeout requirements in the other document.

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Standardise communication to ensure all ECUs uses the same diagnostic communication specifications. ISO standard shall be followed as far as possible unless otherwise specified to reduce cost and make implementation easier.
Source:
Verification Method: Analysis
Legacy ID: 60003v1

The transport/network layer shall be compliant to *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services* with the restrictions/additions as defined by this document. If there are contradictions between this specification and *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services,* then this specification shall override *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services*.

1199 v2 **Precedence of requirements**

Version history:
(2):

Purpose: Define which set of requirements to use for which circumstances.
Source:
Verification Method: Analysis
Legacy ID: 60004v1

If there are other kinds or requirements like legislative requirements (for example OBD requirements), these requirements may have higher priority than the requirements in this specification. However if the other requirements have performance requirements lower than the performance requirements specified in this document, the performance requirements in this document shall be followed. This is valid as long as the performance requirement in this document does not contradict timeout requirements in the other document.

4.3.3.1.1.1 Protocol control information specification

4.3.3.1.1.1.1 BlockSize (BS) parameter definition

When in a non-programming session, both the server and client side shall be able to receive a complete diagnostic message without sending any intermediate flow control frames, to ensure as swift communication as possible. Client side shall be able to receive 4095 bytes diagnostic message without sending any flow control frames i.e. BlockSize = 0x00.

Block size for CAN using 8 bytes frame length
When in a non-programming session the server shall be able to receive diagnostic messages of at least 100 bytes without sending flow control frames. Translated into block size the server shall be able to receive 100/(7 bytes / CF), which rounds up to 15 Consecutive Frames, i.e. the

server shall have a BlockSize parameter set to 15 (0x0F) or larger, e.i. BlockSize 1 to 14 is not allowed. Server may also set BlockSize to 0x00 if it is capable of receiving 4095 bytes diagnostic message without sending Flow Control frames.

Block size for CAN FD using 64 byte frame length
When in a non-programming session the server shall be able to receive diagnostic messages of at least 200 bytes without sending flow control frames. Translated into block size the server shall be able to receive 300/(64 bytes / CF), which rounds up to 5 Consecutive Frames, i.e. the server shall have a BlockSize parameter set to 5 (0x05) or larger, e.i. BlockSize 1 to 4 is not allowed. Server may also set BlockSize to 0x00 if it is capable of receiving 4095 bytes diagnostic message without sending Flow Control frames.

When in programming session server side shall be able to receive a complete diagnostic message of 4095 bytes without sending any intermediate Flow Control frames i.e. BlockSize shall always be 0x00 for all services the server supports.

1200 v4 BlockSize parameter non-programming session server side

Version history:
(4):

Purpose: Define BlockSize for non-programming session server side. For more information see section BlockSize (BS) parameter definition
Source: Section "BlockSize (BS) parameter definition"
Verification Method: Test
Legacy ID: 60006v2

BlockSize shall be 15 (0x0F) or higher for server side. If the server can receive 4095 bytes without intermediate FlowControl frames, the BlockSize parameter shall be set to 0x00.

22281 v1 CAN FD BlockSize parameter non-programming session server side

Version history:
(1):

Purpose: Define BlockSize for non-programming session server side
Source:
Verification Method: Test
Legacy ID:

BlockSize shall be 5 (0x05) or higher for server side. If the server can receive 4095 bytes without intermediate FlowControl frames, the BlockSize parameter shall be set to 0x00.

1201 v3 BlockSize parameter non-programming session client side

Version history:
(3):

Purpose: Define BlockSize for non-programming session client side. For more information see section BlockSize (BS) parameter definition.
Source:
Verification Method: Test

Legacy ID: 60007v2

For ECU gatewaying responses from CAN to another network:
In non-programming session the receiver of an upstream message must respond with BS=0 so that there are never more than one FlowControl N_PDUs per 4095 bytes of data (including the first FlowControl N_PDU) when buffer resources makes 4095 bytes available.
In non-programming session, if the largest available RAM buffer is less than the length of the message, that buffer must be used, and BS set so that all of that buffer is used. This will reduce the number of FlowControl N_PDU used for reception of the complete message.
It is mandatory to use the smallest RAM buffer available that can hold the complete message, when such buffer is available.
Note:
Example A: A 300 bytes message is received, and the only RAM buffer available is 4095 bytes. BS is 0 or at least 42.
Example B: A 300 bytes message is received, and RAM buffers are available with 50 bytes and 100 bytes, so none of the buffers can hold the complete message. A 100 bytes RAM buffer is used in favour of 50 bytes buffers, which reduce the number of FlowControl N_PDUs. BS will be 13.

1202 v2 **BlockSize parameter programming session server side**

Version history:
(2):

Purpose: Define BlockSize for programming session server side. For more information see section BlockSize (BS) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60008v1

BlockSize (BS) shall always be 0x00 in programming session for server side.

1203 v2 **BlockSize parameter programming session client side**

Version history:
(2):

Purpose: Define BlockSize for programming session client side. For more information see section BlockSize (BS) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60009v1

BlockSize (BS) shall always be 0x00 in programming session for client side

4.3.3.1.1.1.2 Separation (STmin) parameter definition

The STmin parameter shall reflect the maximum processing time the ECU needs to have in between each CAN frame, regardless of ECU state or activity. For example if an ECU sets a STmin time of 1ms, the supplier must guarantee that the ECU is always able to process consecutive frames at this rate. Preferably the STmin time shall be as low as possible, but the

server is allowed to set a maximum of 15ms. Client side shall be dimensioned to handle STmin time of 0 which means CAN frames are being sent back to back. The reason why STmin requirement is tougher for gateways than the ECU's on the network is:

- When in non-programming session, most requests requiring some performance are small enough to be fitted into one frame.
- More lenient requirements lessen the burden on the servers, making it possible to use ECU's with less performance as servers.

When in programming session both server and client side shall be able to handle receiving frames with no separation time primarily for 2 purposes.

- Considering the amount of data to transfer via diagnostics when in programming session compared to the other sessions, the need for speed is greater when in programming session.
- Since when in bootloader mode the ECU will not be burden by simultaneously running the normal functionality supported by the ECU, the ECU shall be able to handle this extra processor load.

1204 v1 **Separation time (STmin) non-programming session server side**

Version history:
(1):

Purpose: Define STmin for non-programming session server side. For more information see section Separation (STmin) parameter definition.
Source: Section "Separation (STmin) parameter definition"
Verification Method: Test
Legacy ID: 60010v0

Separation time (STmin) for server side when in non-programming session shall be maximum 15ms.

1205 v3 **Separation time (STmin) non-programming session client side**

Version history:
(3):

Purpose: Define STmin for non-programming session client side. For more information see section Separation (STmin) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60011v2

When in non-programming session the ECU shall be able to receive CF N_PDU with minimum 5ms separation time (i.e. STmin set to 5ms).
Note: It is allowed to support shorter separation time than this requirement specifies.

1206 v2 **Separation time (STmin) programming session server side**

Version history:
(2):

Purpose: Define STmin for programming session server side. For more information see

section Separation (STmin) parameter definition.

Source:
Verification Method: Test
Legacy ID: 60012v1

Separation time (STmin) for programming session shall be maximum 0ms for server side.

1207 v3 **Separation time (STmin) programming session client side**

Version history:
(3):

Purpose: Define STmin for programming session. For more information see section Separation (STmin) parameter definition.
Source:
Verification Method: Test
Legacy ID: 60013v2

When in programming session the ECU shall be able to receive CF N_PDU with minimum 5ms separation time (i.e. STmin set to 5ms).
Note: It is allowed to support shorter separation time than this requirement specifies.

4.3.3.1.1.1.3 Separation time between single frames

In programming session the ECU shall be able to handle receiving single frames (SF) with no separation time. This applies regardless of whether the single frames are received from one or more servers.
Examples for a server:
a: queued request where both the first and second request each fits in a single frame.
Examples for a gateway ECU:
b: a server sends negative response NRC 0x78 and immediately after it sends final response.
c: several servers sends negative response NRC 0x78 and immediately after they send final responses.
In these examples and if using 500kbps bit rate up to four single frames shall be received within one millisecond.

1208 v1 **Separation time between single frames - programming session**

Version history:
(1):

Purpose: To ensure no messages are lost and good timing performance.
Source:
Verification Method: Test
Legacy ID: 380118v0

In programming session the ECU shall handle receiving single frames (SF N_PDU) with as short time apart as the CAN protocol allows. This applies both when the single frames are from the same sender and from different senders.

Note: the shortest separation time of CAN frames is the minimum Interframe Space.

4.3.3.1.1.2 Maximum number of FC. Wait frame transmission (N_WFTmax)

The N_WFTmax counter shall be set to 255 (0xFF). In programming session, the server may use FC.Wait if it receives a request to be queued in server, when it is processing a previously received request. A gateway ECU may use FC.Wait in any situation and session when it does not have enough buffers available for receiving the complete message.

1209 v2 **N_WFTmax value for server side**

Version history:
(2):

Purpose: N_WFTmax counter shall be set to 255 (FF hex) for server side. This requirement is applicable for all sessions. For more information see section Maximum number of FC.Wait frame transmissions (N_WFTmax).
Source:
Verification Method: Analysis
Legacy ID: 60015v1

N_WFTmax counter shall be set to 255 (FF hex) for server side. This requirement is applicable for all sessions.

1210 v2 **N_WFTmax value for client side**

Version history:
(2):

Purpose: Define the value to be used as N_WFTmax for client side. For more information see section Maximum number of FC.Wait frame transmissions (N_WFTmax).
Source:
Verification Method: Test
Legacy ID: 60016v1

N_WFTmax counter shall be set to 255 (FF hex) for client side. This requirement is applicable for all sessions.

1211 v3 **Flowcontrol Wait usage**

Version history:
(3):

Purpose: Use FC.Wait to halt message reception in appropriate situations.
Source:
Verification Method: Test
Legacy ID: 115798v2

Non-gateway server:
In programming session, the server may use FC.Wait if it receives a request when it is

processing a previously received request. In all other situations in all sessions usage of FC.Wait is not allowed.

Gateway ECU:

The gateway ECU shall use FC.Wait if it receives a message when buffers are not available for receiving the complete message. This applies to server side and client side.

4.3.3.1.1.3 Network layer timing

4.3.3.1.1.3.1 Timing parameters

The time it takes to send one CAN frame (N_Ar and N_As) depends on the bit rate of the CAN network and if the bit rate is set to 500kbit/s the lowest possible value for N_Ar and N_As is 250 µs. If a CAN with a different bit rate is used, performance of N_Ar and N_As will have to be recalculated.

Different parameter values applies depending on if the ECU is in programming or non-programming session.

The network layer timing parameters for non-programming session is described in Table – Timing parameters non-programming session.

Item	Parameter	Timeout value	Performance requirement value
1	N_As	1000 ms	-
2	N_Ar	1000 ms	-
3	N_Bs	1000 ms	-
4	N_Br	-	N_Br < 20 ms for FC.CTS and FC.OVFLW. For FC.Wait the periodicity shall be N_Bs * 0.9 = 900ms ± 50ms.
5	N_Cs	-	N_Cs < 20 ms.
6	N_Cr	1000ms	-

Table – Timing parameters non-programming session The network layer timing parameters for programming session is described in Table – Timing parameters programming session.

Item	Parameter	Timeout value	Performance requirement value
1	N_As	1000 ms	-
2	N_Ar	1000 ms	-
3	N_Bs	1000 ms	-
4	N_Br		N_Br < 2000 ms for FC.CTS and FC.OVFLW. For FC.WT the periodicity shall be N_Bs * 0.9 = 900ms ± 50ms.
5	N_Cs		N_Cs < 100 ms
6	N_Cr	1000 ms	-

Table – Timing parameters programming session All requirements from this section are specified in section Requirements from section Timing parameters.

Note: When performance timing is verified, timestamp information normally shows time at start of each frame. In such case it is important to know that N_Ar/N_As is included when e.g. evaluating N_Br and N_Cs performance.

1212 v2 **N_As timeout in non-programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60017v1

N_As timeout value shall be 1000ms in non-programming session.

1213 v2 **N_Ar timeout in non-programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60019v1

N_Ar timeout value shall be 1000ms in non-programming session.

1214 v2 **N_Bs timeout in non-programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60021v1

N_Bs timeout value shall be 1000ms in non-programming session.

1215 v4 **N_Br Performance requirement in non-programming session for FC.CTS and FC.OVFLW**

Version history:
(4):

Purpose: The time spent waiting shall be minimized. Although N_Br time does not affect

the transmission time as much as N_Cs time, the diagnostic kernel scheduling shall still be done with the same frequency

Source: Table - Timing parameters non-programming session, item 4

Verification Method: Test

Legacy ID: 60062v3

N_Br shall be N_Br < 20 ms for FC.CTS and FC.OVFLW when in non-programming session.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.
d. The Autosar CAN Transport Layer configuration parameter CanTpNbr is not configuring the N_Br performance for the first transmitted FlowControl after a FirstFrame although CanTpNbr is stated to be a perfomance configuration parameter. Instead CanTpNbr is configuring an internal timeout of getting internal buffer resources for use in subsequent reception of Consecutive Frames and affect the time between a previous FlowControl and a subsequent FlowControl.

1216 v5 **N_Br Performance requirement in non-programming session for FC.Wait**

Version history:
(5):

Purpose: The time that an ECU may wait before sending a FC.Wait frame must be lower than the specified timeout to ensure the sending ECU does not time out the message. To prevent unnecessary bus load and minimize the risk that N_WFTmax counter is overrun, the periodicity of the FC.Wait frame shall be as high as possible.

Source:

Verification Method: Test

Legacy ID: 60063v4

FC.Wait frames shall be sent with a periodicity of N_Cr * 0.9 = 900ms ± 50ms non-programming session.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.
d. The Autosar CAN Transport Layer configuration parameter CanTpNbr is not configuring the N_Br performance for the first transmitted FlowControl after a FirstFrame although CanTpNbr is stated to be a perfomance configuration parameter. Instead CanTpNbr is configuring an internal

timeout of getting internal buffer resources for use in subsequent reception of Consecutive Frames and affect the time between a previous FlowControl and a subsequent FlowControl.

1217 v5 **N_Cs Performance requirement in non-programming session**

Version history:
(5):

Purpose: The time spent waiting shall be minimized and especially for N_Cs since N_Cs wait time is the factor that affects the transfer rates the most. If the send mechanism is performed in application schedule context (compared to sending on Tx interrupt), the ECU needs to poll the send function more often, and thus less time will be spent on the applications normal functionality.

Source:

Verification Method: Test

Legacy ID: 60064v4

When in non-programming session, the ECU shall be able to send Consecutive Frames with N_Cs less than 20ms.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.
d. The Autosar CAN Transport Layer configuration parameter CanTpNcs is not configuring the N_Cs performance defined by ISO-15765 although it is stated to be a perfomance configuration parameter. Instead CanTpNcs is configuring an internal timeout of getting data to be placed in the next Consecutive Frame. I.e. if no data can be retrieved internally for sending in a new Consecutive Frame, the sender will abort the transmission after CanTpNcs elapsed.

1218 v2 **N_Cr timeout in non-programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.

Source:

Verification Method: Test

Legacy ID: 60081v1

N_Cr timeout value shall be 1000ms in non-programming session.

1219 v2 **N_As timeout in programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60102v1

N_As timeout value shall be 1000ms in programming session.

1220 v3 **N_Ar timeout in programming session**

Version history:
(3):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60103v2

N_Ar timeout value shall be 1000ms in programming session.

1221 v2 **N_Bs timeout in programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60104v1

N_Bs timeout value shall be 1000ms in programming session.

1222 v3 **N_Br performance requirement in programming session for FC.CTS and FC.OVFLW**

Version history:
(3):

Purpose: When in programming session the ECU will not be burdened by

simultaneously running an application and hence the ECU shall be able to respond faster than in non-programming session.

Source: Table - Timing parameters programming session, item 4

Verification Method: Test

Legacy ID: 60105v2

N_Br shall be N_Br < 2000 microseconds for FC.CTS and FC.OVFLW when in programming session.

Note:

a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.

b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.

c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.

1223 v3 N_Br Performance requirement in programming session for FC.Wait

Version history:
(3):

Purpose: The time that an ECU may wait before sending a FC.Wait frame must be lower than the specified timeout to ensure the sending ECU does not time out the message. To prevent unnecessary bus load and minimize the risk that N_WFTmax counter is overrun, the periodicity of the FC.Wait frame shall be as high as possible.

Source:

Verification Method: Test

Legacy ID: 60106v2

FC.Wait frames shall be sent with a periodicity of N_Bs * 0.9 = 900ms ± 50ms when in programming session.

Note:

a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.

b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.

c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.

1224 v3 N_Cs Performance requirement in programming session

Version history:
(3):

Purpose: In programming session the consecutive frames shall be sent as close to back to back as possible for maximum speed.

Source: Table - Timing parameters programming session, item 5
Verification Method: Test
Legacy ID: 60107v2

In programming session N_Cs shall be N_Cs < 100 microseconds.
Note:
a. If other CAN frames with higher priority exist on CAN, then a FC N_PDU may be delayed due to CAN protocol arbitration. The requirement is applicable and verifyable only when no such frame exist.
b. For a client side of a gateway sending parallel requests, the CAN frames will have different CAN identifiers and hence different priority, and actual N_Cs may be higher than the requirement, this is expected.
c. For a server, if other servers are sending responses in parallel then a FC N_PDU may be delayed due to CAN protocol arbitration, this is expected.

1225 v2 **N_Cr timeout in programming session**

Version history:
(2):

Purpose: From a system perspective it is important that both sender and receiver side times out roughly the same time. The timeout value shall be high enough to not be affected by situations like occasional high busloads and low enough to get a user friendly system if for example an ECU is not connected.
Source:
Verification Method: Test
Legacy ID: 60108v1

N_Cr timeout value shall be 1000ms in programming session.

4.3.3.1.1.3.2 Unexpected arrival of N_PDU

In programming session, full duplex shall be supported. In all other sessions half duplex shall be supported.
The reason why full-duplex is to be supported is because of full-duplex makes it possible to queue requests on both the gateways and the servers.

1226 v4 **Duplex communication**

Version history:
(4):

Purpose: Full duplex shall be supported to make it possible to queue requests on both the gateways and the servers in programming session.
Source:
Verification Method: Test
Legacy ID: 60109v3

In programming session, full duplex shall be supported. In all other sessions half duplex shall be supported.

4.3.3.1.2 Data link layer usage

4.3.3.1.2.1 Adressing formats

The addressing format shall be “normal addressing” as specified in *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services*. Only 11bit CAN frame identifier (N_TA and N_SA) shall be used. Both physically and functionally addressed requests shall be supported.

1227 v3 — Addressing format

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define addressing format.
Source:
Verification Method: Analysis
Legacy ID: 60110v1

“Normal addressing” as specified in *[DoCAN_2] Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services* shall be used.

1228 v3 — CAN frame identifier length

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define length of CAN frame identifier.
Source:
Verification Method: Analysis
Legacy ID: 60111v1

11-bit CAN frame identifiers shall be used.

1229 v2 — Supporting functional requests

Version history:
(2):

Purpose: Define support for functional requests.
Source:
Verification Method: Test
Legacy ID: 60112v1

Functional requests shall be supported.

4.3.3.1.2.2 CAN frame data padding

For Classic CAN the DLC (Data Length Code) contained in every CAN frame shall always be set to eight. Any CAN frame with a DLC different from eight shall be considered invalidly formatted and ignored by the recipient.

For CAN FD the DLC (Data Length Code) contained in every CAN frame shall always be set to a value within the range 8 to 15. Any CAN frame with a DLC outside this range shall be considered invalidly formatted and ignored by the recipient.

Unused data shall be padded with 0x00 and the receiver of the frame shall not check the padding.

Note
The number of bytes in the data field of a CAN frame shall be indicated by the DLC. This DLC shall consist of 4 bits. The admissible number of data bytes for Classic CAN frames shall range from 0 to 8. DLCs in the range of 0 to 7 shall indicate data fields of length of 0 to 7 bytes. In Classical CAN frames, all other DLCs shall indicate that the data field is 8 bytes long. In CAN FD frames, DLCs in the range of 0 to 8 shall indicate data fields of length 0 to 8 bytes, while the other DLCs code longer data fields, according to the table below (see *[DoCAN_3] ISO 11898-1*).

Frame types	Data Lenght Code				Number of data bytes
	DLC3	DLC2	DLC1	DLC0	
Classic CAN and CAN FD	0	0	0	0	0
	0	0	0	1	1
	0	0	1	0	2
	0	0	1	1	3
	0	1	0	0	4
	0	1	0	1	5
	0	1	1	0	6
	0	1	1	1	7
	1	0	0	0	8
Classic CAN	1	0 or 1	0 or 1	0 or 1	8
CAN FD	1	0	0	1	12
	1	0	1	0	16
	1	0	1	1	20
	1	1	0	0	24
	1	1	0	1	32
	1	1	1	0	48
	1	1	1	1	64

1230 v3 **Length of Classic CAN frames**

Version history:
(3):

Purpose: Define the length of USDT frames.
Source:
Verification Method: Test
Legacy ID: 60129v1

The DLC (DataLengthCode) of USDT frames on Classic CAN format shall always be set to

eight.

22292 v2 **Length of CAN FD frames**

Version history:
(2):

Purpose: Define the length of USDT frames
Source:
Verification Method: Test
Legacy ID:

The DLC (DataLengthCode) of USDT frames on CAN FD format shall be in the range 8 to 15.

1231 v3 **Handling of CAN requests in regards of DLC**

Version history:
(3):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 60130v1

USDT request frames with a DLC not equal to eight (8) for Classic CAN, or outside the range 8-15 for CAN FD, shall be considered invalidly formatted and be ignored by the recipient.

1232 v2 **Unused data in the CAN frame**

Version history:
(2):

Purpose: Define what to do with data not used in the USDT frame.
Source:
Verification Method: Test
Legacy ID: 60133v1

Unused data shall be padded with 0x00 and the receiver of the frame shall ignore padding.

4.3.3.1.2.3 Use data frame by frame

While receiving a segmented request for TransferData 0x36 the transport protocol shall send all N_Data from each N_PDU immediately to upper layers that contain the flash programming functions (optional for all other services). The ECU must not start flashing the TransferData block after the complete TrasferData block has been received. If the N_Data (transferRequestParameterRecord) is compressed, all N_Data from each N_PDU shall be sent to the layer where decompression is performed, and that layer shall in turn send data as soon as it is decompressed to upper layers that contain the flash programming functions.
This contradicts normal specified transport layer behaviour since it states that the transport layer shall only send complete messages to upper layers. This d ecrease the time it takes to perform TransferData by writing frame by frame to flash memory. The ECU shall be able to write the data while receiving new data and therefore the ECU will not need to write as much

data at the last message and thus the P4 time will be shorter leading to shorter SWDL time.

1233 v2 **Forward N_Data from each N_PDU to upper layer**

Version history:
(2):

Purpose: Decrease the time it takes to perform TransferData via writing frame by frame. The ECU will be able to write the data while receiving new data and thus the ECU will not need to write as much data at the last message and thus the P4 time will be shorter leading to shorter SWDL time.

Source:

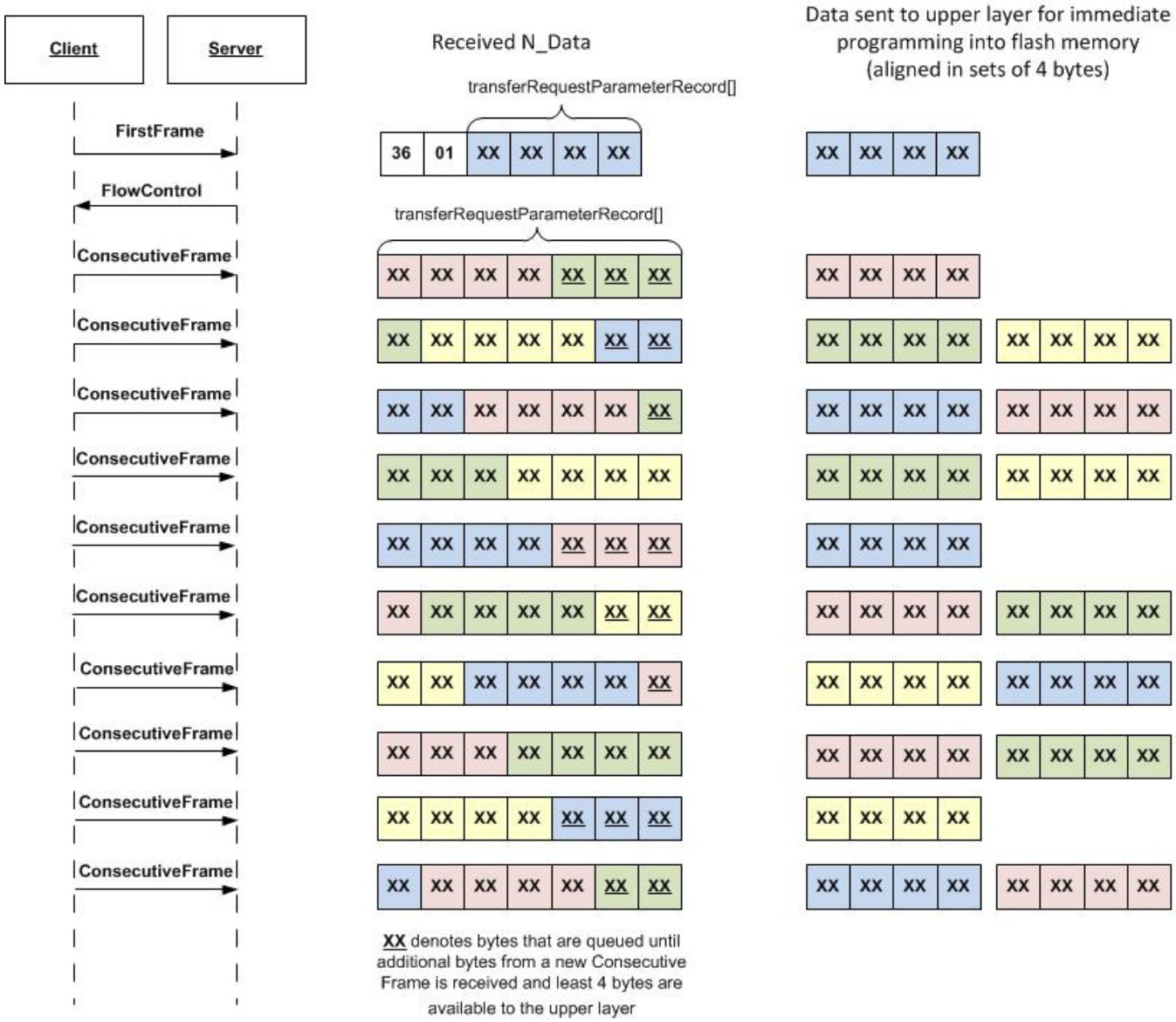
Verification Method: Analysis

Legacy ID: 128908v1

While a diagnsotic server is receiving a segmented request for TransferData 0x36 the transport protocol layer shall send all N_Data from each N_PDU to upper layers that contain the flash programming functions (optional for all other services). If the N_Data is compressed, all N_Data from each N_PDU shall be sent to the layer where decompression is performed, and that layer shall in turn send data as soon as it is decompressed to upper layers that contain the flash programming functions.
Note: this contradicts and shall override ISO specified transport layer behaviour which states that the transport layer shall only send complete diagnostic messages to upper layers.
Depending on programming alignment constrains of the flash memory, the actual programming of bytes may have to be aligned according to the flash memory and not performed immediately when data bytes are available from transport layer or decompression layer.

Example 1: Uncompressed N_Data; a microcontroller has capability to write only 4 bytes at a time into program memory, e.i. not 2 bytes in one event and adjacent 2 bytes in another event. In this case the flash programming function has to reassemble groups of 4 bytes for programming, and must queue any received bytes less than 4 until further bytes are received to get at a group of 4 bytes. In this case 4 bytes of the 6 bytes N_Data from FirstFrame can be programmed immediately, and 2 bytes (6-4) queued and joined with first 2 bytes from the first Consecutive Frame and so on. If the length of the data to be programmed is such that at the end, a group of 4 bytes can not be made, the reaining bytes, less than 4, are written anyway.
Example 2: Uncompressed N_Data; a microcontroller has capability to write only 256 bytes at a time time into program memory. In this case the flash programming function has to reassemble groups of 256 bytes for programming, and must queue any received bytes less than 256. In this case FirstFrame + 36 Consecutive Frames (7 bytes N_Data in FF and 252 bytes N_Data in 36 CF:s for a total of 259 bytes) will be received until flash programming can start when the beginning of TransferData request is received, and 3 bytes (259-256) will be queued and reassembled with the following Consecutive Frames until at least 256 new bytes are received. If the length of the data to be programmed is such that at the end, a group of 256 bytes can not be made, the reaining bytes, less than 256, are written anyway.

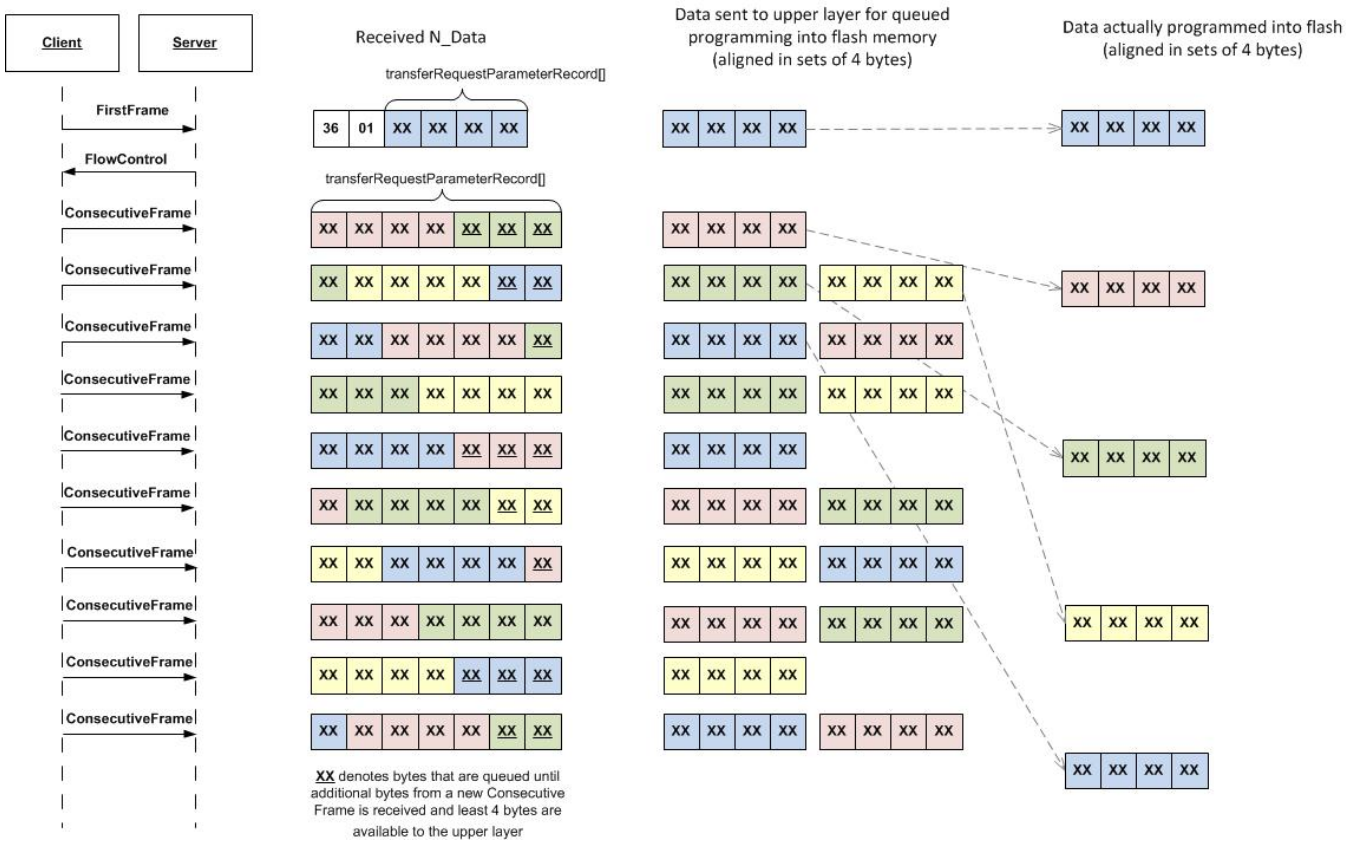
4.3.3.1.2.3.1 Example A



Example A: Alignment of flash memory is on 4 byte boundary with flash programming time which is less than the time until next Consecutive Frame. Bytes are queued and aligned until sets of 4 bytes can be forwarded to the upper layer for immediate programming. The ECU can flash memory at least at the same speed as data from Consecutive Frames are received.

Note: Example A is for Classic CAN (DLC = 8) or CAN FD where DLCs above 8 are not used. A CAN FD example with DLCs above 8 would still require that bytes are queued and aligned for the last fraction of the 4 bytes set in every Consecutive Frame. However, more sets of 4 bytes can immediately be programmed for every Consecutive Frame due to that the DLC is above 8.

4.3.3.1.2.3.2 Example B



Example B: Alignment of flash memory is on 4 byte boundary with flash programming time which is more than the time until next Consecutive Frame. Bytes are aligned in sets of 4 bytes and forwarded to the upper layer for queued programming. The ECU can not flash memory at the same speed as data from Consecutive Frames are received.

Note: Example B is for Classic CAN (DLC = 8) or CAN FD where DLCs above 8 are not used. A CAN FD example with DLCs above 8 would still require that bytes are queued and aligned for the last fraction of the 4 bytes set in every Consecutive Frame. More sets of 4 bytes sets in every Consecutive Frame imply that more data per Consecutive Frame will be forwarded to the upper layer for queued programming.

4.3.3.1.3 Definitions and Abbreviations

Definition	Description
Client	Software entity which uses services of a server. Normally this is a tester.
Client side	The ECU sending a diagnostic request on the specific network. The ECU may either be a gateway sending a diagnostic request from a higher level network or a tester directly connected to the network.
FC.CTS	See [DoCAN_2].
FC.OVFLW	See [DoCAN_2].
FC.Wait	See [DoCAN_2].
N_Ar	See [DoCAN_2].
N_As	See [DoCAN_2].

N_Br	See [DoCAN_2].
N_Bs	See [DoCAN_2].
N_Cr	See [DoCAN_2].
N_Cs	See [DoCAN_2].
N_SA	See [DoCAN_2].
N_TA	See [DoCAN_2].
N_WFTmax	See [DoCAN_2]
Server	Software entity which provides services for clients
Server side	The ECU sending a diagnostic response on the specific network. The ECU may either be a server or a gateway transferring a diagnostic response from a lower level network.
STMin	See [DoCAN_2].
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbreviation	Description
CF	See [DoCAN_2].
DLC	Data Length Code
ECU	Electronic Control Unit
FF	See [DoCAN_2].
OBD	On Board Diagnostics
TP	Transport Protocol
USDT	Unacknowledged Segmented Data Transfer

4.3.3.1.4 Introduction

This document describes specific adaptations for the CAN transport protocol. This document specifies the OSI network layer 3 and transport layer 4 requirements for the diagnostic software component.

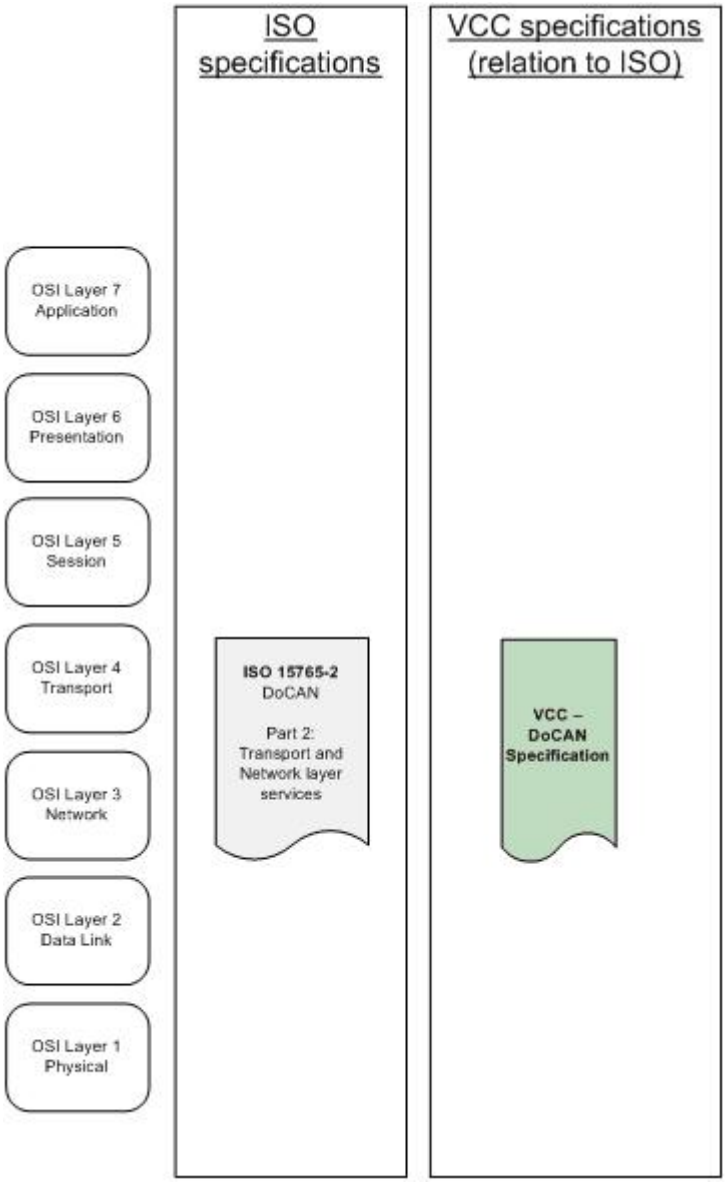


Figure – Document structure

Note
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

4.3.3.1.5 Requisite documents

Reference	Source	Title
[DoCAN_2]	ISO	ISO 15765-2:2016 Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services
[DoCAN_3]	ISO	ISO 11898-1:2015 Road vehicles - Controller area network

		(CAN) - Part 1: Data link layer and physical signalling

Information obtaining ISO documents may be found on the internet: www.iso.org

4.3.4 Guideline for Safety Related Communication

4.3.4.1 Guideline for Safety Related Communication

4.3.4.1.1 Appendix C - Transient Filter

A transient filter is necessary to be tolerant to a number of transient failures in the received data. The filter will also make it possible to require a number of correct messages before the received data is regarded as correct. Such a transient filter can be implemented with a counter. If errors are detected in the received data, the counter is incremented and if correct data is received the counter is decremented.
The counter has programmable counting factors so that the time until a transient fault is regarded permanent and the time from when the error disappears until it is regarded fault free can be individually adjustable. At a certain level of the error counter the error is regarded permanent and failure management actions are taken. The counting stops at a certain level in order to limit recovery time. When the counter has returned to zero it is assumed that the fault has disappeared. The different limits shall be possible to calibrate from zero to 255. The counting factors shall be possible to calibrate from 1 to 10. This principle is described in **Figure: Principle for the transient filter, "permanent error"** for permanent errors and in **Figure: Principle for the transient filter, "intermittent error"** for intermittent errors

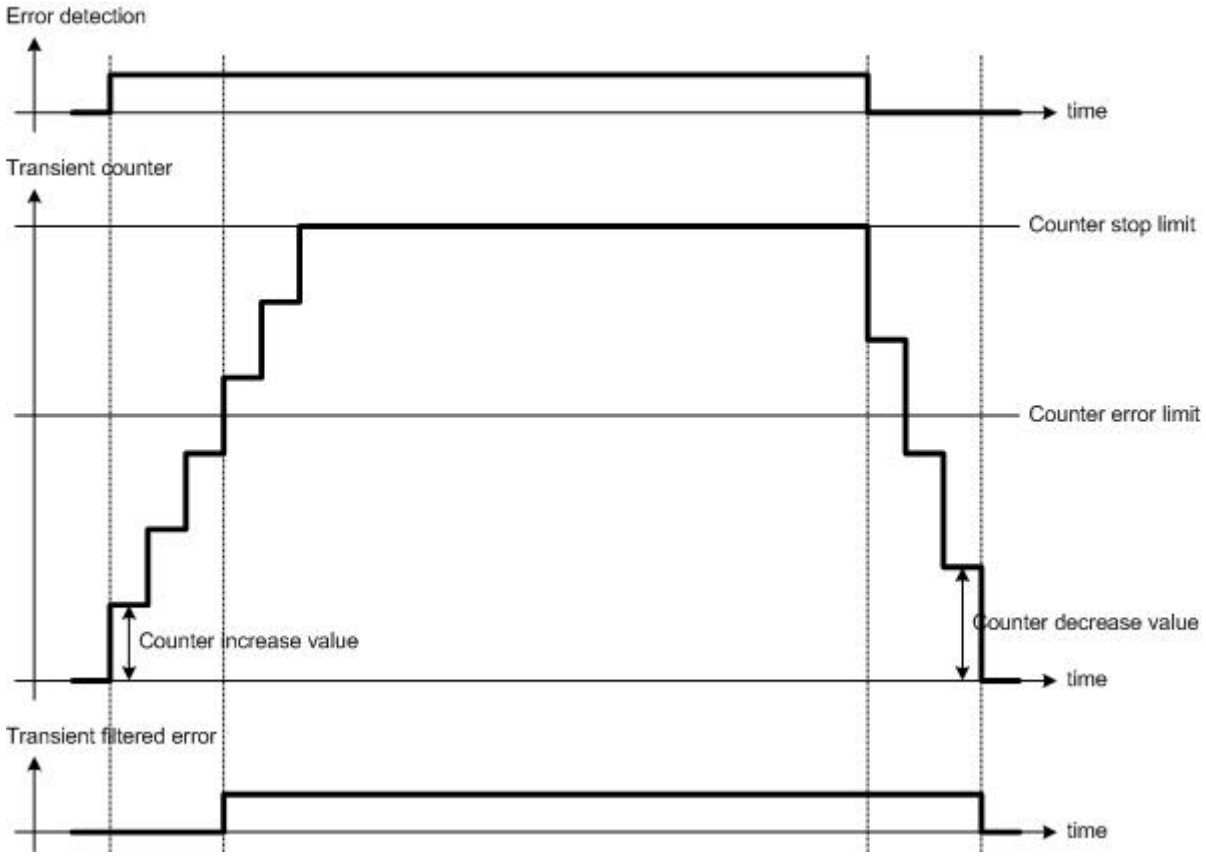


Figure: Principle for the transient filter, "permanent error".

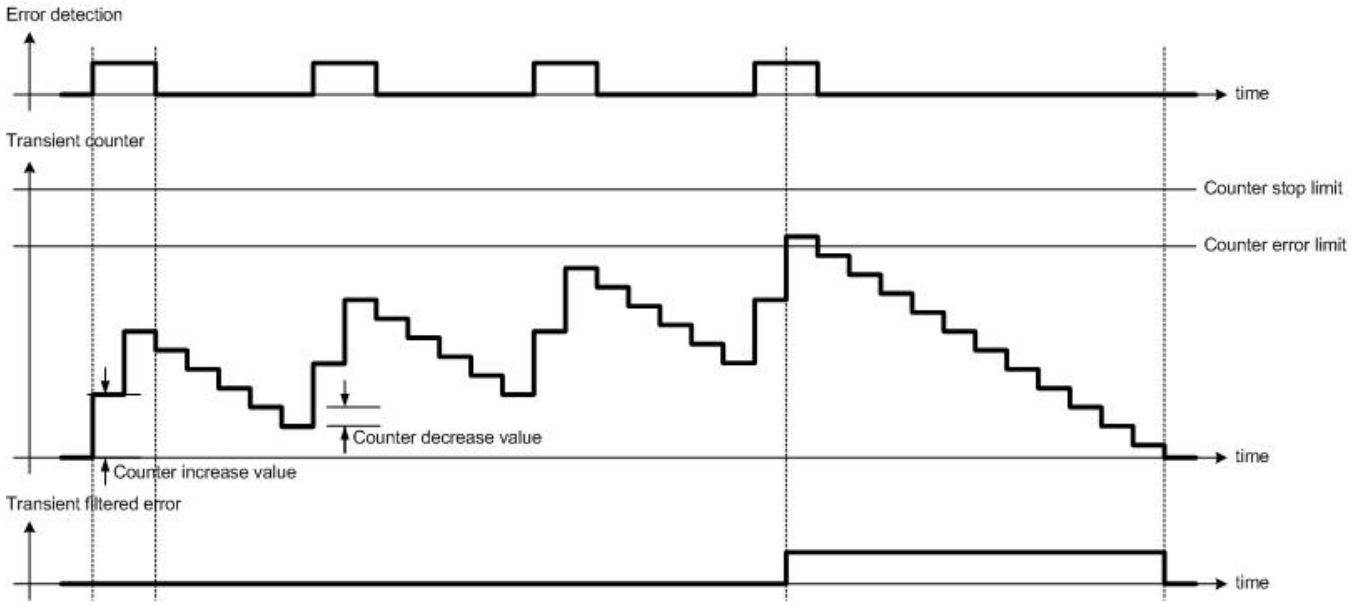


Figure: Principle for the transient filter, "intermittent error".

NOTE: When an incorrect signal is received, a replacement value for the Data usually has to be used temporarily by the application (e.g. the last received correct Data is used until a correct signal is received).

4.3.4.1.2 Appendix B - Examples of CAN Signals Reception

The principle to receive a signal in an asynchronous system is shown in **Figure: Reception of a signal in an asynchronous system**.

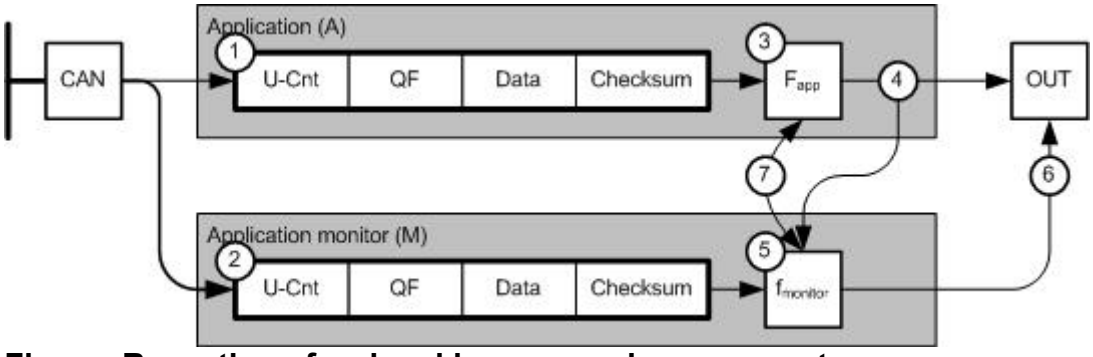


Figure: Reception of a signal in an asynchronous system

Explanations to the numbers in the **Figure: Reception of a signal in an asynchronous system** and description of the execution flow:

1. The signal is received through the CAN interface. The message is transferred unprocessed to the application (A) SW.
2. The same unprocessed message as in 1 is also supplied to the monitor (M) SW.
3. The signal is used as input to (A). This SW has to include the different error detection mechanisms, and also transient filters on the error reactions.
4. Application (A) will generate some kind of function output, which will be used both to control the actual output (e.g. digital out, CAN-message) and as input to (M).
5. (M) will use the signal and the application function output and apply some application

specific plausibility check(s). Those checks are described in the SWRS for the ECU.

6. In case (M) detects error in the plausibility check it will be able to inhibit the output from (A). The specific mechanism is also application dependant, but shall ensure that a faulty application SW should not be able to cause a hazardous situation. The mechanism is described in the SWRS for the ECU.
7. In order to fulfill the requirements (A) and (M) must perform mutual monitoring of the other's error detection capabilities, i.e. (M) must check that (A) detects error(s) correctly and (A) must check that (M) detects error(s) correctly.

4.3.4.1.3 Appendix D - Handling of Difference in Generation and Consumption Rates

Depending upon the practical situation of the relationship between the signal generation frequency and the consumption frequency, we get three (3) different cases:

1. The consumption of the signal is done at a rate which is at least the rate of the generation. This is the ideal case and will allow the receiver to check that all values of the U-Cnt are monotonically increasing (disregarding the U-Cnt wrap-around) and thus provide the strongest error checking capability.
2. The consumption of the signal is done at the same rate as the rate of the generation. In this case the receiving ECU has to take into account the jitter between the producing ECU and itself. The value of U-Cnt may not be updated when expected (no new signal has been received) as well as U-Cnt value might step over one value (one signal has been missed). This is no optimal case, and this has to be taken into account in the receiving node (e.g. in the calibration of the transient filters or in special SW).
3. The consumption of the signal is done at a rate which is slower than the rate of the generation. In this case the receiving ECU has to take into account not only the jitter between the producing ECU and itself, but also the higher production rate. Individual signal will for sure be missed, and U-Cnt values might step over several values. This has to be taken into account in similar ways as in (2) above.

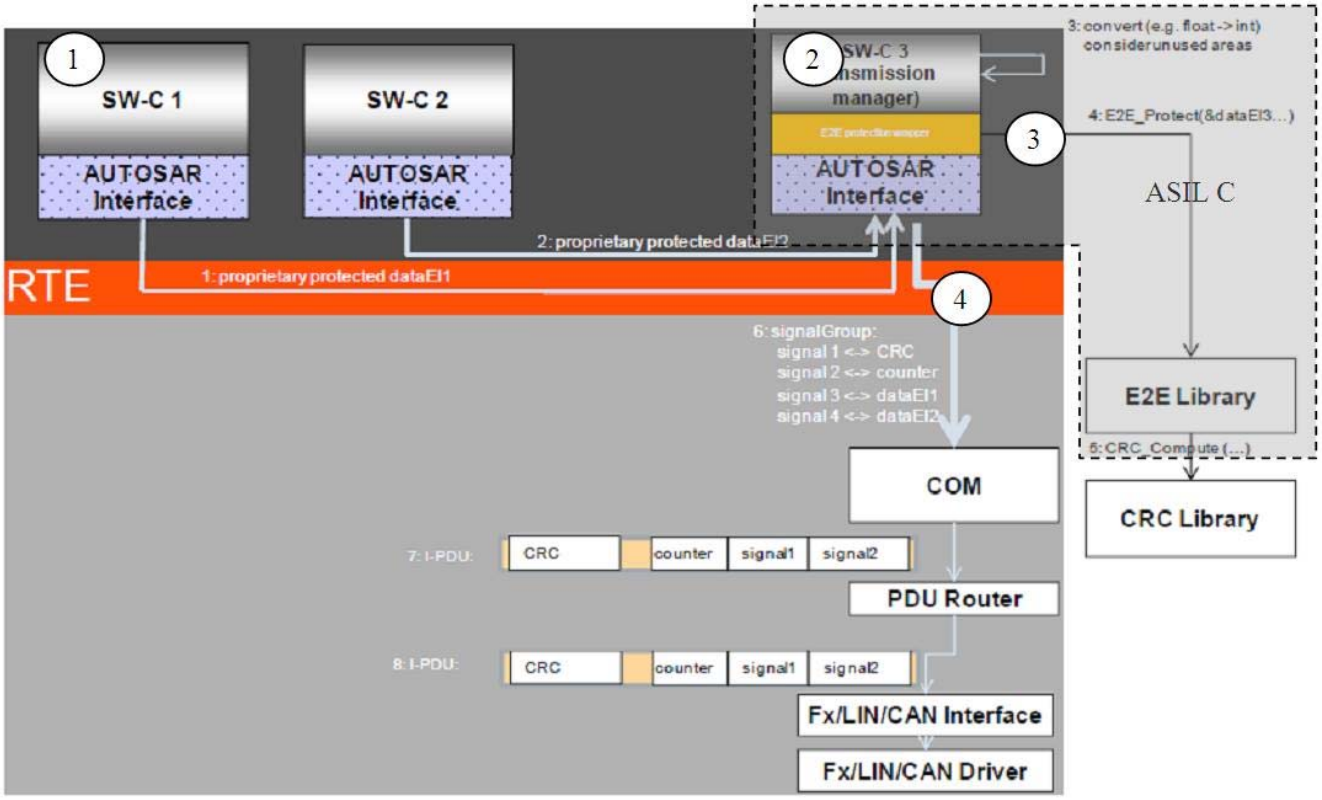
The following algorithm can be used to handle the situation described in (2) above:

```
u_cnt_sum += u_cnt - u_cnt_old - 1;

u_cnt_old = u_cnt;

if ((u_cnt_sum > 1) || (u_cnt_sum < -1)) {
  fault_cnt = min(FC_MAXLIMIT, Fault_cnt + FC_INCR);
  u_cnt_sum = 0;
} else if (u_cnt_sum == 0) {
  fault_cnt = max(0,Fault_cnt-FC_DECR);
}
if (fault_cnt >= FC_FAULTLIMIT) {
  error = true;
} else if (fault_cnt == 0) {
  error = false;
}
```

4.3.4.1.4 Appendix F - Example of Transmission Manager on Sender ECU



Scenario:

1. Data is calculated by SW-C 1 and transferred as dataEl1 to SW-C 3 (ASIL C transmission manager).
2. SW-C 3 performs a sanity check of ASIL C properties of dataEl1.
3. If sanity check shows that ASIL C properties are fulfilled, E2E library is called with dataEl1 as parameter (E2E_Protect(&dataEl1…))
3. If sanity check shows that ASIL C properties are not fulfilled, E2E library is called with safe state of dataEl1 as parameter (E2E_Protect(&safetstatedataEl1…))
4. Signal is transferred to RTE as follows: signal 1 = CRC from E2E, signal 2=counter from E2E, signal 3=dataEl1.

SW-C 3 and E2E lib is implemented in ASIL C part of software. Freedom of interference between partitions SW-C 1 and SW-C 3 according to ISO26262 has to be fulfilled.

Note: signal3 is equal to dataEl1 even if sanity check in SW-C 3 detects that ASIL C properties are not fulfilled. This is due to that SW-C 1 shall always set dataEl1 to safestate in case ASIL C properties are not fulfilled.
This means that CRC will be correct as long as SW-C 1 is calculating data correct with respect to ASIL C properties. If SW-C 1 does not calculate ASIL C properties correctly CRC will be corrupt.
Note: SW-C 2 and dataEl2 is not used in this example.

4.3.4.1.5 Appendix E - Proposal for Distributed Run-Time Fault Injection

Design details regarding the run-time fault injection can vary from implementations but one approach could include the following steps (assuming the transmitting and receiving node are both asynchronous double CPU systems):

Transmitting node
1 time out of N, the application SW should generate an erroneous signal that intentionally triggers the monitor to inhibit the sending of this particular signal. (This will typically be twice per minute (ap-proximately 100 times per hour) to minimize the risk for undiscovered errors and compensate for the fact that the transmitting and receiving nodes are not synchronized.)
In the asynchronous system this will include the switching between Application generating Data error (in which case the signal shall be blocked by a correctly working monitor and resulting in that the receiving node will detect a counter error) and the generation of incorrect checksum values (in which case a correctly working receiving node shall detect the checksum error).

Receiving node
The error reaction (inhibit of application output by the monitor) should be checked at least once every driving cycle (during initialization or shut-down, if not possible during normal operation). Based upon the knowledge that the transmitting node will generate erroneous signals 1 times out of N, an additional error detection mechanism should take this into account. The failure to detect any errors for a certain time (matched against the figure N) is also to be treated as an error.

NOTE: By having the transmitting node performing the run-time fault injection the total need for implementation of run-time fault injection routines will be kept to a minimum. Furthermore the necessity for robustness against transient errors in the receiving ECUs will be visible early in the development phase.

4.3.4.1.6 Appendix A - Examples of Monitored Signal Generation

It is assumed that a signal is produced by an ECU with two independent HW components (e.g. CPUs) sharing one CAN interface. The SW generating the signal can be arranged logically in four (4) different com-positions. Those are illustrated in **Figure: The four (4) different logical compositions for the SW generating signals** and described according to the following (note that the descriptions are simplified, and some details have been left out):

1. The application (A) is responsible for generation of the signal contents, and for the sending of the signal on the bus. The application monitor (M) is responsible for checking that the Data part of the signal is ok, otherwise (M) shall prevent (A) from sending this signal on the bus.
2. The application (A) is responsible for generation of the signal contents. The application monitor (M) is responsible for checking that the Data part of the signal is ok, and for the sending of the signal on the bus. If (M) detects an error in the Data part of the signal, the sending of this signal shall be blocked.
3. The same case as in (1) above, but with a monitor (m) that monitors the application monitor (M). By having (m) performing run-time fault injection into (M), the correct operation of (M) is ensured. This lay-out is necessary when (A) and (M) runs within the same CPU without HW redundancy.
4. Similar to case (3) above. The difference is that the application monitor (M) is

responsible for the sending of the signal on the CAN-bus.

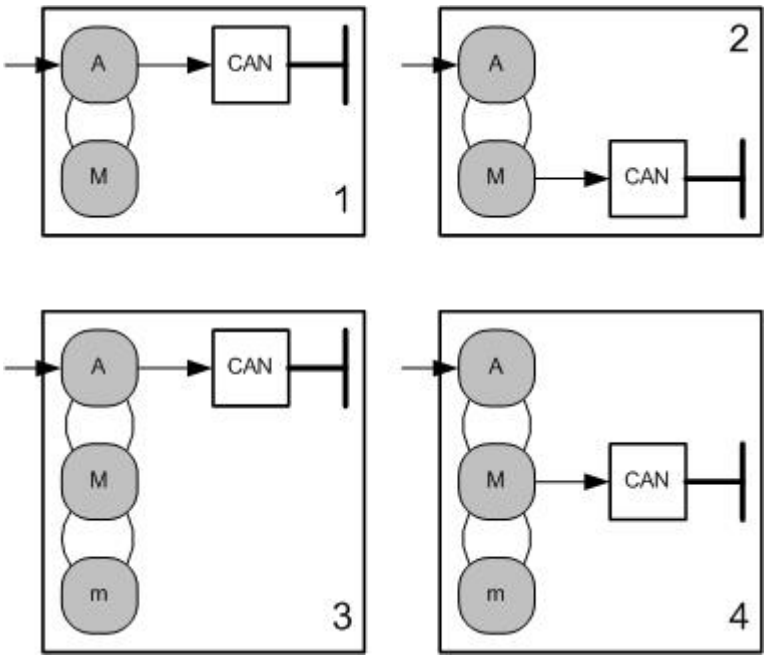


Figure: The four (4) different logical compositions for the SW generating signals.

Those four logical compositions can be mapped into different physical ECU realizations. In **Figure: Four (4) different physical realizations of the SW generating signals** we have identified four (4) different combinations, described according to the following:

- A. Synchronous system where two identical CPUs synchronized in time execute identical SW. HW monitoring is performed on a clock-cycle basis, and ensures the integrity of the generate signal.
- The logical (A) and (M) composition is according to case (1) above.
- B. Synchronous system where one main CPU is executing both the application (A) and the application monitor (M) SW. Dedicated HW (typically an ASIC) houses the monitor (m) monitoring the application monitor (M) by performing run-time fault injection.
- The logical (A), (M) and (m) composition is according to case (3) above.
- C. Asynchronous system where the application (A) is executed on the "main" CPU and the application monitor (M) SW is executed on the "secondary" CPU. The collaboration of (A) and (M) together ensures the integrity of the signal.
- The logical (A) and (M) composition is according to case (1) above.
- D. Similar to case (C) above. The difference is that the logical (A) and (M) composition is according to case (2) above.

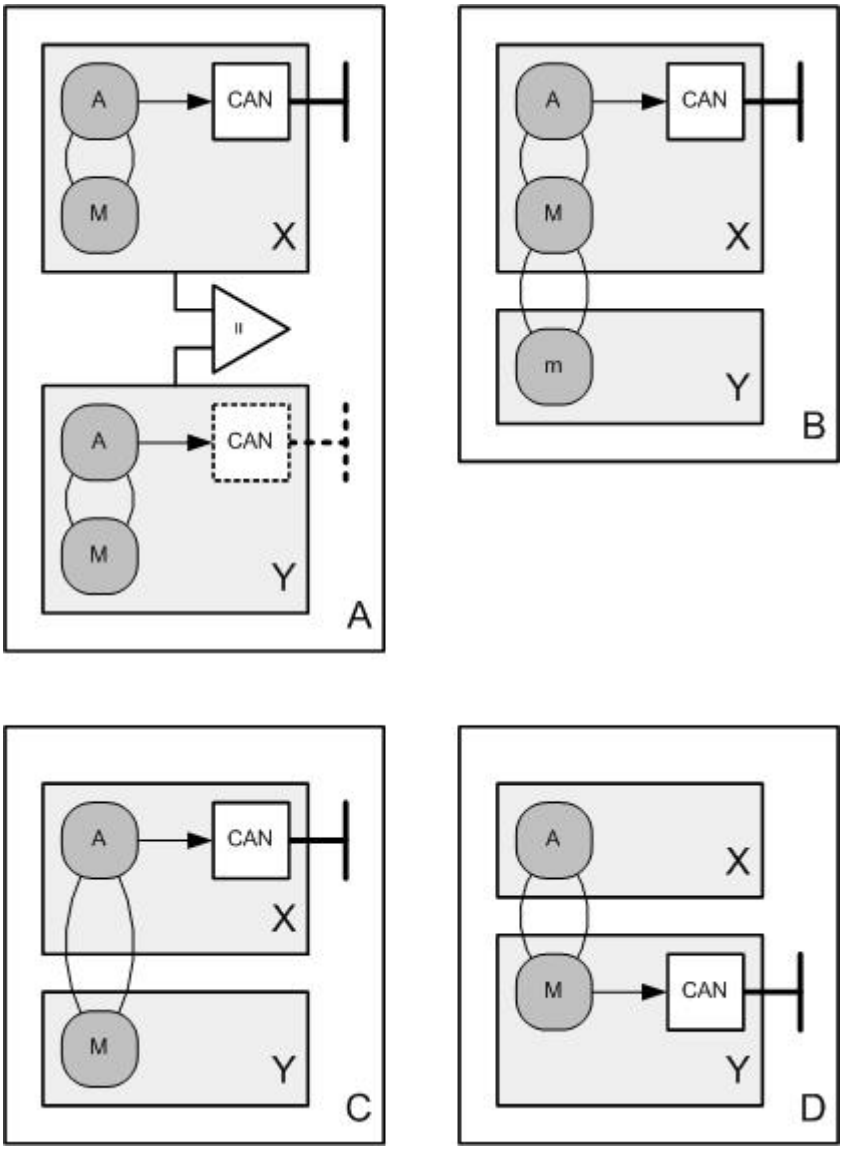


Figure: Four (4) different physical realizations of the SW generating signals.

Note: This Appendix gives examples of possible solutions for appropriately generating signals. Other solutions may exist that are not covered in this Appendix.

4.3.4.1.6.1 Case (A) - System With Two Identical CPUs and SW

The principle to generate a signal in a synchronous system with two identical CPUs as in case (A) is shown in **Figure: Construction of a signal in a synchronous system as in case (A)**.

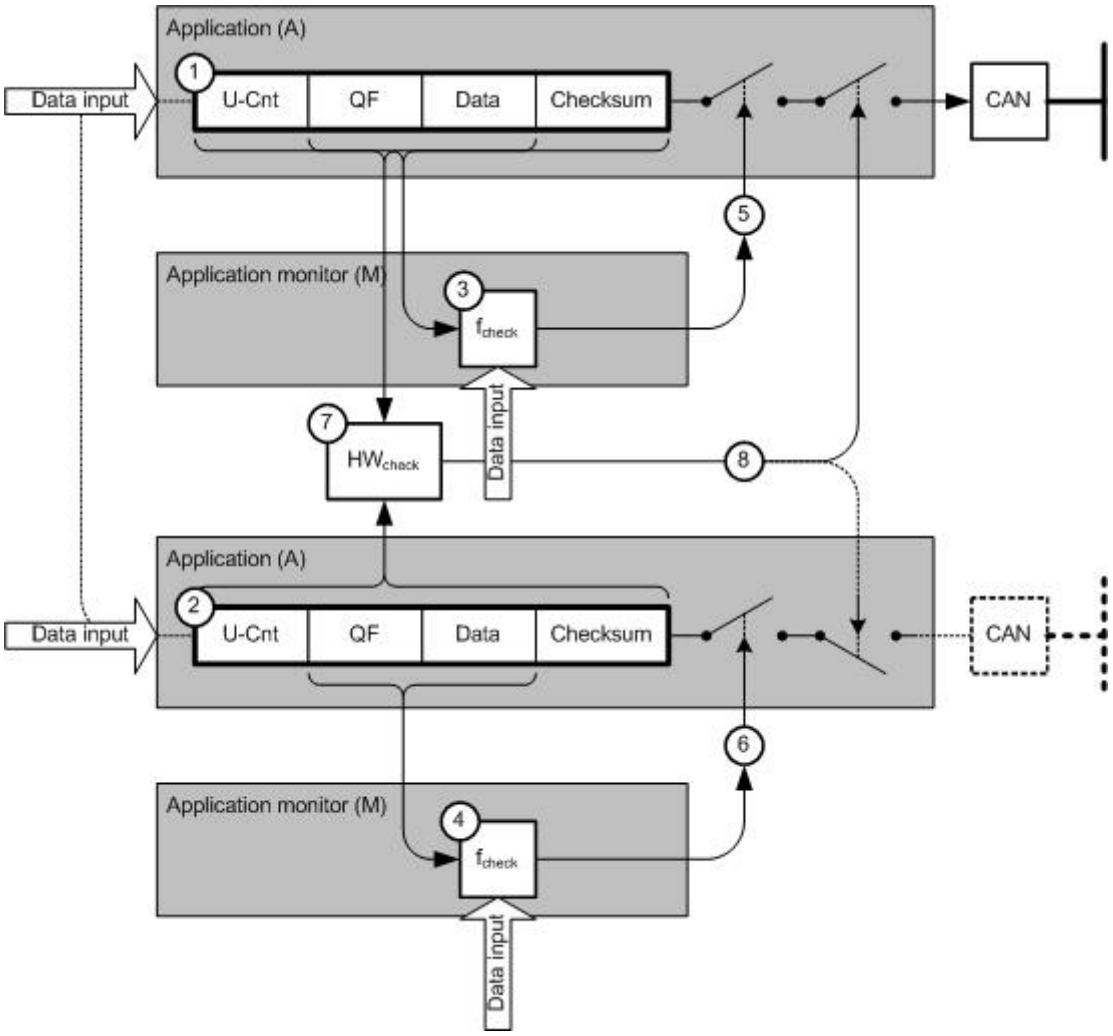


Figure: Construction of a signal in a synchronous system as in case (A)

Explanations to the numbers in **Figure: Construction of a signal in a synchronous system as in case (A)** and description of the execution flow:

1. Data input (e.g. from a sensor) is read in by the application (A) SW in the CPU1. The corresponding U-Cnt, Q-factor and Checksum are constructed within the same critical section.
2. Simultaneously to 1, the CPU2 executes exactly the same SW and constructs an identical message.
3. The message generated by (A) is checked for plausibility by an application monitor (M) SW. This check depends upon the application, but could e.g. be based upon a 2nd data input.
4. Simultaneously to (3), the CPU2 executes exactly the same SW checking its message.
5. If the check is successful the signal will be allowed to be sent out on the CAN-bus, otherwise (M) will block the sending of the signal.
6. Simultaneously to (5), the CPU2 executes the passing of the signal. (For practical reasons, the HW of the CPU2's CAN-controller might be detached from the CAN-bus.)
7. An external check mechanism checks that the two CPUs constructs the same message, and in case of a mismatch the sending of the message is blocked. (The details of the check functionality are de-scribed in the SWRS for the ECU, and are not within the scope of this specification.)

8. If the check of the CPU1 and CPU2 is successful, the signal will be sent out on the CAN-bus, otherwise the external check mechanism will block the sending.

NOTE: It is still required to have a fault tolerant check that the created data value is correct. This is in order to cover design errors in the SW (see points 3 and 4 above).

4.3.4.1.6.2 Case (B) - System With Two Different CPU and-or SW, Alternative 1

The principle to generate a signal in a synchronous system as in case (B) is shown in **Figure: Construction of a signal in a synchronous system as in case (B), alternative 1**.

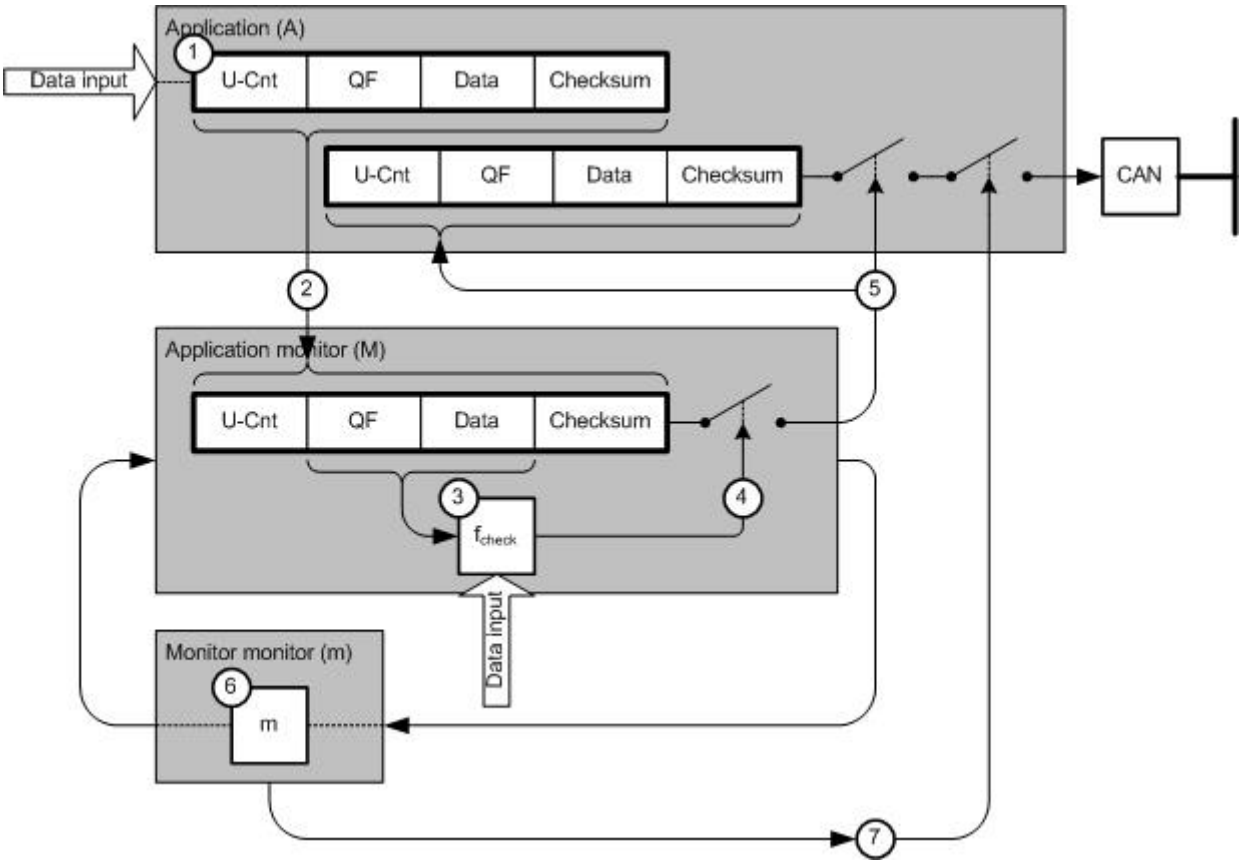


Figure: Construction of a signal in a synchronous system as in case (B), alternative 1.

Explanations to the numbers in the **Figure: Construction of a signal in a synchronous system as in case (B), alternative 1** and description of the execution flow:

1. Data input is read in by the application (A). The corresponding U-Cnt, Q-factor and Checksum are constructed close in time to this event.
2. The message is transferred to the application monitor (M).
3. The message is checked for plausibility in (M). This check depends upon the application, but could e.g. be based upon a 2nd sensor input.
4. The message is re-transferred back to (A), but not when (M) detects an error. I.e. (M) will not send back a message which it has detected as erroneous.
5. Application (A) will send the message out on the CAN-bus. This will only be the case when it has actually received a message from (M), and it is vital to understand that the message that is put for sending on the CAN-bus is not the same message as in the beginning of the flow. The SW performing this sending should be as "simple" as possible and just gateway the message received from (M) out on the CAN-bus without any

modification of the values.

6. Continuously the monitor (m) for the application monitor (M) is running, checking (M) for run-time faults by using e.g. run-time fault injection.
7. If the check of (M) is not successful, (m) shall block the sending of the message.

4.3.4.1.6.3 Case (B) - System with Two Different CPU and-or SW, alternative 2

Another alternative to the solution described above in alternative 1, also matching the case (B) (two different CPU and /or SW) could be;

As in alternative 1, there is a CPU which is regarded the main processing unit in the system, and another HW component (e.g. CPU or ASIC) which is checking the main CPU for faults in run-time. This alternative could be described as in **Figure: Construction of a signal in a synchronous system as in case (B), alternative 2**.

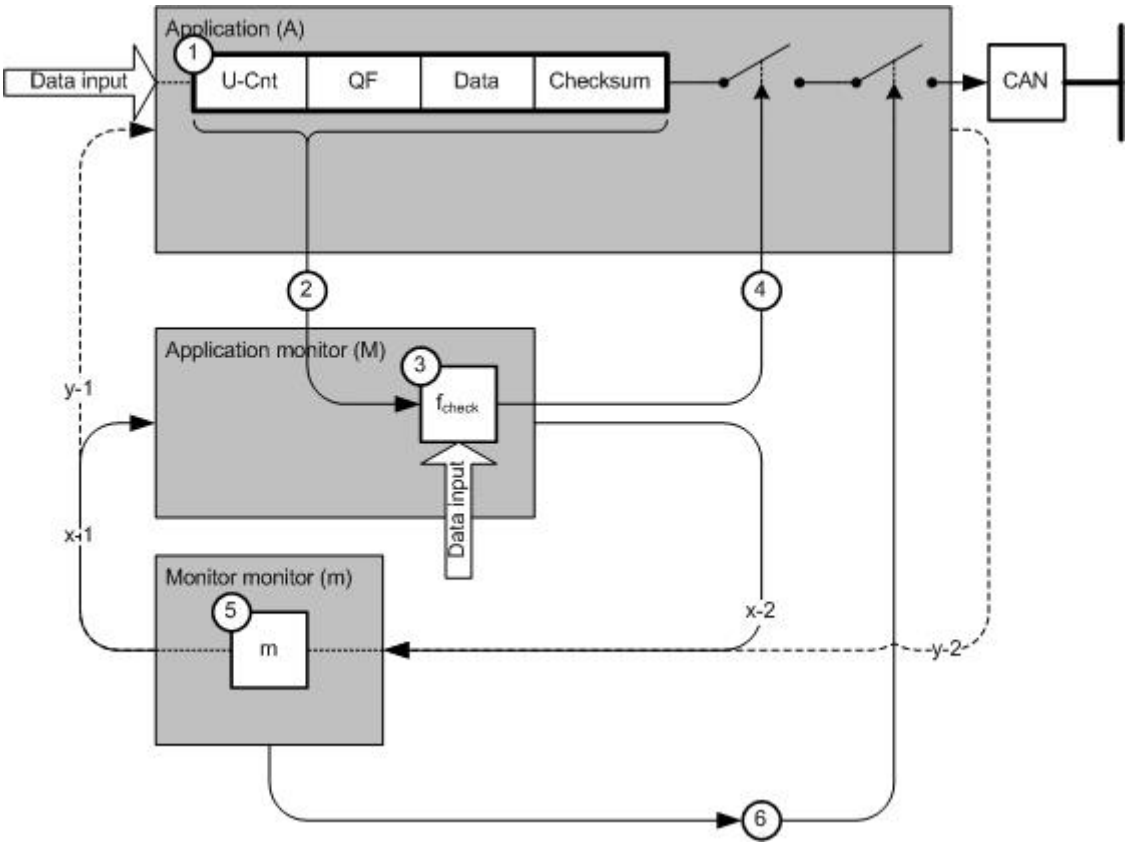


Figure: Construction of a signal in a synchronous system as in case (B), alternative 2.

Explanations to the numbers in the Figure 5 and description of the execution flow:

1. Data input is read in by the application (A). The corresponding U-Cnt, Q-factor and Checksum are constructed close in time to this event.
2. The message is transferred to the application monitor (M).
3. The message is checked for plausibility in (M). This check depends upon the application, but could e.g. be based upon a 2nd sensor input.
4. The application monitor has the possibility to interrupt the sending of the message, if detected as incorrect.
5. The monitor (m) for the application monitor (M) is running continuously, checking (M) for

run-time faults by using e.g. run-time fault injection (x-1 and x-2). Alternatively to this, the (m) could also be used to check the entire application code with e.g. strong memory checks in run-time. In this case (y-1 and y-2) it will be immediately detected if either application (A) or application monitor (M) is altered.

6. If the check performed by (m) is not successful, (m) shall block the sending of the message.

4.3.4.1.6.4 Case (C), asynchronous system, alternative 1

The principle to generate a signal in an asynchronous system as in case (C) is shown in **Figure: Construction of a signal in asynchronous system (alternative 1).**

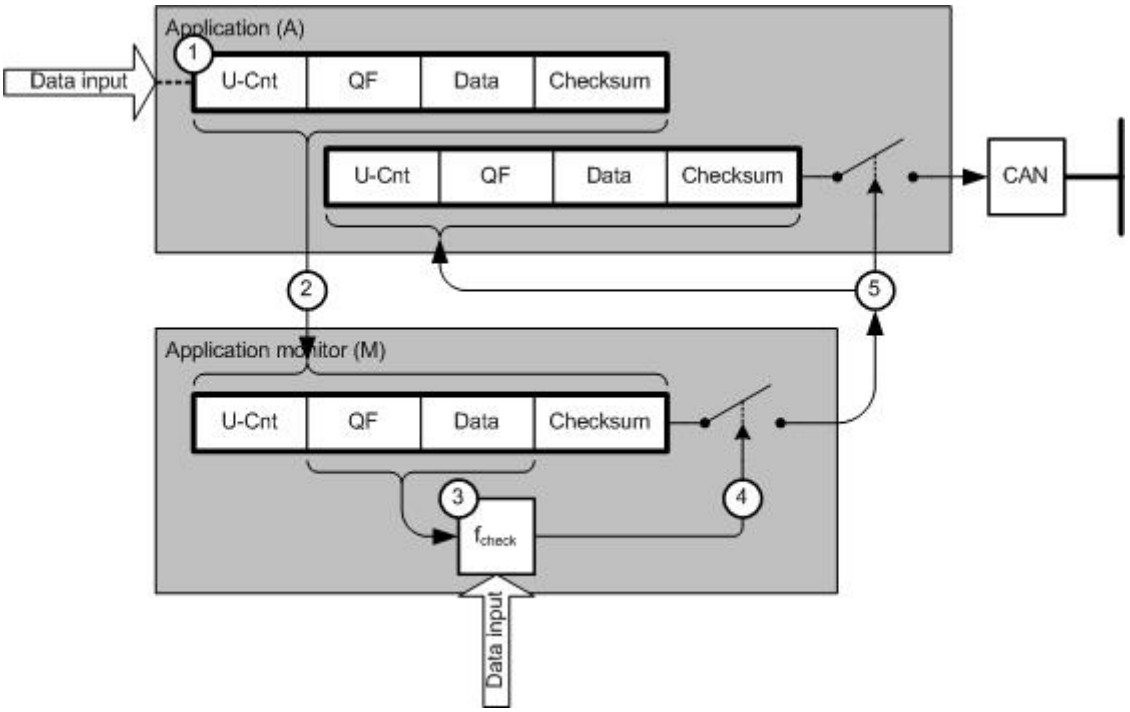


Figure: Construction of a signal in asynchronous system (alternative 1).

Explanations to the numbers in **Figure: Construction of a signal in asynchronous system (alternative 1).** and description of the execution flow:

1. Data input is read in by the application (A). The corresponding U-Cnt, Q-factor and Checksum are constructed close in time to this event.
2. The message is transferred to the application monitor (M).
3. The message is checked for plausibility in (M). This check depends upon the application, but could e.g. be based upon a 2nd sensor input.
4. The message is re-transferred back to (A), but not when (M) detects an error. I.e. (M) will not send back a message which it has detected as erroneous.
5. Application (A) will send the message out on the CAN-bus. This will only be the case when it has actually received a message from (M), and it is vital to understand that the message that is put for sending on the CAN-bus is not the same message as in the beginning of the flow. The SW performing this sending should be as "simple" as possible and just gateway the message received from (M) out on the CAN-bus with-out any modification of the values.

4.3.4.1.6.5 Case (D), asynchronous system, alternative 2

The principle to generate a signal in an asynchronous system as in case (D) is shown in **Figure: Construction of a signal in asynchronous system as in case (D).**

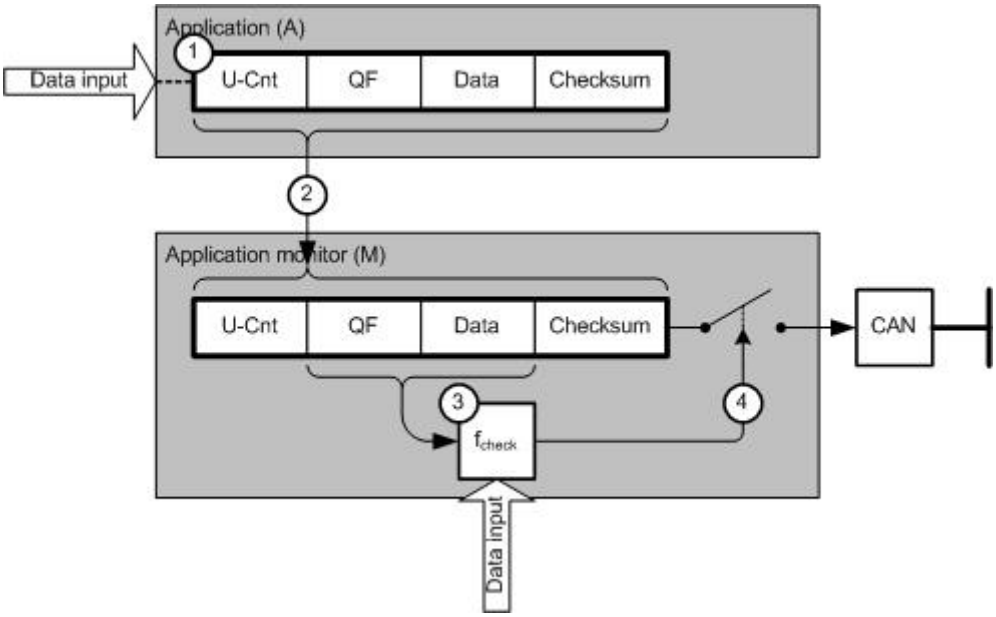


Figure: Construction of a signal in asynchronous system as in case (D).

Explanations to the numbers in the **Figure: Construction of a signal in asynchronous system as in case (D).** and description of the execution flow:

1. Data input is read in by the application (A). The corresponding U-Cnt, Q-factor and Checksum are constructed close in time to this event.
2. The message is transferred to the application monitor (M).
3. The message is checked for plausibility in (M). This check depends upon the application, but could e.g. be based upon a 2nd data input.
4. If the check is successful the signal will be sent out on the CAN-bus, otherwise (M) will block the sending of the signal.

4.3.4.1.7 Definitions and Abbreviations

Definition	Description
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software
COM	An AUTOSAR basic software handling a part of the communication
CRC	Cyclic Redundancy Check
DC	Diagnostic Coverage
E2E	End to End
ECU	Electronic Control Unit
LC	Logical Component
SSE	System Safety Engineer
SW-C	Software Component
TSC	Technical Safety Concept
QF or Q-Factor	Quality Factor

U-Cnt	A 4-bit Update Counter (Counter fulfils same purpose)

4.3.4.1.8 Introduction

4.3.4.1.8.1 What is the purpose of this document

This guideline is aimed at supporting the development of Technical Safety Concepts with respect to system signaling over a communication bus. The guideline consists of two parts. The first part is a guide to be used when selecting suitable safety mechanisms depending on failure modes to be handled and their ASIL. The second part includes a list of suitable safety mechanisms for system signaling with the related failure modes.
The appendix in the document provides a set of safety mechanisms, patterns and examples that relates to different aspects of distributed functionality and safety related communication.

4.3.4.1.8.2 Motivation

This guideline exists to provide guidance and harmonize the use of safety mechanisms in network communication. The guideline is also a part of ISO-26262 compliance for functions, systems, and components.

4.3.4.1.8.3 Who should read this document

This document should be read by function developers, system developers, component developers, test engineers, and System Safety Engineers (SSE) that are involved in the development of safety related functions that use safety related network signaling.

4.3.4.1.8.4 How do you receive more information

Additional details to this guideline are found in the referenced documents or from the CEVT System Safety competence centre.

4.3.4.1.8.5 Requisite Documents

[SafeCom_1] AUTOSAR Specification of SW-C End-to-End Communication Protection Library
[SafeCom_2] AUTOSAR Specification of CRC Routines
[SafeCom_3] Road Vehicles – Functional Safety ISO/FDIS 26262:2011
[SafeCom_5] Guideline for quality factor signals (part of Base Tech SWRS)
[SafeCom_6] Numerical Recipes: The Art of Scientific Computing (3rd ed.). New York: Cambridge University Press. ISBN 978-0-521-88068-8]

For [SafeCom_1] and [SafeCom_2] please refer to the specification revisions that corresponds to the general AUTOSAR Revision in use.

4.3.4.1.9 How To Use This Guideline

This section explains the background to this guideline and its relevance for developing a Technical Safety Concept (TSC).
Note that the safety mechanisms are specified in an ECU platform perspective but they are still aimed to be on system level design when specifying the signal characteristics. Further, the listed safety mechanisms shall not be seen as the only valid mechanisms to use.

This guideline includes AUTOSAR safety mechanisms and relates to ASIL as specified by the ISO 26262.
ISO 26262 [SafeCom_3] requires fault handling to be implemented depending on the ASIL, even if the fault handling mechanisms for different ASILs are not specified. The appropriate safety mechanisms for fault handling to avoid the violation of Safety Goals shall be selected

based on engineering judgment.
In AUTOSAR 4.0 there are certain safety mechanisms provided, like the E2E [SafeCom_1] and CRC lib [SafeCom_2], and they are recommended to be used whenever appropriate.

4.3.4.1.9.1 Limitations of this Guideline

It should be noted that following this guideline is not sufficient to achieve a safe communication according to ISO 26262. It shall be demonstrated that the selected safety mechanisms provide sufficient fault handling capabilities for the considered network.
The recommendations in this guideline are based on the recommendations in [SafeCom_1] for communication using E2E that are developed by safety experts within AUTOSAR for up to ASIL D communication. These recommendations shall be confirmed to be appropriate in the Item, fulfilling the Item's own safety requirements and any external safety requirements. These safety requirements include ISO 26262 requirements on hardware architectural metrics and requirements for safety goal violation rates.
It should be noted that the recommended safety mechanisms in this guideline are safety mechanisms in addition to the safety mechanisms already implemented in the communication bus used. These additional safety mechanisms are intended to protect against faults occurring in the communication stack and also to enhance protection against faults occurring on the communication bus itself. Depending on application and the ASIL of the relevant failure modes, the safety mechanisms in the communication bus may be sufficient.

4.3.4.1.9.2 Relevance of This Guideline When Developing a Technical Safety Concept

In order to achieve safe communication, the complete communication chain between the sender and related receivers shall be considered, including the communication bus and COM BSW. This guideline provides an initial recommendation for suitable fault handling mechanisms to protect safety related communication according to ISO 26262. The chosen fault handling mechanisms shall be documented in the Technical Safety Concept for the Item under development.

4.3.4.1.10 Communication Related Safety Mechanisms

In this chapter communication related safety mechanisms are grouped in a bottom-up perspective. These mechanisms are only a selection of possible mechanisms to use for achieving safe communication.

4.3.4.1.10.1 Lower Level Network Protocol Safety Mechanisms

In the used network protocols CAN, LIN, and Flexray, there are safety mechanisms incorporated that need to be considered when deciding what safety mechanisms to use in the AUTOSAR platform and in the SW-Cs. Note that these safety mechanisms do not cover faults introduced in the communication layers above the protocol layer.
The information here is only for an overview. When deciding on mechanisms like CRC, the complete path of the communication needs to be handled. The CRC on the communication bus may not be sufficient. The next subchapters give an overview of the mechanisms for each protocol.

4.3.4.1.10.1.1 CAN

The -protocol safety related mechanisms included in CAN are:

- 15 bits CRC covering 0-8 data bytes
- Every sent bit is checked for correctness by sender (read back)
- A frame is deleted by all receivers if any receiver detects an error

4.3.4.1.10.1.2 LIN

The in-protocol safety related mechanism for LIN is:

- 8-bits checksum covering 0-8 data bytes

4.3.4.1.10.1.3 Flexray

The in-protocol safety related mechanisms for Flexray are:

- 11-bits CRC bit covering header (20 bits)
- 24-bits CRC bit covering data (0-254 bytes)
- Redundancy with parallel channels (optional)

4.3.4.1.10.2 AUTOSAR Platform Communication Safety Mechanisms

The used versions of the AUTOSAR platform (Rev 4.0 and 4.2) provides two libraries that are used for enhancing safety. These libraries are E2E and CRC.

4.3.4.1.10.2.1 Generic Failure Modes and Detection Mechanisms in AUTOSAR

The table below shows the safety mechanisms included in the AUTOSAR CRC and E2E libraries with the failure modes that the mechanisms can detect. The list of faults could be extended if broken down further, but this level of abstraction is deemed useful as an example on what faults could be handled with the safety mechanisms provided by the AUTOSAR platform. For a more detailed description see [SafeCom_1] - Specification of SW-C End-to-End Communication Protection Library.

Mechanism	Detected failure modes
Counter	Repetition, loss, insertion, out of sequence
Timeout	Loss, delay
Data ID	Insertion, addressing faults, masquerading
CRC	Corruption

4.3.4.1.10.2.2 E2E Library

The following mechanisms are available via two profiles, called Profile 1 and Profile 2.
The mechanisms can be used separately, but DataID requires CRC for transmission and Timeout needs Counter in order to detect timeout. See AUTOSAR E2E Library [SafeCom_1] for details.

- Counter – Profile 1
- * 4bit (explicitly sent) representing numbers from 0 to 14 incremented on every send request.
- * Both Alive Counter and Sequence Counter mechanisms are provided by E2E Profile 1, evaluating the same 4 bits.
- Counter – Profile 2
- * 4bit (explicitly sent) representing numbers from 0 to 15 incremented on every send request.
- Data ID- Both profiles
- * 16 bit, unique number, included in the CRC calculation but not transmitted (implicit transmission)
- * (Note that Profile 1A implicit transmits all 16 bits, while 1B alternates on the low and high 8 bits in transmission)
- Timeout – Only Profile 1
- * Timeout is determined by E2E Library by means of evaluation of the Counter, by a non-

blocking read at the receiver.

- CRC (via CRC lib.) – Profile 1
- * using the Crc_CalculateCRC8 () function, an 8-bit calculation
- CRC (via CRC lib.) – Profile 2
- * using the Crc_CalculateCRC8H2F () function, an 8-bit calculation

4.3.4.1.10.2.3 CRC Library

This library provides CRC (Cyclic Redundancy Code).functions, which are accessible via E2E lib or directly through CRC lib. See AUTOSAR CRC Library [SafeCom_2] for details.
The CRC lib provides the following functionality:

- 8-bit CRC calculation
- * 8-bit SAE J1850 CRC Calculation
- * 8-bit 0x2F polynomial CRC Calculation
- 16-bit CRC calculation based on CCITT CRC16 standard
- 32-bit CRC calculation based IEEE-802.3 CRC32 Ethernet Standard

A rule of thumb for size of CRC to be used is given in [SafeCom_1] and [SafeCom_6] and summarized below where D is the total size of the data to be protected and that includes the data, CRC and counter:

- For CAN and Flexray where D ≤30 bytes: 8-bit CRC
- For LIN where D≤8 bytes: 8-bit CRC
- All other sizes for CAN, Flexray and LIN
- * D < 16K bytes: 16-bit CRC
- * D < 512M bytes: 32-bit CRC

4.3.4.1.10.3 Application Level Safety Mechanisms

On the application level (Logical Component level) additional mechanisms can be implemented that will enhance functional safety of the communication. On this level it is possible to do application specific solutions.

4.3.4.1.10.3.1 Quality Factor

QF is a mechanism used for showing the quality of a signal and the implementation is described in [SafeCom_5] - Guideline for quality factor signals. This concept is not designed to detect of communication faults. However, it is mentioned here because of its usage in functional safety. When used, it is necessary to agree the meaning of the different signal quality values between the signal producer and all signal consumers.
If used, it is recommended that QF is sent in the same frame as the signal and included in the same CRC to detect corruption and to provide some protection against asymmetric behavior.

4.3.4.1.10.3.2 Counter

An application can implement an a counter through a separate signal and place it in a signal group. This counter can be of lower size compared to the counter in the AUTOSAR E2E Library to save bandwidth.

4.3.4.1.10.3.3 Boolean Signals

For Boolean signals, fault tolerance can be increased by using two (2) bits for the signal. If used, coding of the Data shall then be according to the table below.

Data (binary coded)	Description
00	ILLEGAL

01	False
10	True
11	ILLEGAL

4.3.4.1.10.3.4 Oversampling

For signals that are sensitive to the failure modes delay and loss, an application could use oversampling of signals to increase the fault tolerance against these failure modes.

4.3.4.1.10.3.5 Handshake Protocol

To detect different types of communications faults at an application layer, a handshake protocol may be used. One common type of implementation is a challenge-response protocol to detect if an application is working correctly.

4.3.4.1.10.3.6 Range Check and Plausibility Check

To detect different types of communications faults, both range check and plausibility checks can be used by the receiving application where a range check is much simpler. A range check determines if the values of received data fall within an expected range, e.g. vehicle speed could be expected to be between -50 km/h and 200 km/h. A plausibility check on the other hand is more dynamic and determines if the values of received data are likely based on knowledge of the signal's behavior, e.g. it is not likely that vehicle speed change from 0 km/h to 100 km/h between two samples at 100 Hz.

4.3.4.1.10.4 System Level Safety Mechanisms

4.3.4.1.10.4.1 Redundancy (Only relevant for ASIL C and D)

The need for redundancy in signaling depends on ASIL and the robustness requirements. If needed, redundancy can be achieved in several ways. Usage of different communication buses or duplicate frames is examples on how to implement redundant communication. The selection of redundancy method is decided case by case. The system architect, responsible for Sending and Receiver should be involved in these types of considerations.

4.3.4.1.10.5 Other Mechanisms

Here are mechanisms that are implemented as complex drivers or other ways described.

4.3.4.1.10.5.1 Self-detected CPU HW failures

Run-time fault injection is a technique for improving the coverage of a test by introducing faults and see if it is discovered.
An example of this can be found in "Appendix E: PROPOSAL FOR DISTRIBUTED RUN-TIME FAULT INJECTION"

4.3.4.1.10.5.2 HW SW Independence

Two separate SW modules and two separate HW components (e.g. two CPUs with one separate SW module each, or one CPU with two separate SW modules and one ASIC) can be involved in the transmission and reception of the signal. An alternative is that an independent HW (e.g. second CPU or ASIC) is involved in monitoring the main CPUs ability to produce signal(s) correctly. In Appendix A, implementation examples shows different solutions.

4.3.4.1.10.5.3 Monitor

A monitor is a pattern that can be used to implement safety mechanisms. To specify that a monitor should be used and of what type it shall be should be done by the system designer. Selection of monitor type can be supported by first figure in "Appendix A: EXAMPLES OF

MONITORED SIGNAL GENERATION".
A component designer that has received requirements on a monitor can use the second figure in Appendix A to understand how the specified monitor type can be implemented in an ECU.
For signal reception "Appendix B: EXAMPLES OF CAN SIGNALS RECEPTION" can be used for support.
"Appendix F: Example of Transmission Manager on Sender ECU" is another example of a Monitor pattern in an AUTOSAR context.

4.3.4.1.10.5.4 Run-time Fault Injection

Run-time fault injection is a technique for improving the coverage of a test by introducing faults and see if it is discovered.
An example of this can be found in "Appendix E: PROPOSAL FOR DISTRIBUTED RUN-TIME FAULT INJECTION"

4.3.4.1.11 Method

When determining the different fault handling mechanisms to be used, there are several aspects to consider. These aspects are:

- Requirements in functional safety standard used, if used. For SPA, this is in most cases ISO26262. In particular, requirements resulting from HW development may require diagnostic coverage of different degree for faults in communication as a result of the requirements on hardware architectural metrics and safety goal violation rates.
- Requirements on communication from function and system development, including functional and technical safety concepts (FSC and TSC). These requirements may come from several different items.

The safety mechanisms are further described in Section 5.
One approach when determining which fault handling mechanisms to be used is described in Figure: Process for selecting safety mechanisms to protect against communication faults at application level used in and end to end manner. This approach includes the following steps:

1. In the Technical Safety Concept for the Item under development, specifying the safety mechanisms to protect against relevant faults that could occur during communication between applications considering the relevant ASIL. This should be done according to Table: Recommended safety mechanisms for detection of communication faults and Table: Recommended safety mechanisms for handling of communication faults.

1. This step should consider safety mechanisms already implemented in the communication bus, the communication stack, and the applications' use of the relevant signals (e.g. if receiving applications do plausibility checks).

1. Note: There may be requirements from functional and technical safety concept that should be fulfilled by safety mechanisms during communication and are appropriate to document at this step.

2. Carry out a preliminary analysis to judge if the recommended safety mechanisms are likely to fulfill the ISO 26262 requirements, including hardware architectural metrics and safety goal violation rates based on the target values of the components' failure rates.
3. Carry out an analysis to confirm that the selected safety mechanisms fulfill the ISO 26262 requirements on hardware architectural metrics and safety goal violation rates based on the actual components failure rates.

3. This step should also consider the fulfillment of other safety requirements, e.g. from ISO-26262 and the functional and technical safety concepts, on the implemented safety mechanisms.

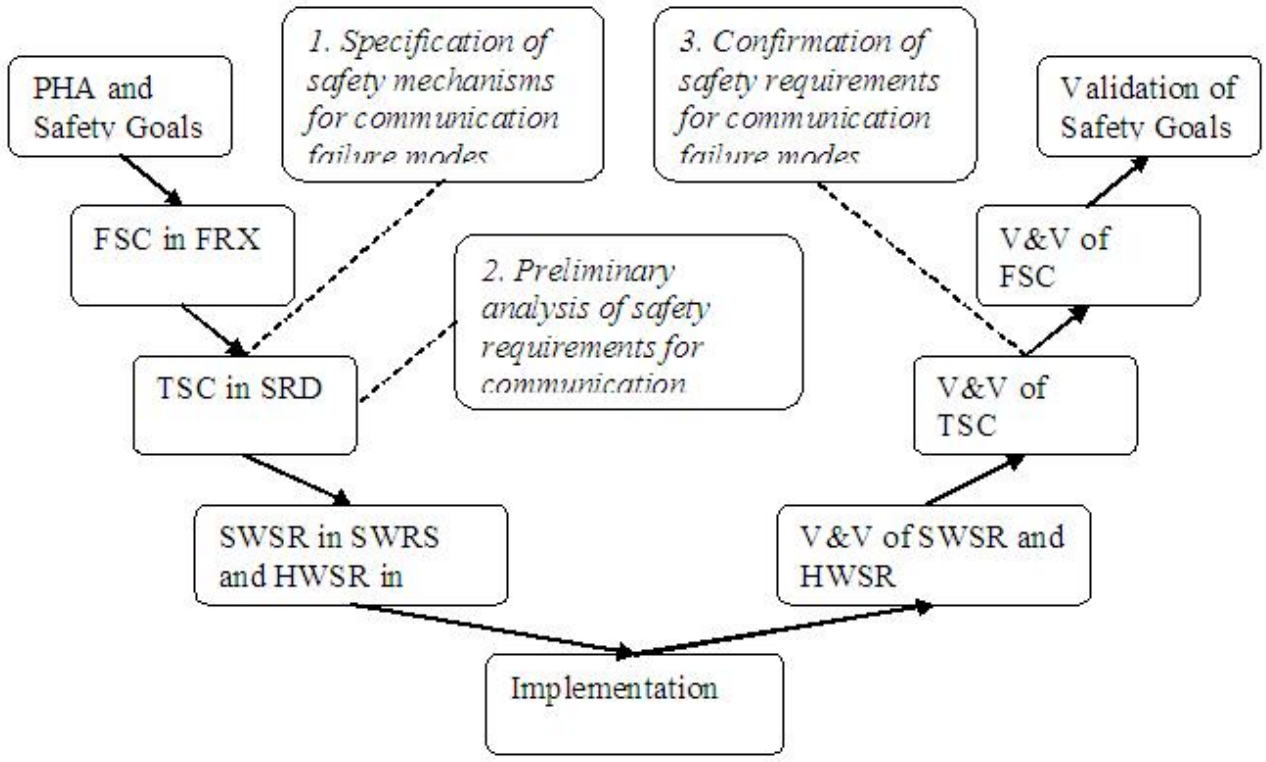


Figure: Process for selecting safety mechanisms to protect against communication faults

Table: Recommended safety mechanisms for detection of communication faults at application level used in an end-to-end manner.

Failure mode	ASIL A	ASIL B	ASIL C	ASIL D
Repetition, loss, insertion, out of sequence	Select appropriate mechanism from Section 5 [1]	Counter [2]	E2E Profile Variant 1A	E2E Profile Variant 1A
Corruption	Select appropriate mechanism from Section 5 [1, 3]	CRC [2, 3]	E2E Profile Variant 1A [3]	E2E Profile Variant 1A [3]
Addressing faults	Select appropriate mechanism from Section 5 [1]	Select appropriate mechanism from Section 5 [1]	E2E Profile Variant 1A	E2E Profile Variant 1A
Delay	Select appropriate mechanism from Section 5 [1]	Select appropriate mechanism from Section 5 [1]	E2E Profile Variant 1A	E2E Profile Variant 1A
Masquerading	Select appropriate mechanism from Section 5 [1]	Select appropriate mechanism from Section 5 [1]	E2E Profile Variant 1A	E2E Profile Variant 1A

[1] – No recommendation for or against any specific measure. Sufficient protection could be available in the communication protocol used or could be solved at application level by using mechanisms described in Section 5.3 or E2E Profile 1A.
[2] – If both Counter and CRC are deemed necessary, using E2E Profile 1A should be considered.
[3] – If using the AUTOSAR CRC then care should be taken so that the size of the data is not

too large. See chapter 5.2.3 for recommendation on this.
Note: Recommended safety mechanisms should be implemented using AUTOSAR's standard libraries.
Note: Often a signal is sensitive to more than one failure mode and hence several safety mechanisms would be needed.

Table: Recommended safety mechanisms for handling of communication faults at application level used in an end-to-end manner.

	ASIL A	ASIL B	ASIL C	ASIL D
Delay including loss	Oversampling at application level			

When designing the functional or technical safety concept of a particular system any user of E2E Library shall ensure that the transmission of one undetected erroneous data element in a sequence of data elements between sender and receiver will not directly lead to the violation of a safety goal of this system.
In other words, SW-C shall be able to handle the reception of one erroneous data element, which error was not detected by the E2E library. What is *not* required is that an SW-C tolerates two consecutive undetected erroneous data elements, because it is enough unlikely that two consecutive Data are wrong AND that for both Data the error remains undetected by the E2E library.

In addition to the safety mechanisms in the table above, the use of quality flags could be considered according to [SafeCom_5] - Guideline for quality factor signals.
It should be noted that the safety mechanisms, including the communication stack in BSW and the AUTOSAR CRC and E2E libraries, should be implemented according to a development process capable of an ASIL of at least the same level as of the supported signal as well as fulfilling safety requirements of ISO-26262.

4.4 Data Security

4.4.1 Random number generation

4.4.1.1 Random number generation

4.4.1.1.1 Requirements

1576 v2 **Entropy for random numbers**

Version history:
(2):

Purpose: Avoid predictable outcome from random number generation, by ensuring that the random number generator is seeded with sufficient entropy.
Source:
Verification Method: Analysis
Legacy ID: 419311v1

The output from the entropy source shall be tested using the tool ENT

(https://www.fourmilab.ch/random/). The entropy generated shall when tested with ent shall be:

- - Entropy: At least 7.5 bits per character or higher.
- - Compression: 0%
- - Chi-Square: Between 200 and 400 and randomly exceed more than 10% and less than 90% of the time.
- - Mean value: Between 126 and 128
- - Monte Carlo: Max 0.5% error
- - Serial correlation: 0.1 or lower

Note; recommendations can be found in NIST SP 800-90A-C or BSI AIS 20 & BSI AIS 31 https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Zertifizierung/Interpretationen/AIS_31_Functionality_classes_for_random_number_generators_e.pdf?__blob=publicationFile

and tool https://www.fourmilab.ch/random/

1577 v1 **Random number generator algorithms**

Version history:
(1):

Purpose:	Ensure that generated random numbers are suitable for use in cryptographic and other security related use cases.
Source:	NIST Special Publication 800-22 Revision 1a
Verification Method:	Analysis
Legacy ID:	434662v0
ASIL:	QM

Random numbers shall be generated by a deterministic random bit generator (DRBG), also known as a cryptographically safe pseudo random number generator. The generator shall be seeded and periodically reseeded from entropy provided by the entropy source.

The generator used shall be one of the hash based or block cipher based generators approved by NIST, see SP 800-90A. That is Hash_DRBG, HMAC_DRBG eller CTR_DRBG. The generator Dual_EC_DRBG shall not be used.

The output from the generator shall be tested with the tool Dieharder (https://www.phy.duke.edu/~rgb/General/dieharder.php). When tested, no test shall fail independent of ECU mode (like after a reboot).

4.4.1.1.2 Introduction

When a random number is needed for a cryptographic application both algorithm and seed need to be chosen with care. Examples of cryptographic application in the automotive context is:

- Challegne response protocols like security access
- TLS
- Passwords

when possible, autosar standard should be used.

Recommendations can be found in NIST SP 800-90A-C
http://csrc.nist.gov/groups/ST/toolkit/random_number.html
or
BSI AIS 20 & BSI AIS 31
https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Zertifizierung/Interpretationen/AIS_31_Functionality_classes_for_random_number_generators_e.pdf?__blob=publicationFile

and tool https://www.fourmilab.ch/random

4.4.2 Security Access algorithm

4.4.2.1 Security Access algorithm

4.4.2.1.1 Configuration of the Level of Protection

The level of protection by using diagnostic service Security Access together with the Security Access Algorithm is configurable with respect to:
1. Fixed Bytes:
Common. The Fixed Bytes are used for a security level by an ECU in several vehicles. Typically it is already programmed when the ECU is delivered or . Hence, a compromised ECU (disclosure of fixed bytes from an ECU) will affect the security of other vehicles.
Unique. ECU individual (unique) Fixed Bytes are used (forward-secrecy, i.e. a compromised ECU will not affect the security of another ECU).
2. Delay Timer introduced after a number of false security access attempts.
3. Delay Timer could be applied upon every start-up.
To guide the system designer, this logical component will specify some basic configurations with respect to the level of protection. Which level to choose is based on the information (asset) to be protected.

Notes:
- The terminology "level of protection" is used in favor to "level of security" as security level is already in use (ISO 14229-1) to define the "area" to be protected in an ECU using Security Access. Each Security Level could be configured separately.
- There should be dedicated Fixed Bytes for different purposes (roles), e.g., for accessing enhanced diagnostic functionality or for ECU reprogramming.
- The level of protection is dependent of the random number generator capabilities (seed generation) and the secure storage of the Fixed Bytes.

Protection Level	**Delay Timer** (activated when max number of false attempts detected)	**Number of false security access attempts**	**Fixed Bytes**
1	N/A	N/A	Common
2	10 seconds	2	Common

3	N/A	N/A	Unique
4	5-60 seconds	2	Unique

Table - Levels of Protection

Level 1: To prevent an unauthorized user that is having limited time and capabilities, or to avoid accidental errors, as the common key must for "Level 1" be assumed to be known by an attacker at some stage.
Level 2: See Level 1. But, adds additional protection (time effort) to reveal the Fixed Bytes used the first time.
Level 3: Using this level limits that unauthorized access (Fixed Bytes revealed) to one ECU also affects the security of a second ECU.
Level 4: See Level 3. But, adds additional protection (time effort) to reveal the Fixed Bytes used.

Physical attacks not considered here, i.e. where Fixed Bytes are extracted from the ECU memory.

1586 v3 — Default values - Delay timers and counters for false attempts

Version history:
(3):

Purpose: To define default values for the parameters used to configure the level of protection
Source:
Verification Method: Test
Legacy ID: 355163v2

The default values shall be set to:
- Delay Timer activated after a number of false Security Access attempts: 10 seconds.
- Delay Timer activated at every start: 5 seconds.
- Number of false Security Access attempt before a delay time is activated: 2.

A function/system designer is free to increase the delay timer activated after a number of false attempts, but decrasing that parameter or change of the other parameters must require an approved deviation.
ECUs having the proper hardware capabilities shall store (in non-volatile memory) the information whether the delay timer after a number of false attempts has been activated or not, i.e. that information shall not be lost at a reset.

1587 v3 — SecurityAccess (27) - time delay after false securityAccess attempts

Version history:
(3):

Purpose: For some scenarios the tools will require two attempts to unlock the ECU.
Source:
Verification Method: Test
Legacy ID: 76145v2

It shall be possible to make at least 1 false SecurityAccess attempt without being penalized by a time delay before the next attempt is accepted.

4.4.2.1.2 Random Number Generation

The generation of the random number, i.e. the Seed the sent by the ECU, is important with respect to the security of algorithm, e.g. identical Seed values will result in that identical Key values (Response Bytes) are computed.

1588 v1 **Seed - Random Number Generation**

Version history:
(1):

Purpose: Random and pseudo-random numbers generated for cryptographic applications should be unpredictable.
Source:
Verification Method: Test + Analysis
Legacy ID: 354418v0
ASIL: QM - Not safety relevant

The Seed generated must be a cryptographically secure random number, meaning that it's generated using either a True Random Number Generator (TRNG) or Pseudo-Random Number Generator (PRNG). This implicit implies that the Seed outputs must not be repeated after an ECU reset. Hence, static Seed shall never be used.

4.4.2.1.3 Security Levels (Areas)

An ECU is divided into several security access levels ("areas"), where each level is protected by a dedicated security access constant. Information or services are by that separated and only accessible to the user having access to the security access constant to a dedicated level. For example, an user shall only be allowed to read some information but not be able to re-program an ECU.

This logical component defines some basic security access levels that shall be applied unless there is a specific need to add some more level in order to separate some information further.

1592 v2 **Security Level usage**

Version history:
(2):

Purpose: Define a basic set of security access levels. Other security access levels are still supported, but needs to specified elsewhere.
Source:
Verification Method: Test
Legacy ID: 436226v0

The ECU shall use the security access levels defined in *Table - Security Access Levels.* Other levels are however also allowed, but requires an analysis to identify the need of separating information or services further.

Security Access Level	Purpose	Comment
01	Used for software download	A vehicle unique security access

		constant shall be generated in the manufacturing process.
03	Car mode transition	A common security access constant shall be applied.
05	General for diagnostic services	This is the default area that shall be applied by the ECU, unless there is a specific need to separate the specific information from others.
11	Immo.	-
-	-	-
1D	Test and development only	Uses the algorithm supported by OEM tool-chain (but not used at manufacturing or workshops)
1F	Legal requirements for ERA-GLONASS	-
-		
31	Vehicle Speed Limitation	
5F	Pyrotechnic end-of-life activation	ISO 26021 requirement

Table - Security Access Levels

4.4.2.1.4 Introduction

Purpose

The purpose of this document is to define the security algorithm to be used with diagnostic service $27 (request security access) for all ECUs supporting diagnostics, regardless of protocol.

Diagnostic service $27 is based upon the tester requesting a random three byte seed from the ECU. The tester then utilizes the algorithm described in this document to calculate a three byte key which is transmitted to the ECU. The ECU utilizes the same algorithm and compares its calculated three byte key to the key submitted by the tester to determine if security access is granted or not.

The algorithm described in this document generates a three byte response (key) to an eight byte challenge. The eight byte challenge consists of three random seed bytes (S1, S2, S3) and five fixed bytes (F1, F2, F3, F4, F5). Note that the five fixed bytes are not transmitted on the network and must be known in advance by both the ECU and tester. These fixed bytes are also referred to as the Security Constant.

The seed bytes are transmitted from S1 (most significant byte of the 3 byte seed) to S3 (least significant byte of the 3 byte seed) in a $67 positive response to a service $27 (security access) diagnostic request. For example, $67 $01 $S1 $S2 $S3.

Key (response) bytes are transmitted from R1 (most significant byte of the 3 byte key) to R3 (least significant byte of the 3 byte key) in a $27 submit key request. For example, $27 $02 $R1 $R2 $R3.

4.4.2.1.5 Algorithm Description

6429 v2 **Security Access Algorithm Description**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Define the algorithm used for Security Access
Source:
Verification Method: Analysis
Legacy ID: 362857v0

The Security Access algorithm description is provided by CEVT Base Technology Team upon request.

4.4.2.1.6 Security Constant

1589 v1 **Distribution of the Fixed Bytes**

Version history:
(1):

Purpose: To define the source provider of the fixed bytes.
Source: -
Verification Method: Test + Inspection
Legacy ID: 353878v0
ASIL: QM - Not safety relevant

The fixed bytes are to be supplied from CEVT. These are not to be chosen by supplier.

1590 v2 **Fixed Bytes default value**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To define the default value of the fixed bytes when no Security Constant has been programmed to the ECU.
Source: Required by diagnostic tools.
Verification Method: Analysis
Legacy ID: 353877v0
ASIL: QM - Not safety relevant

If the fixed bytes has not yet been programmed to the ECU and unlocking by the $27 Security Access service is requested, the ECU shall use a default value where all fixed bytes are set to 0xFF.

1591 v1 **Key Storage**

Version history:
(1):

Purpose: The security of the system depends on the secrecy of the key that is used.

Source:
Verification Method: Test + Inspection
Legacy ID: 360246v0

The Fixed Bytes used shall be stored in a protected area, typically in tamper resistant memory, and must not be possible to read out. If a key update process is required, the security of that process must be ensured.

4.4.3 Software Authentication

4.4.3.1 General Software Authentication

4.4.3.1.1 System Description

To support legacy systems and enable future growth, the Software Authentication concept must be prepared to support several signing methods even though an ECU will only support one. The defined signing method will affect:

- Algorithms and key lengths chosen
- Sign and verify with code signing certificates or by only using the key-pair (no certificates)
- How to transfer the signature data
- How to initiate the verification in the ECU and inform the user of the result
- ...

Software Authentication Generation 2.0 - each software part is signed individually: Each software part is signed individually and each software part is carrying the digital signature, separately generated as previously shown in *Figure - Software signing function overview.*

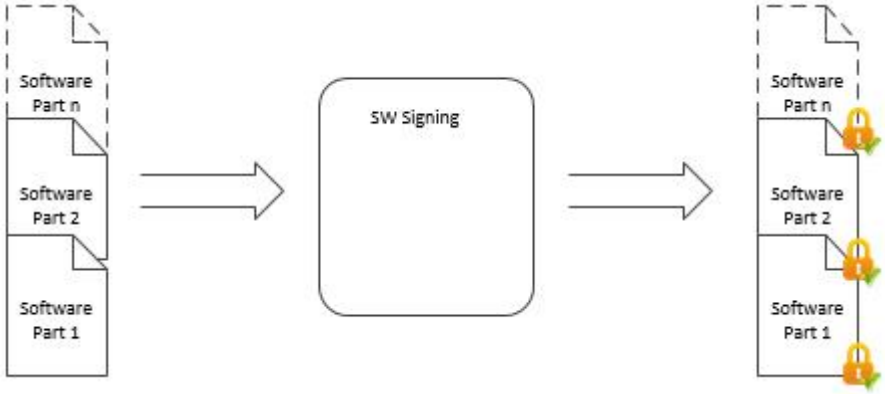


Figure - Software signing of each software part independently

For off-board tool reference purposes (to manage the variability), the legacy concept "Software Authentication Generation 1.0 (Legacy) - complete set of software parts are signed" can be found in *[Software Authentication]*.

This section will describe the following main parts, for the method where each software part is signed individually:

- Software part creation
  - Responsible: ECU Software Supplier
- Sign
  - Responsible: Trust Centre
- Software Download
- Verification

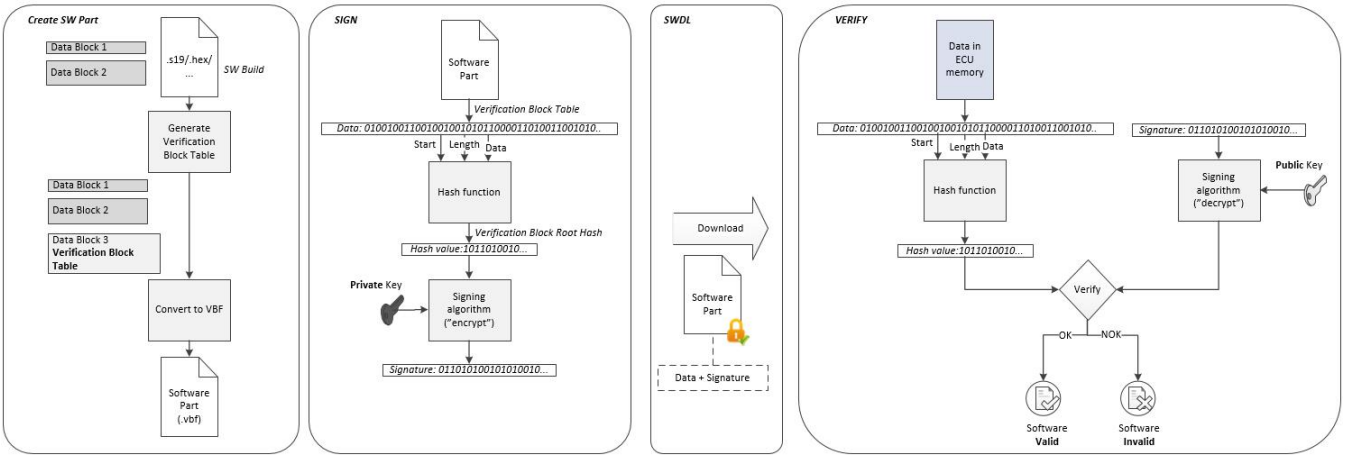


Figure - Create, Sign, Download and Verify a software part

4.4.3.1.1.1 Create Software Part

A software part could be segmented into several data blocks. To verify the data stored, the ECU must have knowledge of the data that must be verified and where it is located. The ECU software supplier will, once the application is build, generate a *Verification Block Table* and append it to the Software Part. This block table contains information of all other data blocks (location and hash values), i.e. it is equivalent to a manifest. The Verification Block Table will be digitally signed and when the ECU has verified it, the ECU could continue to verify all other expected data blocks. Hence, there will be one Verification Block Table per Software Part.

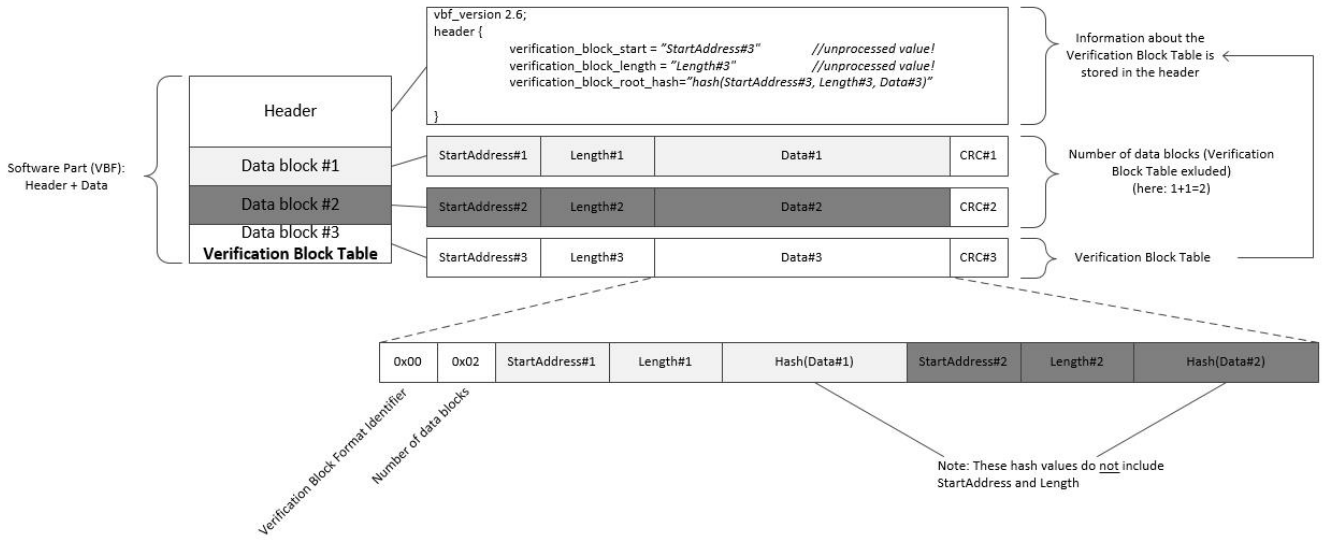


Figure - Verification Block Table format Example.

The ECU software supplier has build an application having two data blocks.
Based on the two data blocks, the ECU software supplier generates and adds a Verification Block Table (a dedicated data block, stored at a known start address):

The start address, length and hash value of the Verification Block table itself are added to the file header. The hash values of the data blocks stored in the Verification Block Table are calculated on the data only, but the hash value of the Verification Block Table (i.e. verification_block_root_hash in software part header) is calculated using also its start address and length. Note that the lengths defined in the VBT are the lengths of the data blocks prior to e.g. any compression is applied.

4.4.3.1.1.1.1 Verification Block Table

1600 v5 **Format of the Verification Block Table**

Version history:
(5): Clarification

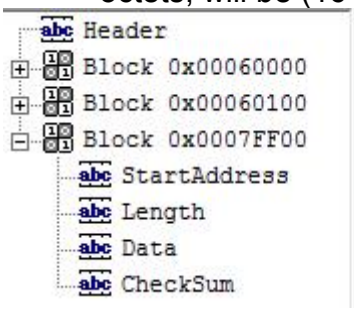
Purpose: Define the format of the verification block table
Source:
Verification Method: Test
Legacy ID: 398360v4

The verification block table, encoded with big endian, shall have the following format:

- Verification Block Table Format Identifier. The verification block will start with most significant byte of this format identifier. It defines the version of the verification block table, mainly intended for tool parsing and format checks in the ECU, i.e. it shall not be used to configure e.g. the actual algorithm to be used by the bootloader. Size 16 bits.
- Number of data blocks. This shall be identical to the number of data blocks included in the software part. The data block containing the verification block table itself shall not be included. Size 16 bits.
- StartAddress [size 32 bits] , Length [size 32 bits], Hash value. Every data blocks shall be defined, except the block table itself. The size of the hash is dependent of the hash function used and is specified elsewhere. The Length is the unprocessed length (see other requirements for more details).
- The data blocks shall be sorted according to the order the data blocks are being programmed to the ECU.

Example.

- SHA-256 hash function is used, i.e. each hash value is 256 bits.
- Number of data blocks are two, i.e. the total length of the verification block table, in octets, will be (16+16+2x(256+32+32))/8=84 octets.




```
0000 : 00 00 00 02 00 06 00 00 00 00 00 13 D7 8E CD 62
0010 : C6 2C F1 F5 75 D7 DE C6 D2 FD 57 DA 06 65 54 FD
0020 : 42 EE E5 C8 67 E7 00 83 BC 94 22 62 00 06 01 00
0030 : 00 01 EF 00 E6 02 7B 3F 95 9F AF 23 1B 38 2F FD
0040 : C6 4E B2 38 25 E6 B7 CC EA 47 F5 19 6D 02 CA 2A
0050 : FD F4 41 88
```

Figure - Example of a VBF file with a verification block table

Figure - Example of a VBF file with a verification block table shows a software part (VBF data file). The data shown belongs to the verification block. The Verification block starts at address 0x0007FF00, i.e. it is the third data block in the figure:
0000
0002
00060000 00000013
D78ECD62C62CF1F575D7DEC6D2FD57DA066554FD42EEE5C867E70083BC942262
00060100 0001EF00
E6027B3F959FAF231B382FFDC64EB23825E6B7CCEA47F5196D02CA2AFDF44188

- The format identifier is 0x0000
- Number of data blocks are two (0x0002)
- Data block 1: Start address 0x00060000 and length 0x00000013. Hash 0xD78ECD...
- Data block 2: Start address 0x00060100 and length 0x0001EF00. Hash 0xE6027B...

1603 v4 **Verification Block Table Format Identifier - Definition**

Version history:
(4): Req change

Purpose: Define the applicable value of the Verification Block Table Format Identifier.
Source:
Verification Method: Test
Legacy ID: 404696v3

The Verification Block Table Format Identifier shall be according to *Table - Verification Block Format Identifier*.

Format of the Verification Block Table	Verification Block Format Identifier	Hash function	Comment
As defined in REQPROD 398360	0x0000	SHA-256	Hash(Data block)
As defined in REQPROD 398360	0x0001	SHA-384	Hash(Data block)
As defined in REQPROD 398360	0x0002	SHA-512	Hash(Data block)
-	0x0003-0xFFFF		Spare

Table - Verification Block Format Identifier

Example.
The signing method used is, as defined in REQPROD 410732, e.g. SHA256_RSA2048. This means that Verification Block Table Format Identifier shall be 0x0000.

1602 v2 **Verification Block Table - Input order to Hash function per data block**

Version history:
(2):

Purpose: Define the input order to the hash function per data block. If the order is changed, the output from the hash function will be changed also.The start address and length of a data block are NOT input to the hash function. The placement of a data block is ensured by explicitly defining the start address and length per corresponding data block: start address, length, hash (data block).

Source:

Verification Method: Test

Legacy ID: 398373v1

The hash value for each data block in the verification block table shall be calculated as *Hash(Data)*, i.e. the start address and length are excluded. The data to be hashed starts with the data at the start address.

1604 v4 Verification Block Table Header Identifiers

Version history:
(4): Clarification

Purpose: Define the required header identifiers required to support the applied signing method

Source:

Verification Method: Test

Legacy ID: 404693v3

The header section of the delivered software part must contain the following identifier with respect to the verification block table, when a verification block table is constructed according to the format in "REQPROD 398360: Format of the Verification Block Table".
Alternative 1 - verification block header identifiers (default configuration):

- **verification_block_root_hash**: The hash value of the unprocessed verification block table.
- **verification_block_start**: The start address of the unprocessed verification block table.
- **verification_block_length**: The length of the unprocessed verification block table.

Upon OEM approval, the header section can exclude the verification_block_start and verification_block_length identifiers:
Alternative 2 - verification block header identifiers:
verification_block_root_hash

This might be applied for ECUs where existing installation and verification mechanisms, using e.g. ECU specific manifests, are to be reused in the ECU to ensure authenticity that is not possible to realize using the Volvo verification block table format.
Example.

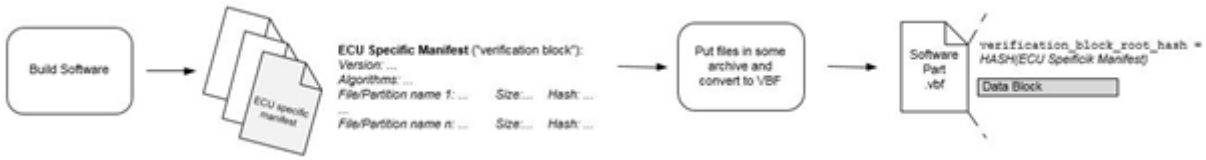


Figure - Example of ECU supplier specific verification block (manifest)

"Figure - Example of ECU supplier specific verification block (manifest)" shows a data file header where verification_block_root_hash is present only.

1601 v4 **Verification Block Root Hash**

Version history:
(4): Clarification

Purpose: Define how to calculate the verification block root hash, carrying the information of unprocessed data that is required to support use cases of having processing methods not necessarily known to the Trust Centre.

Source:

Verification Method: Analysis

Legacy ID: 404694v2

Req:
The verification_block_root_hash must be calculated on the "raw" unprocessed data in the verification block table, i.e. when no prior to any padding, encryption/compression/delta encoding method hasmethods have been applied to the VBT data block. The verification block root hash shall be calculated as Hash(startAddress||length||Verification Block Table data), where || represents the concatenation. The order shall be according to *Figure - Verification Block Root Hash calculation*. The start address (verification_block_start) and length (verification_block_length) are included to ensure the placement of the verification block and shall always be 32 bits long each. The (root) hash function shall be the same as used when calculating the hash value of the data blocks defined in the verification block table (REQPROD 404696).

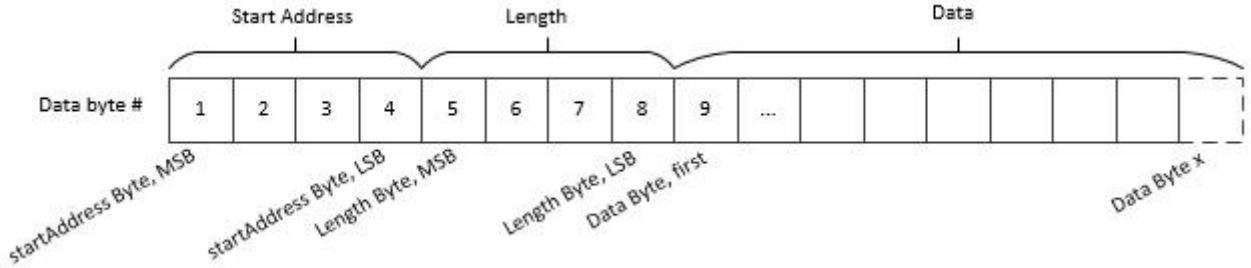


Figure - Verification Block Root Hash calculation
The root hash value shall be added by the software supplier to the header of the software part.

Example 1.
All values are in hexadecimal. SHA-256 is used as hash function.
Hash(startAddress||length||Verification Block Table data)=
Hash(*00 05 FF 00***00 00 00 54** 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1) =
9B B4 E0 92 1C 4B 84 1C 31 C3 CB 3F DD 15 D0 B9 E8 ED 70 5E D3 C2 52 DC C9 2A D5 06 E0 5E 2B AE.

Start Address: *0005FF00*
Length: **00000054**
Data: 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1

Example 2.
The length of the data block for a verification block table is 44 bytes long, prior to any padding or processing method is applied.
All data blocks shall be aligned to a multiple of 8 bytes. Hence, the VBT data block is padded to 48 bytes, but verification_block_length is still 44 (0x2C). It is this length that is used when calculating the verification_block_root_hash.

Note: Be aware of the differences of how to manage the padding of data and start/length information for data blocks that are defined in the verification block table versus the data block containing the VBT.

1605 v3 Verification Block Table on unprocessed data

Version history:
(3): Clarification

Purpose: To enable that the target software is verified.
Source:
Verification Method: Test
Legacy ID: 404184v2

Req:
The data block definitions in the Verification Block Table shall be prior to processing methods have been applied, i.e. prior to the any compression, encryption or delta encoding is applied. If there is a need to align the data blocks to a specific size prior to a processing method is applied (e.g. to match a boundary of a flash sector and to be a multiple of the minimum programmable block size for the specific ECU), it is the padded data blocks that are to be used as inputs when defining the data blocks in the Verification Block Table.

Example.

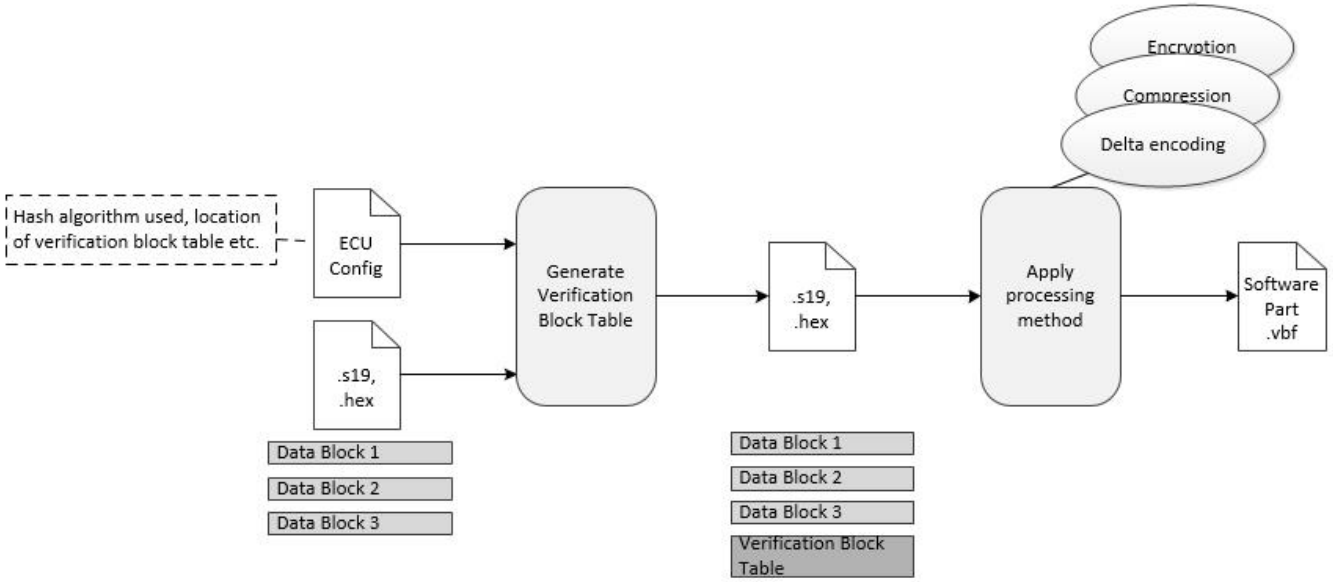


Figure - Generate verification block table on unprocessed data

1606 v3 **Verification Block Table per Software Part**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To enable that each software part is to be signed and verified individually.
Source:
Verification Method: Analysis
Legacy ID: 398371v1

Each single software part must contain a verification block table, i.e. it will be one verification block per logical block defined in the ECU.

130849 v1 **Verification Block Table in a padded data block**

Version history:

Purpose: To clarify how to manage the verification block table when there is a need to align that data block to a specific length, e.g. to match the start and/or end of a data block to the boundary of a flash sector (for an addressed based ECU). This requirement is valid for the data block containing the VBT, i.e. for all other data blocks the padding will be treated as any other data when generating the VBT.
Source:
Verification Method: Test
Legacy ID: 457183v0

Req:
If there is a must to align and pad the data blocks to some specific size, the padded data must have the value of erased memory, i.e. it will be included in the ECU "blank" check.

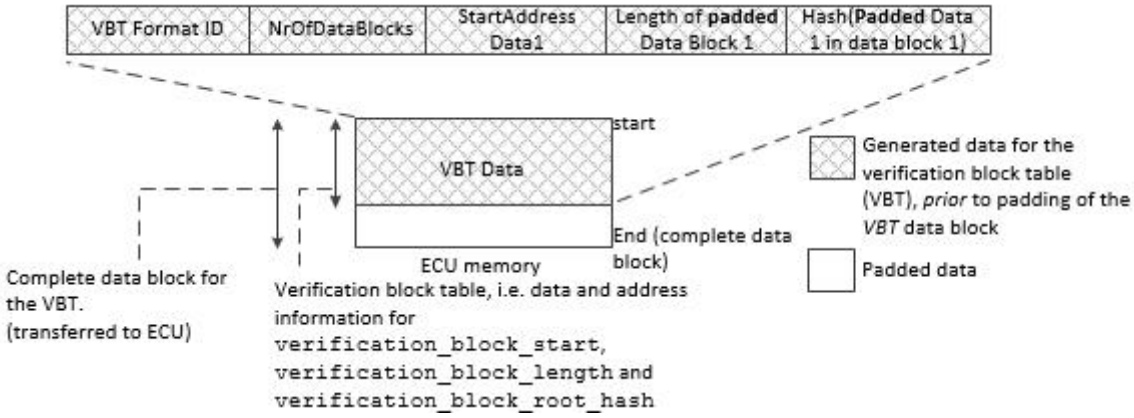


Figure - Data block with aligned verification block table data block

Note: The verification Block Table*_block_start, verification_block_length and verification_block_root_hash shall be* calculated on data (blocks)*generated using the "raw" unprocessed data in the verification block table prior to* some processing methods have been*any padding is applied*, i.e. prior to the any compression, encryption or delta encoding is applied *to this specific data block.*

4.4.3.1.1.2 Sign Software Part

The software is signed with the confidential private key. It is the Verification Block Table that is signed, as defined by the software part header identifier.

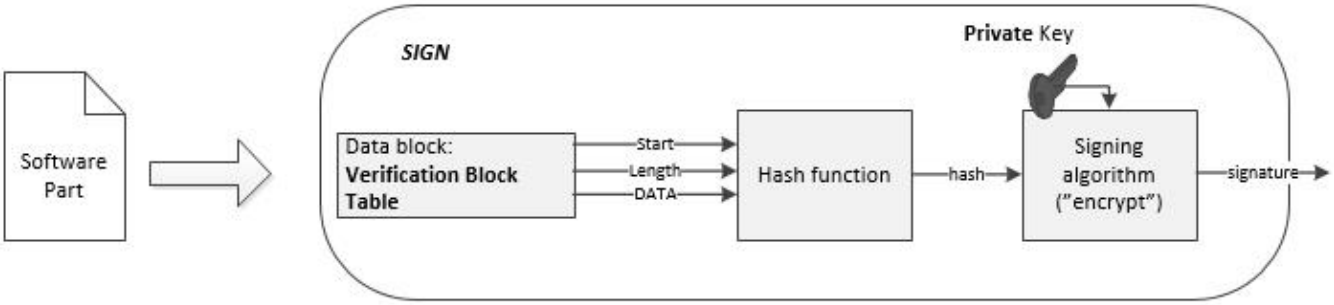


Figure - Software Signing Tool

There are many use-cases to be supported, including different diagnostic clients, in order to download (signed) software to a vehicle:

- key balancing, e.g. having global/common keys used for a period of time or ECU unique keys
- software download at manufacturing, workshops, over-the-air (including vehicle internal tester), test&development, fault-tracking (e.g. when a software download using workshop diagnostic client fails and R&D tools must be used), repair analysis etc.

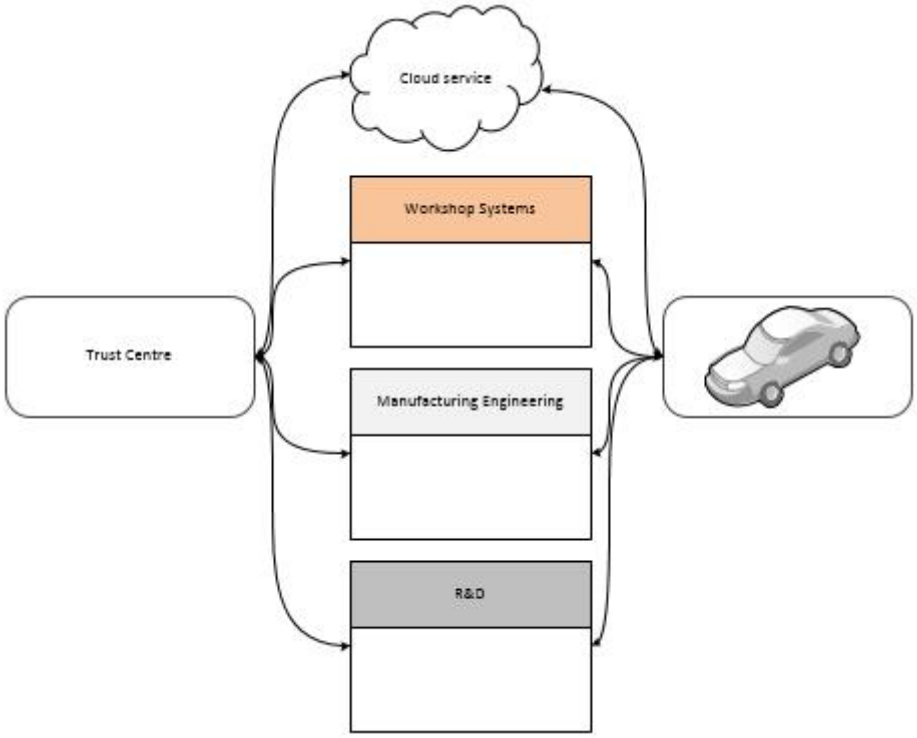


Figure - Example of entities downloading software to a vehicle
To support the variability as shown in *Figure - Example of entities downloading software to a vehicle* as well as balancing key usage, the signature data for production (e.g. customer vehicles) vehicle is kept detached (separate file).

Example.
Unique keys are used.
A software application *12345678AA.vbf* is to be signed using private key#1 for ECU#1 and private key#2 for ECU#2. The sw application part could be programmed in both ECUs but the belonging signature data must be transferred to the correct ECU respectively, otherwise the application will not start.

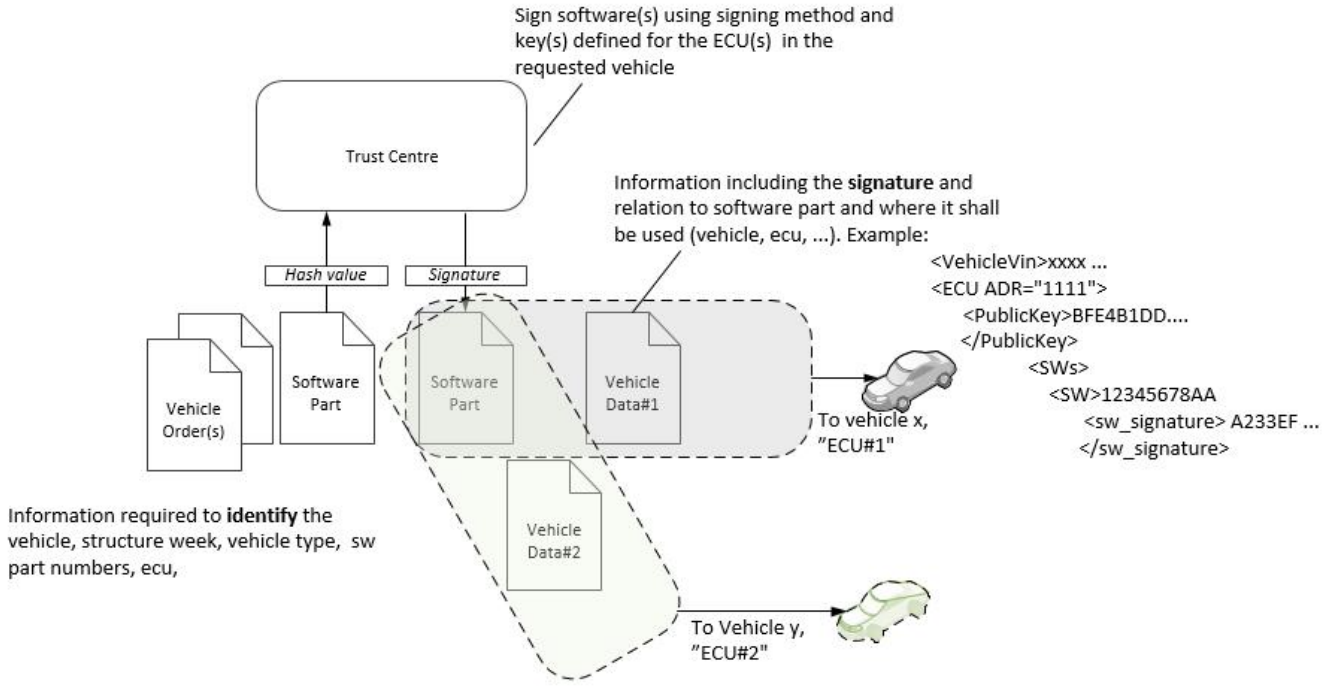


Figure - Example: Detached signature
The *Figure - Example: Detached signature* shows the principle of detached production signature.

Hence, the diagnostic client must have the capability to correlate belonging signature for a software part and transfer it to the vehicle.

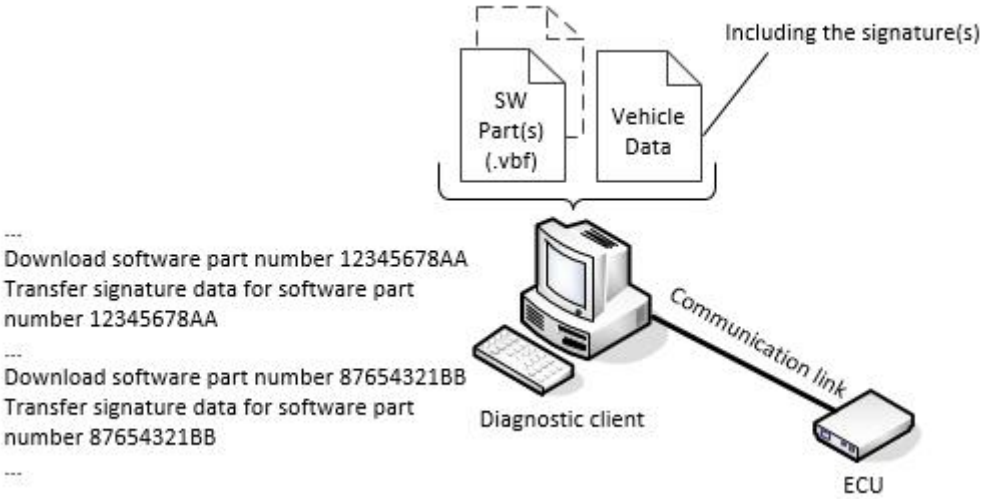


Figure - Download a software and transfer signature data
To support the test&development phases, a development key-pair will be used. To simplify the software part management, the software signature for development will be placed in the part header (the R&D diagnostic client will also have the capability to read the software signature for production from the software part header *[VBF (Versatile Binary Format)]*) .

Refer to section *Key Management for Test and Production ECUs* for further details.

1613 v2 Signing function - Integrity Check

Version history:
(2):

Purpose: To ensure that the software part is not altered when updated, i.e. validate the integrity.
Source:
Verification Method: Test
Legacy ID: 428664v1

The signing tool used must ensure the integrity of the software part. If the tool is updating some software part identifier, the file_checksum shall be checked before and after the update and the checksum shall be identical.

1611 v3 Sign Software Part - Private Key syntax

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define the key representation for key distribution at test&development and for development tool usage.
Source: PKCS #8: Private-Key Information Syntax Standard, RSA Laboratories Technical Note Version 1.2, Revised November 1, 1993. PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012. RFC1421.
Verification Method: Analysis
Legacy ID: 428665v1

The Private key shall be represented in an unencrypted PKCS #8 PrivateKeyInfo format as defined in *PKCS #8: Private-Key Information Syntax Standard, RSA Laboratories Technical Note Version 1.2, Revised November 1, 1993*.
The PrivateKey part of of the PrivateKeyInfo shall be a represented as an ASN.1 type RSAPrivateKey as defined in *PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012.*
The Private Key shall be stored using the PEM format (base64 encoded data) as defined in RFC1421.

Note: *This is the formats to (at least) be supported by signing development tool. The format used by the Trust Centre (production) is specified elsewhere.*

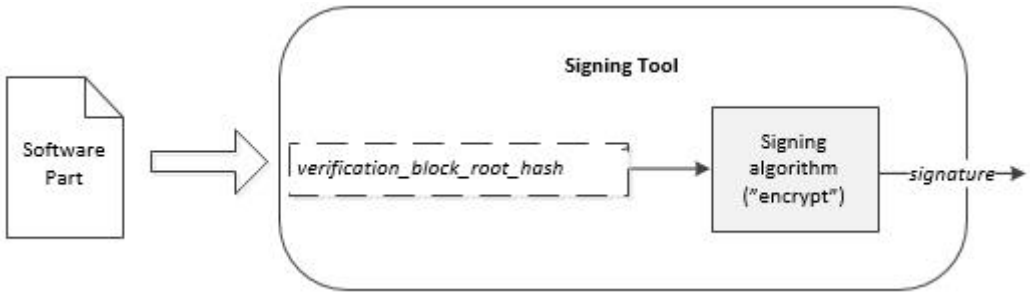
1612 v4 Signing function - Generate signature

Version history:
(4): Clarification

Purpose: Define the input data to the signing function.
Source:
Verification Method: Analysis
Legacy ID: 428667v3

The data to be signed is defined by the verification_block_start and verification_block_length identifiers. These start address, length and data of the addressed data block shall be used as inputs when generating the hash value. The signing entity has two possibilities:
(1) always calculate the hash value over the verification block table and ensure that the content of the table is correct. This is the preferred option.
(2) use the already calculated hash value (verification_block_root_hash).

Example.
The verification_block_root_hash might be used when the processing method is not known (e.g. delta compression method) as defined by the software part data format identifier but it might also be used to optimize off-board functions (e.g. computational time), alternatively when the verification_block_start and verification_block_length are not present in data file header.



Sign - Processing method unknown

1610 v5 **Append signature to software part**

Version history:
(5): Clarification

Purpose: Define how to append the signature to the software part. The signing tools must support the use-case of sign and re-sign software parts, typically during development but also to enable for e.g. key updates. To support the lifecycle of the part, i.e. the software is verified (development) prior to it is released to production, two signature types will be applicable for the final software part (to not have to re-sign the software when it is to be used in a test object versus a production vehicle).

Source:

Verification Method: Analysis

Legacy ID: 428666v2

The digital signature when the software part is signed with the intended private key for **development**, sw_signature_dev, shall be appended to the software part header by the software supplier when a part is delivered.. The resulting sw_signature signed by the trust centre at OEM with the intended key for **production** shall be stored in a separate file, using a standardized format separately (e.g. .xml). The signing tool must support both sign and re-sign functionality of a software part.

Example.
(1) A software part is delivered containing the sw_signature_dev. This software can only be

used in an ECU equipped with the public key for development.
(2) The software part is, when successfully verified, validated and ready for production, signed with the private key for production. This means that two signatures are present (the sw_signature_dev is kept).
(3) This software part can now be used in both a development and production ECUs, but it must be ensured that the corresponding signature is transferred at SWDL.

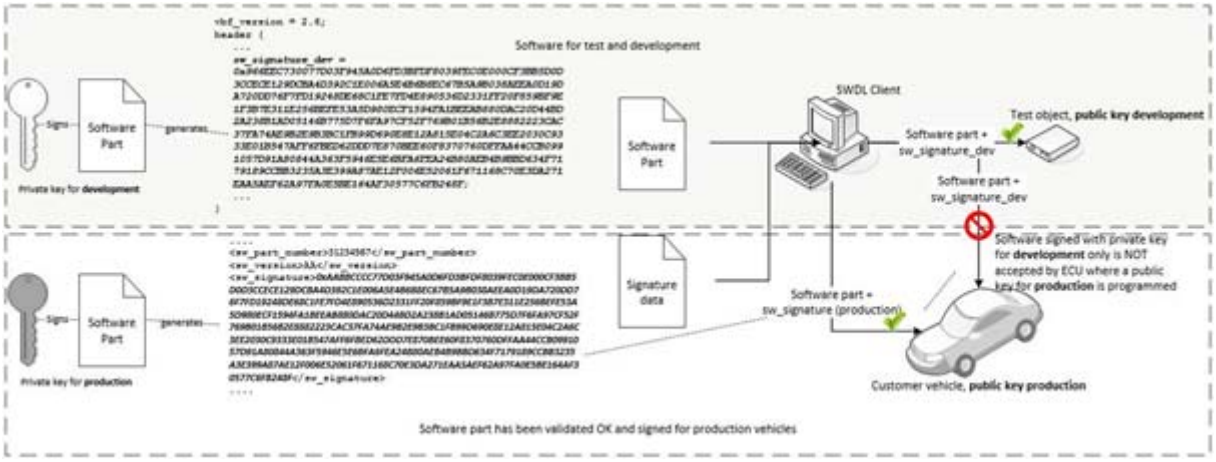


Figure - Signature for test and development and production

4.4.3.1.1.2.1 Sign and verify unprocessed data

At the ECU side (where the software part is verified) it is the target data to be executed that shall be verified, e.g. the data stored in the non-volatile memory (for an application file). Hence, it is the unprocessed data that must be signed, i.e. the data prior to any processing method (**delta encoding**, **compression**, **encryption**, etc.) has been applied.

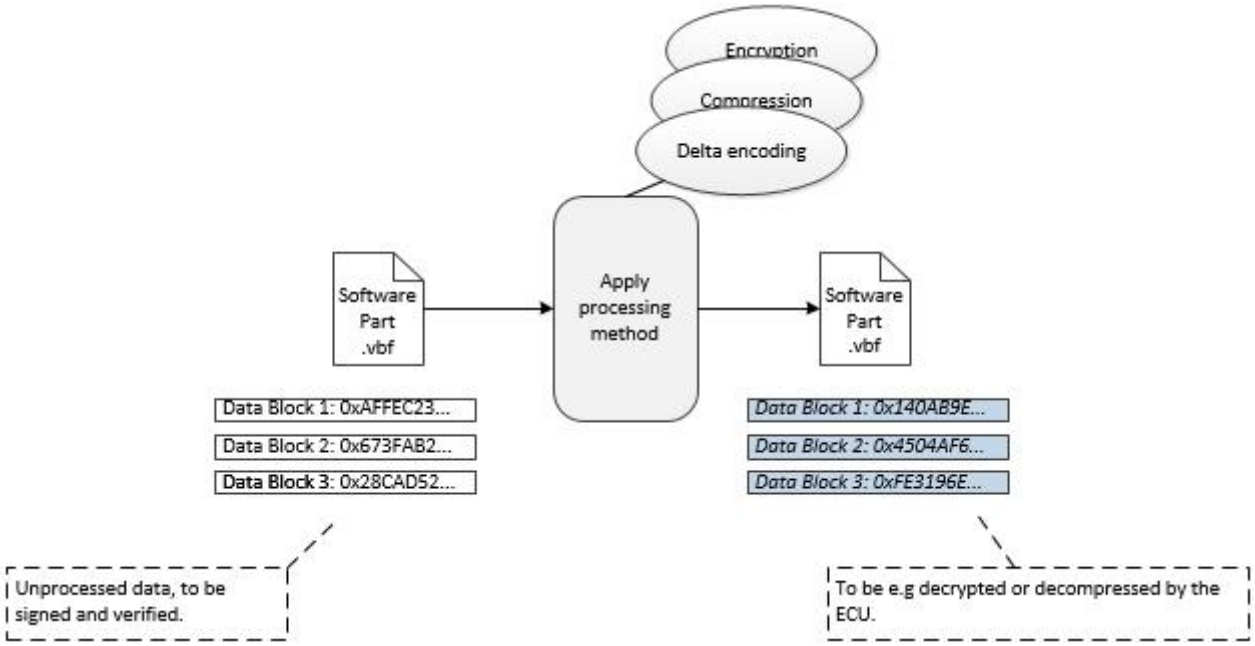


Figure - Signing and verification of unprocessed (target) data

From an ECU point of view, see *Figure - Verification of unprocessed data,* where the data stored in the non-volatile memory is verified,

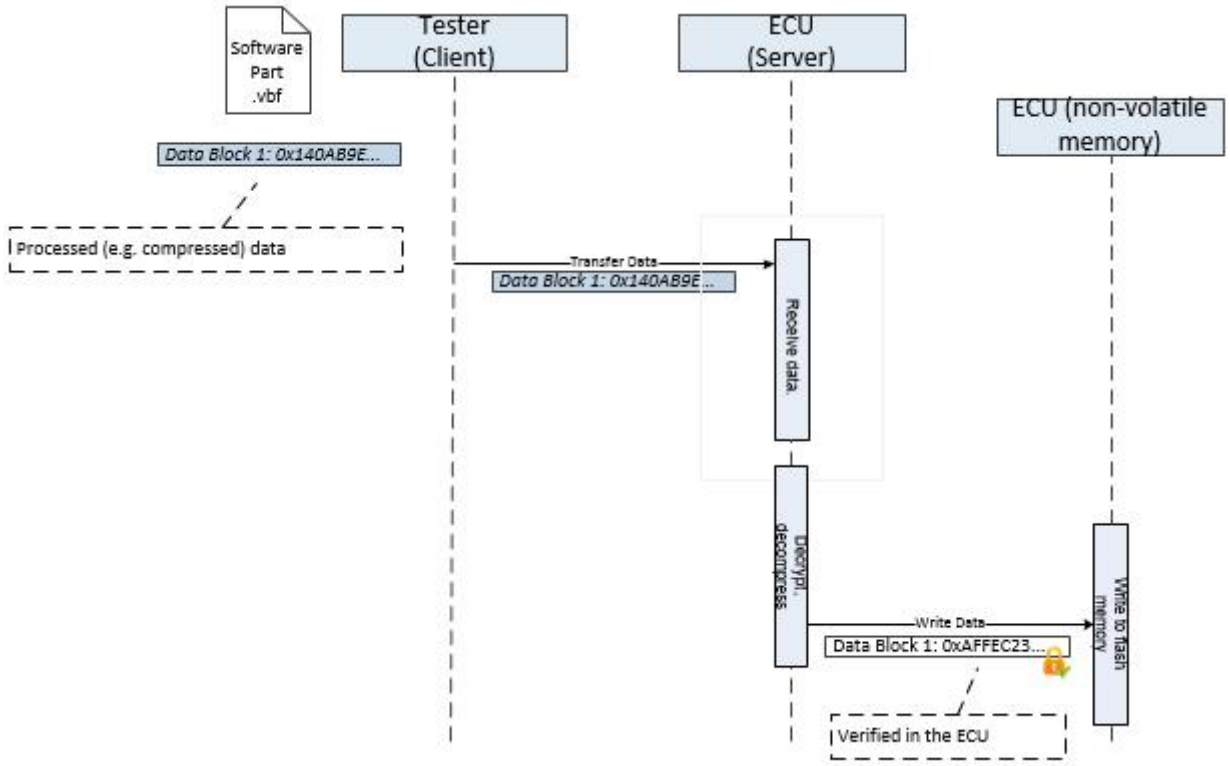


Figure - Verification of unprocessed data

1615 v2 **Sign and verify unprocessed data**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Define how to handle compressed, encrypted or delta encoded software.
Source:
Verification Method: Analysis
Legacy ID: 404594v0

The signing and verification shall be based on unprocessed data.

4.4.3.1.1.2.2 Sign a Delta Software Part

When delta encoding is used, the delta file shall contain information (hash values, signature, ...) identical to the final target software:

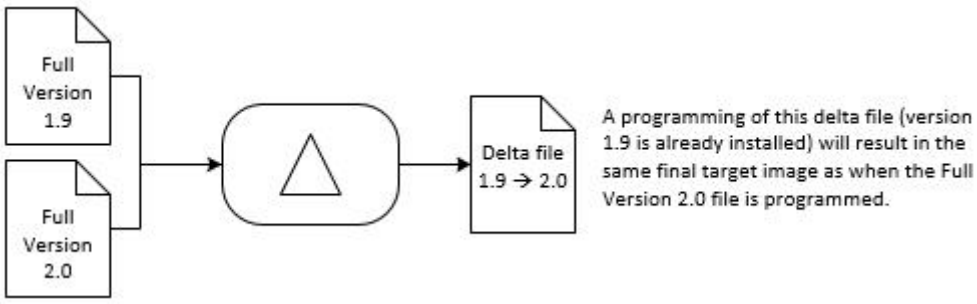


Figure - Signature for a delta file
When a delta file, as shown in *Figure - Signature for a delta file* is programmed, the ECU must finally verify the final data that has been installed in the non-volatile memory.

1614 v5 **Sign a delta file**

Version history:
(5): Clarification

Purpose: Define how to sign a delta file. The programmed (installed) data shall be verified, i.e. not the transferred (streamed) data.

Source:

Verification Method: Analysis

Legacy ID: 429082v3

A delta software part shall contain the same header identifiers as the corresponding full version, with respect to following Software Authentication identifiers: verification_block_start, verification_block_length, verification_block_root_hash and sw_signature_dev.

4.4.3.1.1.3 Download a (signed) software part

This section describes how to download a signed software part and transfer the signature data, where the signature data is transferred after each software part is downloaded.

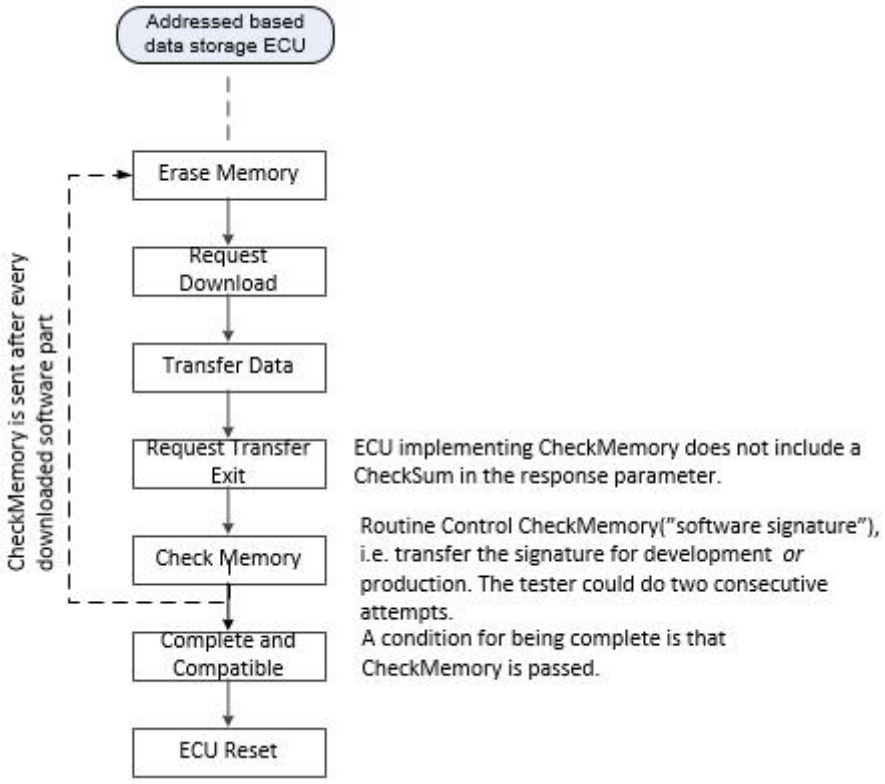


Figure - SWDL of a signed software part

The software signature is transferred when the complete data file has been downloaded using the CheckMemory routine control, which will also initiate the ECU verification. The result values provided in the CheckMemory response will inform the tester whether the verification has passed or not . Note that ECUs implementing CheckMemory shall not respond with a checksum value at the RequestTransferExit service. The CheckMemory return values are specified in *[General Bootloader requirements].* The result value is 0 (zero) when the verification is passed. If the verification is failed, the tester might do a second attempt.

Prior to the programming of a data file starts it must be ensured that the expected public key is stored in the ECU, e.g. either an initial development key is stored and to be kept (test and development only) or the dedicated public key shall be programmed (if it is not already programmed).

4.4.3.1.1.4 Verify a signed software part

This section describes and defines the general ECU verification requirements, relevant to both off- and on-board (ECU) systems. More detailed ECU requirements are however part of *[General Bootloader requirements]*.

An ECU is divided into several “Logical Blocks”, one logical block per software part and each logical block is signed/verified independently. A successful signature verification means that a logical block is complete (complete & compatible). The ECU must know the logical blocks that are mandatory for the whole application to be complete.

The logical block definition, ECU Software Structure, is to be re-programmable in order to support use-case when the software structure is changed from e.g. three to four files or when a software part growths beyond its dedicated range.

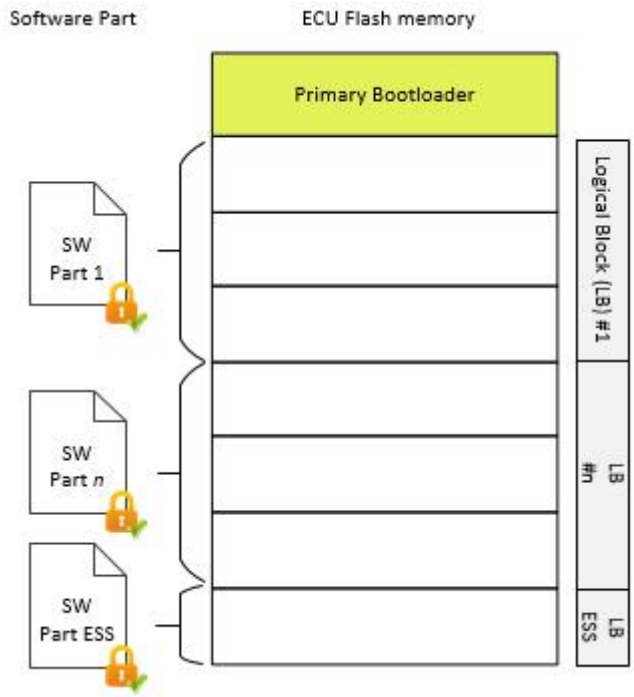


Figure - ECU Software Structure, ESS

The programmed public key is used to verify the software part, i.e. the logical block at:

- Installation, i.e. at Software Download (must).

But, the memory not changed (e.g. memory not used for adaptations, "user space", etc.) after e.g. software download can also be verified:

- At every start of the application (secure boot), starting from a root of trust and trusted read-only start-up code.
- At run-time, i.e. when the application is running.

A software part must have been verified OK for the application to be started in the ECU. The secondary bootloader, stored in volatile memory, is not shown in *Figure - ECU Software Structure, ESS.* The SBL must be successfully verified prior to it is activated.

4.4.3.1.1.5 Key Management for Test and production ECUs

To e.g. simplify the daily life for developers/testers etc. and to keep track of the keys used for signing and verification during test and development, a known key is used. This must not be used in production vehicles.

An empty (e.g. spare part) ECU shall by default contain the defined development key. The initial development key is used to verify software until the production key has been programmed to the ECU (WriteDataByIdentifier). Hence, a transition from that initial development key to a production key must be supported.

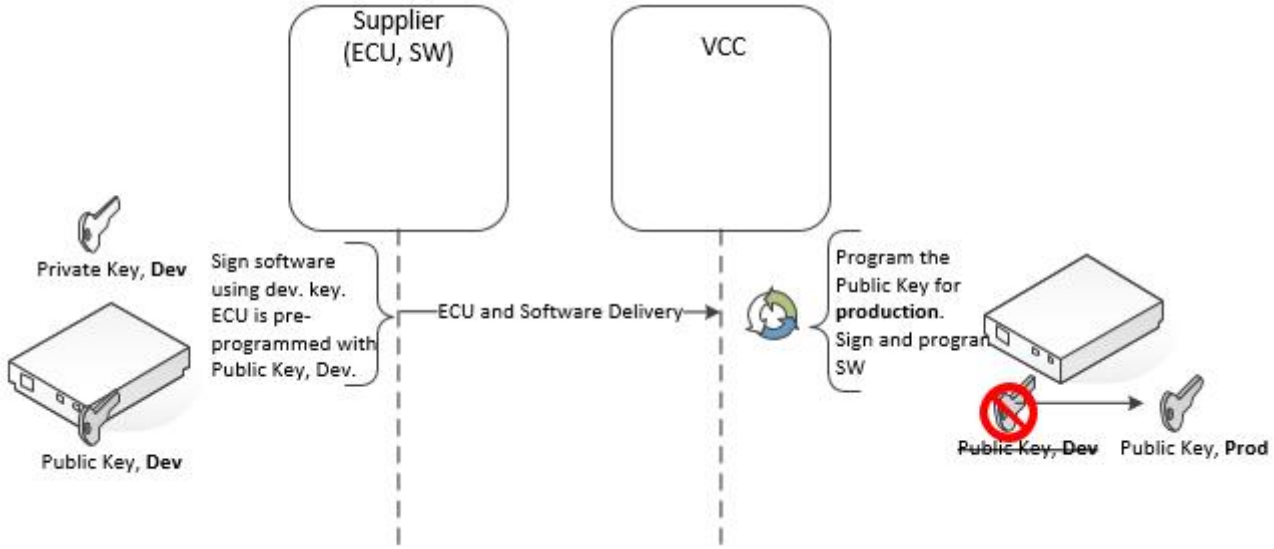


Figure - Transition from Development to Production

To simplify fault tracking, but also to ensure the integrity of the public key when it is programmed for production, a checksum of the public key will be appended when it is stored. That checksum is also possible to read once a public key has been successfully been programmed (i.e. it is not possible to read the checksum when an initial development key has been stored in the ECU by the ECU provider).

To prepare for future growth, where even more additional metadata will be appended to the public key to e.g. support a layered key structure, the term Public Key Type is used. That is used to e.g. inform the trust centre about the key structure supported by the ECU but could also be used by the diagnostic clients programming the public key (and metadata) if there is a need to store future public key types differently (e.g. using different diagnostic services).

Example - Public Key Types.

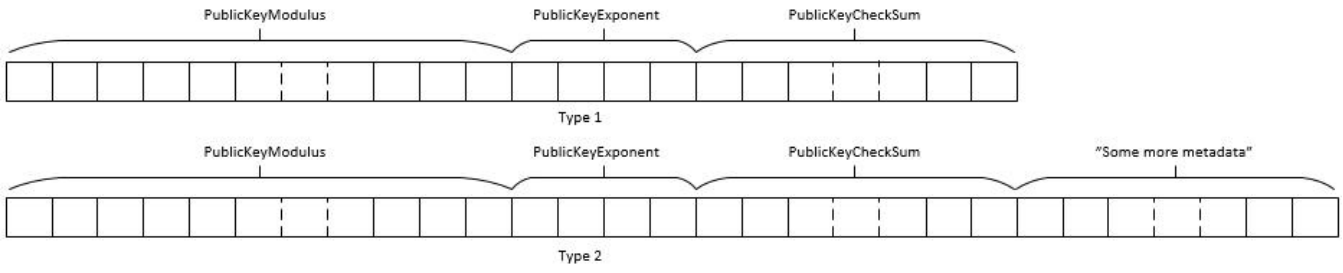


Figure - Public Key Types

The first type in *Figure - Public Key Types* is the format defined in the *Software Settings File* section by the swAuthPublicKeyType as RAW_KEY_WITH_CHECKSUM.

1607 v1 **Production Key programming**

Version history:
(1): Delete the requirement "Production Key programming";

Purpose: To ensure that no production vehicle leaves manufacturing or workshops

equipped with a development key. A development key is to be considered as known, i.e. it adds no security value.

Source:
Verification Method: Test
Legacy ID: 435017v0

It must be ensured that no production vehicles leaves the plant or workshop equipped with the development key, i.e. a valid public key for production shall be programmed to the ECU. This is applicable also for the use-case when no software part at all is being programmed at End-of-Line (e.g. an ECU is pre-programmed with the software, replaced etc.).

This requirement is not applicable for the ECU supplier but for off-board systems where the vehicle is manufactured (or at workshops).

1609 v5 **Software Authentication Test and Development Key**

Version history:
(5):

Purpose: To keep track of the keys used for signing and verification during test and development
Source:
Verification Method: Analysis
Legacy ID: 420232v2

For test and development only, a development key provided by the OEM shall be used. Unless otherwise is explicitly defined by the vehicle project, the following key pair shall be used:

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAt+l0S0X6piDTHDDpY4bp
zV+5k97KRcnWCJT3fbnuqdB4RXaUgJ33BSTXMOLADwRuYFMjvVADvyypu7RcxRFa
Hc4lfUIDT34cej4aaOiaABCNGCisJr1xrkrJuSMLm8EBZ0apAV5w8dm9f1ZLl2Fk
/8HZbpOrQGbVy/QC9fxTEVGpgFwHFqvLmCX+AvOJfleRemTMLHpx6IMzWQqpWSPP
SmvkJBr3jKkEXWW2dIcZQknjaQPdpMl1/qc8B8GRZ1RF/l/PRXL4vUeVuoGnVFBV
KZIvgYJxm0Mc6ycWyofiuoOgHoXvdeRjiC0LU3a2s9ZoGeJsK2dPCp3e/pNCQ86H
rQIDAQAB
-----END PUBLIC KEY-----


Private-Key: (2048 bit)
modulus:
    00:b7:e9:74:4b:45:fa:a6:20:d3:1c:30:e9:63:86:
    e9:cd:5f:b9:93:de:ca:45:c9:d6:08:94:f7:7d:b9:
    ee:a9:d0:78:45:76:94:80:9d:f7:05:24:d7:30:e2:
    c0:0f:04:6e:60:53:23:bd:50:03:bf:2c:a9:bb:b4:
    5c:c5:11:5a:1d:ce:25:7d:42:03:4f:7e:1c:7a:3e:
    1a:68:e8:9a:00:10:8d:18:28:ac:26:bd:71:ae:4a:
    c9:b9:23:0b:9b:c1:01:67:46:a9:01:5e:70:f1:d9:
    bd:7f:56:4b:97:61:64:ff:c1:d9:6e:93:ab:40:66:
    d5:cb:f4:02:f5:fc:53:11:51:a9:80:5c:07:16:ab:
    cb:98:25:fe:02:f3:89:7e:57:91:7a:64:cc:2c:7a:
```

```
    71:e8:83:33:59:0a:a9:59:23:cf:4a:6b:e4:24:1a:
    f7:8c:a9:04:5d:65:b6:74:87:19:42:49:e3:69:03:
    dd:a4:c9:75:fe:a7:3c:07:c1:91:67:54:45:fe:5f:
    cf:45:72:f8:bd:47:95:ba:81:a7:54:50:55:29:92:
    2f:81:82:71:9b:43:1c:eb:27:16:ca:87:e2:ba:83:
    a0:1e:85:ef:75:e4:63:88:2d:0b:53:76:b6:b3:d6:
    68:19:e2:6c:2b:67:4f:0a:9d:de:fe:93:42:43:ce:
    87:ad
publicExponent: 65537 (0x10001)
privateExponent:
    67:bf:c8:3e:2a:95:12:a0:d3:d7:44:74:75:14:07:
    d3:36:dc:2e:e1:f1:03:db:af:e5:99:7b:e0:ae:42:
    48:03:f5:c5:61:f6:b6:73:e6:85:3d:5a:34:16:c6:
    b7:f2:0c:fe:44:08:96:64:8c:28:8d:de:96:a8:51:
    e9:4e:37:a3:36:c7:09:59:73:1a:a6:0f:14:9a:f2:
    35:1a:7a:bd:ec:98:5b:f7:9d:de:20:e2:ff:aa:eb:
    0f:89:08:a4:6e:06:07:a7:e1:f1:86:c0:7a:7f:16:
    1a:be:a8:d8:16:36:6e:dd:81:76:92:d1:79:fc:49:
    41:cc:3e:db:5b:e3:d4:91:62:fa:26:04:1b:1f:5e:
    2b:d4:7c:b2:68:5c:ec:40:f2:c6:e2:83:7d:71:e0:
    f9:fa:41:24:8d:36:1c:e8:c4:05:38:ab:23:37:d1:
    c8:c6:ad:e5:de:98:d5:0a:7e:aa:ba:54:5a:63:d5:
    e1:49:be:84:0d:de:66:71:78:c8:df:5c:53:88:ea:
    00:c1:8d:af:81:9f:0a:c4:a0:6e:b0:f7:da:a1:22:
    72:46:65:68:3b:24:b7:89:d0:ce:de:3a:f3:d5:07:
    94:d7:17:9c:c7:90:77:7b:6c:7a:21:72:e5:17:25:
    8a:84:20:01:1b:9b:e5:96:5c:17:0b:de:85:38:cf:
    95
prime1:
    00:df:d1:18:63:6f:2c:1b:14:3b:95:3d:56:dd:5b:
    6f:01:6e:f9:4c:2b:cb:eb:dd:89:b0:8e:dd:f3:f0:
    0f:f7:71:75:d0:7e:b0:be:6b:51:3a:8c:cf:9b:81:
    fa:34:4f:fc:98:d6:65:b1:2f:82:55:a8:d1:b5:1a:
    af:7b:da:6f:02:43:14:7e:69:52:11:9e:a9:95:44:
    b1:19:7f:7b:7c:67:30:11:67:88:9d:b1:b5:6e:6a:
    69:17:64:57:dc:5e:fd:33:ae:84:dc:2a:73:54:8d:
    be:04:c7:6f:6e:7f:31:f1:04:b3:6b:f7:9f:62:15:
    b0:75:12:a8:4f:02:4a:cb:d7
prime2:
    00:d2:5b:6c:9d:a4:9b:a9:6e:29:67:3f:d1:c2:53:
    1a:50:5b:0a:7c:c1:64:01:ff:20:20:8c:68:32:b1:
    1a:fb:8e:73:54:29:3b:e7:fc:94:0f:63:06:a2:87:
    56:a4:e3:48:75:21:b7:10:97:fc:3a:1d:b3:e2:ab:
    e6:4b:04:8d:de:fb:ac:d3:e4:3d:71:20:da:04:aa:
    e2:98:f4:e3:6c:f4:fa:c3:a7:1f:73:30:4f:3c:fe:
    8a:3e:21:06:e7:02:b4:e5:9d:4e:37:64:67:15:f3:
    64:73:cf:30:2d:54:19:76:22:ee:44:b5:cb:8f:94:
    a0:f5:15:91:68:56:cc:38:1b
exponent1:
    7e:33:cf:06:b2:77:32:45:b4:5b:30:9d:3c:70:04:
    25:d0:c7:6d:b5:fc:64:61:24:f4:93:7a:7f:c4:4b:
```

```
    9c:81:33:a7:7e:e8:76:56:d9:14:a4:b5:a3:c0:24:
    af:3e:b2:f6:13:5e:80:0c:83:f7:7d:1b:d2:7c:db:
    9a:80:ce:bb:7d:cb:9e:84:10:ac:b2:c4:78:d0:a4:
    f3:f5:b8:51:ab:75:a5:3a:b6:04:05:62:82:82:2a:
    03:f0:a6:c2:32:25:9f:f0:b6:25:d7:21:f4:f9:7f:
    bd:fe:1e:cd:35:97:99:89:c7:0a:08:34:ad:00:01:
    e1:e1:c5:59:d7:b7:09:3d
exponent2:
    00:87:f8:a4:9a:b9:9e:0c:c4:b2:6a:94:ec:07:4a:
    24:46:30:b2:f4:b5:24:e9:cd:79:7c:d0:85:41:cf:
    0c:fb:f1:b6:46:7e:68:c4:a9:95:22:e5:05:92:e5:
    1c:72:74:9f:8f:66:fd:a7:f2:36:0d:72:c9:a6:09:
    2b:50:ee:5e:ad:f5:cc:5f:22:b7:3c:7a:d9:b2:0e:
    ab:6d:e7:4d:62:4e:70:11:2b:e3:be:57:49:c0:c9:
    5f:9e:8d:46:a2:e8:32:fa:00:d6:60:23:bc:26:8a:
    2f:32:54:88:75:a4:58:d8:ed:f7:49:de:a0:f7:ec:
    40:a6:6b:0c:94:7f:16:7e:65
coefficient:
    46:53:1a:19:21:2d:fe:36:73:85:3a:71:a2:84:fa:
    5c:35:76:db:10:f0:34:86:14:a9:70:1c:d8:73:df:
    2b:d0:43:d9:73:d2:26:1f:b2:ba:9d:82:57:9b:4c:
    cd:4a:bc:f2:1f:fa:f5:5b:2a:8d:6c:20:39:db:f8:
    55:19:11:15:b4:a9:a1:00:dc:93:31:53:c5:c6:17:
    cb:36:e9:00:d8:6e:c8:75:69:6a:85:4d:42:01:2a:
    86:36:59:fc:d7:0b:57:8a:76:7d:c0:bc:43:8a:4d:
    b1:14:07:4d:28:5a:ec:30:28:93:6e:68:14:1a:c0:
    17:6c:e1:7c:39:19:e6:d8
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAt+l0S0X6piDTHDDpY4bpzV+5k97KRcnWCJT3fbnuqdB4RXaU
gJ33BSTXMOLADwRuYFMjvVADvyypu7RcxRFaHc4lfUIDT34cej4aaOiaABCNGCis
Jr1xrkrJuSMLm8EBZ0apAV5w8dm9f1ZLl2Fk/8HZbpOrQGbVy/QC9fxTEVGpgFwH
FqvLmCX+AvOJfleRemTMLHpx6IMzWQqpWSPPSmvkJBr3jKkEXWW2dIcZQknjaQPd
pMl1/qc8B8GRZ1RF/l/PRXL4vUeVuoGnVFBVKZIvgYJxm0Mc6ycWyofiuoOgHoXv
deRjiC0LU3a2s9ZoGeJsK2dPCp3e/pNCQ86HrQIDAQABAoIBAGe/yD4qlRKg09dE
dHUUB9M23C7h8QPbr+WZe+CuQkgD9cVh9rZz5oU9WjQWxrfyDP5ECJZkjCiN3pao
UelON6M2xwlZcxqmDxSa8jUaer3smFv3nd4g4v+q6w+JCKRuBgen4fGGwHp/Fhq+
qNgWNm7dgXaS0Xn8SUHMPttb49SRYvomBBsfXivUfLJoXOxA8sbig31x4Pn6QSSN
NhzoxAU4qyM30cjGreXemNUKfqq6VFpj1eFJvoQN3mZxeMjfXFOI6gDBja+BnwrE
oG6w99qhInJGZWg7JLeJ0M7eOvPVB5TXF5zHkHd7bHohcuUXJYqEIAEbm+WWXBcL
3oU4z5UCgYEA39EYY28sGxQ7lT1W3VtvAW75TCvL692JsI7d8/AP93F10H6wvmtR
OozPm4H6NE/8mNZlsS+CVajRtRqve9pvAkMUfmlSEZ6plUSxGX97fGcwEWeInbG1
bmppF2RX3F79M66E3CpzVI2+BMdvbn8x8QSza/efYhWwdRKoTwJKy9cCgYEA0lts
naSbqW4pZz/RwlMaUFsKfMFkAf8gIIxoMrEa+45zVCk75/yUD2MGoodWpONIdSG3
EJf8Oh2z4qvmSwSN3vus0+Q9cSDaBKrimPTjbPT6w6cfczBPPP6KPiEG5wK05Z1O
N2RnFfNkc88wLVQZdiLuRLXLj5Sg9RWRaFbMOBsCgYB+M88GsncyRbRbMJ08cAQl
0MdttfxkYST0k3p/xEucgTOnfuh2VtkUpLWjwCSvPrL2E16ADIP3fRvSfNuagM67
fcuehBCsssR40KTz9bhRq3WlOrYEBWKCgioD8KbCMiWf8LYl1yH0+X+9/h7NNZeZ
iccKCDStAAHh4cVZ17cJPQKBgQCH+KSauZ4MxLJqlOwHSiRGMLL0tSTpzXl80IVB
zwz78bZGfmjEqZUi5QWS5RxydJ+PZv2n8jYNcsmmCStQ7l6t9cxfIrc8etmyDqtt
501iTnARK+O+V0nAyV+ejUai6DL6ANZgI7wmii8yVIh1pFjY7fdJ3qD37ECmawyU
fxZ+ZQKBgEZTGhkhLf42c4U6caKE+lw1dtsQ8DSGFKlwHNhz3yvQQ9lz0iYfsrqd
glebTM1KvPIf+vVbKo1sIDnb+FUZERW0qaEA3JMxU8XGF8s26QDYbsh1aWqFTUIB
```

```
KoY2WfzXC1eKdn3AvEOKTbEUB00oWuwwKJNuaBQawBds4Xw5GebY
-----END RSA PRIVATE KEY-----
```

1608 v3 **Public Key Types**

Version history:
(3): Clarification

Purpose: To prepare the interfaces off-board of future public key types and to be able to control the diagnostic clients how to install the public key.

Source:

Verification Method: Analysis

Legacy ID: 442538v1

Information about the public key type must be provided by the entity managing the keys. That information is optionally used to control how the diagnostic client shall program the public key, e.g. what diagnostic service(s) that shall be applied.
The Public key type (PublicKeyModulus||PublicKeyExponent||PublicKeyChecksum) shall be referred to as "RAW_KEY_WITH_CHECKSUM".

This requirement is not applicable for the ECU supplier but for off-board systems.

4.4.3.1.2 Signing Methods

4.4.3.1.2.1 Signing Method - Variability control

Different off-board systems must know: If, when, how etc. software for an ECU shall be signed. This concept uses a dedicated software part, referred to as Software Settings File, to carry this information.

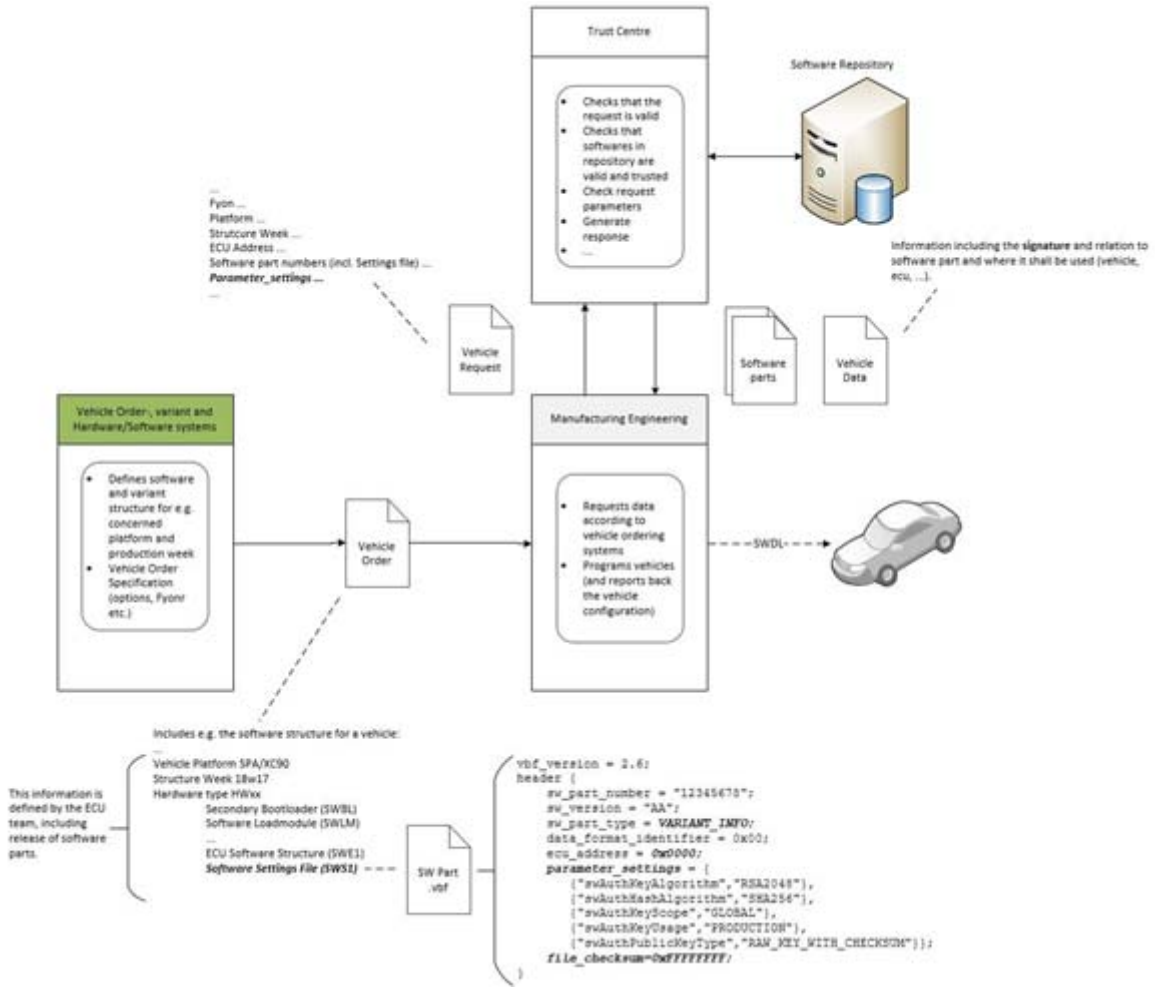


Figure - Software Authentication variability management

The *Figure - Software Authentication variability management* shows a simplified view (e.g. details about information flow is omitted) of how to define and distribute needed variant information.
(i) A separate software part, a Software Settings File, with a specific identifier parameter_settings is applied.
(ii) The Software Settings File is version handled and released as any other software part.
Refer to section "Software Settings File" for further information about that format.

1599 v4 **Signing Method - Variability control**

Version history:
(4): Clarification

Purpose: To control the variability at the back-end systems but also to inform the ECU of the signing method (at design time) that shall be applied. Note that a specific ECU does not have to support all methods, but only one.

Source:
Verification Method: Analysis
Legacy ID: 404595v3

To define and distribute the Signing Method, a Software Settings File (a software part) shall be used for methods where each software part is signed individually.

Note: The notation <HashAlgorithm_SigningAlgorithm_SigningObject>, e.g. SHA256_RSA2048_sw_part is used in nextcoming requirements to define algorithms to be applied. The "sw_part" is used to seperate this method from legacy concepts. See also section "Software Settings File" actual definitions.

1598 v2 Default Signing Method

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Define the default signing method. The ECU shall use the default signing method, unless else is explicitly defined elsewhere, e.g. in the LC : Diagnostics and ECU Platform.
Source:
Verification Method: Analysis
Legacy ID: 410732v0

The default signing method shall be SHA256_RSA2048_sw_part_based.

4.4.3.1.2.2 Signing Method - Algorithm Definitions

1594 v2 Signing Method SHA256_RSA2048 - Hash function

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Define the hash function.
Source: FIPS PUB 180-4, Secure Hash Standard (SHS) FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION, March 2012.
Verification Method: Analysis
Legacy ID: 404598v0

The hash function to be used is the SHA-256 (256 bits) as defined in *FIPS PUB 180-4, Secure Hash Standard (SHS), March 2012*.

1596 v3 Signing Method SHA256_RSA2048 - Public Key Algorithm

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define the public key algorithm.
Source: PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012
Verification Method: Analysis

Legacy ID: 404620v1

The public key algorithm shall be: *PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012*, where the RSASSA-PSS signature scheme shall be used. The signature scheme includes a salt when generating a signature. The salt length shall be set to the same length as the underlying hash value. This means that the resulting signature will be different every time a file is signed.

1595 v2 Signing Method SHA256_RSA2048 - Key Size

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Define the key size for the private- and public key.
Source: PKCS #1 v2.2: RSA Cryptography Standard, RSA Laboratories, October 27, 2012
Verification Method: Analysis
Legacy ID: 404623v0

The key size shall be 2048 bits. This is the size of the modulus part of the key representations.

4.4.3.1.2.3 Signing Method - Key balancing

The software authentication concept must support different strategies with respect to key management and key balancing. Three main security classes with respect to software authenticity are defined in *Table - Software Authentication security classes*. After a threat and risk analysis the proper security class shall be defined and applied for ECUs in the vehicle platform(s). Based on that decision, the affected variant management systems shall be updated accordingly. Hence, both the signing method and key usage must be configurable. In case of an incident, or if a signing key for other reasons must be changed, the key must be able to change (when global keys are used).

1597 v3 Software authentication security classes

Version history:
(3): Clarification

Purpose: To align the terminology but also describe the expected possibilities of how to balance the key usage. This variability must be managed in the variant handling systems off-board.
Source:
Verification Method: Analysis
Legacy ID: 441876v1

The software authentication security classes as defined in the *Table - Software Authentication security classes* must be supported with respect to key management. This means that it shall be possible to configure the key usage in the off-board variant management systems:
1) Integrity check only, i.e. no cryptographic keys needed
2) ECU global keys used for a period of time. This shall be applied by default.
3) ECU unique keys

Security Class – Software	Meaning

Authentication	
SwAuth Class 1	Integrity check only, i.e. only using checksums/hash values, for error detection. This is not part of the General Software Authentication document, but added here for legacy reasons.
SwAuth Class 2	Authenticity (origin + integrity), using common/global keys. A dedicated signing key per ECU type shall anyhow be applied, e.g. one for all door modules and another for the climate control respectively (for segmentation) The key used shall be possible to change after a period of time (structure week) and (optional) vehicle type.
SwAuth Class 3	Authenticity + “copy between vehicles protection”. An ECU unique key shall be applied. This class is recommended for sellable features but also for risk reduction (e.g. a key is revealed). SwAuth Class 3 shall always be applied when there is device unique keys for diagnostic service securityAccess, for programming services.

Table - Software Authentication security classes

4.4.3.1.2.4 Software Settings File

The Software Settings File (SWS1) is used to provide different systems in a controlled manner with the information required to sign a software part. This file is version handled in the systems as any other application software part. See *Figure - Software Authentication variability management.*

130979 v1 **Software Settings - Definition**

Version history:

Purpose: To define the format of the software settings file, used to distribute variant information.

Source:
Verification Method: Analysis
Legacy ID: 456934v0

The software settings file shall be based on the *[LC : Versatile Binary Format Specification]* and the requirements specified there, and contain:
(i) The identifier sw_part_type shall be set to VARIANT_INFO
(ii) The identifier parameter_settings with valid definitions
(iii) The ecu_address shall be set to 0x0000
(iv) No data blocks. That means that the file_checksum will be 0xFFFFFFFF

The valid "name" / "value" definitions for paramater_settings are:
swAuthKeyAlgorithm : RSA2048, RSA4096
swAuthHashAlgorithm : SHA256, SHA384, SHA512
swAuthKeyScope : GLOBAL, NODE_UNIQUE
swAuthKeyUsage : TEST, PRODUCTION
swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM

Example - parameter_settings:
parameter_settings = { {"swAuthKeyAlgorithm","RSA2048"}, {"swAuthKeyScope","GLOBAL"}, "swAuthPublicKeyType","RAW_KEY_WITH_CHECKSUM"}, {"swAuthHashAlgorithm","SHA256"} };

Notes:
(i) The reason for using a general ecu_address is to enable using the same settings file partnumber for many different ECUs that will use identical parameter_settings definitions.
(ii) The swAuthPublicKeyType is shown in "Figure - Public Key Types", where the Type 1 is the RAW_KEY_WITH_CHECKSUM.
(iii) The vbf version to be used is project dependent and to be specified elsewhere.

130980 v1 **Software Settings File - valid part numbers**

Version history:

Purpose: Define software settings file that can be globally used, i.e. used by many ECUs. These are managed by the OEM.
Source:
Verification Method: Analysis
Legacy ID: 458432v0

The software settings files according to "*Table - SWS1 part numbers*" are to be used, unless explicitly defined elsewhere (e.g. due to specific project and ECU needs):

Part number	Settings
32218509	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : GLOBAL swAuthKeyUsage : TEST swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
32218510	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : GLOBAL swAuthKeyUsage : PRODUCTION swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
32218511	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256 swAuthKeyScope : NODE_UNIQUE swAuthKeyUsage : TEST swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM
32218512	swAuthKeyAlgorithm : RSA2048 swAuthHashAlgorithm : SHA256

	swAuthKeyScope : NODE_UNIQUE swAuthKeyUsage : PRODUCTION swAuthPublicKeyType : RAW_KEY_WITH_CHECKSUM

Table - SWS1 part numbers

4.4.3.1.3 Appendix

4.4.3.1.3.1 Example - Data format identifiers and signing (sw_part_based)

Example of a software part with identifiers used for software authentication, where each part is signed individually using the development key.

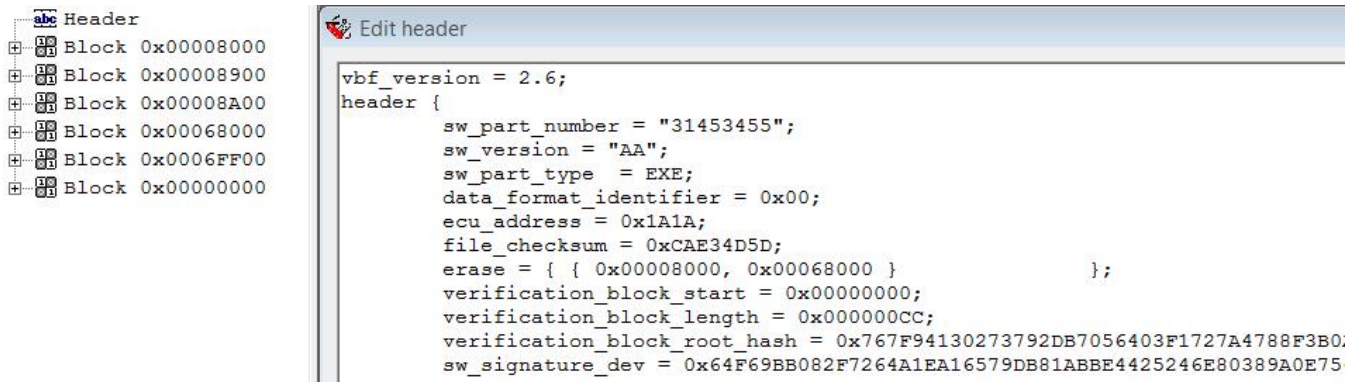


Figure - Example VBF version 2.6

Note that the verification block here starts at address 0x00000000 (verification_block_start), example only.

At the time of this release, the format used by the trust centre to distribute the sw_signature for production to diagnostic clients is TBD. But, the transfer to the ECU will still be by using the routine control CheckMemory.

4.4.3.1.3.2 Example - Software Part evolution

An overview of the evolution of a software part from ECU Software Supplier to Trust Centre is shown in *Figure - Software Part evolution*.

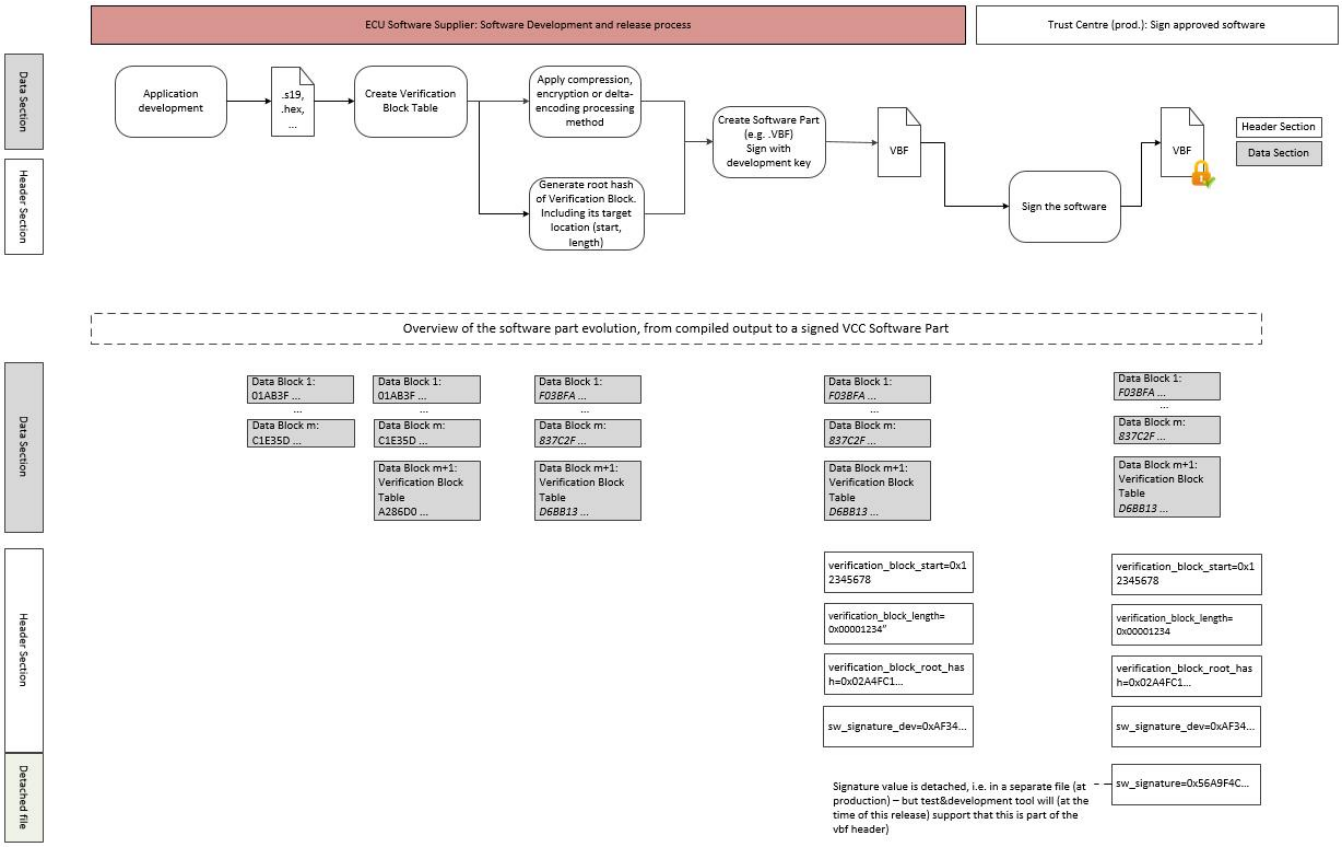


Figure - Software Part evolution

4.4.3.1.3.3 Example - Signing function when the processing method is unknown

Example.
The ECU supplier has used an encryption method where the secret key is not known to the Trust Centre.
The Trust Centre must rely on and sign the software part by using the Verification Block Root Hash value (that is calculated using unprocessed data as input). See *Figure - Signing using Verification Block Root Hash*.

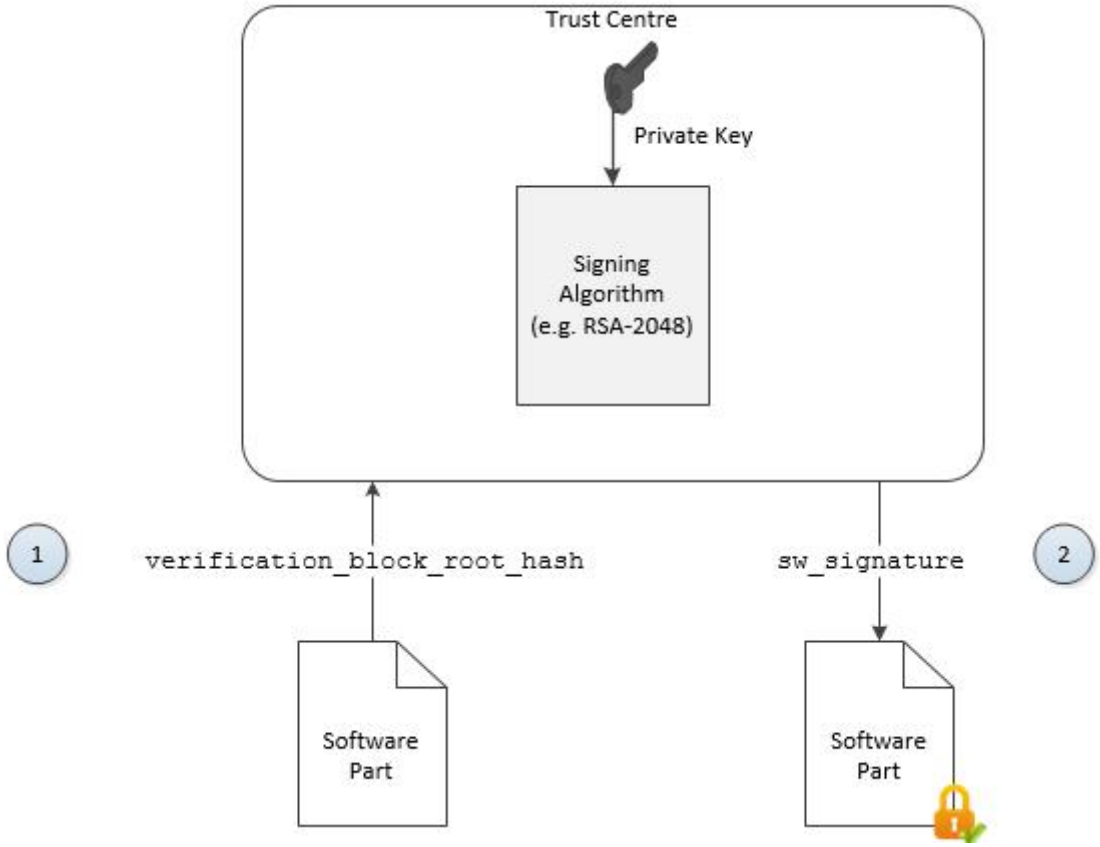


Figure - Signing using Verification Block Root Hash
(1) The hash value (verification_block_root_hash) is transferred to trust centre for signing. Note that the start address and length are included when calculating the hash value.
(2) The resulting signature (sw_signature) is appended to the software part.

The processing method is to be defined by the data_format_identifier *[Versatile Binary Format]*.

4.4.3.1.3.4 Example - PublicKeyCheckSum

The purpose of this example is to show how to calulate the PublicKeyCheckSum, when SHA-256 is used. For simplicity, the previously described initial development key is used as input data for hash calculation.

PublicKeyCheckSum = Hash(*PublicKeyModulus*||PublicKeyExponent) =
Hash(*BF E4 B1 DD 88 BC F8 A4 CE 35 E1 B2 52 1F 33 D3 1B 53 1B 03 82*
8D 69 0A 86 AC D5 24 79 0C 62 D2 70 A6 78 08 2E FF 53 86 EE C7 65 E6
B4 20 0D AE 62 4C 41 23 52 BD B8 3B 92 1C FF 64 E9 25 2A 93 4F ED ED
97 3D 0D 85 D0 85 4A A7 8B 37 68 48 67 BB A7 54 CE 6D 04 52 E9 D9 44
B7 1E AA D2 1F CE 24 C6 CF 35 95 40 8F 89 0A 99 13 59 78 CB 34 18 48
E1 CC 1D 92 1A 97 5F 77 E0 38 D7 92 0D 79 85 AF 9A AA 1B 8D 55 C7 D9
66 15 72 23 4F 56 8C 80 36 15 F3 33 0A 5B 7F 96 C8 36 49 F4 2A A2 B1
0F C3 B0 60 C4 FD E3 E2 39 1C 78 C3 1F B3 40 58 76 AF 97 30 3C 53 5E
AE 7C 71 63 25 AE 94 84 3D 5C B1 88 26 95 4B D1 C9 1A E9 99 9A 4E FE
30 2D C3 6E BD C1 F4 9D EA 97 E8 73 58 C3 31 27 08 42 69 FD AE 5D 04
C5 96 64 FC AC 9E F0 AA 8B 02 B1 30 B6 D1 F3 F4 07 E1 C0 F3 A7 1C B8
0D EF 94 9E 11 00 01 00 01) =

```
63 0B 45 B6 43 C3 E1 55 59 CB FA 73 7C 46 81 6F 47 7A 87 FC CD 1E A5
E5 76 BF 86 09 CD 08 B4 87
```

Programming using the public key DID:
WriteDataByIDentifier_PublicKey(*PublicKeyModulus* || PublicKeyExponent || PublicKeyCheckSum) =
WriteDataByIDentifier_PublicKey(*BF E4 B1 DD 88 BC ...* 00 01 00 01 63 0B 45 ... B4 87)

4.4.3.1.4 Definitions and Abbreviations

Definition	Description
Authentication	Authentication is the process of assuring and verifying authenticity.
Authenticity	The property of being genuine and being able to be verified and trusted.
Digital Signature	Used to assure originator authenticity and data integrity (of a software)
Hash	Hash algorithms are also called digital fingerprinting algorithms. They are irreversible functions that provide a fixed-size hash based on various inputs. Irreversibility and collision resistance are necessary attributes for successful hash functions. Examples of hash functions are Secure Hash Algorithm 2
Integrity	The property that specified information remains unchanged between observations
Key-pair	The key-pair is used with a public key algorithm. The private key, which is secret, and the public key, which is publicly available. The private and public keys are cryptographically related. The private key cannot be derived from the public key. The private key can be used only by its owner, and the public key can be used by third parties to perform operations with the key owner.
Trust Anchor	The public key of trusted entity (root Certificate Authority certificate or the public key only)
Trust Centre	A trusted party of the public key infrastructure, including key management systems, with a central authority.

Table - Software Authentication Definitions

Abbreviation	Description
ECU	Electronic Control Unit
PKI	Public Key Infrastructure. See definition of Trust Centre
PKIX	Public Key Infrastructure X.509
SW	Software
SWDL	Software Download
VBF	Versatile Binary Format.
VBT	Verification Block Table.

Table - Software Authentication Abbreviations

4.4.3.1.5 Requisite documents

The following documents are either referenced by this specification, or contain information that is relevant to this specification.

Source	LC in Elektra
Internal	Versatile Binary Format (VBF)
Internal	General Bootloader requirements

Internal	Software Download and Upload sequences
Internal	Software Authentication for file based systems
Internal	Software Authentication*

* Added here for reference. This is applicable for legacy ECUs not supporting CheckMemory.

The Software Authentication concept includes a list of specifications that has a tight relation to each other, the *"Figure - Software Authentication specification component structure"*, shows the overview and their relation.

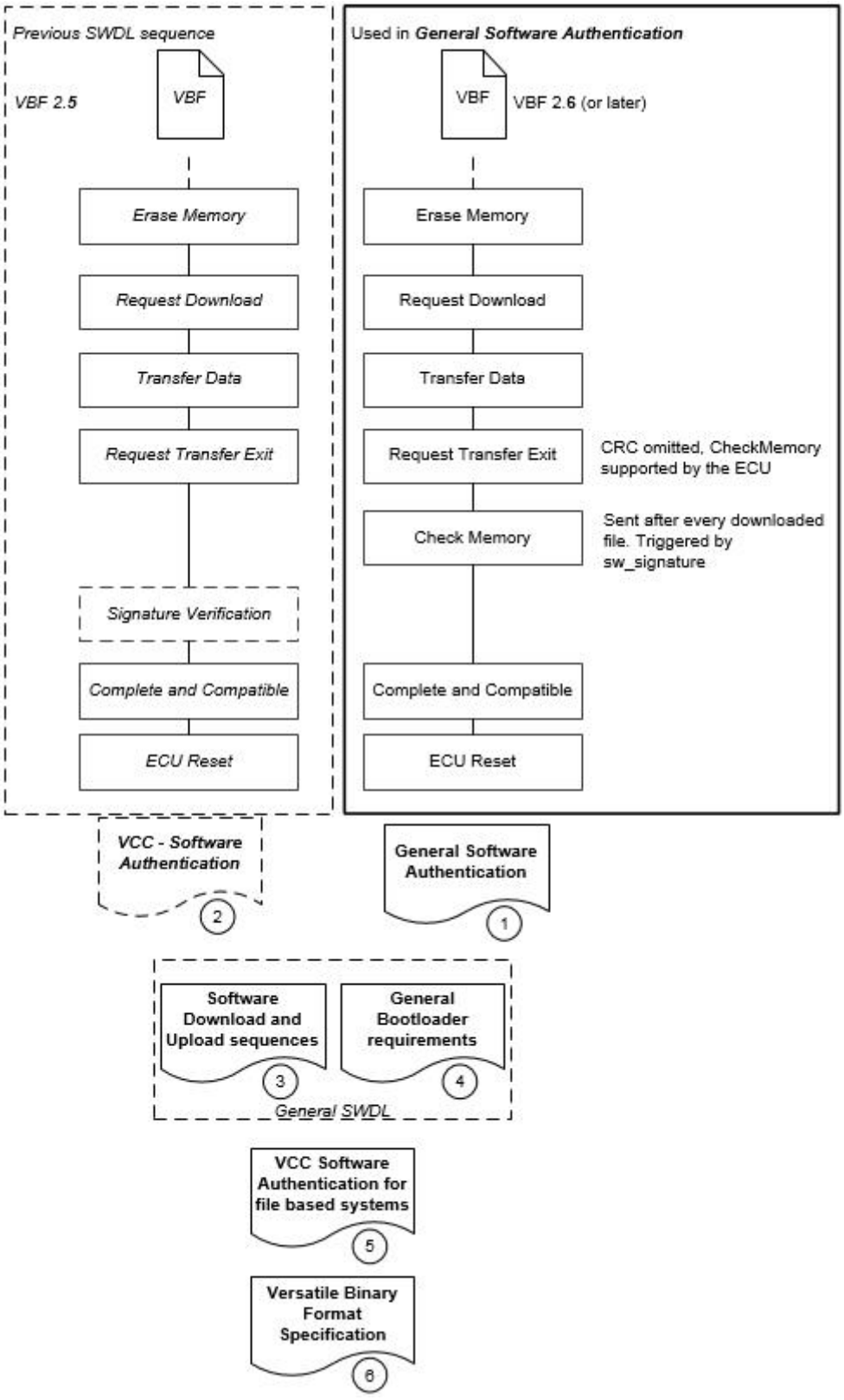


Figure - Software Authentication specification component structure

1 - The *"General Software Authentication"* is defining the software authentication purpose and system overview, together with the applicable signing methods and algorithms.

2 - The " *Software Authentication*" is defining a legacy signing method (for ECUs not supporting

CheckMemory), where the complete set of software parts are signed and the single signature is transferred (downloaded) to the ECU in a dedicated software part. Added here for reference only.
3 and 4 - The *"Software Download and Upload sequences"* and *"General Bootloader requirements*" includes the default software authentication method requirements for the client and server respectively, where each software part is independently signed and the signature is transferred in a CheckMemory routine control. Introduced together with the data file format version VBF 2.6. Note: There is a dedicated " *General Bootloader requirements*" version for ECUs not supporting CheckMemory.
5 - The *"Software Authentication for file based systems"*, defines the requirements when code signing certificates are used in the ECU.
6 - The "*VBF (Versatile Binary Format*)" data file format. VBF 2.6 is required when signing methods using CheckMemory are applied.

4.4.3.1.6 Introduction

The purpose of Software Authentication is to provide the following security services in order to protect the software in the vehicle:

- Authentication (proof of origin): the recipient ECU of the downloaded software shall be confident that the software originates from a legitimate source.
- Integrity: the recipient ECU shall be able to verify that the original software has not been altered (e.g. manipulated by an unauthorized user) in transit.
- Copy Protection: it is used to prevent unauthorized feature activation, by e.g. copying a software from one ECU to another. This is not the same as encryption, for confidentiality. Copy protection is realized by key management.

The two first terms, Authentication (proof of origin) and Integrity, are commonly referred to as Authenticity. It is accomplished by the use of public-key algorithms (asymmetric algorithms), where the key (private key) used to encrypt the software is separated from the key used to decrypt it (public key). The private (secret) key is exclusively known to the originator of the software, thus this key is used for signing (generating a digital signature). The recipient(s) having access to the public key is able to verify the downloadable parts. The software together with the signature shall be sent to the recipient ECU. Standard code signing technology relies on asymmetric cryptography in the form of public key infrastructure (PKI) and provides a well proven quality history.

To summarize: Software Authentication concept is applied to ensure that the software being executed in the vehicle is authentic by:

- Secure download/flashing (prevent "malware" software by verifying the downloaded software part). Focus of this specification.
- Secure boot (prevent from booting modified and unauthorized software)

It does however NOT cover e.g. a secure delivery process of complete file from software supplier to the OEM but describes how to sign the data blocks of a software part once those criteria(s) for signing it are fulfilled. Hence, conditions to ensure that a software part is valid and trusted prior to it is stored in software archives, distributed to e.g. cloud etc. or for performing the actual signing of the data blocks are to be specified elsewhere.

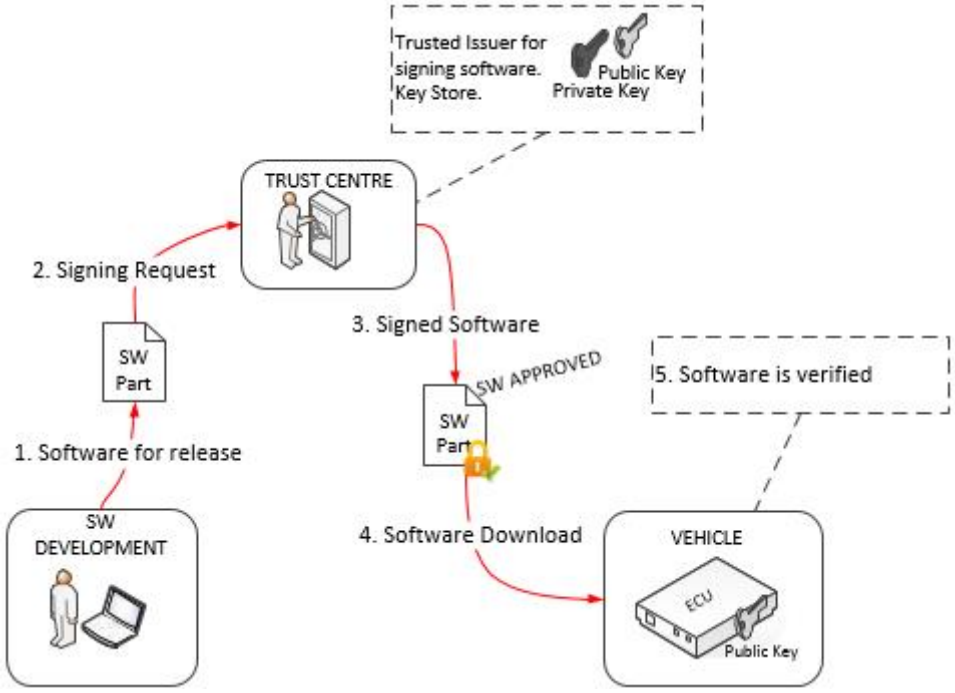


Figure - General Signing model
1. The software is developed and released.
2. The software is sent to the trust centre for signing.
3. The software is signed using a private key and by that a signature (signed software) is generated by the trust centre.
4. The software including signature is stored and transferred (software download) to the ECU.
5. The software stored in the ECU is being verified by using the public key. In order to perform SWDL, authentication of the tester tool is required using Security Access.

In a vehicle the ECU capabilities differs, i.e. some are "high-performance" ECUs but some are having less capabilities. Based on that, the Software Authentication concept has currently only support for a simplified key structure for signing/verifying a software part:

One layer of keys ("raw private/public key pairs") is applied, where only the raw public key data with minimum metadata is used for verification, i.e. no key chain verification as for a more standardized certificate verification of multiple levels (root, policy and issuer authorities). Still, certificates could be used for key management purposes (storage, distribution) and when signing.

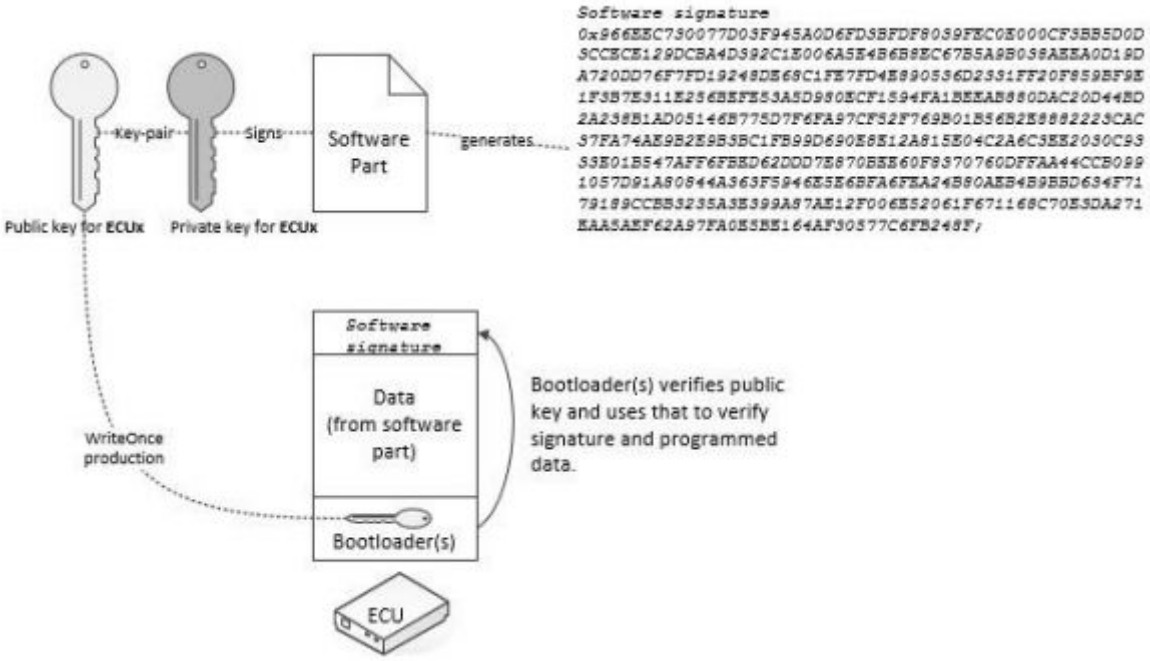


Figure - One-Layer key structure

The figure below gives a brief overview of concerned systems for the Software Authentication concept.

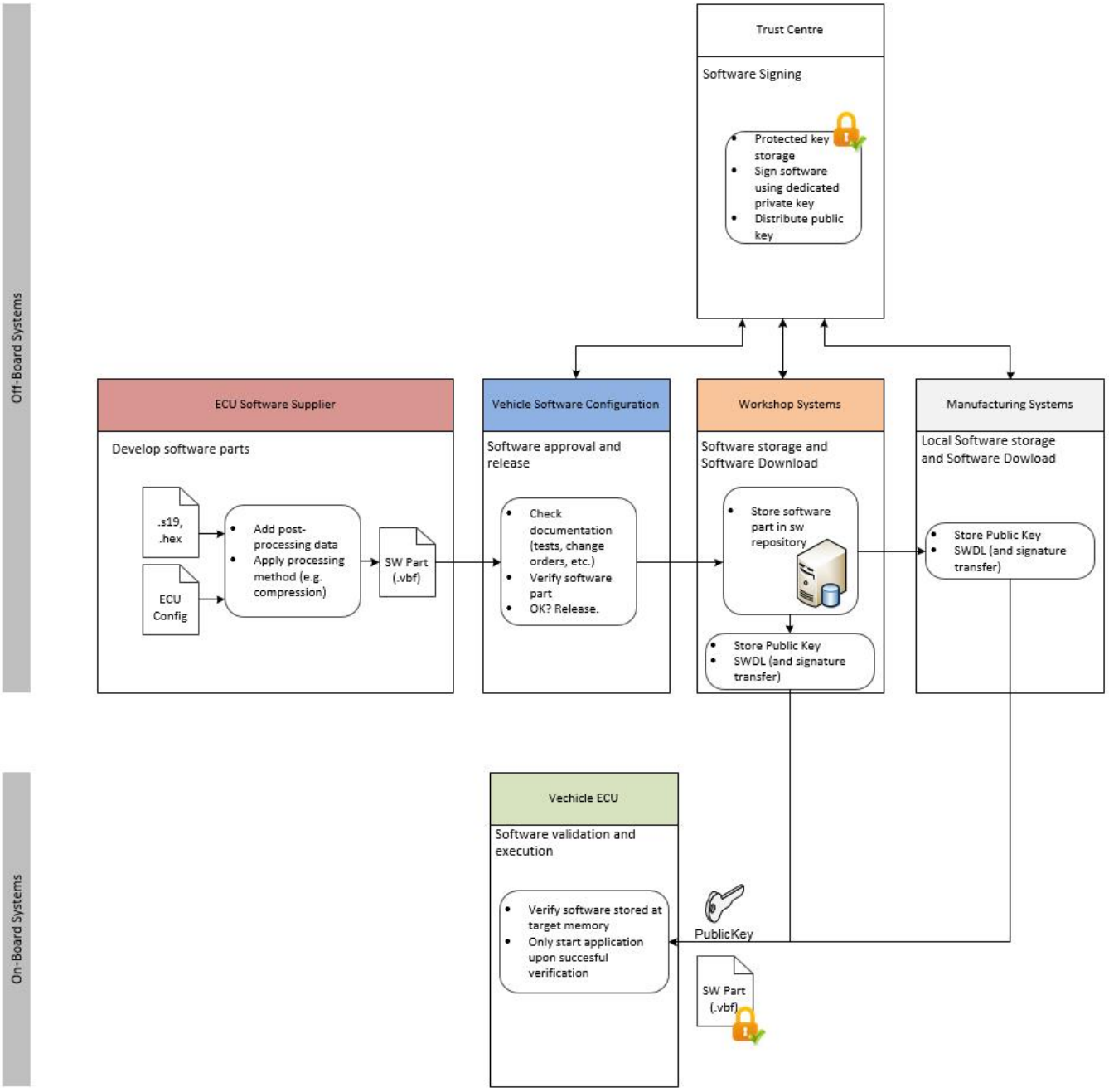


Figure - Software Authentication concerned systems

Figure - Software Authentication concerned systems is a simplified view, where details are omitted (e.g. temporarily software repositories at manufacturing). But, the figures does however illustrate that there are several systems involved which will result in the fact that several interfaces must be defined.

A typical software signing function is described in *Figure - Software signing function overview*. The data that is signed together with the methods (hash function and signing algorithm) must be known to both the signer (Trust Centre) and verifier (ECU). A hash value (digest) is calculated on the data and that hash is being encrypted with the signer's private (secret) key. This encrypted hash is called a signature. The software part together with the signature forms the signed data that is transferred to the ECU.

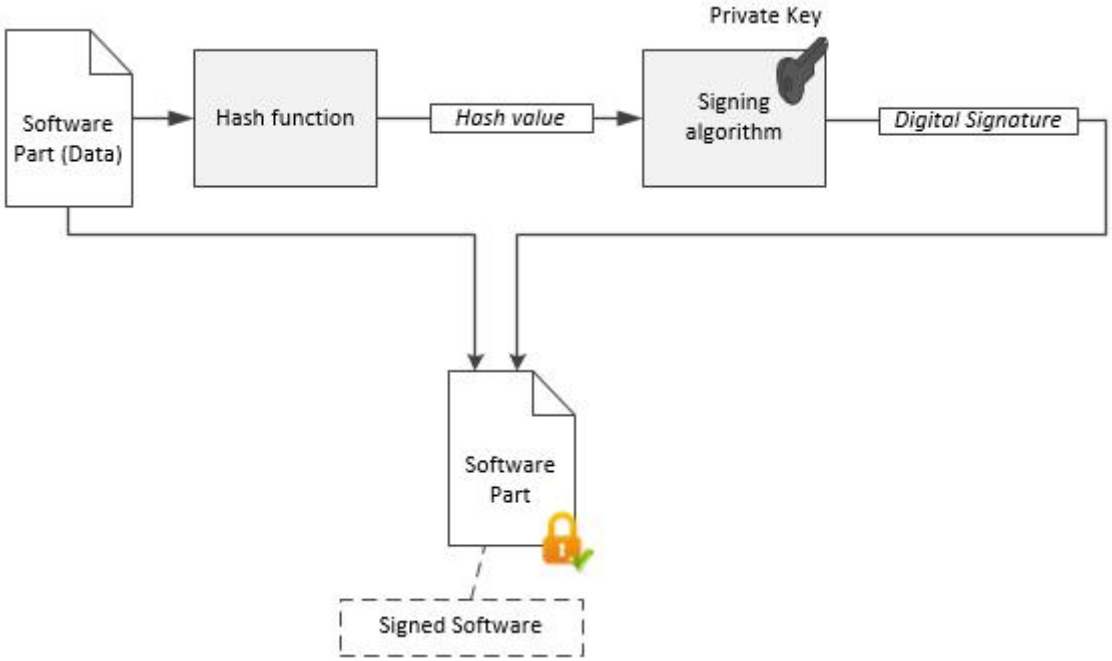


Figure - Software signing function overview

The secret private key is kept protected by the trust centre and must not be exposed, i.e. the data to be signed is an input to the signing function where the private key is stored as shown in *Figure - Trust Centre signing*.

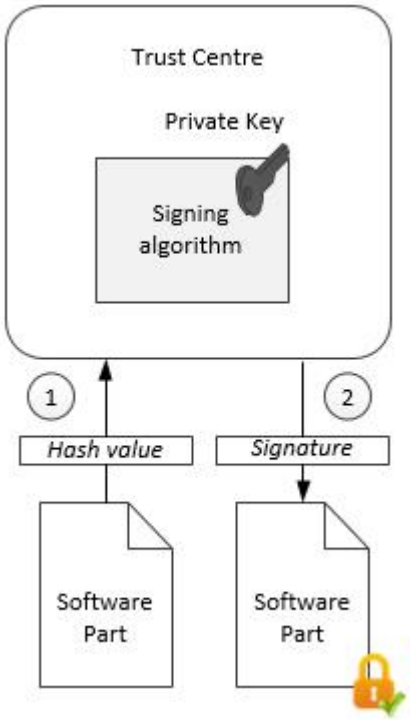


Figure - Trust Centre signing

The software authentication concept must support different signing methods, e.g. for future

growth (change of key lengths, algorithms, etc.) but also for legacy reasons. This variability must be handled in a controlled manner at the off-board systems and because of that the term **Signing Method** is introduced. See chapter *Signing Methods* for further details.

4.4.4 Privacy

4.4.4.1 Privacy in all ECUs

4.4.4.1.1 Requirements

6436 v1 **Privacy Regulations**

Version history:
(1):

Purpose: Privacy Regulations shall be followed
Source:
Verification Method: Documentation
Legacy ID:

All systems storing any personal data shall adhere to privacy regulations or policies applicable (e.g. GDPR, China laws or NHTSA) on the targeted markets.

6437 v2 **Privacy as Default**

Version history:
(2):

Purpose: Prevent personal data to be exposed
Source:
Verification Method: Inspection
Legacy ID:

The main guidelines when developing systems are “privacy as default” and “privacy by design” to prevent data exfiltration and ensure the privacy and integrity of sensitive information.

Whenever it is possible, personal data shall be anonymized or de-identified and encrypted.

It shall not be possible to access other person’s personal data and it shall be possible to erase all personal data on request to eliminate any footprints e.g. when a car is sold or shared.

Any administrator access to personal data shall be restricted to a minimum and actions logged.

6438 v2 **Purpose and Consent**

Version history:
(2):

Purpose: Only necessary data collected, consent given and right to be forgotten
Source:
Verification Method: Documentation

Legacy ID:

A basic principle is that only necessary personal data shall be stored and the purpose for collecting the data must be clearly expressed.

The contractor shall be able to present a list of what data is collected and the purpose must be clearly expressed.

Consent for data collection as well as the possibility to erase personal data from the system is mandatory and shall be considered in the system design.

4.4.4.1.2 Introduction

The protection of personal data is of great concern and applicable for any system using or storing personal information. "Personal data" means any information relating to an identifiable natural person. An identifiable person is one who can be directly or indirectly identified by information.

An identifiable person is one who can be directly or indirectly identified by information such as name, identification number, location data or factors specific to the physical, physiological, genetic, mental, economic, cultural or social identity of that person.

Refer to http://www.eugdpr.org/ for further definition of personal data

4.5 Diagnostics

4.5.1 UDS Data

4.5.1.1 UDS Data

4.5.1.1.1 Requirements

4.5.1.1.1.1 Requirements from section Diagnostic Data

2093 v2 **Compliance to ISO14229-1**

Version history:
(2):

Purpose: The diagnostic strategy is to comply with the general ISO diagnostic specifications.
Source: Section "Diagnostic Data"
Verification Method: Analysis
Legacy ID: 67741v1

All diagnostic data supported by the ECU shall be compliant to *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications . The ECU is

not allowed to support other diagnostic data than specified in section 4 and in each implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP) and by reference *[Data_53] Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 1:* General information , UDS - Data - External publications and *[Data_54] Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 2: Communication requirements* , UDS - Data - External publications unless it is approved by the responsible authority appointed by OEM.

2096 v2 **Diagnostic data for private ECUs**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Specifiy that also private ECUs shall support diagnostic data
Source: Section 4 - DIAGNOSTIC DATA REQUIREMENTS
Verification Method: Analysis
Legacy ID: 174453v0

Private ECUs shall, from a diagnostic point of view, be regarded as if they were internally connected units (e.g. internal I/O-units reading sensors and controlling actuators) within the public ECU master itself. The public ECU master shall, on behalf of the private ECUs, support all the diagnostic data required by this document and that are applicable for the private ECUs.

2094 v1 **Compliance to ISO26021-1**

Version history:
(1):

Purpose: Diagnostics on safety systems must comply with regulations defined by relevant ISO specifications.
Source:
Verification Method: Analysis
Legacy ID: 67742v0

The diagnostic data supported by the ECU in the safetySystemDiagnosticSession shall be defined as specified by reference *[Data_53] Road vehicles - End-of-life activation of on-board pyrotechnic devices — Part 1: General information and use case definitions* , UDS - Data - External publications and *[Data_54] Road vehicles - End-of-life activation of on-board pyrotechnic devices - Part 2: Communication requirements* , UDS - Data - External publications.

2095 v1 **Compliance to ISO26262**

Version history:
(1):

Purpose: Implementation of diagnostics on safety related ECUs must comply with ISO 26262.
Source:
Verification Method: Analysis
Legacy ID: 67743v0

For all ECUs implementing safety requirements with an ASIL higher than QM, it shall be ensured that the diagnostic functions required in this specification cannot violate any of those safety requirements, i.e. to demonstrate independence as required by *[Data_55] Road vehicles - Functional safety , International Organization for Standardization, 2011* , UDS - Data - External publications . Exception to this requirement is allowed only when approved by the responsible authority appointed by OEM and will require that the diagnostic functions required in this specification are developed according to *Road vehicles - Functional safety , International Organization for Standardization, 2011* , UDS - Data - External publications and the allocated ASIL of those safety requirment for wich independence cannot be demonstrated.

4.5.1.1.1.2 Requirements from section Record Data - General

2290 v4 **Vehicle manufacturer specific data records defined in GMRDB**

Version history:
(4):

Purpose: Mandatory data records in are defined in GMRDB.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67761v3

Data records with data identifiers in the ranges 0x0100 to 0xD8FF, 0xDE00 to 0xE2FF, 0xE600 to 0xED1F, 0xED80 to 0xED9F and 0x F010 - 0xF0FF shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

2275 v5 **Access to Vehicle manufacturer specific data records**

Version history:
(5):

Purpose: To ensure that data records can be accessed in the required diagnostic sessions.
Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 67762v4

It shall be possible to access the vehicle manufacturer specific data records with data identifiers in the ranges 0x0100 to 0xD8FF, 0xDE00 to 0xE2FF, 0xE600 to 0xED1F, 0xED80 to 0xED9F and 0xF010 to 0xF0FF, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which services used for access the data records is supported, except programming session. Data items (e.g. external and internal signals) specified in ECU external signals, ECU internal signals, ECU Vehicle status, Network data and Local DTC snapshot data shall be defined in records with data identifiers in this range.

2268 v4 **Access Development specific data records - implementer specified**

Version history:
(4):

Purpose: To ensure that the data records can be accessed in the required diagnostic sessions.

Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 119101v3

It shall be possible to access the Development specific data records (if supported by the ECU), with data identifiers in the ranges 0xD900 to 0xDCFF, E300 to E4FF and 0xEE00 to 0xEFFF, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions except in programmingSession.

2278 v3 Development specific data records - implementer specified

Version history:
(3):

Purpose: ECU Specific data records are not supported by OEM specific tools and must be defined in a separate identifier range.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67763v2

If data records that are needed only during the development of the ECU are defined by the implementer, these data records shall have data identifiers in the ranges 0xD900 to 0xDCFF, E300 to E4FF and 0xEE00 to 0xEFFF.

2267 v1 Access development specific data records

Version history:
(1):

Purpose: To ensure that the data records can be accessed in the required diagnostic sessions.
Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 392596v0

It shall be possible to access the development specific data records (if supported by the ECU), with data identifiers in the ranges 0xE500 to 0xE5FF by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions in which services used for access the data records is supported, except programming session.

2277 v2 Development specific data records

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Development specific data records are not supported by OEM diagnostic tools and must be defined in a separate identifier range.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 392594v0

Data records needed only during the development shall be implemented with data identifiers in the range 0xE500 to 0xE5FF exactly as they are defined in *[Data_4] Global Master Reference Database*.

2279 v1 **DTC Global Snapshot Data records defined in GMRDB**

Version history:
(1):

Purpose: Mandatory data records in are specificed in GMRDB.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67764v0

DTC Global Snapshot Data records with data identifiers in the range 0xDD00 to 0xDD7F, shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

2271 v2 **Access to DTC Global Snapshot Data records**

Version history:
(2):

Purpose: To ensure that DTC Global Snapshot Data records can be accessed in the required diagnostic sessions.
Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 67765v1

It shall be possible to access the DTC Global Snapshot Data records with data identifiers in the range 0xDD00 to 0xDD7F, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which services used for access the data records is supported, except programming session. Data items (e.g. external and internal signals) specified in section *Global DTC snapshot data* shall be defined in records with data identifiers in this range.

2282 v3 **ECU Identification and Configuration data records**

Version history:
(3):

Purpose: Mandatory data records in are defined in GMRDB.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67767v2

ECU Identification and Configuration data records with data identifiers in the range 0xEDA0 to 0xEDFF (VCC) and 0xED20 to 0xED7F (Geely), shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

2272 v3 **Access to ECU Identification and Configuration data records**

Version history:
(3):

Purpose: To ensure that ECU Identification and Configuration data records can be accessed in the required diagnostic sessions.
Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 67801v2

It shall be possible to access the ECU Identification and Configuration data records with data identifiers in the range 0xEDA0 to 0xEDFF(VCC) and 0xED20 to 0xED7F (Geely), by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions. Data items specified in sections *ECU/Vehicle identification* shall be defined in records with data identifiers in this range.

2284 v1 Identification Option - Vehicle Manufacturer Specific data records

Version history:
(1):

Purpose: Mandatory data records in are defined in GMRDB.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67821v0

Identification Option – Vehicle Manufacturer Specific data records with data identifiers in the ranges 0xF100 to 0xF17F, and 0xF1A0 to 0xF1EF, shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

2274 v2 Access to Identification Option - Vehicle Manufacturer Specific data records

Version history:
(2):

Purpose: To ensure that Identification Option - Vehicle Manufacturer Specific data records can be accessed in the required diagnostic sessions.
Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 67841v1

It shall be possible to access the Identification Option – Vehicle Manufacturer Specific data records, with data identifiers in the ranges 0xF100 to 0xF17F, and 0xF1A0 to 0xF1EF, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions. Data items specified in sections 4.1.6 shall be defined in records with data identifiers in these ranges .

2283 v1 Identification Option - ISO specified data records

Version history:
(1):

Purpose: ISO defined mandatory data records are also defined in GMRDB.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis

Legacy ID: 67861v0

Identification Option – ISO specified data records with data identifiers in the range 0xF180 to 0xF19F, shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

2273 v2 **Access to Identification Option - ISO specified data records**

Version history:
(2):

Purpose: To ensure that Identification Option - ISO Specific data records can be accessed in the required diagnostic sessions.
Source: Table "Requirements on implementation of data records"
Verification Method: Test
Legacy ID: 67862v1

It shall be possible to access the Identification Option – ISO Specific data records, with data identifiers in the range 0xF180 to 0xF19F, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions. Data items specified in sections *ECU/Vehicle status* and *ECU/Vehicle identification* shall be defined in records with data identifiers in this range.

2285 v2 **Identification Options-System Supplier Specific data records**

Version history:
(2):

Purpose: System supplier specific data records are not supported by OEM specific tools and must be defined in a separate identifier range.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67864v1

If Identification Options data records, that are not for uesd by OEM, are defined by the system supplier, these data records shall have data identifiers in the range 0xF1F0 to 0xF1FF.

2269 v1 **Access Options-System Supplier Specific data records**

Version history:
(1):

Purpose: To ensure that the data records can be accessed in the required diagnostic sessions.
Source:
Verification Method: Test
Legacy ID: 119121v0

It may be possible to access the Identification Options data records, with data identifiers in the range 0xF1F0 to 0xF1FF, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions except in programmingSession.

2280 v1 **Dynamically Defined Data Identifiers in ascending order**

Version history:
(1):

Purpose: To simplify management of data identifiers in the tools.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67868v0

If Dynamically defined DataIdentifier data records are implemented they shall have data identifiers in the range 0xF300 to 0xF3FF and shall use the identifiers in ascending order starting from data identifier 0xF300 (i.e. if the ECU supports only one single dynamically defined DataIdentifier then its value shall be 0xF300, if the ECU supports two dynamically defined DataIdentifiers then the values shall be 0xF300 and 0xF301, etc.).

2281 v1 **Dynamically defined periodic Data Identifiers in ascending order**

Version history:
(1):

Purpose: To simplify management of data identifiers in the tools
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67865v0

If Dynamically defined periodic DataIdentifier data records are implemented they shall have data identifiers in the range 0xF200 to 0xF240 and shall use the identifiers in ascending order starting from data identifier 0xF200 (i.e. if the ECU supports only one single dynamically defined periodicDataIdentifier then its value shall be 0xF200, if the ECU supports two dynamically defined periodicDataIdentifiers then the values shall be 0xF200 and 0xF201, etc.)

2287 v3 **Supported services for Dynamically Defined Data Identifiers**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that data parameters in dynamically defined data records can be read out by diagnostic services.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67869v1

If Dynamically defined DataIdentifier data records are implemented they shall be supported by services, specified by *[Data_1] UDS Services*, that can dynamically define a data identifier and read the dynamically defined data parameter , in all diagnostic sessions except in programmingSession . They shall not be supported by services that can write or control the dynamically defined data parameters.

2288 v3 **Supported services for Dynamically defined periodic Data Identifiers**

Version history:

(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that data parameters in dynamically defined periodic data records can not be changed by services that can write or control the parameter values.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67867v1

If Dynamically defined periodic DataIdentifier data records are implemented they shall only be supported by services, specified by *[Data_1] UDS Services*, that can dynamically define a data identifier and read the dynamically defined data parameter periodically , in all diagnostic sessions except in programmingSession . They shall not be supported by services that can write or control the dynamically defined data parameters.

2289 v2 System Supplier Specific data records

Version history:
(2):

Purpose: System supplier specific data records are not supported by OEM specific tools and must be defined in a separate identifier range.
Source: Table "Requirements on implementation of data records"
Verification Method: Analysis
Legacy ID: 67943v1

If data records that are not intended for use by OEM are defined by the system supplier, these data records shall have data identifiers in the range 0xFD00 to 0xFEFF.

2270 v1 Access System Supplier Specific data records

Version history:
(1):

Purpose: To ensure that the data records can be accessed in the required diagnostic sessions.
Source:
Verification Method: Test
Legacy ID: 119145v0

It may be possible to access the System Supplier Specific data records, with data identifiers in the range 0xFD00 to 0xFEFF, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions except in programmingSession.

2286 v1 Specification of supported services for each data record

Version history:
(1):

Purpose: To define which services that shall be supported for each data record.
Source: Section "Record Data - General"
Verification Method: Analysis

Legacy ID: 67945v0

Unless specified by this document, the implementer shall specify which data identifiers that the ECU supports, and for each supported data identifier, if the data record shall be possible to read, write and/or control by the services specified in *[Data_1] UDS Services*

2276 v1 **Data types and classifications**

Version history:
(1):

Purpose: To simplify implementation of data interpretation in tools all data must comply with standard definitions.
Source: Section "Record Data - General"
Verification Method: Analysis
Legacy ID: 67946v0

All data records shall be defined according to the data types and classifications described in Appendix A . No other data types and classifications are allowed.

4.5.1.1.1.3 Requirements from section ECU external signals

2175 v1 **Support for external switch input signal data records - application signal**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 1.2
Verification Method: Analysis
Legacy ID: 67983v0

For each external switch input signal a feasibility study shall be performed by the implementer to decide if a data record shall be implemented for reading the value of the switch input signal currently available for the ECU´s application software. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data record to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2173 v1 **Support for external output signal data records - application signal**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 4.1
Verification Method: Analysis

Legacy ID: 68011v0

For each external output signal a separate data record shall be defined within the ECU for reading and controlling the output signal (the desired set point) controlled by the ECU´s application software.

2147 v1 Access to external switch input signal data records

Version history:
(1):

Purpose: The data records for external signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 1,1 and 1.2
Verification Method: Test
Legacy ID: 67984v0

It shall be possible to access external switch input signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2165 v1 Reporting external switch input signal data records while the input para...

Version history:
(1):

Purpose: If the tester is actively controling the input parameter the tester substituted value must be reported in the data record since this is the value that the ECu application is using.
Source: Table "Requirements for definition of ECU external signals in data records" - item 1.1
Verification Method: Test
Legacy ID: 67985v0

If it is possible to control the value of an external switch input signal, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted input value to a diagnostic request to read a data record for an actual digital external switch input signal while the input parameter value is under tester control.

2156 v2 External switch input signals update time

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that the data read from the ECU is not too old.
Source: Table "Requirements for definition of ECU external signals in data records" - item 1.1 and 1.2
Verification Method: Analysis

Legacy ID: 67986v0

For all external switch input signal data records, the value read by the diagnostic services specified in *[Data_1] UDS Services* shall be updated by the ECU each 100 ms or faster.

2169 v1 Support for external analog input signal data records - actual input signal

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 2.1
Verification Method: Analysis
Legacy ID: 68001v0

For each external analog input signal a separate data record shall be defined within the ECU for reading the actual digital input signal read by the ECU. The record value shall be the sampled raw signal, after necessary HW and/or SW filtering of “normal” disturbances.
By examining the raw signal it shall be possible to determine if the signal is correct or if a short to battery, short to ground or open circuit is present.

2157 v1 for external analog input signal data records - application signal

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 2.2
Verification Method: Analysis
Legacy ID: 68002v0

For each external analog signal a feasibility study shall be performed by the implementer to decide if a data record shall be implemented for reading the value of the analog input signal currently available for the ECU´s application software. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data record to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2144 v1 Access to external analog input signal data records

Version history:
(1):

Purpose: The data records for external signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 2.1 and 2.2

Verification Method: Test
Legacy ID: 68003v0

It shall be possible to access external analog input signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2163 v2 Reporting external analog input signal data records while the input parameter value is under tester

Version history:
(2):

Purpose: If the tester is actively controling the input parameter the tester substituted value must be reported in the data record since this is the value that the ECu application is using
Source: Table "Requirements for definition of ECU external signals in data records" - item 2.1
Verification Method: Test
Legacy ID: 68004v1

If it is possible to control the value of an external analog input signal, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted input value to a diagnostic request to read a data record for an actual external analog input signal while the input parameter value is under tester control.

2153 v2 External analog input signals update time

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that the data read from the ECU is not too old.
Source: Table "Requirements for definition of ECU external signals in data records" - item 2.1 and 2.2
Verification Method: Analysis
Legacy ID: 68005v0

For all external analog input signal data records, the value read by the diagnostic services specified in *[Data_1] UDS Services* shall be updated by the ECU each 100 ms or faster.

2171 v2 Support for external FM, PCM, PWM input signal data records - engineering units

Version history:
(2):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" item 3.1

Verification Method: Analysis
Legacy ID: 68006v1

For each external FM, PCM, PWM (or similar) input signal a separate data record shall be defined within the ECU for reading the actual digital input signal read by the ECU. The record value shall be the actual input signal in suitable engineering units.
By examining the value read it shall be possible to decide whether the circuit is correct or if a short to battery voltage, short to ground or open circuit fault is present. If this is not possible then the feasibility study required by item 3.2 of table 4.2 shall be performed to decide if the decision can be made by examining the value of the raw version of the signal instead.

2170 v1 **Support for external FM, PCM, PWM input signal data records - actual inp...**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" item 3.2
Verification Method: Analysis
Legacy ID: 68007v0

For each external FM, PCM, PWM (or similar) input signal a feasibility study shall be performed by the implementer to decide if a data record shall be implemented for reading the value of the input signal read by the ECU. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data record to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The record value shall be the sampled raw signal, after necessary HW and/or SW filtering of “normal” disturbances.

2145 v1 **Access to external FM, PCM, PWM input signal data records**

Version history:
(1):

Purpose: The data records for external signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 3.1 and 3.2
Verification Method: Test
Legacy ID: 68008v0

It shall be possible to access external FM, PCM, PWM (or similar) input signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2164 v1 **Reporting external FM, PCM, PWM input signal data records while the inpu...**

Version history:
(1):

Purpose: If the tester is actively controling the input parameter the tester substituted value must be reported in the data record since this is the value that the ECu application is using.
Source: Table "Requirements for definition of ECU external signals in data records" item 3.1 and 3.2
Verification Method: Test
Legacy ID: 68009v0

If it is possible to control the value of an external FM, PCM, PWM (or similar) input signal, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted input value to a diagnostic request to read a data record for an actual digital external FM, PCM, PWM (or similar) input signal while the input parameter value is under tester control.

2154 v2 **External FM, PCM, PWM input signals update time**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that the data read from the ECU is not too old.
Source: Table "Requirements for definition of ECU external signals in data records" - item 3.1 and 3.2
Verification Method: Analysis
Legacy ID: 68010v0

For all external FM, PCM, PWM (or similar) input signal data records, the value read by the diagnostic services specified in *[Data_1] UDS Services* shall be updated by the ECU each 100 ms or faster.

2172 v1 **Support for external output signal data records - actual output signal**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 4.2
Verification Method: Analysis
Legacy ID: 68012v0

For each external output signal a separate data record shall be defined within the ECU for reading the actual output signal read back by the ECU.

2146 v1 **Access to external output signal data records**

Version history:
(1):

Purpose: The data records for external signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 4.1 and 4.2
Verification Method: Test
Legacy ID: 68013v0

It shall be possible to access external output signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2148 v1 Access to network signal data records - signals received by public ECU

Version history:
(1):

Purpose: The data records for network signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.2
Verification Method: Test
Legacy ID: 68021v0

It shall be possible to access the data records for received network signals, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2149 v1 Access to network signal data records - signals transmitted by public ECU

Version history:
(1):

Purpose: The data records for network signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.1
Verification Method: Test
Legacy ID: 68017v0

It shall be possible to access the data records for transmitted network signals, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2150 v1 **Access to other external signal data records defined by the implementer**

Version history:
(1):

Purpose: The data records for external signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 8
Verification Method: Test
Legacy ID: 68028v0

It shall be possible to access the data records for other external signal defined by the implementer, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2151 v1 **Access to power supply input signal data records**

Version history:
(1):

Purpose: The data records for external signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU external signals in data records" - item 7
Verification Method: Test
Legacy ID: 68024v0

It shall be possible to access power supply input signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2152 v2 **Control of network signal data records - periodic signals transmitted by...**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: The test tool must not change the periodicity of transmitted signals as this could lead to faults detected in other ECUs.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.1
Verification Method: Analysis
Legacy ID: 68019v0

If the network signal is normally sent periodically by the ECU then this must continue during diagnostic control of the signal.

2155 v2 **External output signals update time - actual output signal**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that the data read from the ECU is not too old.
Source: Table "Requirements for definition of ECU external signals in data records" - item 4.2
Verification Method: Analysis
Legacy ID: 68015v0

For all external output signal data records, for the actual output signal read back by the ECU, the value read by the diagnostic services specified in *[Data_1] UDS Services*, shall be updated by the ECU each 100 ms or faster.

2158 v1 **Individual control of external output signals - application signal**

Version history:
(1):

Purpose: To enhance fault tracing capabilities it shall be possible to control the output signals from the ECU from the diagnostic test tool.
Source: Table "Requirements for definition of ECU external signals in data records" - item 4.1
Verification Method: Analysis
Legacy ID: 68014v0

It shall be possible to control each external output signal (the desired set point normally controlled by the ECU´s application software) individually and independently of each other unless there are special safety reasons not to do this.
If any output is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM

2159 v1 **Individual control of network signal data records - signals transmitted ...**

Version history:
(1):

Purpose: To enhance fault tracing capabilities it shall be possible to individually control the network signals transmitted from the ECU from the diagnostic test tool.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.1
Verification Method: Analysis
Legacy ID: 68018v0

If control of the network signals is required (as defined by the feasibility study) it shall be possible to control the signals individually and independently of each other unless there are special safety reasons not to do this.
If any network signal is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM.

2160 v1 **Individual control of other external signal data records defined by the ...**

Version history:
(1):

Purpose: To enhance fault tracing capabilities it shall be possible to individually control the signals from the ECU from the diagnostic test tool.
Source: Table "Requirements for definition of ECU external signals in data records" - item 8
Verification Method: Analysis
Legacy ID: 68041v0

If control of the external signal data records defined by the implementer is required (as defined by the feasibility study) it shall be possible to control the signals individually and independently of each other unless there are special safety reasons not to do this.
If any network signal is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM.

2161 v2 **Power supply input signals update time**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that the data read from the ECU is not too old.
Source: Table "Requirements for definition of ECU external signals in data records" - item 7
Verification Method: Analysis
Legacy ID: 68026v0

For all power supply input signal data records, the value read by the diagnostic services specified in *[Data_1] UDS Services* shall be updated by the ECU each 100 ms or faster.

2162 v1 **Read and control of the network signal data record 0xDD02 transmitted by public ECU**

Version history:
(1):

Purpose: Read and control the network signal is used for test and verification of the DTC ¿ ¿Low supply voltage at the ECU¿ during development of the vehicle.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.3.
Verification Method: Test
Legacy ID: 391971v0

If the ECU is responsible for maintaining and distributing the network signal that contain the data record 0xDD02 – “Vehicle battery voltage”, the value of the network signal shall be possible to read and control.

2166 v1 **Reporting other external signal data records defined by the implementer ...**

Version history:

(1):

Purpose: If the tester is actively controling a signal the tester substituted value must be reported in the data record since this is the value that the ECu application is using.
Source: Table "Requirements for definition of ECU external signals in data records" - item 8
Verification Method: Test
Legacy ID: 68042v0

If it is possible to control the value of an external signal defined by the implementer, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted value to a diagnostic request to read a data record for the received network signal while the signal value is under tester control.

2167 v1 **Reporting power supply input signal data records while the input paramet...**

Version history:
(1):

Purpose: If the tester is actively controling the input parameter the tester substituted value must be reported in the data record since this is the value that the ECu application is using
Source: Table "Requirements for definition of ECU external signals in data records" - item 7
Verification Method: Test
Legacy ID: 68025v0

If it is possible to control the value of power supply input signal, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted input value to a diagnostic request to read a data record for an actual digital external switch input signal while the input parameter value is under tester control.

2168 v1 **Reporting received network signal data records while the input parameter...**

Version history:
(1):

Purpose: If the tester is actively controling the network signal the tester substituted value must be reported in the data record since this is the value that the ECu application is using.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.2
Verification Method: Test
Legacy ID: 68022v0

If it is possible to control the value of a received network signal, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted value to a diagnostic request to read a data record for the received network signal while the signal value is under tester control.

2174 v1 **Support for external switch input signal data records actual input signal**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 1.1
Verification Method: Analysis
Legacy ID: 67982v0

For each external switch input signal a separate data record shall be defined within the ECU for reading the actual digital input signal read by the ECU. The record value shall be the sampled raw signal, after necessary HW and/or SW filtering of “normal” disturbances.

2176 v1 **Support for network signal data records - signals received by public ECU**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU, including network signals.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.2
Verification Method: Analysis
Legacy ID: 68020v0

If the ECU is a public ECU a feasibility study shall be performed by the implementer for each network signal received by the ECU to decide if data records shall be implemented for reading the values of the signals. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data records to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2177 v1 **Support for network signal data records - signals transmitted by public ECU**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU, including network signals.
Source: Table "Requirements for definition of ECU external signals in data records" - item 5.1
Verification Method: Analysis
Legacy ID: 68016v0

If the ECU is a public ECU a feasibility study shall be performed by the implementer for each network signal transmitted by the ECU to decide if data records shall be implemented for reading and/or controlling the values of the signals. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data records to be implemented.

The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2178 v1 Support for other external signal data records defined by the implementer

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU. The implementer shall identify which other signals, not covered by the the requirements in this specification, that shall be monitored.
Source: Table "Requirements for definition of ECU external signals in data records" - item 8
Verification Method: Analysis
Legacy ID: 68027v0

A feasibility study shall be performed by the implementer to decide if data records shall be implemented for reading and/or controlling the values of any additional external signals. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data records to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2179 v1 Support for power supply data record

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control input and output signals from/to the ECU.
Source: Table "Requirements for definition of ECU external signals in data records" - item 7
Verification Method: Analysis
Legacy ID: 68023v0

For each power supply to the ECU a separate data record shall be defined within the ECU for reading the actual voltage input to the ECU. The record value shall be the actual voltage read as an analog input signal by the ECU (i.e. the sampled raw signal), but after necessary HW or/and SW filtering of “normal” disturbances.

4.5.1.1.1.4 Requirements from section ECU internal signals

2180 v1 Access to input from internal HMI signal data records

Version history:
(1):

Purpose: Data records for internal signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 1.1 and 1.2

Verification Method: Test
Legacy ID: 68063v0

It shall be possible to access input from internal HMI signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2181 v1 **Access to other internal signal data records defined by the implementer**

Version history:
(1):

Purpose: The data records for ECU internal signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 3
Verification Method: Test
Legacy ID: 68083v0

It shall be possible to access the data records for other ECU internal signals defined by the implementer, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2182 v1 **Access to output to internal HMI signal data records**

Version history:
(1):

Purpose: The data records for internal signals shall be available in application software but does not have to be supported in boot software.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 2
Verification Method: Test
Legacy ID: 68067v0

It shall be possible to access output to internal HMI signal data records, by the relevant diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions, except programming session.
The access by a specific diagnostic service in a specific session depends on if that specific diagnostic service is supported in that specific session.

2183 v1 **Individual control of other ECU internal signal data records defined by ...**

Version history:
(1):

Purpose: To enhance fault tracing capabilities it shall be possible to individually control

the ECU internal signals from the diagnostic test tool.

Source: Table "Requirements for definition of ECU internal signals in data records" - item 3

Verification Method: Analysis

Legacy ID: 68084v0

If control of the ECU internal signal data records defined by the implementer is required (as defined by the feasibility study) it shall be possible to control the signals individually and independently of each other unless there are special safety reasons not to do this.
If any network signal is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM.

2184 v2 Individual control of output to internal HMI signals

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To enhance fault tracing capabilities it shall be possible to control the output signals from the ECU from the diagnostic test tool.

Source: Table "Requirements for definition of ECU internal signals in data records" - item 2

Verification Method: Analysis

Legacy ID: 68081v0

It shall be possible to control each output to internal HMI signal (the desired set point normally controlled by the ECU´s application software) individually and independently of each other unless there are special safety reasons not to do this.
If any output signal is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM

2185 v1 Reporting input from internal HMI signal data records while the input pa...

Version history:
(1):

Purpose: If the tester is actively controling the input parameter the tester substituted value must be reported in the data record since this is the value that the ECu application is using

Source: Table "Requirements for definition of ECU internal signals in data records" - item 1.1

Verification Method: Test

Legacy ID: 68064v0

If it is possible to control the value of an input from internal HMI signal, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted input value to a diagnostic request to read a data record for an actual digital internal HMI input signal while the input parameter value is under tester control.

2186 v1 Reporting other ECU internal signal data records defined by the implemen...

Version history:

(1):

Purpose: If the tester is actively controling a signal the tester substituted value must be reported in the data record since this is the value that the ECu application is using
Source: Table "Requirements for definition of ECU internal signals in data records" - item 3
Verification Method: Test
Legacy ID: 68085v0

If it is possible to control the value of an ECU internal signal defined by the implementer, by diagnostic services specified in *[Data_1] UDS Services*, the ECU shall report the tester substituted value to a diagnostic request to read a data record for the received network signal while the signal value is under tester control.

2187 v2 Support for input from internal HMI signal data records - actual input signal

Version history:
(2):

Purpose: An important part of the fault tracing strategy is to be able to read and control ECU internal input and output signals.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 1.1
Verification Method: Analysis
Legacy ID: 68061v1

For each input from internal HMI signal a data record shall be defined within the ECU for reading the actual digital input signal read by the ECU. The record value shall be the sampled raw signal, after necessary HW and/or SW filtering of “normal” disturbances.

2188 v1 Support for input from internal HMI signal data records - application si...

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control ECU internal input and output signals.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 1.2
Verification Method: Analysis
Legacy ID: 68062v0

For each input from internal HMI signal a feasibility study shall be performed by the implementer to decide if a data record shall be implemented for reading the value of the internal HMI signal currently available for the ECU´s application software. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data record to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2189 v1 **Support for other internal signal data records defined by the implementer**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control ECU internal input and output. The implementer shall identify which other signals, not covered by the the requirements in this specification, that shall be monitored.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 3
Verification Method: Analysis
Legacy ID: 68082v0

A feasibility study shall be performed by the implementer to decide if data records shall be implemented for reading and/or controlling the values of any additional ECU internal signals. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the data records to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2190 v1 **Support for output to internal HMI signal data records**

Version history:
(1):

Purpose: An important part of the fault tracing strategy is to be able to read and control ECU internal input and output signals.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 2
Verification Method: Analysis
Legacy ID: 68066v0

For each output to internal HMI signal a separate data record shall be defined within the ECU for reading and controlling the output signal (the desired set point) controlled by the ECU's application software.

2191 v2 **Update time for input from internal HMI signal data records**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that the data read from the ECU is not too old.
Source: Table "Requirements for definition of ECU internal signals in data records" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 68065v0

For all internal HMI input signal data records, the value read by the diagnostic services specified in *[Data_1] UDS Services* shall be updated by the ECU each 100 ms or faster.

4.5.1.1.1.5 Requirements from section ECU_Vehicle status

2230 v1 **Active diagnostic session data record**

Version history:
(1):

Purpose: To enable readout of the active diagnostic session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68177v0

A data record with identifier 0xF186 shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2231 v1 **Build List data record**

Version history:
(1):

Purpose: The workshop tool must be able to read out the P-spec for the vehicle.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68163v0

If the ECU is the Car Configuration Domain Master a data record with identifier 0xC011 shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2232 v3 **Car Configuration Parameter Faults data record**

Version history:
(3):

Purpose: To support a more advanced analysis of the vehicle behaviour it must be possible to understand how many incorrect configuration parameters that the ECU has received.
Source: Table "Requirements for definition of ECU/Vehicle status in data records".
Verification Method: Test
Legacy ID: 68175v2

A data record with identifier 0xE103 shall be implemented exactly as defined in *[Data_4] Global Master Reference Database* if the ECU support the DTC U230056 (0xE30056) - Invalid vehicle configuration data.

2233 v2 **Car Mode data record**

Version history:
(2):

Purpose: To enable readout and control of the active Car mode.
Source: Table - Requirements for definition of ECU/Vehicle status in data records
Verification Method: Test

Legacy ID: 161873v1

If the ECU are publisher or subscriber of the car mode signal, a data record with identifier 0xD134 shall be implemented exactly as defined in reference *[Data_4] Global Master Reference Database*.

2234 v4 **Control of Usage Mode data record**

Version history:
(4):

Purpose: To enable control of the active Usage mode.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68174v3

It shall be possible to control the value of the data record 0xDD0A in the ECU that implements the Usage Mode Manager, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for control the data record is supported, except programming session.

2235 v2 **Control or write of Car Mode data record**

Version history:
(2):

Purpose: To enable control or write of the active Car mode.
Source: Table - Requirements for definition of ECU/Vehicle status in data records.
Verification Method: Test
Legacy ID: 161913v1

Whether or not it be possible to control or write the value of the data record 0xD134, by diagnostic services specified in *[Data_1] UDS Services*, is specified in *[Data_3] Vehicle Mode Management - Concept SRD.*

2236 v2 **Customer setting parameters data record**

Version history:
(2):

Purpose: The purpose is to support the ECU replacement process at workshop
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 344378v1

A data record with identifier 0xDED0 shall be implemented if the ECU stores customer setting parameter.
The data record shall only consist of record data that contain information of customer settings stored by the ECU. The data record is used for restoring customer settings in the ECU replacement process at a workshop.
A customer setting is defined as:

• A function that is open for alteration by the customer and/or tester tool it will be possible to change and adapt behavior of the functionality in the car to suit the customer.
• A long term alteration that only is initiated by the customer himself. The ECU is not allowed to change the customer setting parameter. Short term adjustments that, due to safety or other reasons, only are valid in one driving cycle or less and will revert to a predefined value are not to be considered as customer setting parameter.
A setting that can be done by the customer himself and by the tester tool or only by a tester tool (at workshop or remote).

2237 v1 **Diagnostic Read out #1 to #4 data record**

Version history:
(1):

Purpose: To support quality tracking of the ECU.
Source:
Verification Method: Test
Legacy ID: 113861v0

A data record(s) with identifier 0xEDC0 up to 0xEDC3 shall be implemented if required by the responsible authority appointed by OEM.
The data record(s) intended for quality tracking of the ECU is defined by the implementer.
The data record(s) shall only consist of other record data than the record data read by the generic standard read out sequence.

2238 v1 **ECU List data record**

Version history:
(1):

Purpose: The workshop tool must be able to read out what ECUs are actually installed in the vehicle.
Source: "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68141v0

If the ECU is the Car Configuration Domain Master a data record with identifier 0xC010 shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2239 v2 **Partial Network Cluster (PNC) data record**

Version history:
(2):

Purpose: To enable readout and control of the Partial Network Cluster (PNC)
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 178193v1

If the ECU is the Vehicle Mode Management (VMM) master i.e. the ECU that supports the subsystem ‘Internal-VMM implementation‘ (refer to *[Data_3] Vehicle Mode Management -*

Concept SRD for details regarding VMM), a data record with identifier 0xDD0B shall be implemented exactly as defined in reference *[Data_4] Global Master Reference Database.*

2240 v2 **Read Access to Active diagnostic session data record**

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to read out the active diagnostic session from the ECU, independent of the diagnostic session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68178v1

It shall be possible to read the data record 0xF186, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported.

2241 v2 **Read Access to Build List data record**

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to read out the Build List data record in all diagnostic sessions, except in programming session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68165v1

It shall be possible to read the data record 0xC011, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except in programming session.

2242 v3 **Read access to Car Configuration Parameter Faults data record**

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the Car Configuration Parameter Faults data record in all diagnostic sessions, except in programming session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68176v2

It shall be possible to read the data record 0xE103, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except programming session.

2243 v3 **Read access to Car Mode data record**

Version history:

(3):

Purpose: To support diagnostic procedures it must be possible to read out the active Car Mode from the ECU in all diagnostic sessions, except in programming session.
Source: Table - Requirements for definition of ECU/Vehicle status in data records.
Verification Method: Test
Legacy ID: 161893v2

It shall be possible to read the data record 0xD134, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except programming session.

2244 v3 Read access to Customer setting parameters data record

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the Customer setting parameter from the ECU, independent of the diagnostic session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 344379v2

It shall be possible to read the data record 0xDED0, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except programming session.

2245 v1 Read Access to Diagnostic Read out #1 to #4 data record

Version history:
(1):

Purpose: To support quality tracking of the ECU
Source:
Verification Method: Test
Legacy ID: 113863v0

It shall be possible to read the data record(s) with identifier 0xEDC0 up to 0xEDC3, by diagnostic services specified in reference *[Data_1] UDS Services* , in all diagnostic sessions except in programming session.

2246 v2 Read Access to ECU List data record

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to read out the ECU List data record in all diagnostic sessions, except in programming session.
Source: Table 4.4: Table "Requirements for definition of ECU/Vehicle status in data records"

Verification Method: Test
Legacy ID: 68161v1

It shall be possible to read the data record 0xC010, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except in programming session.

2247 v1 Read access to Partial Network Cluster (PNC) data record

Version history:
(1):

Purpose: To support diagnostic procedures it must be possible to read out the active Partial Network Cluster (PNC) from the ECU in all diagnostic sessions, except in programming session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 178213v0

If the support the data identifier 0xDD0B, it shall be possible to read the data record from the ECU, by diagnostic services specified in*[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except programming session.

2248 v2 Read access to RVDC data record

Version history:
(2): Change the Verification Method to Test

Purpose: To support RVDC functionality in the vehicle, it must be possible to read out the RVDC data record.
Source:
Verification Method: Test
Legacy ID: 438802v0

It shall be possible to read the data record 0xEA00, by diagnostic services specified in *[Data_1] UDS Services* and according to *RVDCClient*, UDS DATA, documents.

2249 v3 Read access to System adaption data record

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the System adaption data from the ECU, independent of the diagnostic session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 344382v2

It shall be possible to read the data record 0xDED1, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except programming session.

2250 v3 **Read access to Usage Mode data record**

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the active Usage Mode from the ECU in all diagnostic sessions, except in programming session.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68173v2

It shall be possible to read the data record 0xDD0A, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for read the data record is supported, except programming session.

2251 v1 **Read Access to Vehicle state of health data record**

Version history:
(1):

Purpose: To support quality diagnostics of the ECU.
Source:
Verification Method: Test
Legacy ID: 113866v0

It shall be possible to read the data record(s) with identifier 0xEDC5, by diagnostic services specified in reference *[Data_1] UDS Services* in all diagnostic sessions except in programming session.

2252 v2 **Remote Vehicle Data Collection (RVDC) data record**

Version history:
(2): Change the Verification Method to Test

Purpose: To enable readout of RVDC data record
Source:
Verification Method: Test
Legacy ID: 438801v0

A data record with identifier 0xEA00 shall be implemented according to *RVDCClient*, UDS DATA, documents.
The data record shall consist of a list of other data records that are supported by the ECU.

The content in the data record shall be the following:

Byte	Description
#1-2	First data record identifier supported by the ECU

#3-4 Second data record identifier supported by the ECU

#5-6 Third data record identifier supported by the ECU

.

.

#N-(N+1) N:th data record identifier supported by the ECU

If a data record in the list either is not supported for a specific configuration of the ECU or for some other reason does not have a correct or avaliable content, the value of the record identifier shall be replaced with 0x0000.

2253 v2 System adaption data record

Version history:
(2):

Purpose: The purpose is to support the ECU replacement process at workshop
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 344381v1

A data record with identifier 0xDED1 shall be implemented if the ECU stores system adaption data.
Data records intended for adaption parameters.
The data record shall only contain information of adaption data stored by the ECU. The data record is used for restoring adaption data in the ECU replacement process in workshop.
Adaption parameters are adjusted to the customers usage/driving and/or components performance/life-span in order to optimize the performance and drivability in the vehicle.
The data record shall be used where it is considered suitable that adaption values are transferred from an old to a new ECU in the ECU replacement process at a workshop.

2254 v1 Usage Mode data record

Version history:
(1):

Purpose: To enable readout of the active Usage mode.
Source: Table "Requirements for definition of ECU/Vehicle status in data records"
Verification Method: Test
Legacy ID: 68172v0

A data record with identifier 0xDD0A shall be implemented exactly as defined in reference *[Data_4] Global Master Reference Database*.

2255 v2 Vehicle state of health data record

Version history:

(2):

Purpose: To support remote diagnostics of the ECU.
Source:
Verification Method: Test
Legacy ID: 113864v1

A data record with identifier 0xEDC5 shall be implemented if required by the responsible authority appointed by OEM.
Data record intended for remote diagnostics is defined by the implementer.
The data record shall only consist of record data that the workshop preplanning process need to supplement DTCs and DTC related data in order to identify the root cause to customer complaints.

2256 v3 Write access to Customer setting parameters data record

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to write the Customer setting parameter from the ECU, independent of the diagnostic session.
Source: Table: Requirements for definition of ECU/Vehicle status in data records
Verification Method: Test
Legacy ID: 347297v2

It shall be possible to write the data record 0xDED0, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for write the data record is supported, except programming session.

2257 v3 Write access to System adaption data record

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to write the System adaption data from the ECU, independent of the diagnostic session.
Source: ble: Requirements for definition of ECU/Vehicle status in data records
Verification Method: Test
Legacy ID: 347298v2

It shall be possible to write the data record 0xDED1, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service used for write the data record is supported, except programming session.

4.5.1.1.1.6 Requirements from section ECU_Vehicle identification

2192 v3 Access to Application Diagnostic Database Part Number data record

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the

Application Diagnostic Database Part Number from the ECU, except in programming session.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68190v2

It shall be possible to read the data record with the identifier 0xF120 and the data record with the identifier 0xF1A0, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2193 v5 Access to Complete ECU PartSerial Number data record

Version history:
(5):

Purpose: To support diagnostic procedures it must be possible to read out the Complete ECU Part/Serial Number data record.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68241v4

It shall be possible to read the data record with the identifier 0xEDA0 and the data record with the identifier 0xED20, by diagnostic services specified in *[Data_1] UDS Services*, including programming session running in both primary and secondary bootloader.

2194 v1 Access to Diagnostic Autosar version number

Version history:
(1):

Purpose: To support diagnostic procedures it must be possible to read out the AUTOSAR cluster(s) implemented in the ECU, except in programming session.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68199v0

It shall be possible to read the data record with the identifier 0xF126, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2195 v3 Access to ECU Core Assembly Part Number data record

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the ECU Core Assembly Part Number data record.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68201v2

It shall be possible to read the data record with the identifier 0xF12A and a data record with identifier 0xF1AA, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions, including programming session running in both primary and secondary bootloader. It is allowed to change the value of the data records, by diagnostic service defined in reference *[Data_1] UDS Services*, but only once. Only the secondary bootloader is allowed to support change of the value .

2196 v4 **Access to ECU Delivery Assembly Part Number data record**

Version history:
(4):

Purpose: To support diagnostic procedures it must be possible to read out the ECU Delivery Assembly Part Number data record, except in programming session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68203v3

It shall be possible to read the data record with the identifier 0xF12B and the data record with the identifier 0xF1AB, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions , including programming session running in both primary and secondary bootloader. It is allowed to change the value of the data records, by diagnostic service defined in reference *[Data_1] UDS Services*, but only once. Only the secondary bootloader is allowed to support change of the value.

2197 v2 **Access to ECU Serial Number data record**

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to read out the ECU Serial Number data record.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68231v1

It shall be possible to read the data record with the identifier 0xF18C, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions, including programming session running in both primary and secondary bootloader. It is allowed to change the value of the data record, by diagnostic service defined in reference *[Data_1] UDS Services*, but only once. Only the secondary bootloader is allowed to support the write.

2198 v2 **Access to ECU Software Extended Part Numbers data record**

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to read out the ECU Software Extended Part Numbers data record, except in programming session.
Source: Section "ECU/Vehicle identification"

Verification Method: Test
Legacy ID: 114064v1

It shall be possible to read the data record with the identifier 0xF12F and the data record with the identifier 0xF1AF, by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2199 v3 **Access to ECU Software Part Numbers data record**

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the ECU Software Part Numbers data record, except in programming session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68208v2

It shall be possible to read the data record with the identifier 0xF12E and the data record with the identifier 0xF1AE, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2200 v3 **Access to Primary Bootloader Diagnostic Database Part Number**

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the database key for the diagnostic database used by the ECU's primary bootloader SW.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68192v2

It shall be possible to read the data record with the identifier 0xF121 and and the data record with the identifier 0xF1A1, by diagnostic services specified in reference *[Data_1] UDS Services*, in programming session running in both primary and secondary bootloader. It may be possible to read the data identifiers in other sessions. It is allowed to change the value of the data records, by diagnostic service defined in reference *[Data_1] UDS Services*, but only once. Only the secondary bootloader is allowed to support change of the value.

2201 v4 **Access to Primary Bootloader Software Part Number data record**

Version history:
(4):

Purpose: To support diagnostic procedures it must be possible to read out the Primary Bootloader Software Part Number data record.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68226v3

It shall be possible to read the data record with the identifier 0xF125 and the data record with the identifier 0xF1A5 by diagnostic services specified in reference *[Data_1] UDS Services*, in programming session running in both primary and secondary bootloader. It may be possible to read the data identifiers in other sessions. It is allowed to change the value of the data records, by diagnostic service defined in reference *[Data_1] UDS Services*, but only once. Only the secondary bootloader is allowed to support the write.

2202 v1 **Access to Private ECU(s) or component(s) Serial Number data record**

Version history:
(1):

Purpose: To enable read out of the serial numbers for Private ECU(s) or component(s).
Source:
Verification Method: Test
Legacy ID: 442778v0

It shall be possible to read out the data record with the identifier 0xF13F by diagnostic services specified in reference *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2203 v3 **Access to Secondary Bootloader Diagnostic Database Part Number**

Version history:
(3):

Purpose: To support diagnostic procedures it must be possible to read out the database key for the diagnostic database used by the ECU's secondary bootloader SW, when executing the secondary bootloader in programming session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68195v2

It shall be possible to read the data record with the identifier 0xF122 and the data record with the identifier 0xF1A2 by diagnostic services specified in *[Data_1] UDS Services*, when the ECU is executing the secondary bootloader in programming session. It is not allowed to read the data identifier in other sessions.

2204 v2 **Access to Secondary Bootloader Software Version Number data record**

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to read out the Secondary Bootloader Software Version Number data record.
Source: Section: ECU/Vehicle identification
Verification Method: Test
Legacy ID: 129861v1

It shall be possible to read the data record with the identifier 0xF124, by diagnostic services specified in reference *[Data_1] UDS Services* in programming session running in secondary

bootloader. It is not allowed to read the data identifier in other sessions.

2205 v1 **Access to Vehicle Configuration Parameters data record**

Version history:
(1):

Purpose: To support diagnostic procedures it must be possible to read out the Vehicle Configuration Parameters from the ECU, except in programming session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68186v0

It shall be possible to read the data record with the identifier 0xF106, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions in which the service is supported except programming session.

2206 v1 **Access to Vehicle Identification Number data record**

Version history:
(1):

Purpose: To support diagnostic procedures it must be possible to read out the Vehicle Identification Number data record, except in programming session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68228v0

If the data record with the identifier 0xF190 is implemented in the ECU, it shall be possible to read this data record by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2207 v1 **Access to Vehicle Information Section data record**

Version history:
(1):

Purpose: To support diagnostic procedures it must be possible to read out the Vehicle Information Section data record, except in programming session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68206v0

It shall be possible to read the data record with the identifier 0xF114, by diagnostic services specified in *[Data_1] UDS Services*, in all diagnostic sessions except programming session.

2208 v4 **Application Diagnostic Database Part Number data record**

Version history:
(4): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of a database key for the diagnostic database used by the

ECU application SW.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68189v2

A data record with identifier 0xF120 shall be implemented containing Volvo Cars format Part number.
A data record with identifier 0xF1A0 shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2209 v2 Autosar BSW cluster versions

Version history:
(2):

Purpose: To enable readout of the specific version of the AUTOSAR cluster(s) implemented in the ECU.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68197v1

If the ECU contains any AUTOSAR Basic Software cluster a data record with identifier 0xF126 shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2210 v8 Complete ECU PartSerial Number data record

Version history:
(8): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of the complete set of ECU Part/Serial numbers.

Source: Section "ECU/Vehicle identification"

Verification Method: Test

Legacy ID: 68239v6

A data record with identifier 0xEDA0 shall be implemented containing Volvo Cars format Part number.
A data record with identifier 0xED20 shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.
The data records (called "composite" data record) shall contain several data items in an arbitrary order. Each data item shall consist of a data identifier followed by the record data that is identified by the data identifier.
The "composite" data record with identifier 0xEDA0 shall contain the following data items (the data items is depending of the currently active executing software):

F120 or F121 or F122, Application- or PBL- or SBL Diagnostic Database Part number
F12A, ECU Core Assembly Part Number
F12B, ECU Delivery Assembly Part Number
F18C, ECU Serial Number
F12E or F125 or F124, ECU- or PBL Software Part Number(s) or SBL Software Version Number
F13F, Private ECU(s) or component(s) Serial Number.
The "composite" data record with identifier 0xED20 shall contain the following data items (the data items is depending of the currently active executing software):
F1A0 or F1A1 or F1A2, Application- or PBL- or SBL Diagnostic Database Part number
F1AA, ECU Core Assembly Part Number
F1AB, ECU Delivery Assembly Part Number
F18C, ECU Serial Number
F1AE or F1A5 or F124, ECU- or PBL Software Part Number(s) or SBL Software Version Number
F13F, Private ECU(s) or component(s) Serial Number.

2211 v4 ECU Core Assembly Part Number data record

Version history:
(4): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of a part number that identifies the combination of the ECU hardware and any non-replaceable software (bootloaders and other fixed software).
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68200v2

A data record with identifier 0xF12A shall be implemented containing Volvo Cars format Part number.
A data record with identifier 0xF1AA shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2212 v3 ECU Delivery Assembly Part Number data record

Version history:
(3): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of a part number that identifies the complete ECU at the point of delivery to the assembly plant.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68202v1

A data record with identifier 0xF12B shall be implemented containing Volvo Cars format Part number.

A data record with identifier 0xF1AB shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2213 v2 **ECU Serial Number data record**

Version history:
(2):

Purpose: To enable readout of the supplier unique ECU serial number.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68230v1

A data record with identifier 0xF18C shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*. Serial number data records shall have 8 digits for the serial number, all coded in BCD. The data record shall have a fixed length of 4 bytes, be right justified with any unused digit(s) filled with 0.

2214 v4 **ECU Software Extended Part Numbers data record**

Version history:
(4): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of the extended part numbers for the ECU softwares
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 114062v2

If required by the project responsible OEM, data records shall be implemented according to the following:
A data record with identifier 0xF12F shall be implemented containing Volvo Cars format Part number.
A data record with identifier 0xF1AF shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.
The content of the dataRecord with dataIdentifier 0xF12F shall be the following:

Byte	Description
#1	Total number of ECU Software Part Numbers
#2 - 8	ECU Software #1 Part Number
#9 - 15	ECU Software #2 Part Number
.	.
.	.
#N - (N+6)	ECU Software #X Part Number

The content of the dataRecord with dataIdentifier 0xF1AF shall be the following:

Byte	Description
#1	Total number of ECU Software Part Numbers
#2 - 9	ECU Software #1 Part Number
#10 - 17	ECU Software #2 Part Number
.	.
.	.
#N - (N+7)	ECU Software #X Part Number

The Part Number(s) is only allowed to be used for item(s) that can not be separately downloaded to the ECU according.

2215 v6 ECU Software Part Number data record

Version history:
(6): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of the part number for the ECU software.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68207v4

A data record with identifier 0xF12E shall be implemented containing Volvo Cars format Part numbers.
A data record with identifier 0x F1AE shall be implemented containing Geely format Part numbers.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.
The content of the dataRecord shall be the following:

The content of the dataRecord with identifier 0xF12E shall be the following:

Byte	Description
#1	Total number of ECU Software Part Numbers
#2 - 8	ECU Software #1 Part Number
#9 - 15	ECU Software #2 Part Number
.	.
.	.
#N - (N+6)	ECU Software #X Part Number

The content of the dataRecord with identifier 0xF1AE shall be the following:

Byte	Description
#1	Total number of ECU Software Part Numbers
#2 - 9	ECU Software #1 Part Number
#10 - 17	ECU Software #2 Part Number
.	.
.	.
#N - (N+7)	ECU Software #X Part Number

The Part Number(s) is only allowed to be used for item(s) that can be separately downloaded to the ECU .

Note: If part numbers are not obtained on both formats DID:s on either format may be filled with zeros. The ECU shall however always have at leas one set of valid part numbers.

2216 v3 ECU Software Structure Part Number data record

Version history:
(3): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of the part number of the ECU Software Structure.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 418372v1

If the ECU is supporting the Software Authentication concept where each data file is signed and verified individually as defined in *[Data_10] General Software Authentication* the ECU shall implement:
A data record with identifier 0xF12C containing Volvo Cars format part number.
A data record with identifier 0xF1AC containing Geely format Part number.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.
It shall be possible to read the data records by diagnostic services specified in reference *[Data_1] UDS Services*, in programming session running in both primary and secondary bootloader

Note: Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.

2217 v2 No read of the Security Constant data record

Version history:
(2):

Purpose: For security reasons it must not be possible to read the value of the Security Constant in any diagnostic session.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68182v1

If a data record with identifier 0xF102 is implemented in the ECU it shall not be possible to read the value of this data record by any diagnostic service in any diagnostic session or any kind of external debug interface that may be used without destroying the sealing of the ECU . Note that this requirement is only applicable sealed ECU's.

2218 v2 One time write of the Security Constant data record

Version history:
(2):

Purpose: To limit the possibility to change the value of the security constant to a one time writing of the value.
Source: Section "ECU/Vehicle identification"

Verification Method: Test
Legacy ID: 68183v1

If it is possible to change the value of the data record with identifier 0xF102, by diagnostic service defined in *[Data_1] UDS Services*, the value is allowed to be changed only one single time. Once the value has been changed, it shall not be possible to change the value again. Change of the value is optional only when the ECU is executing the secoundary bootloader - programming session .

2219 v6 Part number data records coding

Version history:
(6): Change the Verification Method from Analysis to Test

Purpose: N/A
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68214v4

Part number data records used by VCC shall have 8 digits for part number + 3 characters (version suffix).
Part number data records used by Geely shall have 10 digits for part number + 3 characters (version suffix).
T he part number digits shall be coded in BCD and the 3 characters shall be coded in ASCII.

The data record used by VCC shall have a fixed length of 7 bytes and the data record used by Geely shall have a fixed length of 8 bytes, be right justified with any unused digit(s) filled with 0.

If version suffix contains one character, there shall be two space characters between the part number digits and the version suffix.
If version suffix contains two characters, there shall be one space character between the part number digits and the version suffix.

This requirement applies to all data records with the following identifiers used by VCC:
F120, F121, F122, F125, F12A, F12B, F12C, F12E, F12F.
This requirement applies to all data records with the following identifiers used by Geely:
F1A0, F1A1, F1A2, F1A5, F1AA, F1AB, F1AC, F1AE, F1AF.

2220 v4 Primary Bootloader Diagnostic Database Part Number

Version history:
(4): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of a database key for the diagnostic database used by the ECU's primary bootloader SW.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68191v2

A data record with identifier 0xF121 shall be implemented containing Volvo Cars format Part number.

A data record with identifier 0xF1A1 shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2221 v4 Primary Bootloader Software Part Number data record

Version history:
(4): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of the part number of the Primary Bootloader SW.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68225v2

A data record with identifier 0xF125 shall be implemented containing Volvo Cars format Part number.
A data record with identifier 0xF1A5 shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2222 v1 Private ECU(s) or component(s) Serial Number data record

Version history:
(1):

Purpose: To support read out of serial numbers for private ECUs or components that are required by legislation to have additional documentation and where the case is that the private ECUs / components does not support to be adressed directly by a tester.
Source:
Verification Method: Test
Legacy ID: 442733v0

A data record with identifier 0xF13F shall be implemented only if the following conditions are fulfilled:

- The ECU is a Public ECU master for Private ECU(s) or component(s) which has additional documentation requirements.
- The Public ECU master does not support gateway functionality for the private ECU(s) / component(s) in question, meaning that a tester can not read the serial number from the Private ECU(s) or component(s) by adressing it directly.

The content of the data record shall only consist of serial number(s) for Private ECU(s) or component(s) that are subject to additional documentation requirements. The decision if a Private ECU or component will be subject for additional documentation is made by responsible authority at OEM.

The content of the data record shall only consist of serial number(s) for those Private ECU(s) or component(s) that can not be directly adressed by a tester.
The lenght of the serial number for a private ECU / component shall be maximum 12 digits.
The coding of the serial number shall be alphanumeric.
The serial number shall be right justified with any unused digits filled with 0, in total 12 bytes.
If a private ECU / component, that is supposed to be present according to the ECU master, fails to answer when the public ECU master collect private ECUs / components part & serial numbers, this shall be shown by filling all bytes in the missing number with 0xFF.
If the configuration of a vehicle excludes a private ECU / component (i.e certain private ECU / component are not mounted in the vehicle) this shall be shown by having all bytes in the serial number for that specific private ECU / component filled with 0x00.
The content of the data record shall be the following:

Byte	Description
#1	Total number of Private ECU(s) / component(s) Serial Numbers
#2 -13	Private ECU / component #1 Serial Number
#14 - 26	Private ECU / component #2 Serial Number
. .	
. .	
#N - (N+12)	Private ECU / component #X Serial Number

2223 v4 **Secondary Bootloader Diagnostic Database Part Number**

Version history:
(4): Clarification about the usage of Geely and Volvo format part numbers.

Purpose: To enable readout of a database key for the diagnostic database used by the ECU's secondary bootloader SW.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68194v2

A data record with identifier 0xF122 shall be implemented containing Volvo Cars format Part number.
A data record with identifier 0xF1A2 shall be implemented containing Geely format Part number.
Both data records shall be implemented in all ECU:s independently of what car brand they shall be used in.
The data records shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*.

2224 v1 **Secondary Bootloader Software Version Number data record**

Version history:
(1):

Purpose: To enable readout of the version number of the Secondary Bootloader SW.
Source: Section: ECU/Vehicle identification
Verification Method: Test
Legacy ID: 129845v0

A data record with identifier 0xF124 shall be implemented exactly as defined in reference

[Data_4] Global Master Reference Database.

2225 v2 Secondary Bootloader Software Version Number data record coding

Version history:
(2):

Purpose: To enable correct interpretation of the Secondary Bootloader Software Version Number.
Source: Section: ECU/Vehicle identification
Verification Method: Inspection
Legacy ID: 129842v1

The Secondary Bootloader Software Version Number 0xF124 shall be BCD encoded, right justified, all unused digit shall be filled with 0 in total 7 bytes.

2226 v1 Vehicle Configuration Parameters data record

Version history:
(1):

Purpose: To enable readout of the Vehicle Configuration Parameters.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68185v0

A data record with identifier 0xF106 shall be implemented, exactly as defined in *[Data_4] Global Master Reference Database*, in the Car Configuration Domain Master, on networks with ECU configuration controlled by an on-board master configuration file.

2227 v1 Vehicle Identification Number data record

Version history:
(1):

Purpose: To enable readout of the standard 17 character VIN (as written on the vehicle identification plate).
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 68227v0

If the ECU has a requirement to store VIN, a data record with identifier 0xF190 shall be implemented exactly as defined in *[Data_4] Global Master Reference Database*

2228 v2 Vehicle Information Section data record

Version history:
(2):

Purpose: To enable readout of Vehicle Information Section.
Source: Section "ECU/Vehicle identification"
Verification Method: Test

Legacy ID: 68205v1

A data record with identifier 0xF114 shall be implemented exactly as defined in reference *[Data_4] Global Master Reference Database*, in the Car Configuration Domain Master, on networks with ECU configuration controlled by an on-board master configuration file.

2229 v3 Write of the Public Key data record

Version history:
(3):

Purpose: To enable programming of the Public Key, for ECUs supporting Software Authentication where only the key (and not certificates) is used to verify the authenticity of the software.
Source: Section "ECU/Vehicle identification"
Verification Method: Test
Legacy ID: 418471v1

If the ECU is supporting the Software Authentication concept where the data file is verified using a Public Key only as defined in *[Data_10] General Software Authentication* the ECU shall implement to:
Write the Public Key data record with identifier 0xD01C, by diagnostic service defined in *[Data_1] UDS Services*, it is to be supported when the ECU is executing the secondary bootloader - programming session.

4.5.1.1.1.7 Requirements from section DTC information - General

2100 v1 DTC Failure Type Byte

Version history:
(1):

Purpose: A unified failure type description shall be used by all DTCs since it will be displayed by the tools.
Source: Table "Requirements on implementation of DTCs"
Verification Method: Analysis
Legacy ID: 68265v0

DTCs supported by an ECU shall use Failure Type Byte as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications.

2101 v3 DTCs defined by GMRDB

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: All DTC information shall be supported by OEM specific tools and must therefore be defined in GMRDB.
Source: Table "Requirements on implementation of DTCs"
Verification Method: Analysis
Legacy ID: 68264v1

DTCs supported by an ECU shall be implemented as defined in *[Data_4] Global Master Reference Database*. Development specific implementer specifed DTCs are excluded from from this requirement.

2102 v1 **DTCs for faults caused by the ECU operating environment or by customer**

Version history:
(1):

Purpose: DTCs identifying faults caused by the ECU operating environment or by customer actions shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source:
Verification Method: Inspection
Legacy ID: 115161v0

DTCs that identifies faults caused by the ECU operating environment or by customer actions shall be defined in the base DTC range U2E00 to U2EFF and according to section *Detection of faults caused by the ECU operating environment or customer actions* .

2103 v3 **DTCs for secondary faults caused by data from other ECU indicated as invalid**

Version history:
(3):

Purpose: DTCs identifying secondary faults (i.e. not root cause9 shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.3
Verification Method: Analysis
Legacy ID: 68270v2

DTCs that identifies faults caused by signal or data value received from another ECU where the signal/data is indicated as invalid by the transmitting ECU, shall be defined in the base DTC range U2C00 to U2CFF and according to section *Detection of faults caused by signals and data values received from other ECUs* .

2104 v3 **DTCs for secondary faults caused by data from other ECU not indicated as invalid**

Version history:
(3):

Purpose: DTCs identifying secondary faults (i.e. not root cause9 shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.4
Verification Method: Analysis
Legacy ID: 68271v2

DTCs that identifies faults caused by signal or data value received from another ECU where the signal/data is **not** indicated as invalid by the transmitting ECU, shall be defined in the base DTC range U2D00 to U2DFF according to section *Detection of faults caused by signals and data*

values received from other ECUs .

2105 v1 **DTCs that are needed only during the development of the ECU**

Version history:
(1):

Purpose: DTCs that are needed only during the development of the ECU shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.7
Verification Method: Test
Legacy ID: 392611v0

DTCs that are needed only during the development or quality tracking of the ECU shall be defined in the base DTC range U2800 to U2BFF.
The requirements on detection of the faults that shall be identified by DTCs in this range are specified in the following sections.
The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range.
The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range.

2106 v5 **DTCs that are needed only during the development of the ECU - implementer specified**

Version history:
(5):

Purpose: DTCs that are needed only during the development of the ECU shall be defined in a separate range in order for the tools to be able to mask them out if required.
Source: Table "Requirements on implementation of DTCs" - item 2.6
Verification Method: Inspection
Legacy ID: 115163v4

DTCs specified by the implementer that are needed only during the development or quality tracking of the ECU shall be defined in the base DTC range U2F00 to U2FFF.
The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Requirements on detection of faults caused by signal/data content received from other ECUs.*
The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range.
The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range.

2107 v3 **ECU operation cycle**

Version history:

(3): Change the Verification Method to Analysis

Purpose: Reference to requirements on an operation cycle that shall be supported by the ECU.
Source: Section "Definitions- Operation cycle"
Verification Method: Analysis
Legacy ID: 392752v1

Refer to *REQPROD 391191 - ECU operation cycle* and *REQPROD 391192 - ECU start up time, in [Data_1] UDS Services*, for definition of an operation cycle and requirements on the ECU to start and stop an operation cycle.

2108 v1 Emissions related DTCs defined by SAEJ2012

Version history:
(1):

Purpose: Emission related DTCs are defined by SAEJ2012.
Source: Table "Requirements on implementation of DTCs" - item 1.1
Verification Method: Analysis
Legacy ID: 68266v0

For all emissions related ECUs, DTCs with base DTC value in any of the ranges P0000 to P0FFF, P2000 to P2FFF or P3400 to P3FFF shall be implemented as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications.

2109 v2 Non emissions related DTCs defined by SAEJ2012

Version history:
(2):

Purpose: Standardized DTCs defined in SAEJ2012 shall be used.
Source: Table "Requirements on implementation of DTCs" - item 2.1
Verification Method: Analysis
Legacy ID: 68267v1

DTCs with base DTC value in any of the ranges B0000 to B0FFF, B3000 to B3FFF, C0000 to C0FFF, C3000 to C3FFF, U0000 to U0FFF or U3000 to U3FFF shall be implemented as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications and in the sections *Detection of faults in external electrical circuits* to *Detection of faults due to configuration errors and software incompatibility* and *DTCs associated with driver indications* .

2110 v2 Reporting of DTC information

Version history:
(2):

Purpose: It must be possible to extract DTC information from the ECU by standardized services.
Source: Table "Requirements on implementation of DTCs"
Verification Method: Analysis

Legacy ID: 68262v1

DTCs supported by an ECU shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*, and be reported not more than once in any response.

4.5.1.1.1.8 Requirements from section Calibration of DTCs

1804 v2 **Calibration of agedDTCLimit**

Version history:
(2):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 7.
Verification Method: Test
Legacy ID: 408301v0

If not otherwise is specified by this document or is approved by CEVT Electrical Architecture, the AgedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC aging counter (OCC2 or OCC5) shall reach the agedDTCLimit when the fault has not been detected at such a long time that it probably will not occur and be detected again. Typically, it is likely that the detected fault has been self-healed or detected by mistake.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

1805 v2 **Calibration of confirmedDTCLimit**

Version history:
(2):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs. item 6
Verification Method: Test
Legacy ID: 408300v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the confirmedDTCLimit shall be calibrated in such a way that the following is met:
• Calibration of DTC test sample failed is equal to the confirmedDTCLimit when it is very likely that that the most customers observe a symptom (caused by the detected fault) and is bothered by the symptom in such extent that they require repair of the vehicle.
Typically, it is very likely that the detected fault can be considered as permanent (lasting), i.e. repetitive and persistent, and that it occurs and causes customer symptom during the most vehicle operations and environmental conditions.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

1806 v1 Calibration of DTC test sample failed

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 1.
Verification Method: Test
Legacy ID: 408292v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTC test sample failed criteria shall be calibrated in such a way that the following is met:
• The DTC test sample shall fail when the monitored item does not meet the design requirements specified by the implementer.
Examples:
1. The DTC test monitors an external circuit connected to the ECU and it runs periodically each 100 ms. The DTC test sample fail when the circuit is short to ground.
2. The DTC test monitors the maximum end-to-end timing of a signal sent on the in-vehicle network. The DTC test sample fail as soon as the maximum end-to-end timing of the received signal is exceeded.
Refer to section *Definitions and Abbreviations* for definition of DTC test sample.

1807 v1 Calibration of DTC test sample passed

Version history:
(1):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 2.
Verification Method: Test
Legacy ID: 408293v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTC test sample passed criteria shall be calibrated in such a way that the following is met:
• The DTC test sample shall pass when the monitored item meet the design requirements specified by the implementer.
Examples:
1. The DTC test monitors an external circuit connected to the ECU and it runs periodically each 100 ms. The DTC test sample pass when the circuit is not short to ground.
2. The DTC test monitors the maximum end-to-end timing of a signal sent on the in-vehicle network. The DTC test sample pass when the maximum end-to-end timing of the received signal is met.
Refer to section *Definitions and Abbreviations* for definition of DTC test sample.

1808 v2 Calibration of DTCTestFailedLimit

Version history:
(2):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Setion Calibration of DTCs, item 4
Verification Method: Test
Legacy ID: 408296v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTCTestFailedLimit shall be calibrated, i.e. the size of the DTC fault detection counter (FDC 10) step-up (increase) value shall be adjusted, in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to the DTCTestFailedLimit (i.e. +127) when it is very likely that the most customers observe a symptom (caused by the detected fault) but probably not all of them is bothered by the symptom in such extent that they require repair of the vehicle.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

1809 v2 Calibration of DTCTestPassedLimit

Version history:
(2):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Setion Calibration of DTCs, item 5
Verification Method: Test
Legacy ID: 408299v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the DTCTestPassedLimit shall be calibrated, i.e. the size of the DTC fault detection counter (FDC 10) step-down (decrease) value shall be adjusted, in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to the DTCTestPassedLimit (i.e. -128) when the fault has not been detected for a while and the monitored object can currently be considered free from the fault.
Typically, the FDC 10 step-down value is equal or a bit less than the FDC 10 step-up value.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

1810 v2 Calibration of unconfirmedDTCLimit

Version history:
(2):

Purpose: To ensure that DTCs is calibrated correctly.
Source: Section Calibration of DTCs, item 3
Verification Method: Test
Legacy ID: 408294v0

If not otherwise is specified by this document or is approved by the responsible authority appointed by OEM, the unconfirmedDTCLimit shall be calibrated in such a way that the following is met:
• The DTC fault detection counter (FDC10) shall reach a value that is equal to or greater than the unconfirmedDTCLimit when it is likely that at least some customers observer a symptom (caused by the detected fault) but probably only some of them is bothered by the symptom in such extent that they require repair of the vehicle.
Typically, it is very likely that the detected fault is intermittent (temporary), i.e. sporadic and transient, and that it occurs and causes customer symptom during only certain (perhaps unusual) vehicle operation and environment conditions.
Note that customer symptoms may be caused by actions (e.g. reconfigurations) taken by the ECU that tries mitigating the effect (e.g. on customer function, engine emissions or system safety) of the detected fault. If the detected fault causes a driver indication (e.g. telltale, text message, etc.), it shall be regarded as a customer symptom.
Refer to *[Data 12] General Diagnostic Guideline* for recommendations and guidelines on how this requirement shall be met.

4.5.1.1.1.9 Requirements from section Detection of faults

1816 v2 **DTC values**

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68892v1

Unless otherwise specified in the following sections, one unique DTC value shall be used to identify each fault that can be detected by a DTC test in an ECU. Grouping of faults to use one single common DTC is not allowed unless explicitly approved by the responsible authority appointed by OEM, or where the tables within this specification already shows the use of the same DTC. Where faults are combined, all of the associated stored DTC information (e.g. the fault detection counter) is also combined and no more than one instance of the DTC shall be reported in any response to the services specified in reference *[Data_1] UDS Services*. Where the implementer identifies a requirement for separate repair action or fault confirmation process for DTCs that are shown with the same DTC then the responsible authority appointed by OEM may permit the use of different DTCs.

1817 v2 **Failure type value for DTCs**

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68893v1

Unless specifically defined in this document, DTCs shall use Failure Type values which are the

same as the DTCFailureTypeByte defined by *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications , as appropriate depending on the type of fault.
A failure type value of 0x00 may only be used if the Base DTC description includes the failure type and is approved in *[Data_4] Global Master Reference Database*. In such cases the Base DTC may not be combined with any other failure type values.

1818 v2 **Generic Test Run Criteria - car mode transition inhibit criteria**

Version history:
(2):

Purpose: Changing car mode may cause transitional effects that may trigger a DTC test. This must be avoided in order not to set false DTCs.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68889v1

A ll DTC tests shall be inhibited during 5 seconds from any transition from one car mode to another. The inhibition shall start at notational consumption of the 'car mode' signal when a change of car mode is requested and shall be active for 5 s ± 2s. Exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM.

1819 v2 **Generic Test Run Criteria - car modes inhibit criteria**

Version history:
(2):

Purpose: In some car modes vehicle functions may be inhibited which may cause other behaviour than normally expected by the DTC tests.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68891v1

A ll DTC tests shall be inhibited when the vehicle is set in any of the car modes Factory, Transport or Crash.

1820 v3 **Generic Test Run Criteria - enable criteria**

Version history:
(3):

Purpose: A general rule is that all DTC tests shall run all the time that the ECU is "in operation".
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68886v2

DTC tests shall be enabled all the time during an operation cycle provided that no inhibited criteria are met.

If the ECU performs any action due to fault detected by the DTC test (e.g. in order to mitigate the effect of the fault on customer functions or system safety), the DTC tests shall not be inhibited after the ECU has performed the action (e.g. has reduced or turned off the customer functions that are affected by the fault). However, if this is not appropriate and if approved by the responsible authority appointed by OEM, the DTC test may be inhibited in this case.

The DTC tests shall not be inhibited (while the ECU is in operation) when the functionality, which can be affected by fault detected by the DTC test, is not active (e.g. is disabled when the functionalty is not used). However, if this is not appropriate and if approved by the responsible authority appointed by OEM, the DTC test may be inhibited in this case.

1821 v4 Generic Test Run Criteria - low power inhibit criteria

Version history:
(4):

Purpose: DTC tests shall be regarded as unreliable when electrical power availability is limited, i.e. the tests shall be inhibited.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68887v3

If the ECU is power supplied from the 12 V system for which the ElPowerLevel signal is applicable, all DTC tests shall be inhibited when electrical power availability is limited, as indicated by ElPowerLevel 1. Exception: A test shall not be inhibited during engine cranking if the test can detect faults that can occur during cranking and that can prevent the transition to engine running. Other exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM

1822 v1 Generic Test Run Criteria - low voltage inhibit criteria

Version history:
(1):

Purpose: DTC tests shall be regarded as unreliable when the ECU supply voltage is low, i.e. the tests shall be inhibited.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68890v0

A ll DTC tests shall be inhibited when voltage supplied to the ECU is lower than required for ensuring correct fault monitoring. The low voltage limit shall be defined by the implementer but shall not be higher than the lower limit of the supply voltage range specified for the function(s) which includes the monitored component/system, unless otherwise defined in this document.

1823 v4 Generic Test Run Criteria - usage mode transition inhibit criteria

Version history:
(4):

Purpose: Changing usage mode may cause transitional effects that may trigger a DTC test. This must be avoided in order not to set false DTCs.
Source: Section "Detection of faults"
Verification Method: Test
Legacy ID: 68888v3

If not otherwise is specified, all DTC tests shall be inhibited during 5 seconds from any transition from one usage mode to another. The inhibition shall start at notational consumption of the 'usage mode' signal when a change of usage mode is requested and shall be active for 5 s ± 2s. Exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM.

1824 v3 — Test Run Criteria - Updating the DTC Fault Detection Counter

Version history:
(3):

Purpose: If a DTC test is stopped by an inhibit critera the actual test may not be allowed to stop due to safety or other functional requirements. In order to stop updateing the DTC information it is sufficient to stop updating FDC10. Note that if the actual test is not stopped and the ECU can take any action due to faults detected by the DTC test (e.g. reduce or shut off its functionality or request a warring indicator) that for sure can cause customer symptom the updating of the FDC10 should not be stopped (i.e. the test is not inhibited in this case).
Source: Section "Detection of faults"
Verification Method: Analysis
Legacy ID: 68885v2

If a DTC test is inhibited it shall stop updating the DTC Fault Detection Counter (FDC10).

4.5.1.1.1.10 Requirements from section Detection of faults in external electrical circuits

1959 v1 — Base DTC value for detection of low supply voltage

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 10.1
Verification Method: Analysis
Legacy ID: 69021v0

Detection of low supply voltage shall use base DTC value U3003 to identify the fault.

1960 v1 — Base DTC value for faults in high side driver outputs

Version history:

(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 6.1, 6.2 and 6.3
Verification Method: Analysis
Legacy ID: 68961v0

High side driver outputs circuits which has fault detection implemented shall have one unique base DTC value for each output circuit.

1961 v1 Base DTC value for low side driver outputs

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 5.1, 5.2 and 5.3
Verification Method: Analysis
Legacy ID: 68918v0

Low side driver outputs circuits which has fault detection implemented shall have one unique base DTC value for each output circuit.

1962 v1 Base DTC value for short circuit and open circuit faults in FM, PCM, PWM...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis
Legacy ID: 68910v0

The ECU shall implement one unique base DTC value for each FM, PCM, PWM and similar input circuit for detection short circuit to ground faults, short circuit to battery and open circuit faults.

1963 v1 Base DTC value for short circuit to battery or open circuit faults in an...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1 and 2.2
Verification Method: Analysis
Legacy ID: 68903v0

The ECU shall implement one unique base DTC value for each analog input circuit for detection

short circuit to ground faults, short circuit to battery and open circuit faults.

1964 v1 **Base DTC value for switch inputs**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 68899v0

The ECU shall implement one unique base DTC value for each switch input circuit for which fault detection is implemented.

1965 v1 **Detection of CS- or CRC-error on private ECU network**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.3
Verification Method: Test
Legacy ID: 68988v0

If the ECU is a Public ECU master on a network and it is able to detect CS- or CRC-error on received messages on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1966 v1 **Detection of data not updated by transmitter on private ECU network**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.5
Verification Method: Test
Legacy ID: 68994v0

If the ECU is a Public ECU master on a network and it is able to detect that received data has not been updated by the transmitting ECU on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support

generation of DTC information) this fault shall also be identified by an appropriate DTC. Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1967 v2 **Detection of faults for switch inputs**

Version history:
(2):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1 and 1.2
Verification Method: Test
Legacy ID: 68895v1

Where a fault in a switch input circuit connected to the ECU is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for a corresponding DTC.
Otherwise, the ECU shall be able to detect faults in all switch input circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1968 v1 **Detection of incorrect timing and format on private ECU network**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.4
Verification Method: Test
Legacy ID: 68991v0

If the ECU is a Public ECU master on a network and it is able to detect incorrect timing, content, structure and/or format of bits, bytes or messages on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC. Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1969 v1 **Detection of low supply voltage**

Version history:
(1):

Purpose: Low supply voltage is a root fault that may be the cause of many other fault symptoms.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 10.1
Verification Method: Test
Legacy ID: 69006v0

The ECU shall be able to detect if its supply voltage is lower than the vehicle battery voltage according to the requirements in section *Requirements from section Requirements for detection of low supply voltage*.

1970 v1 Detection of missed deadline for message on private ECU network

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.6
Verification Method: Test
Legacy ID: 68998v0

If the ECU is a Public ECU master on a network and it is able to detect that a periodic message is not received within its maximum period on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC. Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1971 v1 Detection of other external electrical circuit faults

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 11.1
Verification Method: Test
Legacy ID: 69023v0

Where a fault in electrical circuits connected to the ECU (e.g., cables, connectors, sensors and actuators including both the electrical and mechanical parts of sensors and actuators) is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding

DTC.
Otherwise, the ECU shall be able to detect other faults in electrical circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1972 v1 **Detection of other faults for analog inputs**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.3
Verification Method: Test
Legacy ID: 68906v0

Where a fault in an analog input circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other faults in all analog input circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1973 v1 **Detection of other faults for FM, PCM, PWM etc. inputs**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.3 and 4.3
Verification Method: Test
Legacy ID: 68912v0

Where a fault in a FM, PCM, PWM and similar input circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other (than short circuit and open circuit) faults in all FM, PCM, PWM and similar input circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1974 v1 **Detection of other faults for high side driver outputs**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.3
Verification Method: Test
Legacy ID: 68965v0

Where a fault in a high side driver output circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other (than short circuit and open circuit) faults in all high side driver outputs circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1975 v1 **Detection of other faults for low side driver outputs**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.3
Verification Method: Test
Legacy ID: 68922v0

Where a fault in a low side driver output circuit connected to the ECU, other than short circuit and open circuit, is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other (than short circuit and open circuit) faults in all low side driver outputs circuits connected to the ECU as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1976 v1 **Detection of other faults on private ECU network**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.8
Verification Method: Test
Legacy ID: 69004v0

If the ECU is a Public ECU master on a network and another fault (than the faults identified in items 9.1 through 9.7 in table *Requirements on detection of faults in external electrical circuits connected to the ECU*) on that network is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate Failure Type value for the corresponding DTC.
Otherwise, the ECU shall be able to detect other faults in the private ECU network as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1977 v1 Detection of physical layer faults on private ECU network

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.7
Verification Method: Test
Legacy ID: 69001v0

If the ECU is a Public ECU master on a network and it is able to detect that the physical layer of the communication link has a short to ground, short to battery voltage or open circuit as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1978 v1 Detection of short circuit and open circuit faults in analog inputs

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1 and 2.2
Verification Method: Test
Legacy ID: 68901v0

The ECU shall be able to detect short circuit to ground faults, and short circuit to battery or open circuit faults, for all analog input circuits connected to the ECU.

1979 v1 **Detection of short circuit and open circuit faults in FM, PCM, PWM etc. ...**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.1, 3.2, 4.1 and 4.2
Verification Method: Test
Legacy ID: 68908v0

The ECU shall be able to detect short circuit and open circuit faults in FM, PCM, PWM and similar input circuits connected to the ECU according to one of the following alternatives.
Alternative 1: short circuit to ground faults and short circuit to battery or open circuit faults.
Alternative 2: short circuit to battery faults and short circuit to ground or open circuit faults.

1980 v1 **Detection of short circuit and open circuit faults in high side driver o...**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 6.1 and 6.2
Verification Method: Test
Legacy ID: 68924v0

The ECU shall be able to detect short circuit to ground faults, and short circuit to battery or open circuit faults, for all high side driver outputs circuits connected to the ECU.

1981 v1 **Detection of short circuit and open circuit faults in low side driver ou...**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 5.1 and 5.2
Verification Method: Test
Legacy ID: 68914v0

The ECU shall be able to detect short circuit to battery faults and short circuit to ground or open circuit faults, for all low side driver outputs circuits connected to the ECU.

1982 v4 **Detection of transmit error on private ECU network**

Version history:
(4):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.1
Verification Method: Test
Legacy ID: 68982v3

If the ECU is a Public ECU master on a network and it is able to detect if transmitted message contain data that does not correspond to data read back by the ECU from the network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM) .

1983 v1 Detection of unexpected transmissions on private ECU network

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.2
Verification Method: Test
Legacy ID: 68985v0

If the ECU is a Public ECU master on a network and it is able to detect if the transmission line is not free when no data transmission is expected on that network as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) this fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect this fault as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1984 v1 DTC value for other external electrical circuit faults

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 11.1
Verification Method: Analysis
Legacy ID: 69025v0

There shall be one unique base DTC value for each output circuit which has a fault monitor for

other faults, and for each fault detected on this circuit a unique Failure Type value shall be used

1985 v1 **Failure type value for CS- or CRC-error on private ECU network**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.3
Verification Method: Test
Legacy ID: 68990v0

DTCs for CS- or CRC-error on private ECU network shall have a Failure Type value 0x83.

1986 v1 **Failure type value for detection of low supply voltage**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 10.1
Verification Method: Analysis
Legacy ID: 69022v0

The DTC for detection of low supply voltage shall use Failure Type value 0x62.

1987 v1 **Failure type value for incorrect timing and format on private ECU network**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.4
Verification Method: Test
Legacy ID: 68993v0

DTCs for incorrect timing and format on private ECU network shall have a Failure Type value 0x86.

1988 v1 **Failure type value for missed deadline for message on private ECU network**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.6
Verification Method: Test
Legacy ID: 69000v0

DTCs for missed deadline for message on private ECU network shall have a Failure Type value 0x87.

1989 v1 **Failure type value for physical layer faults on private ECU network**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.7
Verification Method: Test
Legacy ID: 69003v0

DTCs for physical layer faults on private ECU network shall have a Failure Type value 0x01.

1990 v1 **Failure type value for short circuit and open circuit faults in FM, PCM,...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis
Legacy ID: 68911v0

DTCs for faults in FM, PCM, PWM and similar input circuits shall use Failure Type value according to one of the following alternatives
Alternative 1: I nput circuits which have fault detection for short circuit to battery or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x12 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x33 or 0x34, as appropriate depending on the type of fault, shall be used for both faults.
DTCs for short circuit to ground faults shall use Failure Type value 0x11.
Alternative 2: I nput circuits which have fault detection for short circuit to ground or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x11 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x33 or 0x34, as appropriate depending on the type of fault, shall be used for both faults.
DTCs for short circuit to battery faults shall use Failure Type value 0x12.

1991 v1 **Failure type value for short circuit to battery faults in low side drive...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.1

Verification Method: Analysis
Legacy ID: 68920v0

DTCs for short circuit to battery faults for low side driver outputs circuits shall use Failure Type value 0x12.

1992 v1 Failure type value for short circuit to battery or open circuit faults i...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1
Verification Method: Analysis
Legacy ID: 68904v0

DTCs for detection for short circuit to battery or open circuit faults in analog input circuits that can differentiate between the two faults shall use Failure Type value 0x12 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x15 shall be used for both faults.

1993 v3 Failure type value for short circuit to battery or open circuit faults in high side driver outputs

Version history:
(3):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.1
Verification Method: Analysis
Legacy ID: 68962v2

High side driver outputs circuits which have fault detection for short circuit to battery or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x12 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x15 shall be used for both faults.

1994 v1 Failure type value for short circuit to ground faults in analog inputs

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.2
Verification Method: Analysis
Legacy ID: 68905v0

DTCs for short circuit to ground faults for analog inputs shall use Failure Type value 0x11.

1995 v1 Failure type value for short circuit to ground faults in high side drive...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.2
Verification Method: Analysis
Legacy ID: 68963v0

DTCs for short circuit to ground faults for high side driver outputs circuits shall use Failure Type value 0x11.

1996 v1 Failure type value for short circuit to ground or open circuit faults in...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.2
Verification Method: Analysis
Legacy ID: 68919v0

Low side driver outputs circuits which have fault detection for short circuit to ground or open circuit faults implemented that can differentiate between the two faults shall use Failure Type value 0x11 and 0x13, as appropriate depending on the type of fault. If the fault detection can not differentiate between the two faults then Failure Type value 0x14 shall be used for both faults.

1997 v1 Failure type value for stuck at faults for switch inputs

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1
Verification Method: Analysis
Legacy ID: 68900v0

DTCs for ‘stuck at’ faults for switch inputs shall use Failure Type value 0x23 or 0x24, as appropriate depending on the type of fault.

1998 v1 Failure type value for transmit error on private ECU network

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.1
Verification Method: Test
Legacy ID: 68984v0

DTCs for transmit error on private ECU network shall have a Failure Type value 0x86.

1999 v1 — Failure type value for unexpected transmissions on private ECU network

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.2
Verification Method: Test
Legacy ID: 68987v0

DTCs for unexpected transmissions on private ECU network shall have a Failure Type value 0x81.

2000 v1 — Test period time for detection of other external electrical circuit faults

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 11.1
Verification Method: Analysis
Legacy ID: 69024v0

The DTC tests for the detection of other external electrical circuit faults shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2001 v1 — Test period time for detection of other faults on private ECU network

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.8
Verification Method: Analysis
Legacy ID: 69005v0

The DTC tests for the detection of faults in the private communication network shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2002 v1 **Test period time for detection of unexpected transmissions on private EC...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.2
Verification Method: Analysis
Legacy ID: 68986v0

The DTC tests for the detection of unexpected transmissions on private ECU network shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2003 v1 **Test period time for other faults for analog inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.3
Verification Method: Analysis
Legacy ID: 68907v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for analog inputs shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2004 v1 **Test period time for other faults for FM, PCM, PWM etc. inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 3.3 and 4.3
Verification Method: Analysis
Legacy ID: 68913v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for FM, PCM, PWM and similar input circuits shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2005 v1 **Test period time for other faults for high side driver outputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.3
Verification Method: Analysis
Legacy ID: 68966v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for high side driver outputs circuits shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2006 v1 **Test period time for other faults for low side driver outputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.3
Verification Method: Analysis
Legacy ID: 68923v0

The DTC tests for the detection of other (than short circuit and open circuit) faults for low side driver outputs circuits shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2007 v1 **Test period time for other faults for switch inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.2
Verification Method: Analysis
Legacy ID: 68898v0

The DTC tests for the detection of other (than 'stuck at') faults for switch inputs shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2008 v1 **Test period time for physical layer faults on private ECU network**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.7
Verification Method: Analysis
Legacy ID: 69002v0

The DTC tests for the detection of physical layer faults on private ECU network shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2009 v1 **Test period time for short circuit and open circuit faults in analog inputs**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 2.1 and 2.2
Verification Method: Analysis
Legacy ID: 68902v0

The DTC test for the detection short circuit to ground faults, short circuit to battery and open circuit faults in analog inputs shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

2010 v1 **Test period time for short circuit and open circuit faults in FM, PCM, P...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" items 3.1, 3.2, 4.1 and 4.2
Verification Method: Analysis
Legacy ID: 68909v0

The DTC test for the detection short circuit to ground faults, short circuit to battery and open

circuit faults in FM, PCM, PWM and similar input circuits shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

2011 v1 **Test period time for short circuit and open circuit faults in high side ...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 6.1 and 6.2
Verification Method: Analysis
Legacy ID: 68941v0

The DTC test for the detection short circuit to battery faults, short circuit to ground and open circuit faults in high side driver outputs circuits shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

2012 v1 **Test period time for short circuit and open circuit faults in low side d...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - items 5.1 and 5.2
Verification Method: Analysis
Legacy ID: 68915v0

The DTC test for the detection short circuit to ground faults, short circuit to battery and open circuit faults in low side driver outputs circuits shall produce Test Samples periodically every 100 ms or faster while the Test Run Criteria are met.

2013 v3 **Test period time for stuck at faults for switch inputs**

Version history:
(3):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 1.1
Verification Method: Analysis
Legacy ID: 68897v2

The DTC test for the detection of ‘stuck at’ faults for switch inputs shall produce Test Samples periodically every 500 ms or faster while the Test Run Criteria are met.

2014 v1 **Test Run Criteria for CS- or CRC-error on private ECU network**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.3
Verification Method: Analysis
Legacy ID: 68989v0

The DTC test for CS- or CRC-error on private ECU network shall run each time a message is received on the network by the ECU.

2015 v1 Test Run Criteria for data not updated by transmitter on private ECU net...

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.5
Verification Method: Analysis
Legacy ID: 68995v0

The DTC test for data not updated by transmitter on private ECU network shall run each time a message is received on the network by the ECU.

2016 v1 Test Run Criteria for detection of transmit error on private ECU network

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.1
Verification Method: Analysis
Legacy ID: 68983v0

The DTC test for transmit error on private ECU network shall run each time a message is transmitted on the network by the ECU.

2017 v2 Test Run Criteria for fault detection in external circuits

Version history:
(2):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Section "Detection of faults in external electrical circuits"
Verification Method: Analysis

Legacy ID: 68894v1

Any fault detection test for external circuits using shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

2018 v1 Test Run Criteria for incorrect timing and format on private ECU network

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.4
Verification Method: Analysis
Legacy ID: 68992v0

The DTC test for incorrect timing and format on private ECU network shall run each time a message is received on the network by the ECU.

2019 v1 Test Run Criteria for missed deadline for message on private ECU network

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 9.6
Verification Method: Analysis
Legacy ID: 68999v0

The DTC test for missed deadline for message on private ECU network shall run each time a message is expected to be received on the network by the ECU.

2020 v1 Test Run Criteria for short circuit to battery faults in low side driver...

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.1
Verification Method: Test
Legacy ID: 68916v0

The DTC test for detection of short circuit to battery faults in a low side driver output shall run at least while the output is activated.

2021 v1 **Test Run Criteria for short circuit to battery or open circuit faults in high side driver outputs**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table 4-7, item 6.1
Verification Method: Test
Legacy ID: 151986v0

The DTC test for detection of short circuit to battery or open circuit faults in a high side driver output shall run at least while the output is deactivated.

2022 v1 **Test Run Criteria for short circuit to ground faults in high side driver...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 6.2
Verification Method: Test
Legacy ID: 68964v0

The DTC test for detection of short circuit to ground faults in a high side driver output shall run at least while the output is activated.

2023 v1 **Test Run Criteria for short circuit to ground or open circuit faults in ...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in external electrical circuits connected to the ECU" - item 5.2
Verification Method: Test
Legacy ID: 68917v0

The DTC test for detection of short circuit to ground or open circuit faults in a low side driver output shall run at least while the output is deactivated.

2024 v1 **type value for data not updated by transmitter on private ECU network**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in external electrical circuits

connected to the ECU" - item 9.5

Verification Method: Test

Legacy ID: 68996v0

DTCs for data not updated by transmitter on private ECU network shall have a Failure Type value 0x82.

4.5.1.1.1.11 Requirements from section Detection of low supply voltage

2079 v3 **Accuracy of vehicle battery voltage measurement**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: The vehicle battery voltage must be measured with a known minimum accuracy.

Source: Section "Requirements for detection of low supply voltage"

Verification Method: Analysis

Legacy ID: 69501v1

The vehicle battery voltage shall be measured with a minimum accuracy of ±0.25 V .

2080 v1 **AgedDTCLimit or low supply voltage fault detection**

Version history:
(1):

Purpose: To define exact critera for setting the agedDTC status.

Source: Section "Requirements for detection of low supply voltage"

Verification Method: Test

Legacy ID: 409254v0

The DTC shall have agedDTCLimit = 255.

2081 v3 **Averaging of ECU supply voltage measurement**

Version history:
(3):

Purpose: To be able to use the measured ECU supply voltage the measured value must be filtered through an averaging function.

Source: Section "Requirements for detection of low supply voltage"

Verification Method: Analysis

Legacy ID: 69523v2

A new Uecu value shall be calculated every time a sample is taken. It shall be calculated as the average value of the samples taken during the last 500 ms period.

2082 v3 **Averaging of vehicle battery voltage measurement**

Version history:
(3):

Purpose: To be able to use the measured vehicle battery voltage the measured value must be filtered through an averaging function.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Analysis
Legacy ID: 69514v2

A new Ubat value shall be calculated every time a sample is taken. It shall be calculated as the average value of the samples taken during the last 500 ms period.

2083 v1 ConfirmedDTCLimit for low supply voltage fault detection

Version history:
(1):

Purpose: To define exact critera for setting the confirmedDTC status
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Test
Legacy ID: 69532v0

The DTC test for low supply voltage shall have confirmedDTCLimit = 10.

2084 v4 Distribution of the vehicle battery voltage value

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: To be able to use the calculated vehicle battery voltage value in for comparison with the supply voltage in all other ECUs the calculated value must be distributed on the in-vehicle networks.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Analysis
Legacy ID: 69517v2

The calculated Ubat values shall be sent periodically and received by all Public ECUs with a maximum end-to-end time (including sensing, signaling and actuating time) of 100 ms.

2085 v3 ECU supply voltage measurement

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: The ECU supply voltage must be measured with a known minimum accuracy.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Analysis
Legacy ID: 69518v1

The ECU shall measure its supply voltage with a minimum accuracy of ±0.25 V .

2086 v3 Sample time for ECU supply voltage measurement

Version history:

(3): Change the Verification Method from Test to Analysis

Purpose: The ECU supply voltage must be measured with a known minimum sample time.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Analysis
Legacy ID: 69521v1

The ECU supply voltage shall be sampled every 50 ms or faster.

2087 v3 Sample time for vehicle battery voltage measurement

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: The vehicle battery voltage must be measured with a known minimum sample time.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Analysis
Legacy ID: 69503v1

The vehicle battery voltage Ubat shall be sampled every 50 ms or faster.

2088 v1 Test Run Criteria for low supply voltage fault detection

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Test
Legacy ID: 69525v0

Low supply voltage fault detection shall only be done when all of following conditions are fulfilled:
$U_{bat} \geq 12$ V
AND
UsageMode == Driving

2089 v1 Test sample failed criteria for low supply voltage fault detection

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Test
Legacy ID: 69527v0

For each new U_{ecu} value being calculated the ECU shall compare the U_{ecu} value with the received U_{bat} value:
- if $U_{ecu} < (U_{bat} - 3V)$ then the DTC fault detection counter (FDC10) shall be incremented by 3.

2090 v1 **Test sample passed criteria for low supply voltage fault detection**

Version history:
(1):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Test
Legacy ID: 69528v0

For each new U_{ecu} value being calculated the ECU shall compare the U_{ecu} value with the received U_{bat} value:
- if $U_{ecu} >= (U_{bat} - 3V)$ then the DTC fault detection counter (FDC10) shall be decremented by 6

2091 v1 **UnconfirmedDTCLimit for low supply voltage fault detection**

Version history:
(1):

Purpose: To define exact critera for setting the UnconfirmedDTC status
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Test
Legacy ID: 69531v0

The DTC test for low supply voltage shall have unconfirmedDTCLimit = 6.

2092 v1 **Vehicle battery voltage measurement**

Version history:
(1):

Purpose: To define a robust mechanism for measurement of the vehicle battery voltage.
Source: Section "Requirements for detection of low supply voltage"
Verification Method: Analysis
Legacy ID: 69493v0

The vehicle battery voltage shall be measured, by a dedicated ECU, as close to the actual battery terminals as possible (such that the measurement is unaffected by any internal energy storage in the measuring ECU).

4.5.1.1.1.12 Requirements from section Detection of faults in internal electrical circuits

2025 v1 **Base DTC value for detection of data integrity fault in non volatile memory**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69556v0

Detection of data integrity fault in non volatile memory shall use one unique base DTC value for all non volatile memory in the ECU.

2026 v1 Base DTC value for detection of data integrity fault in volatile memory

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69561v0

Detection of data integrity fault in volatile memory shall use one unique base DTC value for all volatile memory in the ECU.

2027 v1 Base DTC value for detection of faults in other internal parts

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69611v0

Detection of faults in other internal parts shall use one unique base DTC value for each internal part of the ECU that is monitored by a test (e.g. component , circuit, system or function) to identify the fault.

2028 v1 Base DTC value for detection of internally connected actuator fault

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 4.1
Verification Method: Analysis
Legacy ID: 69590v0

Detection of internally connected actuator fault shall use one unique base DTC value for each actuator to identify the fault.

2029 v1 **Base DTC value for detection of internally connected sensor fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69602v0

Detection of internally connected sensor fault shall use one unique base DTC value for each sensor to identify the fault.

2030 v2 **Detection of data integrity fault in non volatile memory**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69551v0

Where a data integrity fault in non volatile memory is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect data integrity fault in non volatile memory as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The specific data integrity verification methods used by the ECU are specified by the implementer.

2031 v2 **Detection of data integrity fault in volatile memory**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69558v0

Where a data integrity fault in volatile memory is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to

support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect data integrity fault in volatile memory as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The specific data integrity verification methods used by the ECU are specified by the implementer.

2032 v1 **Detection of faults in other internal parts**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69608v0

Where a fault in another internal part (other than identified in items 1.1 through 5.1 in table *Requirements on detection of faults in internal electrical circuits*) is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect faults in other internal parts as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2033 v2 **Detection of internal device communication error**

Version history:
(2):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69564v1

Where internal device communication errors are detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect internal device communication errors as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved

by the responsible authority appointed by OEM).

2034 v2 **Detection of internal faults not on the main circuit board**

Version history:
(2):

Purpose: Faults not on the main circuit board shall be treated as if they were external faults.
Source: Section "Detection of faults in internal electrical circuits"
Verification Method: Analysis
Legacy ID: 69540v1

Where an ECU, or a non repairable (by the workshop) assembly unit containing the ECU, contains circuitry (actuators, sensors etc) internal to the ECU but not located on the main circuit board, the ECU shall be able to detect faults in these internal electrical circuits according to section *Detection of faults in external electrical circuits* as if they were external electrical circuits.

2035 v1 **Detection of internally connected actuator fault**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 4.1
Verification Method: Analysis
Legacy ID: 69584v0

Where an internally connected actuator fault is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect internally connected actuator faults as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
When determining the requirement for internal actuator failures the implementer shall consider the possible faults of the internal actuator and its internal physical and electrical interface with the ECU hardware.

2036 v1 **Detection of internally connected sensor fault**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1

Verification Method: Analysis
Legacy ID: 69596v0

Where an internally connected sensor fault is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect internally connected sensor faults as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
When determining the requirement for internal sensor failures the implementer shall consider the possible faults of the internal sensor and its internal physical and electrical interface with the ECU hardware.

2037 v1 Detection of programme execution fault

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69573v0

Where a programme execution fault is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect programme execution faults as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
The specific data execution monitoring methods used by the ECU shall be specified by the implementer.

2038 v1 DTC value for detection of internal device communication error

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69571v0

Detection of internal device communication error shall use on unique base DTC value for each monitored device to identify the fault.

2039 v1 **DTC value for detection of programme execution fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69576v0

Detection of programme execution fault shall use one unique base DTC value for each execution unit in the ECU to identify the fault.

2040 v1 **Failure type value for detection of data integrity fault in non volatile...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69557v0

DTCs for detection of data integrity fault in non volatile memory shall use Failure Type value 0x45 or 0x46.

2041 v1 **Failure type value for detection of data integrity fault in volatile memory**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69562v0

DTCs for detection of data integrity faults in volatile memory shall use Failure Type value 0x44.

2042 v1 **Failure type value for detection of faults in other internal parts**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69612v0

DTCs for detection of faults in other internal parts shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications .

2043 v1 **Failure type value for detection of internal device communication error**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69572v0

DTCs for detection of internal device communication error shall use Failure Type value 0x49.

2044 v1 **Failure type value for detection of internally connected actuator fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69595v0

DTCs for detection of internally connected actuator fault shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications .

2045 v1 **Failure type value for detection of internally connected sensor fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 5.1
Verification Method: Analysis
Legacy ID: 69605v0

DTCs for detection of internally connected sensor fault shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications .

2046 v1 **Failure type value for detection of programme execution fault**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69581v0

DTCs for detection of programme execution fault shall use Failure Type value 0x47.

2047 v1 Test period time for detection of faults in other internal parts

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 6.1
Verification Method: Analysis
Legacy ID: 69610v0

The DTC tests for the detection of faults in other internal parts shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2048 v1 Test period time for detection of internally connected actuator fault

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 4.1
Verification Method: Analysis
Legacy ID: 69588v0

The DTC tests for the detection of internally connected actuator fault shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2049 v1 Test period time for detection of internally connected sensor fault

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item

5.1
Verification Method: Analysis
Legacy ID: 69601v0

The DTC tests for the detection of internally connected sensor fault shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2050 v1 Test Run Criteria and Test Period time for detection of data integrity f...

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.2
Verification Method: Analysis
Legacy ID: 69560v0

The test run criteria and test period time for detection of data integrity fault in volatile memory shall be defined by the implementer.

2051 v1 Test Run Criteria and Test Period time for detection of data integrity f...

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 1.1
Verification Method: Analysis
Legacy ID: 69554v0

The test run criteria and test period time for detection of data integrity fault in non volatile memory shall be defined by the implementer.

2052 v2 Test Run Criteria and Test period time for detection of programme execution fault

Version history:
(2):

Purpose: The test run criteria and test period time for detection of programme execution fault are specific for each implementation and must be defined by the implementer.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 3.1
Verification Method: Analysis
Legacy ID: 69575v1

The test run criteria and test period time for detection of programme execution fault shall be defined by the implementer.

2053 v1 **Test Run Criteria for detection of internal device communication error**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults in internal electrical circuits" - item 2.1
Verification Method: Analysis
Legacy ID: 69565v0

If the detection of internal device communication error is implemented the DTC test shall run at each communication with the internal device.

2054 v2 **Test Run Criteria for fault detection in internal circuitsk**

Version history:
(2):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Section "Detection of faults in internal electrical circuits"
Verification Method: Analysis
Legacy ID: 69547v1

Any fault detection test for internal circuits shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

4.5.1.1.1.13 Requirements from section Detection of faults inferred from system behavior

2055 v1 **Base DTC value for comparison of system behaviour with predictive model**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69657v0

Comparison of system behaviour with predictive model shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

2056 v1 **Base DTC value for detection of closed loop control system out of control**

Version history:

(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69662v0

Detection of closed loop control system out of control shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

2057 v1 Base DTC value for detection of failure to react after event

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69644v0

Detection of failure to react after event shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

2058 v1 Base DTC value for detection of other faults inferred from system behaviour

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 5.1
Verification Method: Analysis
Legacy ID: 69669v0

Detection of other faults faults inferred from system behaviour shall use one unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

2059 v1 Base DTC value for detection of system adaptation out of range

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Analysis
Legacy ID: 69666v0

Detection of system adaptation out of range shall use a unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

2060 v2 Comparison of system behaviour with predictive model

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69647v0

If the ECU is able to detect if the measured behaviour of a component or system do not follow the predictions of a simulation of expected behaviour which is being run by the ECU , as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if the measured behaviour of a component or system do not follow the predictions of a simulation of expected behaviour which is being run by the ECU , as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2061 v2 Detection of closed loop control system out of control

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69658v0

If the ECU is able to detect if a closed loop control is not stable under conditions where stability would be expected , as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if a closed loop control is not stable under conditions where stability would be expected , as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2062 v2 **Detection of failure to react after event**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Where no direct fault detection mechanism is available, or where it is inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69628v0

If the ECU is able to detect if sensor readings do not change as a result of some change actuated by the ECU, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if sensor readings do not change as a result of some change actuated by the ECU, as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2063 v1 **Detection of other faults inferred from system behaviour**

Version history:
(1):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 5.1
Verification Method: Analysis
Legacy ID: 69667v0

Where another fault inferred from system behaviour is detected by a fault monitor as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect faults inferred from system behaviour as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2064 v2 **Detection of system adaptation out of range**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Where no direct fault detection mechanism is available, or where it is

inadequate, the ECU shall be able to indirectly detect faults in the system and components associated with the ECU.

Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1

Verification Method: Analysis

Legacy ID: 69663v0

If the ECU is able to detect if a ystem where the ECU adapts to manufacturing tolerance, component ageing etc., has exceeded the limit of its adaptation range , as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect if a ystem where the ECU adapts to manufacturing tolerance, component ageing etc., has exceeded the limit of its adaptation range , as defined by a feasibility study performed by the implementer. (The feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

2065 v1 Failure type value for comparison of system behaviour with predictive model

Version history:
(1):

Purpose: To enable easy identification of specific faults.

Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1

Verification Method: Analysis

Legacy ID: 69653v0

DTCs for comparison of system behaviour with predictive model shall use Failure Type value 0x61.

2066 v1 Failure type value for detection of closed loop control system out of co...

Version history:
(1):

Purpose: To enable easy identification of specific faults.

Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1

Verification Method: Analysis

Legacy ID: 69661v0

DTCs for detection of closed loop control system out of control shall use Failure Type value 0x06.

2067 v1 Failure type value for detection of failure to react after event

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69642v0

DTCs for detection of failure to react after event shall use Failure Type value 0x67.

2068 v1 **Failure type value for detection of other faults inferred from system be...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 5.1
Verification Method: Analysis
Legacy ID: 69670v0

DTCs for detection of other faults faults inferred from system behaviour shall use a unique Failure Type value for each fault type as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications.

2069 v1 **Failure type value for detection of system adaptation out of range**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Analysis
Legacy ID: 69665v0

DTCs for detection of system adaptation out of range shall use Failure Type value 0x91.

2070 v1 **Test period time for comparison of system behaviour with predictive model**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Analysis
Legacy ID: 69652v0

If comparison of system behaviour with predictive model is implemented the DTC test shall run periodically every 100 ms or faster whenever the Test Run Criteria are met.

2071 v1 **Test period time for detection of closed loop control system out of control**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Analysis
Legacy ID: 69660v0

If the detection of closed loop control system out of control is implemented the DTC test shall run periodically every 100 ms or faster whenever the Test Run Criteria are met.

2072 v1 **Test period time for detection of failure to react after event**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Analysis
Legacy ID: 69639v0

If the detection of failure to react after event is implemented the DTC test shall run periodically every 100 ms or faster whenever the Test Run Criteria are met.

2073 v1 **Test period time for detection of other faults inferred from system beha...**

Version history:
(1):

Purpose: To ensure that faults are detected fast enough and that the presence of faults are continuously monitored.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item
Verification Method: Analysis
Legacy ID: 69668v0

The DTC tests specified by the implementer for the detection of othe faults inferred from system behaviour shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

2074 v1 **Test Run Criteria for comparison of system behaviour with predictive model**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 2.1
Verification Method: Test
Legacy ID: 69648v0

If comparison of system behaviour with predictive model is implemented the DTC test shall run whenever the function being simulated in the predictive model is active.

2075 v1 Test Run Criteria for detection of closed loop control system out of con...

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 3.1
Verification Method: Test
Legacy ID: 69659v0

If the detection of closed loop control system out of control is implemented the DTC test shall run whenever stable closed loop control is actively expected.

2076 v1 Test Run Criteria for detection of failure to react after event

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 1.1
Verification Method: Test
Legacy ID: 69631v0

If the detection of failure to react after event is implemented the DTC test shall run whenever changes from event are actively expected.

2077 v1 Test Run Criteria for detection of faults inferred from system behaviour

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source:
Verification Method: Inspection
Legacy ID: 115341v0

Any fault detection test for detection of faults inferred from system behaviour shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

2078 v1 **Test Run Criteria for detection of system adaptation out of range**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults inferred from system behaviour" - item 4.1
Verification Method: Test
Legacy ID: 69664v0

If the detection of system adaptation out of range is implemented the DTC test shall run whenever any adaptation value is updated.

4.5.1.1.1.14 Requirements from section Detection of faults due to configuration errors

1873 v2 **agedDTCLimit for inconsistent or incompatible vehicle configuration data**

Version history:
(2):

Purpose: To define exact critera for DTC aging.
Source: Table "Calibration of configuration DTCs" - item 2.5
Verification Method: Test
Legacy ID: 70833v1

The DTC test for inconsistent or incompatible vehicle configuration data shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1874 v2 **agedDTCLimit for invalid vehicle configuration data**

Version history:
(2):

Purpose: To define exact critera for DTC aging.
Source: Table "Calibration of configuration DTCs" - item 2.3
Verification Method: Test
Legacy ID: 70784v1

The DTC test for invalid vehicle configuration data shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1875 v2 **agedDTCLimit for missing Car Configuration Logic (CCL)**

Version history:

(2):

Purpose: To define exact critera for DTC aging.
Source: Table "Calibration of configuration DTCs" - item 4.2
Verification Method: Test
Legacy ID: 70635v1

The DTC test for missing Car Configuration Logic (CCL) shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1876 v2 agedDTCLimit for missing vehicle configuration data

Version history:
(2):

Purpose: To define exact critera for DTC aging
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 69762v1

The DTC test for missing vehicle configuration data shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1877 v2 agedDTCLimit for missing Vehicle Configuration Data (VCD)

Version history:
(2):

Purpose: To define exact critera for DTC aging
Source: Table "Calibration of configuration DTCs" - item 5.1
Verification Method: Test
Legacy ID: 70607v1

The DTC test for missing Vehicle Configuration Data (VCD) shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1878 v2 agedDTCLimit for new configuration not learned

Version history:
(2):

Purpose: To define exact critera for DTC aging.
Source: Table "Calibration of configuration DTCs" - item 2.4
Verification Method: Test
Legacy ID: 70818v1

The DTC test for new configuration not learned shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1879 v2 **agedDTCLimit for VCD and CCL data inconsistency**

Version history:
(2):

Purpose: To define exact critera for DTC aging
Source: Table "Calibration of configuration DTCs" - item 4.1
Verification Method: Test
Legacy ID: 70723v1

The DTC test for VCD and CCL data inconsistency shall use Operation cycle counter #2 (OCC2) as the counter used for aging and shall have agedDTCLimit = 1 (provided that the DTC is not emission related) .

1880 v1 **Base DTC value for detection of software incompatibility with another ECU**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.1
Verification Method: Analysis
Legacy ID: 70842v0

The DTC test for detection of software incompatibility with another ECU shall use the base DTC value U03XX if defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications , otherwise base DTC value U2DXX shall be used to identify this fault.

1881 v1 **Clearing of DTC for detection of suppler software installed in ECU.**

Version history:
(1):

Purpose: The DTC for indicating that the ECU has supplier specific software installed shall not be possible to clear except by replacing the software.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.2
Verification Method: Analysis
Legacy ID: 70845v0

The DTC U2400-57 shall always be set as long as the supplier specific software is installed in the ECU. It shall not be possible to delete (clear) this DTC except by erasing or replacing the supplier software. The DTC shall not be supported by any OEM authorised application software.

1882 v1 **ConfirmedDTCLimit for inconsistent or incompatible vehicle configuration...**

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 2.5
Verification Method: Test
Legacy ID: 70832v0

The DTC test for inconsistent or incompatible vehicle configuration data shall have confirmedDTCLimit = 1.

1883 v1 ConfirmedDTCLimit for invalid vehicle configuration data

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status.
Source: Table "Calibration of configuration DTCs" - item 2.3
Verification Method: Test
Legacy ID: 70774v0

The DTC test for invalid vehicle configuration data shall have confirmedDTCLimit = 1.

1884 v1 ConfirmedDTCLimit for missing Car Configuration Logic (CCL)

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status.
Source: Table "Calibration of configuration DTCs" - item 4.2
Verification Method: Test
Legacy ID: 70632v0

The DTC test for missing Car Configuration Logic (CCL) shall have confirmedDTCLimit = 1.

1885 v1 ConfirmedDTCLimit for missing vehicle configuration data

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 69743v0

The DTC test for missing vehicle configuration data shall have confirmedDTCLimit = 1.

1886 v1 ConfirmedDTCLimit for missing Vehicle Configuration Data (VCD)

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 5.1
Verification Method: Test

Legacy ID: 70606v0

The DTC test for missing Vehicle Configuration Data (VCD) shall have confirmedDTCLimit = 1.

1887 v1 ConfirmedDTCLimit for new configuration not learned

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status.
Source: Table "Calibration of configuration DTCs" - item 2.4
Verification Method: Test
Legacy ID: 70809v0

The DTC test for new configuration not learned shall have confirmedDTCLimit = 1.

1888 v1 ConfirmedDTCLimit for VCD and CCL data inconsistency

Version history:
(1):

Purpose: To define exact critera for setting confirmedDTC status.
Source: Table "Calibration of configuration DTCs" - item 4.1
Verification Method: Test
Legacy ID: 70721v0

The DTC test for VCD and CCL data inconsistency shall have confirmedDTCLimit = 1.

1889 v3 Detection of data integrity fault in configuration data memory

Version history:
(3):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 3.1; Central Car Configuration - SPA, UDS - Data - Requisite documents
Verification Method: Analysis
Legacy ID: 70656v1

If the ECU is a Car Configuration Domain Master the ECU shall be able to detect data integrity faults in long-term memory used for storing VCD, CCL and VCP data.
The test shall be based on the data integrity mechanism according to *[Data_5] Central Car Configuration*.

1890 v2 Detection of incompatible VIN

Version history:
(2):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing

software components" - item 7.1

Verification Method: Analysis

Legacy ID: 70838v1

If t he ECU has a fault monitor implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) to detect if the locally stored VIN does not match the VIN transmitted from another ECU, this shall also be used as a DTC test.

1891 v2 Detection of inconsistent or incompatible vehicle configuration data

Version history:
(2): Change the Verification Method to Analysis

Purpose: Basic requirement to enable detection of faults.

Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.5

Verification Method: Analysis

Legacy ID: 70823v0

If t he ECU has a fault monitor implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) to detect if any loaded configuration parameter has a valid value but is internally inconsistent or incompatible with some other data or configuration in the ECU or an ECU hardware option, this test shall also be used as a DTC test.

1892 v2 Detection of invalid local configuration data

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that it can be detected if an ECU has invalid local configuration data.

Source: Table "Requirements on detection of faults caused by local ECU Configuration (local parameters)" - item 1.1

Verification Method: Analysis

Legacy ID: 69701v0

If the ECU is required to be programmed with local configuration (or calibration) data for correct operation, the ECU shall be able to detect if it has has one or more required local configuration parameters with unknown or invalid value.

1893 v4 Detection of invalid vehicle configuration data

Version history:
(4):

Purpose: Basic requirement to enable detection of faults.

Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.3; Central Car Configuration - SPA, UDS - Data - Requisite documents

Verification Method: Analysis
Legacy ID: 70745v2

If the ECU is a Car Configuration Subscriber the ECU shall be able to detect invalid vehicle configuration data, according to *[Data_5] Central Car Configuration*.

1894 v5 Detection of missing Car Configuration Logic (CCL)

Version history:
(5): Change the Verification Method from Test to Analysis

Purpose: To ensure that it can be detected if a Car Configuration Domain Master ECU is missing Car Configuration Logic data.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.2; Central Car Configuration - SPA, UDS - Data - Requisite documents
Verification Method: Analysis
Legacy ID: 70611v2

If the ECU is a Car Configuration Domain Master the ECU shall be able to detect missing Car Configuration Logic data (CCL), according to *[Data_5] Central Car Configuration*.

1895 v5 Detection of missing vehicle configuration data (ECU never configured)

Version history:
(5): Change the Verification Method from Test to Analysis

Purpose: To ensure that it can be detected if an ECU has never received the required VCP data.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.1; Central Car Configuration - SPA Req18, UDS - Data - Requisite documents
Verification Method: Analysis
Legacy ID: 69705v2

If the ECU is a Car Configuration Subscriber and it is in ‘Bulk state’ it shall be able to detect missing vehicle configuration data, according to *[Data_5] Central Car Configuration*.

1896 v5 Detection of missing Vehicle Configuration Data (VCD)

Version history:
(5): Change the Verification Method from Test to Analysis

Purpose: To ensure that it can be detected if a Car Configuration Domain Master ECU is missing vehicle configuration data (VCD).
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" ; Central Car Configuration - SPA, UDS - Data - Requisite documents
Verification Method: Analysis
Legacy ID: 69831v2

If the ECU is a Car Configuration Domain Master the ECU shall be able to detect missing Vehicle Configuration Data (VCD), according to reference *[Data_5] Central Car Configuration*.

1897 v5 Detection of new configuration not learned

Version history:
(5): Change the Verification Method from Test to Analysis

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.4
Verification Method: Analysis
Legacy ID: 70793v2

I f the ECU requires an additional control routine to instruct the ECU to learn a changed configuration, the ECU shall be able to detect that a new valid configuration has been received but not learned, according to *[Data_5] Central Car Configuration*.

1898 v1 Detection of software incompatibility with another ECU

Version history:
(1):

Purpose: To ensure that it can be detected if an ECU has software loaded that is incompatible with another ECU.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.1
Verification Method: Analysis
Legacy ID: 70841v0

If t he ECU has a fault monitor implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) where the application/strategy/data level of another ECU can be checked and indicate software incompatibility with the other ECU, this test hsall also be used as a DTC test.

1899 v1 Detection of suppler specific software installed in ECU

Version history:
(1):

Purpose: To ensure that an ECU with supplier specific software installed will not pass undetected through the manufacturing process.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.2
Verification Method: Analysis
Legacy ID: 70844v0

If t he ECU is delivered to CEVT with supplier specific software that must be replaced by CEVT authorised application software as part of the manufacturing process this shall be identified by a confirmed DTC with the DTC value U2400-57.

1900 v3 Detection of VCD and CCL data inconsistency

Version history:
(3):

Purpose: Basic requirement to enable detection of faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.11; Central Car Configuration - SPA, UDS - Data - Requisite documents
Verification Method: Analysis
Legacy ID: 70664v1

If the ECU is a Car Configuration Domain Master the ECU shall be able to detect if incompatible VCD and CCL data are loaded, according to *[Data_5] Central Car Configuration*.

1901 v1 DTC value for detection of CCL part no inconsistency

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.1
Verification Method: Analysis
Legacy ID: 70741v0

Detection of CCL part no inconsistency shall use base DTC value U2302 to identify the fault.

1902 v1 DTC value for detection of data integrity fault in configuration data me...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 3.1
Verification Method: Analysis
Legacy ID: 70662v0

Detection of data integrity faults in configuration data memory shall use base DTC value U2301 to identify the fault.

1903 v2 DTC value for detection of incompatible VIN

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 7.1
Verification Method: Analysis
Legacy ID: 70840v1

Detection of incompatible VIN shall use the base DTC values P0630, P0631, C0545 or C0546 to identify the fault.

1904 v1 **DTC value for detection of inconsistent or incompatible vehicle configur...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.5
Verification Method: Analysis
Legacy ID: 70835v0

T he detection of inconsistent or incompatible configuration shall use base DTC value U2300 to identify the fault.

1905 v1 **DTC value for detection of invalid local configuration data**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by local ECU Configuration (local parameters)" - item 1.1
Verification Method: Analysis
Legacy ID: 69703v0

Detection of invalid local configuration data shall use base DTC value U2101 to identify the fault.

1906 v1 **DTC value for detection of invalid vehicle configuration data**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.3
Verification Method: Analysis
Legacy ID: 70787v0

Detection of invalid configuration data shall use base DTC value U2300 to identify the fault.

1907 v1 **DTC value for detection of missing Car Configuration Logic data**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU

Configuration (global parameters)" - item 4.2
Verification Method: Analysis
Legacy ID: 70654v0

Detection of missing Car Configuration Logic data shall use base DTC value U2302 to identify the fault.

1908 v1 DTC value for detection of missing vehicle configuration data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.1
Verification Method: Analysis
Legacy ID: 69802v0

Detection of missing vehicle configuration data shall use base DTC value U2300 to identify the fault.

1909 v2 DTC value for detection of missing Vehicle Configuration Data (VCD)

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 5.1
Verification Method: Analysis
Legacy ID: 70609v1

Detection of missing vehicle configuration data (VCD) shall use base DTC value U2303 to identify the fault.

1910 v1 DTC value for detection of new configuration not learned

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.4
Verification Method: Analysis
Legacy ID: 70821v0

Detection of new configuration not learned shall use base DTC value U2300 to identify the fault.

1911 v1 Failure type value for detection of CCL part no inconsistency

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.1
Verification Method: Analysis
Legacy ID: 70743v0

DTCs for detection of CCL part no inconsistency shall use Failure Type value 0x57.

1912 v1 Failure type value for detection of data integrity fault in configuratio...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 3.1
Verification Method: Analysis
Legacy ID: 70663v0

DTCs for detection of data integrity faults in configuration data memory shall use Failure Type value 0x41.

1913 v1 Failure type value for detection of inconsistent or incompatible vehicle...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.5
Verification Method: Analysis
Legacy ID: 70837v0

The DTCs for detection of inconsistent or incompatible configuration shall use Failure Type value 0x57.

1914 v1 Failure type value for detection of invalid local configuration data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by local ECU Configuration (local parameters)" - item 1.1
Verification Method: Analysis
Legacy ID: 69704v0

DTCs for detection of invalid local configuration data shall use Failure Type value 0x56.

1915 v1 Failure type value for detection of invalid vehicle configuration data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.3
Verification Method: Analysis
Legacy ID: 70790v0

The DTC for detection of invalid configuration data shall use Failure Type value 0x56.

1916 v1 Failure type value for detection of missing Car Configuration Logic data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.2
Verification Method: Analysis
Legacy ID: 70655v0

DTCs for detection of missing Car Configuration Logic data shall use Failure Type value 0x55.

1917 v2 Failure type value for detection of missing Vehicle Configur. Data (VCD)

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 5.1;
Verification Method: Analysis
Legacy ID: 70610v1

DTCs for detection of missing vehicle configuration data (VCD) shall use Failure Type value 0x55.

1918 v3 Failure type value for detection of missing vehicle configuration data

Version history:
(3):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.1
Verification Method: Analysis
Legacy ID: 69803v2

DTCs for detection of missing vehicle configuration data shall use Failure Type value 0x55.

1919 v1 **Failure type value for detection of software incompatibility with anothe...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 8.1
Verification Method: Analysis
Legacy ID: 70843v0

The DTCs for detection of software incompatibility with another ECU shall use Failure Type value:
0x4A if module must be replaced
0x57 If module must be reprogrammed
0x00 if cause is not known

1920 v1 **Failure type value for new configuration not learned**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.4
Verification Method: Analysis
Legacy ID: 70822v0

The DTC for detection of new configuration not learned shall use Failure Type value 0x51.

1921 v1 **FDC 10 max value for inconsistent or incompatible vehicle configuration ...**

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state.
Source: Table "Calibration of configuration DTCs" - item 2.5
Verification Method: Test
Legacy ID: 70831v0

The DTC test for inconsistent or incompatible vehicle configuration data shall have FDC 10 max value = 127.

1922 v1 **FDC 10 max value for invalid vehicle configuration data**

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state.
Source: Table "Calibration of configuration DTCs" - item 2.3
Verification Method: Test
Legacy ID: 70771v0

The DTC test for invalid vehicle configuration data shall have FDC 10 max value = 127.

1923 v1 FDC 10 max value for missing Car Configuration Logic (CCL)

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state.
Source: Table "Calibration of configuration DTCs" - item 4.2
Verification Method: Test
Legacy ID: 70630v0

The DTC test for missing Car Configuration Logic (CCL) shall have FDC 10 max value = 127.

1924 v1 FDC 10 max value for missing vehicle configuration data

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 69741v0

The DTC test for missing vehicle configuration data shall have FDC 10 max value = 127.

1925 v1 FDC 10 max value for missing Vehicle Configuration Data (VCD)

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state.
Source: Table "Calibration of configuration DTCs" - item 5.1
Verification Method: Test
Legacy ID: 70604v0

The DTC test for missing Vehicle Configuration Data (VCD) shall have FDC 10 max value = 127.

1926 v1 FDC 10 max value for new configuration not learned

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state.
Source: Table "Calibration of configuration DTCs" - item 2.4
Verification Method: Test
Legacy ID: 70804v0

The DTC test for new configuration not learned shall have FDC 10 max value = 127.

1927 v1 **FDC 10 max value for VCD and CCL data inconsistency**

Version history:
(1):

Purpose: To define that the DTC test shall be able to set the Confirmed state.
Source: Table "Calibration of configuration DTCs" - item 4.1
Verification Method: Test
Legacy ID: 70719v0

The DTC test for VCD and CCL data inconsistency shall have FDC 10 max value = 127.

1928 v2 **Test Run Criteria for detection of CCL part no inconsistency**

Version history:
(2):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.1
Verification Method: Analysis
Legacy ID: 70726v0

The DTC test for detection of CCL part no inconsistency shall run whenever the VCP generation routine is executed according to *[Data_5] Central Car Configuration*.

1929 v1 **Test Run Criteria for detection of data integrity fault in configuration...**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 3.1
Verification Method: Analysis
Legacy ID: 70661v0

The DTC test for detection of data integrity faults in configuration data memory shall run according to criteria specified by the implementer.

1930 v1 **Test Run Criteria for detection of incompatible VIN**

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by incompatible or missing software components" - item 7.1
Verification Method: Analysis

Legacy ID: 70839v0

The DTC test for detection of incompatible VIN shall run at least once every operation cycle.

1931 v1 Test Run Criteria for detection of inconsistent or incompatible vehicle ...

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.5
Verification Method: Analysis
Legacy ID: 70834v0

The DTC test for detection of inconsistent or incompatible configuration shall run every time a global configuration parameter used for configuration of the ECU is received until test the test has passed.

1932 v2 Test Run Criteria for detection of invalid local configuration data

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable
Source: Table "Requirements on detection of faults caused by local ECU Configuration (local parameters)" - item 1.1
Verification Method: Analysis
Legacy ID: 69702v0

If the detection of invalid local configuration data is implemented the DTC test shall run whenever a local configuration parameter is used or changed.

1933 v3 Test Run Criteria for detection of missing Car Configuration Logic data

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 4.2
Verification Method: Analysis
Legacy ID: 70637v0

T he DTC test for detection of missing Car Configuration Logic data shall run each time new VCP data is generated according to *[Data_5] Central Car Configuration*.

1934 v2 Test Run Criteria for detection of missing vehicle configuration data

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.1
Verification Method: Analysis
Legacy ID: 69801v0

T he DTC test for detection of missing vehicle configuration data shall be enabled while ECU is in 'Bilk state' and shall run each time a Car Configuration block is received. The test shall be inhibited once it has passed.

1935 v4 Test Run Criteria for detection of missing Vehicle Configuration Data (VCD)

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 5.1
Verification Method: Analysis
Legacy ID: 70608v1

T he DTC test for detection of missing vehicle configuration data (VCD) shall run each time new VCP data is generated according to *[Data_5] Central Car Configuration*.

1936 v1 Test Run Criteria for invalid vehicle configuration data

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by Global ECU Configuration (global parameters)" - item 2.3
Verification Method: Analysis
Legacy ID: 70785v0

The DTC test for detection of invalid configuration data shall run every time a global configuration parameter used for configuration of the ECU is received, until the test has passed.

1937 v1 Test Run Criteria for new configuration not learned

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by Global ECU

Configuration (global parameters)" - item 2.4
Verification Method: Analysis
Legacy ID: 70819v0

The DTC test for new configuration not learned shall run every time a global configuration parameter used for configuration of the ECU is received, until the test has passed.

1938 v1 Test sample failed criteria for inconsistent or incompatible vehicle con...

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.5
Verification Method: Test
Legacy ID: 70825v0

The DTC test for inconsistent or incompatible vehicle configuration data shall use the following criteria for test sample failed:
O ne or more parameters used for configuration of the ECU has a valid value but is internally inconsistent or incompatible with some other data or configuration in the ECU or an ECU hardware option.
The FDC10 shall be increased by 127 each test sample failed .

1939 v1 Test sample failed criteria for invalid vehicle configuration data

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.3
Verification Method: Test
Legacy ID: 70761v0

The DTC test for invalid vehicle configuration data shall use the following criteria for test sample failed:
O ne or more parameters used for configuration of the ECU is received with an unrecognized or invalid value (incl. values 0x00 and 0xFF).
The FDC10 shall be increased by 127 each test sample failed .

1940 v1 Test sample failed criteria for missing Car Configuration Logic (CCL)

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 4.2
Verification Method: Test

Legacy ID: 70623v0

The DTC test for missing Car Configuration Logic (CCL) shall use the following criteria for test sample falied:
No Car Configuration Logic (CCL) file is stored in the CCL memory.
The FDC10 shall be increased by 127 each test sample failed .

1941 v3 Test sample failed criteria for missing vehicle configuration data

Version history:
(3):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 69721v2

The DTC test for missing vehicle configuration data shall use the following criteria for test sample failed:
No valid configuration (i.e. any required parameter missing or with value 0x00 or 0xFF) has been stored in VCP memory.
The FDC10 shall be increased by 127 each test sample failed.

1942 v1 Test sample failed criteria for missing Vehicle Configuration Data (VCD)

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 5.1
Verification Method: Test
Legacy ID: 70594v0

The DTC test for missing Vehicle Configuration Data (VCD) shall use the following criteria for test sample falied:
No Vehicle Configuration Data (VCD) file is stored in the VCD memory.
The FDC10 shall be increased by 127 each test sample failed.

1943 v1 Test sample failed criteria for new configuration not learned

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.4
Verification Method: Test
Legacy ID: 70795v0

The DTC test for new configuration not learned shall use the following criteria for test sample failed:
A changed configuration has been evaluated as valid and the ECU requires an additional control routine to instruct the ECU to learn a changed configuration.
The FDC10 shall be increased by 127 each test sample failed .

1944 v1 Test sample failed criteria for VCD and CCL data inconsistency

Version history:
(1):

Purpose: To define exact critera for test failed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 70681v0

The DTC test for VCD and CCL data inconsistency shall use the following criteria for test sample failed:
The CC-Logic Part No specified in VCD (Information Data) and the CC-Logic Part No specified in CCL memory are **not** equal.
The FDC10 shall be increased by 127 each test sample failed .

1945 v3 Test sample passed criteria for inconsistent or incompatible vehicle configuration data

Version history:
(3):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.5
Verification Method: Test
Legacy ID: 70827v2

The DTC test for inconsistent or incompatible vehicle configuration data shall use the following criteria for test sample passed:
All parameters used for configuration of the ECU have valid values and are internally consistent and compatible with all other data, configuration and hardware options in the ECU.
The FDC10 shall be decreased by 128 each test sample passed .

1946 v3 Test sample passed criteria for invalid vehicle configuration data

Version history:
(3):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.3
Verification Method: Test

Legacy ID: 70766v2

The DTC test for invalid vehicle configuration data shall use the following criteria for test sample passed:
All parameter used for configuration of the ECU have been received with valid values.
The FDC10 shall be decreased by 128 each test sample passed.

1947 v3 Test sample passed criteria for missing vehicle configuration data

Version history:
(3):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 69723v2

The DTC test for missing vehicle configuration data shall use the following criteria for test sample passed:
A valid configuration (all required parameters have valid values) has been stored in VCP memory.
The FDC10 shall be decreased by 128 each test sample passed.

1948 v3 Test sample passed criteria for missing Vehicle Configuration Data (VCD)

Version history:
(3):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 5.1
Verification Method: Test
Legacy ID: 70599v2

The DTC test for missing Vehicle Configuration Data (VCD) shall use the following criteria for test sample passed:
A Vehicle Configuration Data (VCD) file is stored in the VCD memory.
The FDC10 shall be decreased by 128 each test sample passed.

1949 v3 Test sample passed criteria for new configuration not learned

Version history:
(3):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Table "Calibration of configuration DTCs" - item 2.4
Verification Method: Test
Legacy ID: 70796v2

The DTC test for new configuration not learned shall use the following criteria for test sample passed:
The routine required to instruct the ECU to learn a changed configuration has been successfully executed.
The FDC10 shall be decreased by 128 each test sample passed.

1950 v3 Test sample passed criteria for No Car Configuration Logic (CCL) is present in CCL memory.

Version history:
(3):

Purpose: The FDC10 shall be decreased by 128 each test sample passed.
Source: Table "Calibration of configuration DTCs" - item 4.2
Verification Method: Test
Legacy ID: 70625v2

The DTC test No Car Configuration Logic (CCL) is present in CCL memory s hall use the following criteria for test sample passed:
The routine required to instruct the ECU to learn a changed configuration has been successfully executed.
The FDC10 shall be decreased by 128 each test sample passed.

1951 v3 Test sample passed criteria for VCD and CCL data inconsistency

Version history:
(3):

Purpose: To define exact critera for test passed condition and to ensure that all ECUs use the same criteria.
Source: Table 4-13, item 4.1
Verification Method: Test
Legacy ID: 152002v2

The DTC test for VCD and CCL data inconsistency shall use the following criteria for test sample passed:
The CC-Logic Part No specified in VCD (Information Data) and the CC-Logic Part No specified in CCL memory are equal.
The FDC10 shall be decreased by 128 each test sample passed.

1952 v1 UnconfirmedDTCLimit for inconsistent or incompatible vehicle configurati...

Version history:
(1):

Purpose: To define exact critera for setting unconfirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 2.5
Verification Method: Test
Legacy ID: 70829v0

The DTC test for inconsistent or incompatible vehicle configuration data shall have

unconfirmedDTCLimit = 127.

1953 v1 UnconfirmedDTCLimit for invalid vehicle configuration data

Version history:
(1):

Purpose: To define exact critera for setting unconfirmedDTC status.
Source: Table "Calibration of configuration DTCs" - item 2.3
Verification Method: Test
Legacy ID: 70769v0

DTC test for missing vehicle configuration data shall have unconfirmedDTCLimit = 127.

1954 v1 UnconfirmedDTCLimit for missing Car Configuration Logic (CCL)

Version history:
(1):

Purpose: To define exact critera for setting UnconfirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 4.2
Verification Method: Test
Legacy ID: 70627v0

The DTC test for missing Car Configuration Logic (CCL) shall have UnconfirmedDTCLimit = 127.

1955 v1 UnconfirmedDTCLimit for missing vehicle configuration data

Version history:
(1):

Purpose: To define exact critera for setting unconfirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 2.1
Verification Method: Test
Legacy ID: 69724v0

The DTC test for missing vehicle configuration data shall have unconfirmedDTCLimit = 127.

1956 v1 UnconfirmedDTCLimit for missing Vehicle Configuration Data (VCD)

Version history:
(1):

Purpose: To define exact critera for setting unconfirmedDTC status
Source: Table 4.13: Table "Calibration of configuration DTCs" - item 5.1
Verification Method: Test
Legacy ID: 70601v0

The DTC test for missing Vehicle Configuration Data (VCD) shall have unconfirmedDTCLimit = 127.

1957 v1 **UnconfirmedDTCLimit for new configuration not learned**

Version history:
(1):

Purpose: To define exact critera for setting unconfirmedDTC status
Source: Table "Calibration of configuration DTCs" - item 2.4
Verification Method: Test
Legacy ID: 70802v0

The DTC test for new configuration not learned shall have unconfirmedDTCLimit = 127.

1958 v1 **UnconfirmedDTCLimit for VCD and CCL data inconsistency**

Version history:
(1):

Purpose: To define exact critera for setting unconfirmedDTC status.
Source: Table "Calibration of configuration DTCs" - item 4.1
Verification Method: Test
Legacy ID: 70716v0

The DTC test for VCD and CCL data inconsistency shall have unconfirmedDTCLimit = 127.

4.5.1.1.1.15 Requirements from section Detection of faults caused by received signals

1825 v1 **Base DTC value for detection of corrupt data received from other ECU by end-to-end checksum**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332536v0

The DTC test for detection of corrupt data received from other ECU by end-to-end checksum shall use one unique base DTC value for each tested signal, in the range U2D00 to U2DFF, as defined in *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications and *[Data_4] Global Master Reference Database*.

1826 v2 **Base DTC value for detection of corrupt data received from other ECU by rolling counter**

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1

Verification Method: Analysis
Legacy ID: 71754v1

The DTC test for detection of corrupt data received from other ECU by rolling counter shall use one unique base DTC value for each tested signal, in the range U2D00 to U2DFF, as defined in *[Data_51] Diagnostic trouble code definitions* , UDS - Data - External publications and [Data_4] *Global Master Reference Database*.

1827 v3 Base DTC value for detection of corrupt data received from other ECU by up-date-bit

Version history:
(3):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332571v2

The DTC test for detection of corrupt data received from other ECU by up-date-bit shall use one unique base DTC value for each tested signal, in the range U2F00 to U2FFF if the DTC is not emission related and U0100 to U02FF if the DTC is emission related, as defined in *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications and *[Data_4] Global Master Reference Database*.

1828 v1 Base DTC value for detection of other ECU not operating normally

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70872v0

The DTC test for detection of other ECU not operating normally shall use one unique base DTC value for each signal (indicating reduced functionality in the other ECU), in the range U2C00 to U2CFF, as defined in:
[Data_51] Diagnostic trouble code definitions
[Data_4] Global Master Reference Database

1829 v1 Base DTC value for detection of other ECU sending data tagged as invalid

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1

Verification Method: Analysis
Legacy ID: 71743v0

The DTC test for detection of other ECU sending data tagged as invalid shall use one unique base DTC value for each signal, in the range U2C00 to U2CFF, as defined in *[Data_51] Diagnostic trouble code definitions* and *[Data_4] Global Master Reference Database*

1830 v1 Base DTC value for detection of other ECU sending faulty data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71748v0

The DTC test for detection of other ECU sending faulty data shall use one unique base DTC value for each tested signal, in the range U2D00 to U2DFF, as defined in *[Data_51] Diagnostic trouble code definitions* and *[Data_4] Global Master Reference Database*.

1831 v1 Detection of corrupt data received from other ECU by end-to-end checksum

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Test
Legacy ID: 332533v0

If the ECU detects, by the data protection mechanism end-to-end checksum implemented at application level in the ECU, that a data value received from other ECU is corrupted each such fault shall also be identified by an appropriate DTC.

1832 v2 Detection of corrupt data received from other ECU by rolling counter

Version history:
(2):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Test
Legacy ID: 71752v1

If the ECU detects, by the data protection mechanism rolling counter implemented at application level in the ECU, that a data value received from other ECU is corrupted (the data is received without the corresponding rolling count value being properly updated e.g. incremented by the value 1 when expected), as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.

1833 v3 Detection of corrupt data received from other ECU by up-date-bit

Version history:
(3):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: able "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Test
Legacy ID: 332551v2

If the ECU detects, by the data protection mechanism end-to-end checksum implemented at application level in the ECU (the data is received without the corresponding up-date bit value being properly set e.g. set to the value 1 when expected), that a data value received from other ECU is corrupted each such fault may also be identified by an appropriate DTC.
Note: The DTC may be mandatory if the DTC is emission related but this is not specified by this document.
Note: The faults are identified by DTCs that are known as "LostComDTCs".
If the DTC is not emission related the DTC status indicator 30:5 (SI30:5) – warningIndicatorRequestedSinceLastClear is allowed to be set to 1 for these DTCs despite the FDC10 never reach +127 as the DTCs are development specific. Refer to Table: “Requirements on implementation of DTC status indicators” for requirements on set of DTC SI 30:5 to 1. It is up to the implementer to specify the criteria for set the DTC SI 30:5 to 1.

1834 v1 Detection of other ECU not operating normally

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Test
Legacy ID: 70867v0

If the ECU detects that another ECU sends data that indicates it has detected a fault which causes it to operate with reduced or not correct functionality, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.

Otherwise, the ECU shall be able to detect that other ECU not operating normally as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1835 v1 **Detection of other ECU sending data tagged as invalid**

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Test
Legacy ID: 70878v0

If the ECU detects that another ECU sends data tagged as invalid, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that other ECU sends data tagged as invalid as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1836 v1 **Detection of other ECU sending faulty data**

Version history:
(1):

Purpose: To handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Test
Legacy ID: 71746v0

If the ECU detects that another ECU sends data that is not tagged as invalid but still detected as faulty by the ECU, e.g. by plausibility check, as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such fault shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that other ECU sends faulty data as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1837 v2 **Failure type value for detection of corrupt data received from other ECU by end-to-end checksum**

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332537v1

The DTCs for detection of corrupt data received from other ECU by end-to-end checksum shall use Failure Type value 0x83.

1838 v2 **Failure type value for detection of corrupt data received from other ECU by rolling counter**

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Analysis
Legacy ID: 71755v1

The DTCs for detection of corrupt data received from other ECU by rolling counter shall use Failure Type value 0x82.

1839 v1 **Failure type value for detection of corrupt data received from other ECU by up-date bit**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332572v0

The DTCs for detection of corrupt data received from other ECU by up-date bit shall use Failure Type value 0x82.

1840 v1 **Failure type value for detection of other ECU not operating normally**

Version history:
(1):

Purpose: To enable easy identification of specific faults.

Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70875v0

The DTCs for detection of other ECU not operating normally shall use Failure Type value 0x92.

1841 v1 Failure type value for detection of other ECU sending data tagged as inv...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71744v0

The DTCs for detection of other ECU sending data tagged as invalid shall use Failure Type value 0x81.

1842 v1 Failure type value for detection of other ECU sending faulty data

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71750v0

The DTCs for detection of other ECU sending faulty data shall use Failure Type value 0x86.

1843 v1 Root DTC for ECU not operating normally

Version history:
(1):

Purpose: To enable identification of the root cause of the DTC for other ECU not operating normally.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.1
Verification Method: Analysis
Legacy ID: 70877v0

The ECU transmitting a signal that indicates it is not operating normally shall set an appropriate DTC for this fault (this can be either a DTC relating to the root cause of the situation if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU).

1844 v1 **Root DTC for ECU sending data tagged as invalid**

Version history:
(1):

Purpose: To enable identification of the root cause of the DTC for other ECU sending data tagged as invalid.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71745v0

The ECU transmitting a signal that is tagged as invalid shall set an appropriate DTC for this fault (this can be either a DTC relating to the root cause of the situation if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU).

1845 v2 **Test Run Criteria for detection of corrupt data received from other ECU by end-to-end checksum**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332535v0

The DTC test for detection of corrupt data received from other ECU by end-to-end checksum shall run each time updated data, for which a test i simplemented, is received from another ECU.

1846 v2 **Test Run Criteria for detection of corrupt data received from other ECU by rolling counter**

Version history:
(2):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 4.1
Verification Method: Analysis
Legacy ID: 71753v1

The DTC test for detection of corrupt data received from other ECU by rolling counter shall run each time updated data, for which a test i simplemented, is received from another ECU.

1847 v3 **Test Run Criteria for detection of corrupt data received from other ECU by update bit**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: ble "Requirements on detection of faults caused by signal/data content received from other ECUs"
Verification Method: Analysis
Legacy ID: 332552v1

The DTC test for detection of corrupt data received from other ECU by up-date bit shall run each time updated data, for which a test i simplemented, is received from another ECU.

1848 v1 Test Run Criteria for detection of other ECU not operating normally

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 1.
Verification Method: Analysis
Legacy ID: 70869v0

The DTC test for detection of other ECU not operating normally shall run each time the updated data indicating reduced functionality in the other ECU is received.

1849 v1 Test Run Criteria for detection of other ECU sending data tagged as invalid

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 2.1
Verification Method: Analysis
Legacy ID: 71423v0

The DTC test for detection of other ECU sending data tagged as invalid shall run each time updated data is received from another ECU.
The test shall be inhibited if the received data is detected as corrupt by a data protection mechanism.

1850 v1 Test Run Criteria for detection of other ECU sending faulty data

Version history:
(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.

Source: Table "Requirements on detection of faults caused by signal/data content received from other ECUs" - item 3.1
Verification Method: Analysis
Legacy ID: 71747v0

The DTC test for detection of other ECU sending faulty data shall run each time updated data, for which a test i simplemented, is received from another ECU.
If plausibility check is used as fault detection logic the test shall be inhibited if a DTC test for one (or more) of the compared signals (used to evaluate the plausibility of the received signal) has detected a fault on that signal.

4.5.1.1.1.16 Requirements from section Detection of faults caused by the ECU environment

1851 v1 **Base DTC value for detection of events caused by customer operating vehi...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71773v0

If a DTC test for events caused by customer operating vehicle or vehicle equipment outside specification is implemented the test shall use the base DTC value U2E02.

1852 v1 **Base DTC value for detection of events caused by ECU supply voltage is t...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.2
Verification Method: Analysis
Legacy ID: 71788v0

If a DTC test for events caused by ECU supply voltage is too low for full operation is implemented the test shall use the base DTC value U2E04.

1853 v1 **Base DTC value for detection of events caused by ECU supply voltage is t...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.1
Verification Method: Analysis
Legacy ID: 71782v0

If a DTC test for events caused by ECU supply voltage is too high for full operation is implemented the test shall use the base DTC value U2E03.

1854 v2 Base DTC value for detection of events caused by other condition that is out of range for full opera

Version history:
(2):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71796v1

If DTC tests for events caused by other conditions are out of range for full operation is implemented the tests shall use a base DTC value in the range U2E05 to U2EFF. The tests shall use one unique base DTC value for each event.

1855 v1 Base DTC value for detection of faults caused by ambient teperature too ...

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.1
Verification Method: Analysis
Legacy ID: 71760v0

If a DTC test for ambient temperature too high for full and correct operation is implemented the test shall use the base DTC value U2E00.

1856 v1 Base DTC value for detection of faults caused by ambient teperature too low

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.2
Verification Method: Analysis
Legacy ID: 71762v0

If a DTC test for ambient temperature too low for full and correct operation is implemented the test shall use the base DTC value U2E01.

1857 v1 Base DTC value for detection of other faults inferred from system behaviour

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source:
Verification Method: Inspection
Legacy ID: 115381v0

Detection of other faults faults inferred from system behaviour shall use one unique base DTC value for each component, system or part of system that is monitored, to identify the fault.

1858 v2 Detection of events caused by customer operating vehicle or vehicle equi...

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71767v0

If the ECU detects that the vehicle, or vehicle equipment (including the ECU itself), is operated outside of its design specification so that not correct, reduced or no system functionality is available, or a fallback strategy is activated, and this detection is implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such event shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that such events as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1859 v2 Detection of events caused by ECU supply voltage is too high for full op...

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.1
Verification Method: Analysis
Legacy ID: 71779v0

If the ECU is able to detect that its supply voltage is too high for full operation so that not correct, reduced or no system functionality is available and this detection is implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such event shall also be identified

by an appropriate DTC.
Otherwise, the ECU shall be able to detect that such events as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1860 v2 Detection of events caused by ECU supply voltage is too low for full ope...

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.2
Verification Method: Analysis
Legacy ID: 71785v0

If the ECU is able to detect that its supply voltage is too low for full operation so that not correct, reduced or no system functionality is available and this detection is implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such event shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that such events as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1861 v2 Detection of events caused by other condition that is out of range for f...

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To be able to handle the situation where there may be a customer complaint and a symptom associated with the ECU but the root cause is that the ECU is operating outside of its design specification.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71792v0

If the ECU is able to detect that any other condition is out of range for full operation so that not correct, reduced or no system functionality is available and this detection is implemented as part of the ECU functional requirements (i.e. where the primary reason for the fault monitor is another than to support generation of DTC information) each such event shall also be identified by an appropriate DTC.
Otherwise, the ECU shall be able to detect that such events as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution

is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).

1862 v1 **Failure type value for detection of events caused by customer operating ...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71777v0

If a DTC test for events caused by customer operating vehicle or vehicle equipment outside specification is implemented the test shall use Failure Type value 0x68.

1863 v1 **Failure type value for detection of events caused by ECU supply voltage ...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.1
Verification Method: Analysis
Legacy ID: 71783v0

If a DTC test for events caused by ECU supply voltage is too high for full operation is implemented the test shall use Failure Type value 0x68.

1864 v1 **Failure type value for detection of events caused by ECU supply voltage ...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.2
Verification Method: Analysis
Legacy ID: 71790v0

If a DTC test for events caused by ECU supply voltage is too low for full operation is implemented the test shall use Failure Type value 0x68.

1865 v1 **Failure type value for detection of events caused by other condition tha...**

Version history:
(1):

Purpose: To enable easy identification of specific faults.

Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71797v0

If a DTC test for events caused by other condition is out of range for full operation is implemented the test shall use Failure Type value 0x68.

1866 v1 Failure type value for detection of faults caused by ambient teperature

Version history:
(1):

Purpose: To enable easy identification of specific faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 71764v0

If a DTC test for ambient temperature too high or too low for full and correct operation is implemented the test shall use Failure Type value 0x68.

1867 v1 Test period time for detection of events caused by customer operating ve...

Version history:
(1):

Purpose: To allow for fast detection of faults and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 2.1
Verification Method: Analysis
Legacy ID: 71769v0

If a DTC test for events caused by customer operating vehicle or vehicle equipment outside specification is implemented the test shall produce Test Samples periodically every 100 ms or faster whenever the Test Run Criteria are met.

1868 v1 Test period time for detection of events caused by ECU supply voltage is...

Version history:
(1):

Purpose: To allow for fast detection of faults and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.1
Verification Method: Analysis
Legacy ID: 71781v0

If a DTC test for events caused by ECU supply voltage is too high for full operation is implemented the test shall produce Test Samples periodically every 100 ms or faster whenever the Test Run Criteria are met.

1869 v1 **Test period time for detection of events caused by ECU supply voltage is...**

Version history:
(1):

Purpose: To allow for fast detection of faults and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 3.2
Verification Method: Analysis
Legacy ID: 71787v0

If a DTC test for events caused by ECU supply voltage is too low for full operation is implemented the test shall produce Test Samples periodically every 100 ms or faster whenever the Test Run Criteria are met.

1870 v2 **Test period time for detection of events caused by other condition that is out of range for full ope**

Version history:
(2):

Purpose: To allow for fast detection of faults and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 4.1
Verification Method: Analysis
Legacy ID: 71794v1

If a DTC test for events caused by other condition is out of range for full operation is implemented the test shall produce Test Samples periodically, while the Test Run Criteria are met, at a rate defined by the implementer and approved by the responsible authority appointed by OEM. If not defined by the implementer the test period time shall be 100 ms or faster.

1871 v1 **Test period time for detection of faults caused by ambient teperature**

Version history:
(1):

Purpose: To allow for fast detection of fault and the ability to detect intermittent faults.
Source: Table "Requirements on detection of faults caused by the ECU operating environment or customer actions" - item 1.1 and 1.2
Verification Method: Analysis
Legacy ID: 71757v0

If a DTC test for ambient temperature too high or too low for full and correct operation is implemented the test shall produce Test Samples periodically every 100 ms or faster whenever the Test Run Criteria are met.

1872 v1 **Test Run Criteria for detection of faults caused by the ECU operating environment or customer action**

Version history:

(1):

Purpose: To enable a fault detection mechanism with maximum coverage while being robust and reliable.
Source:
Verification Method: Inspection
Legacy ID: 115361v0

Any fault detection test for detection of faults caused by the ECU operating environment or customer actions shall be inhibited when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test.

4.5.1.1.1.17 Requirements from section Detection of events associated with driver indications

1815 v2 **Detected of events associated with driver indications**

Version history:
(2):

Purpose: If a driver indication (e.g. telltale, text message, etc.) exists which, when activated, recommends (e.g. by a service book instruction, text message, etc.) the driver to bring the car to a workshop, the workshop may need to be able to find the root cause for this indication by a DTC.
Source: Section "Detection of events associated with driver indications"
Verification Method: Analysis
Legacy ID: 71801v1

A DTC test indicating the cause of the event that triggers an activation of a driver indication (which when activated, recommends the driver to bring the car to a workshop) shall be implemented as required by a study perfomed by the implementer and approved by the responsible authority appointed by OEM.

4.5.1.1.1.18 Requirements from section DTC status bits

2125 v4 **Support for DTC status bit confirmedDTC**

Version history:
(4):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71827v3

The DTC status bit no 3, confirmedDTC, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications, with the following addition:
The bit shall be set to 1 when the OCC6 reaches the confirmedDTCLimit.The value of the the confirmedDTCLimit shall be in the range of [1, +15].
The bit shall be reset to 0 when the DTC is aged.
The bit shall not be set 1 and it shall be set to 0, in case of the bit is already set to 1, if there

are no aging counter (OCC2 or OCC5) allocated to the DTC due to the long term memory overflow.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

2126 v3 Support for DTC status bit pendingDTC

Version history:
(3):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71824v2

The DTC status bit no 2, pendingDTC, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications. (The status bit number is the bit number of the DTC status byte associated with the DTC).

2127 v2 Support for DTC status bit testFailed

Version history:
(2):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71815v1

The DTC status bit no 0, testFailed, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications , with the following exception:
No additional reset conditions are allowed to be defined by the implementer.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

2128 v2 Support for DTC status bit testFailedSinceLastClear

Version history:
(2):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71829v1

The DTC status bit no 5, testFailedSinceLastClear, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

2129 v2 Support for DTC status bit testFailedThisOperationCycle

Version history:
(2):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71823v1

The DTC status bit no 1, testFailedThisOperationCycle, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

2130 v2 Support for DTC status bit testNotCompletedSinceLast-Clear

Version history:
(2):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71828v1

The DTC status bit no 4, testNotCompletedSinceLast-Clear, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

2131 v2 Support for DTC status bit testNotCompletedThis-OperationCycle

Version history:
(2):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71830v1

The DTC status bit no 6, testNotCompletedThis-OperationCycle, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

2132 v2 Support for DTC status bit warningIndicatorRequested

Version history:
(2):

Purpose: To provide information about the status of an individual DTC to the test tool.
Source: Table "Requirements on implementation of DTC status bits"
Verification Method: Test
Legacy ID: 71831v1

The DTC status bit no 7, warningIndicatorRequested, shall be supported for all DTCs supported by the ECU, as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications , with the following exception:
No additional reset conditions are allowed to be defined by the implementer.
(The status bit number is the bit number of the DTC status byte associated with the DTC.)

4.5.1.1.1.19 Requirements from section DTC snapshot data identification

2118 v1 — Data records not included in local snapshot data

Version history:
(1):

Purpose: To ensure consistent storage of data records in local DTC snapshot data records.
Source: Section "DTC snapshot data identification"
Verification Method: Analysis
Legacy ID: 71837v0

Data records defined either as SystemSupplierSpecific or dynamically defined (as defined according to the table *Requirements on implementation of data records*) shall not be included in local DTC snapshot data.

2119 v1 — Order of multple samples of data records in global and local snapshot data records

Version history:
(1):

Purpose: To ensure consistent storage of data records in local DTC snapshot data records
Source:
Verification Method: Test
Legacy ID: 115401v0

If data is sampled multiple times (record type = 'Multiple samples') before the sampling criteria is satisfied, then the sampled values shall be repeated a defined number of times in the snapshot data record. The order of the DTC snapshot data in the DTC snapshot record shall be such that the data which is sampled first is put at the start of the record (in bytes #1 to n) and the next data sample is appended to data from the first sample (bytes #n+1 to m), and so on.

4.5.1.1.1.20 Requirements from section DTC snapshot sampling criteria

2120 v5 — Sampling criteria for DTCSnapshotRecordNumber 0x20, single sample

Version history:
(5):

Purpose: To define the critera for sampling of snapshot data when the DTC has reached its unconfirmed limit.
Source: Table "Requirements on implementation DTC snapshot sampling criteria"

Verification Method: Test
Legacy ID: 72012v4

When implementing with AUTOSAR 4.0.x:

For all DTCs supported by the ECU DTC snapshot data shall be sampled and stored with DTCSnapshotRecordNumber 0x20 when the following criteria is satisfied for the data associated with the DTC:
Each time the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit until the FDC10 reaches the value +127 (since DTC information was last cleared) and each time the FDC10 reaches the value +127 until the DTC staus bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared).

If FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. If FDC10 reaches the value +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only.

Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).

When implementing with AUTOSAR 4.1 or newer:

For all DTCs supported by the ECU DTC snapshot data shall be sampled and stored with DTCSnapshotRecordNumber 0x20 when the following criteria is satisfied for the data associated with the DTC:
Each time the FDC10 reaches the value +127 until the DTC status bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared).

If FDC10 reaches the value +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only.

Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).

2121 v3 **Sampling criteria for DTCSnapshotRecordNumber 0x21, single sample**

Version history:
(3): Change the Verification Method to Test

Purpose:
Source:
Verification Method: Test
Legacy ID: 441094v0

When implementing with AUTOSAR 4.0x:
This requirement is Not Applicable

When implementing with AUTOSAR 4.1 or newer:
For all DTCs supported by the ECU DTC snapshot data shall be sampled and stored with DTCSnapshotRecordNumber 0x21 when the following criteria is satisfied for the data associated with the DTC:
Each time the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 (since DTC information was last cleared).

If FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only.

Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).

2122 v2 Sampling criteria for DTCSnapshotRecordNumber 0x30 to 0x3F, multiple samples

Version history:
(2):

Purpose: To define the critera for sampling of snapshot data when the DTC has reached its unconfirmed limit.
Source:
Verification Method: Test
Legacy ID: 115421v1

For DTCs supported by the ECU DTC snapshot data may be sampled and stored with DTCSnapshotRecordNumber in the range 0x30 to 0x3F according to the following:
The DTC snapshot data shall be sampled periodically. A number of samplers before and/or after the criteria are satisfied shall be stored (the number of samples, and the periodicity, shall be defined by the implementer). The sampling critera is specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that CEVT tools support the DTC snapshot record.

2123 v1 Sampling criteria for DTCSnapshotRecordNumber 0x40 to 0x4F, single sample

Version history:
(1):

Purpose: To define the critera for sampling of snapshot data.
Source: Table: Requirements on implementation DTC snapshot sampling criteria
Verification Method: Test
Legacy ID: 336996v0

For DTCs supported by the ECU DTC snapshot data may be sampled and stored with DTCSnapshotRecordNumber in the range 0x40 to 0x4F according to the following:
The DTC snapshot data shall be sampled (and stored) once when the criteria is satisfied. The sampling criteria is specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that CEVT tools support the DTC snapshot record.

2124 v1 **Sampling criteria for DTCSnapshotRecordNumber 0x50 to 0x5F, other samples**

Version history:
(1):

Purpose: To define the critera for sampling of snapshot data.
Source: Table: Requirements on implementation DTC snapshot sampling criteria
Verification Method: Test
Legacy ID: 336997v0

For DTCs supported by the ECU DTC snapshot data may be sampled and stored with DTCSnapshotRecordNumber in the range 0x50 to 0x5F according to the following:
The sampling critera is specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that CEVT tools support the DTC snapshot record.

4.5.1.1.1.21 Requirements from section Global DTC snapshot data

2258 v2 **Definition of global real time data record**

Version history:
(2):

Purpose: Ensure that the global real time data has a useful resolution and accuracy.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72018v1

The ECU responsible for maintaining and distributing the real time data shall maintian a global time data record with the following range, resolution and accuracy:
range 0x00000000 to 0xFFFFFFFE
resolution 1 bit = 100 ms
accuracy minimum ±10 minutes/week (±0.1%)
correct value not available = 0xFFFFFFFF
after the global time have reached 0xFFFFFFFE it shall restart from 0x00000000

2259 v3 **Definition of total distance data record**

Version history:
(3):

Purpose: Ensure that the total distance data has a useful resolution and accuracy.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72020v2

The ECU responsible for maintaining and distributing the total distance data shall maintian a total distance data record with the following range, resolution and accuracy:
range minimum 0x000000 to 0x0F4240.
resolution 1 bit = 1 km

the accuracy shall be the same as or better than the accuracy of the total distance displayed in the dash board
correct value not available = 0xFFFFFF

2260 v2 **Definition of vehicle battery voltage data record**

Version history:
(2):

Purpose: Ensure that the vehicle battery voltage data has a useful resolution and accuracy.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72021v1

The ECU responsible for maintaining and distributing the vehicle battery voltage data shall maintian a vehicle battery voltage data record with the following range, resolution and accuracy:
range minimum / 0x00 to 0x60
r esolution 1 bit = 0.25 Volt
the accuracy shall be according to section *Requirements for detection of low supply voltage.*
correct value not available = 0xFF

2261 v3 **Distribution of global DTC snapshot data**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that global snaphsot data that is sampled and stored in each ECU is not too old.
Source: Section "Global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72024v1

Global DTC snapshot data shall be transmitted to all public ECUs by an ECU (one or more) responsible for maintaining and distributing the data according to the following requirements:
- The ECU shall transmit current values of the data periodically each second on the network except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent faster according to the requirements specified in section Additional requirements for detection of low supply voltage.
- The ECU shall transmit the data with a maximum age of 250 ms, except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent with a lower maximim age according to the requirements specified in section Additional requirements for detection of low supply voltage.

2262 v3 **Maintaining global real time after power loss**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure consistency in the distributed global real time value after power

loss.

Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72019v1

The ECU responsible for maintaining and distributing the real time data shall store the current value of global real time in long.term memory at least once each operation cycle. If the ECU loses power supply then, following reconnection of the power supply, it shall continue to count the global real time beginning from the stored value.

2263 v4 Storage of global DTC snapshot data when not available

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: If data received from the network is not availabel at the sampling moment the last received value will be the most useful to store.
Source: Section "Global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72026v2

If global DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data.
If no value has been received in the current operation cycle a value representing "correct value not available", according to specified in table: *Requirements on implementation of global DTC snapshot data*, shall be used and stored as DTC snapshot data.

2264 v10 Supported data records in global DTC snapshot data

Version history:
(10): Change the Verification Method from Test to Analysis

Purpose: To enhance fault tracing and root cause analysis specific vehicle global data shall be stored with each DTC.
Source: Table "Requirements on implementation of global DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72017v8

For all DTCs supported by the ECU the global DTC snapshot data records shall include the following data records, as defined in reference *[Data_4] Global Master Reference Database* and the stored data records shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*:

0xDD00 Global real time.
0xDD01 Total distance.
0xDD02 Vehicle battery voltage.
0xDD0A Usage mode.
0xDD0B Partial Network Cluster (PNC). See note 1
0xDD0C ElPowerLevel.

Note 1:
This DID is not mandatory for all ECU's. It shall only be included in the Global DTC snapshot data when the ECU is required to implement this DID.

4.5.1.1.1.22 Requirements from section Local DTC snapshot data

2265 v2 **Storage of local DTC snapshot data when not available**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: If data received from the network is not availabel at the sampling moment the last received value will be the most useful to store.
Source: Section "Local DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72028v0

If local DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data.
If no value has been received in the current operation cycle a value representing "correct value not available" shall be used and stored as DTC snapshot data.

2266 v2 **Support for local DTC snapshot data**

Version history:
(2):

Purpose: To enhance fault tracing and root cause analysis specific ECU local data shall be stored with each DTC.
Source: Table "Requirements on implementation of local DTC snapshot data"
Verification Method: Analysis
Legacy ID: 72027v1

For all DTCs supported by the ECU local DTC snapshot data shall be implemented as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the DTC snapshot data to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM). The stored local DTC snapshot data records shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

4.5.1.1.1.23 Requirements from section DTC operation cycle counters

2111 v4 **Support for Operation Cycle Counter #1**

Version history:
(4):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"

Verification Method: Test
Legacy ID: 72030v3

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=1 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal to or greater than its unconfirmedDTCLimit or more (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle.
The counter shall be cleared as soon as FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle.
The counter shall be cleared as soon as FDC10 reaches the value +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

2112 v5 **Support for Operation Cycle Counter #2**

Version history:
(5):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test

Legacy ID: 72031v4

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=2 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded.

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as the FDC10 reaches the value -128, unless FDC10 has already reached a value that is equal to or greater than its unconfirmedDTCLimit in the current operation cycle.
The counter shall be cleared as soon as FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded.

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as the FDC10 reaches the value -128, unless FDC10 has already reached the value +127 in the current operation cycle.
The counter shall be cleared as soon as FDC10 reaches the value +127.
If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

2113 v5 Support for Operation Cycle Counter #3

Version history:
(5):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"

Verification Method: Test
Legacy ID: 72032v4

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=3 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared).
All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reached a value that is equal to or greater than its unconfirmedDTCLimit shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached the value +127 (since DTC information was last cleared).
All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reached the value +127 shall be included.

Increment/clear criteria:
The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached the value +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

2114 v4 **Support for Operation Cycle Counter #4**

Version history:
(4):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.

Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72033v3

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=4 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared).

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as FDC10 has reached a value that is equal to or greater than its unconfirmedDTCLimit for the first time.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

When implementing with AUTOSAR 4.1 or newer:
Definition:
Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value +127 (since DTC information was last cleared).

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

2115 v4 **Support for Operation Cycle Counter #5**

Version history:
(4):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72034v3

For all emission related DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=5 shall be implemented according to the following:

Definition:
Number of warm-up cycles since the DTC commanded the MIL to switch off (since DTC information was last cleared).
The warm-up cycle is defined in *[Data_52] OBD II regulation, section 1968.2 of title 13, California Code of Regulations, UDS - Data - External publications*.

Increment/clear criteria:
The counter shall be incremented when warm-up cycle changes from false to true.
The counter shall be cleared when the DTC fault detection counter 10 reach the value of +127.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services.*

2116 v7 Support for Operation Cycle Counter #6

Version history:
(7):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC operation cycle counters"
Verification Method: Test
Legacy ID: 72035v6

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=6 shall be implemented according to the following:

Definition:
The number of consecutive operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value of +127 (since DTC information was last cleared). The operation cycles during which the test was not completed shall be excluded. The minimum range of the counter is 0-15.

Increment/clear criteria:
In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time.
The counter shall be cleared in the beginning of the operation cycle following an operation cycle where FDC10 did not reach the value +127 at any time but -128 (the counter shall maintain its value if the FDC10 did not reach the value -128). The counter may also be cleared when the confirmedDTCLimit is reached.
If any count operation occurs which would cause a counter to roll over past 0xFF, then the count value shall instead be maintained at 0xFF.

Reporting:

The stored counter shall be reported as a 1 byte value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

2117 v6 **Support for Operation Cycle Counter #7**

Version history:
(6):

Purpose: To provide enhanced information about the occurence of a DTC that may be useful in the analysis of the fault.
Source:
Verification Method: Test
Legacy ID: 115561v5

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=7 shall be implemented according to the following:

When implementing with AUTOSAR 4.0.x:
Definition:
Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed or the DTC fault detection counter 10 did not reach a value that is equal to or greater than its unconfirmedDTCLimit shall be excluded.

Increment/clear criteria:
The counter shall be incremented in each operation cycle, following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit, as soon as the following criteria is met: The FDC10 reach the value -128 or a value that is equal to or greater than its unconfirmedDTCLimit.
If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF.

Reporting:
The stored counter shall be reported as a 1 byte unsigned numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

When implementing with AUTOSAR 4.1 or newer:
This requirements is Not Applicable

4.5.1.1.1.24 Requirements from section DTC fault detection counters

2097 v6 **Support for Fault Detection Counter #10**

Version history:
(6):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault, and enables detailed calibration of the

corresponding DTC.

Source: Table "Requirements for the implementation DTC fault detection counters"

Verification Method: Test

Legacy ID: 72036v5

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=10 shall be implemented as defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* (service 19 sub-function 14 and Appendix D.5) , UDS - Data - External publications and according to the following additional definitions:

1. At the start of a new operation cycle the count value shall be 0.
2. The count value shall be maintained between test runs within one and same operation cycle, also when the test run is blocked due to the test run criteria is not satisfied
3. The DTC test may be completed (i.e. the DTC status bit 4 and 6 is set to 0) before FDC10 reaches +127 or -128, and passed (i.e. the DTC status bit 0 is set to 0) before FDC10 reaches -128, where approved by the responsible authority appointed by OEM.
4. The maximum value of the counter may be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) where approved by the responsible authority appointed by OEM or shall be set to +126 if that is specified by this document.
5. If a single test sample of a test returns a fail result, when the current value of the counter is negative, the new count value resulting from the fail result shall be set as if the current count value had been 0 (a test sample represents the fail result shall always causes fault detection counter to decrement above 0).
6. If a single test sample of a test returns a pass result, when the current value of the counter is positive, the new count value resulting from the pass result shall be set as if the current count value had been 0 (a test sample represents the pass result shall always causes fault detection counter to decrement below 0).
7. The count value shall be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. If the number of consecutive test sample failed results is more than 127 to reach the testFailedLimit +127 (from the start value 0) or if the number of consecutive test sample passed results is more than 128 to reach the testPassedLimit -128 (from the start value 0), the count value cannot be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. In this case, and only in this case, may a pre-counter or similar (specified by the implementer) be used in order to reduce the number of updates to 127 (at increases) and 128 (at decreases). Refer to section Definitions and Abbreviations for definition of a test sample. Exception to this requirement (e.g. due to legal requirements) is allowed but only when approved by the responsible authority appointed by OEM.

The stored counter shall be reported as a 1 byte signed numeric value with a range of –128 to + 127 on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

2098 v4 **Support for Fault Detection Counter #11**

Version history: (4):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.

Source: Table "Requirements for the implementation DTC fault detection counters"
Verification Method: Test
Legacy ID: 72038v3

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=11 may be implemented and if it is implemented then shall be according to the following definition:
The count value shall be equal to the maximum value the FDC10 has reached during current operation cycle.
The stored counter shall be reported as a 1 byte signed numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

2099 v2 **Support for Fault Detection Counter #12**

Version history:
(2):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements for the implementation DTC fault detection counters"
Verification Method: Test
Legacy ID: 72039v1

For all DTCs supported by the ECU a counter identified by DTCExtendedDataRecordNumber=12 shall be implemented according to the following definition:
The count value shall be equal to the maximum value, which is equal to or greater than unconfirmedDTCLimit, the FDC10 has reached since the last time DTC information was cleared.
The stored counter shall be reported as a 1 byte signed numeric value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

4.5.1.1.1.25 Requirements from section DTC time stamp

2142 v3 **Support for DTC time stamp #20**

Version history:
(3):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC time stamps"
Verification Method: Test
Legacy ID: 72041v2

For all DTCs supported by the ECU a data record identified by DTCExtendedDataRecordNumber=20 shall be implemented according to the following definition:
The record value shall be equal to the global real time (data record 0xDD00) that is taken the first time FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit, since DTC information was last cleared.

The stored data record shall be reported as a 4 byte value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

2143 v3 **Support for DTC time stamp #21**

Version history:
(3):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC time stamps"
Verification Method: Test
Legacy ID: 72042v2

For all DTCs supported by the ECU a data record identified by DTCExtendedDataRecordNumber=21 shall be implemented according to the following definition:
The record value shall be equal to the global real time (data record 0xDD00) that is taken the latest time FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit, since DTC information was last cleared.
If FDC10 is updated and reaches a value equal to or greater than unconfirmedDTCLimit more than once during an operation cycle then the time stamp shall be taken the first time only.
The stored data record shall be reported as a 4 byte value on request of the diagnostic services specified in reference *[Data_1] UDS Services*.

4.5.1.1.1.26 Requirements from section DTC status indicators

2133 v1 **Support for DTC status indicator #30**

Version history:
(1):

Purpose: To provide enhanced information about the occurence of a fault, that may be useful in the analysis of the fault.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72063v0

For all DTCs supported by the ECU a status indicator record identified by DTCExtendedDataRecordNumber=30 (SI30) shall be implemented
The status indicator record shall be reported as a 1 byte value.

2134 v4 **Support for DTC status indicator #30, AgedDTC.**

Version history:
(4):

Purpose: To provide information about the maximum value of DTC status bit 7 since DTC information was last clearedaged DTC.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test

Legacy ID: 72067v3

When implementing with AUTOSAR 4.0.x:
For all DTCs supported by the ECU, bit no 3 of SI30 shall indicate AgedDTC according to the following definition:
- The bit is set to 1 when the DTC is aged.
- The bit is reset to 0 when the aged DTC reoccurs.

When implementing with AUTOSAR 4.1 or newer:
For all DTCs supported by the ECU, bit no 3 of SI30 shall be set to 0.

2135 v2 Support for DTC status indicator #30, EmissionRelatedDTC

Version history:
(2):

Purpose: To allow for a tool to understand if the DTC is emission related or not.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Analysis
Legacy ID: 72070v1

For all DTCs supported by the ECU, bit no 6 of SI30 shall indicate EmissionRelatedDTC according to the following definition:
The bit shall be set to 1 when the DTC is emission related.

2136 v2 Support for DTC status indicator #30, SymptomSinceLastClear

Version history:
(2):

Purpose: To provide information that a fault indicated by the DTC has caused a customer symptom since DTC information was last clear.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72069v1

For all DTCs supported by the ECU, bit no 4 of SI30 shall be supported as defined by a feasibility study performed by the implementer. (The feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM).
If implemented bit no 4 of SI30 shall indicate SymptomSinceLastClear according to the following definition:
The bit shall be set to 1 when FDC10 is in the range [unconfirmedDTCLimit, +127] and specific conditions specified by the implementer, that need to be fulfilled in order to generate customer symptom, are satisfied.

2137 v2 Support for DTC status indicator #30, testFailedSinceLastClear/Aged

Version history:

(2):

Purpose: To provide information about the DTC test failed since the DTC was last aged or, as long as the DTC have not been aged, since the DTC information was last clear.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72071v1

For all DTCs supported by the ECU, bit no 7 of SI30 shall indicate testFailedSinceLastClear/Aged according to the following definition:
The bit is set to set to 1 wthen DTC test is falied i.e. when the FDC10 reach the value +127.
The bit is reset to 0 when DTC is aged i.e. the SI30:3 is set to1.

2138 v4 Support for DTC status indicator #30, UnconfirmedDTC

Version history:
(4):

Purpose: To provide an early indication that a fault may be currently present.
Source: Table "Requirements on implementation of DTC status indicators";
Verification Method: Test
Legacy ID: 72064v3

When implementing with AUTOSAR 4.0.x:
For all DTCs supported by the ECU, bit no 0 of SI30 shall indicate UnconfirmedDTC according to the following definition:
The bit shall be set to 1 when FDC10 reaches a value in the range [unconfirmedDTCLimit, +127]
The bit shall be reset to 0 when FDC10 reaches the value -128.

The value of the unconfirmedDTCLimit is specified by the implementer according to the requirements specified in section Generic requirements on calibration of DTCs.

The value shall be in the range of [+1, +127].

When implementing with AUTOSAR 4.1 or newer:
For all DTCs supported by the ECU, bit no 0 of SI30 shall be set to 0.

2139 v3 Support for DTC status indicator #30, UnconfirmedDTCSinceLastClear

Version history:
(3):

Purpose: To provide an early indication that a fault may be present or has been present since last clear.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72066v2

When implementing with AUTOSAR 4.0.x:
For all DTCs supported by the ECU, bit no 2 of SI30 shall indicate UnconfirmedDTCSinceLastClear according to the following definition:
- The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127].

When implementing with AUTOSAR 4.1 or newer:
For all DTCs supported by the ECU, bit no 2 of SI30 shall be set to 0.

2140 v3 Support for DTC status indicator #30, UnconfirmedDTCThisOperationCyle

Version history:
(3):

Purpose: To provide an early indication that a fault may be present in the current operation cycle.
Source: Table "Requirements on implementation of DTC status indicators"
Verification Method: Test
Legacy ID: 72065v2

When implementing with AUTOSAR 4.0.x:
For all DTCs supported by the ECU, bit no 1 of SI30 shall indicate UnconfirmedDTCThisOperationCyle according to the following definition:
The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127].
The bit shall be reset to 0 at the start of an operation cycle.

When implementing with AUTOSAR 4.1 or newer:
For all DTCs supported by the ECU, bit no 1 of SI30 shall be set to 0.

2141 v2 Support for DTC status indicator #30, WarningIndicatorRequestedSinceLastClear

Version history:
(2):

Purpose: To provide information about the maximum value of DTC status bit 5 since DTC information was last cleared.
Source:
Verification Method: Test
Legacy ID: 115621v1

For all DTCs supported by the ECU, bit no 5 of SI30 shall indicate WarningIndicatorRequestedSinceLastClear according to the following definition:
The bit shall be set to 1 when DTC status bit no 7 is set to 1.
However, the bit may also be set to 1 even when DTC status bit not 7 – warningIndicatorRequested is not set to 1 (refer to reference [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications regarding requirements on set of DTC status bit not 7 to 1) according to the following:
The bit is set to 1 when the same criteria as required by the above mention reference on set of DTC status bit not 7 – warningIndicatorRequested to 1 is satisfied, but it may also be set to 1 and warning indicator may be requested before DTC status bit 3 – confirmed DTC is set to 1.
Note that in all cases must FDC10 has reached the value +127, since DTC information was latest cleared, before the bit is set to 1 and warring indicator is requested (if not otherwise is

specified in this document).

4.5.1.1.1.27 Requirements from section Store DTC information

2303 v2 DTC storage time

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is up to date when reported by diagnostic service.
Source:
Verification Method: Analysis
Legacy ID: 115623v0

The DTC information shall be stored within a maximum delay time of "DTC storage time" T1=400 ms. The parameter "DTC storage time" T1 is defined as the time from that an event occurs (e.g. a fault is detected) that cause DTC information to be stored (updated) until the DTC information is available for access by diagnostic service specified in reference [1]. If many events (e.g. 10 or more) occur within a time period of T1 then may the "DTC storage time" be increased but only if that is approved by the responsible authority appointed by OEM.

2304 v3 Storage of DTC fault detection counters 10 and 11 in short term memory

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source:
Verification Method: Analysis
Legacy ID: 115663v1

I t shall be possible to store the value of the DTC fault detection counter 0x10 in short term memory for a all DTCs supported by the ECU. If the DTC fault detection counter 0x11 is supported by the ECU then shall it be possible to store the value of the DTC fault detection counter 0x11 in short term memory for all DTCs supported by the ECU.

2305 v2 Storage of DTC fault detection counters 12 in long term memory

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72084v0

I t shall be possible to store the value of the DTC fault detection counter 0x12 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number*

of DTCs for which DTC information shall be possible to store in long-term memory.

2306 v2 **Storage of DTC information in short term memory**

Version history:
(2):

Purpose: To ensure that the requirements for storage in long term memory is also fulfilled by the short term memory.
Source: Section "Store DTC information"
Verification Method: Analysis
Legacy ID: 72072v1

If DTC information is temporarily stored in short term memory before being stored in long term memory it shall be possible to store at least the same amount of DTC information in short term memory as in long term memory.

2307 v2 **Storage of DTC operation cycle counter 5 in long term memory**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72083v0

If the DTC is emission related it shall be possible to store the value of the DTC operation cycle counter 5 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

2308 v3 **Storage of DTC operation cycle counters 1, 2, 3, 4, 6 and 7 in long term memory**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72081v1

It shall be possible to store the values of the DTC operation cycle counters 1, 2, 3, 4, 6 and 7 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

2309 v3 **Storage of DTC snapshot data in long term memory**

Version history:

(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72074v1

It shall be possible to store the values of the DTC snapshot data defined by DTCSnapshotRecordNumber 0x20 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

2310 v3 Storage of DTC status bits no 0, 2, 3, 4 and 5 in long term memory

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72073v1

It shall be possible to store the values of DTC status bits no 0, 2, 3, 4, and 5 in long term memory for all DTCs supported by the ECU.

2311 v3 Storage of DTC status bits no 1, 6 and 7 in short term memory

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source:
Verification Method: Analysis
Legacy ID: 115662v1

It shall be possible to store the values of DTC status bits no 1, 6 and 7 in short term memory for all DTCs supported by the ECU.

2312 v2 Storage of DTC status indicator bit no 1 in short term memory

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source:
Verification Method: Analysis
Legacy ID: 115664v0

It shall be possible to store the value of the DTC status indicator bit no 1 defined by DTCExtendedDataRecordNumber 0x30 in short term memory for a minimum number of DTCs according to Table *Requirements on minimum number of DTCs for which DTC information shall be possible to store* .

2313 v3 Storage of DTC status indicator bits no 0, 2-7 in long term memory

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72121v1

It shall be possible to store the value of the DTC status indicator no 0, 2-7 defined by DTCExtendedDataRecordNumber 0x30 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

2314 v2 Storage of DTC time stamp in long term memory

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure that DTC information is not lost when the ECU is powered down and to define the required storage capacity for the ECU.
Source: Table "Requirements on storage of DTC information in long-term memory"
Verification Method: Analysis
Legacy ID: 72085v0

It shall be possible to store the values of the DTC time stamp defined by DTCExtendedDataRecordNumber 0x20 and 0x21 in long term memory for a minimum number of DTCs according to table *Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory.*

4.5.1.1.1.28 Requirements from section Transfer of DTC information to long term memory

2335 v3 Transfer of DTC information to long term memory

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that no DTC information is lost during normal vehicle operation.
Source: Section "Transfer of DTC information to long term memory"
Verification Method: Analysis
Legacy ID: 72141v1

If the ECU temporarily stores DTC information in short-term memory before storing it in long-term memory, any DTC information that has been added, deleted or changed during the

operation cycle shall be transferred from short-term memory to long-term memory at the end of the operation cycle, and whenever required before it is erased from the short term memory if the erase is caused by normal vehicle operation.
Situations/events, which are not regarded as normal vehicle operation, are:

1) Unexpected disconnection of the ECU power supply while the ECU is in operation, due to Vehicle battery removal, fuse blow, connector disconnection or other similar cause which does not occur as part of normal vehicle operation.
2) ECU supply voltage below minimum specification due to e.g. weak battery or other vehicle related fault.
3) Diagnostic service requests that results in an ECU reset. See *[Data_1] UDS Services*, for services which are allowed to reset the ECU.
4) Additional events, not covered by items 1-3 above, which have been documented by the implementer and approved by the responsible authority by OEM.

DTC information from previous operation cycle shall be retained if temporarily stores DTC information in short-term memory is lost, in the current operation cycle, due to vehicle operation that is not regarded as normal.

4.5.1.1.1.29 Requirements from section Clear DTC information

1811 v3 — Clear DTC information by diagnostic service

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To allow all DTC information to be cleared by request from e.g. the service technician.
Source: Table "Requirements in implementation of clear of DTC information" - item 1
Verification Method: Analysis
Legacy ID: 72146v1

All stored DTC information for all DTCs supported by the ECU , with the exceptions defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications , shall be cleared when requested by diagnostic service specified in *[Data_1] UDS Services*.

1812 v6 — Clear DTC information when a DTC is aged

Version history:
(6): Change the Verification Method from Test to Analysis

Purpose: To allow the DTCConfirmation status to be reset if a failure has not been detected for a defined time.
Source: Table "Requirements in implementation of clear of DTC information" - item 3
Verification Method: Analysis
Legacy ID: 72152v4

When implementing with AUTOSAR 4.0.x:
When the counter used for aging of a DTC reaches the value agedDTCLimit defined for the DTC, no DTC information shall be cleared with some exception (reset of a DTC status bit and

some DTC status indicators) that is specified in the document.
The agedDTCLimit shall be a value in the range of [1, 255]. The counter used for aging shall either be operation cycle counter 2 or 5. The implementer shall specify the value of the agedDTCLimit according to the requirements specified in section *Generic requirements on calibration of DTCs* and which of the two counters shall be used for aging. An addtion to specified above is that emission related DTCs may use both counters for aging. I this case is the DTC aged when both counters have reached their agedDTCLimit.

When implementing with AUTOSAR 4.1 or newer:
When the counter used for aging of a DTC reaches the value agedDTCLimit defined for the DTC, all DRC Snapshot data and Extended data records shall be cleared. All DTC status bits shall be cleared as well, except fro status bit testFailedSinceLastClear, this bit will retain its status.
The agedDTCLimit shall be a value in the range of [1, 255]. The counter used for aging shall either be operation cycle counter 2 or 5. The implementer shall specify the value of the agedDTCLimit according to the requirements specified in section Generic requirements on calibration of DTCs and which of the two counters shall be used for aging. An addtion to specified above is that emission related DTCs may use both counters for aging. I this case is the DTC aged when both counters have reached their agedDTCLimit.

1813 v3 **Clear DTC information when an aged DTC reoccurs**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To ensure that new DTC information is captured when an aged DTC reoccurs.
Source: Table "Requirements in implementation of clear of DTC information" - item 4
Verification Method: Analysis
Legacy ID: 72154v1

When implementing with Autosar 4.0x:
If FDC10 for a DTC reaches a value equal to or greater than unconfirmedDTCLimit, after the DTC has been aged (i.e. the counter used for aging of the DTC has reached the value agedDTCLimit defined for the DTC), all DTC information, except the DTC status bits and FDC10, shall be cleared for that DTC.

When implementing with Autosar 4.1 or newer:
This requirement is Not Applicable

1814 v6 **Clear DTC information when the long term memory overflows**

Version history:
(6): Change the Verification Method from Test to Analysis

Purpose: To define the rules for what DTC information that shall be stored and what shall be discarded in the event that long term DTC memory is full when new DTC information needs to be stored.
Source: Table "Requirements in implementation of clear of DTC information" - item 2
Verification Method: Analysis

Legacy ID: 72148v4

If storage of DTC information is required but available storage for the DTC information in the long term memory is already full, all DTC information for all DTCs supported by the ECU, except the DTC status bits and FDC10, of lowest storage priority (see note 1 below) shall be cleared from the long term memory and replaced by the data that is required to be stored if the data that is required to be stored has higher storage priority according to the following priority order (1 is the highest priority):
1. The DTC is associated with a warningIndicator (i.e. support the value 1 of the DTC status bit 7) and is not aged. See note 2 below.
2. The DTC is not associated with a warningIndicator (i.e. do not support the value 1 of the DTC status bit 7).See note 2 below.
3. The DTC is aged (i.e. the DTC status indicator 30, bit 3 is set to 1).

Excluded from above stated storage priority are developent specific DTCs (refer to table *Requirements on implementation of DTCs* regarding definition of development specific DTCs). They shall have the lowest storage priority (i.e, priority 4) of all DTCs supported by the ECU. However, development specific DTCs may have higher storage priority if the minimum number of DTCs that shall be possible to store in long term memory (according to Table: Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory) is increased with the number of the development specific DTCs that shall have higher storage priority. The mutual storage priority between developments specific DTCs is specified by the implementer.
Safety and emission related DTCs and other specific DTCs may have additional storage priorities than above where approved by the responsible authority appointed by OEM.

Note 1: If several data items (DTC information for several DTCs) of the same storage priority are stored then shall the data item that has the highest value of OOC2 be cleared. If several data items (DTC information for several DTCs) of the same priority and the same value of OCC2 are stored then is it up to the implementer to specify which of these data that shall be cleared.

Note 2: Stored DTC information is not aged when DTC status indicator 30, bit 3 is set to 0. DTC information that is required to be stored is not aged because DTC information is required to be stored when the DTC fault detection counter 10 reach a value in the range [unconfirmedDTCLimit, +127].

4.5.1.1.1.30 Requirements from section Routines - General

2298 v2 **Access to Vehicle manufacturer specific routines**

Version history:
(2):

Purpose: To support diagnostic procedures it must be possible to execute the vehicle manufacturer specific routines in all diagnostic session.
Source: Table "Requirements on implementation of routines" - item 1
Verification Method: Test
Legacy ID: 72158v1

It shall be possible to access control routines with routine identifiers in the ranges 0x0200 to

0xDBFF, and 0xFF00 to 0xFFFF, by diagnostic services specified in *[Data_1] UDS Services*, in the diagnostic sessions that is specified by the implementer (refer to *[Data_1] UDS Services* for restrictions on which diagnostic sessions the different routine types that are allowed in).

2299 v1 **Development Specific routines**

Version history:
(1):

Purpose: Development Specific routines are not supported by OEM specific tools and must be defined in a separate identifier range.
Source: Table "Requirements on implementation of routines" - item 2
Verification Method: Analysis
Legacy ID: 72161v0

If routines that are needed only during the development of the ECU, or for analysis of ECUs returned to the vehicle manufacturer, are defined by the implementer, these routines shall have identifiers in the range 0xDC00 to 0xDFFF.

2300 v1 **ISO specified routines**

Version history:
(1):

Purpose: Routines defined by ISO that shall or may be supported
Source:
Verification Method: Inspection
Legacy ID: 128777v0

Routines with routine identifiers in the ranges 0xFF00 to 0xFF02 shall be implemented as they are defined by reference I*Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications . Routnes specified in section *Supported routines* shall be defined with routine identifiers in these ranges.

2301 v2 **System Supplier Specific routines**

Version history:
(2):

Purpose: Routines needed only by the system supplier shall be defined in a separate range which may not be supported by OEM specific tools.
Source: Table "Requirements on implementation of routines" - item 5
Verification Method: Analysis
Legacy ID: 72166v1

If routines that are needed only by the system supplier are defined by the system supplier, these routines shall have identifiers in the range 0xF000 to 0xFEFF. Routines in this range shall comply with the requirements in reference *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS DATA External documents* , but are not required to comply with requirements in reference *[Data_1] UDS Services.*

2302 v3 **Vehicle manufacturer specific routines defined in GMRDB**

Version history:
(3):

Purpose: Mandatory data records in are defined in GMRDB.
Source: Table "Requirements on implementation of routines" - item 1
Verification Method: Analysis
Legacy ID: 72157v2

Routines with routine identifiers in the ranges 0x0200 to 0xDBFF shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*. Routines specified in section 4.3.2 shall be defined with routine identifiers in these ranges.

4.5.1.1.1.31 Requirements from section Supported routines

2315 v1 **Access to ActivateSBL routine**

Version history:
(1):

Purpose: To support SWDL it must be possible to execute the required routine from SBL and PBL.
Source: Table "Requirements for implementation of routines" - item 3
Verification Method: Test
Legacy ID: 72201v0

It shall be possible to execute the ActivateSBL routine, by diagnostic service request specified in *[Data_1] UDS Services*, when the ECU is executing the primary and secondary bootloader in programming session.

2316 v1 **Access to Check Complete & Compatible routine**

Version history:
(1):

Purpose: To support SWDL it must be possible to execute the required routine from SBL and PBL.
Source: Table "Requirements for implementation of routines" - item 1
Verification Method: Test
Legacy ID: 72170v0

It shall be possible to execute the Check Complete & Compatible routine, by diagnostic services specified in *[Data_1] UDS Services*, when the ECU is executing the primary and secondary bootloader in programming session.

2317 v1 **Access to Check Memory routine**

Version history:
(1):

Purpose: To support SWDL it must be possible to execute the required routine from SBL

and PBL.

Source:

Verification Method: Test

Legacy ID: 418032v0

It shall be possible to execute the Check Memory routine, by diagnostic services specified in *[Data_1] UDS Services*, when the ECU is executing the primary and secondary bootloader in programming session.

2318 v1 Access to Check Programming Pre-conditions routine

Version history:
(1):

Purpose: To support SWDL it must be possible to execute the required routine before the ECU enters programme mode.

Source: Table "Requirements for implementation of routines" - item 2

Verification Method: Test

Legacy ID: 72183v0

It shall be possible to execute the Check Programming Pre-conditions routine, by diagnostic service request specified in *[Data_1] UDS Services*, in the default and extended diagnostic sessions.

2319 v1 Access to Erase Memory routine

Version history:
(1):

Purpose: To support SWDL it must be possible to execute the required routine from SBL.

Source: Table "Requirements for implementation of routines" - item 4

Verification Method: Test

Legacy ID: 72205v0

It shall be possible to execute the Erase Memory routine, by diagnostic service request specified in *[Data_1] UDS Services*, only when the ECU is executing the secondary bootloader in programming session.

2320 v2 Access to Software Installation routine

Version history:
(2):

Purpose: To support SWDL it must be possible to execute the required routine from SBL and PBL.

Source:

Verification Method: Test

Legacy ID: 115690v1

It shall be possible to execute the Software installation routine, by diagnostic service request

specified in reference [1] , when the ECU is executing the primary and secondary bootloader in programming session.

2321 v2 **Access to Software Installation routine**

Version history:
(2):

Purpose: To define the routine type so that the tool can access the routine in the correct way.
Source:
Verification Method: Test
Legacy ID: 115688v1

The Software Installation routine shall be implemented as a type 2 routine.

2322 v1 **ActivateSBL routine**

Version history:
(1):

Purpose: All ECUs must support routines defined for SWDL.
Source: Table "Requirements for implementation of routines" - item 3
Verification Method: Test
Legacy ID: 72185v0

The ActivateSBL routine with routine identifier 0x0301 shall be implemented as defined in *[Data_4] Global Master Reference Database*. The routine shall be used to activate the Secondary Bootloader after it has been downloaded to volatile memory. The ECU shall start executing the secondary bootloader from the memory address defined in the data file containing the Secondary Bootloader.

2323 v2 **ActivateSBL routine response**

Version history:
(2):

Purpose: Type 1 routines shall not send a response until the routine has been completed, i.e. in this case not until the SBL has been activated.
Source: Table "Requirements for implementation of routines" - item 3.
Verification Method: Test
Legacy ID: 72202v1

The final positive response message from the ActivateSBL routine shall be sent when the SBL has been activated, i.e. from the SBL.
If the SBL is already activated at the time of the request the ECU shall respond with a positive response message with RoutineCompleted=0 (Routine is completed).

2324 v1 **ActivateSBL routine type**

Version history:
(1):

Purpose: To define the routine type so that the tool can access the routine in the correct way.
Source: Table 4-30, item 3
Verification Method: Test
Legacy ID: 152003v0

The ActivateSBL routine shall be implemented as a type 1 routine.

2325 v1 Check Complete & Compatible routine

Version history:
(1):

Purpose: All ECUs must support routines defined for SWDL.
Source: Table "Requirements for implementation of routines" - item 1
Verification Method: Test
Legacy ID: 72168v0

The Check Complete & Compatible routine with routine identifier 0x0205 shall be implemented as defined in *[Data_4] Global Master Reference Database* and *[Data_6] General Software Download Specification*.

2326 v1 Check Complete & Compatible routine type

Version history:
(1):

Purpose: To define the routine type so that the tool can access the routine in the correct way.
Source: Table "Requirements for implementation of routines" - item 1
Verification Method: Test
Legacy ID: 72169v0

The Check Complete & Compatible routine shall be implemented as a type 1 routine.

2327 v2 Check Memory routine

Version history:
(2):

Purpose: To initiate the authenticity verification of the downloaded data file at SWDL.
Source:
Verification Method: Test
Legacy ID: 418011v1

If the ECU is supporting the Software Authentication concept as defined in *[Data_10] General Software Authentication*, the ECU shall implement the Check Memory routine with routine identifier 0x0212 as defined in *[Data_4] Global Master Reference Database* and *[Data_6] General Software Download specification*.

2328 v1 **Check Memory routine type**

Version history:
(1):

Purpose: To define the routine type so that the tool can access the routine in the correct way.
Source:
Verification Method: Test
Legacy ID: 418031v0

The Check Memory routine shall be implemented as a type 1 routine.

2329 v1 **Check Programming Pre-conditions routine**

Version history:
(1):

Purpose: All ECUs must support routines defined for SWDL.
The tester will send requests to all ECU to find out if they are prepared to enter program mode. This enables a robust and secure transition from the application to the bootloader by ensuring that all pre-conditions to transition to the bootloader are fulfilled when requested.
Source: Table "Requirements for implementation of routines" - item 2
Verification Method: Test
Legacy ID: 72177v0

The Check Programming Pre-conditions routine with routine identifier 0x0206 shall be implemented as defined in *[Data_4] Global Master Reference Database*. The routine shall check if the ECU, under its current operating conditions, is able to enter program mode.
If the ECU is able to enter program mode or not, i.e. if the criteria for entering program mode are fulfilled or not, at the moment of the routineControl request, shall be indicated by the corresponding RoutineStatusRecord response parameter.

2330 v1 **Check Programming Pre-conditions routine type**

Version history:
(1):

Purpose: To define the routine type so that the tool can access the routine in the correct way.
Source: Table "Requirements for implementation of routines" - item 2
Verification Method: Test
Legacy ID: 72181v0

The Check Programming Pre-conditions routine shall be implemented as a type 1 routine.

2331 v2 **Erase Memory routine**

Version history:
(2):

Purpose: All ECUs must support routines defined for SWDL.
Source: Table "Requirements for implementation of routines" - item 4
Verification Method: Test
Legacy ID: 72203v1

The Erase Memory routine with routine identifier 0xFF00 shall be implemented as defined in *[Data_4] Global Master Reference Database*. The routine shall perform a non-volatile memory erase of the memory blocks defined by the memoryAddress and memorySize request parameters. The routine does not to be supported if the ECU has file system data storage and implements an alternative method e.g RequestFileTransfer – modeOfOperation parameter DeleteFile or ReplaceFile.

2332 v1 Erase Memory routine "already erased" detection

Version history:
(1):

Purpose: To reduce the overall software download time in case the memory is already erased when the ECU is delivered the factory.
Source: Table "Requirements for implementation of routines" - item 4
Verification Method: Test
Legacy ID: 72206v0

If the ECU is delivered to CEVT with the complete programmable memory already erased, the routine shall include an "already erased" detection in order to reduce the overall software download time. If the complete programmable memory is "already erased" and all other preconditions are met (e.g. SBL is downloaded and activated), the routine shall not perform any erase operation and immediately send a final positive response message with RoutineCompleted=1 (Routine is completed).
Once one or more bytes of the ECU programmable memory area has been programmed, the "already erased" status shall be invalidated.
Any erase memory requests when the "already erased" status is invalid shall erase the requested memory area.

2333 v1 Erase Memory routine type

Version history:
(1):

Purpose: To define the routine type so that the tool can access the routine in the correct way.
Source: Table "Requirements for implementation of routines" - item 4
Verification Method: Test
Legacy ID: 72204v0

The Erase Memory routine shall be implemented as a type 1 routine.

2334 v2 Software Installation routine

Version history:
(2):

Purpose: To start installation of a downloaded data file which requires to be installed.
Source:
Verification Method: Test
Legacy ID: 115685v1

The Software Installation routine with routine identifier 0xD000 shall be implemented as defined in reference [4]. The routine is optional and shall be used for ECUs supporting file system storage to start an installation of a downloaded data file.
The routineContolOptionRecord has a dynamical length and shall be equal to the file system location of the ECU where the data file is stored included with the file name with extension.

4.5.1.1.1.32 Requirements from section Response times

2291 v1 **Check Complete & Compatible response time**

Version history:
(1):

Purpose: The Check Complete & Compatible routine must be executed quickly in order to fulfil the timing requirement of SWDL.
Source: Table "Requirements on P4server_max for routines"
Verification Method: Test
Legacy ID: 72214v0

The response time $P4_{server_max}$ for the Check Complete & Compatible routine shall be equal to $P2_{Server_max}$. (The timing parameters $P4_{server_max}$ and $P2_{Server_max}$ are defined in *[Data_2] UDS Session*.)

2292 v1 **Check Memory response time**

Version history:
(1):

Purpose: Check Memory is computional heavy, hence it is time consuming. The response time is also dependent of the pre-calculation capabilities.
Source:
Verification Method: Test
Legacy ID: 418071v0

The response time $P4_{server_max}$for the Check Memory routine shall be 2000 ms.

2293 v1 **Check Programming Pre-conditions response time**

Version history:
(1):

Purpose: The Check Programming Pre-conditions routine must be executed quickly in order to fulfil the timing requirement of SWDL.
Source: Table "Requirements on P4server_max for routines"
Verification Method: Test

Legacy ID: 72217v0

The response time P4 $_{server_max}$ for the Check Programming Pre-conditions routine shall be equal to P2 $_{Server_max}$. (The timing parameters P4 $_{server_max}$ and P2 $_{Server_max}$ are defined in *[Data_2] UDS Session*.)

2294 v1 **Erase Memory response time**

Version history:
(1):

Purpose: Erasing Flash Memory is a time consuming routine that needs longer time than allowed for other routines.
Source: Table "Requirements on P4server_max for routines"
Verification Method: Test
Legacy ID: 72220v0

The response time P4 $_{server_max}$ for the Erase Memory routine shall be 60000 ms. (The timing parameter P4 $_{server_max}$ is defined in reference *[Data_2] UDS Session*.)

2295 v3 **Extended Erase Memory response time**

Version history:
(3):

Purpose: The Erase Memory routine needs longer time to erase a large Flash memory.
Source: Table "Requirements on P4server_max for routines"
Verification Method: Test
Legacy ID: 72221v2

The response time P4 $_{server_max}$ for the Erase Memory routine shall be increased by an additional 20s for each megabyte range of data over 1 megabyte that is requested to be erased. (The timing parameters P4 $_{server_max}$ is defined in reference *[Data_2] UDS Session*.).

2296 v1 **Extended Erase Memory response time - ECU Software Structure**

Version history:
(1):

Purpose: The complete programmable memory will be erased when the ECU Software Structure, ESS, is updated. The Erase Memory routine needs longer time.
Source: Table "Requirements on P4server_max for routines"
Verification Method: Test
Legacy ID: 428976v0

The P4 $_{server_max}$ for an ESS update shall be defined by the size of the total memory, e.g. the sum of all erase requests for all programmable data files.

2297 v2 **Software Installation response time**

Version history:

(2):

Purpose: The response time for a type 2 routine shall be quickly.
Source:
Verification Method: Test
Legacy ID: 115691v1

The response time $P4_{server_max}$ for the Software Installation routine shall be equal to $P2_{Server_max}$. (The timing parameters $P4_{server_max}$ and $P2_{Server_max}$ are defined in reference *[Data_2] UDS Session*).

4.5.1.1.2 APPENDIX

4.5.1.1.2.1 Appendix A

Data records requested and reported via diagnostic services shall fit into one of the data classifications as described below.

Where multi-byte data records are specified and the byte order is not already specified in *[Data_4] Global Master Reference Database* then the order shall be that the most significant byte of the multi-byte record shall be placed in the data byte with the lowest number according to the data byte numbering convention used in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*.

Data type 1- ASCII (American Standard Code for Information Interchange)
The ASCII data type contains one character per byte with a value between 0x00 and 0x7F using the standard ASCII code. ASCII parameters shall always be left justified (i.e., the most significant byte represents the first character) with any unused bytes filled with 0x00.

Data type 2- BCD (Binary Coded Decimal)
The BCD data type contains two numbers per byte with a value between 0 and 9 using the standard 4 bit hex value for each number. BCD parameters shall always be left justified (i.e., the most significant bit represents the first number) with any unused nibbles filled with 0xF.

Data type 3- State Encoded
The state encoded data type is used when hexadecimal values represent unique states (e.g., using different values to represent the states of the ignition switch). State encoded parameters shall be limited to a maximum size of 1 byte (i.e., 256 unique states).

Data type 4- Bit-Mapped
The bit-mapped data type contains the binary state (0/1) of a parameter in a single bit (e.g., the discrete state of an ECU output may be represented as "0" for "Off" and "1" for "On"). If a particular bit-mapped parameter is not supported within a data record that is being reported by the ECU, the value of this bit shall be set to zero.

Data type 5- Unsigned Numeric
The unsigned numeric data type is used to represent data that has a defined range of values and a specific resolution that can be represented through hexadecimal encoding. Each unsigned numeric parameter consists of a size in bits (n), a resolution (r), and an offset. The minimum value of the unsigned parameter is equal to the offset. The maximum value of the

unsigned parameter is equal to (((2 ^{n}-1) * r) + offset). The maximum value of n is 32.

Data type 6- Signed Numeric
The signed numeric data type is used to represent data that has a defined range of values and a specific resolution that can be represented through 2's complement encoding. Each signed numeric parameter consists of a size in bits (n) and a resolution (r). The minimum value of the signed parameter is equal to - (2 $^{(n-1)}$ * r). The maximum value of the signed parameter is equal to ((2 $^{(n-1)}$ - 1) * r). The maximum value of n is 32.

Data type 7- Binary floating point
The binary floating point data type is used to represent data that has a defined range of values and a specific resolution that can be represented through the IEEE 754 single precision standard for binary floating point arithmetic. Each binary floating point parameter consists of a sign bit, exponent, and mantissa that is stored in accordance with single precision IEEE 754.

Data type 8 - Hexadecimal
The hexadecimal data type contains two characters per byte with a value between 0 and 9 and between A and F using the standard 4 bit hex value for each character.

4.5.1.1.3 Overview

N/A (reserved for future use)

4.5.1.1.3.1 Definitions and Abbreviations

4.5.1.1.3.1.1 Abbreviations

Abbreviation	Stands for	Description
ASIL	Automotive Safety Integrity Level	See *Road vehicles — Functional safety, International Organization for Standardization, 2011, UDS - Data - External publications*
D	Don't Care	Implementation of the requirement is not required and not forbidden. The implementer may decide if to implement or not. If implemented it will not be verified by VCC and shall not be expected to be supported by VCC tools.
DID	Data Identifier	
DTC	Diagnostic Trouble Code	
ECU	Electronic Control Unit	
FM	Frequency Modulation	Type of signal modulation
FTB	Failure Type Byte	See *Diagnostic trouble code definitions, UDS - Data - External publications*
GGDS	Generic Global Diagnostic Specification	Previous versions of VCC diagnostics specification
GMRDB	Global Master Reference Data Base	

HMI	Human Machine Interface	
M	Mandatory	
MC	Mandatory on Condition(s)	The condition(s) is specified separately for each occurrence.
MCPS	Mandatory on Condition for primary and secondary bootloader	The condition(s) is specified separately for each occurrence.
MCS	Mandatory on Condition for secondary bootloader	The condition(s) is specified separately for each occurrence.
MPS	Mandatory for primary and secondary bootloader	Shall be supported by the ECU.
MS	Mandatory for secondary bootloader	Shall be supported by the ECU.
N	Not allowed	
N/A	Not applicable	
O	Optional	May be supported by the ECU. If the ECU supports it then shall it be supported exactly as specified.
OS	Optional in secondary bootloader. Not allowed in primary bootloader.	May be supported by the ECU. Whether or not the ECU shall support it is defined by reference *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, 001, 2006-12-01 Corrected 2007-04-15, UDS - Data - External publications* or, if it is not defined by this reference, by the implementer (if not otherwise is explicitly specified by this document).
PCM	Pulse Code Modulation	Type of signal modulation
PWM	Pulse Width Modulation	Type of signal modulation
SBL	Secondary Bootloader	
SWDL	Software Download	Software Download according to reference *[Data_6] General Software Download Specification.*
TRC	Test Run Criteria	
UDS	Unified Diagnostic Services	
VCC	Volvo Car Corporation	

4.5.1.1.3.1.2 Definitions

Definition	Description
DTC Test / Test	Same as 'Test' defined in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* , UDS - Data - External publications, Appendix D.2.1.

Implementer	Person responsible for the diagnostic implementation of the ECU. It may be the vehicle manufacturer or the ECU supplier diagnostic software designer.
Long-term memory	A non-volatile memory (e.g., EEPROM or Flash).
Main circuit board	The circuit board in an ECU containing the micro-controller that supports the requirements specified in *[Data_1] UDS Services*.
Public ECU master	A public ECU responsible to act as a diagnostic server for one or more private ECUs.
Operation cycle	Refer to section *ECU ability to receive requests* in *[Data_1] UDS Services* for definition of an operation cycle and requirements on an ECU to start and stop an operation cycle.
Private ECU	An ECU not addressable from a client/tester point of view. Note: An ECU can partly be public and partly private i.e. some diagnostic services and diagnostic data supported by the ECU are addressable and some are not. Refer to the implementation specification for respective network (e.g. UDSonLIN, UDSonCAN, etc.) for which diagnostic services and diagnostic data supported by the ECU that may be or may not be addressable.
Public ECU	An ECU addressable from a client/tester point of view.
Short-term memory	A volatile memory (e.g. RAM).
Responsible authority appointed by OEM	A person, appointed by OEM for a specific vehicle program, which is responsible for support, review and approval of the implementation of the diagnostic part of VDS4 in the vehicle.
Test Sample	A test sample represents the pass or fail result from a single instance of a DTC test execution when the test run criteria are met. This represents a single sample and therefore not generally a fully matured / qualified condition. A test sample representing a fail will increase the DTC Fault Detection Counter by a specific amount and a test sample representing a pass will decrease the DTC Fault Detection Counter by a specific amount. Note the difference between "test sample passed" and "test passed".

VCC tools	VCC tools include all tools used by VCC R&D, manufacturing and service. These include, but are not limited to: · Development tools (used for e.g. analysis, test, calibration etc.) · Manufacturing tools · Aftersales tools · Databases · Embedded software components

4.5.1.1.3.2 External Publications

Reference number	Source	Title	Version or Date	Document number
[Data_50]	ISO	Road vehicles – Diagnostics systems – Diagnostic services	001, 2006-12-01 Corrected 2007-04-15	ISO/DIS 14229-1
[Data_51]	SAE	Diagnostic trouble code definitions	December 2007	SAE J2012
[Data_52]	CARB	OBD II regulation, section 1968.2 of title 13, California Code of Regulations	The latest available version	N/A
[Data_53]	ISO	Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 1: General information and use case definitions	2008-05-15	ISO26021-1
[Data_54]	ISO	Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 2: Communication requirements	2008-07-15	ISO26021-2
[Data_55]	ISO	Road vehicles — Functional safety , International Organization for Standardization, 2011	2011	ISO-26262
[Data_56] to [Data_99]		Reserved for future use		

Note that in the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by the responsible authority appointed by OEM before the conflict is recognized as resolved.

4.5.1.1.3.3 Requisite Documents

This specification shall be followed and the requirements specified in this document shall not be negotiable.

In the case of any requirement conflict between this document and any of the referenced external documents, the proposed solution to the conflict shall be approved by the responsible authority appointed by CEVT before the conflict is recognized as resolved.

Reference number	Source	Title
[Data_1]	Internal	UDS Services
[Data_2]	Internal	UDS Session
[Data_3]	Internal	Vehicle Mode Management - Concept SRD
[Data_4]	Internal	Global Master Reference Database
[Data_5]	Internal	Central Car Configuration
[Data_6]	Internal	General Software Download Specification
[Data_7]	Internal	Security access algorithm
[Data_8]	Internal	Overview of vehicle configuration & customer settings.
[Data_9]	Internal	FltEgyCns
[Data_10]	Internal	General Software Authentication
[Data_11]	Internal	RVDCclient
[Data_12]	Internal	General Diagnostic Guideline
[Data_13] to [Data_49]		Reserved for future use

4.5.1.1.4 Diagnostic Data

Diagnostic data shall be supported by the ECU as specified by this section. Requirements on diagnostic data are defined for both application and bootloader software. If not otherwise specified when data in programmingSession is defined shall the data to be supported both in

primary and secondary bootloader .

The diagnostic data required by this document shall be compliant to *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications. This section specifies additions and restrictions that are compliant to *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications.

Additionally requirements on diagnostic data that are network dependent are specified in each implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP).

Private ECUs shall, from a diagnostic point of view, be regarded as if they were internally connected units (e.g. internal I/O-units reading sensors and controlling actuators) within the public ECU master itself. The public ECU master shall, on behalf of the private ECUs, support all the diagnostic data required by this document and that are applicable for the private ECUs.

The diagnostic data that shall be supported by the ECU in the safetySystemDiagnosticSession are specified by reference *Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 1: General information and use case definitions*, UDS - Data - External publications and *Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 2:*Communication requirements, UDS - Data - External publications.

The ECU is not allowed to support other diagnostic data unless it is approved by the responsible authority appointed by OEM.

If the ECU implements safety requirements with an ASIL higher than QM, it shall be ensured that diagnostic data, implemented according to this specification, cannot violate any of those safety requirements, i.e. to demonstrate independence as required by *Road vehicles — Functional safety, International Organization for Standardization, 2011*, UDS - Data - External publications. Exception to this requirement is allowed only when approved by the responsible authority appointed by OEM and will require that the diagnostic data required in this specification are developed according to *Road vehicles — Functional safety, International Organization for Standardization, 2011*, UDS - Data - External publications and the allocated ASIL of those safety requirements for wich independence cannot be demonstrated.

Requirements from this section are specified in section "Requirements from section Diagnostic Data".

4.5.1.1.4.1 Record data

4.5.1.1.4.1.1 General

Data records (containing current, frozen and static data) shall be supported by the ECU according to the table below (e.g. in the valid identifier ranges).

Unless specified by this document, the implementer shall specify, for each data identifier, if the data record shall be possible to read, write and/or control by the services specified in *[Data_1] UDS Services*.

The data records shall fit into one of the data types and classifications as described in Appendix A. No other data types and classifications are allowed.

Table: Requirements on implementation of data records

Name	Valid identifier range	Definition, see note 5	Notes	Session, see note 6	
				Other	Program-ming
Vehicle Manufacturer specific	0100 – D8FF DE00 – E2FF E600 – ED1F ED80 - ED9F F010 - F0FF	The data items (e.g. external and internal signals) that shall be defined in these records are specified in sections *ECU external signals*, *ECU internal signals*, *ECU_Vehicle status*, *Network data* and *Local DTC snapshot data.*	1	M	D
Development Specific - implementer specified	D900 – DCFF E300 – E4FF EE00 – EFFF	Data records defined by the implementer. Data records that are needed only during the development of the ECU, for e.g. tuning, calibration and validation of the ECU's software, shall be defined in this range.	4	O	D
Development Specific	E500 - E5FF	Data records that are needed only during the development of the ECU, for e.g. tuning, calibration and validation of the ECU's software, are specified in this range.	4	O	D
DTC Global Snapshot Data	DD00 – DD7F	The data items that (e.g. external and internal signals) shall be defined in these records are specified in section *Global DTC snapshot data.*	1	M	D
Reserved by this document	DD80 – DDFF	This identifier range is reserved for future use.			
ECU Identification and Configuration	EDA0 – EDFF ED20 – ED7F	The data items that shall be defined in these records are specified in section *ECU_Vehicle identification.*	1	M	M
Identification Option – Vehicle Manufacturer Specific	F100 – F17F, F1A0 – F1EF	The data items that shall be defined in these records are specified in section *ECU_Vehicle identification*.	1	M	M
Identification Option – ISO specified	F180 – F19F	The data items that shall be defined in these records are specified in section *ECU_Vehicle status* and *ECU_Vehicle identification.*	1	M	M
Identification Options-System Supplier Specific	F1F0 – F1FF	Data records defined by the system supplier	4	O	D
Dynamically defined periodic DataIdentifier	F200 – F240	Periodic data records defined dynamically.	2	O	D
Reserved by this document	F241 – F2FF	This identifier range is reserved for future use.			

Dynamically Defined-DataIdentifier	F300 – F3FF	Data records defined dynamically.	3	O	D
System Supplier Specific	FD00 – FEFF	Data records defined by the system supplier	4	O	D

Notes in the table above:
1. Data records within this range shall be implemented **exactly** as they are defined in *[Data_4] Global Master Reference Database*.
2. If the ECU supports only one single dynamically defined periodicDataIdentifier then its value shall be 0xF200, if the ECU supports two dynamically defined periodicDataIdentifiers then the values shall be 0xF200 and 0xF201, etc.
Any identifier implemented within this range shall not be supported by services (specified by *[Data_1] UDS Services*) that can write or control the dynamically defined data parameters.
Any identifier implemented within this range shall be supported by services (specified by *[Data_1] UDS Services*) that can dynamically define a data identifier and read the dynamically defined data parameter periodically.
3. If the ECU supports only one single dynamicallyDefinedDataIdentifier then its value shall be 0xF300, if the ECU supports two dynamicallyDefinedDataIdentifiers then the values shall be 0xF300 and 0xF301, etc.
Any identifier implemented within this range shall not be supported by services (specified by *[Data_1] UDS Services*) that can write or control the dynamically defined data parameters.
Any identifier implemented within this range shall be supported by services (specified by *[Data_1] UDS Services*) that can dynamically define a data identifier and read the dynamically defined data parameter..
4. The implementer shall not expect that VCC tools support the data records within this range.
5. The identifiers that the ECU supports are specified by the implementer.
6. Note that “M” means that the ECU shall support data records according to column “Definition” of the table which shall be defined within the data record identifier range according to column “valid identifier range”. Note that “O” means that the ECU may support data records according to column “Definition” of the table which shall be defined within the identifier range according to column “valid identifier range”.

Requirements from this section are specified in section "Requirements from section Record Data - General".

4.5.1.1.4.1.2 ECU external signals

This section is not applicable for programming session.

All external signals (i.e. the signals on all the external circuits connected to the ECU) shall be defined in data records within the ECU according to the table below.

It shall be possible to read and control these data records, according to the table, by diagnostic

services specified in *[Data_1] UDS Services* in all diagnostic sessions except the programming session..

Table: Requirements for definition of ECU external signals in data records

Type of signal	Item	Signals to be defined in data records by the implementer	Diagnostic service, see note 7		Notes
			Read	I/O Control	
Switch input	1.1	Actual digital input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	M	O	1, 6
	1.2	Value currently available for the ECU´s application software (e.g., a filtered value used by the ECU when in normal operation or the replacement value used when the ECU is in limp-home operation).	MC1	O	1
Analog input	2.1	Actual input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	M	O	1, 2, 6
	2.2	Value currently available for the ECU´s application software (e.g. a filtered value used by the ECU when in normal operation or the replacement value used when the ECU is in limp-home operation).	MC1	O	1
FM, PCM, PWM (or similar) input	3.1	Actual FM, PCM, PWM (or similar) input signal in suitable engineering units.	M	O	1, 3, 6
	3.2	Actual digital input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	MC1	O	1, 6
Output	4.1	Output signal (the desired set point) controlled by the ECU´s application software or by the ECU´s diagnostic software (from the tester via a diagnostic service).	M	M	5
	4.2	Actual output signal read back by the ECU as a digital or analog signal.	M	N/A	1
Public ECU network signals	5.1	Network signals (e.g. data items embedded in CAN- or FlexRay-frames) transmitted by the ECU.	MC1	MC1	4, 5
	5.2	Network signals (e.g. data items embedded in CAN- or FlexRay-frames) received by the ECU.	MC1	O	6
	5.3	The network signal that contain the data record 0xDD02 – “Vehicle battery voltage” transmitted by the ECU.	MC2	MC2	4
External electrical circuits connected to private ECUs	6	Note that from a diagnostic point of view, all Private ECUs shall be regarded as if they were internally connected I/O units, reading sensors and controlling actuators, within the Public ECU master itself. The Private ECU master shall, on behalf of the private ECUs, support all the diagnostic data required by this table and that are applicable for the private ECUs.			

Power supply	7	Actual voltage read as an analog input signal by the ECU (i.e. the sampled raw signal), but after necessary HW or/and SW filtering of “normal” disturbances.	M	O	1, 6
Other external signals	8	Specified by the implementer.	MC1	MC1	5, 6

Abbreviations in the table above:
MC1 = A feasibility study is required. The data record(s) shall be implemented if the study shows that a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
MC2 = Mandatory if the ECU is responsible for maintaining and distributing the network signal.
Notes in the table above:
1. The signal read by the diagnostic services shall be updated minimum each 100 ms.
2. By examining the raw signal it shall be possible to determine if the signal is correct or if a short to battery, short to ground or open circuit is present.
3. By examining the value read it will normally be possible to decide whether the circuit is correct or if a short to battery voltage, short to ground or open circuit fault is present. If this is not possible then the feasibility study required by item 3.2 of the table shall be performed to decide if the decision can be made by examining the value of the raw version of the signal instead.
4. If the network signal is normally sent periodically by the ECU then this must continue during diagnostic control of the signal.
5. It shall be possible to control all outputs individually and independently of each other unless there are special safety reasons not to do this. If any output is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM
6. The ECU shall report the tester substituted input value to a diagnostic request to read the signal value while the input parameter value is under tester control.
7. Whether or not the session (in which the data record shall be supported) is all other sessions, or just one or some sessions, depends on which sessions the diagnostic services for read/write/control of the data is supported in.

Requirements from this section are specified in section "Requirements from section ECU external signals".

4.5.1.1.4.1.3 ECU internal signals

This section is not applicable for programming session.

All internal signals (i.e. the signals on all internal circuits connected to the processor of the ECU) shall be defined in data records within the ECU, according to the table below in all diagnostic sessions except the programming session.

It shall be possible to read and control these data records, according to the table, by the diagnostic services specified in *[Data_1] UDS Services*.
Table: Requirements for definition of ECU internal signals in data records

Type of signal	Item	Signals to be defined in data records by the implementer	Diagnostic service, see note 4		Notes
			Read	I/O Control	
Input from internal HMI (e.g., push button)	1.1	Actual digital input signal read by the ECU (i.e., the sampled raw signal), but after necessary HW and/or SW filtering of “normal” disturbances.	M	O	1, 3
	1.2	Value currently available for the ECU´s application software (e.g., a filtered value used by the ECU in normal operation or the replacement value if the ECU is in limp-home operation).	MC	O	1
Output to internal HMI (e.g., LED display)	2	Output signal (the desired set point) controlled by the ECU´s application software or by the ECU´s diagnostic software (from the tester via a diagnostic service).	M	M	2
Other internal signals (e.g., from internal sensors or to internal actuators)	3	Specified by the implementer.	MC	MC	3

Abbreviations in the table above:
MC = A feasibility study is required. This shall be implemented if a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study which shall be approved by the responsible authority appointed by OEM.
Notes in the table above:
1. The signal read by the diagnostic services shall be updated minimum each 100 ms.
2. It shall be possible to control all outputs individually and independently of each other unless there are special safety reasons not to do this. If any output is prohibited to be controlled due to such safety reason, this must be approved by the responsible authority appointed by OEM.
3. The ECU shall report the tester substituted input value to a diagnostic request to read the signal value while the input parameter value is under tester control.
4. Whether or not the session (in which the data record shall be supported) is all other sessions, or just one or some sessions, depends on which sessions the diagnostic services for read/write/control of the data is supported in.

Requirements from this section are specified in section "Requirements from section ECU internal signals"

4.5.1.1.4.1.4 ECU_Vehicle status

Information about ECU/Vehicle status shall be defined in data records within the ECU, according to the table below.
It shall be possible to read the value of the data records by diagnostic services specified in reference *[Data_1] UDS Services* , in different diagnostic sessions as defined in the table.
Table: Requirements for definition of ECU/Vehicle status in data records

Data record	Notes	Session

Identifier, hex	Name	Description		Other ,see note 1	Program-ming
C010	ECU List	List of all ECUs installed in the vehicle		MC1	D
C011	BuildLlist	The P-spec for the vehicle		MC1	D
See note 6	High quiescent current fault identification data	See note 6		See note 6	See note 6
D134	Car Mode	Currently active Car Mode	2	MC2	D
DED0	Customer setting parameters	Data record intended for customer setting parameters. The data record shall only consist of record data that contain information of customer settings stored by the ECU. The data record is used for restoring customer settings in the ECU replacement process at a workshop. A customer setting is defined as: • A function that is open for alteration by the customer and/or tester tool it will be possible to change and adapt behavior of the functionality in the car to suit the customer. • A long term alteration that only is initiated by the customer himself. The ECU is not allowed to change the customer setting parameter. Short term adjustments that, due to safety or other reasons, only are valid in one driving cycle or less and will revert to a predefined value are not to be considered as customer setting parameter. A setting that can be done by the customer himself and by the tester tool or only by a tester tool (at workshop or remote).	5	MC5	D
DED1	System adaption data	Data records intended for adaption parameters. The data record shall only contain information of adaption data stored by the ECU. The data record is used for restoring adaption data in the ECU replacement process in workshop. Adaption parameters are adjusted to the customers usage/driving and/or components performance/life-span in order to optimize the performance and drivability in the vehicle. The data record shall be used where it is considered suitable that adaption values are transferred from an old to a new ECU in the ECU replacement process at a workshop.	5	MC6	D
DD0A	Usage Mode	Currently active Usage Mode	2	M	D
DD0B	Partial Network Cluster (PNC)	Data record defined by *[Data_4] Global Master Reference Database.* Note that the record data is a part of the network management user data and consists of a bit array with one bit for each PNC which indicate whether or not the PNC shall currently be active.		MC4	D

E103	Car Configuration Parameter Faults	List of invalid car configuration parameter values identified by the ECU.		MC7	D
EA00	RVDC Supported Data Records	The data record shall be used by the Remote Vehicle Data Collection (RVDC) functionality. The data record shall consist of a list of other data records that are supported by the ECU.	8	See note 7	See note 7
EDC0 up to EDC3	Diagnostic Read out #1 up to Diagnostic Read out #4	Data record(s) intended for quality tracking of the ECU is defined by the implementer. The data record(s) shall only consist of other record data than the record data read by the generic standard read out sequence.	3	MC3	D
EDC5	Vehicle state of health	Data record intended for remote diagnostics is defined by the implementer. The data record shall only consist of record data that the workshop preplanning process need to supplement DTCs and DTC related data in order to identify the root cause to customer complaints.	3	MC3	D
F186	Active diagnostic session	Currently active diagnostic session		M	M

Abbreviations in the table above :

MC1 = Mandatory for ECU that is the Car Configuration Domain Master according to *[Data_5] Central Car Configuration*.

MC2 = Mandatory for ECUs that are publisher or subscriber of the car mode signal. Refer to *[Data_3] Vehicle Mode Management - Concept SRD* for details regarding car mode signal.

MC3 = Mandatory if required by the responsible authority appointed by OEM.

MC4 = Mandatory if the ECU is the Vehicle Mode Management (VMM) master.

MC5 = Mandatory if the ECU stores customer settings that is expected to be rewritten to a new ECU in the ECU replacement process at a workshop.

MC6 = Mandatory if the ECU stores adaption parameters that is expected to be rewritten to a new ECU in the ECU replacement process at a workshop.

MC7 = Mandatory if the ECU support the DTC U230056 (0xE30056) - Invalid vehicle configuration data.

Notes in the table above:

1. Whether or not the session (in which the data record shall be supported) is all other sessions, or just one or some sessions, depends on which sessions the diagnostic services for read/write/control of the data is supported in. See also note 2 below.
2. Whether or not it shall be possible to control or write the value of the data record 0xD134, by diagnostic services specified in *[Data_1] UDS Services*, is specified in *[Data_3] Vehicle Mode Management - Concept SRD.* It shall be possible to control the value of the data record 0xDD0A in the ECU that implements the Usage Mode Manager, by diagnostic services specified in *[Data_1] UDS Services*.
3. The data record (called "composite" data record) shall contain several data items. Each data item shall consist of a data identifier followed by the record data that is identified by the data identifier.
4. Not in use (reserved for future use) .
5. It shall be possible to write the value by diagnostic service specified in *[Data_1] UDS Services.*
6. See reference *[Data_9] FltEgyCns*.

7. See reference *RVDCClient*, UDS DATA documents.
8. The content in the data record shall be the following:

Byte	Description
#1-2	First data record identifier supported by the ECU
#3-4	Second data record identifier supported by the ECU
#5-6	Third data record identifier supported by the ECU
.	
.	
#N-(N+1)	N:th data record identifier supported by the ECU

If a data record in the list either is not supported for a specific configuration of the ECU or for some other reason does not have a correct or avaliable content, the value of the record identifier shall be replaced with 0x0000.

Requirements from this section are specified in section "Requirements from section ECU_Vehicle status".

4.5.1.1.4.1.5 Vehicle network data

Network specific information shall be defined in data records within the ECU according to requirements defined in the implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP).

4.5.1.1.4.1.6 ECU_Vehicle identification

Information for identification of the ECU and the vehicle shall be defined in data records within the ECU, according to Table 4.5 .

Unless otherwise defined in the table, it shall be possible to read the value of the data records, by diagnostic services specified in reference [1], in different diagnostic sessions as defined in the table.
Unless otherwise defined in the table, it shall not be possible to change the value of the data records by diagnostic services.

Table: Requirements for definition of ECU and vehicle identification in data records

Data record, see note 1				Notes	Session	
Identifier used by VCC, hex	Identifier used by Geely, hex	Name	Description		Other , see note 12	Program-ming
F102	Same as VCC	Security Constant	Security constant according to reference *[Data_7] Security access algorithm* used to calculate the key that is sent in programming session by diagnostic service specified in reference *[Data_1] UDS Services*. It shall not be possible to read the value of the data record by any diagnostic service defined in reference [1] or any kind of external debug interface that may be used without destroying the sealing of the ECU in any diagnostic session.	11	N	OS
F106	Same as VCC	Vehicle Configuration Parameters	See reference *[Data_4] Global Master Reference Database*.		MC1	D

F114	Same as VCC	Vehicle Information Section	See reference *[Data_4] Global Master Reference Database, UDS DATA, documents*.		MC1	D
F18C	Same as VCC	ECU Serial Number	A supplier unique ECU serial number per ECU Core Assembly Part Number.	7, 11	M	MPS
F190	Same as VCC	Vehicle Identification Number	The standard 17 character VIN as written on the vehicle identification plate. See reference *[Data_4] Global Master Reference Database*. Note that the requirement to store and report Vehicle Identification Number is usually dependent upon legislated regulations and/or on the vehicle security (theft deterrence) policy.		MC2	D
F120	F1A0	Application Diagnostic Database Part Number	Part number for diagnostic database used by ECU application SW.	1, 20	M	D
F121	F1A1	Primary Bootloader Diagnostic Database Part Number	Part number for diagnostic database used by ECU primary bootloader SW	1, 11, 20	O	MPS
F122	F1A2	Secondary Bootloader Diagnostic Database Part Number	Part number for diagnostic database used by ECU secondary bootloader SW.	1, 20	N	MS
F124	Same as VCC	Secondary Bootloader Software Version Number	Software Version Number for the Secondary Bootloader SW The Secondary Bootloader Software Version Number shall contain version information values specified by the supplier which allow a tool to uniquely identify the secondary bootloader software resident on the ECU.	16	N	MS
F125	F1A5	Primary Bootloader Software Part Number	Part number for the Primary Bootloader SW	1, 11, 20	O	MPS
F126	Same as VCC	Autosar BSW cluster versions	VCC Autosar Basic Software cluster versions. See reference *[Data_4] Global Master Reference Database*.		MC4	D
F12A	F1AA	ECU Core Assembly Part Number	A number that identifies the combination of the ECU hardware (pc circuit board micro-controller and memory) and any non-replaceable software (bootloaders and other fixed software). The ECU core assembly part number will NOT represent: 1) Any variable aspect of the ECU that is not associated with the circuit board e.g. its color or associated internal actuators or mechatronic parts so long as these parts do not affect the compatibility of any downloadable component	1, 11, 20	M	MPS

			2) Any downloadable software component regardless of whether or not this component is pre-loaded into the ECU. For ECUs not including any items in 1) or 2) above the core assembly number will fully define the part. Note that the ECU core assembly part number will provide sufficient information to fully define the compatibility but not necessarily the correct vehicle applicability of any downloadable software component.			
F12B	F1AB	ECU Delivery Assembly Part Number	A number that identifies the complete ECU at the point of delivery to the assembly plant or the service bay. The number shall not be altered or updated after initial delivery of the ECU. ECU Delivery Assembly Number shall represent the ECU Core Assembly Number 0xF12A (VCC) or 0xF1AA (Geely) and additionally: 1) Any variable aspect of the ECU e.g. its color, associated internal actuators or mechatronic parts. 2) All downloadable software components that are pre-loaded into the ECU. Note that any software download process will alter the ECU software content such that the content referred to by this part number will be incorrect. Therefore the number is only valid at the point of ECU delivery.	1, 11, 20	M	MPS
F12C	F1AC	ECU Software Structure Part Number	Software Part Number for the ECU Software Structure, which is used by ECUs supporting Software Authentication concept where each software part is signed individually.		N	MC7
F12E	F1AE	ECU Software Part Numbers	Complete set of part numbers for the ECU software. The Part Number(s) is only allowed to be used for item(s) that can be separately downloaded to the ECU according to reference *[Data_6] General Software Download Specification*.	1, 2, 20	M	D
F12F	F1AF	ECU Software Extended Part Numbers	Set of part number(s) for the ECU software. The Part Number(s) is only allowed to be used for item(s) that can not be separately downloaded to the ECU according to reference *[Data_6] General Software Download Specification*.	1, 2, 20	MC6	D

F13F	Same as VCC	Private ECU(s) / components Serial Number(s)	A supplier unique Serial number per Private ECU Core Assembly Part Number for a Private ECU or component for which the ECU is Public ECU master.	3, 4, 13, 14	MC3	D
D01C	Same as VCC	Public Key	Public Key used by ECUs supporting Software Authentication concept, where only the key itself (not certificates) is used for authenticity verification of the software		N	OS
EDA0	ED20	Complete ECU Part/Serial Number(s)	Complete set of ECU part/serial numbers.	9, 20	M	MPS

Abbreviations in Table 4.5:
MC1 = Mandatory for Car Configuration Domain Master (according to reference [Data_5] *Central Car Configuration – SPA, UDS DATA, documents*) on networks with ECU configuration controlled by an on-board master configuration file.
MC2 = Mandatory if the ECU has a requirement to store and report VIN.
MC3 = Mandatory if the following conditions are fulfilled:

- The ECU is a Public ECU master for Private ECU(s) or component(s) which has additional documentation requirements.
- The Public ECU master does not support gateway functionality. Thus serial numbers can not be read directly from the Private ECU(s) or component(s) by a tester.

MC4 = Mandatory if the ECU contains any AUTOSAR BSW cluster.
MC5 = Not used
MC6 = Mandatory if required by the responsible authority appointed by OEM. Otherwise optional.
MC7 = Mandatory for ECUs supporting the Software Authentication concept where each data file is signed and verified individually.

Notes in Table 4.5:
1. Data records used by VCC: 8 digits for part number + 3 characters (version suffix).
Data records used by Geely: 10 digits for part number + 3 characters (version suffix).
The part number digits shall be coded in BCD and the 3 characters (version suffix) shall be coded in ASCII, right justified with any unused digit filled with 0 in total 7 bytes (VCC) or in total 8 bytes (Geely).
If version suffix contains one character, there shall be two spaces between part number and version suffix.
If version suffix contains two characters, there shall be one space between part number and version suffix.
Example (applicable to data records with identifier used by VCC):
1234567<space><space>A -> (hex) 01 23 45 67 20 20 41
12345678<space><space>A-> (hex) 12 34 56 78 20 20 41

1234567<space>AB -> (hex) 01 23 45 67 20 41 42
12345678<space>AB -> (hex) 12 34 56 78 20 41 42

1234567ABC -> (hex) 01 23 45 67 41 42 43
12345678ABC -> (hex) 12 34 56 78 41 42 43
2.The content of the dataRecord with the dataIdentifier used by VCC shall be the following:

Byte	Description
#1	Total number of ECU Software Part Numbers
#2 - 8	ECU Software #1 Part Number
#9 - 15	ECU Software #2 Part Number
.	.
.	.
#N - (N+6)	ECU Software #X Part Number

The content of the dataRecord with the dataIdentifier used by Geely shall be the following:

Byte	Description
#1	Total number of ECU Software Part Numbers
#2 - 9	ECU Software #1 Part Number
#10 - 17	ECU Software #2 Part Number
. .	
. .	
#N - (N+7)	ECU Software #X Part Number

3. The content shall only consist of serial number(s) for Private ECU(s) or component(s) that are subject to additional documentation requirements. The decision if a Private ECU or component are required to be subject for additional documentation is made by responsible authority at OEM.

4. The serial number shall be maximum 12 digits long. All digits shall be coded as alphanumeric characters. The serial number shall be right justified with any unused digits filled with 0, in total 12 bytes.

The content of the dataRecord with the dataIdentifier used by VCC shall be the following:

Byte	Description
#1	Total number of Private ECU(s) / component(s) Serial Numbers
#2 -13	Private ECU / component #1 Serial Number
#14 - 26	Private ECU / component #2 Serial Number
. .	
. .	
#N - (N+12)	Private ECU / component #X Serial Number

5. Not in use (reserved for future use)

6. Not in use (reserved for future use)

7.8 digits for serial number. All coded in BCD, right justified with any unused digit filled with 0, in total 4 bytes

Example:

12345678 -> (hex) 12 34 56 78

123456 -> (hex) 00 12 34 56

8. Not in use (reserved for future use)

9. The data record (called "composite" data record) shall contain several data items in an arbitrary order. Each data item shall consist of a data identifier followed by the record data that is identified by the data identifier.

The "composite" data record with identifier 0xEDA0 shall contain the following data items, (the data items are depending of the currently executing software):

Identifier	Description
F120 or F121 or F122,	Application- or PBL- or SBL Diagnostic Database Part Number
F12A,	ECU Core Assembly Part Number
F12B,	ECU Delivery Assembly Part Number
F18C,	ECU Serial Number
F12E or F125 or F124,	ECU- or PBL Software Part Number(s) or SBL Software Version Number.

F13F Private ECU(s) / components Serial Number(s)
The "composite" data record with identifier 0x ED20 shall contain the following data items (the data items is depending of the currently active executing software):
Identifier Description
F1A0 or F1A1 or F1A2, Application- or PBL- or SBL Diagnostic Database Part number
F1AA, ECU Core Assembly Part Number
F1AB, ECU Delivery Assembly Part Number
F18C, ECU Serial Number
F1AE or F1A5 or F124, ECU- or PBL Software Part Number(s) or SBL Software Version Number
F13F Private ECU(s) / components Serial Number(s)
10. Not in use (reserved for future use)
11. Change of the value of data record by diagnostic service defined in reference [1] is optional only when the ECU is executing the secondary bootloader - programming session and only according to the following:
The value is allowed to be changed only one single time. Once the value has been changed, it shall not be possible to change the value again
12. The session in which the data record shall be supported (e.g., all other sessions, or just one or some sessions) depends on which sessions the diagnostic services for read/write/control of the data is supported in.
13. If a private ECU, that supposed to be present according to the ECU master, fails to answer when the public ECU master collect private ECUs part & serial numbers, this shall be shown by filling all bytes in the missing number with 0xFF.
14. If the configuration of a vehicle excludes a private ECU / component (i.e certain private ECU / component are not mounted in the vehicle) this shall be shown by having all bytes in the serial number for that specific private ECU filled with 0x00.
15. Not in use (reserved for future use).16. The Secondaty Bootloader Software Version Number shall be BCD encoded, right justified, all unused digit shall be filled with 0 in total 7 bytes.
17. Not in use (reserved for future use).
18. Not in use (reserved for future use).
19. Not in use (reserved for future use).
20. All ECU:s shall implement both Geely and VCC format DID:s independently of what brand they are planned to be used in.

Requirements from this section are specified in section "Requirements from section ECU_Vehicle identification".

4.5.1.1.4.1.7 DTC information

The requirements in this section are not applicable for programming session.

4.5.1.1.4.1.7.1 General

DTCs shall be supported by the ECU according to the table below (e.g. within the valid display value ranges).
The implementation must use DTCs exactly as they are defined in *[Data_4] Global Master Reference Database*.

Table: Requirements on implementation of DTCs

DTC	M/O, see

				note 1
Name	**Item**	**Valid display value range See [Data_51] Diagnostic trouble code definitions , UDS - Data - External publications .**	**Definition**	
Powertrain (P) DTCs	1.1	P0000YY to P0FFFYY P2000YY to P2FFFYY P3400YY to P3FFFYY	DTCs defined in *[Data_51] Diagnostic trouble code definitions* , and *[Data_4] Global Master Reference Database .* Byte YY is the DTCFailureTypeByte specified by *Diagnostic trouble code definitions .* The requirements on detection of the faults (emission related) that shall be identified by DTCs in this range are not specified by this document. The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*, and be reported not more than once in any response.	MC
	1.2	P1000YY to P1FFFYY P3000YY to P33FFYY	DTCs defined in *[Data_4] Global Master Reference Database* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions .* The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Detection of faults.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
Body (B), Chassis (C), Network and Miscellaneous (U) DTCs	2.1	B0000YY to B0FFFYY B3000YY to B3FFFYY C0000YY to C0FFFYY C3000YY to C3FFFYY U0000YY to U0FFFYY U3000YY to U3FFFYY	DTCs defined in *[Data_51] Diagnostic trouble code definitions ,* and *[Data_4] Global Master Reference Database* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions.* The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Detection of faults .* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.2	B1000YY to B2FFFYY C1000YY to C2FFFYY U1000YY to U28FFYY	DTCs defined in *[Data_4] Global Master Reference Database .* Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions.* The requirements on detection of the faults that shall be identified by DTCs in this range are specified in section *Detection of faults .* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.3	U2C00YY to U2CFFYY	DTCs defined in the *[Data_4] Global Master Reference Database .*	M

			Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions* , The faults that shall be identified by DTCs in this range are faults caused by signal or data value received from another ECU where the signal/data is indicated as invalid by the transmitting ECU. The requirements on detection of these faults are specified in section *Detection of faults caused by received signals.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	
	2.4	U2D00YY to U2DFFYY	DTCs defined in *[Data_4] Global Master Reference Database* . Byte YY is the DTCFailureTypeByte specified by *[Data_51] Diagnostic trouble code definitions* , . The faults that shall be identified by DTCs in this range are faults caused by signal or data value received from another ECU where the signal/data is not indicated as invalid by the transmitting ECU. The requirements on detection of these faults are specified in section *Detection of faults caused by received signals.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.5	U2E00YY to U2EFFYY	DTCs defined in *[Data_4] Global Master Reference Database* . Byte YY is the DTCFailureTypeByte specified by [Data_51] *Diagnostic trouble code definitions* . The faults that shall be identified by DTCs in this range are faults caused by the ECU operating environment or by customer actions The requirements on detection of these faults are specified in section *Detection of faults caused by the ECU environment.* The DTCs shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported not more than once in any response.	M
	2.6	U2F00YY to U2FFFYY	DTCs defined by the implementer. DTCs that are needed only during the development or quality tracking of the ECU shall be specified in this range. The requirements on detection of the faults that shall be identified by DTCs in this range are specified by the implementer and in section *Detection of faults caused by signals and data values received from other ECUs.* The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range.	O

			The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range. The implementer shall not expect that OEM tools support the DTCs within this range.	
	2.7	U2800YY to U2BFFYY	DTCs defined in *[Data_4] Global Master Reference Database*. DTCs that are needed only during the development or quality tracking of the ECU shall be specified in this range. Byte YY is the DTCFailureTypeByte specified by [Data_51] Diagnostic trouble code definitions. The requirements on detection of the faults that shall be identified by DTCs in this range are specified in the following sections. The FDC10 shall increase with the value 1 when the test sample failed criteria is met for the DTCs in this range. The maximum value of the FDC10 shall be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) for the DTCs in this range. The implementer shall not expect that OEM tools support the DTCs within this range.	O

Abbreviations in the table above:
MC = Mandatory for emission related ECUs. Otherwise optional
Notes in the table above:
1. Note that “M” means that the ECU shall support DTCs according to column “Definition” of the table which shall be defined within the DTC range according to column “valid display value range”. Note that “O” means that the ECU may support DTCs according to column “Definition” of the table which shall be defined within the DTC range according to column “valid display value range

Requirements from this section are specified in section "Requirements from section DTC information - General".

4.5.1.1.4.1.7.2 Detection of faults

The ECU shall be able to detect faults, and the detected faults shall be identified by DTCs, according to the following sub-sections, including all tables within these sub-sections.

All tables in the sub-sections uses a unified format, including eight columns of information. The information in each of the columns, and general requirements applicable to all tables, are described below:

Type of item:
General description of the type of item that shall be monitored by the DTC test.
Depending on the subject of the specific table, this can be a description of the type of component, circuit, system, data, ECU etc., that shall be monitored.

Item no.:
A sequential number for each separate item, used for references.

Detected faults/errors:
Definition of the fault, or error, that shall be possible to detect by the DTC test.
The detailed criteria for increasing/decreasing the DTC fault detection counter (see section *DTC fault detection counters*) shall be specified by the implementer, unless it is specified in the table.

Test Period
If a value is included in this column, the DTC test shall produce Test Samples, i.e. the DTC test fault detection logic shall execute and the DTC fault detection counter shall be updated (see section *DTC fault detection counters* for details regarding update of the DTC fault detection counter), periodically at this or a higher rate all the time while the Test Run Criteria are met (see column Test Run Criteria below) unless otherwise is specified by the table.
The maximum period time between Test Samples shall be as specified by the table. However, in cases where the ECU application software uses a signal monitored by the DTC test more frequently than this specified maximum, then the period time shall be reduced to match that used by the application software.
If no value is included in this column, the DTC test shall run one single time when the Test Run Criteria are met.

Test Run Criteria:
The Test Run Criteria defines the criteria that must be met for the DTC test to run.
The Test Run Criteria can be defined both as Enable Criteria, e.g. "While output is activated", and as Inhibit Criteria, e.g. "Not during engine cranking ". The DTC test shall run when all of the Enable Criteria are met and shall be inhibited if any of the Enable Criteria are not met, or if any of the Inhibit Criteria are met (Inhibit Criteria has priority over Enable Criteria, i.e. if at least one Inhibit Criteria is fulfilled while all of the Enable Criteria are met the DTC test shall be inhibited).
If the test is inhibited it shall, as a minimum, stop updating the DTC Fault Detection Counter (FDC10, see section *DTC fault detection counters*). If the actual test can be stopped or not, or if it needs to run for other reasons, e.g. safety related or other application requirements, this shall be defined by the implementer. Note that if the actual test is not stopped and the ECU can take any action due to faults detected by the DTC test (e.g. reduce or shut off its functionality or request a warring indicator) that for sure can cause customer symptom the updating of the FDC10 should not be stopped (i.e. the test is not inhibited in this case).
If the DTC test shall run one single time when the Test Run Criteria are met (no value is included in the "Test Period Time" column) the Test Run Criteria shall include an event description which will be a trigger condition for the test to run.
The implementer shall specify if any partial DTC test results, obtained prior to the DTC test being inhibited by a Test Run Criteria, shall be used when the DTC test resumes execution. Note, however, that the value of the DTC Fault Detection Counter shall be maintained between the DTC test runs (see requirement specified in table "Requirements for the implementation DTC fault detection counters", number 10 found in section *DTC fault detection counters*).

Generic Test Run Criteria:
The following Test Run Criteria are applicable for every DTC test and are therefore not specified in the tables:
Enable criteria.
DTC tests shall be enabled:
· All the time during an operation cycle.

If the ECU performs any action due to fault detected by the DTC test (e.g. in order to mitigate the effect of the fault on customer functions or system safety), the DTC tests shall not be inhibited after the ECU has performed the action (e.g. has reduced or turned off the customer functions that are affected by the fault). However, if this is not appropriate and if approved by the responsible authority appointed by OEM, the DTC test may be inhibited in this case.
The DTC tests shall not be inhibited (while the ECU is in operation) when the functionality, which can be affected by fault detected by the DTC test, is not active (e.g. is disabled when the functionalty is not used). However, if this is not appropriate and if approved by the responsible authority appointed by OEM, the DTC test may be inhibited in this case.
Inhibit criteria.
DTC tests shall be inhibited:
· When electrical power availability is limited, as indicated by the ElPowerLevel 1. (Only applicable for ECUs which are power supplied from the 12 V system for which the ElPowerLevel signal is applicable. See *[Data_3] Vehicle Mode Management - Concept SRD*, for definition of the ElPowerLevel signal).
Exception: A test shall not be inhibited during engine cranking if the test detects faults that can occur during cranking and that can prevent the transition to engine running. Other exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM.
· If not otherwise is specified, during 5 seconds from any transition from one usage mode to another. The inhibition shall start at notational consumption of the 'usage mode' signal when a change of usage mode is requested and shall be active for 5 s ± 2s. This criteria applies regardless of how the usage mode is changed, e.g. by user request or by diagnostic service. See *[Data_3] Vehicle Mode Management - Concept SRD*, for definition of usage modes. Exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM.
· During 5 seconds from any transition from one car mode to another. The inhibition shall start at notational consumption of the 'car mode' signal when a change of car mode is requested and shall be active for 5 s ± 2s. See *[Data_3] Vehicle Mode Management - Concept SRD*, for definition of car modes. Exception is alllowed (e.g. due to safety reasons or legel requirements on emissionrelated faults) if the DTC test is able to detect faults in an accurate and reliable manner without use of these test run criteria but possibly other test run criteria, and if approved by the responsible authority appointed by OEM
· When voltage supplied to the ECU is lower than required for ensuring correct fault monitoring. The low voltage limit shall be defined by the implementer but shall not be higher than the lower limit of the supply voltage range specified for the function(s) which includes the monitored component/system, unless otherwise defined in this document.
· In car modes Factory, Transport and Crash. See *[Data_3] Vehicle Mode Management - Concept SRD*, for definition of car modes. Note that DTC tests that can detect faults that are only related to dynamometer testing environment shall be inhibited in the car mode Dyno. Refer to *[Data_3] Vehicle Mode Management - Concept SRD*, regarding requirements on this.
Specific Test Run Criteria:
Test Run Criteria applicable for specific DTC tests are defined in the tables in the sub-sections or shall be defined by implementer if the table's explictly specify that.
In addition to the Test Run Criteria defined in the tables the implementer may define addtional Test Run Criteria for specific DTC tests if approved by the responsible authority appointed by

OEM.

Failure type:
Failure type is the same as the DTCFailureTypeByte value specified in *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications, unless otherwise specified in the table. The values in the tables are hexadecimal.
A failure type value of 0x00 may only be used if the Base DTC description includes the failure type and is approved in *[Data_4] Global Master Reference Database*. In such cases the Base DTC may not be combined with any other failure type values (i.e. a separate Base DTC must be used for failure type value of 0x00).

Base DTC:
Display value of the base DTC (DTC High Byte and DTC Middle Byte, i.e. excluding the FailureTypeByte) that shall be used to identify the faulty item. If a specific value is not specified by the table, the value used shall be exactly as defined in *[Data_4] Global Master Reference Database*.
Each fault shall be identified by a unique DTC (incl. the FailureTypeByte). Grouping of faults to use one single common DTC is not allowed unless explicitly approved by the responsible authority appointed by OEM, or where the tables within this specification already shows the use of the same DTC. Where faults are combined, all of the associated stored DTC information (e.g. the fault detection counter) is also combined and no more than one instance of the DTC shall be reported in any response to the services specified in *[Data_1] UDS Services*. Where the implementer identifies a requirement for separate repair action or fault confirmation process for DTCs that are shown with the same DTC then the responsible authority appointed by OEM may permit the use of different DTC.

M/O:
Defines if the specified fault detection is mandatory (M) or optional (O) to be implemented in the ECU.

Requirements from this section are specified in section "Requirements from section Detection of faults".

4.5.1.1.4.1.7.2.1 Detection of faults in external electrical circuits

The ECU shall be able to detect faults in all external electrical circuits connected to the ECU. This includes both the parts of the circuits that are external to the ECU (e.g., sensors, actuators, cables and connectors) and the parts that are internal to the ECU (e.g., input filters, input protection and output drivers).
The ECU shall be able to detect open circuit, short to ground, short to battery voltage and other faults in these circuits as specified in the table below

Table: Requirements on detection of faults in external electrical circuits connected to the ECU

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Switch input	1.1	'Stuck at' fault (e.g., activation too long, button with spring return activated for	500 ms		23 or 24	One for each input circuit.	MC1

		longer than 1 minute).					
	1.2	Other faults specified by the implementer (e.g., impossible value of the switch input when another input signal has a certain value).	100 ms See note 1	TRC1	See note 2		MC1
Analog input	2.1	Short to battery voltage or open circuit.	100 ms		12 and 13 or 15	One for each input circuit.	M
	2.2	Short circuit to ground.	100 ms		11		M
	2.3	Other faults specified by the implementer (e.g., too slow or too fast rate of change for a signal, value out of range, implausible combination of signals etc.).	100 ms See note 1	TRC1	See note 2		MC1
FM, PCM, PWM or similar input, alternative 1	3.1	Short circuit to battery voltage or open circuit (low time>maximum or high time>maximum).	100 ms		12 and 13 or 33 or 34	One for each input circuit.	M
	3.2	Short circuit to ground.	100 ms		11		M
	3.3	Other faults specified by the implementer (for examples see the analog input).	100 ms See note 1	TRC1	See note 2		MC1
FM, PCM, PWM or similar input, alternat-ive 2	4.1	Short circuit to battery voltage.	100 ms		12	One for each input circuit.	M
	4.2	Short circuit to ground or open circuit (low time>maximum or high time>maximum).	100 ms		11 and 13 or 33 or 34		M
	4.3	Other faults specified by the implementer (for examples see the analogue input).	100 ms See note 1	TRC1	See note 2		MC1
Output with low side driver	5.1	Short circuit to battery voltage.	100 ms	TRC2	12	One for each output circuit.	M
	5.2	Short circuit to ground or open circuit.	100 ms	TRC3	11 and 13 or 14		M
	5.3	Other faults specified by the implementer (e.g. over-temperature of the low side driver, etc.).	100 ms See note 1	TRC1	See note 2		MC1
Output with	6.1	Short circuit to battery voltage or open circuit.	100 ms	TRC3	12 and 13 or 15	One for each output circuit.	M

high side driver							
	6.2	Short circuit to ground.	100 ms	TRC2	11		M
	6.3	Other faults specified by the implementer (for examples see the low side driver).	100 ms See note 1	TRC1	See note 2		MC1
Public ECU network interface	7	Refer to implementation specification for the respective network (e.g. UDSonCAN, UDSonFR, UDSonIP, UDS onMOST and UDSonLIN). Note that the implementation specification för LIN (UDSonLIN) and CAN (UDSonCAN) specifies a LIN and CAN respectively there an ECU can be regarded as partly public and partly private (some diagnostic services supported by the ECU can be addressed to the ECU while other can not).					
External electrical circuits connected to private ECUs (other than item 9 below)	8	Note that from a diagnostic point of view, all Private ECUs shall be regarded as if they were internally connected I/O units, reading sensors and controlling actuators, within the Public ECU master itself. The Private ECU master shall, on behalf of the private ECUs, support all the diagnostic data required by this table and that are applicable for the private ECUs. The fault detection may, however, be performed by the Private ECUs with the results being sent back to Public ECU master for processing and storage of DTC information.					
Private ECU network interface (except LIN and CAN, see item 7 above)	9.1	Transmitted message contain data that does not correspond to data read back by the ECU.		Each time a message is sent.	86	One for each network.	MC1, MC3
	9.2	The line is not free when no data transmission is expected.	100 ms. See note 1		81		MC1, MC3
	9.3	CS- or CRC-error on received message.		Each time a message is received	83		MC1, MC3
	9.4	Incorrect timing and content of bits, bytes or messages. Incorrect structure and format of bytes or messages.		Each time a message is received	86		MC1, MC3
	9.5	Received data has not been updated by the transmitting ECU.		Each time a message is received	82		MC1, MC3
	9.6	Periodic message is not received within maximum period.		Each time a message is expected to be received.	87		MC1, MC3

	9.7	The physical layer of the serial communication link has a short to ground, short to battery voltage or open circuit.	100 ms. See note 1		01		MC1, MC3
	9.8	Other faults identified by the implementer.	100 ms. See note 1	TRC1	See note 2		MC1, MC3
Power supply	10.1	Low supply voltage at the ECU. The ECU checks if its supply voltage is lower than the measured vehicle battery voltage received from the network. For additional requirements see section *Requirements for detection of low supply voltage*	See section *Requirements for detection of low supply voltage*	See section *Requirements for detection of low supply voltage*	62	U3003	M
Other external electrical circuit.	11.1	Faults identified by the implementer.	100 ms See note 1	TRC1 and additional conditions specified by the implementer	One for each fault. See note 2	One for each circuit.	MC1

Abbreviations in Table 4.7
TRC1 = Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.
TRC2 = At least while output is activated.
TRC3 = At least while output is deactivated.
MC1 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
MC2 = N/A
MC3 = Mandatory if the ECU is a Public ECU master and fulfills the conditions of MC1.

Notes in the table above
1. The defined Test Period Time is the general requirement for this type of fault. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority appointed by OEM.
2. See *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications

Requirements from this section are specified in section "Requirements from section Detection of faults in external electrical circuits".

4.5.1.1.4.1.7.2.2 Detection of low supply voltage

In each ECU supporting DTC U3003-62 (see table *Requirements on detection of faults in external electrical circuits connected to the ECU*) the internal ECU supply voltage, Uecu, shall be measured and compared with the vehicle battery voltage, Ubat, received from the network. A robust strategy for measurement and filtering of the voltages shall be implemented. The actual method shall be designed by the implementer and shall comply with the following requirements.

Vehicle battery voltage:

The vehicle battery voltage Ubat shall be measured, by a dedicated ECU, as close to the actual battery terminals as possible (such that the measurement is unaffected by any internal energy storage in the measuring ECU). The Ubat shall be measured with a minimum accuracy of ±0.25 V and shall be sampled every 50 ms or faster.
A new Ubat value shall be calculated every time a sample is taken. It shall be calculated as the average value of the samples taken during the last 500 ms period.
The calculated Ubat values shall be sent periodically and received by all Public ECUs with maximum end-to-end time (including sensing, signaling and actuating time) of 100 ms.

ECU supply voltage:

The ECU supply voltage Uecu shall be measured with a minimum accuracy of ±0.25 V and shall be sampled every 50 ms or faster.
A new Uecu value shall be calculated every time a sample is taken. It shall be calculated as the average value of the samples taken during the last 500 ms period.
Low supply voltage fault detection shall only be done when all of the following conditions are fulfilled:

(Ubat ≥ 12 V) AND (UsageMode == Driving)

The calculated Uecu value shall be compared with the received Ubat value periodically each 500 ms:

- if Uecu < (Ubat - 3V) then the DTC fault detection counter (FDC10) shall be incremented by 3.
- if Uecu >= (Ubat - 3V) then the DTC fault detection counter (FDC10) shall be decremented by 6.

The DTC shall have unconfirmedDTCLimit = 6.
The DTC shall have confirmedDTCLimit = 10.
The DTC shall have agedDTCLimit = 255.

Requirements from this section are specified in section " *Requirements for detection of low ECU supply voltage*".

4.5.1.1.4.1.7.2.3 Detection of faults in internal electrical circuits

Where an ECU, or a non repairable (by the workshop) assembly unit containing the ECU, circuitry (actuators, sensors etc) internal to the ECU but not located on the main circuit board, the ECU shall be able to detect faults in these internal electrical circuits according to section

Detection of faults in external electrical circuits as if they were external electrical circuits.

The ECU shall be able to detect faults in circuits and components located on the main circuit board of the ECU according to the table below where these have not been already covered by the requirements above.

Table: Requirements on detection of faults in internal electrical circuits

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure Type (FTB)	Base DTC (display value, excl. FTB)	M/O
Memory devices	1.1	Data integrity fault in NVM (Non Volatile Memory e.g. ROM, FLASH used for storing executable code, calibration, configuration or adaptation data etc.). The specific data integrity verification methods used by the ECU are specified by the implementer. See note 2.	Specified by the implementer	Specified by the implementer	45 or 46	One for all NVM	MC
	1.2	Data integrity fault in VM (Volatile Memory, RAM). The specific data integrity verification methods used by the ECU are specified by the implementer.	Specified by the implementer	Specified by the implementer	44	One for all VM	MC
Internal Communi-cation devices	2.1	Internal device communication error (e.g. communication watchdog timeout, data integrity check, etc.)		At each communi-cation with the device	49	One for each device	MC
Programme Executing units	3.1	Programme execution fault The specific data execution monitoring methods used by the ECU are specified by the implementer. Examples of execution monitoring methods: Overall execution time exceeded, e.g. watchdog Task sequence error, e.g. checking the execution path of tasks. Task run error, e.g. checking the number of tasks run within a defined time Task timeout, e.g. checking the time between executions of the same task	Specified by the implementer	Specified by the implementer	47	One for each execution unit	MC

Internally connected actuator	4.1	Faults identified by the implementer. When determining the requirement for internal actuator failures the implementer shall consider the possible faults of the internal actuator and its internal physical and electrical interface with the ECU hardware.	100 ms See note 1		One for each fault. See note 3	One for each actuator	MC
Internally connected sensor	5.1	Faults identified by the implementer. When determining the requirement for internal sensor failures the implementer shall consider the possible faults of the internal sensor and its internal physical and electrical interface with the ECU hardware.	100 ms See note 1		One for each fault. See note 3	One for each sensor	MC
Other internal parts	6.1	Faults specified by the implementer.	100 ms See note 1	TRC1 and additional conditions specified by the implemente r	One for each fault. See note 3	One for each internal part of the ECU that is monitored by a test (e.g. component , circuit, system or functiont)	MC

Abbreviations in the table above:
TRC1 = Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.
MC = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

Notes in the table above:
1. The defined Test Period Time is the general requirement. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority at OEM.
2. Integrity check of Global ECU Configuration data is defined in *Detection of faults due to configuration errors.*
3. See *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications.

Requirements from this section are specified in section "Requirements from section Detection of faults in internal electrical circuits".

4.5.1.1.4.1.7.2.4 Detection of faults due to configuration errors

An ECU shall be able to detect faults caused by errors in the configuration of the ECU, software incompatibility and hardware incompatibility, as specified in the tables found in this section .

Table: Requirements on detection of faults caused by local ECU Configuration (local parameters).

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Local Configuration data	1.1	Invalid configuration data. Compare value of configured item with allowed range of values for that item. The ECU has one or more configured parameters with unknown/invalid values. Refer to *[Data_8] Overview of vehicle configuration & customer* settnings for details regarding local configuration.		Whenever a configured parameter is used AND/OR whenever a configured parameter is changed	56	U2101	MC1

Table: Requirements on detection of faults caused by Global ECU Configuration (global parameters)

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
Global Configuration data	2.1	Missing vehicle configuration data See *[Data_5]Central Car Configuration*, DTC 'Car Configuration – Not Configured', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Enabled while ECU is in 'Bulk state'. Test runs each time a Car Configuration block is received. Test is inhibited once passed.	55	U2300	MC4
	2.2	Reserved for future use.					
	2.3	Invalid vehicle configuration data See *[Data_5] Central Car*		Every time a global configuration parameter used for	56	U2300	MC4

		Configuration, DTC 'Car Configuration – Invalid/Incompatible Configuration', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		configuration of the ECU is received until test passed.			
	2.4	New configuration not learned See *[Data_5] Central Car Configuration*, DTC 'Car Configuration – Not programmed', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Every time a global configuration parameter used for configuration of the ECU is received until test passed	51	U2300	MC5
	2.5	Inconsistent configuration data See table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Every time a global configuration parameter used for configuration of the ECU is received until test passed	57	U2300	MC2
Configuration data storage	3.1	The ECU has corrupt storage of configuration data. See *[Data_5] Central Car Configuration*, Req4, and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Specified by the implementer.	41	U2301	MC3
Car Configuration Logic	4.1	Incompatible Vehicle Configuration Data (VCD) and Car Configuration Logic (CCL) data loaded. See *[Data_5] Central Car Configuration* , DTC 'Car Configuration – VCP Not sustainable', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Whenever the Vehicle Configuration Parameters (VCP) generation routine is executed	57	U2302	MC3
	4.2	No Car Configuration Logic (CCL) is present in CCL memory. See *[Data_5] Central Car Configuration* , DTC 'Car Configuration – CCL Not available', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Whenever the Vehicle Configuration Parameters (VCP) generation routine is executed	55	U2302	MC3

Vehicle Configuration Data	5.1	No Vehicle Configuration Data (VCD) is present in VCD memory. See *[Data_5] Central Car Configuration*, DTC 'Car Configuration – VCD Not available', and table "Calibration of configuration DTCs" in section *Calibration of configuration DTCs.*		Whenever the Vehicle Configuration Parameters (VCP) generation routine is executed	55	U2303	MC3
Public ECU network interface	6.1	Configuration check resulted in disapproved. Refer to implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP).					

Table: Requirements on detection of faults caused by incompatible or missing software components

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
VIN	7.1	Incompatible VIN. The locally stored VIN does not match the VIN transmitted from another ECU		Once per operation cycle	00	P0630, P0631, C0545 or C0546	MC2
ECU Software	8.1	The ECU has a check mechanism where the application/strategy/data level of another ECU can be checked by the ECU. Data Indicates Software incompatibility with another ECU	Test time is determined by the functional need of the ECU to ensure correct operation		4A if module must be re-placed, 57 If module must be repro-grammed or 00 if cause not known	U03XX or U2DXX (software incompatibility with ECU XX. See note 2 for definition.	MC2
	8.2	The ECU is delivered to OEM with supplier specific software that must be replaced by OEM authorised application software as part of the manufacturing process. See note 1.	N/A	N/A	57	U2400 Supplier software installed	MC6

Abbreviations in the three tables above:
MC1 = Mandatory if the ECU is required to be programmed with local configuration

(calibration) data for correct operation.
MC2 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
MC3 = Mandatory for ECU that is the Car Configuration Domain Master according to [Data_5] *Central Car Configuration*.
MC4 = Mandatory for ECU that is a Car Configuration Subscriber according to [Data_5] *Central Car Configuration*.
MC5 = Mandatory if the ECU requires an additional control routine to instruct the ECU to learn a changed configuration.
MC6 = Mandatory if the ECU is delivered to OEM with supplier specific software that must be replaced by OEM authorised application software as part of the manufacturing process.
Notes in the three tables above:
1 This DTC shall always be set as long as the supplier specific software (not intended for use by OEM)§ is installed in the ECU. It shall not be possible to delete (clear) this DTC except by erasing or replacing the supplier specific software. The DTC shall not be supported by any OEM authorised application software.
2 See *[Data_4] Global Master Reference Database* and *[Data_51] Diagnostic trouble code definitions*.
Requirements from this section are specified in section "Requirements from section Detection of faults due to configuration errors".

4.5.1.1.4.1.7.2.5 Calibration of configuration DTCs

The DTCs defined in table *Requirements on detection of faults caused by Global ECU Configuration (global parameters)* shall be calibrated according to table *Calibration of configuration DTCs.*
All DTCs defined in table *Requirements on detection of faults caused by Global ECU Configuration (global parameters)* shall use Operation cycle counter #2 (OCC2) as the counter used for aging provided that the DTCs are not emission related (se section *Clear DTC information* for requirements on DTC aging).

Table : Calibration of configuration DTCs

Item no.	Test sample failed criteria	Increase FDC 10	Test sample passed criteria	Decrease FDC 10	unconfirmedDTCLimit	FDC 10 max value	confirmedDTCLimit	agedDTCLimit
2.1	No valid configuration (any required parameter missing or with value 0x00 or 0xFF) has been stored in VCP memory.	127	A valid configuration (all required parameters have valid values) has been stored in VCP memory.	128	127	127	12727 1 128 127 11	1
2.2	Reserved for future							

	use.							
2.3	One or more parameters used for configuration of the ECU is received with an unrecognized or invalid value (including values 0x00 and 0xFF).	127	All parameter used for configuration of the ECU have been received with valid values.	128	127	127	1	1
2.4	A changed configuration has been evaluated as valid and the ECU requires an additional control routine to instruct the ECU to learn a changed configuration	127	The routine required to instruct the ECU to learn a changed configuration has been successfully executed.	128	127	127	1	1
2.5	One or more parameters used for configuration of the ECU has a valid value but is internally inconsistent or incompatible with	127	All parameters used for configuration of the ECU have valid values and are internally consistent and compatible with all	128	127	127	1	1

	some other data or configuration in the ECU or an ECU hardware option		other data, configuration and hardware options in the ECU.					
3.1	Specified by the implementer							
4.1	The stored VCD file and CCL file are incompatible	127	The stored VCD file and CCL file are compatible	128	127	127	1	1
4.2	No Car Configuration Logic (CCL) file is stored in the CCL memory	127	A Car Configuration Logic (CCL) file is stored in the CCL memory	128	127	127	1	1
5.1	No Vehicle Configuration Data (VCD) file is stored in the VCD memory	127	A Vehicle Configuration Data (VCD) file is stored in the VCD memory	128	127	127	1	1

Requirements from this section are specified in section "Requirements from section Calibration of configuration DTCs".

4.5.1.1.4.1.7.2.6 Detection of faults inferred from system behaviour

The ECU shall be able to detect faults inferred from system behaviour according to the table below.

Table: Requirements on detection of faults inferred from system behaviour.

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB))	M/O
Any component or system	1.1	Failure to react after event. Sensor readings do not change as a result of some change actuated by the ECU (e.g. failure of evaporator temperature to fall after air	100 ms	Changes from event are actively expected.	67	One DTC for each component, system or part of system.	MC

		conditioning compressor is activated).					
	1.2	The measured behaviour of a component or system does not follow the predictions of a simulation of expected behaviour which is being run by the ECU (e.g., the engine temperature does not match with predictions based on ambient temperature, engine power output and time since engine start).	100 ms	Whenever the function being simulated in the predictive model is active	61	One DTC for each component, system or part of system.	MC
	1.3	A closed loop control (e.g. of temperature, motor position, engine speed etc.) is not stable under conditions where stability would be expected	100 ms	Stable closed loop control actively expected.	06	One DTC for each component, system or part of system.	MC
	1.4	System adaptation out of range. A system where the ECU adapts to manufacturing tolerance, component ageing etc. has exceeded the limit of its adaptation range.		Whenever any adaptation value is updated.	91	One DTC for each component, system or part of system.	MC
	1.5	Faults specified by the implementer.	100 ms. See note 1.	TRC1 and additional conditions specified by the implementer	See note 2.	One DTC for each component, system or part of system.	MC

Abbreviations in the table above:
TRC1 = Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.
MC = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

Notes in the table above:
1. The defined Test Period Time is the general requirement for this type of fault. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority at OEM.
2 See *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications.

Requirements from this section are specified in section "Requirements from section Detection of faults inferred from system behaviour".

4.5.1.1.4.1.7.2.7 Detection of faults caused by received signals

The ECU shall be able to detect faults caused by signals and data values received from other ECUs as specified in the table below.
Note: the faults detailed in this table are intended to handle the situation where there may be a customer complaint and a symptom associated with the ECU generating the DTC but the root cause is located on another ECU which may or may not be exhibiting any symptoms.

Table: Requirements on detection of faults caused by signal/data content received from other ECUs.

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure Type (FTB)	Base DTC (display value, excl. FTB)	M/O
Other ECU not operating normally	1.1	Another ECU sends data that indicates it has detected a fault which causes it to operate with reduced or not correct functionality. See note 1.		Each time the updated data (indicating reduced functionality in the other ECU) is received.	92	U2C00 to U2CFF One DTC per signal as defined in (see note 3).	MC1
Other ECU sending data tagged as invalid	2.1	Another ECU has detected a fault which causes data sent by the other ECU to be tagged as invalid. See note 2.		Each time updated data is received from another ECU. TRC1	81	U2C00 to U2CFF One DTC per signal as defined in (see note 3).	MC1
Implausible data from another ECU	3.1	Data from another ECU is not tagged as invalid but detected as faulty by the ECU, e.g. by plausibility check(i.e. when compared with one or more external or internal signals and/or the current operating conditions).		Each time updated data is received. TRC2	86	U2D00 to U2DFF One DTC per signal as defined in (see note 4).	MC1
Corrupt data from another ECU	4.1	Data value from other ECU detected as corrupt by the data protection mechanism “rolling counter” implemented at application level in the ECU. The data is received without the corresponding rolling count value being properly updated (e.g. incremented by the value 1 when expected).		Each time updated data is received.	82	U2D00 to U2DFF One DTC per signal as defined in (see note 4).	MC2
	4.2	Data value from other ECU detected as corrupt by the data protection mechanism “end-to-		Each time updated data is received	83	U2D00 to U2DFF One DTC per	MC2

		end checksum" implemented at application level in the ECU.				signal as defined in (see note 4).	
	4.3	Data value from other ECU detected as corrupt by the data protection mechanism "up-date bit" implemented at application level in the ECU. The data is received without the corresponding up-date bit value being properly set (e.g. set to the value 1 when expected). Note: The faults are identified by DTCs that are known as "LostComDTCs". If the DTC is not emission related the DTC status indicator 30:5 (SI30:5) – warningIndicatorRequestedSinceLastClear is allowed to be set to 1 for these DTCs despite the FDC10 never reach +127 as the DTCs are development specific. Refer to Table: "Requirements on implementation of DTC status indicators" for requirements on set of DTC SI 30:5 to 1. It is up to the implementer to specify the criteria for set the DTC SI 30:5 to 1		Each time updated data is expected to be received.	82	U2F00 to U2FFF if the DTC is not emission related. U0100 to U02FF if the DTC is emission related. One DTC per signal as defined in (see note 3).	O See note 5.

Abbreviations in the table above:
MC1 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements, i.e. where the primary reason for the fault monitor is another than to support generation of DTC information.
Otherwise, a feasibility study shall be performed to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
MC2 = Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
TRC1 = This test shall be inhibited if the received data is detected as corrupt by a data protection mechanism (item 4.1 to 4.3).
TRC2 = If plausibility check is used as fault detection logic the test shall be inhibited if a DTC test for one (or more) of the compared signals (used to evaluate the plausibility of the received signal) has detected a fault on that signal.
Notes in the table above:
1. In all cases the ECU transmitting the signal that indicates it is not operating normally shall set an appropriate DTC. This can be either a DTC relating to the root cause of the

situation if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU.
2. In all cases the ECU transmitting the signal tagged as invalid shall set an appropriate DTC. This can be either a DTC relating to the root cause of the invalid data if the root cause exists in the transmitting ECU, or a secondary DTC if the root cause is not in the transmitting ECU.
3. *[Data_4] Global Master Reference Database* and *[Data_51] Diagnostic trouble code definitions*, UDS - Data - External publications
4. *[Data_4] Global Master Reference Database*
5. The DTC may be mandatory if the DTC is emission related but this is not specified by this document.
Requirements from this section are specified in section "Requirements from section Detection of faults caused by received signals".

4.5.1.1.4.1.7.2.8 Detection of faults caused by the ECU environment

An ECU shall be able to detect faults caused by the ECU operating environment (e.g. temperature, operating voltage, etc.) or by certain customer actions (e.g. operating audio system continuously at maximum volume) where these faults cause symptoms that a customer would both notice and not expect given the (environmental or other) conditions observable by the customer. Detailed requirements and allowed exceptions are specified in the table below

The DTCs detailed in this table are intended to handle the situation where there may be a customer complaint and a symptom associated with the ECU (typically the ECU operates with reduced functionality) but the root cause is that the ECU is operating outside of its design specification. The DTCs in this table are generally set to provide a warning or explanation to the customer which describes the cause of the loss of function and to re-assure them that no repair is necessary. The DTCs in this table are generally set to prevent the ECU being incorrectly suspected as faulty and to prevent unnecessary repair work.

Note that during assembly of the vehicle, all DTCs associated with these faults shall not require any repair action. Furthermore, these faults will most likely not be presented in any of the manufacturing tools.

Table: Requirements on detection of faults caused by the ECU operating environment or customer actions

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure Type (FTB)	Base DTC (display value, excl. FTB)	M/O
Ambient temperature	1.1	Ambient temperature too high for full and correct operation.	100 ms		68	U2E00	O
	1.2	Ambient temperature too low for full and correct operation.	100 ms		68	U2E01	O
Customer actions	2.1	Customer operates vehicle or vehicle equipment outside specification so that	100 ms		68	U2E02	MC

		not correct, reduced or no system functionality is available, or a fallback strategy is activated (e.g. customer action causes temporary component overheating).					
Power supply voltage	3.1	ECU supply voltage is too high for full operation. ECU has symptom of not correct, reduced or no function for a period of time which the customer would notice.	100 ms		68	U2E03	MC
	3.2	ECU supply voltage is too low for full operation at a time when an average customer would not have expected low battery voltage (i.e. not during cranking or when the engine is not running). ECU has symptom of reduced or no function for a period of time which the customer would both notice and not expect (normally an obvious default behaviour latched for the complete operation cycle and/or a warning lamp indication would be required before this DTC may be set).	100 ms	TRC1	68	U2E04	MC
Other condition required for normal ECU functionality	4.1	A condition, not covered by other items in this table, is out of range for full (and correct) operation. ECU has the symptom of not correct, reduced or no function which the customer would both notice and not expect given the conditions observable by the customer.	100 ms. See note 1.	TRC2 and additional conditions specified by the implementer	68	U2E05 to U2EFF. One DTC per condition that is out of range as defined in reference *[Data_4] Global Master Reference Database*.	MC

Abbreviations in the table above:
TRC1 Not when ECU supply voltage check with the vehicle battery voltage has detected a problem (See Table *Requirements on detection of faults in external electrical circuits connected to the ECU*- DTC U3003-62).

TRC2 Not when the fault is a secondary fault that appears as a result of another (primary) fault that is detected by another DTC test. For example a DTC test that has a plausibility check for an analogue sensor signal where the signal is compared with another similar signal. This test shall be inhibited if the DTC test for the compared signal detects a fault on that signal.

MC Mandatory where the fault is detected by a fault monitor as part of the ECU functional requirements.
Otherwise, a feasibility study is required to decide if a technical solution is available and its cost is justified for the fault detection to be implemented. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

Notes in the table above:

1. The defined Test Period Time is the general requirement for this type of fault. Depending of the actual fault detected the general requirement may not be achievable or may not be practical for the type of monitor used. In such cases the implementer may use a different Test Period Time if approved by the responsible authority at OEM.

Requirements from this section are specified in section "Requirements from section Detection of faults caused by the ECU environment".

4.5.1.1.4.1.7.2.9 Detection of events associated with driver indications

If a driver indication (e.g. telltale, text message, etc.) exists which, when activated, recommends (e.g. by a service book instruction, text message, etc.) the driver to bring the car to a workshop, a study shall be performed by the implementer to investigate if the event that triggers the activation of the driver indication shall also be detected by a DTC test. The result of the study shall be approved by the responsible authority appointed by OEM.

Requirements from this section are specified in section "Requirements from section DTCs associated with driver indications".

4.5.1.1.4.1.7.2.10 Detection of faults caused by too high quiescent current

The ECU shall be able to detect faults caused by too high quiescent current according to Table below.

Requirements on detection of faults caused by too high quiescent current.

Type of item	Item no.	Detected faults/errors	Test Period	Test Run Criteria (TRC)	Failure type (FTB)	Base DTC (display value, excl. FTB)	M/O
See note 1.	1	Quiescent current is	See note 1.	See note 1.	See note 1.	See note 1.	See note 1.

too high in the ECU. See note 1.

Notes in the table above:
1. See reference *[Data_9] FltEgyCns*.

4.5.1.1.4.1.7.3 DTC status bits

DTC status bits shall be supported for all DTCs supported by the ECU according to the table below.

The requirements for storing and clearing of the status bits are specified in section *Store DTC information* and *Clear DTC information*.

The stored DTC status byte (that contains the DTC status bits) shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.

Table : Requirements on implementation of DTC status bits

DTC status bits			**M/O**
Bit no See note 1	**Name**	**Definition**	
0 (SB0)	testFailed	See note 2 with the following addition: *-[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* allows for additional reset conditions to be defined by the vehicle manufacturer or implementer. No such additional reset condition of this bit is allowed. Note that the value of this bit shall not be reset to 0 at start of an operation cycle (the value from previous operation cycle shall be kept).	M
1 (SB1)	testFailedThisOperationCycle	See note 2.	M
2 (SB2)	pendingDTC	See note 2.	M
3 (SB3)	confirmedDTC	See note 2 with the following addition: - The bit shall be set to 1 when the trip counter (OCC6) reaches the confirmedDTCLimit. The value of confirmedDTCLimit is specified by the implementer according to the requirements specified in section *Generic requirements on calibration of DTCs*. The value shall be in the range of [1, +15]. Refer to section *DTC operation cycle counters* for definition of the OCC6. - The bit shall be reset to 0 when the DTC is aged. Refer to table *Requirements in implementation of clear of DTC information* in section *Clear DTC information* for when a DTC is aged. - The bit shall not be set to 1 and it shall be reset to 0, in case of the bit is already set to 1, if there are no aging counter (OCC2 or OCC5) allocated to the DTC due to the long term memory overflow. Refer to table *Requirements in implementation of clear of DTC information in section Clear DTC information* for definition of long term memory overflow.	M

4 (SB4)	testNotCompletedSinceLast-Clear	See note 2.	M
5 (SB5)	testFailedSinceLastClear	See note 2.	M
6 (SB6)	testNotCompletedThis-OperationCycle	See note 2.	M
7 (SB7)	warningIndicatorRequested	See note 2 with the following addition: *-[Data_50] Road vehicles – Diagnostics systems – Diagnostic services* allows for additional set and reset conditions to be defined by the vehicle manufacturer or implementer. No such additional reset condition of this bit is allowed. Note that if the bit 3 (SB3) above cannot be set due to there is no aging counter (OCC2 or OCC5) allocated to the DTC then neither can this bit (SB7) be set to 1	M

Notes in the table above:

1. The number is the DTC status bit number of the DTC status byte associated with the DTC.
2. Specified in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications.

Requirements from this section are specified in section "Requirements from section DTC status bits".

4.5.1.1.4.1.7.4 DTC snapshot data

4.5.1.1.4.1.7.4.1 DTC snapshot data identification

The DTC snapshot data, consisting of both global and local DTC snapshot data, is identified by using data record identifiers.

Global DTC snapshot data consists of data that is distributed over the vehicle networks and is received by all Public ECUs. The data record identifiers for global DTC snapshot data are specified in *[Data_4] Global Master Reference Database* as detailed in chapter Global DTC snapshot data.

Local DTC snapshot data is defined separately for each ECU and can include data that is only known to the specific ECU.The data record identifiers for local DTC snapshot data are specified by the implementer as detailed in chapter Local DTC snapshot data.

Data records defined either as SystemSupplierSpecific or dynamically defined, according to table "Requirements on implementation of data records" found in section Record data - General, shall not be included in local DTC snapshot data.

If the data is sampled multiple times (record type = 'Multiple samples') before the sampling criteria is satisfied, then the sampled values shall be repeated a defined number of times in the snapshot data record. The order of the DTC snapshot data in the DTC snapshot record shall be such that the data which is sampled first is put at the start of the record (in bytes #1 to n) and the next data sample is appended to data from the first sample (bytes #n+1 to m), and so on.

See the example in the table below. See *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications* for further details.

Table : Example of identification of DTC snapshot data

DTC hex	DTC snapshot	No. of samples				DTC snapshot data
	record number, hex See section *Global DTC snapshot data*		Type	Data record type	Data record identifier hex	Definition
990022	20	N/A	Global	See note 1.	DD00	Global real time. See section *Global DTC snapshot data.*
					DD01	Total distance. See section *Global DTC snapshot data.*
					DD02	Vehicle battery voltage. See section *Global DTC snapshot data.*
					DD0A	Usage mode. See section *Global DTC snapshot data.*
					DD0B	Partial Network Cluster (PNC). See section *Global DTC snapshot data*.
					DD0C	ElPowerLevel. See section *Global DTC snapshot data.*
			Local	See note 1.	A472	Left Hand Mirror Up Down Switch
					D901	Input from push buttons x3 to x10
					D902	Input from sensor x11
				Development specific	EE00	Output to motor y1 and y2
990122	20	N/A	Global	See note 1.	DD00 – DD02, DD0A - DD0C	See above

			Local	See above	A472, D901, D902, EE00	See above
990322	20	N/A	Global	See note 1.	DD00 – DD02, DD0A - DD0C	See above
			Local	See note 1.	D904	Input from sensor x2
					D905	Input from sensor x12
990422	20	N/A	Global	See note 1.	DD00 – DD02, DD0A - DD0C	See above
			Local	See note 1.	D906	Input from temperature sensor x13
					D907	Input from switches x20 to x27
				Development specific	EE01	Output to motor y3
	30	5	Local	See note 1.	DD02	Vehicle battery voltage.
				See note 1.	D908	Input from sensor x15, sample no. 1 (först sample)
				See note 1.	D908	Input from sensor x15, sample no. 2
				See note 1.	D908	Input from sensor x15, sample no. 3
				See note 1.	D908	Input from sensor x15, sample no.5 (last sample)

Notes in the table above:

1. Specified in *[Data_4] Global Master Reference Database*.

Requirements from this section are specified in section "Requirements from section DTC snapshot dataidentification".

4.5.1.1.4.1.7.4.2 DTC snapshot sampling criteria

DTC snapshot sampling criteria shall be supported for all DTCs supported by the ECU according to the table below.

The DTC snapshot data that shall be sampled and stored when the criteria is satisfied is specified in section *Global DTC snapshot data* and *Local DTC snapshot data.*

Table: Requirements on implementation DTC snapshot sampling criteria - AUTOSAR 4.0.x

DTC snapshot record

Number, hex see note 1	Snapshot Type see note 2	The criteria which triggers the sampling of data (the snapshot)	M/O
00	Specified by [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications.	Specified by *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications*.	O
01-09	N/A	Reserved by this document for future use.	N/A
10-13	N/A	Reserved for previous versions of VCC diagnostics (GGDS).	N/A
20 (SS20)	Single sample	Each time the FDC10 reache a value that is equal to or greater than its unconfirmedDTCLimit until the FDC10 reach the value +127 (since DTC information was last cleared) and each time the FDC10 reach the value +127 until the DTC staus bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared). If FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. If FDC10 reach the value +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).	M
21-2F	Single sample	Reserved by this document for future use.	N/A
30-3F (SS 30-3F)	Multiple samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
40-4F (SS 40-4F)	Single sample	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
50-5F (SS 50-5F)	Other samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
60-FF	N/A	Reserved by this document for future use.	N/A
FF	N/A	Refer to *[Data_1] UDS Services*	N/A

Table: Requirements on implementation DTC snapshot sampling criteria - AUTOSAR 4.1 or newer

DTC snapshot record			
Number, hex	**Snapshot Type see note 2**	**The criteria which triggers the sampling of data (the snapshot)**	**M/O**

see note 1			
00	Specified by [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications.	Specified by [Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications.	O
01-09	N/A	Reserved by this document for future use.	N/A
10-13	N/A	Reserved for previous versions of VCC diagnostics (GGDS).	N/A
20 (SS20)	Single sample	Each time the FDC10 reach the value +127 until the DTC staus bit 3 – confirmedDTC is set to 1 (since DTC information was last cleared). If FDC10 reach the value +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).	M
21 (SS21)	Single Sample	Each time the FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 (since DTC information was last cleared). If FDC10 reaches a value that is equal to or greater than its unconfirmedDTCLimit and less than +127 more than once during an operation cycle then the sampling of data shall be triggered the first time only. Only the last sampled DTC snapshot data shall kept in the memory (the previously sampled DTC snapshot data is overwritten).	M
21-2F	Single sample	Reserved by this document for future use.	N/A
30-3F (SS 30-3F)	Multiple samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
40-4F (SS 40-4F)	Single sample	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
50-5F (SS 50-5F)	Other samples	Specified by the implementer. The DTC snapshot record shall only consist of DTC snapshot data that is needed during the development of the ECU. The implementer shall not expect that OEM tools support the DTC snapshot record.	O
60-FF	N/A	Reserved by this document for future use.	N/A
FF	N/A	Refer to *[Data_1] UDS Services*	N/A

Notes in the tables above:

1. The number is the DTCSnapshotRecordNumber that identifies the failure criteria which triggers the sampling and storage of DTC snapshot data. The text within brackets is the short name of the DTC snapshot record.

2 Single sample: The DTC snapshot data shall be sampled (and stored) once when the criteria is satisfied.

Multiple samples: The DTC snapshot data shall be sampled periodically. A number of samplers before and/or after the criteria are satisfied shall be stored (the number of samples, and the periodicity, shall be defined by the implementer).

Requirements from this section are specified in section "Requirements from section DTC snapshot sampling criteria".

4.5.1.1.4.1.7.4.3 Global DTC snapshot data

Global DTC snapshot data shall be supported for all DTCs supported by the ECU according to the table below.

The data records defined in this table must be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.

The requirements for storing and clearing of the DTC snapshot data are specified in sections *Store DTC information* and *Clear DTC information.*

The criteria that trigger the sampling and storage of DTC snapshot data are specified in section *DTC snapshot sampling criteria* .

If DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data. If no value has been received in the current operation cycle a value representing "correct value not available" shall be used and stored as DTC snapshot data.

Table: Requirements on implementation of global DTC snapshot data

Global DTC snapshot data			**M/O**
Data record identifier, hex	**Name**	**Definition**	
DD00	Global real time	Data record defined in *[Data_4] Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for maintaining and distributing the global real time data: - range 0 to 13.6 year / 0x00000000 to 0xFFFFFFFE, correct value not available = 0xFFFFFFFF - resolution 1 bit = 100 ms. - accuracy minimum ±10 minutes/week (±0.1%). - after the global time have reached 0xFFFFFFFE it shall restart from 0x00000000. - the current value of global real time shall be stored in long.term memory at least once each operation cycle. - If the ECU loses the current value due to loss of power supply then, following reconnection of the power supply, it shall continue to count the global real time beginning from the stored value. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	M
DD01	Total distance	Data record defined in *[Data_4] Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for maintaining and distributing the total distance data: - range minimum 0 to 1 000 000 km / 0x000000 to 0x0F4240, correct value not available = 0xFFFFFF - resolution 1 bit = 1 km	M

		- the accuracy shall be the same as or better than the accuracy of the total distance displayed in the dash board. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	
DD02	Vehicle battery voltage	Data record defined in *[Data_4] Global Master Reference Database* with the following additional requirements that are only applicable to the ECU responsible for measuring and distributing the vehicle battery voltage: - range minimum 0 to 24 V / 0x00 to 0x60, correct value not available = 0xFF - resolution 1 bit = 0.25 V - see section *Detection of faults in external electrical circuits* The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	M
DD03 to DD09	Reserved for previous versions of VCC diagnostics (GGDS)		
DD0A	Usage mode	Data record defined by *[Data_4] Global Master Reference Database*. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	M
DD0B	Partial Network Cluster (PNC)	Data record defined by reference *[Data_4] Global Master Reference Database*. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*.	MC
DD0C	ElPowerLevel	Data record defined by reference *[Data_4] Global Master Reference Database*. The stored DTC snapshot data shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services*.	M

Abbreviations in the table above:
MC = This DID is not mandatory for all ECU's. It shall therefore only be included in Global Snapshot Data in case the ECU is required to implement this DID.

Global DTC snapshot data (see the table above) shall be transmitted to all public ECUs by an ECU (one or more) responsible for maintaining and distributing the data according to the following requirements:

- The ECU shall transmit current values of the data periodically each second on the network except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent faster according to the requirements specified in section *Additional requirements for detection of low supply voltage.*
- The ECU shall transmit the data with a maximum age of 250 ms, except the data record 0xDD02 – "Vehicle battery voltage" which shall be sent with a lower maximim age

according to the requirements specified in section *Additional requirements for detection of low supply voltage*.

4.5.1.1.4.1.7.4.4 Local DTC snapshot data

Local DTC snapshot data shall be supported by the ECU according to the table below :
The data record identifiers shall be defined within the range of identifiers specified for these data records in table "Requirements on implementation of data records", section *Record data - General* .
The requirements for storing and clearing of the DTC snapshot data are specified in sections *Store DTC information* and *Clear DTC information*.
The criteria that trigger the sampling and storage of DTC snapshot data are specified in section *DTC snapshot sampling criteria.*
If DTC snapshot data received from the vehicle network is not available when the sampling criteria is fulfilled, the latest received value shall be used and stored as DTC snapshot data. If no value has been received in the current operation cycle a value representing "correct value not available" shall be used and stored as DTC snapshot data

Table: Requirements on implementation of local DTC snapshot data

Local DTC snapshot data			**M/O**
Data record identifier, hex	**Name**	**Definition**	
Defined by the implementer	Defined by the implementer	Local DTC snapshot data is specified by the implementer. The data shall be chosen so that it will be useful in identification of the conditions (e.g. operation state and surrounding environment of the ECU) which are present whenever a fault (and the fault symptom) occurs. The local DTC snapshot data may be specific for each DTC (e.g. different data for different DTCs). The stored DTC snapshot data shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services*.	MC

Abbreviations in the table:
MC = A feasibility study is required. This shall be implemented if a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.

4.5.1.1.4.1.7.5 DTC extended data

4.5.1.1.4.1.7.5.1 DTC operation cycle counters

DTC operation cycle counters shall be supported for all DTCs supported by the ECU according to the table below :
The requirements for storing and clearing of the counters are specified in sections *Store DTC information* and *Clear DTC information.*

Table : Requirements on implementation of DTC operation cycle counters - AUTOSAR 4.0.x

DTC extended data record			M/O
Number hex, see note 1	**Name**	**Definition**	
01	DTC operation cycle counter #1 (OCC1)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal to or greater than its unconfirmedDTCLimit or more (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle. The counter shall be cleared as soon as FDC10 reaches its unconfirmedDTCLimit. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
02	DTC operation cycle counter #2 (OCC2)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached a value that is equal or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded. Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as the FDC10 reaches the value -128, unless FDC10 has already reached a value that is equal to or greater than its unconfirmedDTCLimit in the current operation cycle. The counter shall be cleared as soon as FDC10 reaches its unconfirmedDTCLimit. If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
03	DTC operation cycle counter #3 (OCC3)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reach a value that is equal to or greater than its unconfirmedDTCLimit shall be included.	M

		Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
04	DTC operation cycle counter #4 (OCC4)	Definition: Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached a value that is equal to or greater than its unconfirmedDTCLimit for the first time. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
05	DTC operation cycle counter #5 (OCC5)/Warm-up cycle counter	Definition: Number of warm-up cycles since the DTC commanded the MIL to switch off (since DTC information was last cleared). The warm-up cycle is defined in *[Data_52] OBD II regulation, section 1968.2 of title 13, California Code of Regulations*, UDS - Data - External publications. Increment/clear criteria: The counter shall be incremented when WUC changes from false to true. The counter shall be cleared when the DTC fault detection counter 10 reach the value of +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte.	MC
06	DTC operation cycle counter #6 (OCC6)	Definition: The number of consecutive operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value of +127 (since DTC information was last cleared). The operation cycles during which the test was not completed shall be excluded. The minimum range of the counter is 0-15.	M

		Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time. The counter shall be cleared in the beginning of the operation cycle following an operation cycle where FDC10 did not reach the value +127 at any time but -128 (the counter shall maintain its value if did not reach the value -128). The counter may also be cleared when the confirmedDTCLimit is reached. If any count operation occurs which would cause a counter to roll over past 0xFF, then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
07	DTC o peration cycle counter #7 (OCC7)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). The operation cycles during which the test was not passed or the DTC fault detection counter 10 did not reach a value that is equal to or greater than its unconfirmedDTCLimit shall be excluded. Increment/clear criteria: The counter shall be incremented in each operation cycle, following the operation cycle where FDC10 first reached a value that is equal to or greater than its unconfirmedDTCLimit, as soon as the following criteria is meet: The FDC10 reach -128 or a value that is equal to or greater than its unconfirmedDTCLimit. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M

Table: Requirements on implementation of DTC operation cycle counters - AUTOSAR 4.1 or newer

DTC extended data record			**M/O**
Number hex, see note 1	**Name**	**Definition**	
01	DTC operation cycle counter	Definition:	M

	#1 (OCC1)	Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). All operation cycles, including those during which the test was not passed shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle. The counter shall be cleared as soon as FDC10 reaches the value +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
02	DTC operation cycle counter #2 (OCC2)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 last reached the value +127 (since DTC information was last cleared). The operation cycles during which the test was not passed shall be excluded. Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as the FDC10 reaches the value -128, unless FDC10 has already reached the value +127 in the current operation cycle. The counter shall be cleared as soon as FDC10 reaches the value +127. If any count operation occurs which would cause a couter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
03	DTC operation cycle counter #3 (OCC3)	Definition: Number of operation cycles (including the current operation cycle) since the DTC fault detection counter 10 first reached the value +127 (since DTC information was last cleared). All operation cycles, including those in which the test has not been passed or the DTC fault detection counter 10 has not reached the value +127 shall be included. Increment/clear criteria: The counter shall be incremented at the beginning of each operation cycle following the operation cycle where FDC10 first reached the value +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting:	M

		The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	
04	DTC operation cycle counter #4 (OCC4)	Definition: Number of operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached a value that is equal to or greater than its unconfirmedDTCLimit (since DTC information was last cleared). Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached a value that is equal to or greater than its unconfirmedDTCLimit for the first time. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	M
05	DTC operation cycle counter #5 (OCC5)/Warm-up cycle counter	Definition: Number of warm-up cycles since the DTC commanded the MIL to switch off (since DTC information was last cleared). The warm-up cycle is defined in *[Data_52] OBD II regulation, section 1968.2 of title 13, California Code of Regulations*, UDS - Data - External publications. Increment/clear criteria: The counter shall be incremented when WUC changes from false to true. The counter shall be cleared when the DTC fault detection counter 10 reach the value of +127. If any count operation occurs which would cause a counter to roll over past 0xFF then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte.	MC
06	DTC operation cycle counter #6 (OCC6)	Definition: The number of consecutive operation cycles (including the current operation cycle) during which the DTC fault detection counter 10 reached the value of +127 (since DTC information was last cleared). The operation cycles during which the test was not completed shall be excluded. The minimum range of the counter is 0-15. Increment/clear criteria: In each operation cycle the counter shall be incremented as soon as FDC10 has reached the value +127 for the first time. The counter shall be cleared in the beginning of the operation cycle following an operation cycle where FDC10 did not reach the value +127 at any time but -128 (the counter shall maintain its value if did not reach the value -128). The counter may also be cleared when the confirmedDTCLimit is reached.	M

		If any count operation occurs which would cause a counter to roll over past 0xFF, then the count value shall instead be maintained at 0xFF. Reporting: The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte unsigned numeric.	

Abbreviations in the table above:
MC = Mandatory for emission related DTCs supported by the ECU.
Notes in the table above:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section "Requirements from section DTC operation cycle counters".

4.5.1.1.4.1.7.5.2 DTC fault detection counters

DTC fault detection counters shall be supported for all DTCs supported by the ECU according to the table below:
The requirements for storing and clearing of the counters are specified in sections *Store DTC information* and *Clear DTC information* .

Table : Requirements for the implementation DTC fault detection counters

DTC extended data record			**M/O**
Number hex, see note 1	**Name**	**Definition**	
10	DTC fault detection counter 10 (FDC10)	The counter is specified by *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications, service 19 sub-function 14 and Appendix D.5 (Appendix D.5 shall be regarded as mandatory and not just an example) for both emission-related and non-emission-related DTCs with the following additions: 1. At the start of a new operation cycle the count value shall be 0. 2. The count value shall be maintained between test runs within one and same operation cycle (also when the test run is blocked due to the test run criteria is not satisfied). 3. The DTC test may be completed (the DTC status bit 4 and 6 is set to 0) before FDC10 reaches +127 or -128, and passed (the DTC status bit 0 is set to 0) before FDC10 reaches -128, where approved by the responsible authority appointed by OEM. 4. The maximum value of the counter may be set to +126 (which means that the value DTC status bits, except the bits 4 and 6, will never be set, they will remain unchanged) where approved by the responsible authority appointed by OEM or shall be set to +126 if that is specified by this document.	M

		5. If a single test sample of a test returns a fail result, when the current value of the counter is negative, the new count value resulting from the fail result shall be set as if the current count value had been 0 (a test sample represents the fail result shall always causes fault detection counter to decrement above 0). 6. If a single test sample of a test returns a pass result, when the current value of the counter is positive, the new count value resulting from the pass result shall be set as if the current count value had been 0 (a test sample represents the pass result shall always causes fault detection counter to decrement below 0). 7. The count value shall be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. If the number of consecutive test sample failed results is more than 127 to reach the testFailedLimit +127 (from the start value 0) or if the number of consecutive test sample passed results is more than 128 to reach the testPassedLimit -128 (from the start value 0), the count value cannot be updated (increased or decreased) each time the DTC test runs and produce a test sample failed or passed result. In this case, and only in this case, may a pre-counter or similar (specified by the implementer) be used in order to reduce the number of updates to 127 (at increases) and 128 (at decreases). Exception to this requirement (e.g. due to legal requirements) is allowed but only when approved by the responsible authority appointed by OEM. Refer to section Definitions and Abbreviations for definition of a test sample. Refer to section DTC status bits for details of the DTC status bits. The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte signed numeric with a range of –128 to + 127.	
11	DTC fault detection counter 11 (FDC11)	The maximum value that the FDC10 has reached during the current operation cycle. The stored counter shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 1 byte signed numeric with a range of –128 to + 127.	O
12	DTC fault detection counter 12 (FDC12)	The maximum value, which is equal to or greater than unconfirmedDTCLimit, that the FDC10 has reached since the last time DTC information was cleared. The stored counter shall be reported on request of the diagnostic services specified in*[Data_1] UDS Services* and be reported as 1 byte signed numeric with a range of –128 to + 127.	M

Notes in the table above:

1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section "Requirements from section DTC fault detection counters".

4.5.1.1.4.1.7.5.3 DTC time stamp

DTC time stamps shall be supported for all DTCs supported by the ECU according to following the table below :
The requirements for storing and clearing of the time stamps are specified in sections *Store DTC information* and *Clear DTC information* .

Table: Requirements on implementation of DTC time stamps

DTC extended data record			**M/O**
Number hex, see note 1	**Name**	**Definition**	
20	DTC time stamp 20 (TS20)	The global real time that is taken the first time, since DTC information was last cleared, FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit. Refer to section *Global DTC snapshot data*, data identifier DD00, for definition of global real time. Refer to section *DTC status indicators* for definition of the unconfirmedDTCLimit. The stored time stamp shall be reported on request of the diagnostic services specified in *[Data_1] UDS - Services*, and be reported as 4 bytes.	M
21	DTC time stamp 21 (TS21)	The global real time that is taken the latest time, since DTC information was last cleared, FDC10 reaches a value that is equal to or greater than unconfirmedDTCLimit. If FDC10 is updated and reaches a value equal to or greater than unconfirmedDTCLimit more than once during an operation cycle then the time stamp shall be taken the first time only. Refer to section *Global DTC snapshot data*, data identifier DD00, for definition of global real time. Refer to section *DTC status indicators* for definition of the unconfirmedDTCLimit. The stored time stamp shall be reported on request of the diagnostic services specified in *[Data_1] UDS Services* and be reported as 4 bytes.	M

Notes in Table 4.23:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section "Requirements from section DTC time stamp"..

4.5.1.1.4.1.7.5.4 DTC status indicators

DTC status indicators shall be supported for all DTCs supported by the ECU according to the table below :
The requirements for storing and clearing of the status indicators in long term memory are specified in sections *Store DTC information* and *Clear DTC information* .

Table: Requirements on implementation of DTC status indicators - AUTOSAR 4.0.x

DTC extended data record					M/O
Number hex, see note 1	**Name**	**Definition**			
30	DTC status indicator 30 (SI30)	The status indicator consists of the following bits:			
		No	Name	Definition	
		0	UnconfirmedDTC	The bit shall indicate whether or not the DTC is currently unconfirmed. The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]. The bit is reset to 0 when the FDC10 reach the value -128. The value of the unconfirmedDTCLimit is specified by the implementer according to the requirements specified in section *Generic requirements on calibration of DTCs*. The value shall be in the range of [+1, +127]. Note that the value of this bit shall not be reset to 0 at start of an operation cycle (the value from previous operation cycle shall be kept).	M
		1	UnconfirmedDTC-ThisOperationCyle	The bit shall indicate whether or not the FDC10 has reached a value in the range [unconfirmedDTCLimit, +127] at any time during the current operation cycle. The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]. The bit is reset to 0 at start of an operation cycle.	M
		2	UnconfirmedDTC-SinceLastClear.	The bit shall indicate whether or not the FDC10 has reached a value in the range [unconfirmedDTCLimit, +127] since DTC information was last cleared) The bit is set to 1 when the FDC10 reach a value in the range [unconfirmedDTCLimit, +127]..	M
		3	AgedDTC	The bit is set to 1 when the DTC is aged. The bit is reset to 0 when the aged DTC reoccurs. Refer to Table 4.27 for when a DTC is aged and when an aged DTC reoccurs.	M
		4	SymptomSinceLast-Clear	The fault detected by the DTC-test has caused a customer symptom since DTC information was last clear. The bit shall be set to 1 when the FDC10 is in the range [unconfirmedDTCLimit, +127] and specific conditions, that need to be fulfilled in order to cause a customer symptom, are satisfied. The specific conditions, specified by the implementer, can be that the customer use the defect function, ECU takes certain actions due to the fault, the ECU is in specific operation or environment condition, etc.	MC1
		5	WarningIndicator-Requested Since-LastClear	Maximum value of DTC status bit 7 since DTC information was last cleared. The bit is set to 1 when DTC status bit no 7 is set to 1. However, the bit may also be set to 1 even when DTC status bit not 7 – warningIndicatorRequested is not set to 1 (refer to reference *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services,*	M

				UDS - Data - External publications regarding requirements on set of DTC status bit not 7 to 1) according to the following: The bit is set to 1 when the same criteria as required by the above mention reference on set of DTC status bit not 7 – warningIndicatorRequested to 1 is satisfied, but it may also be set to 1 and warning indicator may be requested before DTC status bit 3 – confirmed DTC is set to 1. Note that in all cases must FDC10 has reached the value +127, since DTC information was latest cleared, before the bit is set to 1 and warring indicator is requested (if not otherwise is specified in this document).	
		6	EmissionRelatedDTC	The bit is set to 0 when the DTC is not emission related. The bit is set to 1 when the DTC is emission related. Note that a DTC is not emission related as long as ithe FDC10 has not reached the value +127 since DTC information was last cleared and when it is aged i.e. the SI30:3 (bit 3 above) is set to1.	M
		7	TestFailedSinceLastClear/Aged	DTC test failed since the DTC was last aged or, as long as the DTC have not been aged, since the DTC information was last clear. The bit is set to set to 1 when DTC test is failed i.e. when the FDC10 reach the value +127. The bit is reset to 0 when DTC is aged i.e. the SI30:3 (bit 3 above) is set to1.	M
		The stored status indicators shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services* and be reported as 1 byte bit mapped value.			

Table: Requirements on implementation of DTC status indicators - AUTOSAR 4.1 or newer

DTC extended data record					**M/O**
Number hex, see note 1	**Name**	**Definition**			
30	DTC status indicator 30 (SI30)	The status indicator consists of the following bits:			
		No	Name	Definition	
		0	UnconfirmedDTC	Not Used, the bit shall be set to 0	M
		1	UnconfirmedDTC-ThisOperationCyle	Not Used, the bit shall be set to 0	M
		2	UnconfirmedDTC-SinceLastClear.	Not Used, the bit shall be set to 0	M
		3	AgedDTC	Not Used, the bit shall be set to 0	M

		4	SymptomSinceLast-Clear	The fault detected by the DTC-test has caused a customer symptom since DTC information was last clear. The bit shall be set to 1 when the FDC10 is in the range [unconfirmedDTCLimit, +127] and specific conditions, that need to be fulfilled in order to cause a customer symptom, are satisfied. The specific conditions, specified by the implementer, can be that the customer use the defect function, ECU takes certain actions due to the fault, the ECU is in specific operation or environment condition, etc.	MC1
		5	WarningIndicator-Requested Since-LastClear	Maximum value of DTC status bit 7 since DTC information was last cleared. The bit is set to 1 when DTC status bit no 7 is set to 1. However, the bit may also be set to 1 even when DTC status bit not 7 – warningIndicatorRequested is not set to 1 (refer to reference *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications* regarding requirements on set of DTC status bit not 7 to 1) according to the following: The bit is set to 1 when the same criteria as required by the above mention reference on set of DTC status bit not 7 – warningIndicatorRequested to 1 is satisfied, but it may also be set to 1 and warning indicator may be requested before DTC status bit 3 – confirmed DTC is set to 1. Note that in all cases must FDC10 has reached the value +127, since DTC information was latest cleared, before the bit is set to 1 and warring indicator is requested (if not otherwise is specified in this document).	M
		6	EmissionRelatedDTC	The bit is set to 0 when the DTC is not emission related. The bit is set to 1 when the DTC is emission related. Note that a DTC is not emission related as long as ithe FDC10 has not reached the value +127 since DTC information was last cleared and when it is aged i.e. the SI30:3 (bit 3 above) is set to1.	M
		7	TestFailedSinceLastClear/Aged	DTC test failed since the DTC was last aged or, as long as the DTC have not been aged, since the DTC information was last clear. The bit is set to set to 1 when DTC test is failed i.e. when the FDC10 reach the value +127. The bit is reset to 0 when DTC is aged i.e. the SI30:3 (bit 3 above) is set to1.	M
		The stored status indicators shall be reported on request of the diagnostic services specified in reference *[Data_1] UDS Services* and be reported as 1 byte bit mapped value.			

Abbreviations in the table above:
MC1 = A feasibility study is required. This bit shall be implemented if a technical solution is available and its cost is justified. The implementer is responsible for the feasibility study. The feasibility study shall be approved by the responsible authority appointed by OEM.
Notes in the table above:
1. The number is the DTCExtendedDataRecordNumber.

Requirements from this section are specified in section DTC status indicators "DTC status indicators".

4.5.1.1.4.1.7.6 Store DTC information

The ECU shall be able to store the DTC information that are supported by the ECU, i.e. DTC status bits, DTC snapshot data and DTC extended data (refer to sections *DTC status bits* , *DTC snapshot data* and *DTC extended data* for which DTC information that the ECU shall support) according to the table below.

If DTC information is temporarily stored in short-term memory before being stored in long-term memory it shall be possible to store at least the same amount of DTC information in short-term memory as in long-term memory.

The DTC information shall be stored within a maximum delay time of "DTC storage time" T1=400 ms. The parameter "DTC storage time" T1 is defined as the time from that an event occurs (e.g. a fault is detected) that cause DTC information to be stored (updated) until the DTC information is available for access by diagnostic service specified in reference [1]. If many events (e.g. 10 or more) occur within a time period of T1 then may the "DTC storage time" be increased but only if that is approved by the responsible authority appointed by OEM.

Table: Requirements on storage of the DTC information that are supported by the ECU - AUTOSAR 4.0.x

DTC information				**Number of DTC for which it shall be possible to store the DTC information supported by the ECU**				
				In short-term memory		**In long-term memory**		
Name		**Number, hex**		**Minimum number (see note 1 below)**	**All**	**Minimum number (see note 1 below)**	**All**	**Any number specified by the implementer**
DTC status bits (SB)		0, 2, 3, 4, 5					M	
		1, 6			M			
		7			M			O
DTC snapshot data (SD), see note 1		20				M		
		30						O
DTC extended data (ED)	DTC operation cycle counters (OCC)	1, 2, 3, 4, 6, 7				M		
		5				MC1		
	DTC fault detection counter (FDC)	10			M			
		11			O			
		12				M		
	DTC time stamp (TS)	20, 21				M		
	DTC status indicators (SI)	30	Bits 0, 2-7			M		
			Bit 1	M				

Table: Requirements on storage of the DTC information that are supported by the ECU - AUTOSAR 4.1 or newer

DTC information			Number of DTC for which it shall be possible to store the DTC information supported by the ECU				
			In short-term memory		In long-term memory		
Name		Number, hex	Minimum number (see note 1 below)	All	Minimum number (see note 1 below)	All	Any number specified by the implementer
DTC status bits (SB)		0, 2, 3, 4, 5				M	
		1, 6		M			
		7		M			O
DTC snapshot data (SD), see note 1		20			M		
		30					O
DTC extended data (ED)	DTC operation cycle counters (OCC)	1, 2, 3, 4, 6, 7			M		
		5			MC1		
	DTC fault detection counter (FDC)	10		M			
		11		O			
		12			M		
	DTC time stamp (TS)	20, 21			M		
	DTC status inidicators (SI)	30			M		

Abbreviations in the table above:
MC1 = Mandatory if the DTC is emission related.
Notes in the table above:
1. See the table "Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory" below.

Table: Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory

Number of supported DTCs	Minimum number of DTCs for which DTC information shall be possible to store	Notes
0-5	All supported DTCs	
6-50	5	
51-200	5 + 1/30 x (all supported DTCs minus 50)	1
201-	10	

Notes in the table above:
1 The value shall be rounded to the nearest integer value.

Requirements from this section are specified in section "Requirements from Store DTC information".

4.5.1.1.4.1.7.6.1 Transfer of DTC information to long term memory

The following requirements are applicable only when DTC information is temporarily stored in short-term memory (e.g. RAM) before being stored in long-term memory (e.g. Flash).

DTC information shall be transferred from short-term memory to long-term memory whenever necessary in order to ensure that no DTC information is lost during normal vehicle operation (examples of situations/events, which are not regarded as normal vehicle operation, are defined below). As a minimum, DTC information that has been added, deleted or changed shall be transferred from short-term memory to long-term memory at least at the end of each operation cycle.

There will be situations where an ECU resets either due to unexpected operating situations or as a result of requested diagnostic services. These ECU reset situations may result in that the DTC information is not transferred to long-term memory and thereby this DTC information is lost. In such situations only DTC information created in the current operation cycle, prior to the ECU reset, may be lost. DTC information from previous operation cycles shall always be retained.
Examples of acceptable causes which may result in lost DTC information are listed below:

1) Unexpected disconnection of the ECU power supply while the ECU is in operation, due to Vehicle battery removal, fuse blow, connector disconnection or other similar cause which does not occur as part of normal vehicle operation.
2) ECU supply voltage below minimum specification due to e.g. weak battery or other vehicle related fault.
3) Diagnostic service requests that results in an ECU reset. See *[Data_1] UDS Services* for services which are allowed to reset the ECU.
All other situations, not covered by the examples above, which may lead to lost DTC information, shall be identified by the implementer and are accepted only after approval by the responsible authority appointed by OEM.

Requirements from this section are specified in section "Requirements from section Transfer of DTC information to long term memory".

4.5.1.1.4.1.7.7 Clear DTC information

Clearing of stored DTC information (see sections *DTC status bits* , *DTC snapshot data* and *DTC extended data* for definition of the DTC information) shall be supported by the ECU according to the following the table below.
Note that the ECU is not allowed to support other ways to clear stored DTC information diagnostic data unless it is approved by the responsible authority appointed by OEM.

Table: Requirements in implementation of clear of DTC information

Clear of DTC information			
Item	**When DTC information shall be cleared**	**DTC information that shall be cleared**	**M/O**
1	When requested by diagnostic service specified in *[Data_1] UDS Services*.	All with some exceptions. The exceptions are specified by reference [Data_50] *Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications*	MC

2	When the long term memory overflows. The potential for an overflow condition occurs when storage of DTC information is required but available storage for the DTC information in the long term memory is already full. An overflow condition shall be resolved as follows: If the data that is required to be stored has higher storage priority than any of the stored data then shall the stored data of lowest storage priority be cleared (see note 1 below) and replaced by the data that is required to be stored. Otherwise shall the stored data not be cleared and replaced. Note 1: If several data items (DTC information for several DTCs) of the same storage priority are stored then shall the data item that has the highest value of OOC2 be cleared. If several data items (DTC information for several DTCs) of the same priority and the same value of OCC2 are stored then is it up to the implementer to specify which of these data that shall be cleared. The order of storage priority is the following (1 is the highest priority): 1. The DTC is associated with a warningIndicator (i.e. support the value 1 of the DTC status bit 7) and is not aged. See note 2 below. 2. The DTC is not associated with a warningIndicator (i.e. do not support the value 1 of the DTC status bit 7). See note 2 below. 3. The DTC is aged (i.e. the DTC status indicator 30, bit 3 is set to 1). Note 2: Stored DTC information is not aged when DTC status indicator 30, bit 3 is set to 0. DTC information that is required to be stored is not aged because DTC information is required to be stored when the DTC fault detection counter 10 reach a value in the range [unconfirmedDTCLimit, +127]. Excluded from above stated storage priority are development specific DTCs (refer to “Table: Requirements on implementation of DTCs” regarding definition of development specific DTCs). They shall have the lowest storage priority (i.e, priority 4) of all DTCs supported by the ECU. However, development specific DTCs may have higher storage priority if the minimum number of DTCs that shall be possible to store in long term memory (according to Table: Requirements on minimum number of DTCs for which DTC information shall be possible to store in long-term memory) is increased with the number of the development specific DTCs that shall have higher storage priority. The mutual storage priority between developments specific DTCs is specified by the implementer. Safety and emission related DTCs and other specific DTCs may have additional storage priorities than above where approved by the responsible authority appointed by OEM. See section *Requisite documents* for handling of conflicts with legislated requirements. Note 3: If DTC information has not been stored due to memory overflow, then shall the DTC information be stored as soon as memory is available and the criteria for storing are satisfied again.	All DTC information except the DTC status bits and FDC10. Note that the DTC status bit 3 is set to 0, according to Table - *Requirements on implementation of DTC status bits*, if there are no aging counter (OCC2 or OCC5) allocated to the DTC due to the long term memory overflow.	MC

	Refer to section *DTC status bits* for details about the DTC status bits. Refer to section *DTC fault detection counters* for details about the DTC fault detection counter. Refer to section *DTC status indicators* for details about the DTC status indicators.		
3	When a DTC is aged. A DTC is aged when the counter used for aging reach the agedDTCLimit. The agedDTCLimit is a value in the range of [01, 255]. The counter used for aging shall either be operation cycle counter 2 or 5 (warm up cycle counter) if not otherwise is specified by this document. The implementer specifies the value of the agedDTCLimit according to the requirements specified in section *Generic requirements on calibration of DTCs* and which of the two counters shall be used for aging. An addtion to specified above is that emission related DTCs my use both counters for aging. I this case is the DTC aged when both counters has reached its agedDTCLimit . Refer to section *DTC operation cycle counters* for details about the operation cycle counters.	No DTC information. Note that the bit SI 30:3 is set to 1, SI30:6 and SI30:7 are set to 0 according to Table - *Requirements on implementation of DTC status indicators* and SB3 is set to 0 according to Table - *Requirements on implementation of DTC status bits* when a DTC is aged.	MC
4	When an aged DTC reoccurs. An aged DTC reoccurs when the FDC10 reach a value equal to or greater than unconfirmedDTCLimit after that the DTC has been aged. Refer to item 3 of this table for details about aging of DTCs Refer to section *DTC status indicators* for details about the unconfirmedDTCLimit. Note that new DTC information, that replaces the cleared DTC information, shall be sampled and stored as specified in other section of this document, when an aged DTC reoccurs	All DTC information except the DTC status bits and FDC10.	MC

Abbreviations in the table above:
MC = Mandatory for all DTCs supported by the ECU with some exceptions. The exceptions are specified in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*.

Requirements from this section are specified in section "Requirements from section Clear DTC information".

4.5.1.1.4.2 Routines

4.5.1.1.4.2.1 General

Routines shall be supported by the ECU according to the table below (e.g. in the valid identifier ranges). No other routines are allowed unless it is approved by the responsible authority appointed by OEM.

It shall be possible to control the routines and read results using the diagnostic services specified in *[Data_1] UDS Services*.

Table: Requirements on implementation of routines

Item	Routines	Session, see note 5	Notes

	Name	Valid identifier range, hex	Definition	Program-ming	Other, see note 3	
1	Vehicle Manufacturer Specific	0200 – DBFF	Routines are defined in *[Data_4] Global Master Reference Database*. Routines that shall be implemented are specified in section *Supported routines*	M	M	1
2	Development Specific	DC00 – DFFF	Routines defined by the implementer. Routines that are needed only during the development of the ECU, or for analysis of ECUs returned to the vehicle manufacturer, shall be defined in this range.	D	O	2
3	System Supplier Specific	F000 – FEFF	Specified by the system supplier.	D	O	2, 4
4	ISO specified	FF00-FF02	Routines are defined in reference *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services, UDS - Data - External publications,*Routines that shall be implemented are specified in section *Supported routines*	M	D	

Notes in the table above:
1. Routines defined in these identifier ranges shall be implemented exactly as they are defined in *[Data_4] Global Master Reference Database*.
2. The implementer shall not expect that OEM tools support the routines within this range.
3. Whether or not the supported session is all other sessions or just one or some depends on which session the diagnostic services for control and read of the routines is supported in.
4. Routines in this range shall comply with the requirements in *[Data_50] Road vehicles – Diagnostics systems – Diagnostic services*, UDS - Data - External publications, but are not required to comply with requirements in *[Data_1] UDS Services*.
5. Note that “M” means that the ECU shall support routines according to column “Definition” of the table which shall be defined within the routine identifier range according to column “valid identifier range”. Note that “O” means that the ECU may support routine according to column “Definition” of the table which shall be defined within the identifier range according to column “valid display value range”.

Requirements from this section are specified in section "Requirements from section Routines - General"

4.5.1.1.4.2.2 Supported routines

Routines shall be supported by the ECU according to the table below and shall be implemented as they are defined in *[Data_4] Global Master Reference Database*.

Table: Requirements for implementation of routines

Item	Routines				Session			Notes
	Name	Routine identifier,	Routine Type	Definition	Default	Extended	Program-ming	

		hex				Diagnostic		
1	Check Complete & Compatible	0205	1	The routine shall check if the ECU software(s) is complete and compatible according to the requirements defined in reference *[Data_6] General Software Download Specification*.	D	D	MPS	
2	Check Programming Pre-conditions	0206	1	The routine shall check if the ECU, under its current operating conditions, is able to enter program mode .	M	M	N/A	1
3	ActivateSBL	0301	1	The routine s hall be used to activate the Secondary Bootloader after it has been downloaded to volatile memory. The memoryAddress where the ECU shall start executing the secondary bootloader is defined in the data file containing the Secondary Bootloader.	N/A	N/A	MPS	3
4	**Erase Memory**	FF00	1	The routine s hall perform a non-volatile memory erase of the memory blocks defined by the memoryAddress and memorySize request parameters.	N	N	MCS	2
5	Software installation	D000	2	The routine shall start an installation of a downloaded data file. Note; If the start of installation is needed or not depends of type of data file. The data file indicating if the routine shall be performed or not.	N	N	OCPS	4
6	Check Memory	0212	1	The routine shall initiate the ECU authenticity verification according to the requirements defined in reference *[Data_10] General Software Authentication Specification*.	D	D	MCPS	

Abbreviations in t able below :

MCS = Mandatory for secondary bootloader if the ECU don´t has file system data storage and implements an alternative method e.g RequestFileTransfer – modeOfOperation parameter DeleteFile or ReplaceFile.

MCPS = Mandatory on Condition for primary and secondory bootloader. To be implemented by ECUs supporting the Software Authentication concept where each data file is signed and

verified individually.

OCPS= Optional for primary and secondary bootloader if the ECU support file system data storage and is needed to start an installation of a downloaded data file.

Notes in t able below :

1. If the ECU is not able to enter program mode , i.e. the criteria for entering program mode are not fulfilled at the moment of the routineControl request, this shall be indicated by the corresponding RoutineStatusRecord response parameter.

Possible circumstances that can cause an ECU to reject to enter program mode when requested are for example certain safety conditions that may need to be satisfied (e.g. vehicle shall not be moving or engine shall not be running).

If the ECU need time to store data to non-volatile memory before making a mode transition, caused by the service DiagnosticSessionControl (programmingSession), this shall be performed within the time allowed for the mode change as given by the requirement for diagnostic availability specified in reference *[Data_1] UDS Services*.

2. If the ECU is delivered to OEM with the complete programmable memory already erased, the routine shall include an "already erased" detection in order to reduce the overall software download time. If the complete programmable memory is "already erased" and all other preconditions are met (e.g. SBL is downloaded and activated), the routine shall not perform any erase operation and immediately send a final positive response message with RoutineCompleted=1 (Routine is completed) . Once one or more bytes of the ECU programmable memory area has been programmed, the "already erased" status shall be invalidated. Any erase memory requests when the "already erased" status is invalid shall erase the requested memory area .

3. The final positive response message shall be sent when the SBL has been activated, i.e. from the SBL.

If the SBL is already activated at the time of the request the ECU shall respond with a positive response message with RoutineCompleted=0 (Routine is completed).

4. The routineContolOptionRecord has a dynamical length and shall be equal to the file system location of the ECU where the data file is stored included with the file name with extension.

Requirements from this section are specified in section "Requirements from section Supported routines".

4.5.1.1.4.2.3 Response times

General response time requirements for the services to control routines are defined in *[Data_1] UDS Services*.

Exceptions from the general requirements shall be applied according to the table below .

The timing parameters P4 $_{server_max}$ and P2 $_{Server_max}$ are defined in *[Data_2] UDS Session*.

Table : Requirements on P4 $_{server_max}$ for routines

Item	Routine		P4 $_{server_max}$ ms	Notes
	Name	Routine identifier, hex		
1	Check Complete & Compatible	0205	P2 $_{Server_Max}$	
2	Check Programming Pre-conditions	0206	P2 $_{Server_Max}$	
3	Erase Memory	FF00	60000	1
4	Software installation	D000	P2 $_{Server_Max}$	

5	Check Memory	0212	2000	

Notes in the table above:
1. The Erase Memory routine may increase P4 $_{server_max}$ by an additional 20s for each megabyte range of data over 1 megabyte that is requested to be erased. For example, an Erase Memory request with a memorySize value between 1 and 2 megabytes may utilize a P4 $_{server_max}$ time of 80s and an Erase Memory request with a memorySize value between 2 and 3 megabytes may utilize a P4 $_{server_max}$ time of 100s.

Requirements from this section are specified in section "Requirements from section Response times".

4.5.2 UDS Services

4.5.2.1 UDS Services

4.5.2.1.1 Diagnostic service requirements

Diagnostic services shall be supported by the ECU as specified in this section. Note that r equirements on diagnostic services are defined for both diagnostic application software (i.e. software that execute services in other sessions than the programmingSession) and bootloader software (i.e. software that execute services in the programmingSession).

The diagnostic services required by this document are defined by *[Services_9] ISO 14229-1*. This section specifies additions and restrictions that are compliant to *[Services_9] ISO 14229-1.*

Additionally requirements on diagnostic services, which are network dependent, are specified in the implementation specification for the respective networks (e.g. internal specifications UDSonCAN, UDSonFR and UDSonIP).

The diagnostic services that shall be supported by the ECU in the safetySystemDiagnosticSession are specified by *[Services_10] ISO 26021-1 and [Services_11] ISO 26021-2 "End-of-life activation of on-board pyrotechnic devices"*

The ECU is not allowed to support other diagnostic services without approval from CEVT Electrical Architecture.

If the ECU implements safety requirements with an ASIL higher than QM, it shall be ensured that the diagnostic services supported by the ECU in all sessions except in programmingSession cannot violate any of those safety requirements, i.e. to demonstrate independence as required by *[Services_12] ISO-26262*. Exception to this requirement is allowed only when approved by CEVT Electrical Architecture and will require that the diagnostic services are developed according to *[Services_12] ISO-26262* and the allocated ASIL of those safety requirement for which independence cannot be demonstrated.

2357 v3 **Diagnostic service specification**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define standardized diagnostic services,

Source:
Verification Method: Analysis
Legacy ID: 74108v1

The ECU shall be compliant to *[Services_9] ISO 14229-1: Road vehicles – Diagnostics systems – Diagnostic services.*

2356 v2 Allowed diagnostic services

Version history:
(2):

Purpose: Define which diagnostic services to allow. Ensures all suppliers uses the same diagnostic services as well as preventing supplier from using not allowed diagnostic services.
Source: Section "Diagnostic service requirements"
Verification Method: Test
Legacy ID: 74109v1

The ECU is not allowed to support other diagnostic services than specified in section *Diagnostic service requirements* of the applicable implementation specification for the respective networks and by the *[Services_11] ISO 26021-2 "End-of-life activation of on-board pyrotechnic devices"* unless approved by CEVT Electrical Architecture.

2358 v2 Safety requirements

Version history:
(2):

Purpose: Implementation of diagnostics on safety related ECU must comply with ISO26262
Source: Section "Diagnostic service requirements"
Verification Method: Analysis
Legacy ID: 74110v1

If the ECU implements safety requirements with an ASIL higher than QM, it shall be ensured that the diagnostic services supported by the ECU in all sessions except
in programmingSession cannot violate any of those safety requirements, i.e. to demonstrate independence as required by *[Services_12] ISO-26262*. Exception to this requirement is allowed only when approved by CEVT Electrical Architecture and will require that the diagnostic services are developed according to *[Services_12] ISO-26262* and the allocated ASIL of those safety requirement for which independence cannot be demonstrated

4.5.2.1.1.1 Supported services

Diagnostic services shall be supported by the ECU and enabled in diagnostic sessions according to the tables in the following sub sections.

Abbreviations in the table Requirements on diagnostic services that shall be supported by the ECU:
MC1= Mandatory if the ECU is not emission related. Mandatory if the ECU is emission related

and support “calibration DTCs” (one or more). The emission related ECU shall in this case support the groupOfDTC set to value of a “calibration DTC” that is supported by the ECU.
If the ECU is emission related the ECU is allowed to support the groupOfDTC set to value of a non-emission DTCs other than a “calibration DTC”.
If the ECU is emission related the ECU is not allowed to support the groupOfDTC set to value of an emission related DTC.
A “calibration DTC” is a non-emission related DTC that indicates that a specific action (e.g. request a diagnostic service for initiating or calibrating a function controlled by the ECU) that must be performed as a part of the car manufacturing process has not yet been performed and that shall be cleared (by the diagnostic service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of the “calibration DTC”) as soon as the action has been completed.
MC2= Mandatory if the ECU is emission related. Not allowed if the ECU is not emission related.
MC3= Mandatory if the dataRecord consists of more than one parameter (i.e. the dataRecord is packeted or bitmapped). Not allowed if the dataRecord consists of one parameter.
MC4= Mandatory if *[Services_1] UDS Data* specifies that the value of a data record, pointed out by a dataIdentifier, shall be possible to write (by diagnostic service specified by this document). Otherwise, the service WriteDataByIdentifier may be supported by the ECU in one, some or all sessions. When in programming session WriteDataByIdentifier shall only be supported in the secondary bootloader.
MC5= Mandatory if the ECU is emission related and support permanent DTCs.

4.5.2.1.1.1.1 Diagnostic and Communication Management Functional Unit

Index	Session		Sub-function		Data parameters		Session			Notes
	Name	Hex	Name	Hex	Name	Hex	Default	Extended Diagnostic	Progra-mming	
Diagnostic and Communication Management Functional Unit										
1	DiagnosticSessionControl	10					M	M	MPS	1
2			diagnosticSessionType - defaultSession	01, 81	N/A	N/A	M	M	MPS	
3			diagnosticSessionType - prograrnmingSession	02, 82			M	M	MPS	
4			diagnostcSessionType - extendedDiagnosticSessi	03, 83			M	M	N	
5	ECUReset	11					M	M	MPS	
6			resetType - hardReset	01, 81	N/A	N/A	M	M	MPS	
7	SecurityAccess	27					N	O	MPS	2
8			accessType - requestSeed	01, 81	N/A	N/A	N	N	MPS	
9				03-1F, 83-9F			N	O	N	
10				21-3F, AI-BF			N	O	N	
11			accessType - sendKey	02, 82	Three Byte Key secuntyKey -	000000-FFFFFF	N	N	MPS	
12				04-20, 84-A0	secuntyKey - Three Byte Key	000000-FFFFFF	N	O	N	
13				22-40, A2-00	secuityKey — Number of bytes of the key is	Specified by impl.	N	O	N	
14	TesterPresent	3E	zeroSubFunction	00, 80	N/A	N/A	M	M	MPS	

Notes
1. Refer to section *Additional session requirements* for additional session requirements.
2. a.The values of requestSeed (and corresponding sendKey) are divided into the range 0x01-0x1F, 0x83-0x9F (0x02-0x20, 0x84-0xA0) and the range 0x21-0x3F, 0xA1-0xBF (0x22-0x40, 0xA2-0xC0). The first mention range is intended for protecting diagnostic services and data used by CEVT tools while last mention range is intended for protecting diagnostic services and data used during development only.
   - The first range mention above shall use the OEM specified SecurityAccess algorithm.
   - The last range mention above shall use a SecurityAccess algorithm that is provided by the implementer. In order to use this range, an approval by CEVT Electrical Arcitecture is required. Note that CEVT tools are not required to support the range.
   b. Whenever the ECU has responded with a seed on a requestSeed request, the seed shall be valid until:
   - Any new securityAccess request is received except when the new request has a sub-function that is not supported by the ECU and, of course, a valid request with sub-function sendKey which first unlocks the SecurityAccess level and then invalidates the seed.
   - ECU session transition regardless if it is to the same or another session.

2495 v2 **DiagnosticSessionControl (10)**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76123v1

The service diagnosticSessionControl shall be supported by the ECU in defaultSession, extendedDiagnosticSession and programmingSession, both primary and secondary bootloader.

2496 v2 **DiagnosticSessionControl (10) defaultSession (01, 81)**

Version history:
(2):

Purpose: The ECU shall support a method in which the tester can make the ECU revert back to default session.
Source:
Verification Method: Test
Legacy ID: 76132v1

The service diagnosticSessionControl – defaultSession shall be supported by the ECU in defaultSession, extendedDiagnosticSession and programmingSession, both primary and secondary bootloader.

2498 v2 **DiagnosticSessionControl (10) programmingSession (02, 82)**

Version history:
(2):

Purpose: It shall be possible to re-program any ECU on the public network.
Source:
Verification Method: Test
Legacy ID: 76133v1

The service diagnosticSessionControl – programmingSession shall be supported by the ECU in defaultSession, extendedDiagnosticSession and programmingSession, both primary and secondary bootloader.

2497 v1 DiagnosticSessionControl (10) extendedDiagnosticSession (03, 83)

Version history:
(1):

Purpose: ExtendedDiagnosticSession shall be supported since not all services used by the service tools can be performed from defaultSession.
Source:
Verification Method: Test
Legacy ID: 76134v0

The service diagnosticSessionControl – extendedDiagnosticSession shall be supported by the ECU in the defaultSession and the extendedDiagnosticSession and shall not be supported in the programmingSession.

2499 v1 DiagnosticSessionControl (10) - sessionParameterRecord

Version history:
(1):

Purpose: All ECUs shall always use a common P2 time to make timeout handling easier on the client side
Source:
Verification Method: Test
Legacy ID: 76136v0

The ECU shall not use the service diagnosticSessionControl to alter the values of $P2_{Server_max}$ and $P2^*_{Server_max}$ from the values specified in *[Services_2] UDS Session*. The response message data parameter sessionParameterRecord shall always contain these fixed values when the defaultSession, extendedDiagnosticSession or programmingSession is requested.

2500 v4 ECUReset (11)

Version history:
(4):

Purpose: ECU reset is used in the SWDL process and may be useful when testing an ECU.
Source:
Verification Method: Test
Legacy ID: 76139v3

The service ECUReset shall be supported by the ECU in the programmingSession, both primary and secondary bootloader and in the default and the extended session.

2501 v1 ECUReset (11) - hardReset (01, 81)

Version history:
(1):

Purpose: During testing resetting an ECU might be used for several purposes like for example if the tester wishes to use downloaded configuration data right away.
Source:
Verification Method: Test
Legacy ID: 76140v0

The service ECUReset – hardReset shall be supported by the ECU in all sessions where the ECU supports the service ECUReset.

2502 v2 SecurityAccess (27)

Version history:
(2):

Purpose: Define availability of SecurityAccess service.
Source:
Verification Method: Test
Legacy ID: 76141v1

The service SecurityAccess service shall be supported by the ECU in the programmingSession, both primary and secondary bootloader and may be supported in the extendedDiagnosticSession and shall not be supported in the defaultSession.

2506 v3 SecurityAccess (27) - securityAccess algorithm and constant in the lower requestSeed range

Version history:
(3):

Purpose: There shall be an ECU independent method in unlocking the common diagnostic procedures.
Source:
Verification Method: Test
Legacy ID: 76142v2

The requestSeed range 0x01-0x1F, 0x83-0x9F and corresponding sendKey range 0x02-0x20, 0x84-0xA0 shall use the OEM standardized SecurityAccess algorithm sp ecified in reference *[Services_3] Security access algorithm*.

2507 v3 SecurityAccess (27) - securityAccess algorithm and constant in the higher requestSeed range

Version history:
(3):

Purpose: The implementer shall have an ECU dependent method in unlocking the common diagnostic procedures
Source:
Verification Method: Test
Legacy ID: 76144v2

The requestSeed range 0x21-0x3F, 0xA1-0xBF and corresponding sendKey range 0x22-0x40, 0xA2-0xC0 shall use a SecurityAccess algorithm that is provided by the implementer. The number of bytes of the data parameter securityKey is specified by the implementer. In order to use this range, an approval by CEVT Electrical Architecture is required. Note that CEVT tools are not required to support the range.

2511 v2 **SecurityAccess (27) - valid seed**

Version history:
(2):

Purpose: Define so that tester and ECU has a common understanding of when a seed is valid.
Source:
Verification Method: Test
Legacy ID: 76162v1

Whenever the ECU has responded with a seed on a requestSeed request, the seed shall be valid until:
- Any new securityAccess request is received except when the new request has a sub-function that is not supported by the ECU and, of course, a valid request with sub-function sendKey which first unlocks the SecurityAccess level and then invalidates the seed.
- ECU session transition regardless if it is to the same or another session.

2503 v2 **SecurityAccess (27) - requestSeed (01, 81)**

Version history:
(2):

Purpose: programmingSession shall be protected by SecurityAccess
Source:
Verification Method: Test
Legacy ID: 76163v1

The service securityAccess – requestSeed 0x01 shall be supported by the ECU in the programmingSession, both primary and secondary bootloader and shall not be supported by the other sessions.

2504 v1 **SecurityAccess (27) - requestSeed (03-1F, 83-9F)**

Version history:
(1):

Purpose: Some common diagnostic procedures (for example I/O control) shall be

protected with SecurityAccess and shall be possible to unlock via OEM specific tools.

Source:
Verification Method: Test
Legacy ID: 76164v0

The service securityAccess – requestSeed in the range 0x03-0x1F may be supported by the ECU in the extendedDiagnosticSession and shall not be supported in the other sessions.

2505 v1 **SecurityAccess (27) - requestSeed (21-3F, A1-BF)**

Version history:
(1):

Purpose: ECU specific SecurityAccess algorithms may be used but shall be placed in a separate key/seed range to prevent confusion with the general SecurityAccess algorithm.
Source:
Verification Method: Test
Legacy ID: 76165v0

The service securityAccess – requestSeed in the range 0x21-0x3F may be supported by the ECU in the extendedDiagnosticSession and shall not be supported in the other sessions.

2508 v1 **SecurityAccess (27) - sendKey (02, 82)**

Version history:
(1):

Purpose: programmingSession shall be protected by SecurityAccess
Source:
Verification Method: Test
Legacy ID: 76166v0

The service securityAccess – sendKey 02 shall be supported by the ECU in the programmingSession and shall not be supported by the other sessions. The data parameter securityKey shall consist of three bytes.

2509 v1 **SecurityAccess (27) - sendKey (04-20, 84-A0)**

Version history:
(1):

Purpose: Some common diagnostic procedures (for example I/O control) shall be protected with SecurityAccess and shall be possible to unlock via OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76167v0

The service securityAccess – sendKey in the range 0x04-0x20 may be supported by the ECU in

the extendedDiagnosticSession and shall not be supported in the other sessions. The data parameter securityKey shall consist of three bytes.

2510 v2 **SecurityAccess (27) - sendKey (22-40, A2-C0)**

Version history:
(2):

Purpose: ECU specific SecurityAccess algorithms may be used but shall be placed in a separate key/seed range to prevent confusion with the general SecurityAccess algorithm.

Source:

Verification Method: Test

Legacy ID: 76168v1

The service securityAccess – sendKey in the range 0x22-0x40 may be supported by the ECU in the extendedDiagnosticSession and shall not be supported in the other sessions. The number of bytes of the data parameter securityKey is specified by the implementer.

2512 v2 **TesterPresent (3E) - zeroSubFunction (00, 80)**

Version history:
(2):

Purpose: Keep ECUs in a desired session for a longer period of time. Although TesterPresent is not necessary for keeping an ECU in defaultSession, all ECUs shall support TesterPresent in default session since the response of the TesterPresent can be used as an indication of an operational ECU.

Source:

Verification Method: Test

Legacy ID: 76169v1

The service testerPresent – zeroSubFunction shall be supported by the ECU in defaultSession, extendedDiagnosticSession and programmingSession, both primary and secondary bootloader.

4.5.2.1.1.1.2 Data Transmission Functional Unit

Index	Session		Sub-function		Data parameters		Session			Notes
	Name	Hex	Name	Hex	Name	Hex	Default	Extended Diagnostic	Programming	
Data Transmission Functional Unit										
15	ReadDataByIdentifier	22	N/A	N/A	dataIdentifier(-s)	See note 14	M	M	MPS	3
16	ReadMemoryByAddress	23					O	O	N	17
17			N/A	N/A	addressAndLengthFormatIdentifier	14	M	M	N	
18						15 24, 25	O	O	N	
19					memoryAddress	Specified by impl.	M	M	N	
20					memorySize	Specified by impl.	M	M	N	
21	ReadDataByPeriodicIdentifier	2A					N	O	D	4
22			N/A	N/A	transmissionMode - slow	01		O	D	
23					transmissionMode - medium	02	N	O	D	
24					TansmissionMode - fast	03	N	C	D	
25					TansmissionMode - stop	04	N	M	D	
26					penodicDataldendfier(s)	See note 14	N	O	D	
27	DynamicallyDefineDataIdentifier	2C					O	O	N	5, 17
28			defineByIdentifier	01, 81			O	O	N	
29					dynamicallyDefinedDataIdentifier	See note 14	M	M	N	
30			.		sourceDataIdentifier	See note 14	M	M	N	
31					positionInSourceDataRecord	01-FF	M	M	N	
32					memorySize	01-FF	M	M	N	
33			defineByMemoryAddress	02, 82			O	O	N	
34					dynamicallyDefinedDataIdentifier	See note 14	M	M	N	
35					addressAndLengthFormatIdentifier	14	M	M	N	
36						15 24, 25	O	O	N	
37					memoryAddress	Specified by impl.	M	M	N	
38					memorySize	Specified by impl.	M	M	N	
39			clearDynamicallyDefinedDataIdentifier	03, 83			M	M	N	
40					dynamicallyDefinedDataIdentifier	See note 14	O	O	N	
41	WriteDataByIdentifier	2E					MC4	MC4	OS	
42			N/A	N/A	. dataIdentifier	See note 14	M	M	MS	
43							M	M	MS	
44	WriteMemoryByAddress	3D					O	O	N	17
45			N/A	N/A	addressAndLengthFormatIdentifier	14	M	M	N	
46						15 24, 25	O	O	N	
47					memoryAddress	Spec. by impl.	M	M	N	
48					memorySize	Spec. by impl.	M	M	N	
49					dataRecord	Spec. by impl.	M	M	N	

Notes:

3. Minimum 10 dataIdentifiers, or as many dataIdentifiers as implemented, in one single ReadDataByIdentifier request shall be supported by the ECU in default and extended diagnostic session. The ECU is allowed to reject requests with multiple dataIdentifiers when the response is larger than 61 bytes. For example, lager than 1-byte ReadDataByIdentifier response SID (Service Identifier) + 10·2-byte dataIdentifier + 10·4-byte dataRecords. In programmingSession only one dataIdentifier needs to be supported.
4. a. The value (the transmission rate) for the transmissionModes shall be defined by the implementer. The transmissionMode fast may be "as fast as possible", i.e. must not be fixed, and may change over time.
   b. Only response message type #2 is allowed. See also *[Services_9] ISO 14229-1* and the implementation specification for respective network (e.g. UDSonCAN, UDSonFR and UDSonIP)for a detailed description of response message type #2.
5. The dynamically defined data records that the ECU supports shall be kept until they are cleared by the service DynamicallyDefineDataIdentifier – clearDynamicallyDefinedDataIdentifier.
14. Defined in *[Services_1] UDS Data.*
17. The addressAndLengthFormatIdentifier values 0x15 and 0x25 are intended for memory addresses there the most significant byte of the address is used as a memoryIdentifier. The definition of the memoryIdentifier is specified by the implementer.

2472 v2 **ReadDataByIdentifier (22) - dataIdentifier(-s)**

Version history:
(2):

Purpose: It shall be possible to read data from all ECUs
Source:
Verification Method: Test
Legacy ID: 76170v1

The service readDataByIdentifer with the data parameter dataIdentifier(-s) shall be supported by the ECU in defaultSession, extendedDiagnosticSession and programmingSession, both primary and secondary bootloader. The value of the dataIdentifier(s) is defined by *[Services_1] UDS Data.*

2473 v3 **ReadDataByIdentifier (22) - multiple identifiers with one request**

Version history:
(3):

Purpose: The manufacturing and aftersales tool supports reading several DID¿s simultaneously from one ECU. Since the time for each request will be to long, the ECU shall be able to package several data record in one response message
Source:
Verification Method: Test
Legacy ID: 76171v2

Minimum 10 dataIdentifiers, or as many dataIdentifiers as implemented, in one single ReadDataByIdentifier request shall be supported by the ECU in default and extended

diagnostic session. The ECU is allowed to reject requests with multiple dataIdentifiers when the response is larger than 61 bytes. In programmingSession only one dataIdentifier needs to be supported.

2481 v1 **ReadMemoryByAddress (23)**

Version history:
(1):

Purpose: ReadMemoryByAddress shall primarily be used during development or for validation data written by WriteMemoryByAddress.
Source:
Verification Method: Test
Legacy ID: 76172v0

The readMemoryByAddress service may be supported by the ECU in defaultSession and extendedDiagnosticSession and shall not be supported in programmingSession.

2482 v1 **ReadMemoryByAddress (23) - addressAndLengthFormatIdentifier (14)**

Version history:
(1):

Purpose: To make it easier for tools one standardized request shall be supported by all ECUs.
Source:
Verification Method: Test
Legacy ID: 76173v0

The readMemoryByAddress service with the data parameter addressAndLengthFormatIdentifier set to a value of 0x14shall be supported by the ECU in all sessions where the ECU supports the service readMemoryByAddress.

2483 v1 **ReadMemoryByAddress (23) - addressAndLengthFormatIdentifier (24)**

Version history:
(1):

Purpose: Tools shall support addressAndLengthFormatIdentifier 0x24 for when there is a larger amount of data to be read (> 255).
Source:
Verification Method: Test
Legacy ID: 76177v0

The readMemoryByAddress service with the data parameter addressAndLengthFormatIdentifier set to a value of 0x24 may be supported by the ECU in all sessions where the ECU supports the service readMemoryByAddress.

2484 v1 **ReadMemoryByAddress (23) - memoryAddress**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76421v0

The service readMemoryByAddress with the data parameter memoryAddress shall be supported by the ECU in all sessions where the ECU supports the service readMemoryByAddress. The values of the memoryAddress shall be defined by the implementer

2485 v1 ReadMemoryByAddress (23) - memorySize

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76441v0

The service readMemoryByAddress with the data parameter memorySize shall be supported by the ECU in all sessions where the ECU supports the service readMemoryByAddress. The values of the memorySize shall be defined by the implementer.

2474 v2 ReadDataByPeriodicIdentifier (2A)

Version history:
(2):

Purpose: To make it possible to read out data with periodic transmission from ECUs.
Source:
Verification Method: Test
Legacy ID: 76442v1

The service readDataByPeriodicIdentifier may be supported by the ECU in extendedDiagnosticSession and shall not be supported in defaultSession.

2476 v1 ReadDataByPeriodicIdentifier (2A) - response message type

Version history:
(1):

Purpose: Define one method to be used for all periodic identifiers
Source:
Verification Method: Test
Legacy ID: 76443v0

Only response message type #2 is allowed. See also *[Services_9] ISO 14229-1*, and the implementation specification for the respective network (e.g. UDSonCAN, UDSonFR and UDSonIP) for a detailed description of response message type #2.

2479 v1 **ReadDataByPeriodicIdentifier (2A) - transmissionMode slow (01)**

Version history:
(1):

Purpose: Define the transmissionMode slow
Source:
Verification Method: Test
Legacy ID: 76444v0

The service readDataByPeriodicIdentifier with the data parameter transmissionMode slow may be supported by the ECU in all sessions where the ECU supports the service readDataByPeriodicIdentifier. The value of the transmission rate in the transmissionMode slow shall be defined by the implementer.

2478 v1 **ReadDataByPeriodicIdentifier (2A) - transmissionMode medium (02)**

Version history:
(1):

Purpose: Define the transmissionMode medium
Source:
Verification Method: Test
Legacy ID: 76445v0

The service readDataByPeriodicIdentifier with the data parameter transmissionMode set to medium may be supported by the ECU in all sessions where the ECU supports the service readDataByPeriodicIdentifier. The value of the transmission rate in the transmissionMode medium shall be defined by the implementer.

2477 v2 **ReadDataByPeriodicIdentifier (2A) - transmissionMode fast (03)**

Version history:
(2):

Purpose: Define the transmissionMode fast
Source:
Verification Method: Test
Legacy ID: 76446v1

The service readDataByPeriodicIdentifier with the data parameter transmissionMode set to fast may be supported by the ECU in all sessions where the ECU supports the service readDataByPeriodicIdentifier. The value of the transmission rate in the transmissionMode fast shall be defined by the implementer. The transmissionMode fast is allowed to be defined as "as fast as possible" i.e. must not be fixed, it may change over time (e.g. be CPU load dependent).

2480 v1 **ReadDataByPeriodicIdentifier (2A) - transmissionMode stop (04)**

Version history:
(1):

Purpose: Shall be possible to stop reading periodic identifiers without for example

changing session since if a session change is done a lot of other functionality might reset as well.

Source:
Verification Method: Test
Legacy ID: 76447v0

The service readDataByPeriodicIdentifier with the data parameter transmissionMode set to stop shall be supported by the ECU in all sessions where the ECU supports the service readDataByPeriodicIdentifier.

2475 v2 ReadDataByPeriodicIdentifier (2A) - periodicDataIdentifier

Version history:
(2):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76448v1

The service readDataByPeriodicIdentifier with the data parameter periodicDataIdentifier may be supported by the ECU as specified by *[Services_9] ISO 14229-1*, in all sessions where the ECU supports the service readDataByPeriodicIdentifier.

2457 v2 DynamicallyDefineDataIdentifier (2C)

Version history:
(2):

Purpose: In some cases DynamicallyDefineDataIdentifers is used for testing the ECUs.
Source:
Verification Method: Test
Legacy ID: 76449v1

The service dynamicallyDefineDataIdentifier may be supported by the ECU in defaultSession and extendedDiagnosticSession and shall not be supported in programmingSession .

2461 v1 DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01, 81)

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76450v0

The service dynamicallyDefineDataIdentifier - defineByIdentifier may be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier.

2462 v1 **DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - dynamic...**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76461v0

The service dynamicallyDefineDataIdentifier - defineByIdentifier with the data parameter dynamicallyDefinedDataIdentifier shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByIdentifier. The values of the dynamicallyDefinedDataIdentifier are defined by *[Services_1] UDS Data*.

2465 v1 **DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - sourceD...**

Version history:
(1):

Purpose: Compliance with OEM specific tools
Source:
Verification Method: Test
Legacy ID: 76462v0

The service dynamicallyDefineDataIdentifier - defineByIdentifier with the data parameter sourceDataIdentifier shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByIdentifier. The values of the sourceDataIdentifier are defined by *[Services_1] UDS Data*.

2464 v1 **DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - positio...**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76463v0

The service dynamicallyDefineDataIdentifier - defineByIdentifier with the data parameter positionInSourceDataRecord shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByIdentifier.

2463 v1 **DynamicallyDefineDataIdentifier (2C) - defineByIdentifier (01) - memorySize**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test

Legacy ID: 76464v0

The service dynamicallyDefineDataIdentifier - defineByIdentifier with the data parameter memorySize shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByIdentifier.

2471 v1 DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02, 82)

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76465v0

The service dynamicallyDefineDataIdentifier - defineByMemoryAddress may be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier.

2468 v1 DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - dynamicallyDefinedDataIdentifier

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76466v0

The service dynamicallyDefineDataIdentifier - defineByMemoryAddress with the data parameter dynamicallyDefinedDataIdentifier shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByMemoryAddress. The values of the dynamicallyDefinedDataIdentifier are defined by *[Services_1] UDS Data*.

2466 v1 DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - addressAndLengthFormatIdentifier

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76468v0

DynamicThe service dynamicallyDefineDataIdentifier - defineByMemoryAddress with the data parameter addressAndLengthFormatIdentifier set to the value 0x24 may be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByMemoryAddress.

2467 v1 DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - addressAndLengthFormatIdentifier

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76467v0

The service dynamicallyDefineDataIdentifier - defineByMemoryAddress with the data parameter addressAndLengthFormatIdentifier set to the value 0x14 shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByMemoryAddress.

2469 v1 DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - memo...

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76481v0

The service dynamicallyDefineDataIdentifier - defineByMemoryAddress with the data parameter memorySize shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByMemoryAddress. The values of the memorySize shall be defined by the implementer.

2470 v1 DynamicallyDefineDataIdentifier (2C) - defineByMemoryAddress (02) - memo...

Version history:
(1):

Purpose: Compliance with OEM specific tools
Source:
Verification Method: Test
Legacy ID: 76469v0

The service dynamicallyDefineDataIdentifier - defineByMemoryAddress with the data parameter memoryAddress shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier - defineByMemoryAddress. The values of the memoryAddress shall be defined by the implementer.

2458 v2 DynamicallyDefineDataIdentifier (2C) - clear of dynamicallyDefineDataIdentifier

Version history:
(2):

Purpose: Clear of dynamicallyDefineDataIdentifier.

Source:
Verification Method: Test
Legacy ID: 76485v1

The dynamically defined data records that the ECU supports shall be kept until they are cleared by the service DynamicallyDefineDataIdentifier - clearDynamicallyDefinedDataIdentifier.

2459 v1 **DynamicallyDefineDataIdentifier (2C) - clearDynamicallyDefinedDataIdenti...**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76484v0

The service dynamicallyDefineDataIdentifier - clearDynamicallyDefinedDataIdentifier withthe data parameter dynamicallyDefinedDataIdentifier shall be supported by the ECU as specified by *[Services_10] Road vehicles - End-of-life activation of on-board pyrotechnic devices - Part 1*, in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier. The values of the dynamicallyDefinedDataIdentifier are defined by Defined in *[Services_1] UDS Data.*.

2460 v1 **DynamicallyDefineDataIdentifier (2C) - clearDynamicallyDefinedDataIdenti...**

Version history:
(1):

Purpose: Standardised clear procedure to have compliance with OEM specific tools
Source:
Verification Method: Test
Legacy ID: 76482v0

The service dynamicallyDefineDataIdentifier - clearDynamicallyDefinedDataIdentifier shall be supported by the ECU in all sessions where the ECU supports the service dynamicallyDefineDataIdentifier.

2486 v3 **WriteDataByIdentifier (2E)**

Version history:
(3):

Purpose: Primary purpose for using the service WriteDataByIdentifier is to store configuration parameters on an ECU.
Source:
Verification Method: Test
Legacy ID: 76486v2

The service WriteDataByIdentifier shall be supported if *[Services_1] UDS Data* specifies that the value of a data record, pointed out by a dataIdentifier, shall be possible to write (by diagnostic service specified by this document). Otherwise, the service WriteDataByIdentifier

may be supported by the ECU in one, some or all sessions. When in programming session WriteDataByIdentifier shall only be supported in the secondary bootloader.

2487 v1 **WriteDataByIdentifier (2E) - dataIdentifier**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76487v0

The service WriteDataByIdentifier with the data parameter dataIdentifier(-s) shall be supported by the ECU in all sessions where the ECU supports the service WriteDataByIdentifier. The values of the dataIdentifier are defined by *[Services_1] UDS Data*.

2488 v1 **WriteDataByIdentifier (2E) - dataRecord**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76488v0

The service WriteDataByIdentifier with the data parameter dataRecord shall be supported by the ECU in all sessions where the ECU supports the service WriteDataByIdentifier. The values of the dataRecord are defined by the implementer.

2489 v1 **WriteMemoryByAddress (3D)**

Version history:
(1):

Purpose: The service WriteMemoryByAddress may be useful during the development process.
Source:
Verification Method: Test
Legacy ID: 76489v0

The service WriteDataByAddress may be supported by the ECU in defaultSession and extendedDiagnosticSession but not in the programmingSession.

2490 v1 **WriteMemoryByAddress (3D) - addressAndLengthFormatIdentifier (14)**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:

Verification Method: Test
Legacy ID: 76490v0

The service WriteMemoryByAddress with the data parameter addressAndLengthFormatIdentifier set to the value 0x14 shall be supported by the ECU in all sessions where the ECU supports the service WriteMemoryByAddress.

2491 v1 WriteMemoryByAddress (3D) - addressAndLengthFormatIdentifier (24)

Version history:
(1):

Purpose: OEM specific tooling shall support addressAndLengthFormatIdentifier 0x24 for when there is a larger amount of data to write (> 255).
Source:
Verification Method: Test
Legacy ID: 76491v0

The service WriteMemoryByAddress with the data parameter addressAndLengthFormatIdentifier set to the value 0x24 may be supported by the ECU in all sessions where the ECU supports the service WriteMemoryByAddress.

2493 v1 WriteMemoryByAddress (3D) - memoryAddress

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76493v0

The service writeMemoryByAddress with the data parameter memoryAddress shall be supported by the ECU in all sessions where the ECU supports the service writeMemoryByAddress. The values of the memoryAddress shall be defined by the implementer.

2494 v1 WriteMemoryByAddress (3D) - memorySize

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76494v0

The service writeMemoryByAddress with the data parameter memorySize shall be supported by the ECU in all sessions where the ECU supports the service writeMemoryByAddress. The values of the memorySize shall be defined by the implementer.

2492 v1 **WriteMemoryByAddress (3D) - dataRecord**

Version history:
(1):

Purpose: Compliance with OEM specific tools.
Source:
Verification Method: Test
Legacy ID: 76495v0

The service writeMemoryByAddress with the data parameter dataRecord shall be supported by the ECU in all sessions where the ECU supports the service writeMemoryByAddress. The values of the dataRecord shall be defined by the implementer.

4.5.2.1.1.1.3 Stored Data Transmission Functional Unit

Index	Session		Sub-function		Data parameters		Session			Notes
	Name	Hex	Name	Hex	Name	Hex	Default	Extended Diagnostic	Programming	
Stored Data Transmission Functional Unit										
50	ClearDagnosticlacrmation	14					M	M	D	18
51			N/A	N/A	groupOfDTC -Specific DTC	Any 3 byte	MC1	MC1	D	
52					groupOfDTC-AllDTCs in emission related ECU	FFF000	MC2	MC2	D	
53					groupo0fDTC-AllGroups	FFFFFF	M	M	D	
54	ReadDTCInformation 19	19					M	M	D	
55			reportDTCByStatusMask	02, 82			M	M	D	
56					DTCStatusMask	See note 15	M	M	D	
57			reportDTCSnapshotIdentification	03, 83	N/A	N/A	M	M	D	
58			reportDTCSnapshotRecordBy-DTCNumber	04, 84			M	M	D	
59					DTCMaskRecord	See note 14	M	M	D	
60					DTCSnapSnotRecordNumber	FF (=all)	M	M	D	
61						Specific value See note 14	M	M	D	
62			reportGenericExtendedDataRecordBy-DTCNumber	06, 86			M	M	D	
63					DTCMaskRecord	See note 14	M	M	D	
64					DTCExtendedDataRecordNumber	FF (=all)	M	M	D	
65						Specific value See note 14	M	M	D	
66			reportSupportedDTC	0A, 8A	N/A	N/A	O	O	D	
67			reportDTCWithPermanentStatus	15, 95	N/A	N/A	MC5	MC5	D	
^	ReadGenencInformation	AF					O	O	D	11
69			reportGenericSnapshotByDTCNumber	04, 84			M	M	D	
70					DTCMaskRecord	FFFFFF (=all)	M	M	D	
71							M	M	D	
72					DTCSnapshotRecordNumber	FF (=all)	M	M	D	
73					Specific value		M	M	D	
74			reportGenericExtendedDataBy-DTCNumber	06, 86			M	M	D	
75					DTCMaskRecord	FFFFFF (=all)	M	M	D	
76						Specific value See note 14	M	M	D	
I7			.		DTCExtendedDataRecordNumber	FF (=all) Specrbc value	M	M	D	
78						Specific value See note 14	M	M	D	

Notes:

11. Refer to appendix B (section *Appendix B- Additional ReadGenericInformation requirements*) for additional ReadGenericInformation requirements.
14. Defined in *[Services_1] UDS Data.*
18. If the ECU is emission related, the groupOfDTC set to “All Groups” or “All DTCs in emission

related ECU" shall be supported by the ECU only for functional addressed diagnostic request messages. An emission related ECU is not allowed to support this for physical addressed diagnostic request messages. Refer to section Diagnostic message addressing for details regarding functional addressed requests. Note that the requirement "The server shall support its list of diagnostic services regardless of addressing mode (physical, functional addressing type)" specified in reference *[Services_9] ISO 14229-1* shall not apply in this case.

2534 v1 **ClearDiagnosticInformation (14)**

Version history:
(1):

Purpose: Erase DTCs
Source:
Verification Method: Test
Legacy ID: 76496v0

The service ClearDiagnosticInformation shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2537 v3 **ClearDiagnosticInformation (14) - groupOfDTCSpecific DTC**

Version history:
(3):

Purpose: It shall be possible to remove specific DTCs.
Source:
Verification Method: Test
Legacy ID: 76497v2

If the ECU is not emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of any specific DTC shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of a "calibration DTC" shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.
A "calibration DTC" is a non-emission related DTC that indicates that a specific action (e.g. request a diagnostic service for initiating or calibrating a function controlled by the ECU) that must be performed as a part of the car manufacturing process has not yet been performed and that shall be cleared (by the diagnostic service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value of the "calibration DTC") as soon as the action has been completed. This requirement is only applicable if the ECU support "calibration DTCs" (one or more).

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to a value a non-emission related DTC, other than a "calibration DTC", is allowed to be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter

groupOfDTC set to a value of an emission related DTC is not allowed to be supported by the ECU.

2535 v1 ClearDiagnosticInformation (14) - groupOfDTC All DTCs in emission related ECU

Version history:
(1):

Purpose: To shorten the time the ECU is in the workshop or during the assembly process, it shall be possible to clear all DTCs in emission related ECUs with one diagnostic request.

Source:

Verification Method: Test

Legacy ID: 363603v0

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to the value 0xFFF000 – “All DTCs in emission related ECU” shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation but only for functional addressed diagnostic request messages. An emission related ECU is not allowed to support this for physical addressed diagnostic request messages. Refer to section Diagnostic message addressing for details regarding functional addressed requests. Note that the requirement “The server shall support its list of diagnostic services regardless of addressing mode (physical, functional addressing type)” specified in *[Services_9] ISO 14229-1*, shall not apply in this case. Note that this data parameter groupOfDTC – “All DTCs in emission related ECU” is not defined in *[Services_9] ISO 14229-1*.

If the ECU is not emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to the value 0xFFF000 – “All DTCs in emission related ECU” is not allowed to be supported by the ECU.

2536 v2 ClearDiagnosticInformation (14) - groupOfDTC All Groups

Version history:
(2):

Purpose: To shorten the time the ECU is in the workshop or during the assembly process, it shall be possible to clear all DTCs with one diagnostic request.

Source:

Verification Method: Test

Legacy ID: 76498v1

If the ECU is not emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to All Groups shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation.

If the ECU is emission related, the service ClearDiagnosticInformation with the data parameter groupOfDTC set to All Groups shall be supported by the ECU in all sessions where the ECU supports the service ClearDiagnosticInformation but only for functional addressed diagnostic request messages. An emission related ECU is not allowed to support this for physical addressed diagnostic request messages. Refer to section Diagnostic message addressing for details regarding functional addressed requests. Note that the requirement “The server shall

support its list of diagnostic services regardless of addressing mode (physical, functional addressing type)" specified in *[Services_9] ISO 14229-1* shall not apply in this case.

2542 v1 **ReadDTCInformation (19)**

Version history:
(1):

Purpose: Read DTC information.
Source:
Verification Method: Test
Legacy ID: 76499v0

The service ReadDTCInformation shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2543 v2 **ReadDTCInformation (19) - reportDTCByStatusMask (02, 82)**

Version history:
(2):

Purpose: It shall be possible to read out DTCs with a specific status.
Source:
Verification Method: Test
Legacy ID: 76500v1

The service ReadDTCInformation - reportDTCByStatusMask shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2544 v1 **ReadDTCInformation (19) - reportDTCByStatusMask (02, 82) - DTCStatusMask**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76501v0

The service ReadDTCInformation - reportDTCByStatusMask with the data parameter DTCStatusMask shall be supported by the ECU in defaultSesson and extendedDiagnosticSession.

2549 v1 **ReadDTCInformation (19) - reportDTCSnapshotIdentification 03, 83)**

Version history:
(1):

Purpose: Since it's possible to have multiple DTCSnapshot records for one DTC, it must be possible to read out which DTCSnapshot records a specific DTC has.
Source:
Verification Method: Test

Legacy ID: 76502v0

The service ReadDTCInformation - reportDTCSnapshotIdentification shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2550 v2 **ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84)**

Version history:
(2):

Purpose: Snapshot of data values shall be stored along with the DTC when snapshot sampling criteria is fulfilled. This snapshot data shall be possible to read out.
Source:
Verification Method: Test
Legacy ID: 76503v1

The service ReadDTCInformation - reportDTCSnapshotRecordByDTCNumber shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2551 v2 **ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84) - ...**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76504v1

The service ReadDTCInformation - reportDTCSnapshotRecordByDTCNumber with the data parameter DTCMaskRecord shall be supported by the ECU in defaultSession and extendedDiagnosticSession. The values of the DTCMaskRecord are defined by *[Services_1] UDS Data*.

2553 v1 **ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84)-D...**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76505v0

The service ReadDTCInformation - reportDTCSnapshotRecordByDTCNumber with the data parameter DTCSnapShotRecordNumber set to All shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2552 v1 **ReadDTCInformation (19) - reportDTCSnapshotRecordByDTCNumber (04, 84) - ...**

Version history:

(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76506v0

The service ReadDTCInformation - reportDTCSnapshotRecordByDTCNumber with the data parameter DTCSnapShotRecordNumber set to Specific value shall be supported by the ECU in defaultSession and extendedDiagnosticSession. The Specific values of the DTCMaskRecord are defined by *[Services_1] UDS Data*.

2545 v2 ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 84)

Version history:
(2):

Purpose: Extended data, consisting of extended status information, shall be stored along with the DTC. The extended data identified at the time of request shall be possible to read out.
Source:
Verification Method: Test
Legacy ID: 76508v1

The service ReadDTCInformation - reportDTCExtendedDataRecordByDTCNumber shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2546 v1 ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 86...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76509v0

The service ReadDTCInformation - reportDTCExtendedDataRecordByDTCNumber with the data parameter DTCMaskRecord shall be supported by the ECU in defaultSession and extendedDiagnosticSession. The values of the DTCMaskRecord are defined by *[Services_1] UDS Data*.

2547 v1 ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 86...

Version history:
(1):

Purpose:
Source:

Verification Method: Test
Legacy ID: 76510v0

The service ReadDTCInformation - reportDTCExtendedDataRecordByDTCNumber with the data parameter DTCExtendedDataRecordNumber set to 'All' shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2548 v1 ReadDTCInformation (19) - reportDTCExtendedDataRecordByDTCNumber (06, 86...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76512v0

The service ReadDTCInformation - reportDTCExtendedDataRecordByDTCNumber with the data parameter DTCExtendedDataRecordNumber set to a specific value shall be supported by the ECU in defaultSession and extendedDiagnosticSession. The Specific values of the DTCMaskRecord are defined by *[Services_1] UDS Data*.

2555 v1 ReadDTCInformation (19) - reportSupportedDTC (0A)

Version history:
(1):

Purpose: Read out supported DTCs from an ECU
Source:
Verification Method: Test
Legacy ID: 76514v0

The service ReadDTCInformation - reportSupportedDTC may be supported by the ECU in defaultSession and extendedDiagnosticSession.

2554 v2 ReadDTCInformation (19) - reportDTCWithPermanentStatus (15)

Version history:
(2):

Purpose: Fulfill legislated requirements.
Source:
Verification Method: Test
Legacy ID: 76515v1

If the ECU is emission related and support permanent DTCs, the service ReadDTCInformation – reportDTCWithPermanentStatus shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2556 v2 ReadGenericInformation (AF)

Version history:
(2):

Purpose: To be able to read relevant DTC data from DTCs stored in the ECU in a fast and simple way.
Source:
Verification Method: Test
Legacy ID: 76517v1

The service ReadGenericInformation may be supported by the ECU in defaultSession and/or extendedDiagnosticSession.

2557 v2 ReadGenericInformation (AF) - reportDTCSnapshotByDTCNumber (04, 84)

Version history:
(2):

Purpose: Snapshot of data values shall be stored along with the DTC when snapshot sampling criteria is fulfilled. This snapshot data shall be possible to read out.
Source:
Verification Method: Test
Legacy ID: 76519v1

If the service ReadGenericInformation is supported by the ECU, the subfunction reportGenericSnapshotByDTCNumber shall be supported by by the ECU in defaultSession and extendedDiagnosticSession.

2558 v1 ReadGenericInformation (AF) - reportDTCSnapshotByDTCNumber (04, 84) - DT...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76520v0

The data parameter DTCMaskRecord for the service/subfunction ReadGenericInformation - reportDTCSnapshotRecordByDTCNumber shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2559 v1 ReadGenericInformation (AF) - reportDTCSnapshotByDTCNumber (04, 84) - DT...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76524v0

The data parameter DTCSnapShotRecordNumber for the service/subfunction

ReadGenericInformation - reportDTCSnapshotRecordByDTCNumber shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2562 v2 ReadGenericInformation (AF) - reportGenericExtendedDataByDTCNumber (06, 86)

Version history:
(2):

Purpose: Extended data, consisting of extended status information, shall be stored along with the DTC. The extended data identified at the time of request shall be possible to read out.
Source:
Verification Method: Test
Legacy ID: 76527v1

If the service ReadGenericInformation is supported by the ECU, the subfunction reportGenericExtendedDataByDTCNumber shall be supported by by the ECU in defaultSession and extendedDiagnosticSession.

2560 v1 ReadGenericInformation (AF) - reportGenericExtendedDataByDTCNumber (06, ...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76529v0

The data parameter DTCMaskRecord for the service/subfunction ReadGenericInformation - reportGenericExtendedDataByDTCNumber shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2561 v2 ReadGenericInformation (AF) - reportGenericExtendedDataByDTCNumber (06, ...

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76532v1

The data parameter DTCExtendedDataRecordNumber for the service/subfunction ReadGenericInformation - reportGenericExtendedDataByDTCNumber shall be supported by the ECU in defaultSession and extendedDiagnosticSession.

2572 v1 Values of DTCExtendedDataRecordNumber

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76533v0

The values of the data parameter DTCExtendedDataRecordNumber for the service/subfunction ReadGenericInformation - reportGenericExtendedDataByDTCNumber shall be used as defined by reference [1] and the additional value of 0xFF which shall be used to identify all extended data record values supported by the ECU.

2541 v1 **Format of ReadGenericInformation request message**

Version history:
(1):

Purpose: The request message format for the service AF is not defined by ISO and must therefore be clearly defined by this specification.
Source:
Verification Method: Test
Legacy ID: 76535v0

The ECU shall accept request messages for the service ReadGenericInformation that uses the same format as for the service ReadDTCInformation, based on the corresponding sub-function parameter used, as defined in *[Services_9] ISO 14229-1*.

2540 v1 **Definitions of ReadGenericInformation sub-function and data parameters**

Version history:
(1):

Purpose: The definitions of the request message sub-function and data parameters for the service AF are not defined by ISO and must therefore be clearly defined by this specification.
Source:
Verification Method: Test
Legacy ID: 76540v0

The ECU shall accept request message sub-function and data parameters for the service ReadGenericInformation that are defined according to the same definitions as for the service ReadDTCInformation, as defined in *[Services_9] ISO 14229-1*, with the following addition for the data parameter DTCMaskRecord:
If this data parameter equals 0xFFFFFF, the request concerns all DTC's, and data shall be reported for all DTC's, for which data is available, in the positive response.

2570 v1 **Response message format for ReadGenericInformation - reportGenericExtend...**

Version history:
(1):

Purpose: The format of the response message for the service AF is not defined by ISO

and must therefore be clearly defined by this specification

Source:

Verification Method: Test

Legacy ID: 76544v0

If the ECU has received a request message for the service ReadGenericInformation with the sub-function reportGenericExtendedDataByDTCNumber and the ECU shall send a positive response, the ECU shall send a positive response with the response message format as defined by *Requirements on positive response format for service AF 06 (hex).*

2571 v1 **Response message format for ReadGenericInformation - reportGenericSnapsh...**

Version history:
(1):

Purpose: The format of the response message for the service AF is not defined by ISO and must therefore be clearly defined by this specification.

Source:

Verification Method: Test

Legacy ID: 76541v0

If the ECU has received a request message for the service ReadGenericInformation with the sub-function reportGenericSnapshotByDTCNumber and the ECU shall send a positive response, the ECU shall send a positive response with the response message format as defined by *Requirements on positive response format for service AF 04 (hex).*

2538 v2 **Data to be reported in ReadGenericInformation positive repsonse**

Version history:
(2):

Purpose: The amount of data that can be reported in a positive response to the service ReadGenericInformation must be limited to what is relevant to report.

Source:

Verification Method: Test

Legacy ID: 76547v1

A positive response to a request for the service ReadGenericInformation shall only report data from the DTCs for which DTC event data is stored in long-term memory according to *[Services_1] UDS Data*.

2569 v1 **ReadGenericInformation positive response message data parameters definie...**

Version history:
(1):

Purpose: The service ReadGenericInformation is based on the ISO defined service ReadDTCInformation and shall therefore use the same definitions for the response message data parameters

Source:

Verification Method: Test

Legacy ID: 76548v0

The ECU shall use ReadGenericInformation positive response message data parameters according to the same definitions as for the service ReadDTCInformation as defined in *[Services_9] ISO 14229-1*.

2539 v2 Definition of ReadGenericInformation positive response message data para...

Version history:
(2):

Purpose: The service ReadGenericInformation uses additional response message data parameters which are not defined by ISO and must therefore be clearly defined in this specification.
Source:
Verification Method: Test
Legacy ID: 76561v1

The ReadGenericInformation positive response message data parameter DTCStatusIndicators shall be a one byte parameter with the same definition as for the DTCExtendedDataRecordNumber 0x30 defined in *[Services_1] UDS Data*.

2568 v2 ReadGenericInformation negative response code subFunctionNotSupported

Version history:
(2):

Purpose: The negative response codes that must be supported for the service AF are not defined by ISO and must therefore be clearly defined by this specification.
Source:
Verification Method: Test
Legacy ID: 76582v1

If the ECU has received a request message for the service ReadGenericInformation with the sub-function different from 0x04 or 0x06 the ECU shall send a negative response with response code 0x12 - subFunctionNotSupported.

2564 v2 ReadGenericInformation negative response code incorrectMessageLengthOrInvalidFormat

Version history:
(2):

Purpose: The negative response codes that must be supported for the service AF are not defined by ISO and must therefore be clearly defined by this specification.
Source:
Verification Method: Test
Legacy ID: 76583v1

If the ECU has received a request message for the service ReadGenericInformation where the length and/or format of the request message is incorrect the ECU shall send a negative response with response code 0x13 - incorrectMessageLengthOrInvalidFormat.

2567 v2 **ReadGenericInformation negative response code requestOutOfRange**

Version history:
(2):

Purpose: The negative response codes that must be supported for the service AF are not defined by ISO and must therefore be clearly defined by this specification.
Source:
Verification Method: Test
Legacy ID: 76584v1

If the ECU has received a request message for the service ReadGenericInformation where
DTCMaskRecord is different from 0xFFFFFF and different from a supported DTC number
or
the sub-function is 0x04 and DTCSnapshotRecord is different from 0x FF and different from a supported DTCSnapshotRecordNumber
or
the sub-function is 0x06 and DTCExtendedDataRecord is different from 0x FF and different from a supported DTCExtendedDataRecordNumber
the ECU shall send a negative response with response code 0x31 - requestOutOfRange

2563 v3 **ReadGenericInformation negative response code conditionsNotCorrect**

Version history:
(3):

Purpose: The negative response codes that must be supported for the service AF are not defined by ISO and must therefore be clearly defined by this specification
Source:
Verification Method: Test
Legacy ID: 76585v2

If the ECU has received a request message for the service ReadGenericInformation where
DTCMaskRecord = 0xFFFFFF and no DTC event data has been stored for any DTC
or
DTCMaskRecord = a supported DTC number and no DTC event data has been stored for this DTC,
the ECU shall send a negative response with response code 0x22 - conditionsNotCorrect

2566 v2 **ReadGenericInformation negative response code requestCorrectlyReceived-ResponsePending**

Version history:
(2):

Purpose: The negative response codes that must be supported for the service AF are not defined by ISO and must therefore be clearly defined by this specification.
Source:

Verification Method: Test
Legacy ID: 76586v1

If the ECU has received a request message for the service ReadGenericInformation and is not able to respond with a positive response within the required response time the ECU is allowed to send a negative response with the response code 0x78 - requestCorrectlyReceived-ResponsePending.

2565 v2 ReadGenericInformation negative response code order of priority

Version history:
(2):

Purpose: The negative response codes that must be supported for the service AF are not defined by ISO and must therefore be clearly defined by this specification.
Source:
Verification Method: Test
Legacy ID: 76587v1

If the criteria for sending a negative response to a request for the service ReadGenericInformation is fulfilled for more than one negative response code the ECU shall send only one negative response with the response code with the highest priority according to the following priority order (highest priority first): 0x12, 0x13, 0x31, 0x22, 0x78.

2573 v1 Values of DTCMaskRecord

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76530v0

The values of the data parameter DTCMaskRecord for the service/subfunction ReadGenericInformation - reportGenericExtendedDataByDTCNumber shall be used as defined by *[Services_1] UDS Data*, and the additional value of 0xFFFFFF which shall be used to identify all DTC values supported by the ECU.

2574 v1 Values of DTCMaskRecord

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76522v0

The values of the data parameter DTCMaskRecord for the service/subfunction ReadGenericInformation - reportDTCSnapshotRecordByDTCNumber shall be used as defined

by *[Services_1] UDS Data*, and the additional value of 0xFFFFFF which shall be used to identify all DTC values supported by the ECU.

2575 v1 **Values of DTCSnapShotRecordNumber**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76525v0

The values of the data parameter DTCSnapShotRecordNumber for the service/subfunction ReadGenericInformation - reportDTCSnapshotRecordByDTCNumber shall be used as defined by *[Services_1] UDS Data*, and the additional value of 0xFF which shall be used to identify all snapshot record values supported by the ECU.

4.5.2.1.1.1.4 InputOutput Control Functional Unit

Index	Session		Sub-function		Data parameters		Session			Notes
	Name	Hex	Name	Hex	Name	Hex	Default	Extended Diagnostic	Progra-mming	
InputOutput Control Functional Unit										
79	InputOutputControlByIdentifier	2F					N	M	N	6
80			N/A	N/A	dataIdentifier	See note 14	N	M	N	7
81					inputOutputContolParameter-returnControlToECU	00	N	M	N	
82					InputOutputControlParameter-freezeCurrentState	02	N	O	N	
83					InputOutputControlParameter-shortTermAdjustment	03	N	M	N	
84					ControtState-dataRecord	Spec. by impl.	N	M	N	
85					controlMask	See note 15	N	MC3	N	

Notes:
6. a. From the time the request is received by the ECU until the ECU regains control, the substituted values of the data record (referenced by the dataIdentifier) shall remain static. If the substituted values need to change over time, for example due to safety reasons, the service routineControl (0x31) shall be used.
   b. The service InputOutputControlByIdentifier (0x2F) is the only service intended for temporarily controlling inputs (e.g., real world sensors) and outputs (e.g., real world actuators). Unlike service writeDataByIdentifier (0x2E) and writeMemoryByAddress (0x3D), the values substituted by the service 0x2F shall always be immediately effective and revert back to the normal value (as determined by the control system) when the ECU regains control over the values.
7. a. The ECU shall support several consecutive shortTermAdjustment requests without any returnControlToECU (0x00) requests in between.
   b. The ECU shall accept a correct returnControlToECU (0x00) request regardless of the value of the controlMask bits (0 or 1).
   c. The controlState parameter (of the controlStatusRecord) of the positive response shall

always be equal to the size of the dataRecord (referenced by the dataIdentifier) and shall represent the actual current value(s) for each parameter within the dataIdentifier, independent of the value of the controlMask (i.e., the values that would be returned if the data were to be included in a positive response to a service readDataByIdentifier (0x22) request at that time).

14. Defined in *[Services_1] UDS Data.*
15. Defined in *[Services_9] ISO 14229-1.*

2513 v2 InputOutputControlByIdentifier (2F)

Version history:
(2):

Purpose: By controlling inputs and outputs it is possible to make quick fault tracing methods. For example if there is a problem with an electric window, simulating button presses via InputOutputControlByIdentifier can rule out if the problem is on the button side or if on the electric engine side.

Source:

Verification Method: Test

Legacy ID: 76588v1

The service InputOutputControlByIdentifier shall be supported by the ECU in extendedDiagnosticSession if *[Services_1] UDS Data* specifies that the value of a data record, pointed out by a dataIdentifier, shall be possible to controllable by the service InputOutputControlByIdentifier. Otherwise the service InputOutputControlByIdentifier may be supported in extendedDiagnosticSession. The service InputOutputControlByIdentifier shall not be supported by the ECU in defaultSession or programmingSession.

2519 v2 InputOutputControlByIdentifier (2F) - dataIdentifier

Version history:
(2):

Purpose:

Source:

Verification Method: Test

Legacy ID: 76589v1

The service InputOutputControlByIdentifier with data parameter dataIdentifier shall be supported by the ECU in all sessions where the ECU s upports the service InputOutputControlByIdentifier. The dataIdentifier is defined by *[Services_1] UDS Data*.

2521 v2 InputOutputControlByIdentifier (2F) - inputOutputControlParameter returnControlToECU (00)

Version history:
(2):

Purpose: Used for handing back the control of a specific dataIdentifier to the ECU.

Source:

Verification Method: Test

Legacy ID: 76590v1

The service InputOutputControlByIdentifier with data parameter inputOutputControlParameter set to returnControlToECU shall be supported by the ECU in all sessions where the ECU supports the service InputOutputControlByIdentifier.

2520 v2 InputOutputControlByIdentifier (2F) - inputOutputControlParameter freezeCurrentState (02)

Version history:
(2):

Purpose: Freeze input output state when a fault occurs.
Source:
Verification Method: Test
Legacy ID: 76591v1

The service InputOutputControlByIdentifier with data parameter inputOutputControlParameter set to freezeCurrentState may be supported by the ECU in all sessions where the ECU supports the service InputOutputControlByIdentifier.

2522 v2 InputOutputControlByIdentifier (2F) - inputOutputControlParameter shortTermAdjustment (03)

Version history:
(2):

Purpose: Control inputs outputs in order to check for faults.
Source:
Verification Method: Test
Legacy ID: 76592v1

The service InputOutputControlByIdentifier with data parameter shortTermAdjustment shall be supported by the ECU in all sessions where the ECU supports the service InputOutputControlByIdentifier.

2517 v3 InputOutputControlByIdentifier (2F) - controlState data record

Version history:
(3):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76593v2

The service InputOutputControlByIdentifier with data parameter controlState set to dataRecord shall be supported by the ECU in all sessions where the ECU supports the service InputOutputControlByIdentifier. The values of dataRecord are defined specified by the implementer .

2515 v3 InputOutputControlByIdentifier (2F) - controlMask

Version history:
(3):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76594v2

The service InputOutputControlByIdentifier with data parameter controlMask shall be supported by the ECU as specified in *[Services_9] ISO 14229-1* if the dataRecord consists of more than one parameter (i.e. the dataRecord is packeted or bitmapped) but not if the dataRecord consists of only one parameter, in all sessions where the ECU supports the service InputOutputControlByIdentifier.
The length of the controlMask depends on the dataRecord that are defined by *[Services_1] UDS Data*.

2523 v1 InputOutputControlByIdentifier (2F) - Static substituted values

Version history:
(1):

Purpose: Control outputs.
Source:
Verification Method: Test
Legacy ID: 76595v0

From the time the request is received by the ECU until the ECU regains control, the substituted values of the data record (referenced by the dataIdentifier) shall remain static.

2524 v1 InputOutputControlByIdentifier (2F) - Temporarily controlling

Version history:
(1):

Purpose: Control outputs
Source:
Verification Method: Test
Legacy ID: 76596v0

The service InputOutputControlByIdentifier (0x2F) is the only service intended for temporarily controlling inputs (e.g., real world sensors) and outputs (e.g., real world actuators). Unlike service writeDataByIdentifier (0x2E) and writeMemoryByAddress (0x3D), the values substituted by the service 0x2F shall always be immediately effective and revert back to the normal value (as determined by the control system) when the ECU regains control over the values.

2514 v1 InputOutputControlByIdentifier (2F) - Consecutive shortTermAdjustment re...

Version history:
(1):

Purpose: Control outputs
Source:

Verification Method: Test
Legacy ID: 76597v0

The ECU shall support several consecutive shortTermAdjustment requests without any returnControlToECU (0x00) requests in between.

2516 v1 InputOutputControlByIdentifier (2F) - ControlMask bit values of returnCo...

Version history:
(1):

Purpose: Control outputs
Source:
Verification Method: Test
Legacy ID: 76598v0

The ECU shall accept a correct returnControlToECU (0x00) request regardless of the value of the controlMask bits (0 or 1).

2518 v1 InputOutputControlByIdentifier (2F) - controlState parameter of the posi...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76600v0

The controlState parameter (of the controlStatusRecord) of positive response messages shall always be equal to the size of the dataRecord (referenced by the dataIdentifier) and shall represent the actual current value(s) for each parameter within the dataIdentifier, independent of the value of the controlMask (i.e., the values that would be returned if the data were to be included in a positive response to a readDataByIdentifier request at that time).

4.5.2.1.1.1.5 Remote Activation of Routine Functional Unit

Index	Session		Sub-function		Data parameters		Session			Notes
	Name	Hex	Name	Hex	Name	Hex	Default	Extended Diagnostic	Progra-mming	
Remote Activation of Routine Functional Unit										
86	RoutineControl	31					M	M	MPS	8
87			RoutineControlType-startRoutine	01, 81			M	M	MPS	
88					routineIdentifier	See note 14	M	M	MPS	
89	RoutineControl 31				routineControlOptionRecord	See note 16	O	O	MPS	
90			RoutineConlrolType-stopRoutine	02, 82			O	O	OPS	
91					routineIdentifier	See note 14	M	M	MPS	
92					routineControlOptionRecord	See note 16	O	O	OPS	
93			RoutineControlType - requestRoutineResults	03, 83			O	O	OPS	
94					routineIdentifier		M	M	MPS	

Notes:
8. In defaultSession the only allowed RoutineControl type is type 1- Short routine. Refer to section *Additional routine requirements* for additional routine requirements. Note that whether or not the ECU shall support a specific sub-function depends on the routine type and is specified by section *Additional routine requirements.*
14. Defined in *[Services_1] UDS Data.*
16. Defined in *[Services_4] Global Master Reference Database*.

2525 v2 **RoutineControl (31)**

Version history:
(2):

Purpose: To be able to execute specific functionality in the ECU.
Source:
Verification Method: Test
Legacy ID: 76601v1

The service RoutineControl shall be supported by the ECU in defaultSession, extendedDiagnosticSession and programmingSession, both primary and secondary bootloader. In defaultSession the only allowed RoutineControl type is type 1- Short routine.

2529 v1 **RoutineControl (31) - RoutineControlType startRoutine (01)**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76602v0

The service RoutineControl with the sub-function RoutineControlType set to startRoutine shall

be supported by the ECU in all sessions where the ECU supports the service RoutineControl.

2530 v1 RoutineControl (31) - RoutineControlType startRoutine (01) - routineIden...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76603v0

The service RoutineControl with the sub-function RoutineControlType set to startRoutine and with the data identifier routineIdentifier shall be supported by the ECU in all sessions where the ECU support the service with the sub-function RoutineControlType set to startRoutine. The values of routineIdentifier are defined by *[Services_1] UDS Data*.

2528 v2 RoutineControl (31) - RoutineControlType startRoutine - routineControlOptionRecord

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76621v1

The service RoutineControl with the sub-function RoutineControlType set to startRoutine and with the data identifier routineControlOptionRecord may be supported by the ECU in all sessions where the ECU support the service RoutineControl with the sub-function RoutineControlType set to startRoutine. The values of routineControlOptionRecord are defined by *[Services_4] Global Master Reference Database*.

2531 v1 RoutineControl (31) - RoutineControlType stopRoutine (02)

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76622v0

The service RoutineControl with the sub-function RoutineControlType set to stopRoutine shall be supported by the ECU as specified in section *Additional routine requirements* in all sessions where the ECU supports the service RoutineControl.

2533 v1 RoutineControl (31) - RoutineControlType stopRoutine (02) - routineIdent...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76669v0

The service RoutineControl with the sub-function RoutineControlType set to stopRoutine and with the data identifier routineIdentifier shall be supported by the ECU in all sessions where the ECU support the service with the sub-function RoutineControlType set to stopRoutine. The values of routineIdentifier are defined by *[Services_1] UDS Data*.

2532 v2 RoutineControl (31) - RoutineControlType stopRoutine (02) - routineControlOptionRecord

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76670v1

The service RoutineControl with the sub-function RoutineControlType set to stopRoutine and with the data identifier routineControlOptionRecord may be supported by the ECU in all sessions where the ECU support the service RoutineControl with the sub-function RoutineControlType set to stopRoutine. The values of routineControlOptionRecord are defined by *[Services_4] Global Master Reference Database*.

2526 v1 RoutineControl (31) - RoutineControlType requestRoutineResults (03)

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76671v0

The service RoutineControl with the sub-function RoutineControlType set to requestRoutineResults shall be supported by the ECU as specified in section *Additional routine requirements* in all sessions where the ECU supports the service RoutineControl.

2527 v1 RoutineControl (31) - RoutineControlType requestRoutineResults (03) - ro...

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76672v0

The service RoutineControl with the sub-function RoutineControlType set to requestRoutineResults and with the data identifier routineIdentifier shall be supported by the ECU in all sessions where the ECU support the service with the sub-function RoutineControlType set to requestRoutineResults. The values of routineIdentifier are defined by *[Services_1] UDS Data*.

4.5.2.1.1.1.6 Upload download functional unit

Index	Session		Sub-function		Data parameters		Session			Notes
	Name	Hex	Name	Hex	Name	Hex	Default	Extended Diagnostic	Programming	
Upload download functional unit										
95	RequestDownload	34					N	N	MPS	12
96			N/A	N/A	dataFormatIdentifier	00	N	N	MPS	
97						10	N	N	MPS	
98						80	N	N	OPS	
99						01-0F, 11-7F, 81-FF	N	N	OPS	
100					addressAndLengthFormatIdentifier	44	N	N	MPS	
101					memoryAdress	0000-FFFF	N	N	MPS	
102					memorySize	0000-FFFF	N	N	MPS	
103	RequestUpload	35					N	N	OS	
104			N/A	N/A	dataFormatIdentifier	0	N	N	MS	
105					addressAndLengthFormatIdentifier	44	N	N	MS	
106					memoryAdress	0000-FFFF	N	N	MS	
107					memorySize	0000-FFFF	N	N	MS	
108	TransferData	36					N	N	MPS	9
109			N/A	N/A	blockSequenceCounter	00-FF	N	N	MPS	
110					transferRequestParameterRecord	00-FF	N	N	MPS	
111	RequestTransferExit	37	N/A	N/A	N/A	N/A	N	N	MPS	10
112	RequestFileTransfer	38					N	N	OPS	13
113			N/A	N/A	modeOfOperation	01-03	N	N	MPS	
114						04-05	N	N	OPS	
115					filePathAndNameLength	0000-FFFF	N	N	MPS	
116					filePathAndName	00-...	N	N	MPS	
117					dataFormatIdentifier	00	N	N	MPS	
118						10	N	N	MPS	
119						80	N	N	OPS	
120						01-0F, 11-7F, 81-FF	N	N	OPS	
121					fileSizeParametelLength	00-F F	N	N	MPS	
122					fileSizeUnCompressed	00-...	N	N	MPS	
123					fileSizeCompressed	00-...	N	N	MPS	

9. The TransferData service request includes a blockSequenceCounter to allow for an improved error handling in case a TransferData service fails during a sequence of multiple TransferData requests. The use cases (a-d) for blockSequenceCounter, specified in *[Services_9] ISO 14229-1*, shall be handled by the tester and the ECU. It is recommended that a tester retransmits the transferData service (i.e. with the same blockSequenceCounter and data) at least two times if no response is received from the ECU. If an ECU receives a transferData request during an active download sequence with the same blockSequenceCounter as the last accepted transferData request, it shall respond with a positive response without writing the data once again to its memory.
10. The transferResponseParameterRecord shall contain a checksum calculated on all data bytes written to the non-volatile memory defined by the diagnostic services RequestDownload, RequestUpload or RequestFileTransfer. Exception for RequestDownload: The checksum shall be omitted if the ECU has implemented the CheckMemory routine as defined in *[Services_8] General Software Authentication.*
    If the data transfer is initiated by the RequestDownload or RequestUpload services a two byte checksum shall be calculated based on the CRC16-CCITT algorithm (initial value 0xFFFF and normal representation)
    If the data transfer is initiated by the RequestFileTransfer service a four byte checksum shall be calculated based on the CRC32 algorithm (initial value 0xFFFFFFFF and normal representation).
    Hint. For algorithm compliance checking use one of the examples described in section 7.1. The checksum shall always be calculated on data read from the actual non-volatile memory as defined by the RequestDownload, RequestUpload or RequestFileTransfer services. Note that the ECU is not required to do anything more with the CRC data than to send it back to the tester as a response for further validation.
12. If the ECU supports the RequestFileTransfer service which is an alternative method for file based data storage the RequestDownload service may not need to be supported,
    The compression method "Delta encoding" is optional only if the size of the software file is under 10Mb or if the programming time is under 10 minutes. (See *[Services_7] General SWDL Specification* for more info*.)*
13. The RequestFileTransfer service is an alternative method if the ECU implementes a file system data storage and replaces the RequestDownload and RequestUpload service. The RequestFileTransfer service shall be implemented as defined in *ISO 14229-1:2013 (Second Edition)*. *Note: The referenced* ISO 14229-1:2013 (Second Edition) *is only to be used in the purpose of the RequestFileTransfer service.*
    The compression method "Delta encoding" is optional only if the size of the software file is under 10Mb or if the programming time is under 10 minutes. *(*See *[Services_7] General SWDL Specification* for more info*.)*

2583 v2 **RequestDownload (34)**

Version history:
(2):

Purpose: Used in the SWDL process.
Source:
Verification Method: Test
Legacy ID: 76673v1

The service RequestDownload shall be supported in programmingSession, both primary and

secondary bootloader but shall not be used in any other session. If the ECU supports the RequestFileTransfer service which is an alternative method for file based data storage the RequestDownload service may not need to be supported.

2577 v3 **RequestDownload - dataFormatIdentifier (00)**

Version history:
(3):

Purpose: Support a simple standard method for downloading data.
Source:
Verification Method: Test
Legacy ID: 76675v2

RequestDownload - dataFormatIdentifier 0x00 no compression and no encryption shall be supported in all sessions supporting RequestDownload (0x34).

2579 v3 **RequestDownload - dataFormatIdentifier (10)**

Version history:
(3): Change the Verification Method to Test

Purpose: Support of compression method for downloading data to decrease the programming time
Source:
Verification Method: Test
Legacy ID: 130461v1

RequestDownload – dataFormatIdentifier 0x10 compression method #1, as specified in *[Services_5] Data Compression and Encryption* and no encryption shall be supported in all sessions supporting RequestDownload (0x34).

2580 v2 **RequestDownload - dataFormatIdentifier (80)**

Version history:
(2):

Purpose: Support of delta encoding method for downloading data to decrease the programming time
Source:
Verification Method: Test
Legacy ID: 409100v1

RequestDownload – dataFormatIdentifier 0x80 Delta encoding, as specified in *[Services_6] Delta Encoding Specification* and no encryption shall be supported in all sessions supporting RequestDownload (0x34).

2578 v4 **RequestDownload - dataFormatIdentifier (01-0F and 11-7F and 81-FF)**

Version history:
(4):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76676v3

RequestDownload - dataFormatIdentifier range 0x01-0x0F and 0x11-0x1F and 0x81-0xFF are optional in all sessions supporting RequestDownload.

2576 v2 **RequestDownload - addressAndLengthFormatIdentifier (44)**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76677v1

RequestDownload - addressAndLengthFormatIdentifier, with the value 0x44 (4 bytes MemoryAddress and 4 bytes memorySize), shall be supported in all sessions supporting RequestDownload (0x34).

2581 v2 **RequestDownload - memoryAdress**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76679v1

RequestDownload - memoryAdress shall be supported in all sessions supporting RequestDownload.

2582 v2 **RequestDownload - memorySize**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76681v1

RequestDownload - memorySize shall be supported in all sessions supporting RequestDownload.

2606 v3 **RequestUpload (35)**

Version history:
(3):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76683v2

The service RequestUpload may be supported in the secondary bootloader - programmingSession but shall not be used in any other session.

2603 v2 **RequestUpload - dataFormatIdentifier**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76685v1

RequestUpload - dataFormatIdentifier with the value 0x00 shall be supported in all sessions supporting RequestUpload.

2602 v2 **RequestUpload - addressAndLengthFormatIdentifier (44)**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76686v1

RequestUpload - addressAndLengthFormatIdentifier, with the value 0x44 (4 bytes MemoryAddress and 4 bytes memorySize), shall be supported in all sessions supporting RequestUpload (0x35).

2604 v2 **RequestUpload - memoryAdress**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76687v1

RequestUpload - memoryAdress shall be supported in all sessions supporting RequestUpload

2605 v2 **RequestUpload - memorySize**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76688v1

RequestUpload - memorySize shall be supported in all sessions supporting RequestUpload.

2610 v3 **TransferData (36)**

Version history:
(3):

Purpose: Transfers data via the client and the ECU.
Source:
Verification Method: Test
Legacy ID: 76690v2

The service TransferData (0x36) shall be supported in programmingSession, both primary and secondary bootloader but shall not be used in any other session.

2611 v2 **TransferData (36) blockSequenceCounter handling**

Version history:
(2):

Purpose: Define which ISO specified use cases to support to standardize fault behaviour in between different bootloaders.
Source:
Verification Method: Test
Legacy ID: 76691v1

The use cases (a-d) for blockSequenceCounter, specified in *[Services_9] ISO 14229-1*, shall be handled by the tester and the ECU

2612 v2 **TransferData (36) blockSequenceCounter received with the same value**

Version history:
(2):

Purpose: Define behaviour of how a ECU shall react when receiving the same blockSequenceCounter value twice.
Source:
Verification Method: Test
Legacy ID: 76692v1

If an ECU receives a transferData request during an active download sequence with the same blockSequenceCounter as the last accepted transferData request, it shall respond with a positive response without writing the data once again to its memory.

2607 v2 **TransferData - blockSequenceCounter**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76693v1

TransferData - blockSequenceCounter shall be supported in all sessions supporting TransferData.

2608 v2 TransferData - transferRequestParameterRecord

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76694v1

TransferData - transferRequestParameterRecord shall be supported in all sessions supporting TransferData, if the data direction is from the Tester to the ECU i.e during download of data.

2609 v2 TransferData - transferResponseParameterRecord

Version history:
(2): Change the Verification Method to Test

Purpose:
Source:
Verification Method: Test
Legacy ID: 130582v0

TransferData - transferResponseParameterRecord shall be supported in all sessions supporting TransferData, if the data direction is from the ECU to the Tester i.e during upload of data.

2597 v2 RequestTransferExit (37)

Version history:
(2):

Purpose: Stops a SWDL process.
Source:
Verification Method: Test
Legacy ID: 76695v1

The service RequestTransferExit shall be supported in programmingSession, both primary and secondary bootloader but shall not be used in any other session.

2598 v5 RequestTransferExit (37) transferResponseParameterRecord

Version history:
(5):

Purpose: There are two transferResponseParameterRecord alternatives: with/without CRC.
An integrity check of the programmed data is performed at the ECU authenticity verification (Check Memory), according to the Software Authentication concept. Hence, the calculation of a checksum at RequestTransferExit is omitted (time consuming) for that case. A checksum calculation is also not suitable when delta encoding is used, as the location and number of transferred data block and written data block might differ.

Source:

Verification Method: Test

Legacy ID: 76696v4

If the ECU does not support Software Authentication (CheckMemory routine) as defined in *[Services_8] General Software Authentication*, the transferResponseParameterRecord shall contain a checksum for downloaded or uploaded data block.
If the ECU supports Software Authentication, no transferResponseParameterRecord shall be used for downloaded block but only for upload.
Examples - Data transfer initiated by RequestDownload

1. CheckMemory is not implemented.

Table - Example 1: RequestTransferExit positive response when CheckMemory is not implemented

Tester	Description	Data								Explanation
Tx	**RequestTransferExit (S.F)**	**01**	**37**	00	00	00	00	00	00	
Rx	Positive response	**03**	**77**	**xx**	**xx**	00	00	00	00	2-byte CRC

2. CheckMemory is implemented.

Table - Example 2: RequestTransferExit positive response when CheckMemory is implemented (i.e. Software Authentication supported)

Tester	Description	Data								Explanation
Tx	**RequestTransferExit (S.F)**	**01**	**37**	00	00	00	00	00	00	
Rx	Positive response	**01**	**77**	00	00	00	00	00	00	CRC omitted

2599 v4 **RequestTransferExit (37) transferResponseParameterRecord - algorithm**

Version history:
(4):

Purpose: Define the checksum algorithm.

Source:

Verification Method: Test

Legacy ID: 76697v3

If the transfer of data is initiated by RequestDownload or RequestUpload the

transferResponseParameterRecord checksum calculation shall use the CRC16-CCITT algorithm (initial value 0xFFFF and normal representation). The checksum shall be calculated on all data bytes as defined by the diagnostic services RequestDownload or RequestUpload. Hint. For algorithm compliance checking: use the example described in section *Appendix A- Suggestion code for transferResponseParameterRecord calculation.*

2600 v1 **RequestTransferExit (37) transferResponseParameterRecord - algorithm**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 113061v0

If the transfer of data is initiated by the RequestFileTransfer service the (transferResponseParameterRecord) shall consist of a four byte checksum based on the CRC32 algorithm (initial value 0xFFFFFFFF and normal representation) calculation on all data bytes as defined by the diagnostic services RequestFileTransfer. Hint. For algorithm compliance checking use the example described in section *Appendix A- Suggestion code for transferResponseParameterRecord calculation* .

2601 v5 **RequestTransferExit (37) transferResponseParameterRecord - use non-volatile memory**

Version history:
(5):

Purpose:
Source:
Verification Method: Test
Legacy ID: 76698v4

The checksum (transferResponseParameterRecord) shall always be calculated on data read from the actual non-volatile memory as defined by the RequestDownload, RequestUpload or RequestFileTransfer .

2596 v2 **RequestFileTransfer (38)**

Version history:
(2):

Purpose: Used in the SWDL process
Source:
Verification Method: Test
Legacy ID: 113145v1

The RequestFileTransfer service is optional in programmingSession, both primary and secondary bootloader but shall not be used in any other session.
The RequestFileTransfer service is an alternative method if the ECU implementes a file system

data storage and replaces the RequestDownload and RequestUpload service. The RequestFileTransfer service shall be implemented as defined in *ISO 14229-1:2013(Second Edition)*.
Note: The referenced *ISO 14229-1:2013 (Second Edition)* is only to be used in the purpose of the RequestFileTransfer service.

2593 v1 RequestFileTransfer - modeOfOperation (01-03)

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 113153v0

RequestFileTransfer – modeOfOperation (0x01-0x03) shall be supported in all sessions supporting RequestFileTransfer (0x38).

2594 v1 RequestFileTransfer - modeOfOperation (03 - ReplaceFile)

Version history:
(1):

Purpose: Clarification of ISO requirement.
Source:
Verification Method: Test
Legacy ID: 113156v0

If the modeOfOperation parameter 0x03 ReplaceFile is used and the file not is stored at the location shall the file be added.

2595 v1 RequestFileTransfer - modeOfOperation (04-05)

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 113159v0

RequestFileTransfer – modeOfOperation (0x04-0x05) is optional in all sessions supporting RequestFileTransfer (0x38).

2589 v1 RequestFileTransfer - filePathAndNameLength

Version history:
(1):

Purpose:
Source:

Verification Method: Test
Legacy ID: 113163v0

RequestFileTransfer – filePathAndNameLength shall be supported in all sessions supporting RequestFileTransfer (0x38).

2588 v1 **RequestFileTransfer - filePathAndName**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 113167v0

RequestFileTransfer – filePathAndName shall be supported in all sessions supporting RequestFileTransfer (0x38).

2584 v2 **RequestFileTransfer - dataFormatIdentifier (00)**

Version history:
(2):

Purpose: Support a simple standard method for downloading data
Source:
Verification Method: Test
Legacy ID: 113150v1

RequestFileTransfer – dataFormatIdentifier 0x00 no compression and no encryption shall be supported in all sessions supporting RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

2586 v3 **RequestFileTransfer - dataFormatIdentifier (10)**

Version history:
(3): Change the Verification Method to Test

Purpose: Support of compression method for downloading data to decrease the programming time.
Source:
Verification Method: Test
Legacy ID: 130613v1

RequestFileTransfer – dataFormatIdentifier 0x10 compression method #1, specified in reference *[Services_5] Data Compression and Encryption* and no encryption shall be supported in all sessions supporting RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

2587 v2 **RequestFileTransfer - dataFormatIdentifier (80)**

Version history:

(2):

Purpose: Support of delta encoding method for downloading data to decrease the programming time

Source:

Verification Method: Test

Legacy ID: 409103v1

RequestFileTransfer – dataFormatIdentifier 0x80 Delta encoding, as specified in reference *[Services_6] Delta encoding* and no encryption shall be supported in all sessions supporting RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

2585 v3 RequestFileTransfer - dataFormatIdentifier (01-0F and 11-7F and 81-FF)

Version history:
(3):

Purpose:

Source:

Verification Method: Test

Legacy ID: 113152v2

RequestFileTransfer – dataFormatIdentifier range 0x01-0x0F and 0x11-0x7F and 0x81-0xFF is optional in all sessions supporting RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

2591 v2 RequestFileTransfer - fileSizeParameterLenght

Version history:
(2):

Purpose:

Source:

Verification Method: Test

Legacy ID: 113161v1

RequestFileTransfer – fileSizeParameterLenght shall be supported in all sessions supporting RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

2592 v2 RequestFileTransfer - fileSizeUnCompressed

Version history:
(2):

Purpose:

Source:

Verification Method: Test

Legacy ID: 113165v1

RequestFileTransfer – fileSizeUnCompressed shall be supported in all sessions supporting

RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

2590 v2 **RequestFileTransfer - fileSizeCompressed**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 113166v1

RequestFileTransfer – fileSizeCompressed shall be supported in all sessions supporting RequestFileTransfer (0x38). The presence of the parameter depends on the modeOfOperation parameter.

4.5.2.1.1.1.7 Additional routine requirements

The positive response on the service routineControl request, regardless of sub-function, shall have the data parameter RoutineStatusRecord. The first byte of the data parameter RoutineStatusRecord shall be the RoutineInfo byte. The RoutineInfo byte shall consist of two fields where bits 7-4 defines RoutineType (see table *RoutineType definition*) and bits 3-0 defines RoutineStatus (see table *RoutineStatus definition*).

The result of the routine shall (at least) be available, via requestRoutineResults, as long as the diagnostic session is kept or until the routine is re-started.

Table: Routine type definition

Index	Hex	Name	Definition
1	0	Reserved	Reserved by this document
2	1	Type 1- Short routine	The routine stops after a certain time when it is completed. The routine is completed before a positive response on startRoutine request is sent. A positive response on startRoutine request shall be sent and only be sent in the following situations : • The entry conditions have been fulilled and the routine has been completed (and stopped). • The entry conditions have not been fulfilled. • Any of exit conditions have occurred. The sub-function stopRoutine shall not be supported. It is not possible to abort a short routine by diagostic service during its execution as the positive response on startRoutine request is sent after the routine has been stopped (aborted or completed). The sub-function requestRoutineResults may be supported.
3	2	Type 2- Long routine	The routine stops after a certain time when it is completed. The routine may be completed before or after a positive response on startRoutine request is sent. A positive response on startRoutine request shall be sent and only be

			sent in the following situations: • The entry conditions have been fulilled and the routine has been started or completed (and stopped). • The entry conditions have not been fulfilled. • Any of exit conditions have occurred. The sub-function stopRoutine shall be supported. It shall be possible to abort a routine during its execution, before it is completed, by stopRoutine requests (after a positive response on startRoutine request has been sent). The sub-function RequestRoutineResults shall be supported. It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests (after a positive response on startRoutine request has been sent) both while the routine executes and after it has been stopped (aborted or completed). Note that the routine type is compliant with the "Method B" specified in *[Services_9] ISO 14229-1*.
4	3	Type 3 - Continuous routine	The routine is stopped by stopRoutine request. The routine don't need a certain time in order to be completed. It is completed when it is stopped by stopRoutine request. A positive response on startRoutine request shall be sent and only be sent in the following situations: • The entry conditions have been fulilled and the routine has started. • The entry conditions have not been fulfilled. The sub-function stopRoutine shall be supported. The sub-function RequestRoutineResults shall be supported. It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests both while the routine executes and after it has been stopped (aborted or completed). Note that the routine type is compliant with the "Method A" specified in *[Services_9] ISO 14229-1*.
5	4-F	Reserved	Reserved by this document

Table: Routine status definition

Index	Hex	Name	Definition
1	0	RoutineCompleted	The routine completed all requested functionality (successfully or not). The routine results (if any), readable via service routineControl and/or any other service, are valid.
2	1	RoutineAborted	The routine aborted before completion of all requested functionality due to any of the following reasons: 1. The routine has not started due to the entry conditions are not fulfilled. 2. The routine has been aborted due to any of the exit conditions have occurred. 3. The routine has been aborted by a stopRoutine request (only applicable for routine type 2).
3	2	RoutineExcecutes	The routine is under execution and not yet completed (only applicable

			for routine type 2 and 3).
4	3-F	Reserved	Reserved by this document

2340 v2 **RoutineInfo byte**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 74112v1

The positive response on the service routineControl request, regardless of sub-function, shall have the data parameter RoutineStatusRecord. The first byte of the data parameter RoutineStatusRecord shall be the RoutineInfo byte. The RoutineInfo byte shall consist of two fields where bits 7-4 defines RoutineType and bits 3-0 defines RoutineStatus.

bit 7	bit 4	bit 3	bit 0
RoutineType		RoutineStatus	

The value of the RoutineType shall be one of the following:

Hex	RoutineType
0x1	Type 1 - Short Routine
0x2	Type 2 - Long Routine
0x3	Type 3 - Continuous Routine

The value of the RoutineStatus shall be one of the following:

Hex	RoutineStatus
0x0	RoutineCompleted
0x1	RoutineAborted
0x2	RoutineExecutes

2337 v1 **Routine result availability**

Version history:
(1):

Purpose: Define how long time the route shall keep the result of the routine.
Source:
Verification Method: Test
Legacy ID: 74114v0

The result of the routine shall (at least) be available, via requestRoutineResults as long as the

diagnostic session is kept or until the routine is re-started.

2338 v1 **Routine Type 1- Short routine, definition**

Version history:
(1):

Purpose: Quick and easy method for perfoming simpler issues. One advantage with this routine is that it may be run in defaultSession.
Source:
Verification Method: Test
Legacy ID: 74115v0

The routine stops after a certain time when it is completed. The routine is completed before a positive response on startRoutine request is sent **.**

2339 v3 **Routine Type 1- Short routine, positive response on startRoutine**

Version history:
(3):

Purpose: Quick and easy method for perfoming simpler issues. One advantage with this routine is that it may be run in defaultSession.
Source:
Verification Method: Test
Legacy ID: 74116v2

Routine Type 1- Short routine : A positive response on startRoutine request shall be sent and only be sent in the following situations:

- The entry conditions have been fulfilled and the routine has been completed (and stopped).
- The entry conditions have not been fulfilled.
- Any of exit conditions have occurred.

2344 v2 **Type 1- Short routine, sub-function requestRoutineResults**

Version history:
(2):

Purpose: Quick and easy method for perfoming simpler issues. One advantage with this routine is that it may be run in defaultSession.
Source:
Verification Method: Test
Legacy ID: 74118v1

Routine Type 1- Short routine: The sub-function requestRoutineResults may be supported.

2345 v1 **Type 2- Long routine, definition**

Version history:
(1):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74119v0

The routine stops after a certain time when it is completed. The routine is completed before or after a positive response on startRoutine request is sent.

2346 v3 **Type 2- Long routine, positive response on startRoutine**

Version history:
(3):

Purpose: Inform the tester if the routine is still running or if the routine have been stopped.
Source:
Verification Method: Test
Legacy ID: 74120v2

Type 2- Long routine : A positive response on startRoutine request shall be sent and only be sent in the following situations:

- The entry conditions have been fulfilled and the routine has been started or completed (and stopped).
- The entry conditions have not been fulfilled.
- Any of exit conditions have occurred.

2348 v1 **Type 2- Long routine, sub-function stopRoutine**

Version history:
(1):

Purpose:
Source:
Verification Method: Test
Legacy ID: 74121v0

Type 2- Long routine :
The sub-function stopRoutine shall be supported.
It shall be possible to abort a routine under execution, before it is completed, by stopRoutine requests (after a positive response on startRoutine request has been sent)..

2347 v1 **Type 2- Long routine, sub-function RequestRoutineResults**

Version history:
(1):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74122v0

Type 2- Long routine :
The sub-function RequestRoutineResults shall be supported .
It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests (after a positive response on startRoutine request has been sent) both while the routine executes and after it has been stopped (aborted or completed).

2349 v1 **Type 3 - Continuous routine, definition**

Version history:
(1):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74123v0

Type 3 - Continuous routine: The routine is stopped by stopRoutine request. The routine don't need a certain time in order to be completed. It is completed when it is stopped by stopRoutine request.

2350 v2 **Type 3 - Continuous routine, positive response on startRoutine**

Version history:
(2):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74124v1

Type 3 - Continuous routine: A positive response on startRoutine request shall be sent and only be sent in the following situations:

- The entry conditions have been fulilled and the routine has started.
- The entry conditions have not been fulfilled.

2352 v1 **Type 3 - Continuous routine, sub-function stopRoutine**

Version history:
(1):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74125v0

Type 3 - Continuous routine: The sub-function stopRoutine shall be supported.

2351 v1 **Type 3 - Continuous routine, sub-function RequestRoutineResults**

Version history:
(1):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74126v0

Type 3 - Continuous routine: The sub-function RequestRoutineResults shall be supported.
It shall be possible to read the RoutineInfo byte by RequestRoutineResults requests both while the routine executes and after it has been stopped (aborted or completed).

2342 v2 **RoutineStatus, routineCompleted**

Version history:
(2):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74127v1

RoutineStatus, routineCompleted: The routine completed all requested functionality (successfully or not). The routine results (if any), readable via service routineControl and/or any other service, are valid.

2341 v3 **RoutineStatus, RoutineAborted**

Version history:
(3):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74128v2

RoutineStatus, RoutineAborted :
The routine aborted before c ompletion of all requested functionality due to any of the following reasons:
1. The routine has not started due to the entry conditions are not fulfilled.
2. The routine has been aborted due to any of the exit conditions have occurred.
3. The routine has been aborted by a stopRoutine request (only applicable for routine type 2).

2343 v1 **RoutineStatus, RoutineExcecutes**

Version history:
(1):

Purpose: Control routines
Source:
Verification Method: Test
Legacy ID: 74130v0

RoutineStatus, RoutineExcecutes: The routine is under execution and not yet completed (only

applicable for routine type 2 and 3).

4.5.2.1.1.1.8 Additional session requirements

4.5.2.1.1.1.8.1 General

When the ECU transitions from any diagnostic session to another diagnostic session (including the currently active session), the ECU shall reset all active diagnostic functionality requested by diagnostic services that is not supported in the new diagnostic session or that are protected by security access. Note that this means that the ECU shall also reset the security level to the locked state if the ECU was previously unlocked.

When the ECU transitions from any non-default diagnostic session to the same non-default diagnostic session, the ECU shall maintain all active diagnostic functionality requested by diagnostic services, with the exception of the functionality that are protected by security access.

Note that written changes to non-volatile memory shall remain regardless of diagnostic session transitions.

2354 v2 **Reset active diagnostic functionality at session transitions**

Version history:
(2):

Purpose: Define session transition behaviour.
Source:
Verification Method: Test
Legacy ID: 74131v1

When the ECU transitions from any diagnostic session to another diagnostic session (including the currently active session), the ECU shall reset all active diagnostic functionality requested by diagnostic services that is not supported in the new diagnostic session or that are protected by security access. The ECU shall also reset the security level to the locked state if the ECU was previously unlocked.

2353 v1 **Maintain active diagnostic functionality at session transitions**

Version history:
(1):

Purpose: Define session transition behaviour.
Source:
Verification Method: Test
Legacy ID: 74133v0

When the ECU transitions from any non-default diagnostic session to the same non-default diagnostic session, the ECU shall maintain all active diagnostic functionality requested by diagnostic services, with the exception of the functionality that are protected by security access.

Note that written changes to non-volatile memory shall remain regardless of diagnostic session transitions.

4.5.2.1.1.1.8.2 Response message data parameter sessionParameterRecord

The structure and content of the positive diagnostic response message data parameter sessionParameterRecord is defined in table SessionParameterRecord structure definition and table SessionParameterRecord content definition . Thus, the following specified in *[Services_9] ISO 14229-1* regarding the sessionParameterRecord shall be ignored: "The content and structure of this parameter record is data-link-layer-specific and can be found in the implementation specification(s) of this part of ISO 14229".

Table: SessionParameterRecord structure definition

Byte pos. in record	Description	Value	M/O
	sessionParameterRecord[] = [		
#1	P2 $_{Server_max}$ (high byte)	0x00 – 0xFF	M
#2	P2 $_{Server_max}$ (low byte)	0x00 – 0xFF	M
#3	P2* $_{Server_max}$ (high byte)	0x00 – 0xFF	M
#4	P2* $_{Server_max}$ (low byte)]	0x00 – 0xFF	M

Table: SessionParameterRecord content definition

Parameter	Description	# of bytes	Resolution	Value	Note
P2 $_{Server_max}$	Default P2 $_{Server_max}$ timing supported by the server for the activated diagnostic session. Note that the activated diagnostic session is the same as the requested diagnostic session.	2	1 ms	See note 2	1
P2* $_{Server_max}$	Enhanced (NRC 0x78) P2 $_{Server_max}$ supported by the server for the activated diagnostic session. Note that the activated diagnostic session is the same as the requested diagnostic session.	2	10 ms	See note 2	1

Notes in table SessionParameterRecord content definition:

1. The ECU shall not use the service to alter the values of P2 $_{Server_max}$ and P2* $_{Server_max}$ from the values specified in *[Services_2] UDS Session*. The response message data parameter sessionParameterRecord shall contain the fixed values of P2 $_{Server_max}$ and P2* $_{Server_max}$ defined in *[Services_2] UDS Session*.
2. See *[Services_2] UDS Session*

2355 v1 **SessionParameterRecord definition**

Version history:
(1):

Purpose: Define SessionParameterRecord
Source:
Verification Method: Test
Legacy ID: 74134v0

The structure and content of the positive diagnostic response message data parameter sessionParameterRecord is defined in table *SessionParameterRecord structure definition* and table *SessionParameterRecord content definition* .

4.5.2.1.1.2 Response times

The maximum response time on the diagnostic service requests supported by the ECU shall be as specified in the table below.

The response time parameter $P4_{Server_max}$ is defined in reference [2]. Exceptions to $P4_{Server_max}$ are not allowed unless when CEVT Electrical Architecture approves it. For not supported diagnostic services, the ECU shall use $P2_{Server_max}$ as $P4_{Server_max}$. Requirements on the timing parameter $P2_{Server_max}$ is specified in *[Services_2] UDS Session*.

If a diagnostic service is allowed to make an ECU reset the response shall be sent before performing the ECU reset. See section 5.3 for which services that are allowed to reset the ECU. See section 4.6 for details regarding the ECU timing requirements of responding to the next tester request following a positive response on a diagnostic service that reset the ECU.

If data written or cleared by a diagnostic service shall be stored in non-volatile memory, the write or clear operation to the non-volatile memory shall be completed before a positive response of the diagnostic service is sent. An exception to this is service clearDiagnosticInformation which shall be performed as specified by *[Services_1] UDS Data*.

Table: Requirements on the value of response time parameter P4Server_max

Index	Service		Sub-parameter		$P4_{Server_max}$	Note
	Name	Hex	Name	Hex		
1	DiagnosticSessionControl	10			$P2_{Server_max}$	
2	ECUReset	11			$P2_{Server_max}$	
3	SecurityAccess	27			$P2_{Server_max}$	
4	TesterPresent	3E			$P2_{Server_max}$	
5	ReadDataByIdentifier	22			$P2_{Server_max}$	
6	ReadMemoryByAddress	23			$P2_{Server_max}$	
7	ReadDataByPeriodicIdentifier	2A			200 ms	
8	DynamicallyDefineDataIdentifier	2C			200 ms	
9	WriteDataByIdentifier	2E			5000 ms	
10	WriteMemoryByAddress	3D			5000 ms	
11	ClearDiagnosticInformation	14			1000 ms	
12	ReadDTCInformation	19			$P2_{Server_max}$	
13	ReadGenericInformation	AF			1000 ms	
14	InputOutputControlByIdentifier	2F			$P2_{Server_max}$	
15	RoutineControl	31	StartRoutine, RoutineType = 1	01	5000 ms	1
16			All other routine requests		200 ms	1
17	RequestDownload	34			1000 ms	
18	RequestUpload	35			1000 ms	
19	TransferData	36			5000 ms	
20	RequestTransferExit	37			15000 ms	
21	RequestFileTransfer	38			1000 ms	

Notes in Table - Requirements on the value of response time parameter P4Server_max:

1. This is general response time requirements. Exceptions from the general requirements are specified in *[Services_1] UDS Data*.

2436 v2 **Response time on the diagnostic service requests**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: The P4Server_max time defined shall be used to ensure ECUs does not take unnecessarily long time to repond to tester requests.

Source:

Verification Method: Analysis

Legacy ID: 74135v0

Exceptions to $P4_{Server_max}$ are not allowed unless approved By CEVT Electrical Architecture.

2412 v2 **If a diagnostic service makes a reset, the response shall be sent before...**

Version history:

(2): Change the Verification Method from Test to Analysis

Purpose: All ECU¿s need to follow the same sequence since failure to do so may result in different ECU¿s being in different states.
Source:
Verification Method: Analysis
Legacy ID: 74141v0

If a diagnostic service is allowed to make a reset as specified in section *Effect on the ECU operation*, the response shall be sent before performing the reset.

2413 v4 Non volatile data shall be written before responding on the diagnostic request

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: Data written or cleared to an ECU shall be stored in long term memory immediately by the ECU so that it can be read out what is programmed to the ECU:s.
Source:
Verification Method: Analysis
Legacy ID: 74142v2

If data written or cleared by a diagnostic service shall be stored in non-volatile memory, the write or clear operation to the non-volatile memory shall be completed before a positive response of the diagnostic service is sent. An exception to this is service clearDiagnosticInformation which shall be performed as specified by *[Services_1] UDS Data*.

2416 v1 P4Server_max response time for diagnosticSessionControl (10)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74143v0

Maximum response time for the service diagnosticSessionControl (0x10) is $P2_{Server_max}$.

2418 v1 P4Server_max response time for ECUReset (11)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74144v0

Maximum response time for the service ECUReset (0x11) is $P2_{Server_max}$.

2431 v1 **P4Server_max response time for securityAccess (27)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74145v0

Maximum response time for the service securityAccess (0x27) is $P2_{Server_max}$.

2432 v1 **P4Server_max response time for testerPresent (3E)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74146v0

Maximum response time for the service testerPresent (0x3E) is $P2_{Server_max}$.

2420 v1 **P4Server_max response time for readDataByIdentifier (22)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74147v0

Maximum response time for the service readDataByIdentifier (0x22) is $P2_{Server_max}$.

2424 v1 **P4Server_max response time for readMemoryByAddress (23)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74149v0

Maximum response time for the service readMemoryByAddress (0x23) is $P2_{Server_max}$.

2421 v1 **P4Server_max response time for readDataByPeriodicIdentifier (2A)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74150v0

Maximum response time for the service readDataByPeriodicIdentifier (0x2A) is 200 ms.

2417 v1 **P4Server_max response time for dynamicallyDefineDataIdentifier (2C)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74152v0

Maximum response time for the service dynamicallyDefineDataIdentifier (0x2C) is 200 ms.

2434 v1 **P4Server_max response time for writeDataByIdentifier (2E)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74153v0

Maximum response time for the service writeDataByIdentifier (0x2E) is 5000ms.

2435 v1 **P4Server_max response time for writeMemoryByAddress (3D)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74154v0

Maximum response time for the service writeMemoryByAddress (0x3D) is 5000ms.

2415 v1 P4Server_max response time for clearDiagnosticInformation (14)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74155v0

Maximum response time for the service clearDiagnosticInformation (0x14) is 1000ms.

2422 v1 P4Server_max response time for readDTCInformation (19)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74156v0

Maximum response time for the service readDTCInformation (0x19) is $P2_{Server_max}$.

2423 v1 P4Server_max response time for readGenericInformation (AF)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74157v0

Maximum response time for the service readGenericInformation (0xAF) is 1000 ms.

2419 v1 P4Server_max response time for inputOutputControlByIdentifier (2F)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74158v0

Maximum response time for the service inputOutputControlByIdentifier (0x2F) is P2 $_{Server_max}$.

2430 v1 **P4Server_max response time for routineControl (31) startRoutine (1), rou...**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74159v0

Maximum response time for the service routineControl (0x31) startRoutine (1), routineType = 1 is 5000ms. This is general response time requirements. Exceptions from the general requirements are specified in *[Services_1] UDS Data*.

2429 v2 **P4Server_max response time for routineControl (31) except for startRoutine (1), routineType = 1**

Version history:
(2):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74160v1

Maximum response time for the service routineControl (0x31) except for startRoutine (1), routineType = 1 is 200 ms. Exceptions from the general requirements are specified in *[Services_1] UDS Data*.

2425 v1 **P4Server_max response time for requestDownload (34)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74161v0

Maximum response time for the service requestDownload (0x34) is 1000ms.

2428 v1 **P4Server_max response time for requestUpload (35)**

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74162v0

Maximum response time for the service requestUpload (0x35) is 1000ms.

2433 v1 P4Server_max response time for transferData (36)

Version history:
(1):

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 74163v0

Maximum response time for the service transferData (0x36) is 5000ms.

2427 v2 P4Server_max response time for requestTransferExit (37)

Version history:
(2):

Purpose: Define the ECU performance time (P4Server) for the diagnostic response.
Source:
Verification Method: Test
Legacy ID: 74164v1

Maximum response time for the service requestTransferExit (0x37) is 1000ms.

2426 v2 P4Server_max response time for requestFileTransfer (38)

Version history:
(2): Change the Verification Method to Test

Purpose: Define the time the OEM tools shall wait before assuming the diagnostic response shall be considered to have timed out.
Source:
Verification Method: Test
Legacy ID: 130641v0

Maximum response time for the service requestFileTransfer (0x38) is 1000ms.

2414 v1 P4Server_max for services not supported by the ECU

Version history:
(1):

Purpose: ECU shall respond swiftly even though service is not supported by the ECU

since failure to do so may cause problems using functional requests with SPRMB bit set.

Source:

Verification Method: Test

Legacy ID: 74140v0

For not supported diagnostic services, the ECU shall use P2 $_{Server_max}$ as P4 $_{Server_max}$.

4.5.2.1.1.3 Effect on the ECU operation

The diagnostic service's affect on the ECU operation shall be kept to a minimum and are allowed only as specified in the table below.

The possible options are:
Allowed – The service may affect the ECU's ability to execute non-diagnostic tasks. The service may only affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes).
Not allowed – The service shall not affect the ECU's ability to execute non-diagnostic tasks at any time.

Table: Requirements on which diagnostic services are allowed to affect the ECU operation

Index	Service/Sub-function		Effect on the ECU operation allowed	Notes
	Name	Hex		
1	DiagnosticSessionControl	10		
	- Transition from and to programmingSession (excluding programmingSession to programmingSession)		Allowed	
	- All other session transitions		Not allowed	
2	ECUReset	11	Allowed	
3	SecurityAccess	27	Not allowed	
4	TesterPresent	3E	Not allowed	
5	ReadDataByIdentifier	22	Not allowed	
6	ReadMemoryByAddress	23	Not allowed	
7	ReadDataByPeriodicIdentifier	2A	Not allowed	
8	DynamicallyDefineDataIdentifier	2C	Not allowed	
9	WriteDataByIdentifier	2E	Allowed except reset the ECU	
10	WriteMemoryByAddress	3D	Allowed except reset the ECU	
11	ClearDiagnosticInformation	14	Not allowed	
12	ReadDTCInformation	19	Not allowed	
13	ReadGenericInformation	AF	Not allowed	
14	InputOutputControlByIdentifier	2F	Allowed except reset the ECU	
15	RoutineControl	31	Allowed except reset the ECU	
16	RequestDownload	34	N/A	
17	RequestUpload	35	N/A	
18	TransferData	36	N/A	
19	RequestTransferExit	37	N/A	
20	RequestFileTransfer	38	N/A	

2365 v1 **Service diagnosticSessionControl (10) - transition from and to programmi...**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.

Source:

Verification Method: Test

Legacy ID: 74165v0

Transition from and to programmingSession (excluding programmingSession to programmingSession) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes).

2364 v2 **Service diagnosticSessionControl (10) - all other session transissions than from and to programmingS**

Version history:

(2):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.

Source:

Verification Method: Test

Legacy ID: 74166v1

All other transitions than from and to programmingSession (excluding programmingSession to programmingSession) shall not affect the ECU's ability to execute non diagnostic tasks.

2367 v1 Service ECUReset (11) affecting ECU functionality

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.

Source:

Verification Method: Test

Legacy ID: 74167v0

The service ECUReset (0x11) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes).

2375 v1 Service securityAccess (27) affecting ECU functionality

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.

Source:

Verification Method: Test

Legacy ID: 74169v0

The service securityAccess (0x27) shall not affect the ECU's ability to execute non diagnostic tasks.

2376 v1 Service testerPresent (3E) affecting ECU functionality

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.

Source:
Verification Method: Test
Legacy ID: 74170v0

The service testerPresent (0x3E) shall not affect the ECU's ability to execute non diagnostic tasks.

2369 v1 **Service readDataByIdentifier (22) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74172v0

The service readDataByIdentifier (0x22) shall not affect the ECU's ability to execute non diagnostic tasks

2373 v1 **Service readMemoryByAddress (23) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74173v0

The service readMemoryByAddress (0x23) shall not affect the ECU's ability to execute non diagnostic tasks.

2370 v1 **Service readDataByPeriodicIdentifier (2A) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74174v0

The service readDataByPeriodicIdentifier (0x2A) shall not affect the ECU's ability to execute non diagnostic tasks.

2366 v1 **Service dynamicallyDefineDataIdentifier (2C) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74175v0

The service dynamicallyDefineDataIdentifier (0x2C) shall not affect the ECU's ability to execute non diagnostic tasks.

2377 v1 **Service writeDataByIdentifier (2E) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74176v0

The service writeDataByIdentifier (0x2E) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

2378 v1 **Service writeMemoryByAddress (3D) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74177v0

The service writeMemoryByAddress (0x3D) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

2363 v1 **Service clearDiagnosticInformation (14) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74179v0

The service clearDiagnosticInformation (0x14) shall not affect the ECU's ability to execute non diagnostic tasks.

2371 v1 **Service ReadDTCInformation (19) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74180v0

The service ReadDTCInformation (0x19) shall not affect the ECU's ability to execute non diagnostic tasks.

2372 v1 **Service readGenericInformation (AF) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.
Source:
Verification Method: Test
Legacy ID: 74182v0

The service readGenericInformation (0xAF) shall not affect the ECU's ability to execute non diagnostic tasks.

2368 v1 **Service inputOutputControlByIdentifier (2F) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is

limited.

Source:

Verification Method: Test

Legacy ID: 74183v0

The service inputOutputControlByIdentifier (0x2F) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

2374 v1 **Service routineControl (31) affecting ECU functionality**

Version history:
(1):

Purpose: The purpose of diagnostics is to be able to monitor functionality. If the diagnostics affects the functionality, the usefulness of the diagnostics is limited.

Source:

Verification Method: Test

Legacy ID: 74184v0

The service routineControl (0x31) is allowed to affect the ECU's ability to execute non diagnostic tasks. The service is only allowed to affect execution of the non-diagnostic tasks during the execution of the diagnostic service. After the diagnostic service is completed any effect on the non-diagnostic tasks is not allowed anymore (normal operational functionality resumes). The service is not allowed to reset the ECU .

4.5.2.1.1.4 Entry conditions

The entry conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed or required only as specified by the table below . Note that this is applicable on all diagnostic service requests that the ECU receives (refer to section *ECU ability to receive requests* for requirements on when the ECU shall be able to receive diagnostic service requests).

Table: Requirements on which diagnostic services that are allowed or required to have entry conditions

Index	Service/Sub-function		Entry conditions allowed or required	Notes
	Name	Hex		
1	DiagnosticSessionControl	10		
	- Change to programmingSession (excluding programmingSession to programmingSession)		Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC. The implementer shall implement the ECU's condition for entering programmingSession based on the allocated functionality. The condition shall ensure a defined and safe vehicle state when entering programmingSession and shall at a minimum include vehicle speed < 3km/h if not otherwise approved by the responsible authority appointed by VCC In an impaired vehicle or in a stand alone scenario if the vehicle signal(s) used in the evaluation of the condition e.g. speed and/or "main propulsion system not active" shall the safety mechanism not prevent the ECU to change to programmingSession to allow SWDL.	1
	- All other session transitions		Not allowed	
2	ECUReset	11	Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC.	1
3	SecurityAccess	27	Not allowed	
4	TesterPresent	3E	Not allowed	
5	ReadDataByIdentifier	22	Not allowed	
6	ReadMemoryByAddress	23	Not allowed	
7	ReadDataByPeriodicIdentifier	2A	Not allowed	
8	DynamicallyDefineDataIdentifier	2C	Not allowed	
9	WriteDataByIdentifier	2E	Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC.	1
10	WriteMemoryByAddress	3D	Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC.	1
11	ClearDiagnosticInformation	14	Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC.	1
12	ReadDTCInformation	19	Not allowed	
13	ReadGenericInformation	AF	Not allowed	
14	InputOutputControlByIdentifier	2F	Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC.	1
15	RoutineControl	31	Required according to note otherwise allowed, but only if approved by the responsible authority appointed by VCC.	1
16	RequestDownload	34	Not allowed	
17	RequestUpload	35	Not allowed	
18	TransferData	36	Not allowed	
19	RequestTransferExit	37	Not allowed	
20	RequestFileTransfer	38	Not allowed	

Notes in the table - **Requirements on which diagnostic services that are allowed or required to have entry conditions**:

1. If the ECU implement safety requirements with an ASIL higher than QM (see *[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2398 v3 **Entry conditions for the service diagnosticSessionConrol (10), changing to programmingSession (02)**

Version history:
(3):

Purpose: To ensure a defined and safe vehicle state upon entering programming session.
Source:
Verification Method: Test
Legacy ID: 74450v2

Entry conditions for service diagnosticSessionConrol (0x10), changing to programmingSession (0x02) (excluding programmingSession to programmingSession):
The implementer shall implement the ECU's condition for entering programmingSession based on the allocated functionality. The condition shall ensure a defined and safe vehicle state when entering programmingSession and shall at a minimum include vehicle speed < 3km/h if not otherwise approved by CEVT Electrical Architecture.
In an impaired vehicle or in a stand alone scenario if the vehicle signal(s) used in the evaluation of the condition e.g. speed and/or "main propulsion system not active" shall the safety mechanism not prevent the ECU to change to programmingSession to allow SWDL.
If the ECU implement safety requirements with an ASIL higher than QM (see *[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2399 v1 **Entry conditions for the service diagnosticSessionConrol (10), other ses...**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74452v0

Entry conditions for service diagnosticSessionConrol (0x10), other session transitions than changing to programmingSession (0x02) (excluding programmingSession to programmingSession) are not allowed.

2397 v2 **Entry conditions for service ECUReset (11)**

Version history:
(2):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the

diagnostic user may have problems actually fulfilling the entry condition.

Source:

Verification Method: Test

Legacy ID: 74454v1

Entry conditions for service ECUReset (0x11) are allowed only if approved by CEVT Electrical Architecture.
If the ECU implement safety requirements with an ASIL higher than QM (see *[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2392 v1 **Entry conditions for diagnostic service SecurityAccess (27)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.

Source:

Verification Method: Test

Legacy ID: 74455v0

Entry conditions for service SecurityAccess (0x27) are not allowed.

2393 v1 **Entry conditions for diagnostic service TesterPresent (3E)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.

Source:

Verification Method: Test

Legacy ID: 74456v0

Entry conditions for service TesterPresent (0x3E) are not allowed.

2382 v1 **Entry conditions for diagnostic service ReadDataByIdentifier (22)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.

Source:
Verification Method: Test
Legacy ID: 74458v0

Entry conditions for service ReadDataByIdentifier (0x22) are not allowed.

2386 v1 **Entry conditions for diagnostic service ReadMemoryByAddress (23)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74459v0

Entry conditions for service ReadMemoryByAddress (0x23) are not allowed.

2383 v1 **Entry conditions for diagnostic service ReadDataByPeriodicIdentifier (2A)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74460v0

Entry conditions for service ReadDataByPeriodicIdentifier (0x2A) are not allowed.

2380 v1 **Entry conditions for diagnostic service DynamicallyDefineDataIdentifier (2C)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74461v0

Entry conditions for service DynamicallyDefineDataIdentifier (0x2C) are not allowed.

2395 v2 **Entry conditions for diagnostic service WriteDataByIdentifier (2E)**

Version history:
(2):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74462v1

Entry conditions for service WriteDataByIdentifier (0x2E) are allowed only if approved by CEVT Electrical Architecture.
If the ECU implement safety requirements with an ASIL higher than QM (see *[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2396 v2 **Entry conditions for diagnostic service WriteMemoryByAddress (3D)**

Version history:
(2):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74463v1

Entry conditions for service WriteMemoryByAddress (0x3D) are allowed only if approved by CEVT Electrical Architecture.

If the ECU implement safety requirements with an ASIL higher than QM (see *[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2379 v2 **Entry conditions for diagnostic service ClearDiagnosticInformation (14)**

Version history:
(2):

Purpose:
Source:
Verification Method: Test
Legacy ID: 74465v1

If the ECU implement safety requirements with an ASIL higher than QM (see *[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2384 v1 **Entry conditions for diagnostic service ReadDTCInformation (19)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74466v0

Entry conditions for service ReadDTCInformation (0x19) are not allowed.

2385 v1 **Entry conditions for diagnostic service ReadGenericInformation (AF)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74467v0

Entry conditions for service ReadGenericInformation (0xAF) are not allowed.

2381 v3 **Entry conditions for diagnostic service InputOutputControlByIdentifier (2F)**

Version history:
(3):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74468v2

Entry conditions for service InputOutputControlByIdentifier (0x2F) are allowed only if approved by CEVT Electrical Architecture.
If the ECU implement safety requirement s with an ASIL higher than QM (see *[Services_12]*

ISO 26262) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2391 v3 **Entry conditions for diagnostic service RoutineControl (31)**

Version history:
(3):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74472v2

Entry conditions for service RoutineControl (0x31) are allowed only if approved by CEVT Electrical Architecture.
If the ECU implement safety requirements with an ASIL higher than QM (*[Services_12] ISO 26262*) it shall, in all situations when diagnostic services may violate any of those safety requirements, reject the critical diagnostic service requests. Note that if the ECU rejects such critical diagnostic service requests, this requires an approval by CEVT Electrical Architecture.

2387 v1 **Entry conditions for diagnostic service RequestDownload (34)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74474v0

Entry conditions for service RequestDownload (0x34) are not allowed.

2390 v1 **Entry conditions for diagnostic service RequestUpload (35)**

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74475v0

Entry conditions for service RequestUpload (0x35) are not allowed.

2394 v1 Entry conditions for diagnostic service TransferData (36)

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74476v0

Entry conditions for service TransferData (0x36) are not allowed.

2389 v1 Entry conditions for diagnostic service RequestTransferExit (37)

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 74477v0

Entry conditions for service RequestTransferExit (0x37) are not allowed.

2388 v1 Entry conditions for diagnostic service RequestFileTransfer (38)

Version history:
(1):

Purpose: Entry conditions for diagnostic services shall be avoided since entry condition limits the diagnostic functionality. It may also be hard to both convey the information of the entry condition to the diagnostic user as well as the diagnostic user may have problems actually fulfilling the entry condition.
Source:
Verification Method: Test
Legacy ID: 113261v0

Entry conditions for service RequestFileTransfer (0x38) are not allowed.

4.5.2.1.1.5 Exit conditions

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by CEVT Electrical Architecture.

2400 v3 **Exit conditions during diagnostics services**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Exit conditions for diagnostic services shall be avoided since exit condition limits the diagnostic functionality.

Source:

Verification Method: Analysis

Legacy ID: 74479v1

The exit conditions for the diagnostic services supported by the ECU shall be kept to a minimum and are allowed only if approved by CEVT Electrical Architecture.

4.5.2.1.1.6 Security access

The diagnostic services supported by the ECU are allowed or required to be protected by security access only as specified in the table below. Note that in case of the implementer choose to protect a service that is allowed to be protected according to the table below the implementer must not protect all data that is accessible for read, write or control by the diagnostic service but just a part of it.

Table: Requirements on which diagnostic services that are allowed or required to be protected by security access

Index	Service		Security access allowed or required	Notes
	Name	Hex		
1	DiagnosticSessionControl	10	Not allowed	
2	ECUReset	11	Not allowed	
3	SecurityAccess	27	N/A	
4	TesterPresent	3E	Not allowed	
5	ReadDataByIdentifier	22		
	- Read by system supplier specific dataIdentifiers		Allowed	
	- Read by other dataIdentifier		Allowed, but only if approved by the responsible authority appointed by VCC	
6	ReadMemoryByAddress	23	Allowed	
7	ReadDataByPeriodicIdentifier	2A	Not allowed	
8	DynamicallyDefineDataIdentifier	2C	Required according to note otherwise allowed	1
9	WriteDataByIdentifier	2E	Required in programmingSession otherwise allowed	
10	WriteMemoryByAddress	3D	Allowed	
11	ClearDiagnosticInformation	14	Not allowed	
12	ReadDTCInformation	19	Not allowed	
13	ReadGenericInformation	AF	Not allowed	
14	InputOutputControlByIdentifier	2F	Allowed	
15	RoutineControl	31	Required in prgorammingSession otherwise allowed	
16	RequestDownload	34	Required	
17	RequestUpload	35	Required	
18	TransferData	36	Not allowed	2
19	RequestTransferExit	37	Not allowed	2
20	RequestFileTransfer	38	Required	

Notes in the table - Requirements on which diagnostic services that are allowed or required to be protected by security access.

1. If data that is read by service readDataByIdentifier or readMemoryByAddress is protected by security access, then the service shall be protected by security access when including this same data (completely or partly) in the dynamically defined dataIdentifier.

2. The TransferData and RequestTransferExit services are implicitly blocked when the ECU is locked. In other words, the ECU is required to respond with requestSequenceError if a TransferData or RequestTransferExit is received without an active RequestDownload, RequestUpload or RequestFileTransfer service.

2437 v2 **Security access for service diagnosticSessionControl (10)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.

Source:

Verification Method: Test

Legacy ID: 74480v1

The service DiagnosticSessionControl (0x10) is not allowed to be protected by security access.

2440 v2 Security access protection for service ECUReset (11)

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74483v1

The service ECUReset (0x11) is not allowed to be protected by security access.

2453 v2 Security access protection for service testerPresent (3E)

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74485v1

The service TesterPresent (0x3E) is not allowed to be protected by security access.

2443 v3 Security access protection for service readDataByIdentifier (22), system supplier specific dataIdent

Version history:
(3):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74486v2

The service ReadDataByIdentifier (0x22), read by system supplier specific dataIdentifiers, is allowed to be protected by security access.

2442 v2 Security access protection for service readDataByIdentifier (22), other than system supplier specifi

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:

Verification Method: Test
Legacy ID: 74487v1

The service ReadDataByIdentifier (0x22), read by other than system supplier specific dataIdentifiers, is allowed to be protected by security access only if approved by CEVT Electrical Architecture.

2447 v2 **Security access protection for service readMemoryByAddress (23)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74488v1

The service readMemoryByAddress (0x23) is allowed to be protected by security access.

2444 v2 **Security access protection for service readDataByPeriodicIdentifier (2A)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74490v1

The service readDataByPeriodicIdentifier (0x2A) is not allowed to be protected by security access.

2439 v2 **Security access protection for service dynamicallyDefineDataIdentifier (2C)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74491v1

The service dynamicallyDefineDataIdentifier (0x2C) is allowed to be protected by security access only in the following situation: If data that is read by service readDataByIdentifier (0x22) or readMemoryByAddress (0x23) is protected by security access, then the service shall be protected by security access when including this same data (completely or partly) in the dynamically defined dataIdentifier.

2455 v2 **Security access protection for service writeDataByIdentifier (2E)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74492v1

The service writeDataByIdentifier (0x2E) is required to be protected by security access in programmingSession and otherwise allowed.

2456 v2 **Security access protection for service writeMemoryByAddress (3D)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74493v1

The service writeMemoryByAddress (0x3D) is allowed to be protected by security access.

2438 v2 **Security access protection for service clearDiagnosticInformation (14)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74494v1

The service clearDiagnosticInformation (0x14) is not allowed to be protected by security access.

2445 v2 **Security access protection for service readDTCInformation (19)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74495v1

The service readDTCInformation (0x19) is not allowed to be protected by security access.

2446 v2 **Security access protection for service readGenericInformation (AF)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74496v1

The service readGenericInformation (0xAF) is not allowed to be protected by security access.

2441 v3 **Security access protection for service inputOutputControlByIdentifier (2F)**

Version history:
(3):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74497v2

The service inputOutputControlByIdentifier (0x2F) is allowed to be protected by security access.

2452 v2 **Security access protection for service routineControl (31)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74498v1

The service routineControl (0x31) is required to be protected by security access in programmingSession and otherwise allowed .

2448 v2 **Security access protection for service requestDownload (34)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74499v1

The service requestDownload (0x34) is required to be protected by security access.

2451 v2 **Security access protection for service requestUpload (35)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74500v1

The service requestUpload (0x35) is required to be protected by security access.

2454 v2 **Security access protection for service transferData (36)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74501v1

The service transferData (0x36) is not allowed to be protected by security access.

2450 v2 **Security access protection for service requestTransferExit (37)**

Version history:
(2):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 74502v1

The service requestTransferExit (0x37) is not allowed to be protected by security access.

2449 v1 **Security access protection for service requestFileTransfer (38)**

Version history:
(1):

Purpose: The usage of security access to protect diagnostic services shall only be used when required and/or judged as needed.
Source:
Verification Method: Test
Legacy ID: 113281v0

The service requestFileTransfer (0x38) is required to be protected by security access.

4.5.2.1.1.7 Negative response

The negative response codes sent by negative responses on the diagnostic services requests supported by the ECU shall be s as specified in the table below.

Table: Requirements on which negative response codes that are allowed

Index	Name	Hex	Negative response code allowed
1	GeneralReject	10	Not allowed
2	BusyRepeatRequest	21	Not allowed unless otherwise is specified by this document.
3	Other	Other	Allowed *accoridng to [Services_9] ISO 14229-1* unless otherwise is specified by this document or in the implementation specification for the respective networks (e.g. UDSonCAN, UDSonFR and UDSonIP).

2406 v2 **Negative response code generalReject (10)**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Standardise the negative response codes that an ECU may send to make it easier to understand why a ECU rejects a diagnostic service request.
Source:
Verification Method: Analysis
Legacy ID: 74506v0

The negative response code generalReject (0x10) sent by negative responses on diagnostic services requests is not allowed.

2405 v4 **Negative response code busyRepeatRequest (21)**

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: Standardise the negative response codes that an ECU may send to make it easier to understand why a ECU rejects a diagnostic service request.
Source:
Verification Method: Analysis
Legacy ID: 74507v2

The negative response code busyRepeatRequest (0x21) sent by negative responses on diagnostic services requests is not allowed unless otherwise is specified by this document or in the implementation specification for the respective networks (e.g. UDSonCAN, UDSonFR and UDSonIP).

2407 v4 **Negative response code other than generalReject (10) and busyRepeatRequest (21).**

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: Standardise the negative response codes that an ECU may send to make it easier to understand why a ECU rejects a diagnostic service request.

Source:

Verification Method: Analysis

Legacy ID: 74510v2

Negative response code other than generalReject (10) and busyRepeatRequest (21) sent by negative responses on diagnostic services requests is allowed as specified by *[Services_9] ISO 14229-1* unless otherwise is specified by this document or in the implementation specification for the respective networks (e.g. UDSonCAN, UDSonFR and UDSonIP).

4.5.2.1.2 Appendix

4.5.2.1.2.1 Appendix A- Suggestion code for transferResponseParameterRecord calculation

4.5.2.1.2.1.1 CRC16-CITT C-code example 1 (fast)

This example uses a look-up table with pre-calculated CRCs for fast calculation.

\+ fast calculation
\- large code size

```
/* function declarations */
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

unsigned int crctab[256] =
{
   0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7,
   0x8108, 0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF,
   0x1231, 0x0210, 0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6,
   0x9339, 0x8318, 0xB37B, 0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE,
   0x2462, 0x3443, 0x0420, 0x1401, 0x64E6, 0x74C7, 0x44A4, 0x5485,
   0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE, 0xF5CF, 0xC5AC, 0xD58D,
   0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6, 0x5695, 0x46B4,
```

```
   0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D, 0xC7BC,
   0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
   0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B,
   0x5AF5, 0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12,
   0xDBFD, 0xCBDC, 0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A,
   0x6CA6, 0x7C87, 0x4CE4, 0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41,
   0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD, 0xAD2A, 0xBD0B, 0x8D68, 0x9D49,
   0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13, 0x2E32, 0x1E51, 0x0E70,
   0xFF9F, 0xEFBE, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A, 0x9F59, 0x8F78,
   0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E, 0xE16F,
   0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
   0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E,
   0x02B1, 0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256,
   0xB5EA, 0xA5CB, 0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D,
   0x34E2, 0x24C3, 0x14A0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
   0xA7DB, 0xB7FA, 0x8799, 0x97B8, 0xE75F, 0xF77E, 0xC71D, 0xD73C,
   0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657, 0x7676, 0x4615, 0x5634,
   0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9, 0xB98A, 0xA9AB,
   0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882, 0x28A3,
   0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABBB, 0xBB9A,
   0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92,
   0xFD2E, 0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9,
   0x7C26, 0x6C07, 0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1,
   0xEF1F, 0xFF3E, 0xCF5D, 0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8,
   0x6E17, 0x7E36, 0x4E55, 0x5E74, 0x2E93, 0x3EB2, 0x0ED1, 0x1EF0
};

/* driver */
void main(void)
{
   unsigned int crc;

   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
}

/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i;

   for(i=0;i<size;i++) {
     tmp=(crc>>8)^data[i];
     crc=(crc<<8)^crctab[tmp];
   }
   return crc;
}
```

4.5.2.1.2.1.2 CRC16-CITT C-code example 2 (slow)

This example is not using any look-up table, only shift and XOR operations.

\+ small code size
\- slow calculation

```
#define                 MASK                    0x1021                  /* CRC16-CITT
mask */

/* function declarations */
```

```
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

/* driver */
void main(void)
{
   unsigned int crc;

   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
}

/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i,j;

   for(i=0;i<size;i++){
      tmp=data[i]<<8;
      for(j=0;j<8;j++){
         if((crc ^ tmp) & 0x8000) crc=(crc<<1)^MASK;
         else crc<<=1;
         tmp<<=1;
      }
   }
   return crc;
}
```

4.5.2.1.2.1.3 CRC32 C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
void generate_crc32_table(void);
unsigned long calc_crc32(unsigned long size, unsigned char *data);

/* variables */
unsigned long crc32;
static unsigned long crc32_table[256];
```

```
/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};


/* driver */
void main(void)
{
   unsigned long crc32;

   generate_crc32_table();
   crc32=calc_crc32(256,data);

   while(1); /* the result of the above calculation shall be: crc32=0x29058c73 */
}


/* calculate the crc32 value */
unsigned long calc_crc32(unsigned long size, unsigned char *data)
{

   unsigned long i, crc;

   crc=0xffffffff;          // initial value

   for(i=0;i<size;i++) {
      crc = ((crc>>8)&0x00FFFFFF)^crc32_table[(int)(crc^data[i])&0xff];
   }

   return ~crc;
}



#define CRC32_POLY 0xedb88320 // 0x04c11db7 bit reversed

/* generate the lock-up table */
void generate_crc32_table(void)

{
   int i, j;
   unsigned long crc;

   for(i=0;i<256;i++) {
      crc=i;
      for(j=8;j>0;j--) {
```

```
        if(crc&1) {
           crc=(crc>>1)^CRC32_POLY;
        }
        else{
           crc>>=1;
        }
     }
     crc32_table[i]=crc;
  }
}

// As an alternative to the "generate_crc32_table()" function the following table
can be used

/*
static unsigned long crc32_tab[256] =
{  0x00000000L, 0x77073096L, 0xee0e612cL, 0x990951baL, 0x076dc419L, 0x706af48fL,
   0xe963a535L, 0x9e6495a3L, 0x0edb8832L, 0x79dcb8a4L, 0xe0d5e91eL, 0x97d2d988L,
   0x09b64c2bL, 0x7eb17cbdL, 0xe7b82d07L, 0x90bf1d91L, 0x1db71064L, 0x6ab020f2L,
   0xf3b97148L, 0x84be41deL, 0x1adad47dL, 0x6ddde4ebL, 0xf4d4b551L, 0x83d385c7L,
   0x136c9856L, 0x646ba8c0L, 0xfd62f97aL, 0x8a65c9ecL, 0x14015c4fL, 0x63066cd9L,
   0xfa0f3d63L, 0x8d080df5L, 0x3b6e20c8L, 0x4c69105eL, 0xd56041e4L, 0xa2677172L,
   0x3c03e4d1L, 0x4b04d447L, 0xd20d85fdL, 0xa50ab56bL, 0x35b5a8faL, 0x42b2986cL,
   0xdbbbc9d6L, 0xacbcf940L, 0x32d86ce3L, 0x45df5c75L, 0xdcd60dcfL, 0xabd13d59L,
   0x26d930acL, 0x51de003aL, 0xc8d75180L, 0xbfd06116L, 0x21b4f4b5L, 0x56b3c423L,
   0xcfba9599L, 0xb8bda50fL, 0x2802b89eL, 0x5f058808L, 0xc60cd9b2L, 0xb10be924L,
   0x2f6f7c87L, 0x58684c11L, 0xc1611dabL, 0xb6662d3dL, 0x76dc4190L, 0x01db7106L,
   0x98d220bcL, 0xefd5102aL, 0x71b18589L, 0x06b6b51fL, 0x9fbfe4a5L, 0xe8b8d433L,
   0x7807c9a2L, 0x0f00f934L, 0x9609a88eL, 0xe10e9818L, 0x7f6a0dbbL, 0x086d3d2dL,
   0x91646c97L, 0xe6635c01L, 0x6b6b51f4L, 0x1c6c6162L, 0x856530d8L, 0xf262004eL,
   0x6c0695edL, 0x1b01a57bL, 0x8208f4c1L, 0xf50fc457L, 0x65b0d9c6L, 0x12b7e950L,
   0x8bbeb8eaL, 0xfcb9887cL, 0x62dd1ddfL, 0x15da2d49L, 0x8cd37cf3L, 0xfbd44c65L,
   0x4db26158L, 0x3ab551ceL, 0xa3bc0074L, 0xd4bb30e2L, 0x4adfa541L, 0x3dd895d7L,
   0xa4d1c46dL, 0xd3d6f4fbL, 0x4369e96aL, 0x346ed9fcL, 0xad678846L, 0xda60b8d0L,
   0x44042d73L, 0x33031de5L, 0xaa0a4c5fL, 0xdd0d7cc9L, 0x5005713cL, 0x270241aaL,
   0xbe0b1010L, 0xc90c2086L, 0x5768b525L, 0x206f85b3L, 0xb966d409L, 0xce61e49fL,
   0x5edef90eL, 0x29d9c998L, 0xb0d09822L, 0xc7d7a8b4L, 0x59b33d17L, 0x2eb40d81L,
   0xb7bd5c3bL, 0xc0ba6cadL, 0xedb88320L, 0x9abfb3b6L, 0x03b6e20cL, 0x74b1d29aL,
   0xead54739L, 0x9dd277afL, 0x04db2615L, 0x73dc1683L, 0xe3630b12L, 0x94643b84L,
   0x0d6d6a3eL, 0x7a6a5aa8L, 0xe40ecf0bL, 0x9309ff9dL, 0x0a00ae27L, 0x7d079eb1L,
   0xf00f9344L, 0x8708a3d2L, 0x1e01f268L, 0x6906c2feL, 0xf762575dL, 0x806567cbL,
   0x196c3671L, 0x6e6b06e7L, 0xfed41b76L, 0x89d32be0L, 0x10da7a5aL, 0x67dd4accL,
   0xf9b9df6fL, 0x8ebeeff9L, 0x17b7be43L, 0x60b08ed5L, 0xd6d6a3e8L, 0xa1d1937eL,
   0x38d8c2c4L, 0x4fdff252L, 0xd1bb67f1L, 0xa6bc5767L, 0x3fb506ddL, 0x48b2364bL,
   0xd80d2bdaL, 0xaf0a1b4cL, 0x36034af6L, 0x41047a60L, 0xdf60efc3L, 0xa867df55L,
   0x316e8eefL, 0x4669be79L, 0xcb61b38cL, 0xbc66831aL, 0x256fd2a0L, 0x5268e236L,
   0xcc0c7795L, 0xbb0b4703L, 0x220216b9L, 0x5505262fL, 0xc5ba3bbeL, 0xb2bd0b28L,
   0x2bb45a92L, 0x5cb36a04L, 0xc2d7ffa7L, 0xb5d0cf31L, 0x2cd99e8bL, 0x5bdeae1dL,
   0x9b64c2b0L, 0xec63f226L, 0x756aa39cL, 0x026d930aL, 0x9c0906a9L, 0xeb0e363fL,
   0x72076785L, 0x05005713L, 0x95bf4a82L, 0xe2b87a14L, 0x7bb12baeL, 0x0cb61b38L,
   0x92d28e9bL, 0xe5d5be0dL, 0x7cdcefb7L, 0x0bdbdf21L, 0x86d3d2d4L, 0xf1d4e242L,
   0x68ddb3f8L, 0x1fda836eL, 0x81be16cdL, 0xf6b9265bL, 0x6fb077e1L, 0x18b74777L,
   0x88085ae6L, 0xff0f6a70L, 0x66063bcaL, 0x11010b5cL, 0x8f659effL, 0xf862ae69L,
   0x616bffd3L, 0x166ccf45L, 0xa00ae278L, 0xd70dd2eeL, 0x4e048354L, 0x3903b3c2L,
   0xa7672661L, 0xd06016f7L, 0x4969474dL, 0x3e6e77dbL, 0xaed16a4aL, 0xd9d65adcL,
   0x40df0b66L, 0x37d83bf0L, 0xa9bcae53L, 0xdebb9ec5L, 0x47b2cf7fL, 0x30b5ffe9L,
   0xbdbdf21cL, 0xcabac28aL, 0x53b39330L, 0x24b4a3a6L, 0xbad03605L, 0xcdd70693L,
   0x54de5729L, 0x23d967bfL, 0xb3667a2eL, 0xc4614ab8L, 0x5d681b02L, 0x2a6f2b94L,
   0xb40bbe37L, 0xc30c8ea1L, 0x5a05df1bL, 0x2d02ef8dL
};
*/
```

4.5.2.1.2.2 Appendix B- Additional ReadGenericInformation requirements

This appendix defines additional requirements for the service ReadGenericInformation (AF hex).

4.5.2.1.2.2.1 Request message definition

The ReadGenericInformation service shall use request message sub-function and data parameters as defined in previous chapters. The format of the request message shall be the same as for the service ReadDTCInformation, based on the corresponding sub-function parameter used, as defined in *[Services_9] ISO 14229-1*.

4.5.2.1.2.2.2 Request message data parameter definition

The ReadGenericInformation service shall use the same definitions of the request message sub function and data parameters as the service ReadDTCInformation, as defined in *[Services_9] ISO 14229-1*, with the additions defined in table - Request data parameter definition.

Table: Request data parameter definition

Definition
DTCMaskRecord . If this field = 0xFFFFFF, the request concerns all DTC's, and data shall be reported for all DTC's for which data is available in the positive response.

4.5.2.1.2.2.3 Positive response message definition

Positive response(s) to the ReadGenericInformation service requests depend on the sub-function in the service request. The tables below define the response message formats of each sub-function parameter.

For each sub-function only data for the minimum number of DTCs (for which DTC information shall be possible to store in long-term memory - see *[Services_1] UDS Data*) shall be reported in the positive response.

Table Requirements on positive response format for service AF 04 (hex) describes the positive response format for the following sub-functions of this service:
reportGenericSnapshotByDTCNumber.

Table: Requirements on positive response format for service AF 04 (hex)

Byte	Parameter Name	M/O	Hex Value
#1	ReadGenericInformation response Service Id	M	EF
#2	Sub-function = [reportGenericSnapshotByDTCNumber]	 M	 04
#3-6	DTCAndStatusRecord[] #1 = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 00-FF 00-FF 00-FF 00-FF
#7	DTCStatusIndicators #1	M	00-FF
#8	DTCSnapshotRecordNumber #1	M	00-FF
#9	DTCGenericSnapshotRecordNumberOfIdentifiers #1	M	00-FF
	DTCSnapshotRecord[] #1 = [		
#	dataIdentifier#1	M	0000-FFFF

#	snapshotData#1.1 byte #1 : snapshotData#1.1 byte #p	M	00-FF 00-FF
:	:		
#	dataIdentifier#w	MC1	0000-FFFF
#	snapshotData#w.1 byte #1 : snapshotData#w.1 byte #m	MC1	00-FF 00-FF
:	:		
#	DTCAndStatusRecord[] #2 = [		
	DTCHighByte	MC2	00-FF
	DTCMiddleByte	MC2	00-FF
	DTCLowByte	MC2	00-FF
	statusOfDTC]	MC2	00-FF
#	DTCStatusIndicators #2	MC2	00-FF
#	DTCSnapshotRecordNumber #y	MC2	00-FF
#	DTCGenericSnapshotRecordNumberOfIdentifiers#y	MC2	00-FF
	DTCSnapshotRecord[] #y = [		
#	dataIdentifier#1	MC2	0000-FFFF
#	snapshotData#1.1 byte #1 : snapshotData#1.1 byte #t	MC2	00-FF 00-FF
:	:		
#	dataIdentifier#q	MC1	0000-FFFF
#	snapshotData#q.1 byte #1 : snapshotData#q.1 byte #u	MC1	00-FF 00-FF
:	:		
#	DTCAndStatusRecord[] #x = [		
	DTCHighByte	MC3	00-FF
	DTCMiddleByte	MC3	00-FF
	DTCLowByte	MC3	00-FF
	statusOfDTC]	MC3	00-FF
#	DTCStatusIndicators #x	MC3	00-FF
#	DTCSnapshotRecordNumber #z	MC3	00-FF
#	DTCGenericSnapshotRecordNumberOfIdentifiers#z	MC3	00-FF
	DTCSnapshotRecord[] #z = [		
#	dataIdentifier#1	MC3	0000-FFFF
#	snapshotData#1.1 byte #1 : snapshotData#1.1 byte #r	MC3	00-FF 00-FF
:	:		
#	dataIdentifier#v	MC1	0000-FFFF

#r	snapshotData#v.1 byte #1 : snapshotData#v.1 byte #s	MC1	00-FF
#r			00-FF

Abbreviations in Table - Requirements on positive response format for service AF 04 (hex)

MC1 The parameter is only present if more than one dataIdentifier is defined for the DTCSnapshotRecord.

MC2 The parameter is only present if DTCSnapshotRecordNumber is set to 0xFF in the request and more than one DTCSnapshotRecord is available to be reported for the DTC.

MC3 The parameter is only present if DTCMaskRecord is set to 0xFFFFFF in the request and more than one DTC have DTCSnapshotRecord available to be reported.

Table - Requirements on positive response format for service AF 06 (hex) describes the positive response format for the following sub-functions of this service: reportGenericExtendedDataByDTCNumber.

Table - Requirements on positive response format for service AF 06 (hex)

Byte	Parameter Name	M/O	Hex Value
#1	ReadGenericInformation response Service Id	M	EF
#2	Sub-function = [reportGenericExtendedDataByDTCNumber]	M	06
#3-6	DTCAndStatusRecord[] #1 = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 M M M M	 00-FF 00-FF 00-FF 00-FF
#7	DTCStatusIndicators #1	M	00-FF
#8	NumberOfDTCExtendedDataRecords	M	00-FF
#9	DTCExtendedDataRecordNumber #1	M	00-FF
#10 : #10+p-1	DTCExtendedDataRecord [] #1 = [extendedData#1 byte #1 : extendedData#1 byte #p	M	00-FF : 00-FF
:	:		
#t	DTCExtendedDataRecordNumber #n	MC1	00-FF
#t : #t+p-1	DTCExtendedDataRecord [] #n = [extendedData#n byte #1 : extendedData#n byte #q	MC1	00-FF : 00-FF
:	:		
#	DTCAndStatusRecord[] #x = [DTCHighByte DTCMiddleByte DTCLowByte statusOfDTC]	 MC2 MC2 MC2 MC2	 00-FF 00-FF 00-FF 00-FF

#	DTCStatusIndicators #x	MC2	00-FF
#	DTCExtendedDataRecordNumber #1	MC2	00-FF
# : #	DTCExtendedDataRecord [] #1 = [extendedData#1 byte #1 : extendedData#1 byte #p	MC2	 00-FF : 00-FF
:	:		
#	DTCExtendedDataRecordNumber #n	MC3	00-FF
# : #	DTCExtendedDataRecord [] #n = [extendedData#n byte #1 : extendedData#n byte #q	MC3	 00-FF : 00-FF

Abbreviations in Table - Requirements on positive response format for service AF 04 (hex):
MC1 The parameter is only present if all DTCExtendedDataRecords are requested to be reported (DTCExtendedDataRecordNumber is set to 0xFF in the request)
MC2 The parameter is only present if all DTCs are requested to be reported (DTCMaskRecord is set to 0xFFFFFF in the request).
MC3 The parameter is only present if all DTCs and all DTCExtendedDataRecords are requested to be reported (DTCMaskRecord is set to 0xFFFFFF, and DTCExtendedDataRecordNumber is set to 0xFF, in the request).

4.5.2.1.2.2.4 Positive response message data parameter definition

The ReadGenericInformation service shall use the same definitions of the positive response message data parameters as the service ReadDTCInformation, as defined in *[Services_9] ISO 14229-1* with the additions defined in Table Response message data parameter definition .

Table: Response message data parameter definition

Definition
DTCStatusIndicators This parameter is the DTCExtendedDataRecordNumber 0x30 for the DTC. See reference *[Services_1] UDS Data* for definition of DTCExtendedDataRecordNumber.
NumberOfDTCExtendedDataRecords This single-byte parameter shows the number of DTCExtendedDataRecords, associated with the DTC, that follows after this parameter in the response message.

4.5.2.1.2.2.5 Supported negative response codes (NRC_)

The ReadGenericInformation service shall support negative response codes defined in t able - Supported negative response codes , in decreasing order of priority (If the conditions for several negative codes are satisfied concurrently, the higher priority response code in the table shall be sent).

Table: Supported negative response codes

Hex	Description
12	**subFunctionNotSupported** - sub-function different from 0x04 or 0x06 in the request.
13	**incorrectMessageLengthOrInvalidFormat** - The length and/or format of the request message is incorrect.

31	**requestOutOfRange** - DTCMaskRecord is different from 0xFFFFFF and different from a supported DTC number. or - sub-function is 0x04 and DTCSnapshotRecord is different from 0x FF and different from a supported DTCSnapshotRecordNumber. or - sub-function is 0x06 and DTCExtendedDataRecord is different from 0x FF and different from a supported DTCExtendedDataRecordNumber.
22	**conditionsNotCorrect** - DTCMaskRecord = 0xFFFFFF and no snapshot data has been stored for any DTC. or - DTCMaskRecord = a supported DTC number and no snapshot data has been stored for this DTC.
78	**requestCorrectlyReceived-ResponsePending** - Depending on the amount of data in the response NRC 78 will be used to respond within the required response time.

4.5.2.1.3 Introduction

4.5.2.1.3.1 Requisite documents

This specification shall be followed and the requirements specified in this document are not negotiable.

In the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by CEVT Electrical Architecture.

Reference	Source	Document
[Services_1]	Internal	UDS Data
[Services_2]	Internal	UDS Session
[Services_3]	Internal	Security access algorithm
[Services_4]	Internal - CarCom database	Global Master Reference Database
[Services_5]	Internal	Data Compression and Encryption
[Services_6]	Internal	Delta Encoding
[Services_7]	Internal	General SWDL Specification
[Services_8]	Internal	General Software Authentication
[Services_9]	ISO	ISO 14229-1: Road vehicles – Diagnostics systems – Diagnostic services (001, 2006-12-01 Corrected 2007-04-15)
[Services_10]	ISO	ISO 26021-1: Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 1: General information and use case definitions (2008-05-15)
[Services_11]	ISO	ISO 26021-2: Road vehicles — End-of-life activation of on-board pyrotechnic devices — Part 2: Communication requirements (2008-

		07-15)
[Services_12]	ISO	ISO-26262: Road vehicles — Functional safety, International Organization for Standardization (2011)
[Services_13]	ISO	ISO-13400-2: Road vehicles – Diagnostic Communication over Internet Protocol (DoIP) – Part 2: Network and Transport Layer Requirements and Services

Notes:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

4.5.2.1.3.2 Definitions and Abbreviations

Definition	Description
Backbone Network	Internal network to connect domains on a high level
Domain ID	Part of the ECU Address, states which Domain the ECU is physical connected to.
Domain Network	Internal network for connection of nodes within a domain
Domain Network GW	Gateway between Backbone Network (or other internal Network as e.g. internal IP Network) and Domain Network
ECU ID	Part of the ECU Address, unique identifier for each ECU on a network.
ECU sleep mode	ECU sleep mode is defined as an ECU activity level when the ECU has no (functional) activity and has reached its absolute lowest power consumption, but still need to be powered e.g. to maintain long term memory or check for wake-up requests.
Entry condition	An entry condition for a specific diagnostic service is defined as any condition that that is necessary to meet for the diagnostic service to be successfully executed. This shall include ECU and vehicle operation conditions, surrounding environment conditions and other conditions that is necessary to be meet. Note that this includes also conditions that are initiated by other diagnostic services or obtained by the technician performs a certain actions.
Exit condition	An exit condition for a specific diagnostic service is defined as any condition that will cause active diagnostic functionality requested by the diagnostic service to abort.
External Network	Network for connection of an external tester. Used for software download and diagnostics
Implementer	Person responsible for the diagnostic implementation of the ECU. It may be the vehicle manufacturer or the ECU supplier diagnostic software designer.
Internal Network	Network for communication within the vehicle
Network ID	Part of the ECU Address, states which Network the ECU is physical connected to.
Network frame ID	Part of a data frame, transmitted on a network, which is used to identify the content of the frame, and/or the origin of the frame, and/or the destination of the frame.
Operation cycle	Refer to section *ECU ability to receive requests* for definition of an operation cycle and requirements on an ECU to start and stop an operation cycle.

Public ECU	An ECU addressable from a client/tester point of view.
Public ECU master	A public ECU responsible to act as a diagnostic server for one or more private ECUs.
Private ECU	An ECU not addressable from a client/tester point of view. Note: An ECU can partly be public and partly private i.e. some diagnostic services and diagnostic data supported by the ECU are addressable and some are not. Refer to the implementation specification for respective network (e.g. VCC – UDSonLIN, UDSonCAN, etc.) for which diagnostic services and diagnostic data supported by the ECU that may be or may not be addressable.
Server	Software entity of an ECU which provides services for clients (testers)
Sub Network	Internal network for low level communication within a domain
Sub Network GW	Gateway between Domain Network and Sub Network

Abbreviation	Description
ASIL	Automotive Safety Integrity Level. Defined by *Road vehicles - Functional safety, International Organization for Standardization* , VCC UDS Services - External publications
D	Do not care. Implementation of the requirement is not required and not forbidden. The implementer may decide if to implement or not. If implemented it will not be verified by VCC and shall not be expected to be supported by VCC tools.
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
M	Mandatory. Shall be supported by the ECU.
MC	Mandatory on Condition(s). The condition(s) is specified separately for each occurrence.
MPS	Mandatory for primary and secondary bootloader. Shall be supported by the ECU.
MS	Mandatory for secondary bootloader. Shall be supported by the ECU.
N	Not allowed. Shall not be supported by the ECU.
N/A	Not applicable
O	Optional. May be supported by the ECU. Whether or not the ECU shall support it is defined by *Road vehicles – Diagnostics systems – Diagnostic services*, VCC UDS Services - External publications , if it is not defined by this reference , by the implementer (if not otherwise is explicitly specified by this document).
OS	Optional in secondary bootloader. Not allowed in primary bootloader. May be supported by the ECU. Whether or not the ECU shall support it is defined by *Road vehicles – Diagnostics systems – Diagnostic services*, VCC UDS Services - External publications , if it is not defined by this reference , by the implementer (if not otherwise is explicitly specified by this document).
OPS	Optional for primary and secondary bootloader. May be supported by the ECU. Whether or not the ECU shall support it is defined by *Road vehicles – Diagnostics systems – Diagnostic services*, VCC UDS Services - External publications , if it is not defined by this reference , by the implementer (if not

	otherwise is explicitly specified by this document).
PBL	Primary Bootloader
QM	Quality Management. Defined by *Road vehicles - Functional safety, International Organization for Standardization*, VCC UDS Services - External publications
SA	Source Address
SBL	Secondary Bootloader
SWDL	Software Download
TA	Target Address
UDS	Unified Diagnostic Services

4.5.2.1.4 Diagnostic message addressing

The normal vehicle network configuration will be a multi level network, as described in the figure *Typical vehicle network topology* below , with both public and private ECUs on the different networks. This chapter defines requirements for diagnostic message addressing for all public ECUs on any network. It is however not applicable to ECUs on networks where the network frame ID is less than 16 bits. For further information regarding addressing on networks with a network frame ID less than 16 bits, see the corresponding implementation specification.

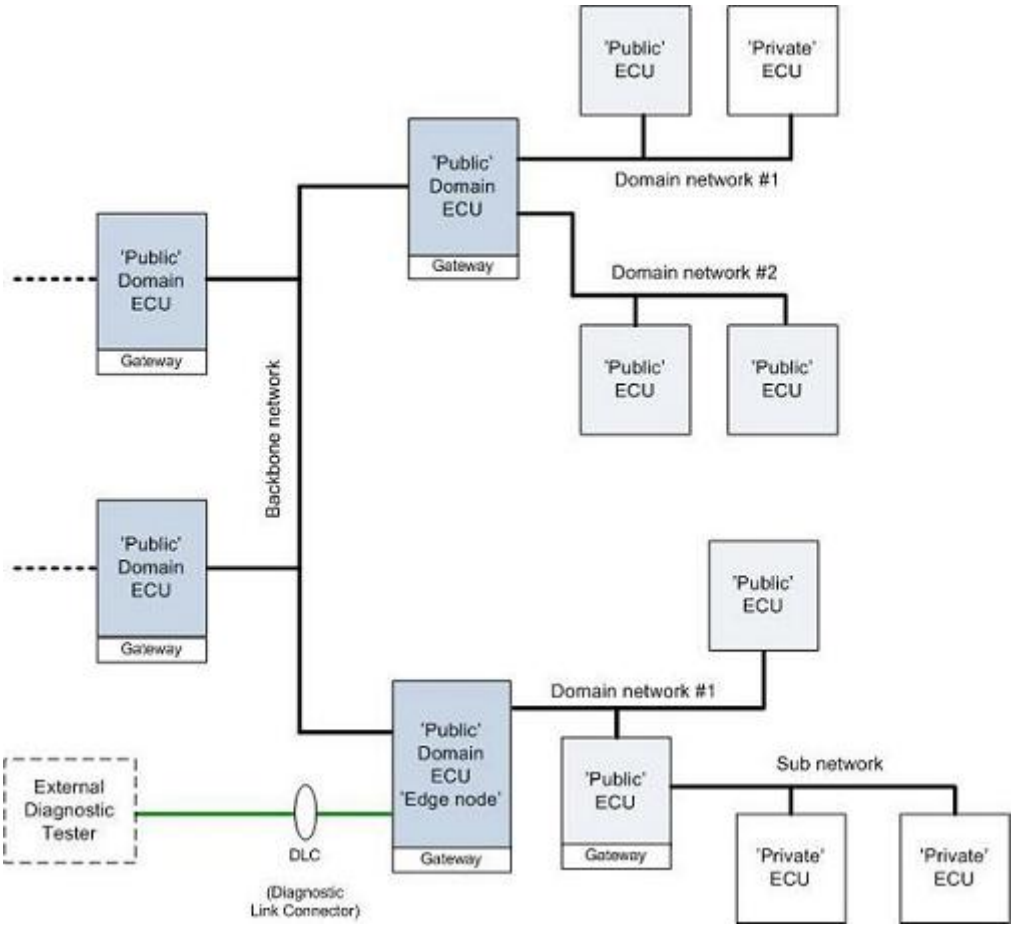


Figure - Typical vehicle network topology

A distributed server system shall be used where all public ECUs shall have their own server.

To make it possible for the tester to send a specific request to a public ECU on a network in the vehicle, each public ECU needs to have its own unique address. The address will be dependent on which network the ECU is located on and which domain the network belongs to.

The logical address range 0x1000 – 0x7FFF (Vehicle manufacturer specific) defined in *[Services_13] ISO-13400-2 Road vehicles – Diagnostic Communication over Internet Protocol (DoIP) – Part 2: Network and Transport Layer Requirements and Services*, where the range 0x1000 to 0x1FFF is used as addresses for the public ECUs in the platforms using this specification.

The logical address range 0x0E80 – 0x0EFF (External vehicle manufacture / aftersales enhanced diagnostic test equipment) defined in *[Services_13] ISO-13400-2 Road vehicles – Diagnostic Communication over Internet Protocol (DoIP) – Part 2: Network and Transport Layer Requirements and Services*, is used for the external diagnostic tester. Only 0x0E80 can be used, (using another address would affect internal routing tables).

4.5.2.1.4.1 Logical address format for diagnostic request and response

The addressing format used for diagnostic request (target address) and response (source address) is divided into three parts where the combination of Domain ID, Network ID and ECU ID is utilized, see the figure below. This addressing format gives the advantage of less complex gateway implementations.
The addressing format supports both physical and functional addressing for request messages. All diagnostic services defined in this document shall be supported for both physical and functional addressing unless otherwise is specified in this document.

Bit 12-15	0x1 (VCC address range)
Bit 8-11	Domain ID
Bit 4-7	Network ID
Bit 0-3	ECU ID

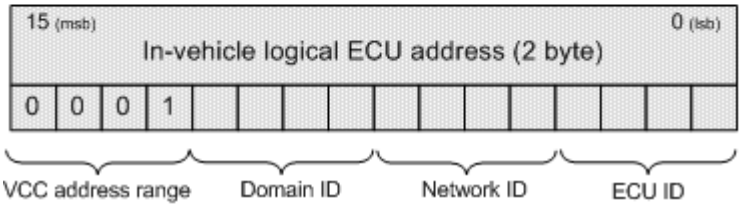


Figure – addressing format

The maximum number of ECUs per network range is limited to 15 since the 16'th (bit 0-3 equal to 0xF) is allocated for functional requests. If there is a need to have more than 15 ECUs within a network the network can be extended by assigning two or more logical Network IDs to the same physically network i.e. the ECU's will physically be on the same network but will have different Network IDs.

The maximum number of networks per domain is limited to 7 since Networks IDs (0x0 to 0x6)

are assigned for communication with external diagnostic tester and Networks IDs (0x7 to 0xD) reserved for future communication with internal diagnostic tester. If there is a need to have more than 7 physical networks within a domain the domain can be extended by assigning two or more logical Domain IDs.

2336 v1 **Unique ECU address**

Version history:
(1):

Purpose: Define how the unique ECU ID is decided for each network.
Source:
Verification Method: Analysis
Legacy ID: 74081v0

Of the 16 bits that is used for TA and SA bit 0-3 shall define ECU ID, bit 4-7 shall define Network ID, bit 8-11 shall define Domain ID and bit 12-15 shall define the address range and shall be set to 0x1.

4.5.2.1.4.2 Physical logical address range

Each ECU shall have one vehicle unique physical logical address assigned within the ranges defined by the table below.

The system architect defining each ECU physical logical address needs to be aware of the parallel request algorithm which the tester uses to distribute the diagnostic request messages during software download based on the logical address. The ECUs which takes the longest time to re-program shall have the lowest Domain/Network/ECU ID.
The system architect must also make sure that a Network ID equal to 0xD is not assigned to a CAN network including emissions related ECUs (this is necessary in order to avoid possible conflicts with request messages sent from an OBD scan tool).
The system architect shall check respective transport/network layer specification (e.g. CAN, LIN, IP, FlexRay, MOST150), regarding limitations when assigning a physical address to an ECU.

Table – Physical logical address range

(msb) (lsb) Physical logical address range				**Comment**
Address range Bit 15 – 12	**Domain ID Bit 11 – 8**	**Network ID Bit 7 – 4**	**ECU ID Bit 3 – 0**	
0x1	0x0 to 0xE	0x0 to 0x6	0x0 to 0xE	Reserved for external diagnostic tester communication
0x1	0x0 to 0xE	0x7 to 0xD	0x0 to 0xE	Reserved for future internal diagnostic tester communication

2408 v3 **Allocating ECU addresses**

Version history:
(3):

Purpose: Describe how to define server ECU address (i.e. TA for request and SA for response). The ECU address distribution method chosen shall:

- make as easy gateway implementation as possible
- support a method for load balancing certain types of system requests.

Source:
Verification Method: Analysis
Legacy ID: 74083v2

The ECU shall have one vehicle unique physical logical address assigned within the ranges defined by the table *Physical logical address range* .

Table – Physical logical address range

(msb) (lsb) Physical logical address range				**Comment**
Address range Bit 15 – 12	**Domain ID Bit 11 – 8**	**Network ID Bit 7 – 4**	**ECU ID Bit 3 – 0**	
0x1	0x0 to 0xE	0x0 to 0x6	0x0 to 0xE	Reserved for external diagnostic tester communication
0x1	0x0 to 0xE	0x7 to 0xD	0x0 to 0xE	Reserved for future internal diagnostic tester communication

2409 v1 **Assigning domain ID address**

Version history:
(1):

Purpose: The scheduling method, for the order that SWDL shall be performed, relies on that larger (in regards of SWDL time) domains shall be programmed first. The method shall be vehicle platform independent.

Source:
Verification Method: Analysis
Legacy ID: 74084v0

Domain ID's range shall be assigned based on the time it takes to make a SWDL for the specific domain. The domain that takes the longest time to perform a SWDL shall have the lower ID's and so on.

2411 v1 **Assigning network ID address**

Version history:
(1):

Purpose: The scheduling method, for order that SWDL shall be performed, relies on that larger (in regards of SWDL time) networks shall be programmed first. The method shall be vehicle platform independent.

Source:
Verification Method: Analysis
Legacy ID: 74086v0

Network ID's range shall be assigned based on the time it takes to make a SWDL for the specific network. The network that takes the longest time to perform a SWDL shall have the

lowest ID and so on.

2410 v1 **Assigning ECU ID address**

Version history:
(1):

Purpose: The scheduling method, for order that SWDL shall be performed, relies on that larger (in regards of SWDL time) ECU¿s shall be programmed first. The method shall be vehicle platform independent.
Source:
Verification Method: Analysis
Legacy ID: 74088v0

ECU ID's range shall be assigned based on the time it takes to make a SWDL for the specific ECU. The ECU that takes the longest time to perform a SWDL shall have the lowest ID and so on.

4.5.2.1.4.3 Functional logical address range

It shall be possible via a functional logical address, to send a request to all (public) ECUs in the complete vehicle. However, in programmingSession there may be ECUs on a specific network not supporting functional addressed requests where a response is expected. .Refer to the implementation specification for respective network (e.g. UDSonLIN) for more information.
A functional request message must be unsegmented on all networks, i.e. the tester shall only send functional request messages that can be transported in a single frame on all networks (with public ECUs) in the vehicle. E.g., for a LIN network the largest functional request message is 6 bytes (payload data).
The ECU shall support the functional address specified in the table below.

Table – Functional logical address range

(msb) Functional logical address range (lsb)				Comment
address range Bit 15 – 12	Domain ID Bit 11 – 8	Network ID Bit 7 – 4	ECU ID Bit 3 – 0	
0x1	0xF	0xF	0xF	All ECUs on all networks within all domains are functionally addressed

2401 v3 **Functional addressing specific networks**

Version history:
(3):

Purpose: From a testing point of view it might be good to read information from several ECUs simultaneously for example when testing network management status.
Source: Section "Functional logical address range"
Verification Method: Test
Legacy ID: 74102v2

The ECU shall support the functional logical address as specified in table *Functional logical address range* .

Table – Functional logical address range

(msb) Functional logical address range (lsb)	Comment

address range Bit 15 – 12	Domain ID Bit 11 – 8	Network ID Bit 7 – 4	ECU ID Bit 3 – 0	
0x1	0xF	0xF	0xF	All ECUs on all networks within all domains are functionally addressed

2402 v3 **Functional request shall be unsegmented on all networks**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: A functional request must be unsegmented on all networks. For CAN network the largest value is 6 bytes.
Source: Section "Functional logical address range"
Verification Method: Analysis
Legacy ID: 74104v1

The tester shall not send larger functional request than 6 bytes in the data field.

4.5.2.1.4.4 Logical address range for periodic response messages

The logical address range for periodic responses message type #2 (see *[Services_9] ISO 14229-1 Road vehicles – Diagnostics systems – Diagnostic services*), used as source address for the ReadDataByPeriodicIdentifiers service (0x2A), is defined in the table below. The Domain ID shall represent the Domain which the ECU is logically connected to.

Table– Periodic response logical address range

(msb) Periodical response address range (lsb)		
address range Bit 15 – 12	**Domain ID Bit 11 – 8**	**Bit 7 – 0**
0x1 to 0x7	0x0	0xE0 to 0xFF
0x1 to 0x7	0x1	0xE0 to 0xFF
0x1 to 0x7	0x2	0xE0 to 0xFF
0x1 to 0x7	0x3	0xE0 to 0xFF
0x1 to 0x7	0x4	0xE0 to 0xFF
0x1 to 0x7	0x5	0xE0 to 0xFF
0x1 to 0x7	0x5	0xE0 to 0xFF
0x1 to 0x7	0x6	0xE0 to 0xFF
0x1 to 0x7	0x7	0xE0 to 0xFF
0x1 to 0x7	0x8	0xE0 to 0xFF
0x1 to 0x7	0x9	0xE0 to 0xFF
0x1 to 0x7	0xA	0xE0 to 0xFF
0x1 to 0x7	0xB	0xE0 to 0xFF
0x1 to 0x7	0xC	0xE0 to 0xFF
0x1 to 0x7	0xD	0xE0 to 0xFF
0x1 to 0x7	0xE	0xE0 to 0xFF

2404 v2 **Logical address range for periodic response messages**

Version history:

(2):

Purpose: To define the source address range used for periodical response messages.
Source:
Verification Method: Analysis
Legacy ID: 74105v1

The logical address range for periodic responses message type #2 (see *[Services_9] ISO 14229-1 Road vehicles – Diagnostics systems – Diagnostic services*), used as source address for the ReadDataByPeriodicIdentifiers service (0x2A), is defined in the table below. The Domain ID shall represent the Domain which the ECU is logically connected to.

Table– Periodic response logical address range

(msb) Periodical response address range (lsb)		
address range Bit 15 – 12	**Domain ID Bit 11 – 8**	**Bit 7 – 0**
0x1 to 0x7	0x0	0xE0 to 0xFF
0x1 to 0x7	0x1	0xE0 to 0xFF
0x1 to 0x7	0x2	0xE0 to 0xFF
0x1 to 0x7	0x3	0xE0 to 0xFF
0x1 to 0x7	0x4	0xE0 to 0xFF
0x1 to 0x7	0x5	0xE0 to 0xFF
0x1 to 0x7	0x5	0xE0 to 0xFF
0x1 to 0x7	0x6	0xE0 to 0xFF
0x1 to 0x7	0x7	0xE0 to 0xFF
0x1 to 0x7	0x8	0xE0 to 0xFF
0x1 to 0x7	0x9	0xE0 to 0xFF
0x1 to 0x7	0xA	0xE0 to 0xFF
0x1 to 0x7	0xB	0xE0 to 0xFF
0x1 to 0x7	0xC	0xE0 to 0xFF
0x1 to 0x7	0xD	0xE0 to 0xFF
0x1 to 0x7	0xE	0xE0 to 0xFF

4.5.2.1.4.5 Logical address range for external diagnostic tester

The logical address range used for the external diagnostic tester is defined in the table below.

Table – External diagnostic tester logical address range

(msb) External tester logical address range (lsb)				Comment
Bit 15 – 12	**Bit 11 – 8**	**Bit 7 – 4**	**Bit 3 – 0**	
0x0	0xE	0x8	0x0	External diagnostic tester
0x0	0xE	0x8	0x1 to 0xF	Reserved
0x0	0xE	0x9 to 0xF	0x0 to 0xF	Reserved

2403 v1 **Logical address for external diagnostic tester**

Version history:
(1):

Purpose: To define the logical address used for the external diagnostic tester.
Source:
Verification Method: Analysis
Legacy ID: 74106v0

The external diagnostic tester shall be assigned the logical address 0x0E80.

4.5.2.1.4.6 ECU ability to receive requests

As long as the ECU is within an operation cycle, the ECU shall all the time be able to receive diagnostic service requests and send a response (positive or negative) on the requests. Refer to section *Entry conditions* for requirements on when the ECU shall be able to execute the received requests.

The ECU shall start an operation cycle at the latest when the ECU has completed its start-up sequence.
The ECU shall complete its start-up sequence within 2500 ms after an event that initiates a start-up sequence. The implementer defines the events that initiate a start-up sequence. However, minimum the following events shall initiate a startup sequence:
1. Power up (the power supply is connected to the ECU) or waking up from sleep (the ECU receives a request to wake up) or other equivalent modes defined by the implementer.
2. An ECU hard reset is triggered. In case of an ECU hard reset is triggered by a diagnostic service and if not otherwise is specified by the diagnostic service that trig the ECU hard reset, the ECU reset shall be triggered latest when the diagnostic service has been executed successfully. See *[Services_9] Road vehicles – Diagnostics systems – Diagnostic services*.

The ECU shall stop an operation cycle at the earliest when the ECU start to perform its shut-down sequence and at the latest when the ECU has completed its shut-down sequence and prior to the following ECU events:
1. Power down (the power supply is disconnected from the ECU).
2. Sleep mode (the ECU goes to sleep mode) or other equivalent modes defined by the implementer.

The detailed criteria for operation cycle start and stop are defined by the implementer for review and approval by CEVT Electrical Architecture.

The ECU shall be able to receive a complete request message, receiving a queued request excluded (refer to *[Services_2] UDS Session* for definition of queued request), without to halt the message , while sending of it is in progress , with the help of services in lower OSI-layer e.g network/transport layer.

All requirements from this section are also specified in section "Requirements from section ECU ability to receive requests".

2360 v4 **ECU ability to receive requests**

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: Define a time the ECU is allowed to be unavailable in regards of diagnostic communication when powering up and the time shall be short enough to not be a problem for manufacturing and aftersales.
Source: Section "ECU ability to receive requests"
Verification Method: Analysis
Legacy ID: 74107v2

As long as the ECU is within an operation cycle, the ECU shall all the time be able to receive diagnostic service requests and send a response (positive or negative) on the requests. Refer to section *Entry conditions* for requirements on when the ECU shall be able to execute the received requests.

2362 v2 ECU start up time

Version history:
(2):

Purpose: Define a time the ECU is allowed to be unavailable in regards of diagnostic communication when powering up and the time shall be short enough to not be a problem for manufacturing and aftersales.
Source: Section "ECU ability to receive requests"
Verification Method: Test
Legacy ID: 391192v1

The ECU shall complete its start-up sequence within 2500 ms after an event that initiates a start-up sequence. The implementer defines the events that initiate a start-up sequence. However, minimum the following events shall initiate a startup sequence:
1. Power up (the power supply is connected to the ECU) or waking up from sleep (the ECU receives a request to wake up) or other equivalent modes defined by the implementer.
2. An ECU hard reset is triggered. In case of an ECU hard reset is triggered by a diagnostic service and if not otherwise is specified by the diagnostic service that trig the ECU hard reset, the ECU reset shall be triggered latest when the diagnostic service has been executed successfully. See reference *[Services_9] Road vehicles – Diagnostics systems – Diagnostic services*
for definition of 'ECU hard reset'.

2361 v2 ECU operation cycle

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: Define a time the ECU is allowed to be unavailable in regards of diagnostic communication when powering up and the time shall be short enough to not be a problem for manufacturing and aftersales.
Source: Section "ECU ability to receive requests"
Verification Method: Analysis
Legacy ID: 391191v0

The ECU shall start an operation cycle at the latest when the ECU has completed its start-up sequence.

The ECU shall stop the operation cycle at the earliest when the ECU start to perform its shut-down sequence and at the latest when the ECU has completed its shut-down sequence and prior to the following ECU events:
1. Power down (the power supply is disconnected from the ECU).
2. Sleep mode (the ECU goes to sleep mode) or other equivalent modes defined by the implementer.

The detailed criteria for operation cycle start and stop are defined by the implementer for review and approval by CEVT Electrical Architecture.

2359 v2 **ECU ability to receive a complete message**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To ensure performance and predictable response times. This requirement does not apply to request and response messages gatewayed thru a gateway.
Source:
Verification Method: Analysis
Legacy ID: 128838v0

The ECU shall be able to receive a complete request message, receiving a queued request excluded (refer to *[Services_2] UDS Session* for definition of queued request), without to halt the message, while sending of it is in progress, with the help of services in lower OSI-layer e.g network/transport layer.

4.5.3 UDS Session

4.5.3.1 UDS Session generic reqs

4.5.3.1.1 Message timing definition

The timing handling during communication required by this document shall be compliant to *[Session_3] ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer services*, section “Timing handling during communication”, with the restrictions/additions as defined by this specification.

2613 v4 **Session Layer**

Version history:
(4):

Purpose: Standardise communication to ensure all ECUs uses the same diagnostic communication specifications. ISO standard shall be followed to reduce cost and make implementation easier
Source: Road vehicles - Unified diagnostic services - Part 2: Session layer services
Verification Method: Inspection
Legacy ID: 113482v3

The session layer defined in *[Session_3] Road vehicles — Unified diagnostic services — Part 2: Session layer services* shall be implemented with restriction/additions defined by the

requirements in this specification.

4.5.3.1.2 Overview

What is the purpose of this document?

This document specifies OEM specific requirements for OSI layer 5 – Session Layer Services for the diagnostic software component. The session layer services shall be compliant to ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer services see requisite documents, with the restrictions/additions as defined by the subsequent sub-sections.

This document is intended to provide guidance and requirements to architects, system designers, application developers, and suppliers that are to deliver diagnostics compliant with production vehicle programs. It shall serve as the cross car line compliance reference for implementation of vehicle diagnostics. Adherence to the specifications contained within this document is required to ensure compatibility within the corporate product line from vehicle to vehicle, model year to model year and maintain compatibility with industry driven standards.

This document specifies OEM specific requirements. If there are other kinds or requirements like legislative requirements (for example OBD requirements), these requirements may have higher priority than the requirements in this specification. However if the other requirements have performance requirements lower than the performance requirements specified in this document, the performance requirements in this document shall be followed. This is valid as long as the performance requirement in this document does not contradict timeout requirements in the other document.

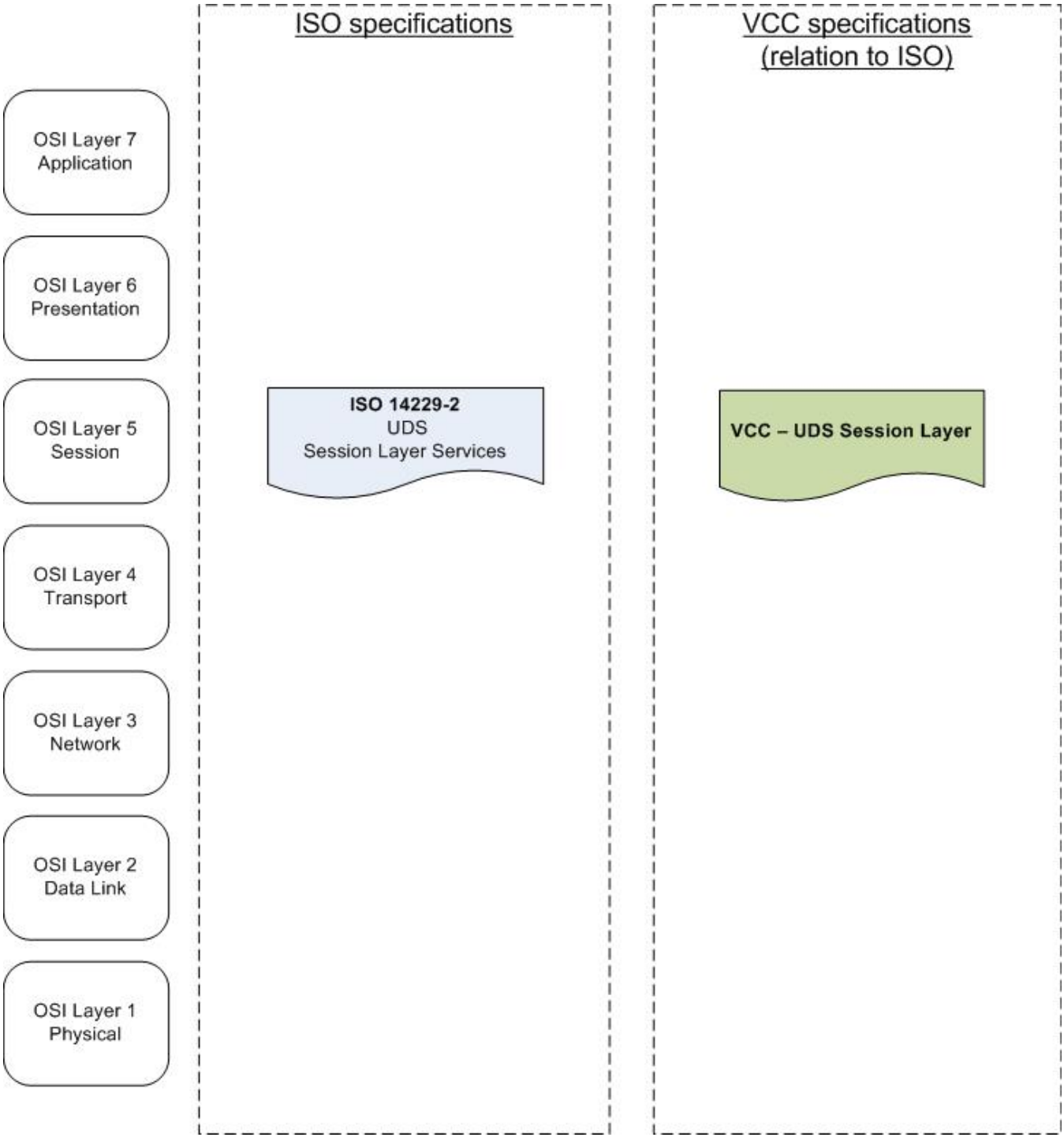


Figure - Relation to ISO document

Who should read this document?

The document is mainly intended for the developer of the actual software.

If the software product is a part of a larger system and interacts with other software products, this document also should be read and approved by the system designers and the designers responsible for each interacting software product.

4.5.3.1.2.1 Definitions and Abbrevations

Definition	Description
A_Data	A_Data is a string of byte data defined for each individual application layer service: The A_Data string shall start with the Service Identifier followed by all service specific parameters from the application layer service data unit.
Client	Function that is part of the tester and that makes use of the diagnostic services.
Edge node	An ECU (internal vehicle network node) which is the network node that is exposed towards the external network. The node implements a diagnostic over IP-protocol (DoIP - ISO13400) and provides diagnostic access to test equipment.
Final response	A final response is a positive response or a negative response other than response code 0x78 "requestCorrectlyReceived-ResponsePending".
Server	Software entity which provides services for clients
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an in-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbr.	Description
ECU	Electronic Control Unit
SPRMIB	SuppressPositiveResponseMessageIndicationBit

4.5.3.1.2.2 Requisite documents

Reference	Source	Title
[Session_1]	Internal	UDS Services
[Session_2]	Internal	DoIP Communication
[Session_3]	ISO	ISO 14229-2: Road vehicles – Unified diagnostic services - Part 2: Session layer services. Edition 1: 2013
[Session_4]	ISO	ISO 13400-2: Road vehicles – Diagnostic Communication over Internet Protocol (DoIP)

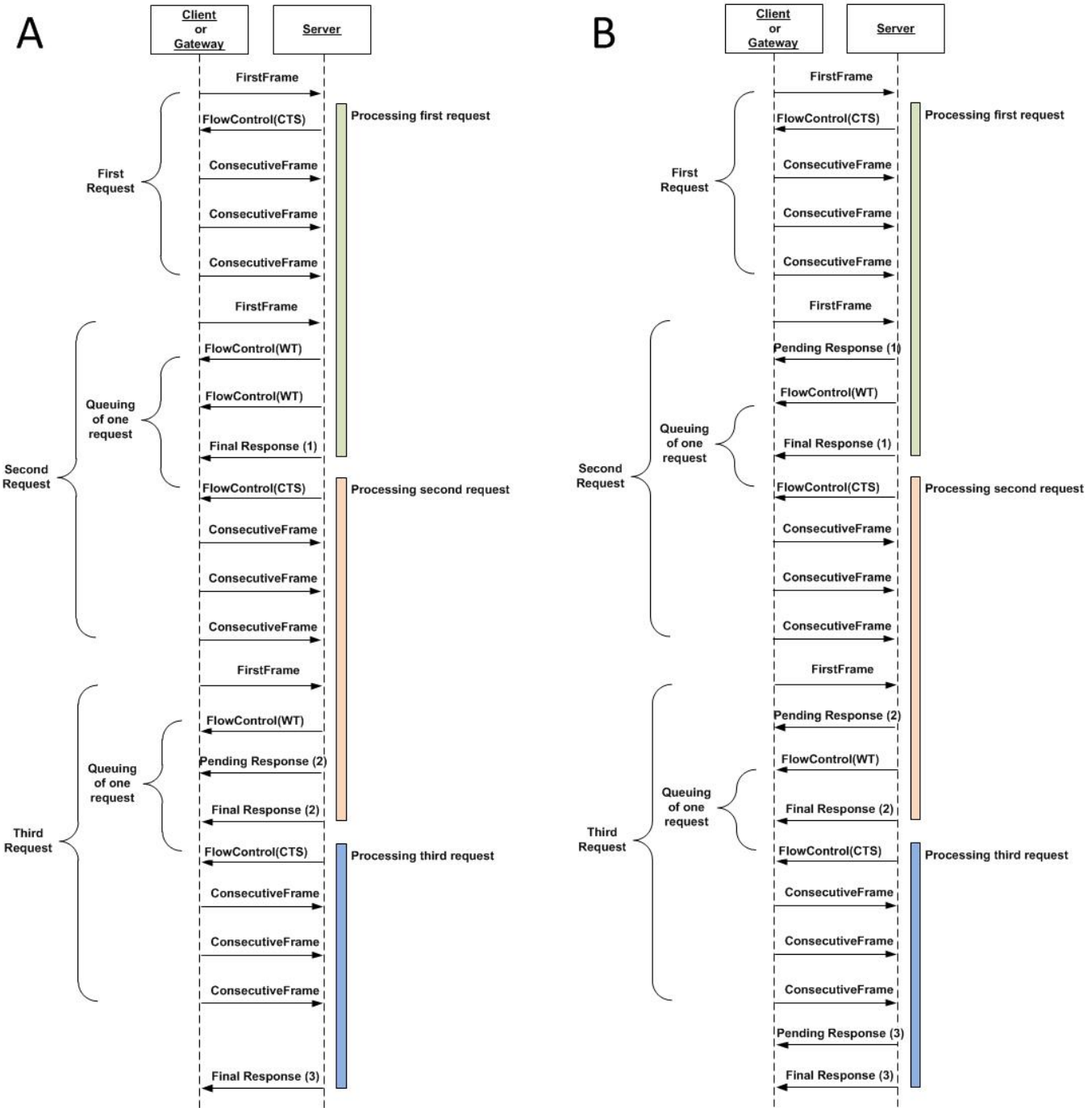
Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: http://www.iso.org/

4.5.3.2 UDS Session ECU reqs

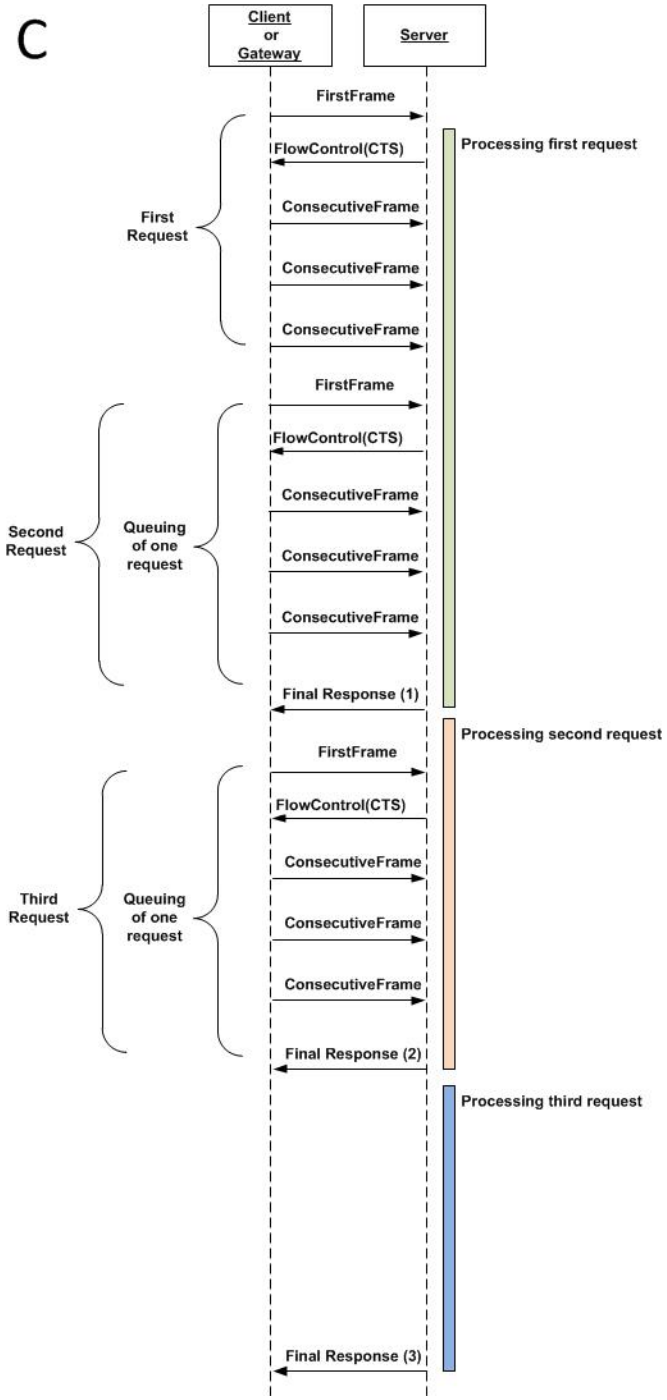
4.5.3.2.1 Appendix

4.5.3.2.1.1 Example, queuing of requests in a server

The sequence below shows examples where multiple requests are sent to a server, a typical case is in programming session where the client will send additional request to be queued in the server. The requests are typically TransferData with Block 1, 2, 3 and so on where the data is known in advance and do not depend on any data received in a previous response.



Examples of queuing of requests towards a CAN server.
The server is not able to receive complete queued message while processing the current message. A pending response, NRC 0x78, for the current message is sent by ECU in various order compared to the FC.WAIT for the queued message.

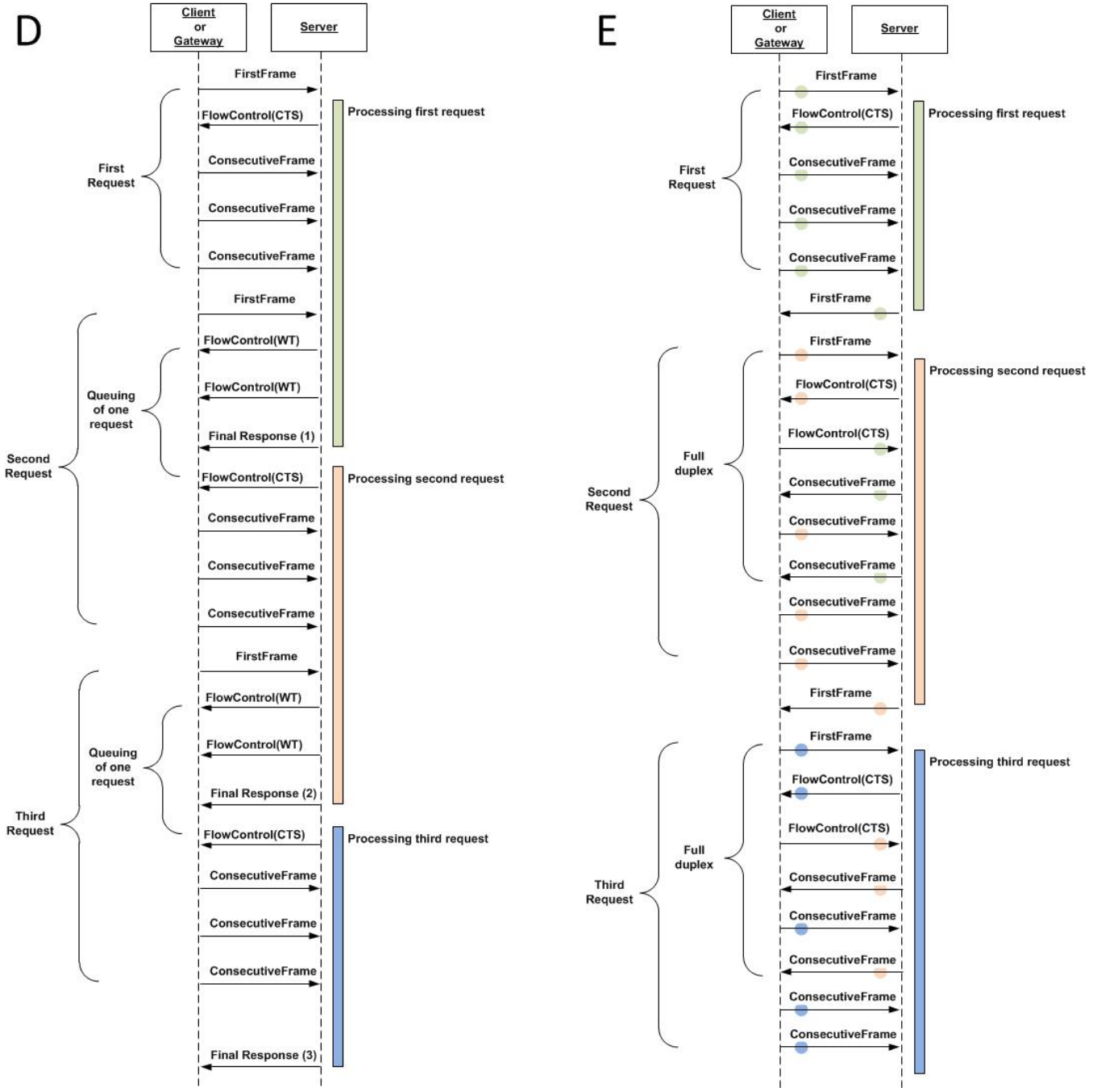


Examples of queuing of requests towards a CAN server.
The server is able to receive complete queued message, and stores it internally until the previous message has been processed and its response sent.

4.5.3.2.1.2 Example, queuing of requests versus full duplex

The purpose of the sequences below is to clarify the difference between queued requests, an OEM specific feature, and full duplex communication. If the server has not yet begun responding (i.e. has not yet sent a NRC 0x78 or a final reponse) to a previous request and a new request has been transmitted to the server, or begin to be transmitted in case of segmented request to server, the second request is considered queued. Since it is not possible

to determine by observing network communication between a request that has been processed completely inside a server but not yet responded to and a request that has not yet been processed completely by a server, the observable network communication will be used to judge if a specific sequence is queued requests or full duplex communication.
The sequences shows examples where multiple requests are sent to a server, a typical case in programming session where the client will send additional request to be queued in the server. If gateways are located between client and server, due to delays in gateways, the second request (believed by client to be queued in vehicle) may arrive to the server slightly differently now and then in time relation to the server responses. The server may already have started responding to a previous request. In that case the second request is not considered to be queued, as percieved by server, but is instead only full duplex communication.



Examples of queuing of requests towards a CAN server and full duplex communication. In sequence to left is the server has not yet begun responding to the second request and that request is therefore considered to be a queued request.
In sequence to right the server has begun the response by a First Frame (processing of request is completed but response not yet completely sent) when a second request is received. Since the (segmented) response is in progress, and a new request is received within the response transmission it is full duplex instead of queued request.

4.5.3.2.2 Timing handling during communication

The timing handling during communication required by this document is compliant to *[Session_3] ISO 14229-2, Road vehicles – Unified diagnostic services - Part 2: Session layer*

services, section “Timing handling during communication” with the restrictions/additions as defined by the subsequent sub-sections.

4.5.3.2.2.1 Queued request messages to one server

Messages sent in-between the client and the server may pass several gateways which results in greater latency, compared to if the client was directly connected to same network as the server. To make the client server communication faster, a feature is introduced named “queued request”. The method used is to send an extra request directly after sending the first request (i.e. receiving the response of the first request). The time saved by using queued request is the response time up to the client + client processing the response and then send a new request + the time for the second request to be sent from the client to the server through all gateways.
A LIN slave shall not implement support for queued request. The LIN network/transport layer standard is not allowing this feature. The LIN master is therefore required to buffer the second (queued) request.

2618 v4 **Server support of queued requests in programming session**

Version history:
(4):

Purpose: To reduce the (delta)P2Request time.
Source:
Verification Method: Test
Legacy ID: 56192v3

In programming session the server shall be able to handle the minimum of one (1) queued physical request during processing of an already received physical request which do not lead to a server reset , meaning the server shall store the second request in a FIFO queue for later processing.
If the server isn’t able to receive the complete second message due to lack of receive buffer space the server shall halt the message with the help of the network/transport layer e.g. WAIT on CAN/FlexRay or window size on TCP. Messages that can be fully received in addition to the minimum one queued request shall be stored in queue. If the queue is full, any further requests shall be ignored.

2619 v2 **Start of P2Server for queued request in programming session**

Version history:
(2):

Purpose: To reduce latency when the server shall start process the queued request.
Source:
Verification Method: Test
Legacy ID: 56194v1

In programming session, the server shall start the $P2_{Server}$ timer for the queued request once the final response (indicated via Server T_Data.con) of the previous request is transmitted.
If the queued request is not completely received at the server when the final response of the previous request is transmitted (e.g. the message is halted or a large queued request is received) shall the server start the $P2_{Server}$ timer when the complete queued message is received (indicated via Server T_Data.ind).

2616 v2 **Order of responses in programming session**

Version history:
(2):

Purpose: The order the server responds to queued requests must be defined.
Source:
Verification Method: Test
Legacy ID: 56197v1

In programming session, physical diagnostic requests addressed to a server shall be processed and responded to in the order they were received.

2617 v6 **Server behavior for additional client requests in programming session**

Version history:
(6):

Purpose: If the server is processing a request and receives a request that is supposed to be placed in the receive queue and the receive queue is full, a predictable behaviour is required.
Source:
Verification Method: Test
Legacy ID: 56200v5

In programming session, if the server is processing a request and it receives a request that is supposed to be placed in the receive queue and the receive queue is full; the request just received shall be ignored.
Note: This requirement applies regardless of the actual size of the receive queue.
Example assuming minimum 1 queued request is required; If the server is processing a first request and queues a second request, and then a third request appears, the third request shall be ignored unless it can be queued as well.

4.5.3.2.2.2 P4 definition

P4Server_max is the maximum time allowed for a server from reception of a request until start of transmission of the final response. P4Server_max is specific for each diagnostic request the server supports.

2614 v2 **P4Server_max definition**

Version history:
(2):

Purpose: To define and set the performance timing conditions how long time the server is allowed to process a diagnostic request.
Source:
Verification Method: Test
Legacy ID: 56203v1

The timing parameter $P4_{Server_max}$ is the maximum time between the reception of a request (Server T_Data.ind) and the start of transmission of the final response (Server T_Data.req).

Note: A final response is a positive response or a negative response other than response code 0x78 “requestCorrectlyReceived-ResponsePending". In case of a request to schedule periodic responses, the initial positive or negative response that indicates the acceptance or non-acceptance of the request to schedule periodic responses shall be considered the final response.

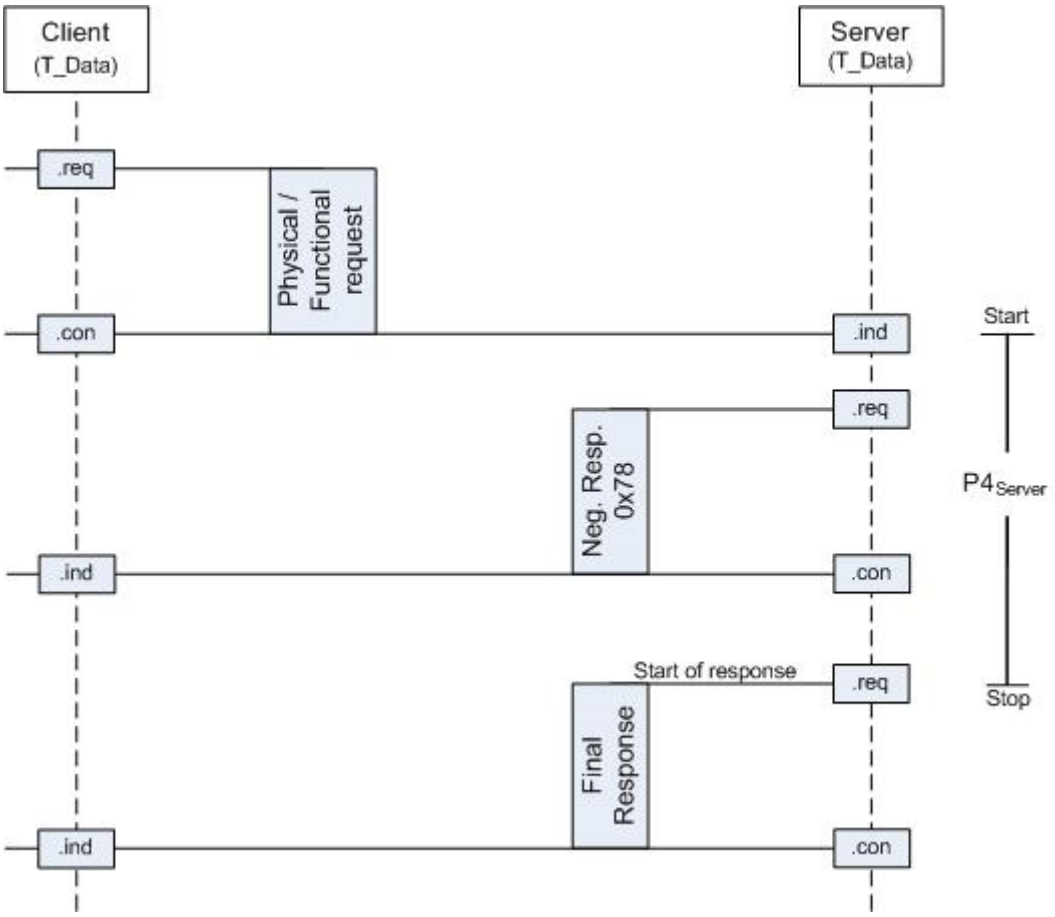


Figure – Definition P4 $_{Server}$ is the actual time which is allowed to be maximum P4 $_{Server_max}$

2615 v2 **P4Server_max equal to P2Server_max**

Version history:
(2):

Purpose: To define the behaviour when the P4Server_max is equal to P2Server_max
Source:
Verification Method: Test
Legacy ID: 56204v1

The server is not allowed to response with a negative response code 0x78 (requestCorrectlyReceived-ResponsePending) if P4 $_{Server_max}$ is the same as P2 $_{Server_max}$.
Note: The value of P2 $_{Server_max}$ is defined in section Timing parameters. P4 $_{Server_max}$ is defined for each diagnostic service in *[Session_1] UDS Services*.

4.5.3.2.3 Timing parameters

The definition of the session layer timing parameters required by this document is compliant to *[Session_3] ISO 14229-2* with the restrictions/additions as defined by the subsequent sub-sections.

4.5.3.2.3.1 Message timing definition

4.5.3.2.3.1.1 P2Server

2622 v3 — P2Server_max - non programming session

Version history:
(3):

Purpose: P2Server_max is the maximum time for the server to start with the response message after the reception of a request message.
Source:
Verification Method: Test
Legacy ID: 56151v2

The maximum time for $P2_{Server}$ in all sessions except programmingSession is 50 ms.

2623 v2 — P2Server_max - ProgrammingSession

Version history:
(2):

Purpose: P2Server_max is the maximum time for the server to start with the response message after the reception of a request message
Source:
Verification Method: Test
Legacy ID: 56150v1

The maximum time for $P2_{Server}$ in programmingSession is 25 ms.

2624 v2 — P2server_min

Version history:
(2):

Purpose: P2Server_min is the minimum time for the server to start with the response message after the reception of a request message.
Source:
Verification Method: Analysis
Legacy ID: 56148v1

The minimum time for $P2_{Server}$ in all sessions is 0ms.

4.5.3.2.3.1.2 P2(star)Server

2620 v2 — P2(star)Server_max

Version history:
(2):

Purpose: P2*Server_max is the maximum time for the server to start with the response message after the transmission of a negative response 0x78 (enhanced response timing).
Source:
Verification Method: Test
Legacy ID: 56155v1

The maximum time for P2* $_{Server}$ in all sessions is 5000 ms.

2621 v3 P2(star)Server_min

Version history:
(3):

Purpose: P2*Server_min is specified differently for final response and pending response. For final response it shall be as short as possible to allow good timing performance. For pending response it must be a suitable value so that a server do not significantly increase bus load by consecutive pending responses.
Source:
Verification Method: Test
Legacy ID: 56154v2

For a final response the minimum time for P2* $_{Server}$ in all sessions is 0ms.
During the enhanced response timing, the minimum time between the transmission of consecutive negative messages (each with negative response code 0x78) shall be 0.3 * P2* $_{Server_max}$, in order to avoid flooding the data link with unnecessary negative response messages.

4.5.4 UDSonCAN

4.5.4.1 UDSonCAN - Generic

4.5.4.1.1 Transport_Network Layer

The transport/network layer required by this document shall be compliant to *[UDSonCAN_4] DoCAN*.

2656 v2 Transport_Network layer

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To define the transport/network layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 55630v0

The CAN transport/network layer required by this document shall be compliant with

[UDSonCAN_4] DoCAN.

4.5.4.1.2 Data Link Layer

The data link layer required by this document shall be compliant to *[UDSonCAN_4] CAN Data Link Layer Specification*.

2653 v3 **CAN data link layer**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: To define the data link layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 55631v1

The data link layer required by this document shall be compliant to *[UDSonCAN_4] CAN Data Link Layer Specification*.

4.5.4.1.3 Physical Layer

The physical layer required by this document shall be compliant to *[UDSonCAN_8] CAN Physical Layer Specification*.

2654 v2 **CAN physical layer**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To define the physical link layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 240274v0

The physical layer required by this document shall be compliant to *[UDSonCAN_8] CAN Physical Layer Specification*.

4.5.4.1.4 Introduction

This document specifies OEM specific requirements for an ECU on the CAN network. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing an ECU connected to a CAN network. Figure - " *Document structure*" , shows the specifications that are related for a CAN ECU.

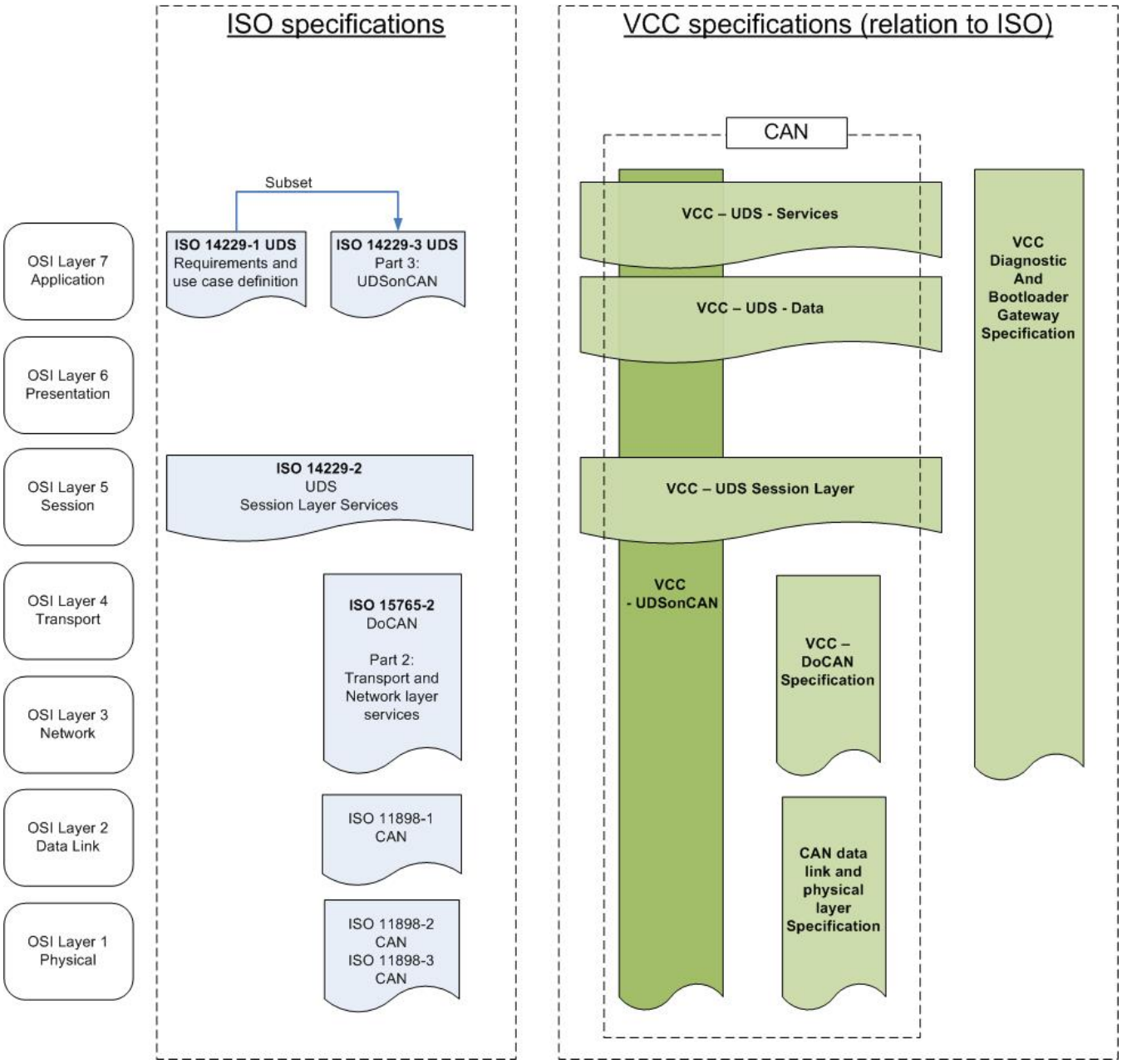


Figure - Document structure

4.5.4.1.4.1 Requisite documents

Note that in the case of any requirement conflict between this document and any of the referenced documents, the proposed solution to the conflict shall be approved by CEVT Electrical Architecture.

Reference number	Source	Title
[UDSonCAN_1]	Internal	UDS Services
[UDSonCAN_2]	Internal	UDS Data
[UDSonCAN_3]	Internal	UDS Session
[UDSonCAN_4]	Internal	DoCAN

[UDSonCAN_5]	Internal	CAN Data Link Layer Specification
[UDSonCAN_6]	Internal	Diagnostic And Bootloader Gateway Specification
[UDSonCAN_7]	Internal	Autosar Network Management- additional requirements
[UDSonCAN_8]	Internal - Base Technology DPR	CAN Physical Layer Specification
[UDSonCAN_9]	Internal	Bootloader_UDSonCAN
[UDSonCAN_10]	Internal - CarCom Database	Global Master Reference Database
[UDSonCAN_11]	Internal	General diagnostic guideline
[UDSonCAN_12]	ISO	ISO 14229-3: Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN) Edition 1, 2012

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

4.5.4.1.4.2 Definitions and Abbreviations

Definition	Description
CANH	HIGH-level CAN bus line
CANL	LOW-level CAN bus line
Do not care	Implementation of the requirement is not required and not forbidden. The implementer may decide if to implement or not. If implemented it will not be verified and shall not be expected to be supported by tools.
Missing frame master	The missing frame master for a network is the ECU which is connected to a higher hierarchal network and act as a gateway for diagnostic request messages from the higher hierarchal network to the ECUs on the network. See *General diagnostic guideline.*
Missing frame slave	All ECUs, except the missing frame master, on a network. See *Missing frame guideline* requisite documents.
Network configuration master	The ECU that is responsibly for the network configuration.
Private ECU	An ECU not addressable from a client/tester point of view. Note: *An ECU can partly be public and partly private i.e. some diagnostic services and diagnostic data supported by the ECU are addressable and some are not.*
Public ECU	An ECU addressable from a client/tester point of view.
Public ECU Master	A public ECU responsible to act as a diagnostic server for one or more private ECUs.
Tester	A system that controls functions such as test, inspection, monitoring,

	or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
TRC	The Test Run Criteria defines the criteria that must be met for the DTC test to run, i.e. for the DTC test fault detection logic to execute and the DTC fault detection counter to be up-dated. *See UDS Services* requisite documents for further detailed explanation.

Abbreviation	Description
CAN	Controller Area Network
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
MFCS	Missing frame Counter Signal. 1 byte counter sent by the missing frame master which all missing frame slaves shall use to ensure that the missing frame slave has continuous network access.
TRC	Test Run Criteria
UDS	Unified Diagnostic Services

4.5.4.1.4.3 Overview

This document specifies the CAN specific diagnostic requirements that shall apply to a CAN based ECU. All diagnostic messages on CAN shall be sent and received according to this and referenced documents in section Requisite documents .

Additional requirements are added if the ECU shall act as a diagnostic gateway to a sub network which is defined in the *[UDSonCAN_6] Diagnostic And Bootloader Gateway Specification*.

Depending on configuration, this document may consist in different combinations of the following sections:

– Generic CAN ECU (Applicable to all diagnosable and/or programmable CAN ECUs)

– Public CAN ECU (Applicable to all CAN ECUs supporting a total diagnostic implementation)

– Private Programmable CAN ECU (Applicable for CAN ECUs with "limited" diagnostic implementation)

For those ECUs who implements a "limited" diagnostic functionality to support software download the Public ECU master is responsible on the private programmable CAN ECU behalf to response on DTCs etc.

Note

This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

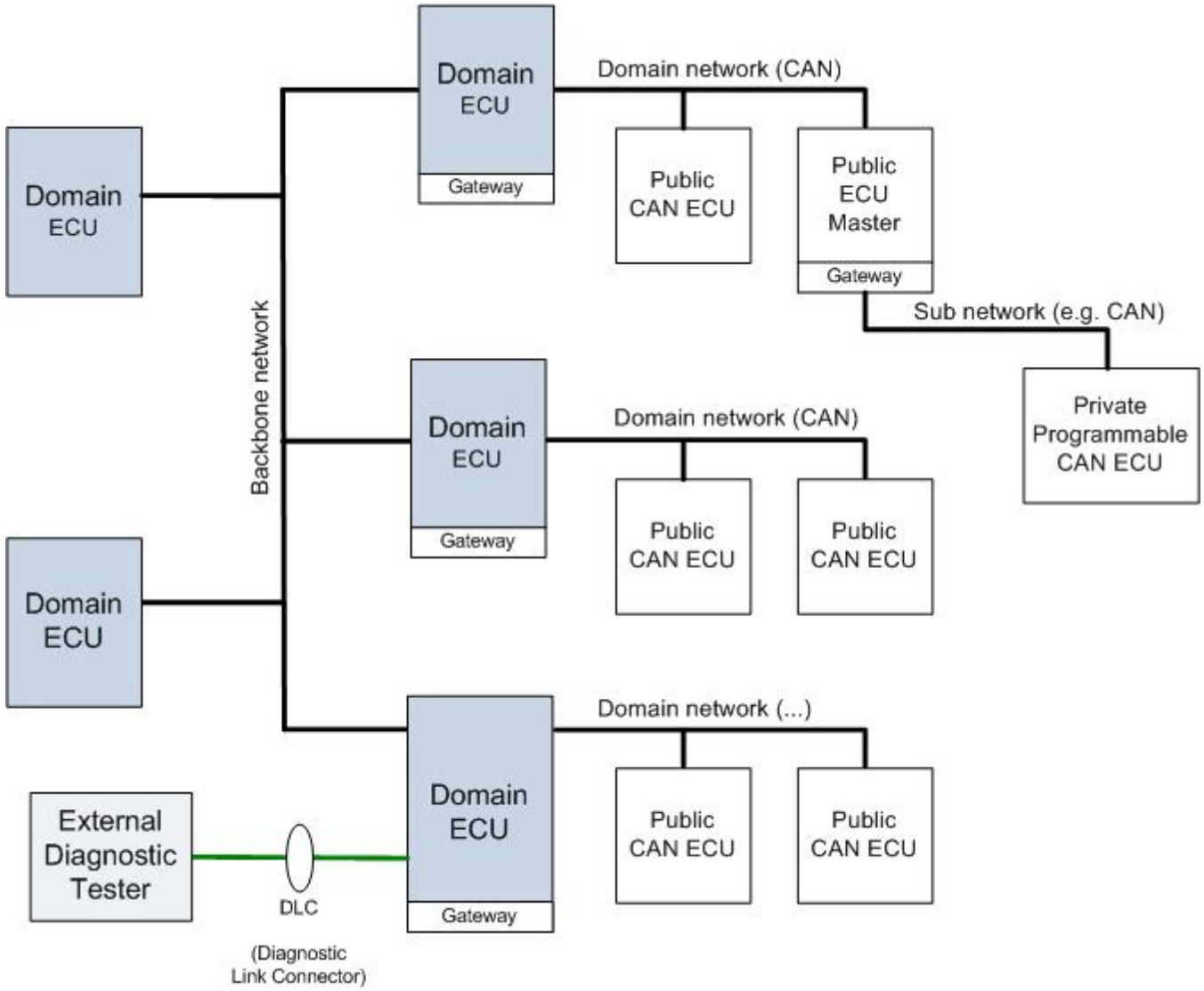


Figure: Example of the vehicle network topology

4.5.4.1.5 Application Layer

The application layer services shall be compliant to *[UDSonCAN_12] Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN)*.

The ECU application shall be compliant to *[UDSonCAN_9] Bootloader_UDSonCAN* in regards of what to do when making an ECU reset as well as when receiving a request to go to programming session.

2652 v2 **Application layer**

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To define the application layer used for communicating on CAN
Source:
Verification Method: Analysis
Legacy ID: 55619v0

The application layer shall be compliant to *[UDSonCAN_12] Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN).*

2651 v3 Application bootloader requirement

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define application behavior when transition to the bootloader and vice versa.
Source:
Verification Method: Analysis
Legacy ID: 55626v1

The application shall comply with the requirements as specified in *[UDSonCAN_9] Bootloader_UDSonCAN, chapter Flow chart diagram (Application).*

4.5.4.1.6 Session Layer

The session layer required by this document shall be compliant to *[UDSonCAN_3] UDS Session* with the restrictions/additions as defined by the subsequent sub-sections.

2655 v2 Diagnostic Session Layer

Version history:
(2): Change the Verification Method from Test to Analysis

Purpose: To define the Session layer used by the application.
Source:
Verification Method: Analysis
Legacy ID: 55627v0

The session layer required by this document shall be compliant to *[UDSonCAN_3] UDS Session*.

4.5.4.1.6.1 Addressing USDT frames

Since 11 bit CAN address mode with normal addressing format (see *[UDSonCAN_4] DoCAN)* is chosen, a fixed transmission ID is used. The ECU shall use the response ID specified in the table below - ECU Diagnostic Reception / Transmission CAN IDs as N_TA (Session layer Target Address).

ECU Reception ID Tester Transmit Request)	**ECU Transmission ID** (Tester Receive Request)
0x700–0x70F (1)	0x600–0x60F (1)
0x710	0x610
0x711	0x611
0x712	0x612
…	…
0x71E	0x61E
0x7FF	Functional request to all networks within all domains

Table - ECU Diagnostic Reception / Transmission CAN IDs

Note (1): The ECU reception ID from 0x700 to 0x70F is reserved due to the domain ECU always shall have network ID equal to zero(0).

Each ECU shall support a total of three (3) USDT CAN diagnostic IDs:
- Physically addressed requests (or ECU Diagnostic Reception ID).
- Physically and functionally addressed responses or (ECU Diagnostic Transmission ID). The ECU Transmission ID is always equal to the value of the
  ECU Reception ID minus 0x100.
- Functionally addressed requests to all ECUs in all domains. Value 0x7FF.

Conversion from the Logical address (ECU Address) to 11 bit CAN ID.
Example: If the logical address (i.e. TA and SA) is 0x1234 (Domain ID=0x2, Network ID=0x3 and ECU ID=0x4) the CAN ID 0x734 shall be used.

2650 v4 **Addressing USDT frames**

Version history:
(4): Change the Verification Method from Test to Analysis

Purpose: Define which CAN IDs an ECU shall listen on
Source:
Verification Method: Analysis
Legacy ID: 55628v2

Each ECU shall support a total of three (3) USDT CAN diagnostic IDs.
· Physically addressed requests (or ECU Diagnostic Reception ID).
· Physically and functionally addressed responses or (ECU Diagnostic Transmission ID). The ECU Transmission ID is always equal to the value of the ECU Reception ID minus 0x100.
· Functionally addressed requests to all ECUs in all domains. Value 0x7FF.

4.5.4.2 UDSonCAN - Public CAN ECU

4.5.4.2.1 Diagnostic Data

The diagnostic data required are specified in *[UDSonCAN_2] UDS Data,* with the restrictions and additions in subsequent subsections.

2657 v2 **Application diagnostic data**

Version history:
(2):

Purpose: Define which data parameters each CAN ECU shall or may support.
Source:
Verification Method: Analysis
Legacy ID: 55606v1

The CAN application shall support the diagnostic data as defined in *[UDSonCAN_2] UDS Data*.

4.5.4.2.1.1 Record Data

This section describes specific CAN record data requirements.

Data record			Notes	M/O
Identifier (hex)	**Name**	**Definition**		
Defined in *[UDSonCAN_10] Global Master Reference Database*	Network management state history buffer	See *[UDSonCAN_7] Autosar Network Management - additional requirements* for additional details.		M

Table - Summary of CAN specific record data

Abbreviations in the table above

M Mandatory for all CAN ECUs.

2790 v2 Network management history record data

Version history:
(2):

Purpose: Network management concept validation.
Source: Section "Record data"
Verification Method: Test
Legacy ID: 55612v1

Each ECU shall implement one CAN network management state history buffer DID per CAN network the ECU is connected to. The network management state history buffer DID shall follow the definition specified in *[UDSonCAN_7] Autosar Network Management - additional requirements.*

2791 v3 Network management state history buffer _ Identifier

Version history:
(3):

Purpose: Define the record data DID
Source: Section "Record data"
Verification Method: Test
Legacy ID: 55613v1

CAN ECUs shall implement network management state history buffer DID as defined in *[UDSonCAN_10] Global Master Reference Database*.

4.5.4.2.1.2 Diagnostic trouble code information

The table below specifies additional CAN requirements relative the general set of requirements in *[UDSonCAN_2] UDS Data*. Further reference on how to implement and use the missing frame concept can be found in *[UDSonCAN_11] General diagnostic guideline*.

Type of	Item	Detected fault	Test Period	Test Run Criteria	Failure	DTC ID	M/O

Detection			Time	(TRC) See note 1	type (hex)		
Public ECU network interface	1	Bus-off		When the ECU enters or exit the bus-off state. See note 8.	00 or 88. See note 9.	See note 6.	M
	2	Frame configuration error		Every time configuration control has entered the state "check completed". See note 8.	00	See note 6.	MC2
	3	Permanent missing frame	Note 3	Note 2	00	See note 6.	MC1
	5	CANH circuit short to battery voltage	At least once per 100 ms.		00	See note 6 and 7.	MC1
	6	CANL circuit short to ground	At least once per 100 ms.		00	See note 6 and 7.	MC1
	7	CANH circuit short to ground	At least once per 100 ms.		00	See note 6 and 7.	MC1
	8	CANL circuit short to battery voltage	At least once per 100 ms.		00	See note 6 and 7.	MC1
	9	CANH circuit short to CANL circuit	At least once per 100 ms.		00	See note 6 and 7.	MC1
	10	Common CAN electrical fault	At least once per 100 ms.		01	See note 6 and 7.	MC1

Table - Summary of CAN related external circuit faults

Abbreviations inthe table above:

M Mandatory.
MC1 Mandatory if the ECU is the missing frame master.
MC2 Mandatory if the ECU is not the network configuration master

Notes inthe table above:

1. If not otherwise stated, Generic Test Run Criteria shall apply, see *[UDSonCAN_2] UDS Data.*

2. Following aspects of missing frame TRC have to be considered:
   1. The missing frame master, and not the missing frame slaves, shall manage all the enabling and disable conditions (i.e. set TRC to true or false) for the missing frame DTCs, both for permanent and intermittent missing frame.
   2. The missing frame detection shall only be based on periodic frames sent in at least the usage mode "Driving".
   3. All parameters used as test run criteria for missing frame tests shall be received outside the network that is being tested.
   4. The testing for missing frames shall only be done when the frame is expected; for example missing frame master shall be configured such that it is aware if an optional missing frame slave is present (mounted or not) on the bus or not.
   5. The missing frame master shall be aware of different operation modes preventing the missing frame slaves to send frames. For example if a missing frame slave is in power-off, low-power, sleep or equivalent mode, the missing frame slave might be prevented from sending frames. The missing frame master shall be aware when the missing frame slave is prevented from sending frames and shall not set any missing frame DTCs during

this these modes.

6. Missing frame tests shall only be done when the ECU is the usage mode "Driving".
7. Missing frame tests shall never be done during combustion engine crank.
8. The missing frame master shall never set TRC true until 5±2 seconds after all criteria for a DTC test, for permanent and intermittent missing frame DTCs, to run, is satisfied. This inhibit time period include the same inhibit time period, that starts after transition from one usage to another or one car mode to another, specified as Generic Test Run Criteria in *[UDSonCAN_2] UDS Data* (i.e. no additional time period is needed).
9. The voltage of the vehicle battery, Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V. Note: The battery voltage shall be measured as close to the battery as possible in order to minimize the measuring fault.
10. If a missing frame master does not receive the missing frame TRC no missing frame DTCs shall be set.
11. Missing frame tests shall never be done when fault is detected on the parameters used as test run criteria (e.g., when the value of parameters, used as test run criteria, distributed on an in-vehicle network is not updated (indicated by an update bit) or not correct (indicated by a quality factor)).

3. Each missing frame slave shall periodically send at least one frame with a shorter periodicity than 100 ms. To save bandwidth on the CAN net, first check if there is an existing frame the missing frame master can perform the DTC test on. Secondly the send frequency of an existing frame shall be increased to match the 100 ms requirement. The missing frame master shall check permanent missing frame DTCs every 150ms-170ms.

4. Not in use

5. Not in use

6. Defined in *[UDSonCAN_10] Global Master Reference Database*.

7. The following faults specified in the table may be identified by one common DTC, the DTC specified by item 10 of the table, instead of one single DTC for each fault if approved by CEVT Electrical Architecture:
Item 5: CANH circuit short to battery voltage
Item 6: CANL circuit short to ground
Item 7: CANH circuit short to ground
Item 8: CANL circuit short to battery voltage
Item 9: CANH circuit short to CANL circuit

8. The inhibit criteria “during 5 seconds from any transition from one usage mode to another” according to the Generic Test Run Criteria, specified in *[UDSonCAN_2] UDS Data*, is not applicable for this DTC.

9. The failure type shall be 0x00 if the ECU is emission related and 0x88 if the ECU is not emission related.

Item no.	Test sample failed criteria	Increase FDC10 (# of steps)	Test sample passed criteria	Decrease FDC10 (# of steps)	Unconfirmed DTCLimit	FDC10 max value	DTCConfirmedLimit	DTCAged Limit
1	The reported value of the DTC fault	According to *Autosar*	The reported value of the DTC fault	According to Autosar	The same value as the	127	1	255

	detection counter shall increase when the CAN controller is in bus-off state.	*Network Management-additional requirements*, VCC - UDSonCAN - Requisite documents	detection counter shall decrease when the CAN controller is in bus-on (i.e. not in bus-off) state.	Network Management-additional requirements, VCC - UDSonCAN - Requisite documents	increase value			
2	The reported value of the DTC fault detection counter shall increase when the ECU fails to recognize correct configuration identification.	127	The reported value of the DTC fault detection counter shall decrease as soon as the ECU succeeds in recognizing configuration identification.	255	127	127	1	1
3	The reported value of the DTC fault detection counter, for a specific ECU, shall increase when t he missing frame master have not received the periodic missing frame slave frame, for that specific ECU, in the test period time.	9	The reported value of the DTC fault detection counter, for a specific ECU, shall decrease when t he missing frame master have received the periodic missing frame slave frame, for that specific ECU, in the test period time.	9	18	127	3	255
5	The reported value of the DTC fault detection counter shall increase when CANH circuit short to battery voltage is detected.	3	The reported value of the DTC fault detection counter shall decrease when not CANH circuit short to battery voltage is detected.	2	5	127	3	255
6	The reported value of the DTC fault detection counter shall increase when CANL circuit short to ground is detected.	3	The reported value of the DTC fault detection counter shall decrease when not CANL circuit short to ground is detected.	2	5	127	3	255
7	The reported value of the DTC fault detection counter shall increase when CANH circuit short to ground is detected.	3	The reported value of the DTC fault detection counter shall decrease when not CANH circuit short to ground is detected.	2	5	127	3	255
8	The reported value of the DTC fault detection counter shall increase when CANL circuit short to battery voltage is detected.	3	The reported value of the DTC fault detection counter shall decrease when not CANL circuit short to battery voltage is detected.	2	5	127	3	255
9	The reported value of the DTC fault detection counter shall increase when CANH circuit short to CANL circuit is detected.	3	The reported value of the DTC fault detection counter shall decrease when not CANH circuit short to CANL circuit is detected.	2	5	127	3	255
10	The reported value	3	The reported value	2	5	127	3	255

	of the DTC fault detection counter shall increase when the ECU detects that at least one of the physical bus error flags is set.		of the DTC fault detection counter shall decrease when the ECU detects that none of the physical bus error flags is set.					

Table – Calibration of CAN DTCs

2750 v3 **DTC Generic Test Run Criteria**

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Ensure correct setting of DTCs
Source:
Verification Method: Analysis
Legacy ID: 340531v1

If not otherwise stated, Generic Test Run Criteria shall apply, see *[UDSonCAN_2] UDS Data*.

2659 v2 **Bus-off**

Version history:
(2):

Purpose: The bus-off state means there has been a fault in sending or receiving CAN frames and usually this will make customer observable symptoms and hence a DTC shall be set showing that the network has been in bus-off state.
Source: Table: "Summary of CAN related external circuit faults" Item 1
Verification Method: Test
Legacy ID: 55684v1

All ECUs on the network shall be able to detect and set bus-off DTCs when the CAN controller has gotten into the state bus-off see *[UDSonCAN_7] Autosar Network Management- additional requirements* for further reference.

2667 v3 **Bus-off _ Test period**

Version history:
(3):

Purpose: Preferably the bus-off state shall be detected via interrupt but if not possible a minimum standardized frequency shall be used.
Source: Table: "Summary of CAN related external circuit faults" Item 1
Verification Method: Test
Legacy ID: 55693v2

The bus-off check shall be tested according to specified by *[UDSonCAN_7] Autosar Network Management- additional requirements* .

2670 v3 **Bus-off _ TRC**

Version history:
(3):

Purpose: A standardized test run criteria shall be used to make it easier for fault tracers to know that the test has been performed.
Source: Table: "Summary of CAN related external circuit faults" Item 1
Verification Method: Test
Legacy ID: 55699v2

The TRC to perform the bus-off check shall be according to the General Test Run Criteria specified in *[UDSonCAN_2] UDS Data*. However, the inhibit criteria “during 5 seconds from any transition from one usage mode to another” according to the Generic Test Run Criteria is not applicable for this DTC.

2664 v3 Bus-off _ Failure type

Version history:
(3):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 1
Verification Method: Test
Legacy ID: 55702v2

For bus-off DTCs the failure type shall be 0x00 if the ECU is emission related and 0x88 if the ECU is not emission related.

2661 v2 Bus-off _ DTC ID

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 1
Verification Method: Test
Legacy ID: 55704v1

The bus-off DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database*.

2668 v3 Bus-off _ Test sample failed criteria

Version history:
(3):

Purpose: Each time the CAN controller gets into the state bus-off, the ECU shall consider the DTC as set.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Test

Legacy ID: 55705v2

The reported value of the bus-off DTC fault detection counter shall increase according to specified by *[UDSonCAN_7] Autosar Network Management- additional requirements* .

2666 v3 **Bus-off _ Increase FDC10 steps**

Version history:
(3):

Purpose: Each time the CAN controller gets into the state bus-off, the ECU shall consider the DTC as set.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Analysis
Legacy ID: 55723v2

The bus-off FDC10 shall be increase according to specified by *[UDSonCAN_7] Autosar Network Management- additional requirements .*

2669 v4 **Bus-off _ Test sample passed criteria**

Version history:
(4):

Purpose: Each time the CAN controller gets into the state bus-on, the ECU shall consider the DTC as not set.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Test
Legacy ID: 55725v3

The reported value of the bus-off DTC fault detection counter shall decrease according to specified by *[UDSonCAN_7] Autosar Network Management- additional requirements*.

2660 v3 **Bus-off _ Decrease FDC10 steps**

Version history:
(3):

Purpose: Each time the CAN controller exits the state bus-off, the ECU shall consider the DTC as set.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Test
Legacy ID: 55741v2

The bus-off FDC10 shall be decrease according to specified by *[UDSonCAN_7] Autosar Network Management- additional requirements* .

2662 v2 **Bus-off _ DTCAgedLimit**

Version history:
(2):

Purpose: Due to the seriousness of the DTC.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Test
Legacy ID: 55782v1

The bus-off DTC shall use DTCAgedLimit = 255.

2663 v2 **Bus-off _ DTCConfirmedLimit**

Version history:
(2):

Purpose: Due to the nature of the DTC, the DTC shall be considered confirmed as soon as a test failed has been run.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Test
Legacy ID: 55781v1

The bus-off DTC shall use DTCConfirmedLimit = 1.

2665 v2 **Bus-off _ FDC10 max value**

Version history:
(2):

Purpose: If a Bus-off occurs it is assumed it has such a big impact on the functionality of the system that it shall cause setting the confirm bit of the DTC.
Source: Table "Calibration of CAN DTCs" Item 1
Verification Method: Test
Legacy ID: 66341v1

The DTC test for Bus-off shall have FDC10 max value = 127.

2671 v2 **Bus-off _ UnconfirmedDTCLimit**

Version history:
(2):

Purpose: Due to the nature of the DTC, the DTC shall be considered unconfirmed as soon as a test passed has been run.
Source: Table: "Calibration of CAN DTCs" Item 1
Verification Method: Test
Legacy ID: 55761v1

The bus-off DTC test shall use UnconfirmedDTCLimit = the same value as the increase value of the FDC10.

2789 v2 **Separating type of electrical fault**

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table - Summary of CAN related external circuit faults
Verification Method: Test
Legacy ID: 179113v1

The following faults specified in *Table - Summary of CAN related external circuit faults* may be identified by one common DTC, the DTC specified by item 10 of the table, instead of one single DTC for each fault if approved by CEVT Electrical Architecture:
Item 5: CANH circuit short to battery voltage
Item 6: CANL circuit short to ground
Item 7: CANH circuit short to ground
Item 8: CANL circuit short to battery voltage
Item 9: CANH circuit short to CANL circuit.

2672 v2 CANH circuit short to battery voltage

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table: "Summary of CAN related external circuit faults" Item 5
Verification Method: Test
Legacy ID: 56057v1

If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANH circuit short to battery voltage. Note that there shall be one unique DTC for each public network in the vehicle.

2680 v2 CANH circuit short to battery voltage _ Test period

Version history:
(2):

Purpose: A standard time shall be used in detecting all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 5
Verification Method: Test
Legacy ID: 56058v1

The CANH circuit short to battery voltage check shall be tested periodically at least every 100 ms.

2683 v2 CANH circuit short to battery voltage _ TRC

Version history:

(2):

Purpose: Default test run criteria shall be used for all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 5
Verification Method: Test
Legacy ID: 56059v1

The TRC to perform the CANH circuit short to battery voltage check shall be according to the general Test Run Criteria specified in *[UDSonCAN_2] UDS Data*.

2677 v2 CANH circuit short to battery voltage _ Failure type

Version history:
(2):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 5
Verification Method: Test
Legacy ID: 56060v1

For CANH circuit short to battery voltage DTCs the failure type 0x00 shall be used.

2674 v2 CANH circuit short to battery voltage _ DTC ID

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 5
Verification Method: Test
Legacy ID: 56061v1

The CANH circuit short to battery voltage DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database*.

2681 v2 CANH circuit short to battery voltage _ Test sample failed criteria

Version history:
(2):

Purpose: Define when to increment FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56062v1

The reported value of the DTC fault detection counter shall increase when CANH circuit short to battery voltage is detected.

2679 v2 **CANH circuit short to battery voltage _ Increase FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56070v1

The CANH circuit short to battery voltage FDC10 shall be increased by 3 at every test sample failed.

2682 v2 **CANH circuit short to battery voltage _ Test sample passed criteria**

Version history:
(2):

Purpose: Define when to decrement FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56078v1

The reported value of the DTC fault detection counter shall decrease when not CANH circuit short to battery voltage is detected.

2673 v2 **CANH circuit short to battery voltage _ Decrease FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56079v1

The CANH circuit short to battery voltage FDC10 shall be decreased by 2 at every test sample passed.

2684 v2 **CANH circuit short to battery voltage _ UnconfirmedDTCLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56081v1

The CANH circuit short to battery voltage DTC test shall use UnconfirmedDTCLimit = 5.

2678 v2 **CANH circuit short to battery voltage _ FDC10 max value**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table "Calibration of CAN DTCs" Item 5
Verification Method: Test
Legacy ID: 66746v1

The DTC test for CANH circuit short to battery voltage shall have FDC10 max value = 127.

2676 v2 **CANH circuit short to battery voltage _ DTCConfirmedLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56082v1

The CANH circuit short to battery voltage DTC shall use DTCConfirmedLimit = 3.

2675 v2 **CANH circuit short to battery voltage _ DTCAgedLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 5 slave
Verification Method: Test
Legacy ID: 56085v1

The CANH circuit short to battery voltage DTC shall use DTCAgedLimit = 255.

2685 v2 **CANH circuit short to CANL circuit**

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table: "Summary of CAN related external circuit faults" Item 9
Verification Method: Test

Legacy ID: 56190v1

If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANH circuit short to CANL circuit. Note that there shall be one unique DTC for each public network in the vehicle.

2693 v2 CANH circuit short to CANL circuit _ Test period

Version history:
(2):

Purpose: A standard time shall be used in detecting all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 9
Verification Method: Test
Legacy ID: 56191v1

The CANH circuit short to CANL circuit check shall be tested periodically at least every 100 ms.

2696 v3 CANH circuit short to CANL circuit _ TRC

Version history:
(3): Change the Verification Method to Test

Purpose: Default test run criteria shall be used for all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 9
Verification Method: Test
Legacy ID: 56193v1

The TRC to perform the CANH circuit short to CANL circuit check shall be according to the general Test Run Criteria specified in *[UDSonCAN_2] UDS Data*.

2690 v2 CANH circuit short to CANL circuit _ Failure type

Version history:
(2):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 9
Verification Method: Test
Legacy ID: 56267v1

For CANH circuit short to CANL circuit DTCs the failure type 0x00 shall be used.

2687 v2 CANH circuit short to CANL circuit _ DTC ID

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between

different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 9
Verification Method: Test
Legacy ID: 56268v1

The CANH circuit short to CANL circuit DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database*.

2695 v2 **CANH circuit short to CANL circuit _ Test sample passed criteria**

Version history:
(2):

Purpose: Define when to decrement FDC for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 7.9
Verification Method: Test
Legacy ID: 56275v1

The reported value of the DTC fault detection counter shall decrease when not CANH circuit short to CANL circuit is detected.

2692 v2 **CANH circuit short to CANL circuit _ Increase FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 56273v1

The CANH circuit short to CANL circuit FDC10 shall be increased by 3 at every test sample failed.

2694 v2 **CANH circuit short to CANL circuit _ Test sample failed criteria**

Version history:
(2):

Purpose: Define when to increment FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 56271v1

The reported value of the DTC fault detection counter shall increase when CANH circuit short to CANL circuit is detected.

2686 v2 **CANH circuit short to CANL circuit _ Decrease FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 56276v1

The CANH circuit short to CANL circuit FDC10 shall be decreased by 2 at every test sample passed.

2697 v2 CANH circuit short to CANL circuit _ UnconfirmedDTCLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible
Source: Table: "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 56277v1

The CANH circuit short to CANL circuit DTC test shall use UnconfirmedDTCLimit = 5

2691 v2 CANH circuit short to CANL circuit _ FDC10 max value

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 66781v1

The DTC test for CANH circuit short to CANL circuit shall have FDC10 max value = 127.

2689 v2 CANH circuit short to CANL circuit _ DTCConfirmedLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 56278v1

The CANH circuit short to CANL circuit DTC shall use DTCConfirmedLimit = 3.

2688 v2 CANH circuit short to CANL circuit _ DTCAgedLimit

Version history:

(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 9
Verification Method: Test
Legacy ID: 56279v1

The CANH circuit short to CANL circuit DTC shall use DTCAgedLimit = 255.

2698 v2 CANH circuit short to ground

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table: "Summary of CAN related external circuit faults" Item 7
Verification Method: Test
Legacy ID: 56105v1

If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANH circuit short to ground. Note that there shall be one unique DTC for each public network in the vehicle.

2706 v2 CANH circuit short to ground _ Test period

Version history:
(2):

Purpose: A standard time shall be used in detecting all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 7
Verification Method: Test
Legacy ID: 56107v1

The CANH circuit short to ground check shall be tested periodically at least every 100 ms.

2709 v2 CANH circuit short to ground _ TRC

Version history:
(2):

Purpose: Default test run criteria shall be used for all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 7
Verification Method: Test
Legacy ID: 56108v1

The TRC to perform the CANH circuit short to ground check shall be according to the general Test Run Criteria specified in *[UDSonCAN_2] UDS Data*.

2703 v2 **CANH circuit short to ground _ Failure type**

Version history:
(2):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 7
Verification Method: Test
Legacy ID: 56112v1

For CANH circuit short to ground DTCs the failure type 0x00 shall be used.

2700 v2 **CANH circuit short to ground _ DTC ID**

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 7
Verification Method: Test
Legacy ID: 56119v1

The CANH circuit short to ground DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database*.

2708 v2 **CANH circuit short to ground _ Test sample passed criteria**

Version history:
(2):

Purpose: Define when to decrement FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56144v1

The reported value of the DTC fault detection counter shall decrease when not CANH circuit short to ground is detected.

2705 v2 **CANH circuit short to ground _ Increase FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56143v1

The CANH circuit short to ground FDC10 shall be increased by 3 at every test sample failed.

2707 v2 **CANH circuit short to ground _ Test sample failed criteria**

Version history:
(2):

Purpose: Define when to increment FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56122v1

The reported value of the DTC fault detection counter shall increase when CANH circuit short to ground is detected.

2699 v2 **CANH circuit short to ground _ Decrease FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56146v1

The CANH circuit short to ground FDC10 shall be decreased by 2 at every test sample passed.

2710 v2 **CANH circuit short to ground _ UnconfirmedDTCLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56149v1

The CANH circuit short to ground DTC test shall use UnconfirmedDTCLimit = 5.

2704 v2 **CANH circuit short to ground _ FDC10 max value**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 66761v1

The DTC test for CANH circuit short to ground shall have FDC10 max value = 127.

2702 v2 **CANH circuit short to ground _ DTCConfirmedLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56152v1

The CANH circuit short to ground DTC shall use DTCConfirmedLimit = 3.

2701 v2 **CANH circuit short to ground _ DTCAgedLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 7
Verification Method: Test
Legacy ID: 56163v1

The CANH circuit short to ground DTC shall use DTCAgedLimit = 255.

2711 v2 **CANL circuit short to battery voltage**

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table: "Summary of CAN related external circuit faults" Item 8
Verification Method: Test
Legacy ID: 56165v1

If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANL circuit short to battery voltage. Note that there shall be one unique DTC for each public network in the vehicle.

2719 v2 **CANL circuit short to battery voltage _ Test period**

Version history:
(2):

Purpose: A standard time shall be used in detecting all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 8
Verification Method: Test
Legacy ID: 56167v1

The CANL circuit short to battery voltage check shall be tested periodically at least every 100 ms.

2716 v2 CANL circuit short to battery voltage _ Failure type

Version history:
(2):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 8
Verification Method: Test
Legacy ID: 56174v1

For CANL circuit short to battery voltage DTCc the failure type 0x00 shall be used.

2722 v2 CANL circuit short to battery voltage _ TRC

Version history:
(2):

Purpose: Default test run criteria shall be used for all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 8
Verification Method: Test
Legacy ID: 56170v1

The TRC to perform the CANL circuit short to battery voltage check shall be according to the general Test Run Criteria specified in *[UDSonCAN_2] UDS Data*.

2713 v2 CANL circuit short to battery voltage _ DTC ID

Version history:
(2):

Purpose: The OEMshall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 8
Verification Method: Test
Legacy ID: 56176v1

The CANL circuit short to battery voltage DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database*.

2720 v2 CANL circuit short to battery voltage _ Test sample failed criteria

Version history:

(2):

Purpose: Define when to increment FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56179v1

The reported value of the DTC fault detection counter shall increase when CANL circuit short to battery voltage is detected.

2718 v2 CANL circuit short to battery voltage _ Increase FDC10 steps

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56183v1

The CANL circuit short to battery voltage FDC10 shall be increased by 3 at every test sample failed.

2721 v2 CANL circuit short to battery voltage _ Test sample passed criteria

Version history:
(2):

Purpose: Define when to decrement FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56185v1

The reported value of the DTC fault detection counter shall decrease when not CANL circuit short to battery voltage is detected.

2712 v2 CANL circuit short to battery voltage _ Decrease FDC steps

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56186v1

The CANL circuit short to battery voltage FDC10 shall be decreased by 2 at every test sample passed.

2723 v2 **CANL circuit short to battery voltage _ UnconfirmedDTCLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56187v1

The CANL circuit short to battery voltage DTC test shall use UnconfirmedDTCLimit = 5.

2717 v2 **CANL circuit short to battery voltage _ FDC10 max value**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 66762v1

The DTC test for CANL circuit short to battery voltage shall have FDC10 max value = 127.

2715 v2 **CANL circuit short to battery voltage _ DTCConfirmedLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56188v1

The CANL circuit short to battery voltage DTC shall use DTCConfirmedLimit = 3.

2714 v2 **CANL circuit short to battery voltage _ DTCAgedLimit**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 8
Verification Method: Test
Legacy ID: 56189v1

The CANL circuit short to battery voltage DTC shall use DTCAgedLimit = 255.

2724 v2 CANL circuit short to ground

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table: "Summary of CAN related external circuit faults" Item 6
Verification Method: Test
Legacy ID: 56086v1

If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting CANL circuit short to ground. Note that there shall be one unique DTC for each public network in the vehicle.

2732 v2 CANL circuit short to ground _ Test period

Version history:
(2):

Purpose: A standard time shall be used in detecting all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 6
Verification Method: Test
Legacy ID: 56091v1

The CANL circuit short to ground check shall be tested periodically at least every 100 ms.

2729 v2 CANL circuit short to ground _ Failure type

Version history:
(2):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 6
Verification Method: Test
Legacy ID: 56094v1

For CANL circuit short to ground DTCs the failure type 0x00 shall be used.

2735 v2 CANL circuit short to ground _ TRC

Version history:
(2):

Purpose: Default test run criteria shall be used for all physical bus faults.
Source: Table: "Summary of CAN related external circuit faults" Item 6
Verification Method: Test

Legacy ID: 56092v1

The TRC to perform the CANL circuit short to ground check shall be according to the general Test Run Criteria specified in *[UDSonCAN_2] UDS Data*.

2726 v2 **CANL circuit short to ground _ DTC ID**

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 6
Verification Method: Test
Legacy ID: 56095v1

The CANL circuit short to ground DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database.*

2733 v2 **CANL circuit short to ground _ Test sample failed criteria**

Version history:
(2):

Purpose: Define when to increment FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 56096v1

The reported value of the DTC fault detection counter shall increase when CANL circuit short to ground is detected.

2731 v2 **CANL circuit short to ground _ Increase FDC10 steps**

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 56097v1

The CANL circuit short to ground FDC10 shall be increased by 3 at every test sample failed.

2734 v2 **CANL circuit short to ground _ Test sample passed criteria**

Version history:
(2):

Purpose: Define when to decrement FDC10 for each physical bus fault.
Source: Table: "Calibration of CAN DTCs" Item 6

Verification Method: Test
Legacy ID: 56098v1

The reported value of the DTC fault detection counter shall decrease when not CANL circuit short to ground is detected.

2725 v2 CANL circuit short to ground _ Decrease FDC10 steps

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 56099v1

The CANL circuit short to ground FDC10 shall be decreased by 2 at every test sample passed.

2736 v2 CANL circuit short to ground _ UnconfirmedDTCLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 56100v1

The CANL circuit short to ground DTC test shall use UnconfirmedDTCLimit = 5.

2730 v2 CANL circuit short to ground _ FDC10 max value

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 66752v1

The DTC test for CANL circuit short to ground shall have FDC10 max value = 127.

2728 v2 CANL circuit short to ground _ DTCConfirmedLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.

Source: Table: "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 56101v1

The CANL circuit short to ground DTC shall use DTCConfirmedLimit = 3.

2727 v2 CANL circuit short to ground _ DTCAgedLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: "Calibration of CAN DTCs" Item 6
Verification Method: Test
Legacy ID: 56104v1

The CANL circuit short to ground DTC shall use DTCAgedLimit = 255.

2737 v2 Common CAN electrical fault

Version history:
(2):

Purpose: Defining what the cause of the electrical fault may help in the fault tracing process. It is enough if the ECU closest to the tester checks electrical faults on the bus since if the fault is present, it will only be the ECU closest to the tester that will be able to report the status.
Source: Table: Summary of CAN related external circuit faults
Verification Method: Test
Legacy ID: 180733v1

If the ECU is connected to more then one public network and the ECU is the ECU transferring diagnostic messages in between the two networks, the ECU shall support detecting Common CAN electrical fault

2749 v2 Common CAN electrical fault _Test period

Version history:
(2):

Purpose: A standard time shall be used in detecting all physical bus faults.
Source: Table: Summary of CAN related external circuit faults, item10
Verification Method: Test
Legacy ID: 180753v1

The physical bus error status shall be read periodically at least every 100 ms.

2742 v3 Common CAN electrical fault _ Failure type

Version history:
(3):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: able: Summary of CAN related external circuit faults, item10
Verification Method: Test
Legacy ID: 180755v2

Common CAN electrical fault DTCs shall use the failure type 0x01.

2747 v2 Common CAN electrical fault _ TRC

Version history:
(2):

Purpose: Default test run criteria shall be used for all physical bus faults.
Source: Table: Summary of CAN related external circuit faults, item10
Verification Method: Test
Legacy ID: 180754v1

The TRC to perform Common CAN electrical fault check shall be according to the general Test Run Criteria specified in *[UDSonCAN_2] UDS Data*.

2739 v2 Common CAN electrical fault _ DTC ID

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: Summary of CAN related external circuit faults, item 10
Verification Method: Test
Legacy ID: 180756v1

The common CAN electrical fault DTC ID including the failure type byte value is defined in *[UDSonCAN_10] Global Master Reference Database*.

2745 v2 Common CAN electrical fault _ Test sample failed criteria

Version history:
(2):

Purpose: Define when to update the DTC fault detection counter.
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180757v1

The reported value of the DTC fault detection counter shall increase when the ECU detects that at least one of the physical bus error flags is set.

2744 v2 Common CAN electrical fault _ Increase FDC10 steps

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180758v1

The Common CAN electrical fault shall be increased by 3 at every test sample failed.

2746 v2 Common CAN electrical fault _ Test sample passed criteria

Version history:
(2):

Purpose: Define when to update the DTC fault detection counter.
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180759v1

The reported value of the DTC fault detection counter shall decrease when the ECU detects that none of the physical bus error flags is set.

2738 v2 Common CAN electrical fault _ Decrease FDC10 steps

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180760v1

The Common CAN electrical fault FDC10 shall be decreased by 2 at every test sample passed.

2748 v2 Common CAN electrical fault _ UnconfirmedDTCLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180761v1

The Common CAN electrical fault DTC test shall use UnconfirmedDTCLimit = 5.

2743 v2 Common CAN electrical fault _ FDC10 max value

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: able: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180762v1

The DTC test for Common CAN electrical fault shall have FDC10 max value = 127.

2741 v2 Common CAN electrical fault _ DTCConfirmedLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180763v1

The Common CAN electrical fault DTC shall use DTCConfirmedLimit = 3.

2740 v2 Common CAN electrical fault _ DTCAgedLimit

Version history:
(2):

Purpose: Generic DTC configuration shall be used for electrical faults to have as similar behaviour regarding DTC setting as possible.
Source: Table: Calibration of CAN DTCs, item10
Verification Method: Test
Legacy ID: 180764v1

The Common CAN electrical fault DTC shall use DTCAgedLimit = 255.

2766 v2 Permanent missing frame

Version history:
(2):

Purpose: Permanent missing frame DTCs shall be used since it may in some cases help pinpointing the location of a network problem as well as proving a rough functional test on operability of an ECU.
Source: Table: "Summary of CAN related external circuit faults" Item 3
Verification Method: Test
Legacy ID: 55841v1

It is mandatory for missing frame masters to detect missing frames and set permanent missing frame DTCs.

2768 v2 **Permanent missing frame _ Test period missing frame slave**

Version history:
(2):

Purpose: A standardized test period should be used for testing the DTC so the behavior is as similar as possible between the ECUs.
Source: Table: "Summary of CAN related external circuit faults" Item 3
Verification Method: Test
Legacy ID: 55861v1

Each missing frame slave shall periodically send at least one frame with a shorter periodicity than 100 ms. To save bandwidth on the CAN net, first check if there is an existing frame the gateway can perform the DTC test on. Secondly the send frequency of an existing frame shall be increased to match the 100 ms requirement.

2767 v2 **Permanent missing frame _ Test period missing frame master frame reception**

Version history:
(2):

Purpose: Missing frame master shall base its permanent missing frame DTC detection on a signal frame reception timeout.
Source: Table: "Summary of CAN related external circuit faults" Item 3
Verification Method: Analysis
Legacy ID: 55881v1

The missing frame master shall be configured to receive the periodically transmitted frame from each missing frame slave within the supervised network.

2776 v2 **Permanent missing frame _ Test period missing frame master**

Version history:
(2):

Purpose: Define how long time the missing frame master shall wait for each frame.
Source: Table: "Summary of CAN related external circuit faults" Item 3
Verification Method: Test
Legacy ID: 55883v1

The missing frame master shall check permanent missing frame DTCs every 150ms-170ms.

2764 v4 **Permanent missing frame _ TRC 1**

Version history:
(4):

Purpose: Only one ECU shall administer missing frame TRC to be sure the same TRC are used for all ECUs in the network.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4;
Verification Method: Analysis
Legacy ID: 55885v3

The missing frame master, and not the missing frame slaves, shall manage all the enabling and disable conditions (i.e. set TRC to true or false) for the missing frame DTCs, both for permanent and intermittent missing frame.

2765 v7 **Permanent missing frame _ TRC 2**

Version history:
(7):

Purpose: Missing frame DTCs shall be set in a predictable fashion (i.e. not allowed using event based frames for missing frames monitoring).
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Analysis
Legacy ID: 55888v6

The missing frame detection shall only be based on periodic frames sent in at least the usage mode "Driving".

2780 v3 **Permanent missing frame _ TRC 3**

Version history:
(3):

Purpose: If these parameters are sent from a missing frame slave within the network being supervised, the TRC would not be possible to obtain by the missing frame master during an error situation.
Source: Table: "Summary of CAN related external circuit faults" Item 7.3 and 7.4
Verification Method: Analysis
Legacy ID: 55891v2

All parameters used as test run criteria for missing frame tests shall be received outside the network that is being tested.

2781 v3 **Permanent missing frame _ TRC 4**

Version history:
(3):

Purpose: It must be possible to have different network configurations in regards of mounted ECUs on a network, without the missing frame master reporting missing slaves.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Test
Legacy ID: 55921v2

The testing for missing frames shall only be done when the frame is expected; i.e. missing frame master shall be configured such that it is aware if an optional missing frame slave is present (mounted or not) on the bus or not.

2782 v4 **Permanent missing frame _ TRC 5**

Version history:
(4):

Purpose: Prevent setting missing frame DTC due to the mode the ECU currently has, prevents the ECU from sending the frame.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4;
Verification Method: Test
Legacy ID: 55922v3

The missing frame master shall be aware of different operation modes preventing the missing frame slaves to send frames. For example if a missing frame slave is in power-off, low-power, sleep or equivalent mode, the missing frame slave might be prevented from sending frames. The missing frame master shall be aware when the missing frame slave is prevented from sending frames and shall not set any missing frame DTCs during this these modes.

2783 v5 Permanent missing frame _ TRC 6

Version history:
(5):

Purpose: Missing frames shall only be tested during usage modes when all ECUs are expected to be active.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Test
Legacy ID: 55925v4

Missing frame tests shall only be done when the ECU is the usage mode "Driving"

2784 v4 Permanent missing frame _ TRC 7

Version history:
(4):

Purpose: Missing frames shall only be tested during usage modes when all ECUs are expected to be active and non-transient operation mode.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Test
Legacy ID: 55927v3

Missing frame tests shall never be done during combustion engine crank.

2785 v5 Permanent missing frame _ TRC 8

Version history:
(5):

Purpose: To ensure missing frame DTCs are not falsely set due to differences in ECU power-up times, missing frames shall only be tested during non-transient conditions.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Test
Legacy ID: 55930v4

The missing frame master shall never set TRC true until 5±2 seconds after all criteria for a DTC test, for permanent and intermittent missing frame DTCs, to run, is satisfied. This inhibit time period include the inhibit time period, that starts after transition from one usage to another or one car mode to another, specified as Generic Test Run Criteria in *[UDSonCAN_2] UDS Data* (i.e. no additional time period is needed).

2786 v3 **Permanent missing frame _ TRC 9**

Version history:
(3):

Purpose: Missing frames shall only be tested within the battery voltage when communication networks shall be active.
Source: Table: "Summary of CAN related external circuit faults" Item 7.3 and 7.4
Verification Method: Test
Legacy ID: 55932v2

The voltage of the vehicle battery Ubat, shall be within: 10,0 ≤ Ubat ≤ 15,4V.
Note: The battery voltage shall be measured as close to the battery as possible in order to minimize the measuring fault.

2778 v3 **Permanent missing frame _ TRC 10**

Version history:
(3):

Purpose: Prevent setting fault DTCs, due to vehicle not being in a normal state.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Test
Legacy ID: 55934v2

If a missing frame master does not receive the missing frame TRC no missing frame DTCs shall be set.

2779 v2 **Permanent missing frame _ TRC 11**

Version history:
(2):

Purpose: Prevent setting fault DTCs, due to vehicle not being in a normal state.
Source: Table: "Summary of CAN related external circuit faults" Item 3 and 4
Verification Method: Test
Legacy ID: 398851v1

Missing frame tests shall never be done when fault is detected on the parameters used as test run criteria (e.g., when the value of parameters, used as test run criteria, distributed on an in-vehicle network is not updated (indicated by an update bit) or not correct (indicated by a quality factor)).

2773 v2 **Permanent missing frame _ Failure type**

Version history:
(2):

Purpose: The OEM shall define which failure type that shall be used to minimize the differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 3
Verification Method: Test
Legacy ID: 55935v1

For permanent missing frame DTCs the failure type 0x00 shall be used.

2770 v2 Permanent missing frame _ DTC ID

Version history:
(2):

Purpose: The OEM shall define which DTC ID to use to minimize differences in between different ECUs, so that administration is kept to a minimum.
Source: Table: "Summary of CAN related external circuit faults" Item 3
Verification Method: Test
Legacy ID: 55936v1

The permanent missing frame DTC ID shall be as defined in *[UDSonCAN_10] Global Master Reference Database*.

2777 v2 Permanent missing frame _ Test sample failed criteria

Version history:
(2):

Purpose: The missing frame master shall be able to detect when the missing frame slaves fails to send their respective periodic frame.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55937v1

The reported value of the DTC fault detection counter, for a specific ECU, shall increase when the missing frame master have not received the periodic missing frame slave frame, for that specific ECU, in the test period time.

2775 v2 Permanent missing frame _ Increase FDC10 steps

Version history:
(2):

Purpose: Permanent missing frame DTCs shall be set with filtering to be sure the validity of the DTC. However the DTC must be set within 3 seconds to not have a gap in between permanent and intermittent missing frame detection.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55938v1

The missing frame permanent FDC shall be increased by 9 at every test sample failed.

2788 v2 **Permanent missing frame _ Test sample passed criteria**

Version history:
(2):

Purpose: The missing frame master shall be able to detect when the missing frame slaves succeeds in sending their respective periodic frame.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55939v1

The reported value of the DTC fault detection counter, for a specific ECU, shall decrease when the missing frame master have received the periodic missing frame slave frame, for that specific ECU, in the test period time.

2769 v2 **Permanent missing frame _ Decrease FDC10 steps**

Version history:
(2):

Purpose: Permanent missing frame DTCs shall be set with filtering to be sure the validity of the DTC.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55940v1

The permanent missing frame FDC10 shall be decreased by 9 at every test sample passed.

2787 v2 **Permanent missing frame _ UnconfirmedDTCLimit**

Version history:
(2):

Purpose: Due to the seriousness of the error when an ECU is unresponsive, it is good to see shorter occurrences of when ECUs fails to send frames.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55942v1

The permanent missing frame DTC test shall use UnconfirmedDTCLimit = 18.

2774 v2 **Permanent missing frame _ FDC10 max value**

Version history:
(2):

Purpose: If a permanent missing frame occurs it is assumed it has big impact on the functionality of the system and hence it shall be able to set the confirm bit.
Source: Table 3 item 3

Verification Method: Test
Legacy ID: 152142v1

The DTC test for permanent missing frame shall have FDC10 max value = 127

2772 v3 Permanent missing frame _ DTCConfirmedLimit

Version history:
(3):

Purpose: Permanent missing frame DTCs shall be set with filtering to be sure the validity of the DTC.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55944v2

The permanent missing frame DTC shall use DTCConfirmedLimit = 3.

2771 v2 Permanent missing frame _ DTCAgedLimit

Version history:
(2):

Purpose: Due to the seriousness of the DTC.
Source: Table: "Calibration of CAN DTCs" Item 3
Verification Method: Test
Legacy ID: 55945v1

The permanent missing frame DTC shall use DTCAgedLimit = 255.

4.5.4.2.2 Diagnostic Services

The required diagnostic services are specified in *[UDSonCAN_1] UDS Services*.

2658 v3 Diagnostic services

Version history:
(3): Change the Verification Method from Test to Analysis

Purpose: Define which services each CAN ECU shall or may support.
Source: Section "Diagnostic Services"
Verification Method: Analysis
Legacy ID: 55596v1

The CAN application shall support the diagnostic services as defined in *[UDSonCAN_1] UDS Services*.

4.6 Software Download

4.6.1 General Software Download

4.6.1.1 Generic SWDL

4.6.1.1.1 Requisite documents

The following documents are either referenced by this specification, or contain information that is relevant to this specification.

Reference	Source	Document
[SWDL_1]	Internal	VBF (Versatile Binary Format)
[SWDL_2]	Internal	VSF (Volvo Cars Software Format)
[SWDL_3]	Internal	Data Compression and Encryption
[SWDL_4]	Internal	Delta encoding
[SWDL_5]	Internal	UDS Services
[SWDL_6]	Internal	UDS Data
[SWDL_7]	Internal	UDS Session
[SWDL_8]	Internal	Diagnostic and Bootloader Gateway
[SWDL_9]	Internal	Software Authentication
[SWDL_10]	Internal	General Software Authentication
[SWDL_11]	ISO	ISO14229-1: Road vehicles — Unified diagnostic services (UDS) — Part 1: Specification and requirements. Revision as stated in *[SWDL_6] UDS Data.*

Note:
Internal references is typically found as a separate chapter in the ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

4.6.1.1.1.1 Referenced document, structure and relations

The Software Download concept includes a list of specifications that has a tight relation to each other, the Figure " *SWDL specification structure*" , shows the overview and their relation to the ISO documents which is the base.

All listed documents are not applicable for each and every ECU. Which specifications that are applicable depends of type of data link, whether the ECU is a gateway or not etc. It is each SW-responsible to request the applicable requirements to each ECU supplier. Note that the specific data link, network and transport protocol specifications not are referenced from this document.

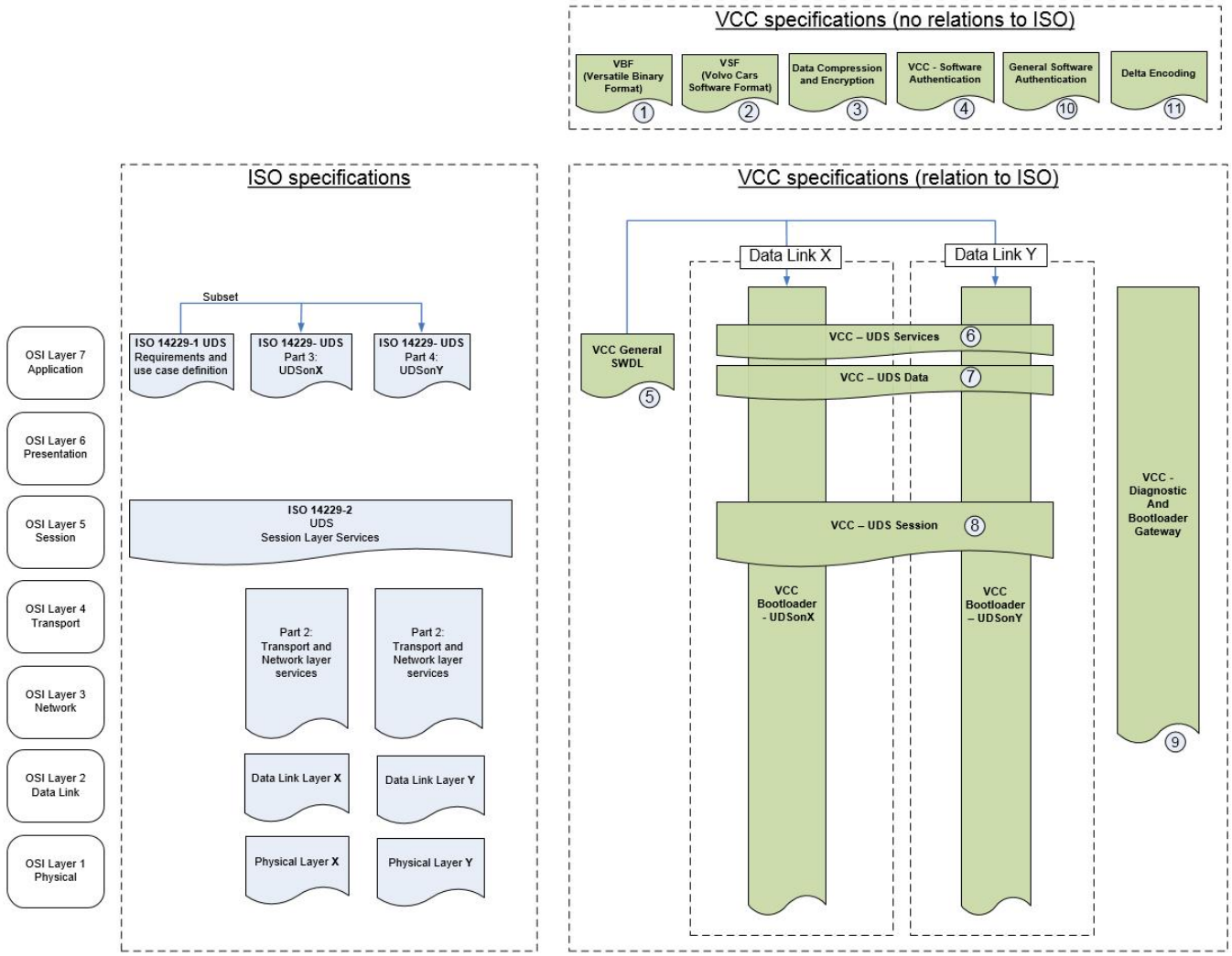


Figure - SWDL specification structure

1 - The "*VBF (Versatile Binary Format*)" data file format is supporting ECUs with addressed based memory data storage.

2 - The "*VSF (Volvo Cars Software Format)"* data file format is supporting ECUs with both file system and/or memory addressed based data storage.

3 - The "*Data Compression and Encryption*" specifies how data stored in the data file shall be compressed and how the data shall be decompressed by the ECU. The specification includes a reference implementation of the two algorithms.

4 - The "*Software Authentication*" is defining a method to protect that the data file origins from a legitimated source.
5 - The "*General Software Download*", is the top document for the software download concept. It describes the background and the non-network dependent requirements for a bootloader as well as the programming sequences used for software download.

6 - The "*UDS Services*" specifies which diagnostic services that is applicable for the bootloader (programmingSession).

7 - The "*UDS Data*" specifies the data used by each diagnostic service that is applicable for the bootloader (programmingSession).

8 - The "*UDS Session*" specifies the requirements for the session layer.

9 - The "*Diagnostic and Bootloader Gateway"* specifies the gateway requirements for those ECUs that act as a diagnostic gateway.
10 - The "*General Software Authentication*" specifies the Software Authentication concepts, i.e. methods used to ensure that the software is authentic.
11 - The "*Delta Encoding"* specifies the requirements to enable delta encoding as compression method for data files.

4.6.1.1.2 Definitions and Abbreviations

Definition	Description
Broken software	The software present in the ECU is only partly downloaded.
Cold Start	The start-up of the ECU from a powered-down or off state.
Compatible check	Check that all data files fit together and the ECU is capable to starts its normal application functionality.
Complete check	Check that every individual data file is present, this can be done that each file has a signature placed in the beginning and end of each file that are checked.
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an on-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.
Final response	Positive or a negative response other than response code 0x78 "requestCorrectlyReceived-ResponsePending"
Program mode	The ECU executing the primary or secondary bootloader
Public ECU	An ECU addressable from a client/tester point of view.
Private ECU	An ECU not addressable from a client/tester point of view.
Warm start	The start-up of the ECU initiated by the application.

Abbr.	Description
CRC	Cyclic Redundancy Check
ECU	Electronic Control Unit
HW	HardWare
NRC	Negative Response Code
PBL	Primary Bootloader
SBL	Secondary Bootloader
SPRMIB	SuppressPositiveResponseMessageIndicationBit
SW	SoftWare
SWDL	Software Download
UDS	Unified Diagnostic Services

4.6.1.1.3 Overview

Software Download involves the transfer of information from a tester to an ECU using data stored in a file. The data transferred may contain configuration or calibration data for the ECU or completely new software. It also includes uploading of data from the ECU to the tester for debugging.
The Software Download concept provides a means for a tester to send multiple bytes of data to an ECU.

4.6.1.1.4 Introduction

Electronic Control Units (ECUs) have demonstrated continued increase in feature, count, memory size, I/O circuit count, and part number complexity. All areas of the vehicle electrical/electronic system are affected. Electronic Control Units are experiencing late and frequent changes many of which affect software. These trends will undoubtedly continue into the foreseeable future.

There is a need to reduce vehicle build and service complexity, reduce ECU supplier complexity, and allow for maximum flexibility in software design by establishing designs and procedures providing the ability to configure and program ECUs in assembly and in service. Configuration and module programming will allow module design and release activities to minimize part complexity, economies of scale and allow bookshelf software to be utilized.

By having software download possibility in all ECUs with complex functionality the cost of correcting software errors is significantly reduced and new functions can be added after production. The software download functionality is used both in the production line to customize the hardware for the specific car and in workshops to correct errors or improve functions. The software download concept is based on a primary bootloader that is fixed in the ECU and always executed at power on before the application is started and a secondary bootloader, including non-volatile memory programming routines that is downloaded before downloading the rest of the applications and configuration files.

Important is to reduce the download time as much as possible to enable a convenient user experience and reduced assembly and workshop time/cost spent on the programming phase. Important technologies such as compression and delta encoding of data reduce the software size as well as using available bandwidth by parallel download of multiple ECUs is therefore motivated.

4.6.1.1.5 What is the purpose of this document

This specification is the top document of the SoftWare DownLoad (SWDL) concept. The specification describes the background and the non-data link dependent requirements for a bootloader as well as the programming sequences used for software download. For details on specific protocols, data links, components and algorithms, please read the respectively documents.
This specification is intended to provide guidance and requirements to architects, system designers, application developers, and ECU suppliers. Adherence to the specifications contained within this document is required to ensure compatibility within the corporate product line from vehicle to vehicle, model year to model year and maintain compatibility with the industry driven standards.

Important to understand is that this specification only specifies the general non network specific

requirements for a bootloader.

4.6.1.2 General Bootloader requirements

4.6.1.2.1 Software Authentication

4.6.1.2.1.1 Software Authentication - Introduction

The purpose of Software Authentication is described in *[General Software Authentication]*, i.e. to verify that the installed software is authentic (originates from a legitimate source and has not been altered). To perform the authenticity verification, using digital signatures, the ECU must (at least) know:

• The data that must be checked for authenticity and where it is located in the memory. Verification Block Tables are used *[General Software Authentication]*. The verification block table (one per software part) contains the placement and hash value for the expected programmed data blocks.

• How to get the corresponding digital signature.

• Conditions for starting the verification and what is required for the verification to pass or fail respectively. If a complete verification is not performed at every application start, information about the verification status (validity status information) must be stored.

• How many software parts (and their location) that must be successfully verified in order to start the application: ECU Software Structure and Logical Blocks are introduced.

The ECU authenticity verification (for a single software part, i.e. a logical block) is performed according to *Figure - Authenticity verification principle*:

(1) - The data blocks of the software part is programmed to the ECU. The corresponding digital signature is transferred using a Check Memory control routine, which will also initiate the verification.

(2) - The signed verification block table is verified. The placement of the verification block is ensured as its start address and length parameters are part of the signature generation, i.e. the length (unprocessed) must be calculated by the ECU (the start address is known for the updated logical block).

(3) - The programmed data blocks, according to the authenticated verification block, are then verified. The placement of all ingoing data blocks are ensured as their start address and lengths together with the corresponding hash values are part of the verification block table.

(4) - The remaining memory area within the logical block shall not have been programmed (the verification block table only contains the programmed data blocks). The ECU verifies that these parts are "blank" (not programmed), e.g. the value of erased memory (hardware dependent) or filled with some known value (e.g. for the SBL). It must be ensured that erased state is a valid state for the used HW.

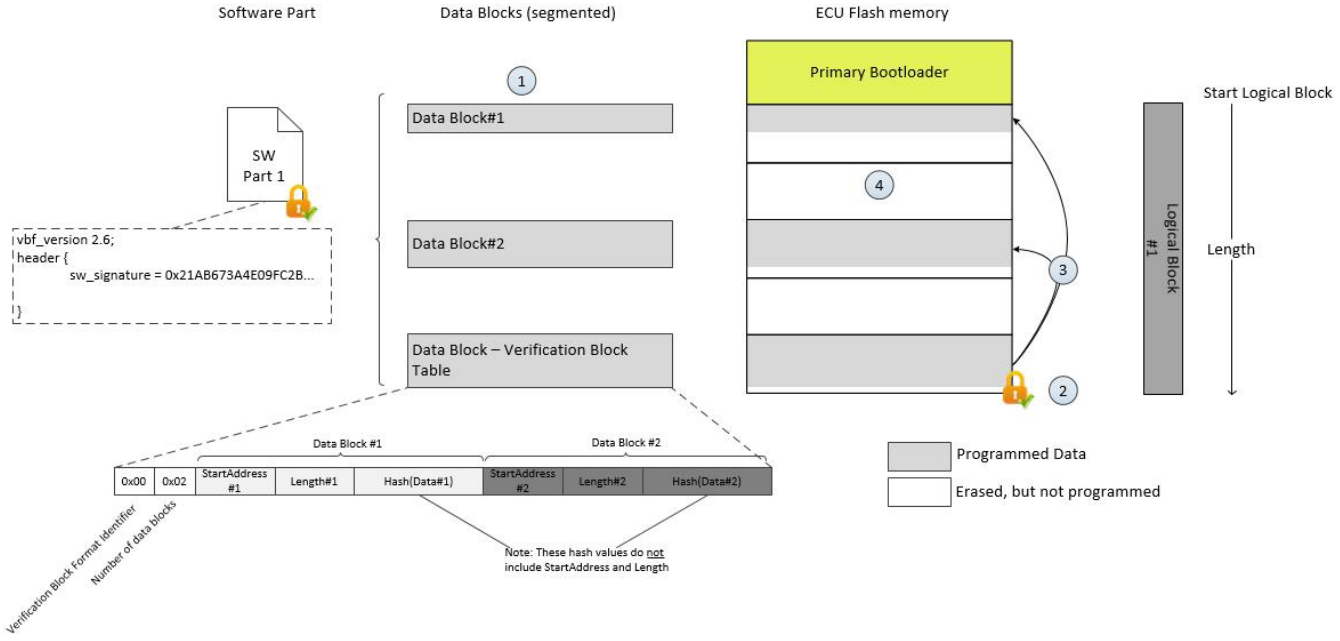


Figure - Authenticity verification principle

The *Figure - Authenticity verification principle* does not show all details of the software preparation (off-board) nor the signing. The purpose is to introduce the reader of how the verification is performed.

At software **download**, the ECU must:

- Check that the erase request matches a logical block range (to ensure that previously installed data is erased)
- Mark the logical block as invalid (i.e. it has not passed the signature verification), prior to any data erase/write operations.
- Check at every Request Download that the logical block is marked as invalid.
- Check at Request Download and at Transfer Data that only one logical block has been addressed. Note that the "compressed" (processed) memory size is used as input parameter to the Request Download, i.e. actual programmed length might differ (typically longer).
- Check at Request Download and at Transfer Data that no memory area is addressed that is exclusive to the bootloader. It must also be ensured that no memory is addressed that is not define, e.g. outside the range of where the application data is allowed to be programmed.
- At Check Memory, verify according to (1) to (4) in *Figure - Authenticity verification principle*.
- Only upon a successful authenticity verification, set the validity status for the verified logical block to valid.

At every ECU **start**, the bootloader must:

- Verify that all logical block are present and valid (i.e. signature verification is passed). The number of logical blocks and corresponding locations are defined by the ECU Software Structure, i.e. this must be authenticated first. All software must have passed the authenticity verification in order to be executed in the ECU.
- If the verification is failed, the complete part of the complete and compatible function must fail and the application is not started.

4.6.1.2.1.2 Software Structure

The ECU is to be organized into different "logical blocks", the ECU software structure. Each logical block is to be verified (and signed) individually and because of that it must only be one software part (e.g. a VBF) allocated per logical block. A software part is typically segmented into one or many data blocks. That means that all data blocks must reside in the dedicated logical block in order to be able to verify each logical block individually.

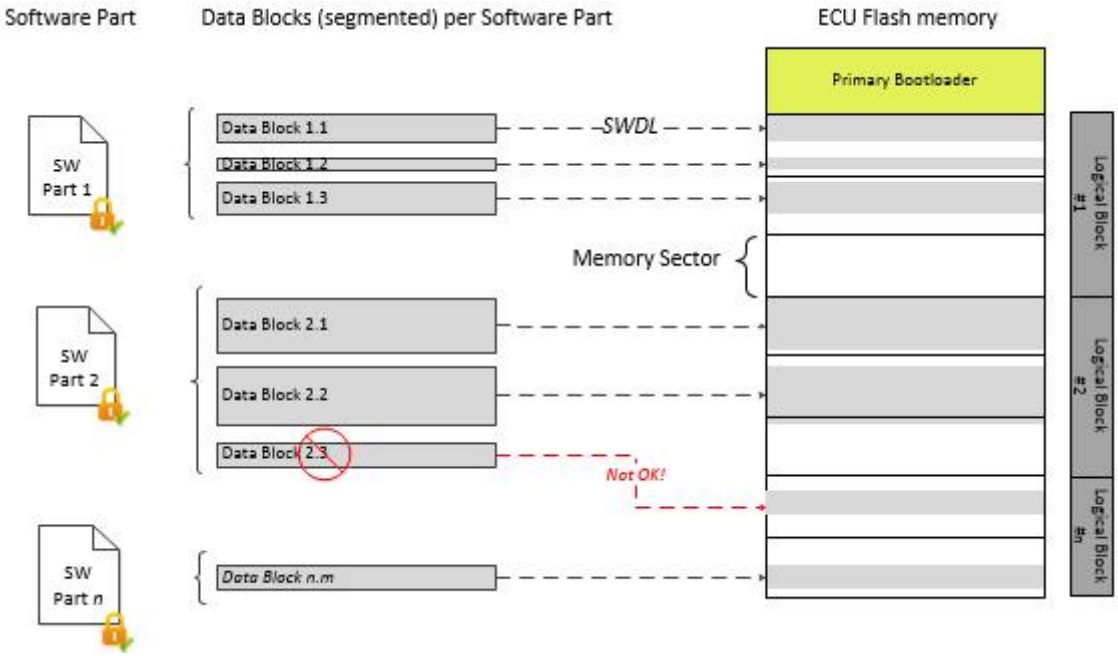


Figure - Logical Block Definition

In *Figure - Logical Block Definition,* a software part *n* is segmented into *m* data blocks and is dedicated to the logical block *n*.

3146 v3 **Programming of Logical Blocks**

Version history:
(3):

Purpose: Define properties for a logical block, that are required for the ECU to be able to manage each logical block independently but also to simplify the signing process at the trust centre.

Source:

Verification Method: Test + Analysis

Legacy ID: 400425v2

Each logical block shall be possible to sign separately, i.e. each logical block must be possible to erase, program and verify independently in the ECU.

3147 v3 **Software parts per logical block**

Version history:
(3):

Purpose: Define the software structure to enable that each software part is possible to

individually sign and verify.

Source:

Verification Method: Test

Legacy ID: 397292v2

The content (data blocks) of a software part must be within a single logical block range, i.e. two different logical blocks must not be addressed by a single software part.

3145 v2 Number of logical blocks

Version history:
(2):

Purpose: Define the minimum number of logical block. The actual number of logical blocks are defined by the ECU Software Structure.

Source:

Verification Method: Test + Analysis

Legacy ID: 397293v1

The ECU shall contain at least one logical block. The maximum number of blocks shall be statically configured in the bootloader.

3143 v4 Logical Block - Properties

Version history:
(4):

Purpose: Define the constraints of a logical block

Source:

Verification Method: Test

Legacy ID: 397291v3

The non-volatile memory for software download shall be divided into logical blocks that must start and end at the boundary of a flash sector. The logical blocks must not share a common flash sector, i.e. logical blocks must not overlap or contain another logical block. Logical blocks must also not cross different memory devices. It shall be contiguous.
Clarification for ECUs having file-based data storage: erasure or modification of a single logical block should not affect contents of other logical blocks (unless explicitly specified in other requirements (e.g. for ESS update)).

3144 v5 Logical Block Declaration

Version history:
(5):

Purpose: To define the properties for each logical block

Source:

Verification Method: Test + Analysis

Legacy ID: 397432v4

For each individual logical block, following properties must be defined:

1. its start address and length, i.e. the location (mandatory)
2. where the verification block table is located (mandatory). The verification block contains information (location and hash value) of the data blocks that shall have been programmed to the logical block. It is typically the first downloaded data block of a Software Part.
3. how to get the corresponding signature (mandatory) and where is shall be stored (optional). Storing the signature in a non-volatile memory is only applicable for use cases where it is required to perform a signature verification at a later time, i.e. the verification is not only performed at the programming sequence.
4. where the validity status information is located (mandatory)
5. where the erase status information is located (mandatory), used to identify if a logical block is erased or not, to e.g. not erase memory when it is already erased (time consuming).
6. where the compatibility information is located (optional). Used to decide the compatibility to other downloaded parts. This must be known to the bootloader *if* the check is done by the bootloader itself. See section for Complete&Compatible function for further details.

Example.
A logical block will contain 1:m data blocks and 1 verification block table.

4.6.1.2.1.2.1 ECU Software Structure (ESS)

The **E**CU **S**oftware **S**tructure (**ESS**) carries the information of all logical block ranges and properties. It shall be stored in a dedicated logical block which position is known to the primary bootloader, e.g. a flash sector exclusively reserved to the ESS. It is however re-programmable as any other software part, according to the Programming sequence.

Typical use cases for updating the ESS are when a software part growths beyond its previously reserved logical block or when a software part is divided into two parts.

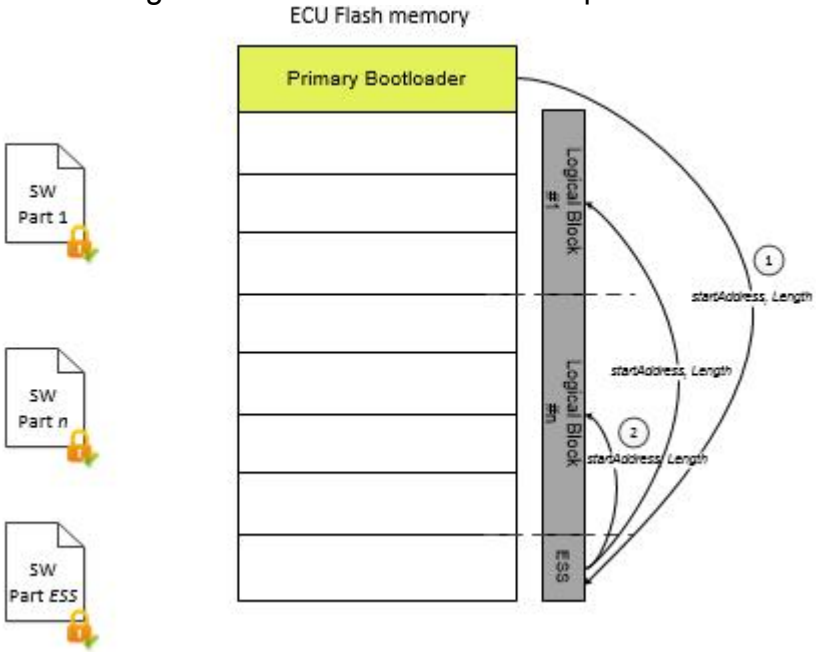


Figure - Re-programmable ECU Software Structure

(1) The location of the ESS is known by the primary bootloader.

(2) The bootloader uses the definitions in the ESS to identify the properties of the other logical blocks. The ESS must first be verified (valid) before its definitions are applied.

When there is no valid ESS stored in the dedicated logical block, the ECU must only allow software download of the SBL and to ESS as the information in the ESS is required in order to validate other logical blocks.

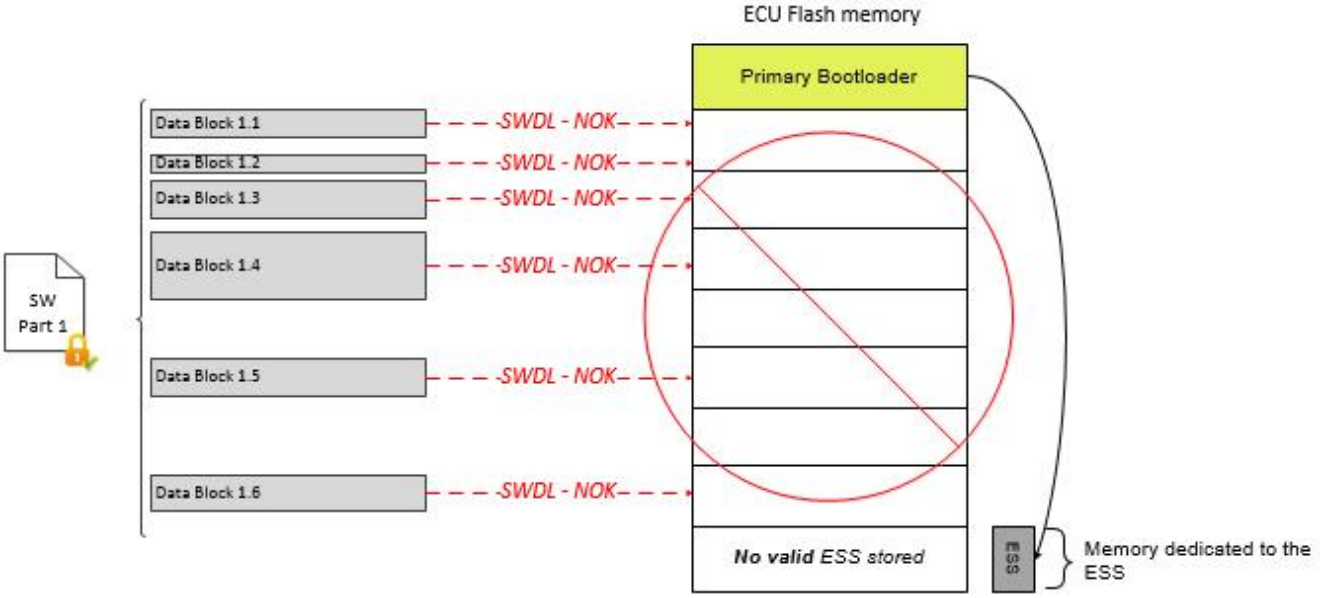


Figure - Restricted SWDL when no valid ESS stored

To ensure a known memory state when the ESS is updated, the bootloader shall erase the complete flash upon every ESS update.

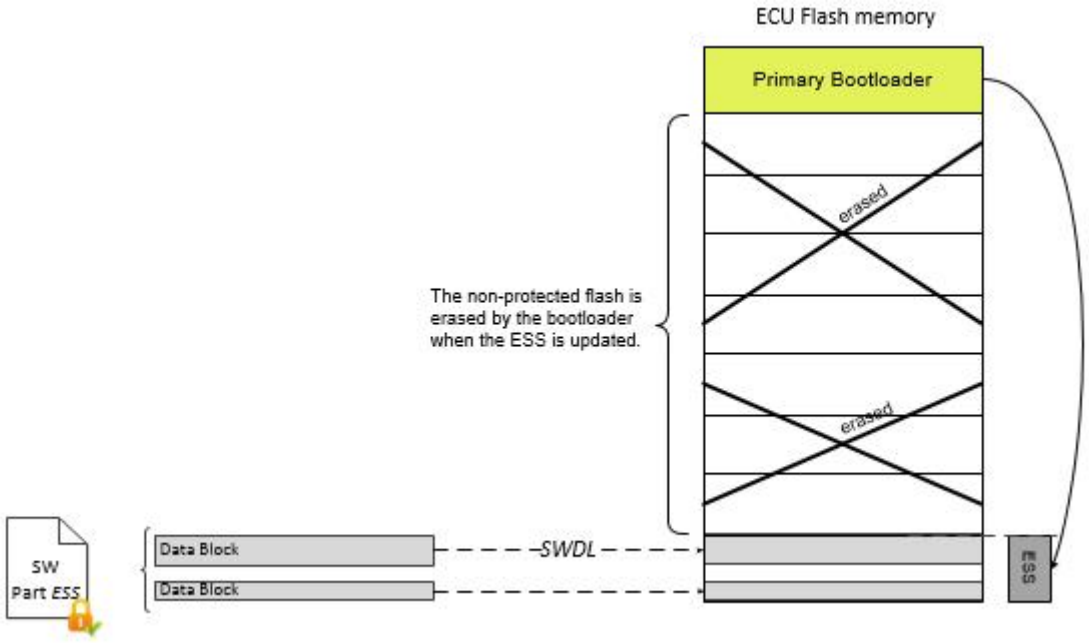


Figure - Non-protected flash to be erased when the ESS is updated

Version history:
(2):

Purpose: To reduce the complexity of the bootloader implementation with respect to ESS update use case and for the bootloader to know where to find the logical block definitions
Source:
Verification Method: Test
Legacy ID: 397435v1

The ECU Software Structure (ESS) shall be stored at a reserved logical block (flash sector), i.e. dedicated to the ESS only.

3151 v3 Reprogrammable ECU Software Structure (ESS)

Version history:
(3):

Purpose: To enable updates of the ESS without requiring a hardware or bootloader update. Typical use cases are when software parts growths in size, the software structure is changed (e.g. a software part is divided into two parts or vice versa).
Source:
Verification Method: Test
Legacy ID: 397436v2

The ESS shall be reprogrammable according to the Programming Sequence described in this document, i.e. as any other software part.

3150 v2 Immediately apply ESS update

Version history:
(2):

Purpose: The ESS is downloaded as any other software parts. For example, an ECU reset cannot be added to the programming sequence every time an ESS is updated.
Source:
Verification Method: Test
Legacy ID: 411891v1

The ESS updates must be applied immediately after being verified correctly, i.e. an initiation of the bootloader must not be required. This means that another software part (e.g. EXE) shall be possible to program next to the ESS without requiring an ECU reset.

3152 v4 Restricted software download when no valid ESS stored

Version history:
(4):

Purpose: To enable ECU Software Structure (ESS) update when there is no valid ESS stored.

Source:
Verification Method: Test
Legacy ID: 397438v3

If there is no valid ESS stored, the software download must be restricted to the SBL and the logical block dedicated to the ESS only. That means that attempts to address other logical blocks shall be rejected. The ECU shall inform the tester of the aborted erase memory attempt or aborted request download.
Note: *The SBL is verified by the primary bootloader.*

3154 v4 Validation of the ECU Software Structure

Version history:
(4):

Purpose: The verification of the programmed data could be bypassed if the logical block carrying the ESS is not valid.
Source:
Verification Method: Test
Legacy ID: 404771v3

The ESS must be validated before the bootloader applies the ESS definitions. The minimum requirements are that it shall be verified as any other logical blocks and ensure that the complete memory is covered by the defined logical blocks.

Note: *The ECU is recommended to add additional checks to e.g. ensure that the format or length is correct, even though these parts are to be verified prior to the software part is signed.*

3149 v3 Erase memory when ECU Software Structure (ESS) is updated

Version history:
(3):

Purpose: To ensure a known memory state when the ESS is updated.
Source:
Verification Method: Test + Analysis
Legacy ID: 397440v2

The complete programmable flash memory shall be erased when the logical block dedicated to the ESS is (internally) addressed for erasure, i.e. being updated and its validity is changed from valid to invalid.

Example.
The bootloader detects that the ESS is addressed for erasure. The bootloader will ensure that all flash memory is erased according to a defined physical flash layout. Note that a logical block shall be set to 'invalid' prior to it is erased. The erase might be initiated by either EraseMemory or RequestDownload.

4.6.1.2.1.2.2 ECU Software Structure - Declaration

The format of the ECU Software Structure is design specific and will not be specified in details.

Some parts might be explicitly defined in the structure itself or derived from another known location, e.g. the logical block range.

Example.

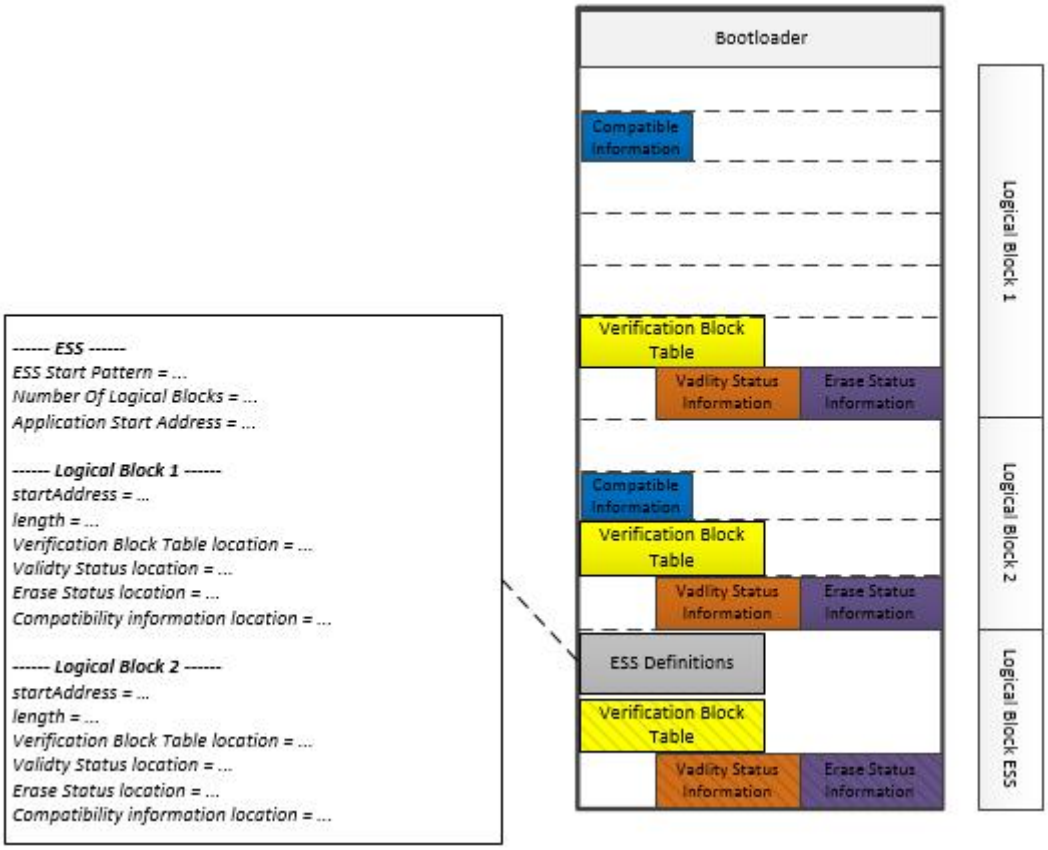


Figure - Example of ESS Declaration

To have a simplified view, the Figure - Example of ESS Declaration shows when Validity Status information and Erase Status information are stored within each logical block range. These could also be stored elsewhere, e.g. in a dedicated memory exclusive to the bootloader. See separate section for Validity Status as it has dependencies to the ECU verification being executed at start (boot). Previous figure shows the case where the Validity Status is used to indicate the latest software download status. The recommendation is to verify all software also at boot.

3148 v3 **ECU Software Structure Declaration**

Version history:
(3):

Purpose: Define the properties of the ECU Software Structure. The supplier is responsible to specify the format of the ESS in detail. This degree of freedom is left to the supplier in order to not interfere with supplier specific tool chain and implementations.

Source:

Verification Method: Test + Analysis

Legacy ID: 399873v2

The *ECU Software Structure* shall at least contain:

- A pattern known to the bootloader, defining the existence of the structure, i.e. always starting with a known pattern.
- The number of logical blocks defined in the ESS.
- The definitions of all logical blocks, see *REQPROD Logical Block Declaration*.

4.6.1.2.1.2.3 Root ECU Software Structure

The *ECU Software Structure* is stored in a dedicated logical block. But, the bootloader must know the properties of that logical block. Hence, the term ***Root** ECU Software Structure* is introduced. This structure is stored in a protected area known to the bootloader (e.g. in the bootloader area itself).

3155 v3 **Root ECU Software Structure Declaration**

Version history:
(3):

Purpose: Define ESS properties that must be stored in a protected area known to, or part of, the bootloader. This properties are required when there is no ESS stored and for the bootloader to be able to verify that the stored ESS is valid (i.e. has passed the authenticity verification).

Source:

Verification Method: Test + Analysis

Legacy ID: 399878v2

The ECU shall in a protected (not possible to alter nor read) area statically store the definitions required to verify the logical block dedicated to the ECU Software Structure, i.e:
(i) start address and length (mandatory)
(ii) where the verification block table is located (mandatory)
(iii) where the validity status information is located (mandatory)
(iv) where the erase status information is located (mandatory)
(v) where the compatibility information is located (optional)
(vi) where the signature of the verification block table is to be stored (optional).

4.6.1.2.1.3 Erase Status Information

Per logical block, the bootloader shall able to identify if the logical block is erased or not. This is typically used by the bootloader to not erase memory (time consuming) when it's already erased, but also to ensure that the memory has been erased as expected for use cases where this is required.

3128 v4 **Erase Status Information**

Version history:
(4):

Purpose: To reduce software download time and to ensure that the memory has been erased as expected.

Source:

Verification Method: Test

Legacy ID: 401108v3

The ECU shall reserve some dedicated memory to the Erase Status Information. This information must be exclusively handled by the bootloader. If the logical block is "already erased" and an valid erase request is detected, the ECU shall not perform any erase operations. Once one or more bytes of the programmable area has been programmed, the addressed logical block must be considered as "not erased" until it is ensured that the complete block has been erased again. The ECU supplier must ensure that a "virgin" ECU has this information set to erased and that a completely programmed ECU has the erase information set to "not erased".

4.6.1.2.1.4 Verification Block Table

A *Verification Block Table* is introduced, representing the expected segmented data blocks (including address, length and hash values per data block) of a complete software part. By that, the signature could be created on the verification block table. When that block is verified correctly at the ECU, the ECU could continue to verify also the other data blocks that has been programmed. Hence, the security principle of "chain-of-trust" is applied. Note that the verification block table is downloaded as any other data block, i.e. the verification block is a separate data block with defined properties.

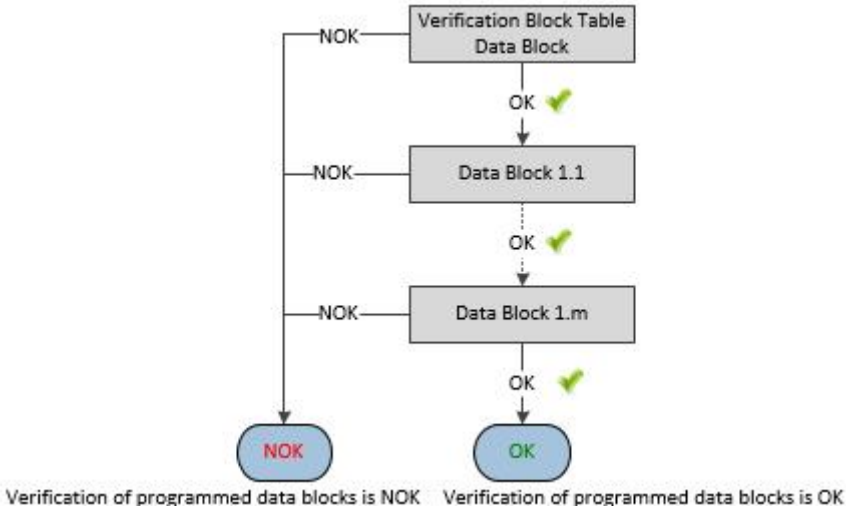


Figure - Verification of programmed data blocks

The verification block table must be put in a separate data block (i.e. downloaded as a separate data block to the ECU) and it shall contain:

- Verification Block Table format identifier, e.g. a version number if the format of the table is changed in the future
- Number of data blocks (the verification block table itself is excluded)
- StartAddress data block 1, length data block 1, hash value of data block 1
- ...
- StartAddress data block m, length data block m, hash value of data block m

Note that there is one verification block table per every software part.

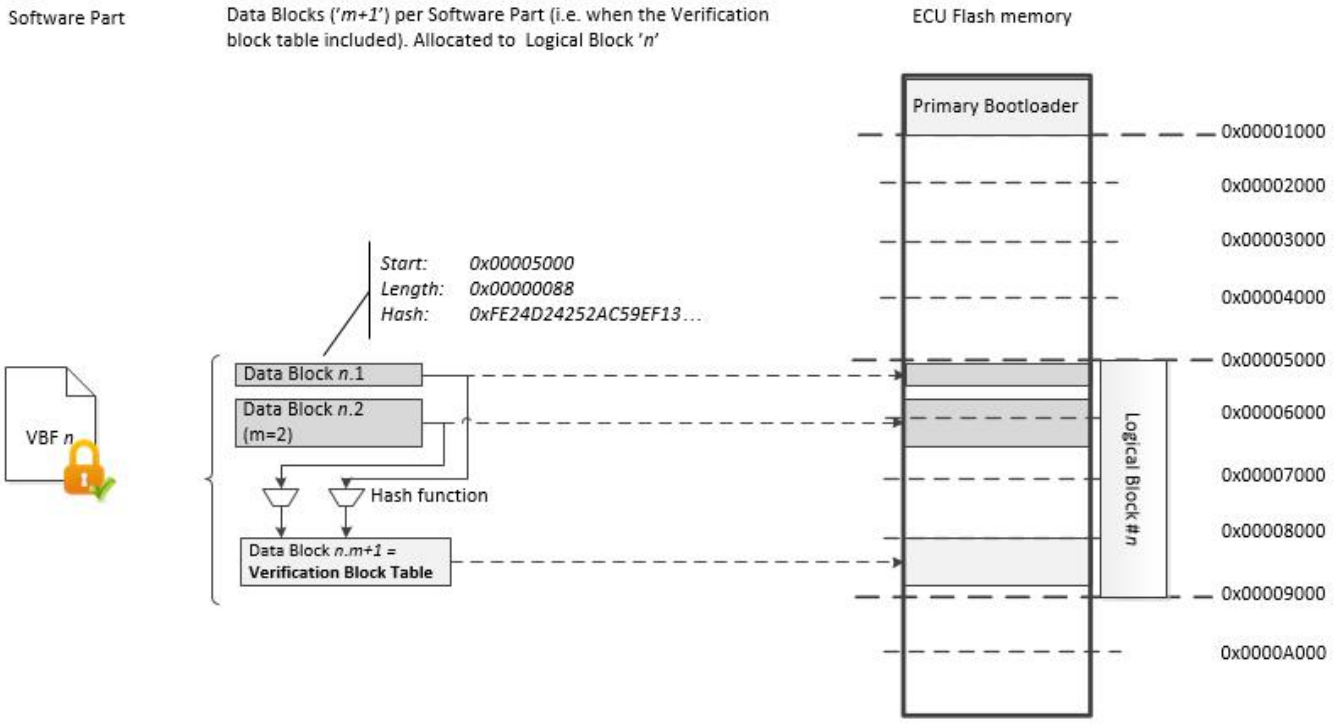


Figure - Verification Block Table overview

In *Figure - Verification Block Table overview* it is shown how the verification block table is constructed. The hash values are calculated on the data parts only, i.e. the start address and length is not used as input to the hash function. The placement of each data block is ensured as the start address and length are also defined in the table. The format of the table is defined in *[General Software Authentication].*

3161 v4 **Allocation of the Verification Block Table**

Version history:
(3): Change the Verification Method from Test to Analysis+Test
(4): Clarifiction

Purpose: The position of the verification block table must be known by the bootloader in order to perform the verification. The verification block must not be part of any other data block, as the verification block table contains hash values of all other data blocks, i.e. it cannot contain the hash of itself.

Source:

Verification Method: Analysis AND Test

Legacy ID: 398372v2

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

3162 v2 **Verification Block Table - Reserve space at application build process**

Version history:
(2):

Purpose: The verification block table is part of the post processing sequence, meaning that it is generated after the actual software is build.

Source:

Verification Method: Test + Analysis

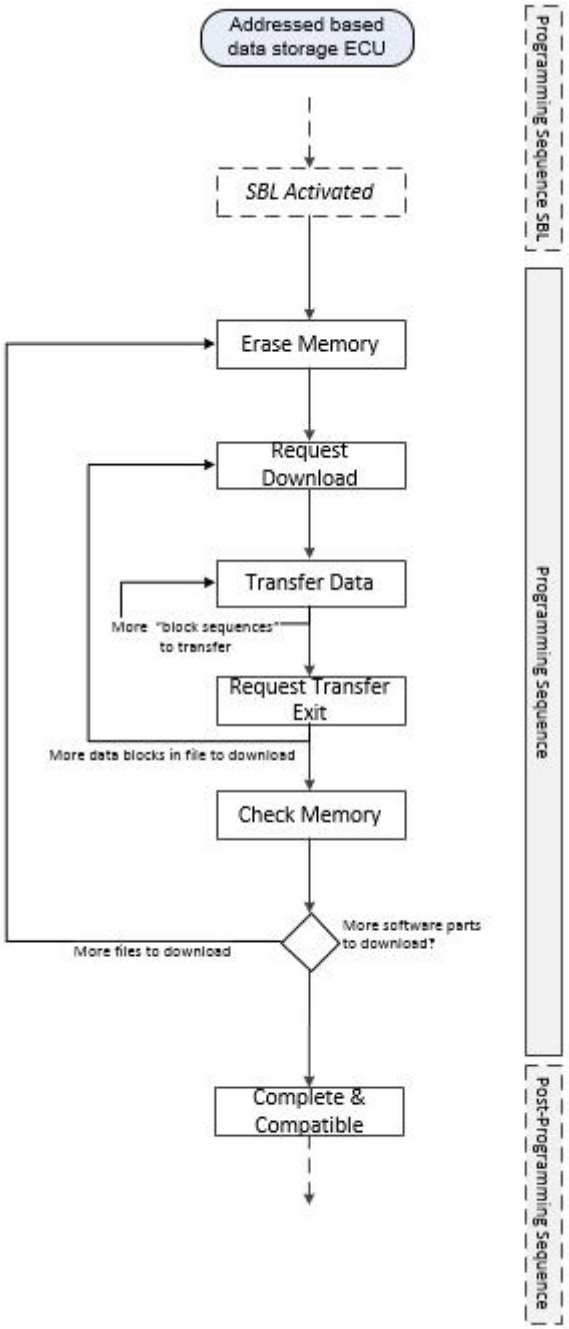
Legacy ID: 402292v1

The application software build process must reserve memory space for the verification block table within a logical block. It is the software supplier that creates the block table and includes it in the software part.

4.6.1.2.1.5 Validation of ECU software(s) - Authenticity Verification

4.6.1.2.1.5.1 Authenticity Verification - Overview

An overview of the authenticity verification was previously described in the beginning of the section *Software Authentication - Introduction*. The programming sequence, highlighting the authenticity verification parts is shown in *Figure - Programming Sequence and authenticity verification*.



1. The addressed logical block shall be set to invalid, when the memory is to be erased. Then, start to erase the complete logical block if it is not already erased. The memory size in erase memory request) shall match the size of the logical block(s).

2. Data transfer to a single logical block is initiated. For every request download, always check that the logical block is invalid. Only one logical block and no restricted area shall be addressed by the download requests (abort otherwise).

3. The data blocks are programmed. Again, data must not be programmed to restricted areas. If a processing method (e.g. compression) is applied, the bootloader will not at RequestDownload know the final length of the data to be written

4. Verify the authenticity (signature verification) of the updated logical block). The start address and length of the verification block table is used as input to the verification function. Unused areas shall be blank/empty. If the check is failed, the addressed logical block shall be kept as invalid. A successful verification must be required to set the logical block to valid. The CheckMemory will contain the corresponding signature to the downloaded data. The verification result is stored (validity status) and used as input to Complete&Compatible check. The verification is done on data in the non-volatile memory. It is not allowed to pre-calculate and verify received data in the volatile memory.

5. If an application call-out is performed (e.g. for comaptibility), the call must be within a already verified (valid) range.

Note: *When the application is to be started, the application start address shall also be within a verified range.*

Figure - Programming Sequence and authenticity verification.

4.6.1.2.1.5.2 Validation of Logical Blocks

3159 v3 **Conditions for a logical block to be invalid**

Version history:
(3):

Purpose: A logical block must be verified with respect to its authenticity every time it is modified. Any condition that triggers a change of memory (erase/write) of a

logical block shall result in that the logical block is set to (or kept) invalid.

Source:

Verification Method: Test

Legacy ID: 400994v2

When the ECU internal conditions for altering (erase/write) a logical block is true, e.g. triggered by a physical/functional EraseMemory, RequestDownload etc., request, the ECU must ensure that the validity status of the addressed logical block is set to invalid prior to the actual memory erasure (or data write) operation is started. The logical block shall be invalid if the authenticity verification is failed. For every RequestDownload, the ECU shall always check that the logical block is 'invalid', otherwise the ECU shall:

- (i) abort the request, for ECUs where a EraseMemory request must be received prior to the RequestDownload *or*
- (ii) set the logical block to 'invalid' and continue with the requested service, where no EraseMemory is required

3158 v4 **Condition for a logical block to be valid**

Version history:
(4):

Purpose: A logical block must be verified successfully with respect to its authenticity every time it has been altered, in order to be considered as valid.

Source:

Verification Method: Test

Legacy ID: 400992v3

A logical block must only be set to valid if it has passed the authenticity verification as defined in *Figure - Authenticity Verification function*. The verification shall be done on data in the non-volatile memory (i.e. at the target location).

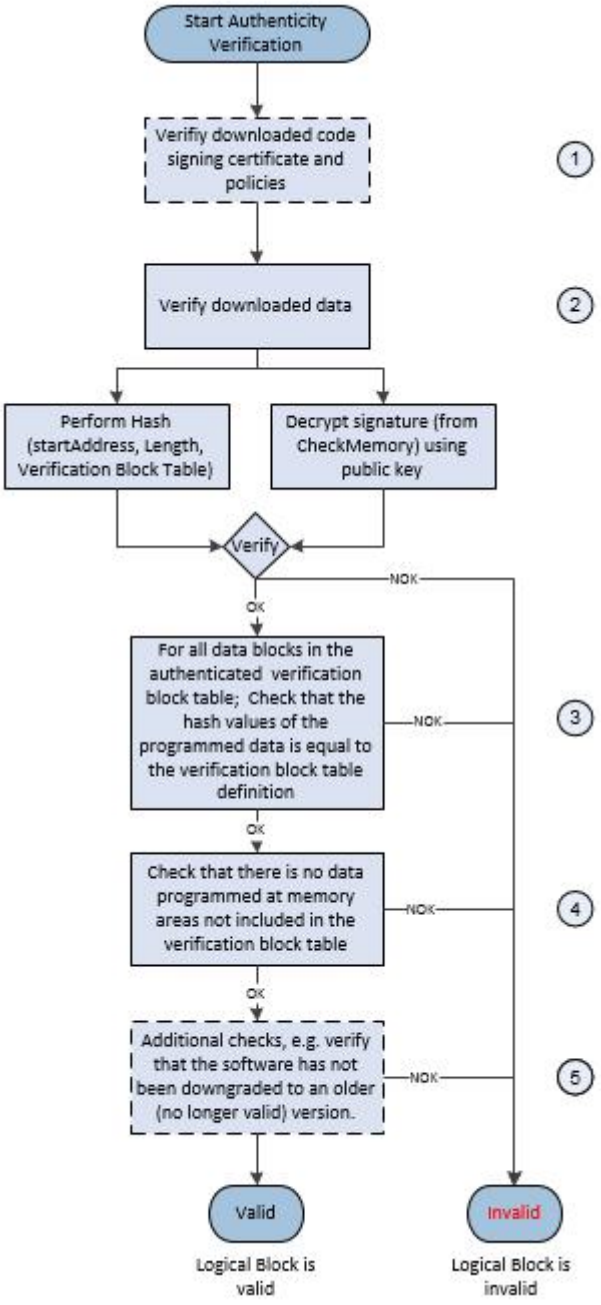


Figure - Authenticity Verification function

(1) - Applicable only if code signing certificates are applied, see *[Software Authentication for file based systems*] for additional requirements.
(2) - Signature verification. Note that the start address and length are used as input to ensure the placement, i.e. the length must for some use cases be calculated (delta encoding). The format and length of the verification block table shall be verified. Prior to the signature verification is performed, the integrity of the public key shall be ensured.
(3) - Verify that the hash value (per data block) is identical to the definition in the verification block table.
(4) - Verify that the non-programmed areas (not part of the authenticated verification block table) are erased. Previously programmed data that is no longer in use shall have been erased

("uninstalled").
(5) - Additional checks, e.g. if the ECU has support to prevent downgrade, meaning that an older (no longer valid) software part must not be installed as it might include some vulnerabilities. CEVT to be consulted if such protection shall be applied (processes must be defined at off-board to support such a use case).

When the authenticity verification function is completed, it must be ensured that the verification status (validity status information) is stored correctly.
Mandatory for the ECU Software Structure: The Logical Blocks defined in the programmed ESS covers the complete memory, i.e. all memory are always to be verified.

4.6.1.2.1.5.3 Validity Status Information

A software part is considered as complete (valid) when the authenticity verification of the belonging logical block is passed. Hence, besides the verificaiton at download the ECU must either verify the software at every boot (recommended) or store information in a protected non-volatile memory about the verification status of each logical block (e.g. valid or invalid), with respect to the authenticity verification at software download. This information is referred to as validity information. Every time a logical block is altered, e.g. erased or programmed, its status must be set to invalid until a successful verification has been completed.

For ECUs verifying the authenticity of the bootloader and application software(s) prior to every execution (e.g. at start), there is no need to store the validity status information in a non-volatile memory. Due to e.g. start-up timing requirements, the ECU possibilities to do the signature check (as in CheckMemory) at every start is limited. To be able to perform a verification at start, the solutions provided by the ECU hardware platform in terms of cryptographic support is to be used.

3160 v6 **Validity Status Information - Properties**

Version history:
(6): Clarification

Purpose: The validity information is the status of the authenticity verification. Only a trusted entity must be able to store (alter) the validity information. A minimum size is defined to avoid that a "bit-flip" result in that a logical block is set to 'valid'.

Source:

Verification Method: Test + Analysis

Legacy ID: 400961v6

The validity status information for each logical block, i.e. if the block has passed- or failed the authenticity verification at software download, shall have a minimum length of 8 octets. One single value shall represent ‘Valid’ and all other values shall be interpreted as ‘Invalid’. ‘Valid’ must not be represented by setting all bytes to the same value, e.g. 0xFF, or to the value of erased memory.
If the authenticity verification is not performed at every boot sequence, the validity information must be stored in a non-volatile protected (secure) memory. The best available solution provided by the ECU shall be used (hardware security module or equivalent, EEPROM, a pattern in flash etc.). The validity status must not be altered by other means than by the

trustworthy verifying component, i.e. it must not be altered by the application or by a tester tool or any other unauthorized entity addressing that area. It must also not be possible to read, or upload, using a tester or debugger tool.

Note: *An ECU unique value for 'Valid' shall be applied.*

4.6.1.2.1.5.4 Public Key Storage

See *[General Software Authentication]* for requirements about public key programming during test and production.

3156 v3 — Public Key storage

Version history:
(3): Clarification

Purpose: The trust anchor shall not be possible to manipulate, that could be used to bypass the protection.
Source:
Verification Method: Test + Analysis
Legacy ID: 366329v2

The trust anchor, i.e. the public key, shall be stored in a protected (read-only) area, typically in a hardware secure module or a protected boot area. It shall be one-time-programmable.

3157 v3 — Re-programmable initial development key

Version history:
(3): Change the Verification Method from Test to Analysis+Test

Purpose: Define key management with respect to the test, development and release (production).
Source:
Verification Method: Analysis AND Test
Legacy ID: 429095v1

The ECU shall use an initial public key for development purposes. It is stored in the ECU by the ECU supplier (e.g. "virgin" ECU). When the initial development key is stored in the ECU, the public key for production shall always be possible to program. When a valid public key has been programmed, the initial development key must no longer be used by the ECU. The development key defined in *[General Software Authentication]* shall be applied unless else is explicitly specified.

4.6.1.2.1.6 Programming Sequence - Authenticity Verification

This section will show define the ECU actions part of the programming sequence, as previously shown in *Figure - Programming Sequence and authenticity verification*.

4.6.1.2.1.6.1 Memory Erasure

3134 v4 — Erase Identifier - restrictions

Version history:

(4):

Purpose: To ensure that a complete logical block is erased is required for several purposes: (1) To ensure that all information about validity status is always erased (if part of the logical block). Otherwise, implicitly knowledge of the bootloader implementation is required to ensure that it is erased. (2) No unknown data is stored in the addressed logical block, i.e. previously installed data deleted ("uninstalled). (3) It will also enable simplified off-board review of the logical block definitions, by checking that the erase identifier is identical to the expected logical block.

Source:

Verification Method: Test

Legacy ID: 400878v3

The erase identifier range(s) of the software part header must match a valid logical block. The ECU shall at an *eraseMemory* request verify that the erase startAddress and Length matches a logical block range, otherwise the ECU shall abort the request. If the eraseMemory request is valid, the ECU must ensure that the complete memory for the addressed logical block is erased.

4.6.1.2.1.6.2 Request Download

3135 v5 — Request Download - restrictions on number of addressed logical blocks

Version history:
(5): Clarification

Purpose: To verify that there is only one software part allocated to each logical block and to enable verification for the downloaded data range, i.e. each logical block is verified independently.

Source:

Verification Method: Test

Legacy ID: 400894v3

The ECU shall check that only one logical block is being updated for a single downloaded software part, i.e. one logical block is allowed to be addressed from a valid RequestDownload to a CheckMemory request is detected. If any memory outside that first logical block is addressed by a RequestDownload request(s), e.g. a second logical block or some location not defined by the ESS at all, the ECU shall abort the RequestDownload and inform the client via a NRC.

As the processed length (compressed/encrypted) length is used as memorySize parameter of the RequestDownload request, there are scenarios when it is not possible to detect this at the RequestDownload stage but identified later when the data is uncompressed/decrypted and about to be written to the memory. Anyhow, the download shall be aborted when it is detected.

To be able to recover when download sequence is aborted, for example trying download the same software part once again or continue downloading the next software part (another logical block), the ECU shall upon a rejected request enter a state where no logical block is active and wait for next request to erase or program the memory.

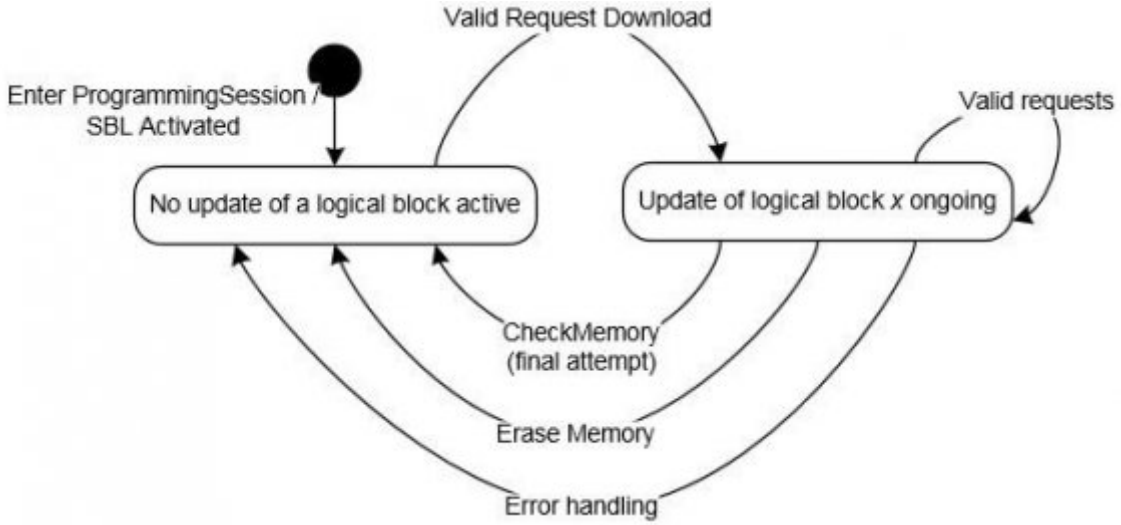


Figure - Simplified logical block active states overview

Notes:
- "Figure Simplified logical block active states overview" shows the principle only, details are omitted. 'Valid requests' are e.g. ReadDataByIdentifer requests, TransferData, RequestsTransferExit. "Error handling" includes the previously described scenario, where e.g. a RequestDownload addresses a logical block that is the 'active' block.
- The figure does not show the actual verification status of each logical block.

Example.
- the ECU is having a file based memory structure, not requiring an explicit erase memory request prior to request download. SBL is already verified and activated.

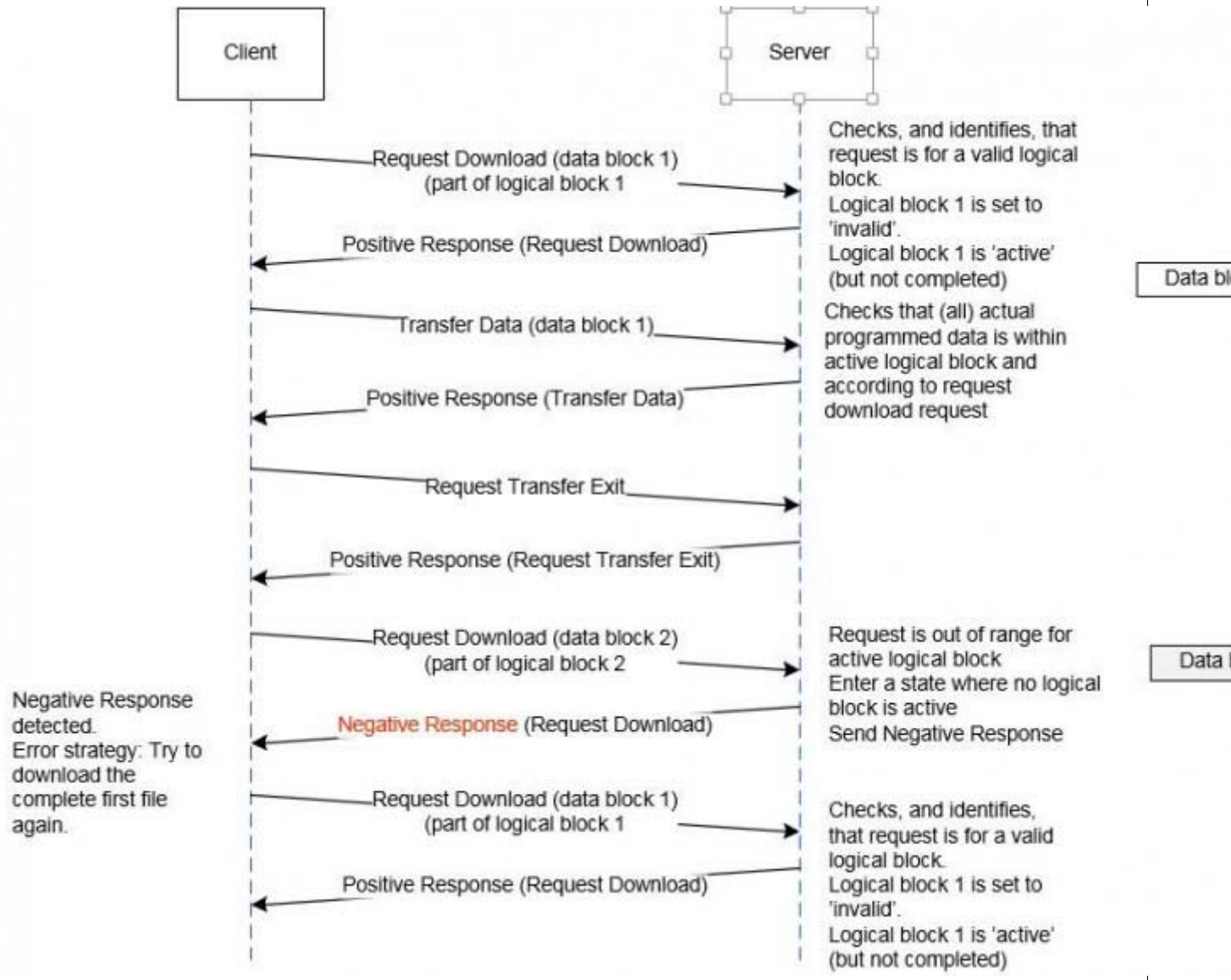


Figure - Example of ECU reaction after failed request (out of active logical block)

4.6.1.2.1.6.3 Check Memory

3129 v3	Check Memory - Return values

Version history:
(3):

Purpose: Define the Check Memory response values.

Source:
Verification Method: Test
Legacy ID: 405471v2

The result values from the CheckMemory request shall be one byte long and represented according to *Table - CheckMemory return Values*. This means that the authenticity verification for a given logical block is passed when the routine returns 0 (CheckMemoryResultValue is 0x00)

Return values from the CheckMemory routine - CheckMemoryResponseValue	
Value	**Description**
Others	*User defined*
0x0B	*Certificate verification failed, i.e. applicable for ECUs supporting code signing certificates.*
0x0A	*Error during storage of validity status information*
0x09	*An additional processor reported failed verification, i.e. only applicable for ECUs having additional processors.*
0x08	*ESS does not cover all programmable memory, i.e. only applicable when the logical block for the ESS is verified.*
0x07	*Read error during hash calculation over memory content.*
0x06	*No data downloaded at all, nothing to verify.*
0x05	*The “blank” check failed, i.e. unexpected data at areas not included in the authenticated verification block table (e.g. data has not been "uninstalled").*
0x04	*The address or hash values of the downloaded data blocks does not match the expected values*
0x03	*Invalid format or length of the Verification Block Table*
0x02	*The public key (“root certificate”) is not stored or is not complete (integrity check failed).*
0x01	*The signed data could not be authenticated, i.e. signature verification failed*
0x00	*The verification is passed*

Table - CheckMemory return values

3130 v3 **Check Memory - verification of programmed data**

Version history:
(3):

Purpose: The CheckMemory is executed in the ECU via the routineIdentifier(Check Memory). This will inform the ECU that a software part is downloaded and that the authenticity verification of the stored data shall be started. For some scenarios, the software download tools will require two attempts (e.g. for development and production units)

Source:
Verification Method: Test
Legacy ID: 405174v2

The ECU shall start the authenticity verification of programmed data when the routineIdentifier

CheckMemory is received. It shall be possible to make two consecutive CheckMemory attempts (i.e. one false attempt is possible). The verification result from the latest CheckMemory attempt shall be applied in the ECU. If the CheckMemory verification has failed and no more attempts are allowed, a new data transfer of the software part must be performed in order to verify the software at a CheckMemory request again. If a third consecutive CheckMemory request is sent, that request must be aborted.

Tester	Description	Data								Explanation
~~Tx~~	CheckMemory request (f.f)	11	04	31	01	02	12	xx	xx	
Rx	Flow control	30	00	00	00	00	00	00	00	
~~Tx~~	CheckMemory request (c.f)	21	xx	xx	xx	xx	xx	xx	xx	
...	...	...								
	CheckMemory request (c.f)	25	xx	xx	00	00	00	00	00	
Rx	Negative response	03	7F	31	78	00	00	00	00	Response pending (if needed)
Rx	Positive response	06	71	01	02	12	10	01	00	Verification **failed**
~~Tx~~	CheckMemory request (f.f)	11	04	31	01	02	12	~~yy~~	~~yy~~	A second consecutive CheckMemory attempt is ok, as no other request has been sent in between the CheckMemory requests.
Rx	Flow control	30	00	00	00	00	00	00	00	
~~Tx~~	CheckMemory request (c.f)	21	~~yy~~	~~yy~~	~~yy~~	~~yy~~	~~yy~~	~~yy~~	~~yy~~	
...	...	...								
	CheckMemory request (c.f)	25	~~yy~~	~~yy~~	00	00	00	00	00	
Rx	Negative response	03	7F	31	78	00	00	00	00	
Rx	Positive response	06	71	01	02	12	10	00	00	Verification **successful**

Table - CheckMemory requests

Table - CheckMemory requests shows when two CheckMemory attempts are possible, i.e. that no other request is sent in between. To save time, the ECU shall at the second CheckMemory attempt reuse the previously performed verifications, e.g. hash calculation.

4.6.1.2.1.6.4 Complete and Compatible

The section *[Validation of ECU software(s) - Complete and Compatible]* will define the relation between the complete check and authenticity verification, where a successful authenticity verification is required to pass the complete check. There are however use cases where exceptions might be required. Typical use cases are when e.g. an "APP" has failed the verification and the fault will be indicated elsewhere, i.e. the "APP" will not be started even though the main application is started. The responsibility of how to inform the user is left to the function to decide.

4.6.1.2.1.6.5 ECU Reset - Application start

3133 v1 **Check Memory - Application Start**

Version history:
(1):

Purpose: To ensure that the application is executed from a trusted location.
Source:
Verification Method: Test
Legacy ID: 416011v0

When the application start up conditions are fulfilled, according to the Complete&Compatible function, the application start address must be within the range of a validated logical block.

4.6.1.2.1.6.6 Download and activate the SBL

It is important that the authenticity of the secondary bootloader (SBL) is verified prior to its activated. Otherwise, there is an increased risk of having the primary bootloader replaced or by other means bypass the provided verification mechanisms and start the application. The principle of verifying the SBL is the same as for any other logical blocks in the non-volatile memory, but for the SBL it is recommended to implement a single data block (but with addition of a *verification block table*). This might result in a simplified verification of the SBL. A single block SBL is however not suitable for ECU hardware architectures having additional microcontrollers, but only one SBL (software part) is used. Refer to requirements about secondary processors.

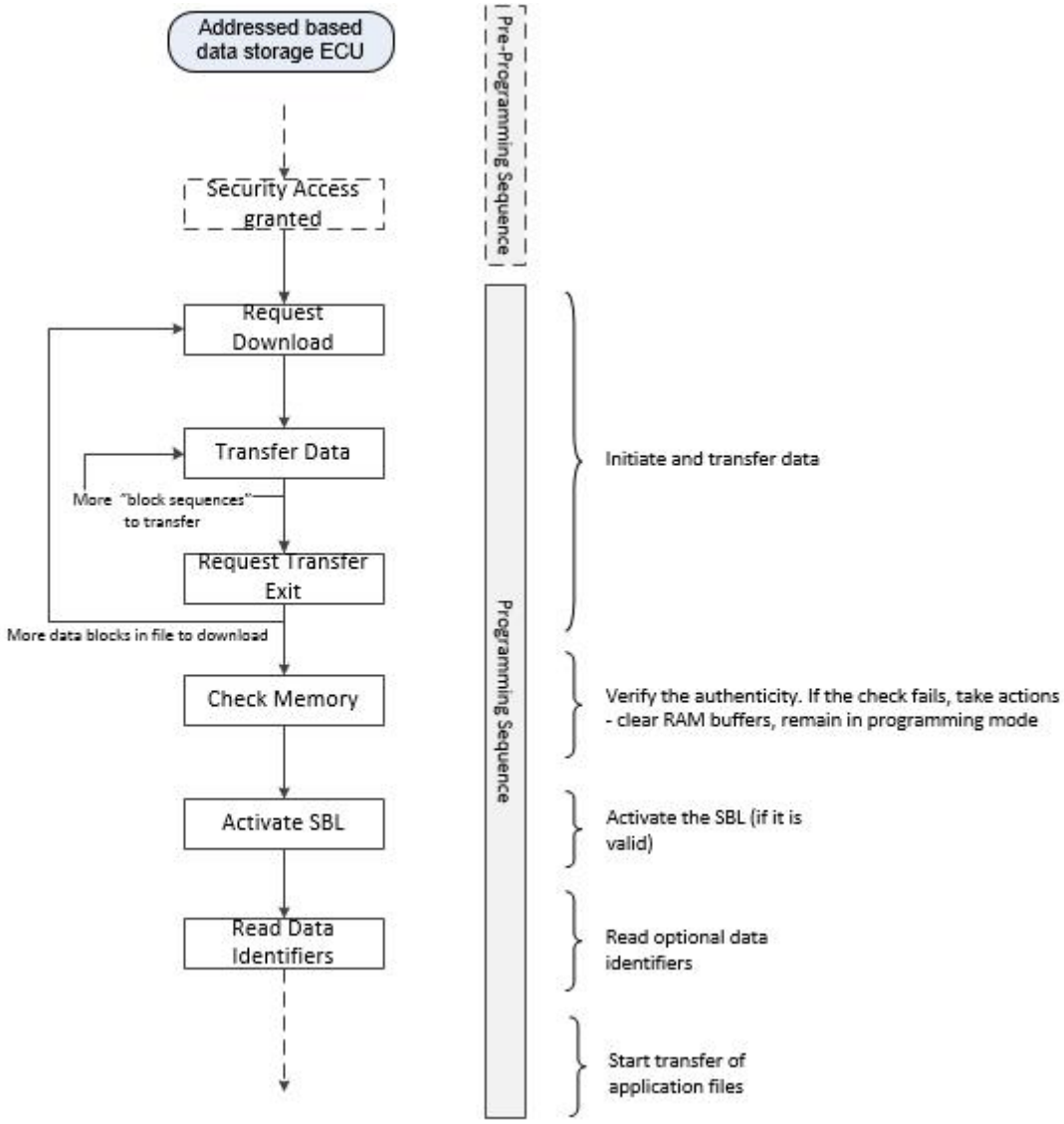


Figure - Overview - Programming Sequence SBL

The *Figure - Overview - Programming Sequence SBL* shows an overview of the programming sequence, a more detailed description of the Tester-ECU requests and responses are to be shown in next coming sections.

3131 v4 **Secondary Bootloader - Condition for activation**

Version history:
(4):

Purpose: The authenticity of the secondary bootloader must be verified. Otherwise, the verification mechanism could be bypassed and arbitrary application could be started.
Source:
Verification Method: Test
Legacy ID: 400704v3

The primary bootloader must ensure that the authenticity verification of the secondary bootloader is passed prior to it is activated. If the verification fails, the ECU shall remain in the primary bootloader and the volatile memory buffer shall be cleared. The used address parameter of the *ActivateSBL* routine control must be within the address range of the validated block(s).

3132 v2 **Secondary Bootloader Verification of unused data**

Version history:
(2):

Purpose: The authenticity verification shall check that there is no data programmed to memory not part of the authenticated verification block table. The SBL is stored in a volatile memory, RAM, and dependent of the actual RAM implemenation it might be faster for the bootloader to fill (write) the unused memory with known data (e.g. 0xFF) instead of checking (read) that no data has been programmed to memory not defined in the verification block.
Source:
Verification Method: Test
Legacy ID: 416451v1

The bootloader shall ensure that there is no SBL data programmed to memory that is not defined in the verification block table.

4.6.1.2.1.6.7 Request Upload

3136 v2 **RequestUpload - Restricted use**

Version history:
(2): Clarification

Purpose: Define restricted areas and recommendations if the RequestUpload service is supported.
Source:

Verification Method:

Legacy ID: 429415v1

If the RequestUpload service is supported by the ECU, the following areas shall be blocked and not possible to upload data from:
(i) Boot area and where the verification function is stored
(ii) Area where the public key is stored
(iii) Area where validity status information is stored
(iv) Area where any confidential key is stored

Note: For intellectual properties rights (e.g. when the software part is encrypted at download), the service shall be used with care.

4.6.1.2.1.7 Additional requirements for ECUs having multiple processors

The term main processor is in this context used to define the microcontroller having the interface to the network used for software download. The requirements in this section are valid for ECUs where all software to the secondary processor is not directly sent to the secondary processor but being addressed to the main processor, i.e. it is considered as one single ECU from a tester point of view.

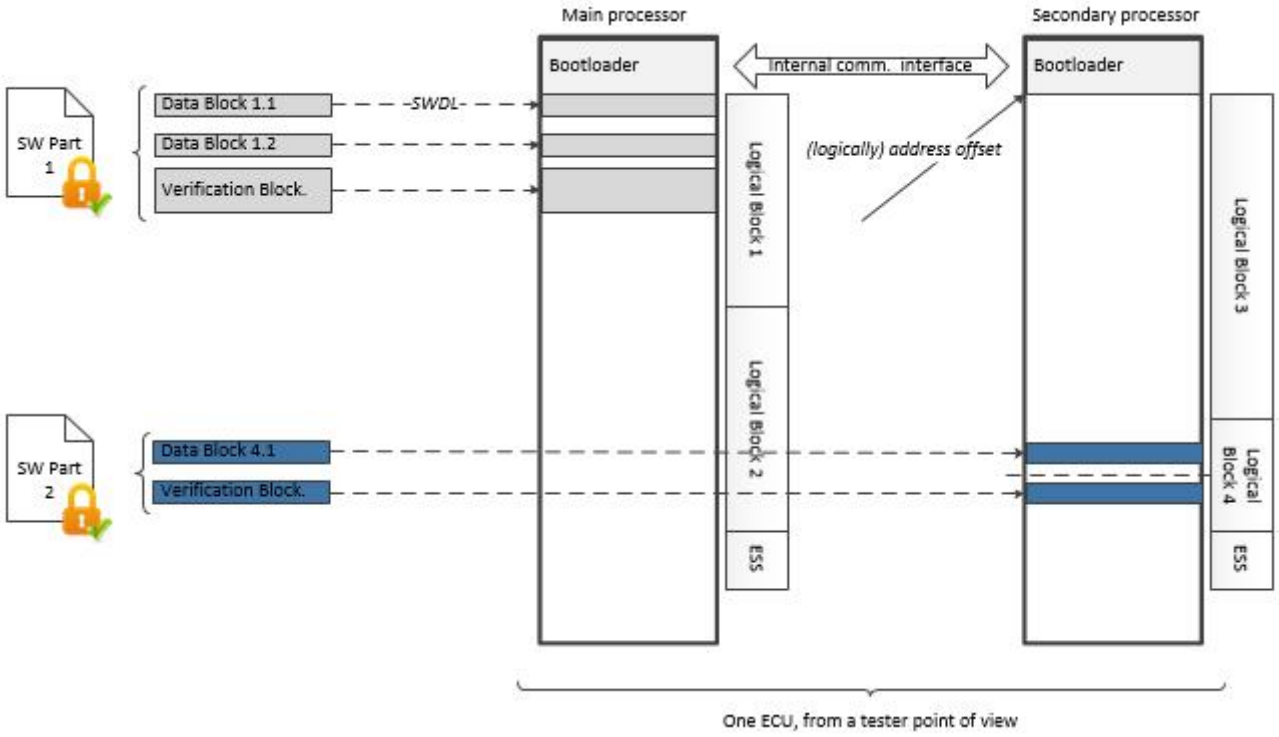


Figure - Additional processors - Overview

The bootloader of the main processor must be able to identify the software parts belonging to a memory device at the secondary processor, typically by using an address offset, and transfer it accordingly. It is recommended that the main processors keeps some information about the memory layout of also the secondary processor in order to do a first filtering before the data is

transferred to the secondary processor. The principle to apply is that the logical blocks shall be verified by the entity where it is executed.

3124 v2 **Multiple processors - verification principle**

Version history:
(2):

Purpose: From a security point of view, the software is to be finally verified at the target processor. This will reduce the risk of having software altered when it is in transit.

Source:

Verification Method: Test + Analysis

Legacy ID: 401206v1

The logical block shall be verified by the target processor respectively, i.e. either by the main- or the secondary processor.

Note: *The actual ECU implementation is highly HW dependent, e.g. capabilities of the processors and internal interfaces.*

4.6.1.2.1.7.1 Multiple processors - Secondary Bootloader

The Secondary Bootloader is a special scenario, where it is required to address two (main/secondary processor) different memory devices in a single software part. See *Figure - SBL verification for ECUs having additional processor*.

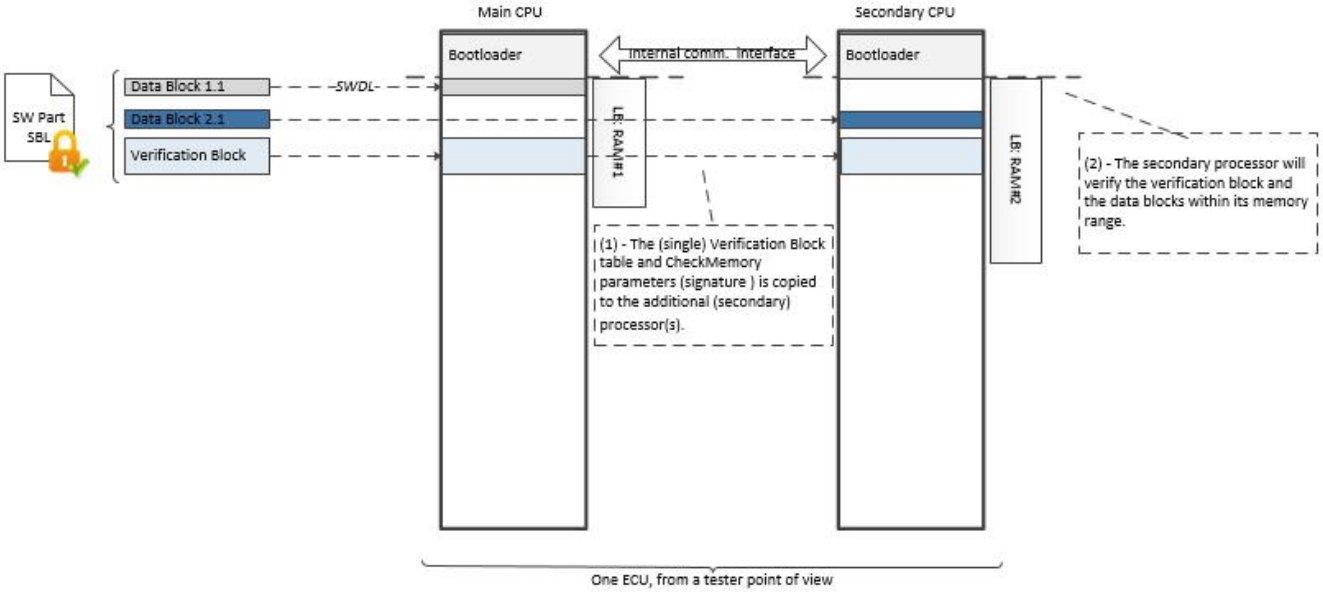


Figure - SBL verification for ECUs having additional processor

- **Responsibility of the main processor:**
- The main processor will verify the authenticity of the verification block table and verify and the data block within its memory range. The main processor will request the verification result from the secondary processor, i.e. for the data blocks addressed to the secondary processor. Hence, the main processor must forward a copy of the verification

block, its signature and all data blocks belonging to the secondary processor.

- **Responsibility of the secondary processor:**
- The secondary processor will also verify the authenticity of the verification block table but not verify the number of data blocks part of the verification block. The actual number of downloaded data blocks, verification block excluded, shall be 1. The secondary processor shall verify that the start address, length and hash value of the downloaded data block is identical to the values defined in the verification block. It shall report the verification result to the main processor.

3125 v2 **Multiple processors - Main processor SBL verification**

Version history:
(2):

Purpose: Define the verification performed by the main processor with respect to the SBL.
Source:
Verification Method: Test + Analysis
Legacy ID: 402299v1

The main processor shall (at least) verify the data blocks dedicated to memory space of the main processor itself. The main processor shall forward the data blocks belonging to the secondary processor together with a copy of the verification block table.

Note: *The verification is, besides this exception, to be performed as for any other logical block (e.g. including signature verification etc*.).

3126 v3 **Multiple processors - Secondary processor SBL verification**

Version history:
(3):

Purpose: Define the verification performed by the secondary processor with respect to the SBL.
Source:
Verification Method: Test + Analysis
Legacy ID: 402300v2

The secondary processor shall first verify the verification block table and then the downloaded data blocks within its memory space. The secondary processor will not receive all data blocks as defined in the verification block table.

4.6.1.2.1.8 Additional requirements for ECUs implementing delta encoding

The delta file will contain information about the target version, as for any other processing method.

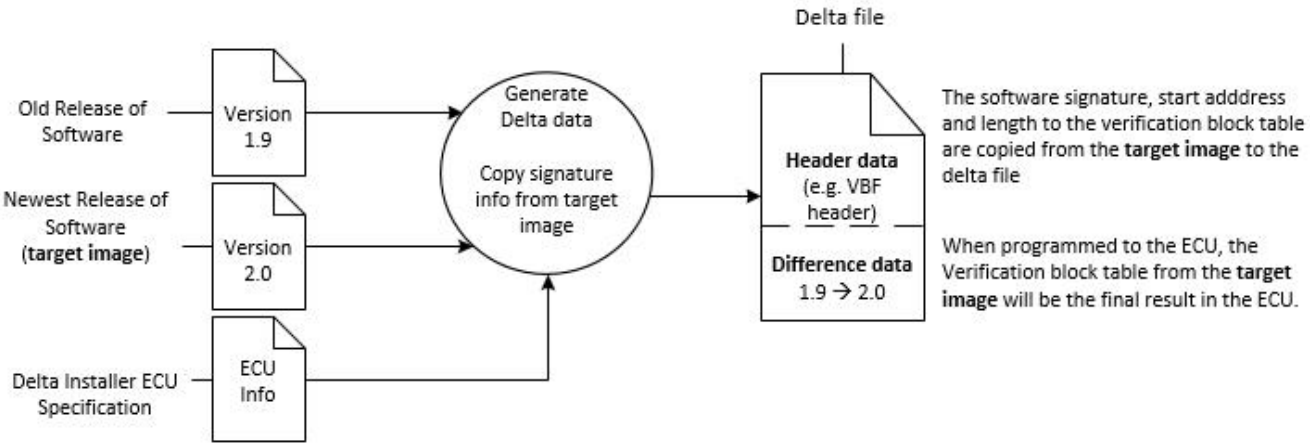


Figure - Content of a delta file

Figure - Content of a delta file shows that the required header information for authenticity verification is added to the delta file, i.e. the signature of the target software (version 2.0).

For ECUs implementing delta encoding, there are some conflicting requirements, e.g. that a complete logical block must always be erased but delta requirement to only erase sectors that are altered (no eraseMemory requests are required by the client). If the ECU is storing Validity Status Information in the flash memory within a logical block range, that flash sector will require some "special handling" to ensure that the altered logical block is always set to invalid.

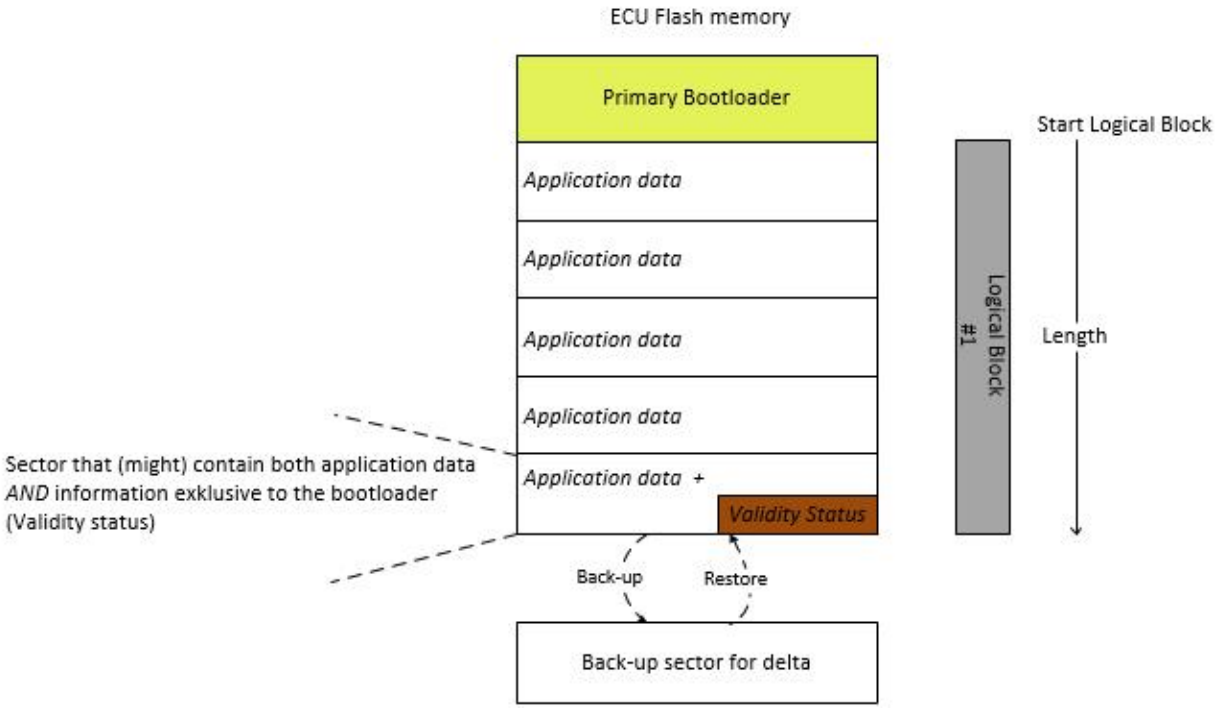


Figure - Delta programming and validity status

Figure - Delta programming and validity status shows a scenario where the ECU is storing the validity status in a flash sector, which also could contain application data (to not waste a single sector for validity status). The bootloader shall always handle that "shared" sector no matter if

the application data part has been altered or not. See example:

Example.
(1) - Some part in the logical block is about to be altered.
(2) - The bootloader must always set the logical block to invalid, prior to the memory is altered, i.e. the Validity Status is set to invalid at the RequestDownload request (erase memory request is not applied for delta encoding)
(3) - When the Validity Status is erased, the shared sector is copied to the delta back-up sector.
(4) - The shared sector is erased
(5) - The data stored in the back-up sector is copied to the shared sector again, now with the validity status set to invalid.
(6) - Start the delta download
(7) - Verify the logical block

It is ECU specific how the Validity Status is stored, previous section is to illustrate an example only.

3127 v2 **Delta encoding - Verification**

Version history:
(2):

Purpose: The verification of a delta installation shall fulfill the same level of authenticity verification as for any other processing method. Due to the nature of a delta installation, some requirements are clarified by this dedicated requirement.
Source:
Verification Method: Test
Legacy ID: 416271v1

When a delta encoding processing method is used, following must (still apply):

- The final target shall be verified, i.e. the unprocessed data after the delta encoding has been applied.
- The verification block table and CheckMemory parameters shall be generated from the target software and copied to the delta file when it is generated.
- A single logical block shall be addressed, e.g. the "delta-library" used must not itself perform any read/erase operations (these must be requested from the bootloader)

4.6.1.2.1.9 Signing method

3137 v2 **Software authentication signing method and algorithms**

Version history:
(2):

Purpose: Define how to identify the signing methods to be used, e.g. the public-key algorithm, key sizes etc.
Source:
Verification Method: Test
Legacy ID: 420111v1

The ECU shall implement the default signing method as defined in *[General Software*

Authentication].

4.6.1.2.1.10 Software Authentication - Generic security requirements

This section contains the generic Software Authentication security requirements. The actual implementation might differ, e.g. dependent of if code signing certificates or only the key-pair are used when signing/verifying the software, when the ECU could perform the authenticity verification etc.

3138 v2 — Allocation of the Authenticity Verification function

Version history:
(2):

Purpose: Tamper resistance, to prevent to that the function is bypassed.
Source:
Verification Method: Analysis
Legacy ID: 366327v1

The authenticity verification function shall be stored in a protected area. If a hardware security module is deployed to the ECU this is required to be used for signature verification function.

3141 v2 — Authenticity verification at software download (Program mode)

Version history:
(2):

Purpose: Verifying code after a software download (at installation) is the minimum level of the concept.
Source:
Verification Method: Test + Analysis
Legacy ID: 360452v1

The authenticity verification function shall always be performed at software download (installation).

3139 v3 — Authenticity verification at every start-up

Version history:
(3): Clarification

Purpose: Define when the ECU authenticity verification shall be started (mandatory on condtion)
Source:
Verification Method: Test + Analysis
Legacy ID: 419731v1

If the ECU is able to fulfill the application start-up conditions, the authenticity verification shall be started every time the (main) application is to be started (via bootloader), typically at every ECU Reset or Power-Up. Hence, the validity status information shall not be permanently stored in the ECU.

Note: *Design consideration - if the verification using digital signatures is not feasible every start, the ECU could after a successful signature verification at software download calculate a message authentication code (MAC) over the verified logical block. During start, that MAC is verified. ECU internal secret keys shall be applied in that case, both the keys and MAC shall be stored in protected memory. This shall be considered if the ECU has the capabilities to calculate and store the MAC and still fulfill the start-up timing* If the verification cannot be performed at start (boot), it must be proven by the ECU supplier by providing e.g. hardware- or performance figures why it is not possible and how the system is hardened against any updates except the defined software download sequence. If any software fails to be authenticated during boot, that sequence shall be aborted.

Design considerations: If the signature verification (as part of CheckMemory) could not be applied at start-up at any stage, the boot and application software(s) are to be signed (and verified) explicitly for verification at boot. These methods relies on solutions using either symmetric or asymmetric keys as provided by the ECU hardware. It relies on HW support to provide the initial root-of-trust (read only) code and a one-time-programmable verification key. Unless others are explicitly required, any others keys but the public key used at CheckMemory are supplier responsibility. Symmetric key solutions (e.g. AES-CMAC calculated over a logical block) requires that the confidential key is in a secured memory, protected from being altered and read as well as ensuring that a verification request origins from a trustworthy component in a secure environment.

Note 1: *Adaptation data that is changed when application is running is not part of start-up check.*
Example.

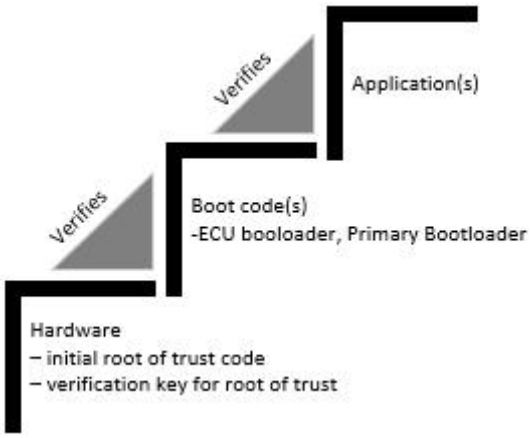


Figure - Chain-of-trust for authenticity verification at start

Note 2: *If the ECU calculates a cryptographic message authentication code (MAC, e.g. AES-CMAC) upon a valid signature verification at software download and verifies that MAC at every start-up, device unique keys must be used.*

3140 v2	Authenticity verification at run time (main application running)

Version history:
(2):

Purpose: This requirement is optional. Authenticity checks could be performed at some time(s) when the main application is up and running. This run time check is to be performed by an additional trustworthy entity, a hardware secure module, and not the main application itself.

Source:

Verification Method: Test + Analysis

Legacy ID: 419911v1

An optional check could be performed by an trustworthy entity, e.g. a hardware security module, when the main application is running. What to verify and how to react is ECU and function specific, but a minimum action is to store an event (security audit log) that is possible to read using a diagnostic identifier.

3142 v1	Debug tools and ports

Version history:
(1):

Purpose: To avoid memory manipulation and e.g. deactivate the verification function or program software.

Source:

Verification Method: Test + Analysis

Legacy ID: 419132v0

The ECU must be protected from all kind of memory manipulations (e.g. via diagnostic services or debuggers) except the diagnostic services supported and specified by in *Global Master Reference Database*.

4.6.1.2.2 Validation of ECU software(s) - Complete and Compatible

The bootloader shall at every start (initiation phase) and upon request from the diagnostic tester (routineIdentifier(Check Complete & Compatible), check if the ECU software(s) is complete and compatible before starting execution of the application.

The purpose of the "Complete & Compatible Check" is to ensure that the expected software parts are programmed successfully and are valid (complete), e.g. to detect if any part of the software is missing, broken (partly present/downloaded) or has been altered. This check shall also decide whether the combination of stored software parts are compatible to each other. The compatible check can be done in many different ways and will be different from case to case. It is either exclusively handled by the bootloader or by using a call-out to a function block that is part of the application. The compatibility check is only performed once the complete check has passed successfully.

The Complete & Compatible check shall be implemented in such a way that it must not be updated for each new or modified data file.

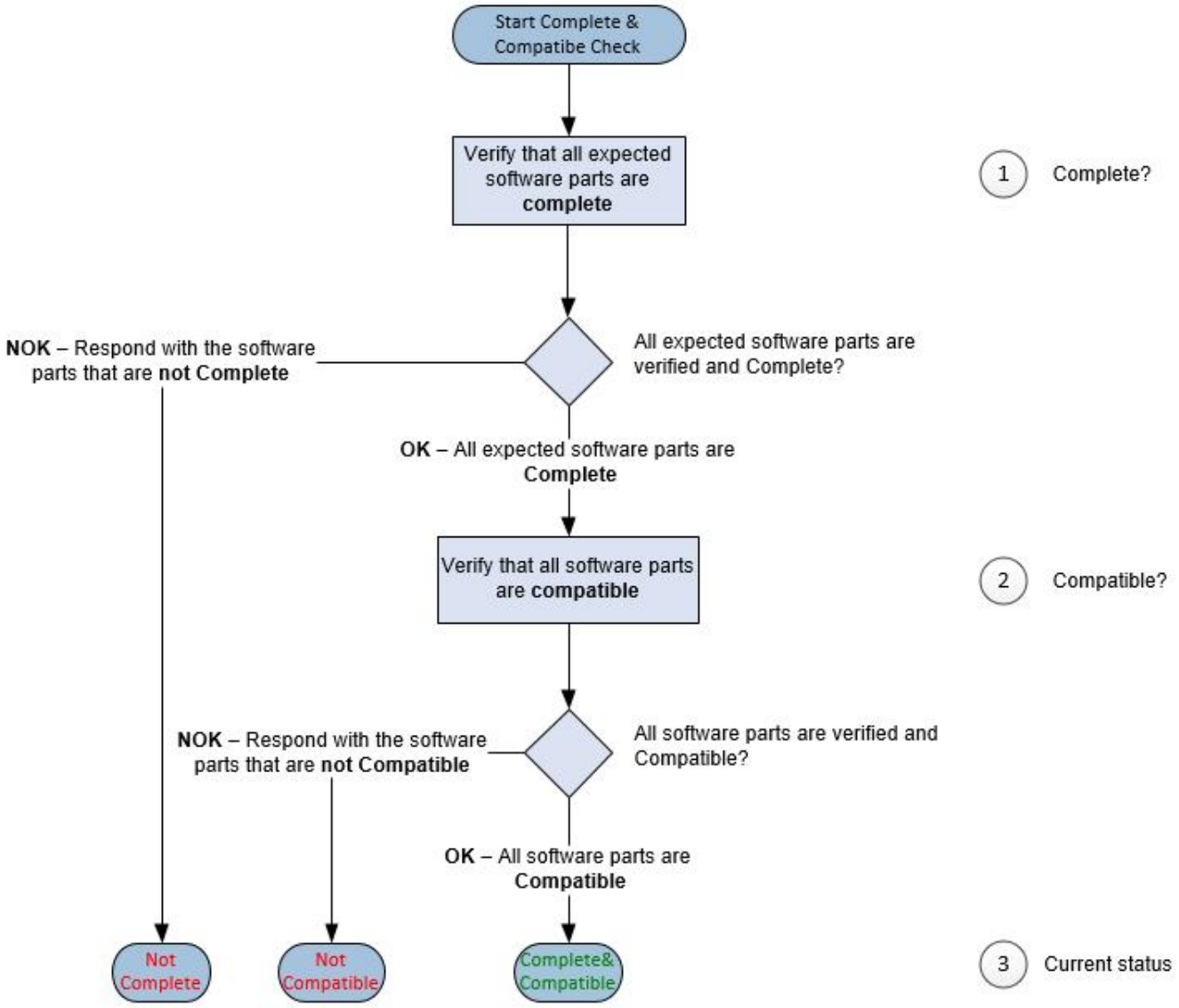


Figure - CompleteCompatibleFunction()

The check shown in *Figure - CompleteCompatibleFunction()* will be referred to as the **CompleteCompatibleFunction()**:

1. Check that software parts are complete.
2. Check that the software parts are compatible. The compatibility check shall only be performed if all expected software parts are complete.
3. The final result of the Complete & Compatible check, including the individual status per software part. The application must not be started unless both the Complete and Compatible check are passed successfully.

3179 v2 Validation of ECU Software(s)

Version history:
(2):

Purpose: To prevent start of incomplete or incompatible software(s).
Source:
Verification Method: Test
Legacy ID: 53935v1

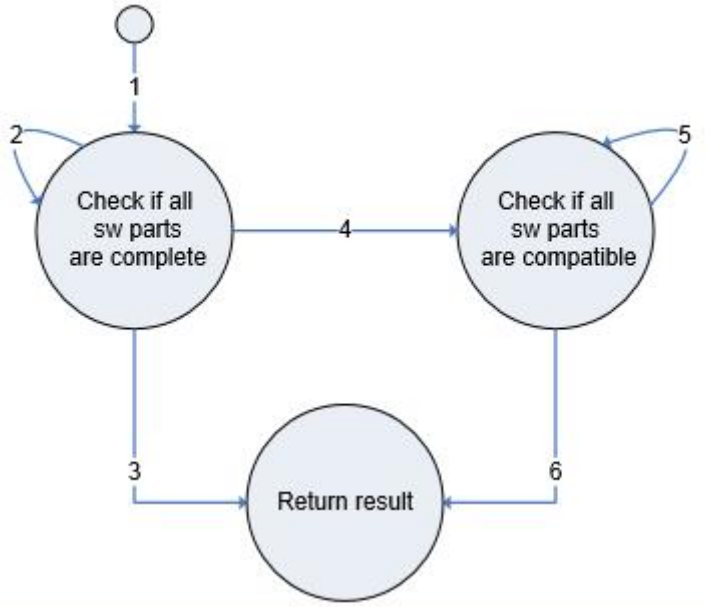
The bootloader shall at every start check if the ECU software(s) is complete and compatible before starting execution of the application.

3178 v5 Complete and Compatible sequence diagram

Version history:
(5):

Purpose: Definition in which order the ECU shall validate the stored software(s).
Source:
Verification Method: Test
Legacy ID: 53959v4

The ECU shall first check that all software parts are complete prior to the compatibility check is performed. If the ECU contains incomplete software(s) it shall not check whether the software(s) is compatible. Hence, the ECU cannot indicate that both the complete and compatible check has failed. The software parts shall be checked in the order defined in *Figure - CompleteCompatibleFunction() sequence diagram.*



Step	Description
1	The bootloader starts the Complete & Compatible function
2	The Complete & Compatible function checks if stored SW parts(s) is **complete** in the following order: (1) ESS (2) EXE (3) Signal Configuration (4) Car Config (if applicable) (5..) Used defined SW parts(s)
3	Return result. If any SW part(s) is incomplete the Complete & Compatible function shall indicate that in the response
4	If all SW part(s) are complete
5	The Complete & Compatible function checks if stored SW parts(s) is **compatible,** using the same order as defined in step 2 but ESS is optional.
6	Return result

Figure - CompleteCompatibleFunction() sequence diagram

4.6.1.2.2.1 Complete and Compatible - Complete check

3183 v3 **Complete and Compatible - Definition of being Complete**

Version history:
(3):

Purpose: The purpose with the complete validation is check that present softwares have been installed (programmed) as expected, i.e. an integrity check. This requirement defines the condition for the whole application to be considered as complete.

Source:

Verification Method: Demonstration

Legacy ID: 403711v2

To be considered complete, all expected software parts must have passed the authenticity verification (using signatures). Hence, the CompleteCompatibleFunction() shall return that a software part is complete or incomplete according to that verification result. See *Figure - CompleteCompatibleFunction() Complete response*.

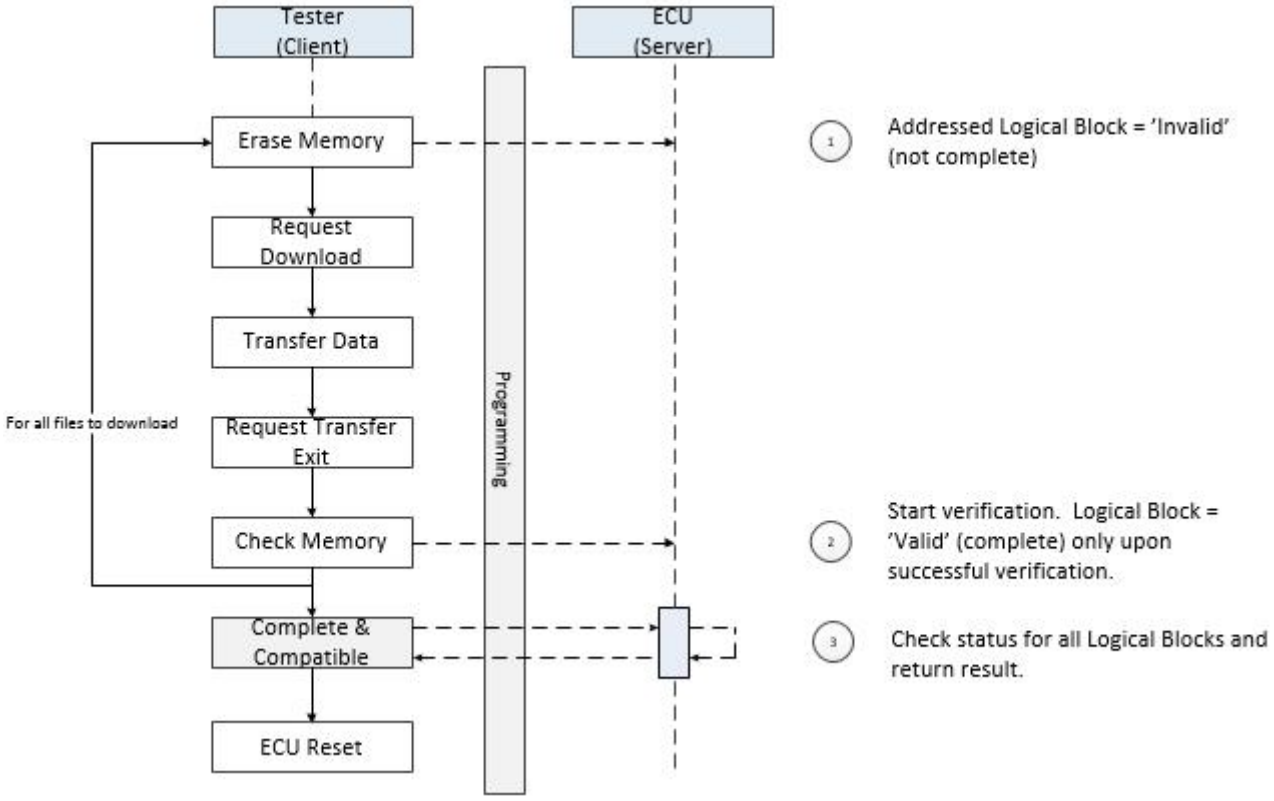


Figure - CompleteCompatibleFunction() Complete response

It might be a few exceptions where the main application might be started even though a software part has not passed the authenticity verification, e.g. an "APP" where the failed verification results in inhibiting the verified application from starting and this is indicated by the main application (function) itself. Every exception must be agreed with CEVT.

The ECU is free to add additional conditions for being complete, e.g. some known patterns in the beginning and end of each software part. That is however left to the ECU designer to decide. Ideally, such patterns are checked prior to the software part is being signed and it will be implicitly verified as part of the authenticity verification.

3182 v4 **Complete and Compatible - Complete function**

Version history:
(4):

Purpose: Define the responsible of performing the complete check. This verification must be performed by a trusted entity, i.e. no call-out to a non-verified area.

Source:

Verification Method: Test + Analysis

Legacy ID: 401157v3

The complete check is always to be performed by the bootloader.

4.6.1.2.2.2 Complete and Compatible - Compatible check

3181 v4 Complete&Compatible - Definition of being Compatible

Version history:
(4):

Purpose: The purpose with the compatibility validation is to detect if present softwares are compatible in such way that the ECU is able to start and communicate. Hence, the ECU application could inform the user by e.g. setting a diagnostic trouble code.

Source:

Verification Method:

Legacy ID: 401140v3

The ECU is compatible if it is at least able to start the application, is able to respond to diagnostics and alert the user if a function is disabled or degraded. An ECU that is compatible shall always be able to enter Programming Session and be re-programmed.

Note: *The application must react accordingly, i.e. enter a defined fail-safe mode if it is started but a function is disabled or degraded. Such a behavior is however the responsibility of the application to decide.*

3180 v3 Complete and Compatible - Compatible function

Version history:
(3):

Purpose: Define alternatives of how to perform the compatibility check, to not limit the ECU designer to use a specific method.

Source:

Verification Method: Inspection

Legacy ID: 401156v2

The compatible check is either exclusively executed by the bootloader or via a call-out to a function placed in a downloaded software part (e.g. application). The bootloader must always verify that the compatible function applied is available and is complete, i.e. it has passed the complete check (resides in a verified memory location) and the existence is verified by e.g. some known start- and end-patterns prior to execution.

Note 1: *It is the responsibility of the bootloader and application developer to agree upon how to realize this check. Please pay attention to e.g. post-build (e.g. reconfiguration of the communication matrix by moving an existing signal) consistency check requirements, which might require information about the application build process.*
Note 2: *See also "Appendix: Section Examples of different compatible functions".*

4.6.1.2.2.3 CompleteCompatibleFunction()

3185 v4 CompleteCompatibleFunction()

Version history:
(4):

Purpose: To define the functionality of the CompleteCompatibleFunction().
Source:
Verification Method: Test
Legacy ID: 53957v3

The CompleteCompatibleFunction() is invoked from the bootloader via either the routine (Check Complete & Compatible) or during the Init phase. The CompleteCompatibleFunction() shall be implemented in a way that allows new or modified software part without an updated application(EXE). The CompleteCompatibleFunction() shall detect if any of the software parts are not as expected, missing, broken (partly downloaded), altered etc. The function is also responsible to decide whether the combination of stored software parts are compatible to each other.

3186 v7 CompleteCompatibleFunction() return value

Version history:
(7):

Purpose: Definition of the bitcoded CompleteCompatibleFunction() return value. When the complete check is part of the bootloader, the bootloader might have limited knowledge of the relation between a logical block and actual sw part type (DATA, EXE).
Source:
Verification Method: Test
Legacy ID: 53958v6

The return value from the CompleteCompatibleFunction() shall be 4 bytes long and bit coded and contain the type of failure and which software part(s) that has failed. If a failure has occurred the bit value is set to one (1), not effected bits are set to zero (0). This means the Complete and compatible function has “PASSED”, when complete and compatible function returns 0. The diagnostic database shall contain the actual return values for an ECU.

Example.
Bit (4) shall be set to "0" if the ECU doesn't supports a software part (e.g. the Car Configuration).

Return values from CompleteCompatibleFunction()

Bit number	Description
Bit 6-31 (msb)	User defined software parts(s). One bit per SW part which is documented by each ECU respectively.
Bit 5	ECU Software Structure
Bit 4	Car Config, if applicable.
Bit 3	Signal Configuration

Bit 2 Executable
Bit 1 Compatible check
Bit 0 (lsb) Complete check
Table - Bit coded return value from the CompleteCompatibleFunction()

3187 v4 Maximum time to perform the Complete&Compatible validation

Version history:
(4):

Purpose: To define the maximum time to perform the Complete & Compatible validation of ECU software(s), to fulfill the ECU start-up timing requirement.
Source:
Verification Method: Test
Legacy ID: 53970v3

The maximum time to perform the total Complete & Compatible sequence described in *Figure - CompleteCompatibleFunction()* shall not exceed 20ms.

3184 v2 CompleteCompatibleFunction - error handling

Version history:
(2):

Purpose: Define a error handling strategy, where a restricted approach is applied meaning that the application must not be started unless it is proven to be valid.
Source:
Verification Method: Inspection
Legacy ID: 405051v1

In case of an unexpected fault during the CompleteCompatibleFunction() that prevents the function to be completed and the root cause cannot be derived to a specific software part, the ECU shall be considered as not complete (or not compatible respectively, dependent of the check that was not completed). This shall be indicated in the response to the routineIdentifier(Check Complete & Compatible) request. If the bootloader cannot identify which part that failed the verification, the bit for the Executable shall be set.

4.6.1.2.3 Time window for PROG detection

If not explicitly stated in respectively bootloader implementation specification shall the primary bootloader (PBL) have a time window (Timeout_Prog) for detection of the DiagnosticSessionControl service. This backdoor solution shall be used if the ECU believes it has valid software(s) present, but the application software is not correctly jumping to the PBL. This method may only be used in a stand alone scenario, due to the impossibility for the gateway ECUs to transmit the DiagnosticSessionControl service in time.

The tester shall send out the DiagnosticSessionControl service with diagnosticSessionType set to programmingSession and suppressPosRspMsgIndicationBit equal to TRUE during a minimum time of 5 seconds. The repetition rate of the DiagnosticSessionControl service shall be sent within 5ms but preferably at 2 ms or faster.

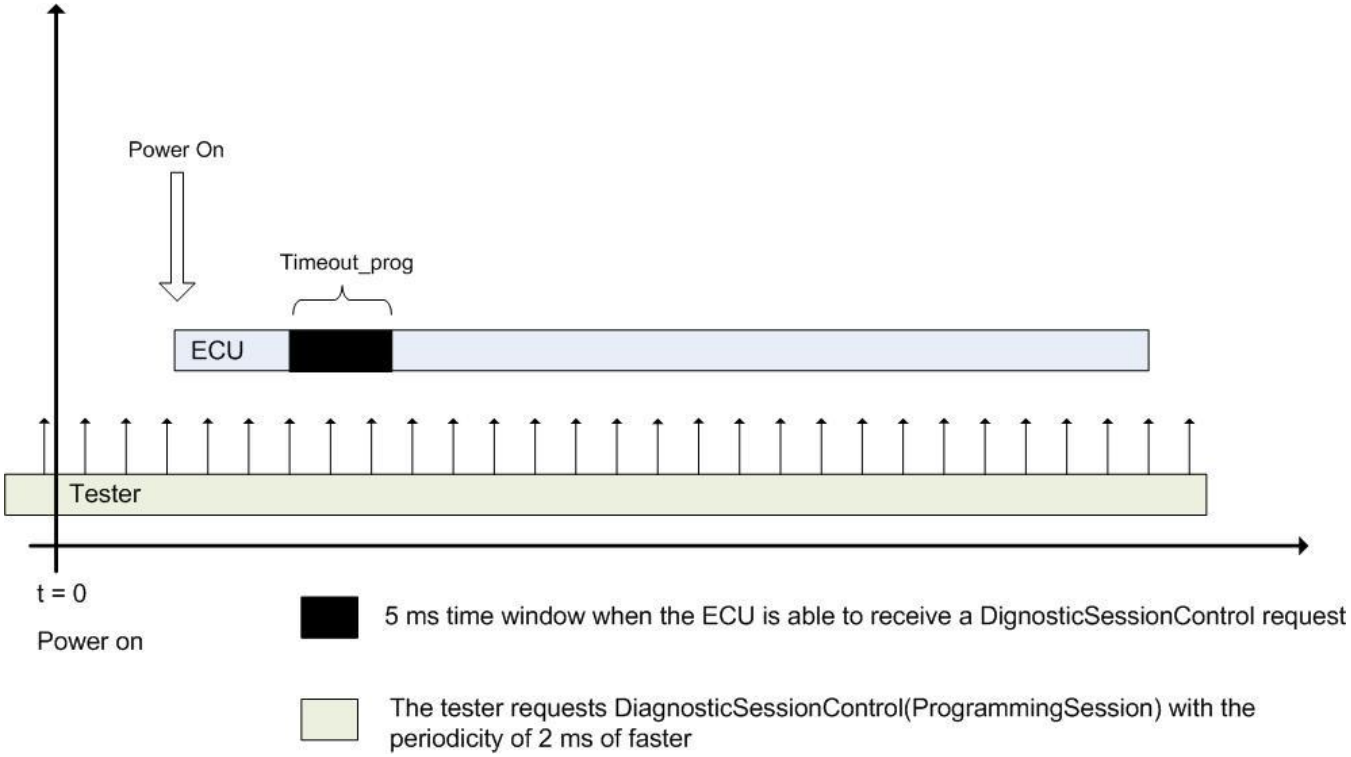


Figure - Typical boot sequence of ECU's in a vehicle

3164 v2 **Time window (Timeout_Prog)**

Version history:
(2):

Purpose: To have the possibility to reprogram the ECU believes if it believes it has valid software(s) present, but the application software is not correctly jumping to the PBL.

Source:

Verification Method: Test

Legacy ID: 53933v1

The primary bootloader (PBL) shall have a time window (Timeout_Prog) of 5 ms (-0% / +10%) for detection of the DiagnosticSessionControl service. If a DiagnosticSessionControl service with diagnosticSessionType equal to programmingSession is received during the time window the ECU shall enter programmingSession state.

The Timeout_prog timer is started when the ECU is able to receive incoming messages.

4.6.1.2.4 ECU program mode

Program mode means that the ECU executing the primary or secondary bootloader.
This section describes the bootloader state and flow chart diagram in a general way. The specific implementation requirements for the bootloader are data link dependent and are therefore specified in respectively data link bootloader specification.

3114 v2 **Disable monitoring and logging of DTCs**

Version history:
(2):

Purpose: To prevent logging and monitoring of DTCs when the bootloader executing.
Source:
Verification Method: Test
Legacy ID: 53929v1

The ECU shall ensure that the monitoring and logging of all diagnostic trouble codes (DTCs) is disabled when executing the bootloader (programmingSession).

3115 v2 **Suspend non diagnostic communication**

Version history:
(2):

Purpose: To only have diagnostic communication between tester and ECU when executing the bootloader.
Source:
Verification Method: Test
Legacy ID: 53927v1

Whenever an ECU is executing the bootloader, it shall ensure that non diagnostic communication e.g. network management messages between ECUs are suspended.

4.6.1.2.4.1 Simplified bootloader state and flow chart diagram

The bootloader state diagram, Figure " *Simplified bootloader state diagram"* shows a simplified example of the different states the ECU can enter in the bootloader. The state diagram is in detailed described in each bootloader implementation specification for respectively network.

- From power on or reset initiated by the application the PBL shall start executing the initialization state where the PBL decide whether the application shall be started or not.
- The primary bootloader (PBL) shall have a time window (Timeout_Prog) for detection of the DiagnosticSessionControl service. This backdoor solution shall be used if the ECU believes it has a valid software(s) present, but the application software is not jumping to the PBL or if the application is not able to process a diagnostic request (see section "*Time window for PROG detection"*).
- The programmingSession state is entered if the reset was initiated by the diagnostic request DiagnosticSessionControl(programmingSession) in the application (Warm start) or if no valid software(s) occurs (see section "*Enter the bootloader (PBL) from the application"* and "*Validation of ECU Software(s)"*).
- The application is started if the start-up was a cold start and valid software(s) is present.

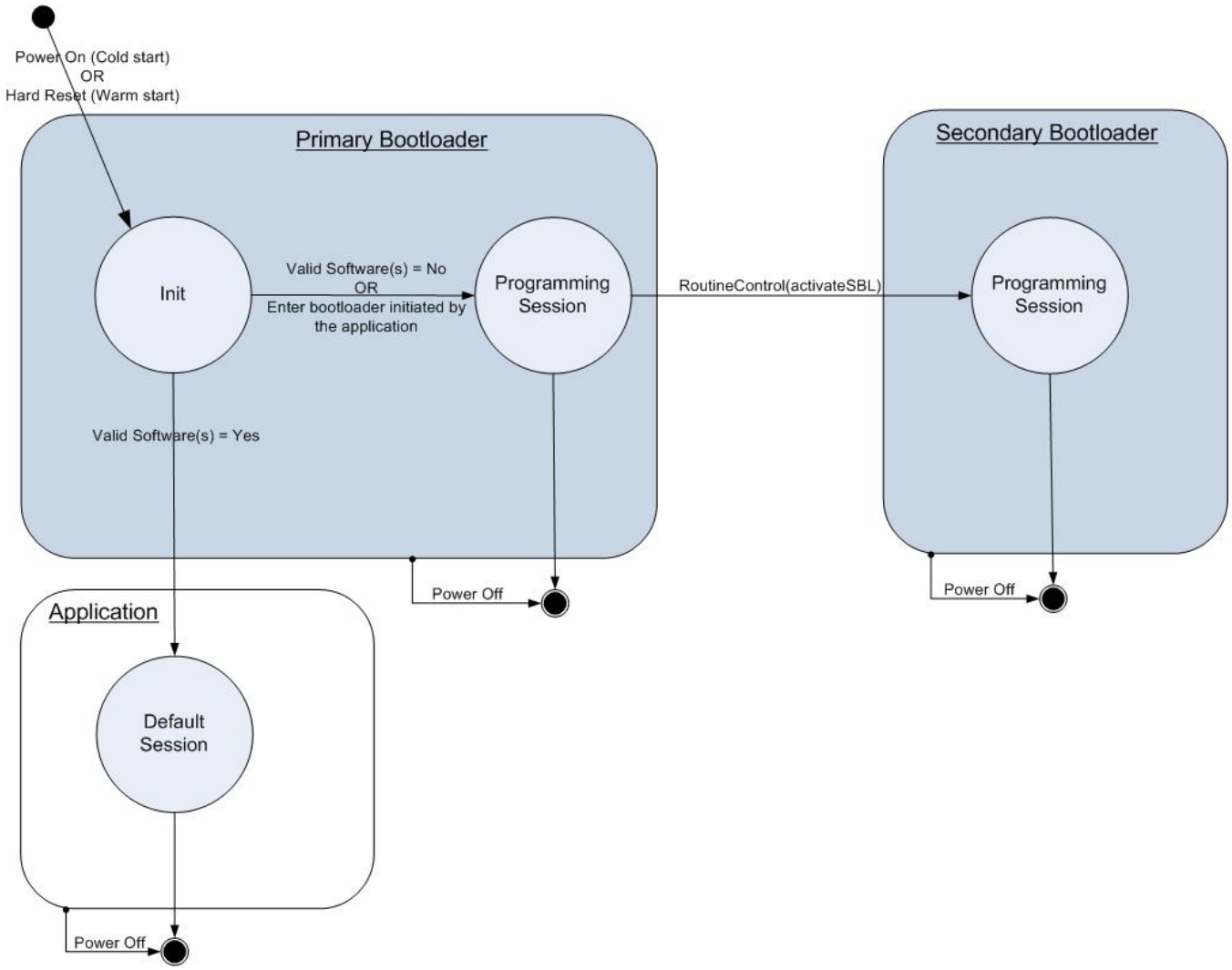


Figure - Simplified bootloader state diagram

The bootloader flow chart diagram, Figure " *Simplified bootloader flow chart diagram"* below shows a simplified example. The flow chart diagram is in detail described in each bootloader implementation specification for respectively data link network.

The Complete and Compatible function which is responsible to make the decision whether valid software(s) is present is in more detail described in section " *Validation of ECU Software(s)"*.

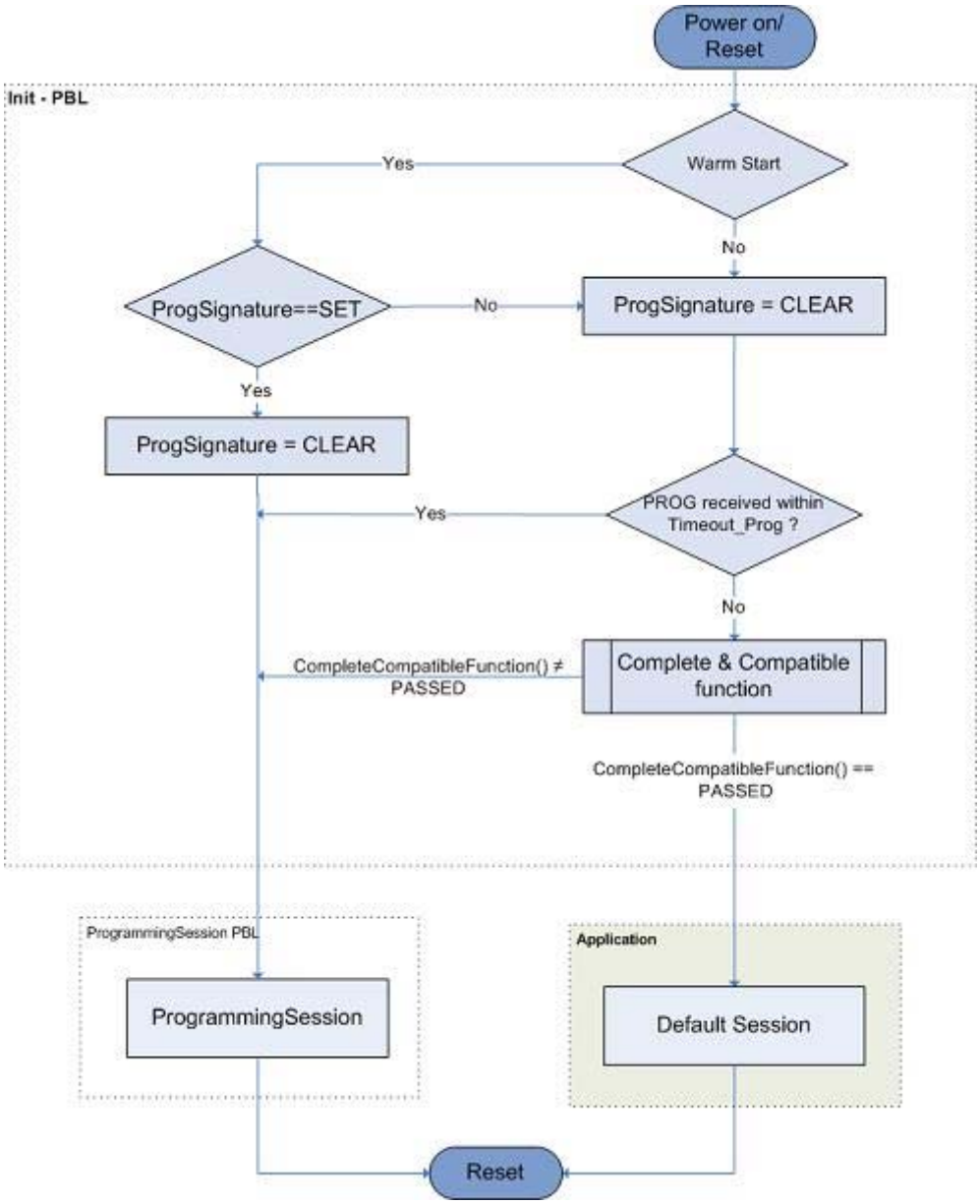


Figure - Simplified bootloader flow chart diagram

4.6.1.2.5 Enter the bootloader (PBL) from the application

If not explicitly stated in respectively bootloader implementation specification the application shall write a signature to the volatile memory after receiving the DiagnosticSessionControl(programmingSession) request.

Signature	**Type**	**Value** (Hex)	**Description**
ProgSignature	SET	0x50726f675369676e	The ProgSignature shall be written to a fixed address in the volatile memory, with a length of 8 bytes and a recommended value of 0x50726f675369676e (equal to text string "ProgSign" in ASCII-format).
ProgSignature	CLEAR	0x00000000000000	ProgSignature shall be cleared by set the fixed

0	address to zero (0).

Table - ProgSignature value and format

It must be possible to enter the primary bootloader by only receiving the DiagnosticSessionControl(programmingSession) service once, which is the normal way of entering the bootloader. To prevent the application to restart after reset must the application therefore indicate to the bootloader with a signature that programmingSession state where requested.

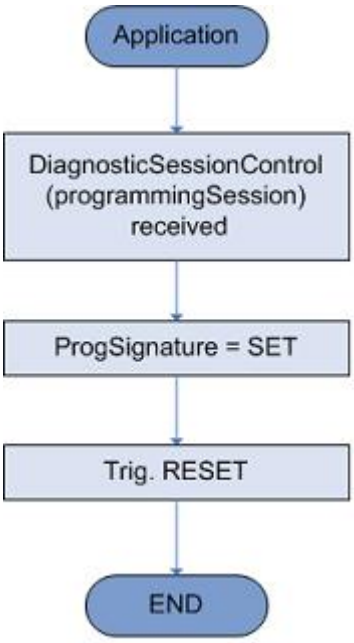


Figure - Write ProgSignature flow chart diagram

3116 v2 **ProgSignature - CLEAR**

Version history:
(2):

Purpose: To prevent unwanted bootloader activation
Source:
Verification Method: Test
Legacy ID: 53932v1

The ProgSignature shall be cleared with a recommended value of 0x0000000000000000 by the bootloader before starting the application.

3117 v2 **ProgSignature - SET**

Version history:
(2):

Purpose: To be able to enter the primary bootloader from the application by sending the DiagnosticSessionControl(programmingSession) service once.

Source:
Verification Method: Test
Legacy ID: 53930v1

The ProgSignature shall be written to a fixed address in the volatile memory, with a length of 8 bytes and a recommended value of 0x50726f675369676e (equal to text string "ProgSign" in ASCII-format) by the application after receiving the DiagnosticSessionControl(programmingSession) request.

Note: The fixed address must be synchronized between the application and bootloader.

4.6.1.2.6 Pre-calculated hash

To decrease the total programming time the ECU could pre-calculate, i.e. calculate hash of data, before receiving the diagnostic CheckMemory request.

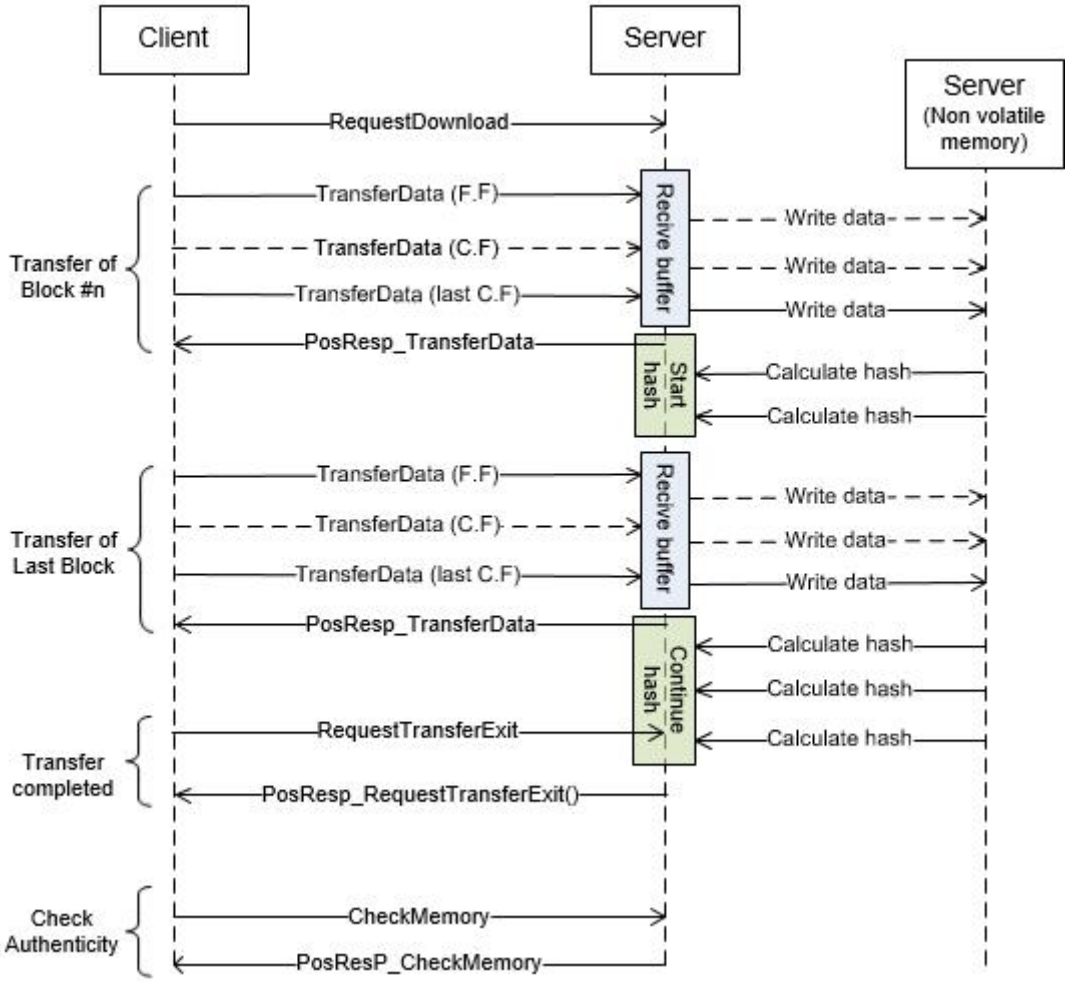


Figure - Pre-Calculated Checksum

This could be realized in several ways, e.g. by always having the verification block table transferred first (first RequestDownload) and having the data blocks in the block table sorted according to the downloaded order. Each downloaded block could then be checked against the block table.

3122 v3 **Pre-calculated hash**

Version history:
(3):

Purpose: To decrease the total programming time.
Source:
Verification Method: Analysis
Legacy ID: 53980v2

The ECU shall in between each TransferData request calculate the hash value, when next TransferData request is received the calculation shall be halted and continue after the data block is written to the non-volatile memory. The ECU must therefore remember where the calculation shall continue.

Important is that the pre-calculation must be done on data in the non-volatile memory. It is not allowed to pre-calculate received data in the volatile memory.

4.6.1.2.7 Appendix

4.6.1.2.7.1 Examples of different scenarios for the Complete & Compatible function

All bits are equal to '0'	The ECU contains complete and compatible software parts. The ECU can start with full normal operational functionality
Bit 6 to 31: is set to '1'	Failure within user defined software parts(s).
Bit 5: is set to '1'	Failure within the ECU Software Structure
Bit 4: is set to '1'	Failure within the Car Configuration (not applicable for every ECUs)
Bit 3: is set to '1'	Failure within the Signal Configuration
Bit 2: is set to '1'	Failure within the ExecutableSoftware
Bit 1: is set to '1'	The ECU contains incompatible software part(s)
Bit 0: is set to '1'	The ECU contains incomplete software part(s)

- If the Signal Configuration data file is created with same fix and target file as the application but with a different net file the returned value from the CompleteCompatibleFunction() shall be compatible.
(0b00000000000000000000000000000000)
- If the Signal Configuration data file is created with the same net and target file as the application but with a different fix file the returned value from the CompleteCompatibleFunction() shall be incompatible.
(0b00000000000000000000000000001010)
- If a data file is modified but still is supported by the application the returned value from the CompleteCompatibleFunction() shall be complete and compatible.
(0b00000000000000000000000000000000)

- If a data file is present (bit 6, user defined software) which makes the application incapable of responding on e.g. diagnostic requests the returned value from the CompleteCompatibleFunction() shall be incompatible (0b00000000000000000000000001000010)
- If a data file (bit 6, user defined software) is missing or broken the returned value from the CompleteCompatibleFunction() shall be incomplete. (0b00000000000000000000000001000001)
- If the EXE data file is broken or corrupt but the CompleteCompatibleFunction block is complete the returned value from the CompleteCompatibleFunction() shall be incomplete. (0b00000000000000000000000000000101)
- If the Signal Configuration data file is missing or broken the returned value from the CompleteCompatibleFunction() shall be incomplete. (0b00000000000000000000000000001001)
- If the bootloader doesn't find correct value or can't read the StartCompleteCompatibleSignature or EndCompleteCompatibleSignature the bootloader shall treat this as incomplete EXE file to further validations and return the value of (0b00000000000000000000000000000101)

4.6.1.2.7.2 Examples of different Compatible functions

4.6.1.2.7.2.1 Bootloader Compatbile check

If the bootloader is performing the compatibility check itself, it could (for example) using some patterns where the locations are defined by the ESS (Compatibility Information).

Example.

- Each logical block contains compatibility Information in the "header/info" part, according to the TIER 1 defined format. This header part must support functional growth, i.e. that the software structure is changed from e.g. three to four logical blocks.
- The dependencies to the application is checked, for each software part.

4.6.1.2.7.2.2 Application Complete & Compatible block

To support use-cases where the compatibility check is highly dependent to the application build process, i.e. where the bootloader does not have access to all required information, a possible realization is put the compatibility check into the application (EXE) and let the bootloader to a call-out to that block. This might be required when the ECU shall support the AUTOSAR post-build loadable concept, where it shall be possible to reconfigure the communication matrix (typically move an existing signal from one frame to another, add a signal gateway, etc.)

Example.
A *Application Complete & Compatible block* is defined and part of the EXE software part. The structure and location of this block is known to the bootloader.

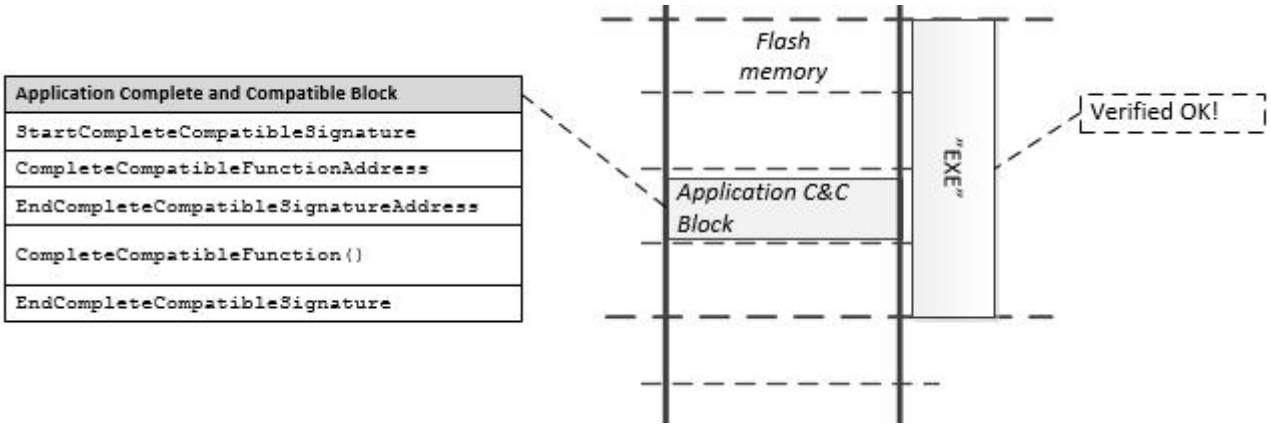
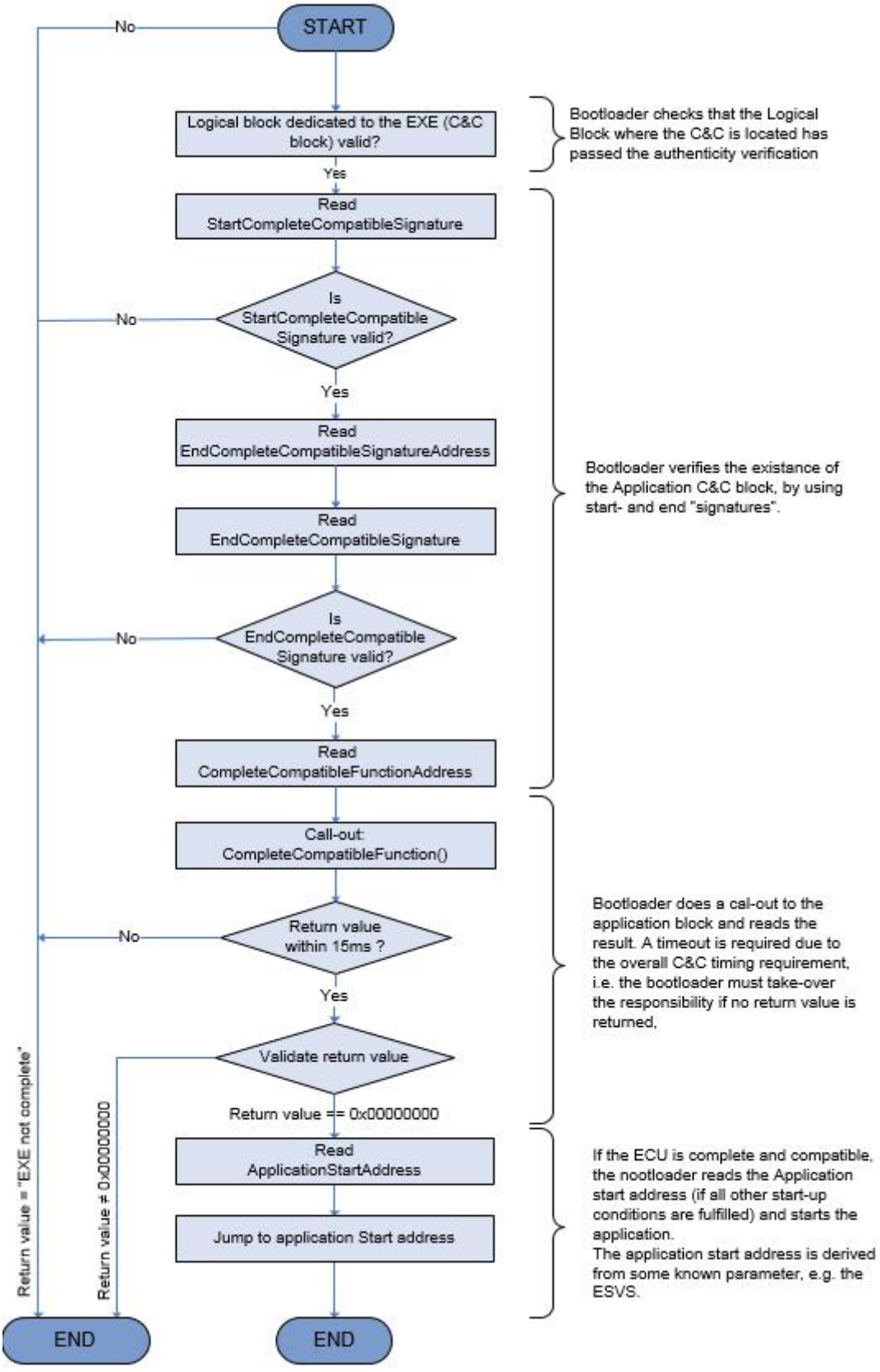


Figure - Application Complete & Compatible block

- StartCompleteCompatibleSignature: To verify the existence (start) of the Application Complete & Compatible block
- CompleteCompatibleFunctionAddress: To define the size and location of the CompeteCompatibleFunction(), i.e. here it is only the compatibility check.
- EndCompleteCompatibleSignatureAdress: To be possible to identify where the EndCompleteCompatibleSignature is located
- Complete&Compatible(): The function invoked from the bootloader, using a call-out.
- EndCompleteCompatibleSignature: To verify the existence (end) of the Application Complete & Compatible block.



Example - Application (EXE) Complete and Compatible block

Note 1: *In case of no response value received, and the a timer has expired, the bootloader*

terminate the execution of the application block and treat the EXE as incomplete during further validations.
Note 2: *The location and size of the Application Complete & Compatible block must be agreed between the bootloader and the application developer.*
Note 3: *No call-out is allowed unless it is proven that the call-out address is located in a valid (verified) location, as shown in "Figure - Application Complete & Compatible block".*

4.6.1.2.8 Writing of received data to non-volatile memory

To decrease the total programming time it is important to start writing data to the non-volatile memory before the complete message is received, decompression included if supported. The positive response shall be sent after all data included in the TransferData request is written to the non-volatile memory.

If the message is interrupted or no response is given to the client the block will be resent. As non-volatile memories normally only allow one write operation for each erase operation the ECU must know how many data bytes it has written to the non-volatile memory. If the client transfers the same block again the ECU shall count the number of received data bytes and start to write data at the first unwritten non-volatile memory position.

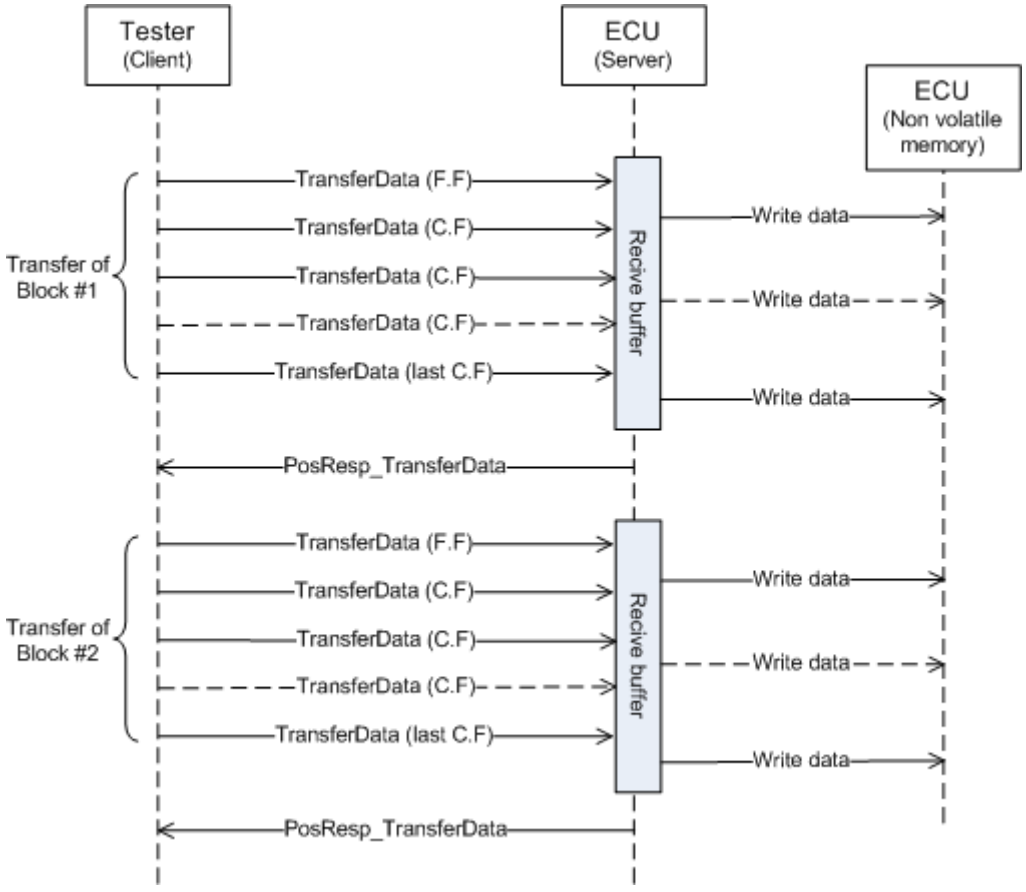


Figure - Writing of data to non-volatile memory

3188 v2	**Writing of received data**

Version history:
(2):

Purpose: To decrease the total programming time the ECU shall start writing data to the non-volatile memory before the complete message is received.

Source:

Verification Method: Analysis

Legacy ID: 53978v1

The ECU shall start writing data to the non-volatile memory before the complete message is received, decompression included if supported. The positive response shall be sent after all data included in the TransferData request is written to the non-volatile memory.

As non-volatile memories normally only allow one write operation for each erase operation the ECU must know how many data bytes it has written to the non-volatile memory. If the client transfers the same block again the ECU shall count the number of received data bytes and start to write data at the first unwritten non-volatile memory position.

4.6.1.2.9 File download order

The data files which are downloadable to the ECU shall be capable of being downloaded to the ECU individually or in any grouping combination independent of order. The only exception to this is that the secondary bootloader shall always be required to be downloaded first when supported. As an example, if the ECU supports a secondary bootloader (SW - A), Application (SW - B), and a Signal configuration (SW - C), then the following download sequences shall be supported:

- Transition to programmingSession, download SW - A then B
- Transition to programmingSession, download SW - A then C
- Transition to programmingSession, download SW - A, B, then C
- Transition to programmingSession, download SW - A, C, then B

3118 v4 **File download order**

Version history:
(4):

Purpose: To be able to download data files to the ECU individually or in any grouping combination independent of order.

Source:

Verification Method: Test

Legacy ID: 53973v3

No restrictions on the download order/sequence of the downloadable data files are allowed, with the exception of the SBL which shall be downloaded first.

It shall be possible to download each data file separately without any requirements of downloading other data files before, exception is the SBL.

Note: *If an ECU Software Structure (ESS) is to be downloaded, it shall always be downloaded next to the SBL. Added here for information, it is a client requirement.*

4.6.1.2.10 Separate memory sectors

To decrease the total programming time it shall be possible to replace only the specific data file

that shall be replaced and not being forced to replace all software(s) that already are present in the ECU. Since the erase of the memory only can be performed for a complete memory sector shall all data files which are possible to download to the ECU not share same memory sector.

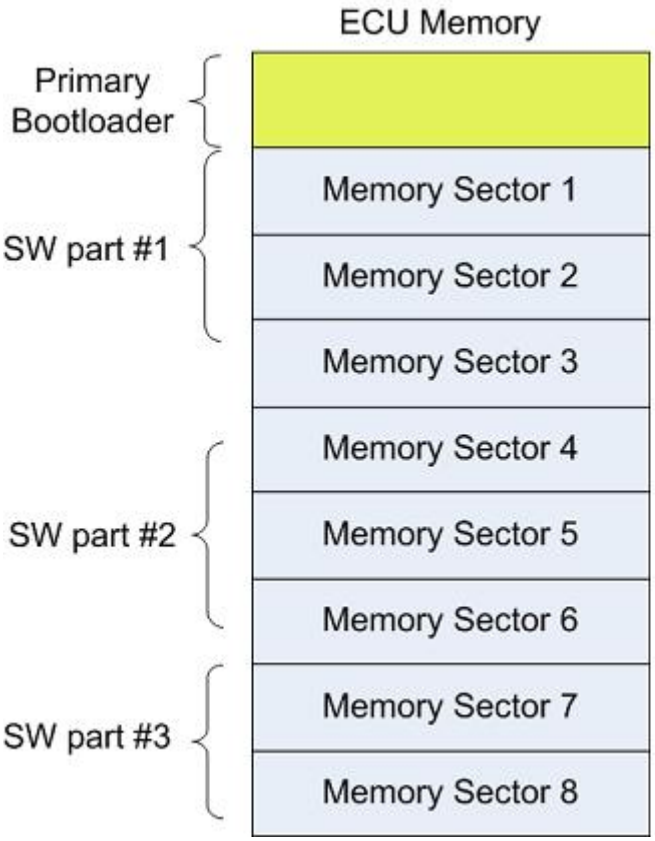


Figure - Separate memory sectors

3123 v2 **Separate memory sectors**

Version history:
(2):

Purpose: To decrease the total programming time it shall be possible to replace only the specific data file that shall be replaced and not being forced to replace all software(s) that already is present in the ECU.

Source:

Verification Method: Test

Legacy ID: 53975v1

Two or more data files shall not share a memory sector.

4.6.1.2.11 Delta encoding

The purpose with Delta encoding is to compress the size of the data to be transmitted to the ECU in order to reduce the time/cost spent during the programming phase. The delta encoding method is to calculate the difference between the new and the old software version, where only the difference between the two versions is transferred to the target ECU.

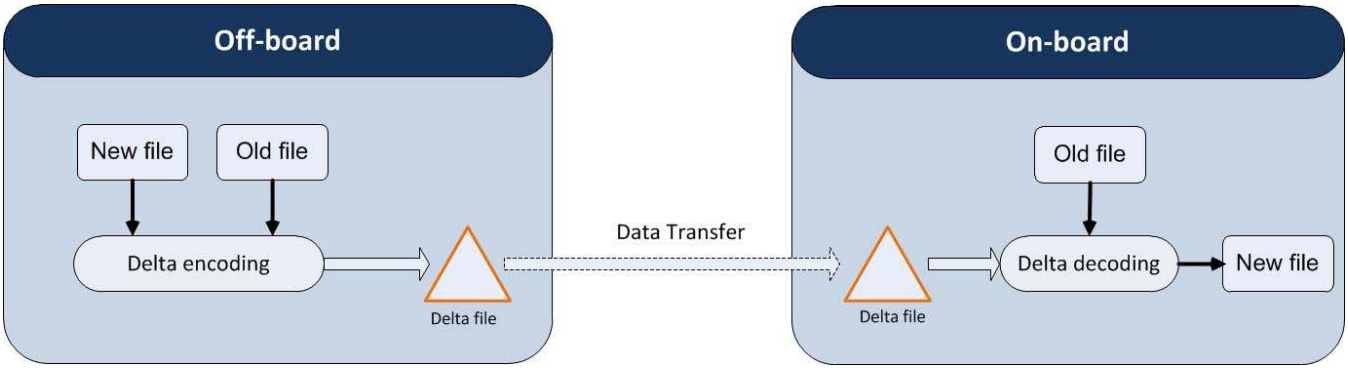


Figure - Principal of delta encoding and decoding

3113 v3 **Delta encoding**

Version history:
(3): Clarification

Purpose: To reduce the time spent on the programming phase and the time spent in transfer over data communication networks.
Source:
Verification Method: Analysis
Legacy ID: 396753v2

Delta encoding and decoding method is required when a data file that is contained within a software delivery exceeds 10MB in size.

4.6.1.2.12 Handling of distribution relays

3119 v5 **Handling of distribution relays**

Version history:
(5):

Purpose: To avoid interruptions on any ongoing communication.
Source:
Verification Method: Test
Legacy ID: 53834v4

An ECU controlling the supply to other ECUs must ensure a stable supply during the transition from the application to the bootloader and vice versa. No glitches or other interruptions are allowed if the supply was enabled in application state. While the bootloader executes, all supplies that feed other ECUs shall be enabled.

4.6.1.2.13 Calibration and customer data

3111 v2 **Calibration and Customer data**

Version history:
(2):

Purpose: To preserve calibration and customer data after the ECU is re-programmed.
Source:
Verification Method: Test
Legacy ID: 53835v1

To avoid re-calibration or re-installation of customer data after ECU programming, the calibration- or customer data shall be preserved. Typical cases are immobilizer codes, mp3 files, navigation waypoints, driver settings, necessary settings in driveline ECU or adaption's etc.

4.6.1.2.14 Internal data transfer speed

3120 v3 **Internal data transfer speed**

Version history:
(3):

Purpose: To improve the total programming time
Source:
Verification Method: Analysis
Legacy ID: 53816v2

To improve the total programming time the internal transfer time between the bus communication controller and the memory (where the data is stored) shall not exceed the memory writing time.

Note: The non-volatile memory itself shall set the dimensioning factor for a SWDL operation.

4.6.1.2.15 Target

Allow any microprocessor based ECU to be configured and fully programmed at any assembly plant or dealership, via approved data links.
The following requirement principles shall be met for software download to an ECU.

3163 v2 **Non-volatile memory for software storage**

Version history:
(2):

Purpose: To be able to modify/upgrade stored software(s)
Source:
Verification Method: Test
Legacy ID: 53791v1

All software(s) shall be stored in a non-volatile memory, SBL excluded.

4.6.1.2.16 Power consumption

3121 v2 **Power Consumption**

Version history:

(2):

Purpose: To minimize the power consumption in programming mode
Source:
Verification Method: Test
Legacy ID: 53802v1

When an ECU executing either the primary or secondary bootloader, all of the ECU's I/O shall be set into a safe and minimize power consumption where components cannot be damaged and the area is safe for people working on the vehicle.

Minimize power consumption means that the ECU only shall supply the necessary components to perform software download.

4.6.1.2.17 Additional bootloader requirements for private ECU

An ECU may not be addressable from a tester point of view (private ECU). If this private ECU have programmable memory areas that supports software download then additional bootloader functionality executing on the public ECU will be required. In these cases the bootloader needs to establish communication with the private ECU for the purposes of software download to the ECU.

This specification does not include any detail for this communication method and any such additional bootloader resource shall be defined by the implementer. However, the following requirements shall be complied.

3106 v2 **Interrupted communication to private ECU**

Version history:
(2):

Purpose: Interrupted communication to the private network ECU shall not prevent subsequent reprogramming
Source:
Verification Method: Test
Legacy ID: 53919v1

The communication method implemented between the public ECU and the private ECU shall be protected such that erasing any or all of the programmable memory areas of the private ECU and removing the ECU power at any time shall not prevent the subsequent reprogramming of any normally programmable memory area in the private ECU. In addition, the complete or partial downloading of any non operational or partly operational software into the programmable memory areas of the private ECU shall not prevent the subsequent reprogramming of any normally programmable memory area.

3107 v2 **Programming sequence for private ECU**

Version history:
(2):

Purpose: To keep the same tester programming sequence as for the public ECU.
Source:

Verification Method: Test
Legacy ID: 53918v1

The programming and erasure of any memory area on a private ECU shall be implemented such that the tester programming sequence have the same format as for a memory area supported on the public connected ECU (i.e., the memory of the private ECU shall appear to be mapped into the memory space of the processor on the public ECU for the purposes of software download).

Note: To keep the same programming sequence shall the private ECU share the same SBL as for the public ECU since only one SBL per ECU address is allowed.

3108 v2 **Read information from a private ECU**

Version history:
(2):

Purpose: To be able to read information from a private ECU.
Source:
Verification Method: Test
Legacy ID: 53921v1

The public ECU is responsible to respond on the behalf of the private ECU regarding software and hardware part numbers as well as if the software stored on the private ECU is complete and compatible.

4.6.1.2.18 Data compression and encryption

To decrease the time to download software to an ECU, compression of the data stream to an ECU shall be used. This is implemented with one algorithm for the compression and one for the decompression of received data. Only the tool that makes the data file and the ECU that is receiving the content of the compressed data file need to implement the compression and decompression algorithms respectively. The ECU shall still support non compressed data files.

The tester informs the ECU whether the data is compressed or not.

3112 v4 **Support for (un)compressed data**

Version history:
(4): Clarification

Purpose: To decrease the time to download data the ECU shall support decompression of the received compressed data.
Source:
Verification Method: Test
Legacy ID: 53923v3

To decrease the software download time of an ECU,compression of data stream to an ECU shall be used, while the ECU shall still support non-compressed data files.The ECU shall implement the decompression algorithm specified in the reference [Data Compression and Encryption].

Only the tool that makes the data file and the ECU that is receiving the content of the compressed data file need to implement the compression and decopression algorithms respectively.

The tester informs the ECU whether the data is compressed or not.

4.6.1.2.19 Additional bootloader requirements for multiple networks

Some ECU hardware architectures may include additional networks which are connected to an ECU. When developing the bootloader shall each requirement for respectively network be considered.

This specification does not include any detail for how to merge the different bootloader implementation requirements and any such additional bootloader functionality shall be defined by the implementer. However, the following requirements shall be complied.

3104 v2 **ECUs with multiple networks**

Version history:
(2):

Purpose: To define the implementation of ECUs with multiple networks.
Source:
Verification Method: Test
Legacy ID: 53913v1

The bootloader shall only support software download from the network which is defined to be the incoming network for diagnostic communication.

3105 v2 **Gateway requirement**

Version history:
(2):

Purpose: To define the requirements with shall apply for an ECU which acts as a diagnostic gateway in the bootloader.
Source:
Verification Method: Test
Legacy ID: 53916v1

An ECU which has more than one network connected that acts as a diagnostic gateway shall implement the requirements defined in *[SWDL_8] Diagnostic and Bootloader Gateway*.

4.6.1.2.20 Two level bootloader

The bootloader is divided into two separate parts; a primary bootloader (PBL) and a secondary bootloader (SBL). The PBL is designed to be permanently placed in a protected flash boot sector, which is activated from reset. The SBL is intended to be downloaded by the PBL into the CPU internal volatile memory, from where it is executed and thereafter deleted after each use. The SBL can be described as a superset of the PBL, adding functions for erase and program

the non-volatile memory.

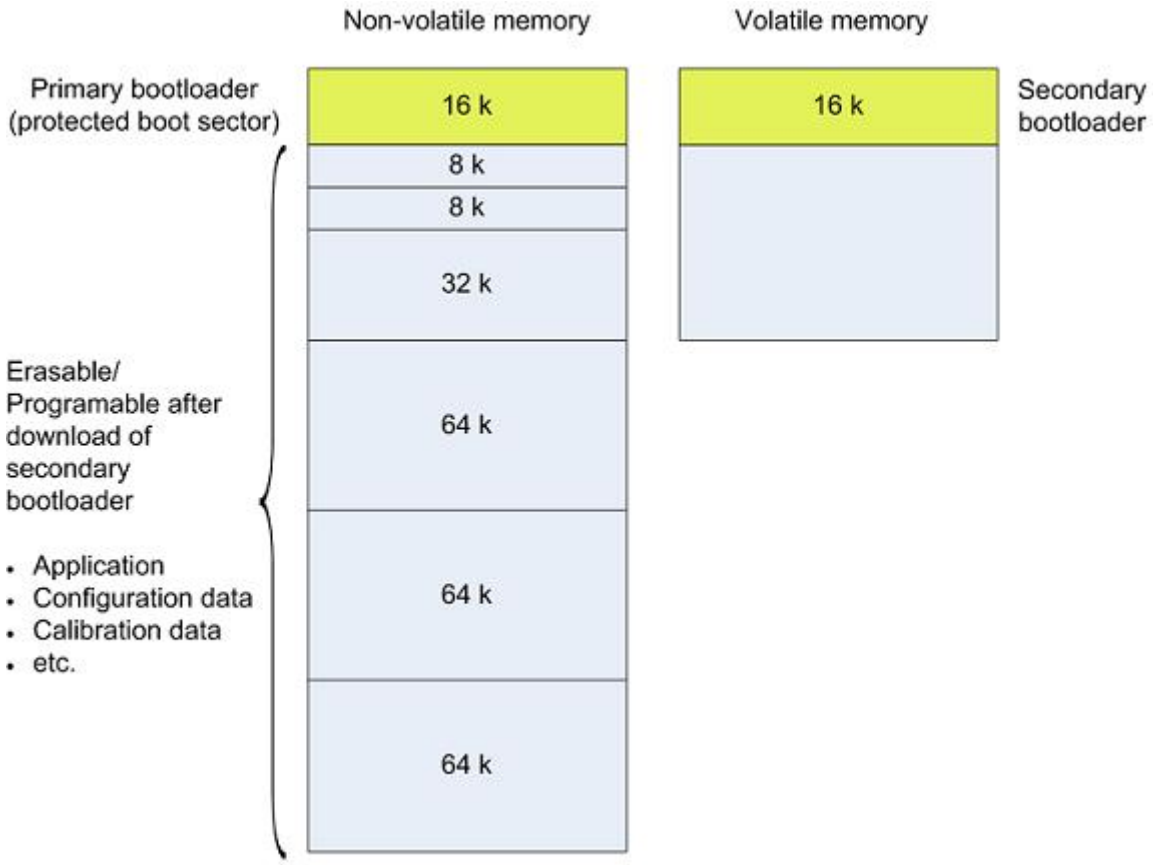


Figure - Typical ECU memory map

3165 v2 **Two level bootloader**

Version history:
(2):

Purpose: For security and data integrity purpose the ECU shall support two levels of bootloaders, PBL and SBL.

Source:

Verification Method: Test

Legacy ID: 53836v1

An ECU shall support the software download concept by the use of two bootloaders; primary- and secondary bootloader.

4.6.1.2.20.1 Primary Bootloader (PBL)

The memory area containing the PBL shall be protected from erasure to eliminate the possibility of accidentally erasing it, but if the processor architecture permits this protection to be configured such that it is removable then the protection shall be set up such that it could be removed if needed. The PBL shall not be able to overwrite or modify any of its own memory and the standard SBL shall not be able to overwrite or erase the PBL.

The PBL shall due to security and data integrity only be capable of writing data to the volatile memory. Erasure and programming to the non-volatile memory shall only be possible when the secondary bootloader is downloaded and activated.

When an ECU executes the bootloader all I/O shall be set into state where components cannot be damaged and the area is safe for people working on the vehicle. The PBL must also enable any I/O that is required to power other ECUs that support software download.

3166 v2 PBL can only write data to volatile memory

Version history:
(2):

Purpose: Due to security and integrity purpose shall the PBL not be able to write or erase data in the non-volatile memory.
Source:
Verification Method: Test
Legacy ID: 53838v1

The primary bootloader shall only be capable of writing data to the volatile memory. Erasure or writing to non-volatile memory is not allowed.

3167 v2 PBL in non-volatile memory

Version history:
(2):

Purpose: The PBL shall always be present in an ECU.
Source:
Verification Method: Test
Legacy ID: 53837v1

The primary bootloader shall be pre-programmed in the non-volatile memory by the ECU manufacturer.

3168 v2 Pre-condition state of the ECUs IO

Version history:
(2):

Purpose: To prevent unwanted states of the ECU's I/O.
Source:
Verification Method: Test
Legacy ID: 53840v1

When an ECU executes the bootloader all I/O shall be set into state where components cannot be damaged and the area is safe for people working on the vehicle. The PBL must also enable any I/O that is required to power other ECUs that support software download.

3169 v2 Protected boot sector

Version history:
(2):

Purpose: To avoid unintentional erase of the PBL.
Source:
Verification Method: Test
Legacy ID: 53839v1

The memory area containing the PBL shall be protected from erasure to eliminate the possibility of unintentional erasure .

Note: The protection shall be set up such that it could be removed if a special SBL were to be written for this purpose, however the standard SBL or the application shall not be able to overwrite or erase the PBL.

4.6.1.2.20.1.1 Primary bootloader diagnostic data

3170 v2 **Primary bootloader diagnostic data**

Version history:
(2):

Purpose: To define the diagnostic data supported in the primary bootloader.
Source:
Verification Method: Test
Legacy ID: 53843v1

The PBL shall support the diagnostic data required in the programmingSession which are specified in *[SWDL_6] UDS Data*.

4.6.1.2.20.1.2 Primary bootloader diagnostic services

3171 v2 **Primary bootloader diagnostic services**

Version history:
(2):

Purpose: To define the diagnostic services supported in the primary bootloader.
Source:
Verification Method: Test
Legacy ID: 53841v1

The PBL shall support the diagnostic services required in the programmingSession which are specified in *[SWDL_5] UDS Services*.

4.6.1.2.20.2 Secondary Bootloader (SBL)

The SBL includes all routines for erase and program of data to the non-volatile memory.

The SBL is downloaded and activated with the help of the PBL into the volatile memory. The SBL diagnostic services are now added (super set to the PBL). This means that all PBL diagnostic services shall be capable of being executed in the SBL, exception is reprogramming and activation of the SBL itself.

After download activation of the SBL in the volatile memory it shall not be possible to make a new download to volatile memory, the ECU shall reject to execute the request with an NRC. Before a new download to the volatile memory can be performed, an exit from the programmingSession is required (see Figure " *Simplified bootloader state diagram"*). The reason for this is to avoid overwriting the SBL.

3172 v4 **Prevent the possibility to download the SBL when already activated**

Version history:
(4):

Purpose: To avoid overwriting the SBL.
Source:
Verification Method: Test
Legacy ID: 53857v3

After download and activation of the SBL in the volatile memory it shall not be possible to make a new SBL download to volatile memory, the ECU shall reject to execute the request with an NRC. Before a new SBL download to the volatile memory can be performed a hard reset of the ECU is required.
Before the SBL activation request is received a new download of the SBL shall be possible to perform.

3173 v2 **SBL in volatile memory**

Version history:
(2):

Purpose: To avoid resident functionality stored in the ECU which has the possibility to erase and write data to the non-volatile memory.
Source:
Verification Method: Test
Legacy ID: 53855v1

The Secondary bootloader shall be stored in the volatile memory.

3174 v2 **Support of erase and write data to non-volatile memory**

Version history:
(2):

Purpose: Only the Secondary Bootloader (SBL) shall be able to erase and write data to the non-volatile memory.
Source:
Verification Method: Test
Legacy ID: 53854v1

The Secondary Bootloader shall upon request erase and write data to the ECU non-volatile memory, except for the non-volatile memory area containing the primary bootloader.

3175 v3 **The SBL shall include all PBL functionality**

Version history:
(3):

Purpose: To simplify the bootloader implementation, to have the possibility to do a workaround if an error has occurred in the PBL and that no calls are necessary from the SBL to the PBL.
Source:
Verification Method: Test
Legacy ID: 53856v2

The SBL shall include the complete PBL functionality.

4.6.1.2.20.2.1 Secondary bootloader diagnostic data

3176 v2 **Secondary bootloader diagnostic data**

Version history:
(2):

Purpose: To define the diagnostic data supported in the secondary bootloader.
Source:
Verification Method: Test
Legacy ID: 53909v1

The SBL shall support the diagnostic data required in programmingSession which are specified in *[SWDL_6] UDS Data*.

4.6.1.2.20.2.2 Secondary bootloader diagnostic services

3177 v2 **Secondary bootloader diagnostic services**

Version history:
(2):

Purpose: To define the diagnostic services supported in the secondary bootloader.
Source:
Verification Method: Test
Legacy ID: 53858v1

The SBL shall support the diagnostic services required in programmingSession which are specified in *[SWDL_5] UDS Services*.

4.6.1.2.21 Additional bootloader requirements for secondary processors

Some ECU hardware architectures may include additional microcontrollers (secondary processors). If these microcontrollers have programmable memory areas additional bootloader functionality executing on the secondary processor is necessary. In these cases the bootloader will need to establish communication with the secondary processor.

This specification does not include any detail for this communication method and any such

additional bootloader resource shall be defined by the implementer. However, the following requirements shall be complied.

3109 v2 **Interrupted communication to the secondary processor**

Version history:
(2):

Purpose: Interrupted communication to the secondary processor shall not prevent subsequent reprogramming.
Source:
Verification Method: Test
Legacy ID: 53912v1

The download communication method implemented on the secondary processor shall be protected such that aborting download or removing ECU power at any time during the download shall not prevent subsequent reprogramming. In addition the complete or partial downloading of any non operational or partly operational software into the secondary processor shall not prevent the subsequent reprogramming of any normally programmable memory area.

3110 v2 **Programming sequence for secondary processors**

Version history:
(2):

Purpose: To keep the same tester programming sequence as for a single processor ECU.
Source:
Verification Method: Test
Legacy ID: 53911v1

The programming and erasure of the additional memory areas shall be implemented such that the tester programming sequence have the same format as for a memory area supported on the main processor (in other words the memory of the secondary processors shall appear to be mapped into the memory space of the main processor for the purposes of software download).

Note: To keep the same programming sequence shall the secondary processors share the same SBL since only one SBL per ECU is allowed.

4.6.2 Data Compression and Encryption

4.6.2.1 Data Compression and Encryption

4.6.2.1.1 Software Component Requirements

4.6.2.1.1.1 Compression Method Selection

4.6.2.1.1.1.1 Method Selection

132335 v1 **Data Compression Method Selection**

Version history:

Purpose: State the compression method selection
Source:
Verification Method: Analysis
Legacy ID: 467271v2

The compression method #2 LZMA shall be used for data compression. The only exception is when it is not possible to implement LZMA in design. For example, if there is not enough RAM (less than 10KB) to support LZMA decompression in ECU, in this case compression method #1 LZSS is used.

4.6.2.1.1.2 Compression Method #1 Requirements

4.6.2.1.1.2.1 Data Compression

3243 v3 **Data compression method 1**

Version history:
(3): Clarification

Purpose: State the compression method
Source: Data Compression
Verification Method: Test
Legacy ID: 52677v2

The data compression method #1 shall be LZSS.

3240 v3 **Data compression Method 1 parameters**

Version history:
(3): Clarification

Purpose: State the compression method #1 parameters
Source: Data Compression
Verification Method: Analysis
Legacy ID: 52683v2

The size of the sliding window in the LZSS compression method shall be 1024 bytes.

3241 v3 **Data compression Method 1 parameters**

Version history:
(3): Clarification

Purpose: State the compression method #1 parameters
Source: Data Compression
Verification Method: Analysis

Legacy ID: 52687v2

The size of the look-ahead buffer in the LZSS compression method shall be 16 bytes.

3242 v3 **Data compression Method 1 parameters**

Version history:
(3): Clarification

Purpose: State the compression parameters
Source: Data Compression
Verification Method: Test
Legacy ID: 52688v1

First bit in a compressed data segment shall indicate if the following data in the segment is an uncompressed byte or a position/length-pair. '1' indicates an uncompressed byte and '0' a position/length-pair.

4.6.2.1.1.3 Compression Method #2 Requirements

4.6.2.1.1.3.1 Data Compression

132947 v1 **Data Compression Method 2**

Version history:

Purpose: State the compression method #2
Source:
Verification Method: Test
Legacy ID: 466659v0

The data compression/decompression method #2 shall be LZMA.
Note:
Data_format_identifier for compression method #2 is specified in *[VCC-UDS Services, REQPROD 459337].*
LZMA SDK (version 18.01) and specification can be downloaded from http://www.7-zip.org/sdk.html

132949 v1 **Data Compression Method 2 Parameters**

Version history:

Purpose: State the compression method #2
Source:
Verification Method: Test
Legacy ID: 466661v0

The data compression ratio varies by using different compression parameters. The most significant change comes from the selection of dictionary size. The general rule is that bigger dictionary can make the compressed data smaller in size. We set three compression levels and each level includes its default parameters. The selection of levels depends on how much RAM memory is available for decompression in the designed ECU. For example, if an ECU has only

about 30KB RAM for decompression, it should take level low: dictionary size 4KB, lc =3, lp =0, and pb=2.

Compression level	Required RAM	dictSize	lc	lp	pb
Low	20KB	4KB (1<<12 bytes)	3	0	2
Medium	144KB	128KB (1<<17)	3	0	2
High	16MB + 16KB	16MB (1<<24)	3	0	2

Table – LZMA Compression Level

Most users just need to choose the suitable compression level with default parameters rather than consider all compression parameters. However, certain advanced users may need freely combine the values of different parameters. It shall be able to select the full value range of all compression parameters, for example, the value of lc shall be freely selected from 0 to 8. The parameters value range can be found in LZMA SDK.

132953 v1 **Data Compression Method 2 Data Format**

Version history:

Purpose: State the compression method #2
Source:
Verification Method:
Legacy ID:

The LZMA SDK defines LZMA compressed data format which includes special LZMA properties, dictionary size, uncompressed size and compressed data. When using LZMA as the method to compress VBF file, each VBF datablock shall be compressed as this data format. Furthermore, the length of uncompressed data shall be saved in uncompressed size, which means uncompressed size cannot be set as unknown, and it is not allowed to add any "end marker" to indicate the end of a datablock.

4.6.2.1.1.4 Compression Method #3 Requirements

4.6.2.1.1.4.1 Data Compression

123905 v2 **Data compression method 3**

Version history:

Purpose: State the compression method
Source:
Verification Method: Test

Legacy ID:

The data compression/decompression method shall be Gzip.

124148 v2 **Data compression Method 3 parameters**

Version history:

Purpose: State the compression parameters
Source:
Verification Method: Analysis
Legacy ID:

gzip_comp_level: gzip compression ratio level, there are totally 9 acceptable values in the range from 1 to 9, 1 means compress fast, 9 means compress best. The default level used in Geely is 9.

4.6.2.1.2 Appendix

4.6.2.1.2.1 Compression Method #1 Examples

4.6.2.1.2.1.1 LZSS Compression C-code Example

This example serves as a LZSS compression implementation. The example code follows the MISRA guidelines and uses size specific data types that have to be defined regarding the used hardware target.

The compression function LZSS_CompressData, read uncompressed data from an input buffer with byte access and writes compressed data to an output buffer with bit access with the following functions:

```
void InputByte(LZSS_OutBuffer_T, Uint8_T byte)
                  Reads one byte from a data input buffer.

void OutputBit(LZSS_OutBuffer_T buffer, Uint8_T bitVal)
                  Writes one bit with value bitValue to a data output buffer

void OutputBits(LZSS_InBuffer_T buffer, Uint32_T dataValue, Uint8_T numberOfBits)
                  Writes numberOfBits with value dataValue to a data output buffer.

/*
*-----------------------------------------------------------------------------
* COPYRIGHT (c) Volvo Cars Corporation 2007.
*
* The copyright of the computer program(s) herein is the property of
* Volvo Cars. The program(s) may be copied and
* used only with the written permission from Volvo Cars.
*
*-----------------------------------------------------------------------------
* IDENTIFICATION
*
* Product Name: General ECU
* Module Abbr : Compression
* File Name   : lzss_comp.c
* Date        : 2007-06-19
```

```
* Revision    : 1.1
*
*-----------------------------------------------------------------------------
* CONTENTS
*
*1      DESCRIPTION
*
*2      DEFINITIONS
*       2.1           Constants
*             2.2     Local functions (forward declarations)
*       2.3           Variables
*       2.4           Macros
*
*       3       LOCAL FUNCTIONS
*               3.1     LZSS_InitTree
*               3.2     LZSS_ContractNode
*               3.3     LZSS_ReplaceNode
*               3.4     LZSS_FindNextNode
*               3.5     LZSS_DeleteNode
*               3.6     LZSS_AddNode
*
*4      GLOBAL FUNCTIONS
*       4.1     LZSS_CompressData
*
*-----------------------------------------------------------------------------
*/
/*
*-----------------------------------------------------------------------------
*1      DESCRIPTION
*-----------------------------------------------------------------------------
*
* This file implements the LZSS compression used for the converter tool.
* LZSS is a dictionary compression method and uses a sliding window as
* dictionary. Compression is achieved by replacing byte arrays found in
* previously read data with position/length-pairs pointing in to the sliding
* window. If the position/length-pair takes up more bits than the byte array
* that it tries to compress, the data is left uncompressed. This is indicated
* with a single bit flag, '1' indicates that data to follow is uncompressed and
* '0' indicates that data to follow is a position/length-pair.
* To speed up the compression algorithm a binary search tree is used to store
* previously compressed data.
*
* Code in this module is based on the code examples in the book "The Data
* Compression Book" by Mark Nelson, ISBN 1-55851-434-1
*/
#include <bitio.h>    /* Data IO functions */
#include <aptypes.h>  /* MISRA type definitions */
#include <lzss_comp.h>

/*
*-----------------------------------------------------------------------------
*2      DEFINITIONS
*-----------------------------------------------------------------------------
*/

/*
*-----------------------------------------------------------------------------
*2.1 Constants
*-----------------------------------------------------------------------------
*/

/*
```

```
* Number of bits allocated to indices into the text window
*/
#define LZSS_INDEX_BIT_COUNT      (Uint8_T)10

/**
* @brief Number of bits allocated for the length of an encode phrase
**/
#define LZSS_LENGTH_BIT_COUNT     (Uint8_T)4

/**
* @brief Size of the sliding window
**/
#define LZSS_WINDOW_SIZE          (Uint16_T)( 1 << LZSS_INDEX_BIT_COUNT )

/*
* Number of bytes to encode a byte array.
* Used to calculate if compression should be done or not
*/
#define LZSS_BREAK_EVEN           (Uint8_T)( ( 1 + LZSS_INDEX_BIT_COUNT + \
                                               LZSS_LENGTH_BIT_COUNT ) / 9 )

/*
* End of stream indicator
*/
#define LZSS_END_OF_STREAM        (Uint8_T)0

/*
* Size of the look-ahead buffer
*/
#define LZSS_RAW_LOOK_AHEAD_SIZE  (Uint8_T)( 1 << LZSS_LENGTH_BIT_COUNT )

/*
* The real look-ahead size, e.g. the maximum number of bytes to match
*/
#define LZSS_LOOK_AHEAD_SIZE      (Uint8_T)( LZSS_RAW_LOOK_AHEAD_SIZE + \
                                               LZSS_BREAK_EVEN )

/*
* Value of the tree root
*/
#define LZSS_TREE_ROOT            LZSS_WINDOW_SIZE

/*
* Null index for the tree structure
*/
#define UNUSED                    (Uint8_T)0

/*
*-----------------------------------------------------------------------------
*    2.2 Local functions (forward declarations)
*-----------------------------------------------------------------------------
*/

void LZSS_InitTree( Uint16_T r );

void LZSS_ContractNode( Uint16_T old_node, Uint16_T new_node );

void LZSS_ReplaceNode( Uint16_T old_node, Uint16_T new_node );

Uint16_T LZSS_FindNextNode( Uint16_T node );
```

```
void LZSS_DeleteNode( Uint16_T p );

Uint8_T LZSS_AddNode( Uint16_T new_node, Uint16_T *match_position );


/*
*-------------------------------------------------------------------------------
*2.3    Variables
*-------------------------------------------------------------------------------
*/

/*
* The sliding window used by the decompression algorithm
*/
Uint8_T LZSS_window[ LZSS_WINDOW_SIZE ];

/*
* Binary tree of all of the byte arrays in the window.
*/
struct {
    Uint16_T parent;
    Uint16_T smallChild;
    Uint16_T largeChild;
} tree[ LZSS_WINDOW_SIZE + 1 ];


/*
*-------------------------------------------------------------------------------
*2.4    Macros
*-------------------------------------------------------------------------------
*/

/*
* Arithmetic modulo to get the correct index in the sliding window
*/
#define LZSS_MOD_WINDOW( a )           (Uint16_T)( ( a ) & \
                                                   ( LZSS_WINDOW_SIZE - 1 ) )


/*
*-------------------------------------------------------------------------------
*3      LOCAL FUNCTIONS
*-------------------------------------------------------------------------------
*/

/*
*-------------------------------------------------------------------------------
*   3.1 LZSS_InitTree
*-------------------------------------------------------------------------------
*
* LZSS Initialize tree
*
* Initialize the binary tree used by the compression to
* keep track of previously compressed byte arrays
*
* param rootChild     Root node to the tree
*
* return Void
*
*/
void LZSS_InitTree( Uint16_T rootChild )
{
```

```
/* Add a root child to initialize the tree */
    tree[ LZSS_TREE_ROOT ].parent = UNUSED;
    tree[ LZSS_TREE_ROOT ].smallChild = UNUSED;
    tree[ LZSS_TREE_ROOT ].largeChild = rootChild;
    /* Set child parent to the tree root */
    tree[ rootChild ].parent = LZSS_TREE_ROOT;
    tree[ rootChild ].largeChild = UNUSED;
    tree[ rootChild ].smallChild = UNUSED;
}

/*
*-------------------------------------------------------------------------------
*   3.2 LZSS_ContractNode
*-------------------------------------------------------------------------------
*
* LZSS contract node
*
* Replace a node with another node already in the tree. This function assumes
* that newNode is a child to the old node.
*
* param oldNode       Node to remove
* param newNode       Node to move to the oldNode position
*
* return Void
*
*/
void LZSS_ContractNode( Uint16_T oldNode, Uint16_T newNode )
{
/* Move the parent node to the new node */
    tree[ newNode ].parent = tree[ oldNode ].parent;
    /* Change the parent on the child node */
    if ( tree[ tree[ oldNode ].parent ].largeChild == oldNode )
    {
        tree[ tree[ oldNode ].parent ].largeChild = newNode;
    }
    else
    {
        tree[ tree[ oldNode ].parent ].smallChild = newNode;
    }
    /* Detach the old node */
    tree[ oldNode ].parent = UNUSED;
    tree[ oldNode ].smallChild = UNUSED;
    tree[ oldNode ].largeChild = UNUSED;
}

/*
*-------------------------------------------------------------------------------
*   3.3 LZSS_ReplaceNode
*-------------------------------------------------------------------------------
*
* LZSS replace node
*
* Replace a node with a new node that was not previously in the tree
*
* param oldNode       Node to remove
* param newNode       New node not previously in the tree
*
* return Void
*
*/
void LZSS_ReplaceNode( Uint16_T oldNode, Uint16_T newNode )
{
```

```
    /* If the old node is the smallChild*/
    if ( tree[ tree[ oldNode ].parent ].smallChild == oldNode )
    {
        /* Set the new node as smallChild*/
        tree[ tree[ oldNode ].parent ].smallChild = newNode;
    }
    else
    {
        /* Else set the new node as largeChild*/
        tree[ tree[ oldNode ].parent ].largeChild = newNode;
    }
    /* Insert the old node at the new node index in the tree array*/
    tree[ newNode ] = tree[ oldNode ];
    /* Set the small child parent to the new node*/
    tree[ tree[ newNode ].smallChild ].parent = newNode;
    /* Set the large child parent to the new node*/
    tree[ tree[ newNode ].largeChild ].parent = newNode;
    /* Detach the old node */
    tree[ oldNode ].parent = UNUSED;
    tree[ oldNode ].smallChild = UNUSED;
    tree[ oldNode ].largeChild = UNUSED;
}

/*
*-----------------------------------------------------------------------------
*   3.4 LZSS_FindNextNode
*-----------------------------------------------------------------------------
*
* LZSS find node
*
* Find the largest node under the argument node. Used when deleting
* a node. This routine assumes that the argument node has a smaller child.
*
* param node   Node to search from
*
* return largest node under the argument node
*
*/
Uint16_T LZSS_FindNextNode( Uint16_T node )
{
    Uint16_T next = UNUSED;

/* Start at the small child of the node*/
    next = tree[ node ].smallChild;
    /* And find the largest node under the small child*/
    while ( tree[ next ].largeChild != UNUSED )
    {
        next = tree[ next ].largeChild;
    }
    return( next );
}

/*
*-----------------------------------------------------------------------------
*   3.5 LZSS_DeleteNode
*-----------------------------------------------------------------------------
*
* LZSS delete node
*
* Deletes a node from the binary tree.
*
* param node   Node to delete
```

```
*
* return void
*
*/
void LZSS_DeleteNode(Uint16_T node)
{
Uint16_T replNode = 0;

/* If the large child is empty, just move up the small child*/
if(tree[node].largeChild == UNUSED)
{
        LZSS_ContractNode( node, tree[ node ].smallChild );
    }
    /* If the small child is empty, just move up the large child */
    elseif ( tree[ node ].smallChild == UNUSED )
    {
        LZSS_ContractNode( node, tree[ node ].largeChild );
    }
    /* Otherwise, replace the node to be deleted with the largest node on
     * the small child side of the tree */
    else
    {
        /* Find replacment node.
         * LZSS_FindNextNode will always get the largest node on the small side
          * and the found node will not have any larger child */
        replNode = LZSS_FindNextNode(node);

        /* Contract the replacement node with its small child */
        LZSS_ContractNode(replNode, tree[replNode].smallChild);

        /* Replace the node to delete with the found node */
        LZSS_ReplaceNode(node, replNode);
    }
}

/*
*-----------------------------------------------------------------------------
*   3.6 LZSS_AddNode
*-----------------------------------------------------------------------------
*
* LZSS add node
*
* In this routine most of the encoder work is done. The routine adds
* a new node to the binary tree. It finds the best match among all existing
* nodes already in the tree and returns the length and the position of the match
* If the new_node has a duplicate in the tree, the old_node is deleted, for
* reasons of efficiency.
*
* param newNode       Node to add to the binary tree (pointer to the byte array)
* param matchPos      Return pointer to the node position in the window
*
* return length of the match
*
*/
Uint8_T LZSS_AddNode( Uint16_T newNode, Uint16_T *matchPos )
{
    Uint8_T i = 0;
    Uint16_T testNode = 0;
    Int16_T delta = 0;
    Uint8_T matchLen = 0;
    Uint16_T *child;
    BOOL_T matchDone = FALSE;
```

```
    Uint8_T retVal = 0;

/* If the new byte is END_OF_STREAM, exit */
    if ( newNode == LZSS_END_OF_STREAM )
    {
        return( 0 );
    }
    /* Get the first node to test with */
    testNode = tree[ LZSS_TREE_ROOT ].largeChild;
    /* Set the length of the match to 0 */
    matchLen = 0;

    while ( !matchDone )
    {
        i = 0;
        delta = 0;
        /* Look for matches in the tree. Test current test node with the new
         * node until they differ */
        while((i < LZSS_LOOK_AHEAD_SIZE) && (delta == 0))
        {
            delta = LZSS_window[ LZSS_MOD_WINDOW( newNode + (Uint16_T)i ) ] -
                    LZSS_window[ LZSS_MOD_WINDOW( testNode + (Uint16_T)i ) ];

        i++;
        }
        /* If the nodes didn't match every LZSS_LOOK_AHEAD_SIZE bytes
         * decrease i to get the current match length */
        if(delta != 0)
        {
        i--;
        }
        /* If i larger than previous match in the tree */
        if ( i >= matchLen )
        {
        /* Set new match length to i */
            matchLen = i;
            /* Set match position to the test node position */
            *matchPos = testNode;
            /* If the byte array in the look-ahead buffer is an exact match of
             * the test node, replace the test node with the new node to remove
             * redundant data */
            if ( matchLen >= LZSS_LOOK_AHEAD_SIZE )
            {
                LZSS_ReplaceNode( testNode, newNode );
                /* Search finished */
                retVal = matchLen;
                matchDone = TRUE;
            }
        }
        if(!matchDone)
       {
        /* If the new node is larger than the test node */
        if ( delta >= 0 )
        {
              /* Continue to test with the larger child */
            child = &tree[ testNode ].largeChild;
        }
        /* If new node is smaller than or equal to the test node */
        else
        {
              /* Continue to test with smaller child */
            child = &tree[ testNode ].smallChild;
```

```
        }
        /* If the new test node is a leaf */
        if ( *child == UNUSED )
        {
              /* Add the new node to the tree */
            *child = newNode;
            tree[ newNode ].parent = testNode;
            tree[ newNode ].largeChild = UNUSED;
            tree[ newNode ].smallChild = UNUSED;
            /* Search finished */
            retVal = matchLen;
            matchDone = TRUE;
        }
        /* Continue to test the child*/
        testNode = *child;
       }
    }
    return retVal;
}

/*
*-----------------------------------------------------------------------------
*4      GLOBAL FUNCTIONS
*-----------------------------------------------------------------------------
*/

/*
*-----------------------------------------------------------------------------
*   4.1 LZSS_CompressData
*-----------------------------------------------------------------------------
*
* LZSS compress data
*
* This routine compress data from an in buffer using the LZSS algorithm. LZSS is
* a dictionary compression method where the dictionary is a sliding window. The
* routine uses a look-ahead buffer and finds matches in previously read bytes
* stored in a sliding window.
* If a match is found an index/length pair is written to the output buffer
* (index is the position in the sliding window). If no match is found the read
* byte is just bypassed to the out buffer. To indicate if the data is an
* index/length pair or a plain text byte a single bit is used as indicator flag
* and is written to the out buffer.
* To speed up the compression a binary tree is used for storing previously
* compressed data.

*
* param inBuf  Input data buffer with byte access
* param outBufoutput data buffer with bit access
*
* return void
*
*/
void LZSS_CompressData(LZSS_InputBuffer_T *inBuf, LZSS_OutputBuffer_T *outBuf )
{
/** Index var. for loop */
    Uint8_T    i             = 0;
    /** Byte read from input stream */
    Uint16_T   inByte               = 0;
    /** Number of bytes in the look head buffer */
    Uint8_T    aheadBytes    = 0;
    /** Current position in the window */
    Uint16_T   winPos               = 1;
```

```
    /** Number of bytes to replace in the window */
    Uint8_T    replCnt       = 0;
    /** Length of the data match found in the window */
    Uint8_T    matchLen      = 0;
    /** Position in the window of the data match */
    Uint16_T   matchPos      = 0;
    /** Indicator of End Of Stream reached */
    BOOL_T            eosReached    = FALSE;

    /* Start with filling up the look-ahead buffer */
        while( (aheadBytes< LZSS_LOOK_AHEAD_SIZE) && (!eosReached))
          {
             /* Get next input byte */
             inByte = InputByte( inBuf );
             /* If input stream is finished, exit */
             if ( inByte == LZSS_END_OF_INPUT_STREAM )
             {
                eosReached = TRUE;
             }
             else
             {
                /* Add byte to wondow */
                LZSS_window[ winPos + (Uint16_T)aheadBytes ] = (Uint8_T) inByte;
                /* Increase look-ahead bytes*/
                aheadBytes++;
             }

          }
    /* Initialize the tree */
    LZSS_InitTree( winPos );
    /* While there still are bytes in the look ahead buffer, loop */
    while ( aheadBytes > 0 )
    {
        /* If previously match length greater than look ahead bytes it's
         * not possible to code correctly */
        if ( matchLen > aheadBytes )
        {
        /* Set matched length to look-ahead buffer length */
            matchLen = aheadBytes;
        }
        /* If the match is smaller than the compressed data (position/length-
         * pair) there will be negative compression so just output the byte */
        if ( matchLen <= LZSS_BREAK_EVEN )
        {
            /* Set consumed bytes in input stream to 1 */
            replCnt = 1;
            /* Indicate that next byte in output stream is uncompressed
             * by output a '1' */
            OutputBit( outBuf, (Uint8_T)1 );
            /* Output uncompressed byte */
            OutputBits( outBuf,
                        (Uint32_T) LZSS_window[ winPos ], (Uint8_T)8 );
        }
        /* The match is larger than a position/length pair, compression can be
         * done */
        else
        {
        /* Indicate that the following bits are compressed data by output a
             * '0' */
            OutputBit( outBuf, (Uint8_T)0 );
            /* Output position in the window */
            OutputBits( outBuf,
```

```
                        (Uint32_T) matchPos, LZSS_INDEX_BIT_COUNT );
            /* Output the length of the match, (length is the number of bytes
             * that is greater than LZSS_BREAK_EVEN) */
            OutputBits( outBuf,
                        (Uint32_T) ( matchLen - ( LZSS_BREAK_EVEN + 1 ) ),
                        LZSS_LENGTH_BIT_COUNT );
            /* Set consumed bytes in input stream to the length of the match
*/
            replCnt = matchLen;
        }
        /* Delete consumed bytes and add new bytes in the window */
        for ( i = 0 ; i < replCnt ; i++ )
        {
        /* Remove consumed byte from the window */
            LZSS_DeleteNode( LZSS_MOD_WINDOW( winPos +
                                (Uint16_T)LZSS_LOOK_AHEAD_SIZE ) );
            /* Get next input byte */
            inByte = InputByte( inBuf );
            /* If input stream is finished */
            if ( inByte == LZSS_END_OF_INPUT_STREAM )
            {
               /* Decrease the look-ahead bytes */
                aheadBytes--;
            }
            else
            {
               /* Add the new byte from the input stream to the window */
                LZSS_window[ LZSS_MOD_WINDOW( winPos +
                           (Uint16_T)LZSS_LOOK_AHEAD_SIZE ) ]= (Uint8_T) inByte;
            }
            /* Increase the position in the window */
            winPos = LZSS_MOD_WINDOW( winPos + (Uint16_T)1 );
            /* If there still are bytes in the look-ahead buffer */
            if ( aheadBytes != 0 )
            {
               /* Add byte position to the tree and get the length of the
                 * match */
                matchLen = LZSS_AddNode( winPos, &matchPos );
            }
        }
    }
    /* Input stream finished, write end of stream to the output buffer
     * uncompressed */
    OutputBit( outBuf, (Uint8_T)0 );
    OutputBits( outBuf, (Uint32_T) LZSS_END_OF_STREAM, LZSS_INDEX_BIT_COUNT );
}


/*
*-----------------------------------------------------------------------------
* End of file
*-----------------------------------------------------------------------------
*/
```

4.6.2.1.2.1.2 LZSS Decompression C-code Example

This example serves as a LZSS decompression implementation. The example code follows the MISRA guidelines and uses size specific data types that have to be defined regarding the used hardware target.

The decompression function LZSS_ExpandData , read compressed data from an input buffer

with bit access and writes decompressed data to an output buffer with byte access with the following functions:

```
Uint8_T InputBit(LZSS_InBuffer_T buffer)
                  Read one bit from a data input buffer.

Uint32_T InputBits(LZSS_InBuffer_T buffer, Uint8_T numberOfBits)
                  Read numberOfBits from a data input buffer.

void OutputByte(LZSS_OutBuffer_T, Uint8_T byte)
                  Writes one byte to a data output buffer.


/*
*-----------------------------------------------------------------------------
* COPYRIGHT (c) Volvo Cars Corporation  2007.
*
* The copyright of the computer program(s) herein is the property of
* Volvo Cars. The program(s) may be copied and
* used only with the written permission from Volvo Cars.
*
*-----------------------------------------------------------------------------
* IDENTIFICATION
*
* Product Name: General ECU
* Module Abbr : Bootloader Decompression
* File Name   : lzss_decomp.c
* Date        : 2007-06-14
* Revision    : 1.0
*
*-----------------------------------------------------------------------------
* CONTENTS
*
*1      DESCRIPTION
*
*2      DEFINITIONS
*       2.1           Constants
*       2.2           Variables
*       2.3           Macros
*
*3      GLOBAL FUNCTIONS
*       3.1 LZSS_ExpandData
*
*-----------------------------------------------------------------------------
*/

/*
*-----------------------------------------------------------------------------
*1      DESCRIPTION
*-----------------------------------------------------------------------------
*
* This file implements the LZSS decompression used in ECU bootloaders.
* LZSS is a dictionary compression method and uses a sliding window as
* dictionary. Compression is achieved by replacing byte arrays found in
* previously read data with position/length-pairs pointing in to the sliding
* window. If the position/length-pair takes up more bits than the byte array
* that it tries to compress, the data is left uncompressed. This is indicated
* with a single bit flag, '1' indicates that data to follow is uncompressed and
* '0' indicates that data to follow is a position/length-pair.
*
* Code in this module is based on the code examples in the book "The Data
```

```
* Compression Book" by Mark Nelson, ISBN 1-55851-434-1
*
*/
#include <bitio.h>           /* Data IO functions */
#include <aptypes.h>         /* MISRA type definitions */
#include <lzss_decomp.h>

/*
*-------------------------------------------------------------------------------
*2      DEFINITIONS
*-------------------------------------------------------------------------------
*/

/*
*-------------------------------------------------------------------------------
*2.1 Constants
*-------------------------------------------------------------------------------
*/

/*
* Number of bits allocated to indices into the text window
*/
#define LZSS_INDEX_BIT_COUNT      (Uint8_T)10

/*
* Number of bits allocated for the length of an encode phrase
*/
#define LZSS_LENGTH_BIT_COUNT     (Uint8_T)4

/*
* Size of the sliding window
*/
#define LZSS_WINDOW_SIZE          (Uint16_T)( 1 << LZSS_INDEX_BIT_COUNT )

/*
* Number of bytes to encode a byte array.
* Used to calculate if compression should be done or not
*/
#define LZSS_BREAK_EVEN           (Uint8_T)( ( 1 + LZSS_INDEX_BIT_COUNT + \
                                               LZSS_LENGTH_BIT_COUNT ) / 9 )

/*
* End of stream indicator
*/
#define LZSS_END_OF_STREAM        (Uint8_T)0

/*
*-------------------------------------------------------------------------------
*2.2 Variables
*-------------------------------------------------------------------------------
*/

/*
* The sliding window used by the decompression algorithm
*/
Uint8_T LZSS_window[ LZSS_WINDOW_SIZE ];

/*
*-------------------------------------------------------------------------------
*2.3 Macros
*-------------------------------------------------------------------------------
*/
```

```
/*
* Arithmetic modulo to get the correct index in the sliding window
*/
#define LZSS_MOD_WINDOW( a )          (Uint16_T)( ( a ) & \
                                                  ( LZSS_WINDOW_SIZE - 1 ) )

/*
*-------------------------------------------------------------------------------
*3      GLOBAL FUNCTIONS
*-------------------------------------------------------------------------------
*/


/*
*-------------------------------------------------------------------------------
*   3.1 LZSS_ExpandData
*-------------------------------------------------------------------------------
*
* LZSS expand data stream
*
* This routine expands data from an in buffer with the LZSS algorithm.
* The routine read in flag bits to decide whether to read in an uncompressed
* character or an index/length pair and expands the data stream to an out
* buffer.
*
* param inBuf     Input data buffer with bit access
* param outBuf    Output data buffer with byte access
*
* return Void
*
*/
void LZSS_ExpandData(LZSS_InputBuffer_T *inBuf, LZSS_OutputBuffer_T *outBuf)
{
    /** Index var. for loop */
    Uint8_T i = 0;
    /** Current window position */
    Uint16_T winPos = 1;
    /** Byte to write to output buffer */
    Uint8_T outByte = 0;
    /** Length of the data match found */
    Uint8_T matchLen = 0;
    /** Position in the window of the data match */
    Uint16_T matchPos = 0;
    /** Indicator of End Of Stream reached */
    BOOL_T     eosReached = FALSE;

    while ( !eosReached )
    {
        /* If next bit is 1, next byte is uncompressed*/
        if ( InputBit( inBuf ) == 1)
        {
            /* Get uncompressed byte */
            outByte = (Uint8_T) InputBits( inBuf, (Uint8_T)8 );
            /* Output byte*/
            OutputByte(outByte, outBuf);
            /* Add byte in window */
            LZSS_window[ winPos ] = outByte;
            /* Increase window position */
            winPos = LZSS_MOD_WINDOW( winPos + 1 );
        }
        /* If next bit is 0, compressed data follows */
```

```
        else
        {
        /* Get compressed data as window position of match*/
            matchPos = (Uint16_T) InputBits( inBuf, LZSS_INDEX_BIT_COUNT );
            /* If end of stream, exit */
            if ( matchPos == LZSS_END_OF_STREAM )
            {
                eosReached = TRUE;
            }
            else
            {
            /* Get length of string match */
            matchLen = (Uint8_T) InputBits( inBuf, LZSS_LENGTH_BIT_COUNT );
            /* Add break even + 1 to get the correct length. Length zero and
                 * the break even value are subtracted from the length during
                 * compression to save space. */
            matchLen = matchLen + (LZSS_BREAK_EVEN +1);
            /* For every byte in match */
            for ( i = 0 ; i < matchLen ; i++ )
            {
              /* Get matching byte from window */
                outByte = LZSS_window[ LZSS_MOD_WINDOW( matchPos + i ) ];
                /* Output byte */
                OutputByte(outByte, outBuf);
                /* Add matched byte to current window position */
                LZSS_window[ winPos ] = outByte;
                /* Increase window position */
                winPos = LZSS_MOD_WINDOW( winPos + 1 );
            }
            }
        }
    }
}


/*
*-----------------------------------------------------------------------------
* End of file
*-----------------------------------------------------------------------------
*/
```

4.6.2.1.2.1.3 Implementation Advice Regarding Decompression

The decompression algorithm shall be implemented by each ECU specified (elsewhere) to have this capability. The compressed data to be transferred to an ECU may be divided into several data blocks, each having a well defined start address and length (or stop address) and may further generate one or several transferData messages.

The decompression algorithm consumes data bit by bit via the InputBit and InputBits functions (operating on LZSS_InBuffer_T), that in turn should by-pass received data (bits) to the decompression algorithm. Examples of these functions (and buffer LZSS_InBuffer_T) are not provided here, it is up to each supplier to implement these.

Implementations of both the decompression algorithm along with bit buffers have demonstrated a problem, worthwhile addressing for others to avoid, which occurs between two dataTransfer messages.
Figure: A bit stream, below illustrates two dataTransfer messages, and the containing bytes and bits.

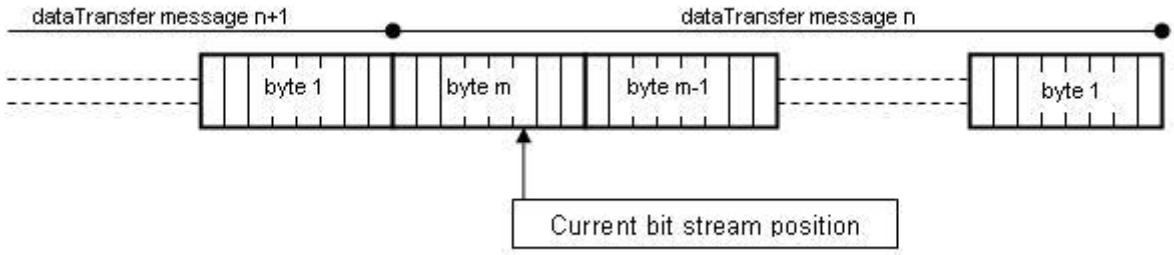


Figure: A bit stream

Suppose that each received dataTransfer message is stored in a buffer, on which the functions InputBit and InputBits have bit access to support the decompression algorithm.

If the capacity of the buffer is equal to the maximum size of a dataTransfer message, then a problem will occur when the current bit position is near the end of the buffer and the decompression algorithm requires more bits than currently stored – bits that are available in the next dataTransfer message.

Care must therefore be taken when consuming bits that may be part of two consecutive dataTransfer messages, illustrated by the Figure: The bit buffer problem illustrated, below. A solution to this problem is not presented here but this information is hopefully of use when designing the bit buffer.

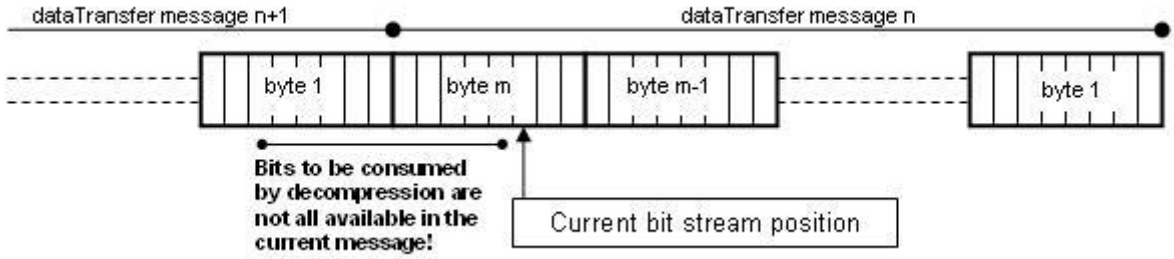


Figure: The bit buffer problem illustrated

4.6.2.1.3 Data Encryption

Not applicable in this document issue.

4.6.2.1.4 Overview

Vehicle software is increasing in size. To be able to reduce the time to download software into a vehicle the data stream transfer must be reduced by means of a compression method. Two compression methods are approved by The OEM: method #1 LZSS and method #2 LZMA. LZMA shall be used, unless it is not possible to implement LZMA in design.

This document states the requirements for data compression/decompression used in ECU software download.

4.6.2.1.4.1 Definitions and Abbreviations

Abbreviations	Stands for	Description
ECU	Electronic Control Unit.	In vehicle electronic control unit with a computer/microcontroller.
LZSS	Lempel Ziv Storer Szymanski	Compression method #1.
LZMA	Lempel–Ziv–Markov chain Algorithm	Compression method #2.
VBF	Versatile Binary Format	A data file format used at the OEM.

4.6.2.1.4.2 External Publications

[1] "The Data Compression Book" by Mark Nelson, ISBN 1-55851-434-1.
[2] LZMA Specification, can be downloaded from http://www.7-zip.org/sdk.html

4.6.2.1.5 Software Compression

4.6.2.1.5.1 Compression Method #1

Software compression is used to minimize the size of the actual data to transmit to in-vehicle ECUs during software download. The compression is performed outside the vehicle before the data is transmitted and the software decompression unpacks the received data in the vehicle ECUs during software download.

Note: Compression method #1 is the default algorithm for compression and decompression.

4.6.2.1.5.1.1 Data Compression

The compression method for compressing the data #1 shall be **Lempel-Ziv-Storer-Szymanski** (**LZSS**) which is a lossless data compression method, a derivative of LZ77 (Abraham Lempel and Jacob Ziv, 1977) that was created in 1982 by James Storer and Thomas Szymanski.

LZSS is a dictionary encoding technique. Unlike Huffman coding, which attempts to reduce the average amount of bits required to represent a symbol, LZSS attempts to replace a string of symbols with a reference to a dictionary location of the same string.

The dictionary in LZSS is a sliding window. Compression is achieved by replacing byte arrays found in previously read data with position/length-pairs pointing in to the sliding window. If the position/length-pair takes up more bits than the byte array that it tries to compress, the data is left uncompressed. This is indicated with a single bit flag, '1' indicates that data to follow is uncompressed and '0' indicates that data to follow is a position/length-pair. A flag bit and an uncompressed byte or a position/length-pair are in this document called a compressed data segment.

To speed up the compression algorithm a binary search tree is used to store previously compressed data.

Appendix: Compression Method #1 Examples, contains C-code examples implementing LZSS compression and decompression.

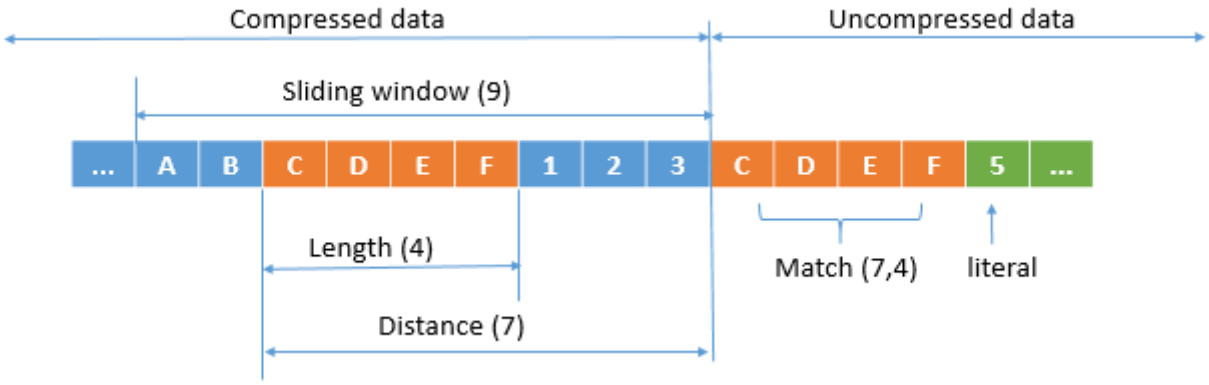
4.6.2.1.5.2 Compression Method #2

Software compression is used to minimize the size of the actual data to transmit to in-vehicle ECUs during software download. The compression is performed outside the vehicle before the data is transmitted and the software decompression unpacks the received data in the vehicle ECUs during software download.

4.6.2.1.5.2.1 Data Compression

The Lempel–Ziv–Markov chain Algorithm (LZMA) is a lossless data compression method, developed by Igor Pavlov. Just as LZSS is a derivative of LZ77, LZMA uses a dictionary sliding window compression algorithm as well but its dictionary size can be much bigger than LZSS, and LZMA also borrows the ideas from Markov chain and range encoding.

LZMA stream is encoded to the sequence that consists of matches and literals. For a block of uncompressed data, if there is the same data (a match) in sliding window, then the uncompressed data is represented as a pair of two numbers (distance and length pair). All uncompressed bytes that cannot be encoded as matches are encoded as literals.



According to LZMA SDK, the RAM memory required in compression/decompression can be calculated in below formulas:

- for compression: (dictSize * 11.5 + 6 MB) + state_size
- for decompression: dictSize + state_size
- state_size = (4 + (1.5 << (lc + lp))) KB

Here are explanations for parameters in the formulas:

- dictSize (dictionary size): LZMA SDK recommends the dictionary should be larger than 4 KB, and sets the default value as 16 MB = (1 << 24) bytes.

- lc (literal context): It assumes how many higher bits of the previous uncompressed byte correlate with the next byte. For example, in typical English text, an upper-case letter is often followed by a lower-case letter, and a lower-case letter is usually followed by another lower-case letter. In the US-ASCII character set, the highest three bits are 010 for upper-case letters and 011 for lower-case letters. When lc is at least 3, the literal

coding can take advantage of this property in the uncompressed data. The value range of lc is from 0 to 8. The default value in LZMA SDK is 3.

- lp (literal position): Sets the number of literal position bits. This affects what kind of alignment in the uncompressed data is assumed when encoding literals. The lp is intended for periodical data when the period is equal to 2^lp. For example, for 32-bit (4 bytes) periodical data you can use lp=2. The range of lp is from 0 to 4. The default value is 0.

- pb (position bit): Sets the number of position bits. Pb is not in the RAM consumption formulas, but it may affect the size of compressed file. Similar to lp, pb affects what kind of alignment in the uncompressed data is assumed in general. It is also intended for periodical data when period is equal to 2^pb. When the alignment is known, setting the number of position bits accordingly may reduce the compressed file size a little. For example, with text files having one-byte alignment (ASCII, UTF-8), using `pb = 0` can improve compression slightly, For UTF-16 text, set pb = 1 is a good choice. The range of pb is from 0 to 4. The default value is 2.

Above parameters are saved in LZMA properties (5 bytes, including dictionary size) which is in the beginning of a LZMA data block in compressed file. The LZMA data format is shown in following figure, note that uncompressed size is required.



Offset	Size	Description
0	1	Special LZMA properties (lc, lp, pb in encoded form)
1	4	Dictionary size (little endian)
5	8	Uncompressed size (little endian)
13		Compressed data

Selecting different values of these parameters can generate many combinations. Our testing result shows that the size of compressed files are quite close when we use different values of lc and lp, however, the file size can vary largely while the dictionary size changes, and the bigger dictionary size always make the size of the compressed file smaller.

4.6.2.1.5.3 Compression Method #3

Software compression is used to minimize the size of the actual data to transmit to in-vehicle ECUs during software download. The compression is performed outside the vehicle before the data is transmitted and the software decompression unpacks the received data in the vehicle ECUs during software download.

4.6.2.1.5.3.1 Data Compression

The method for compressing the data shall be Gzip.
Detail method and algorithm for compression and decompression can be find in:

https://www.gnu.org/software/gzip/

4.6.2.1.5.3.2 Data Decompression

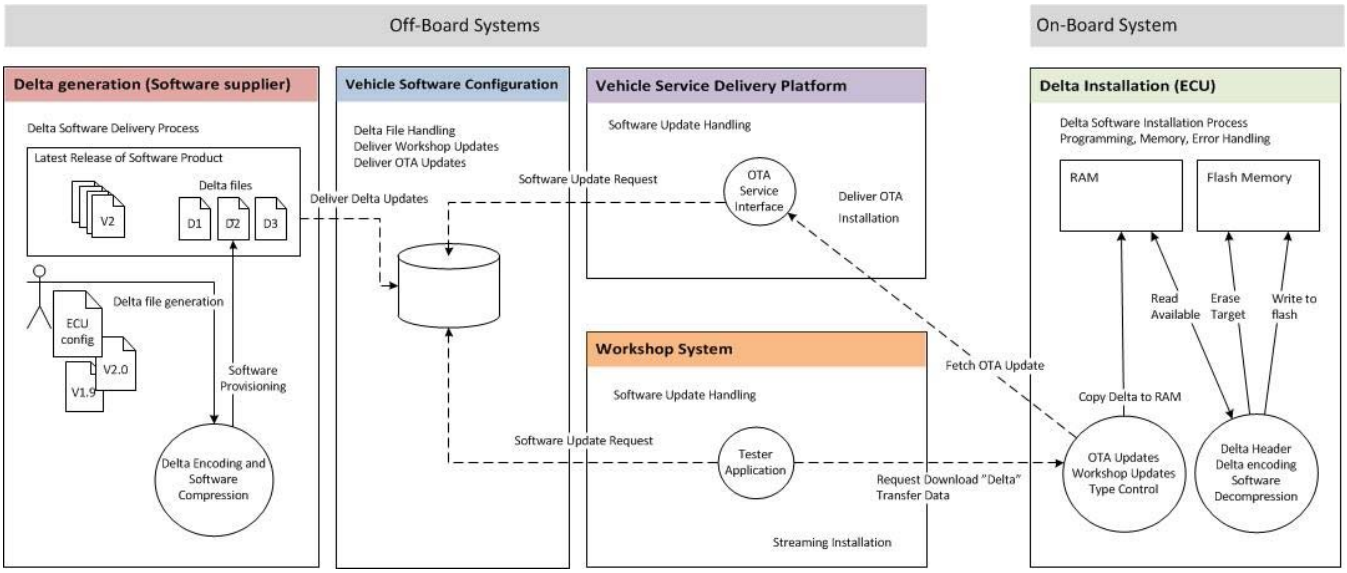
The method for compressing the data shall be Gzip.
Detail method and algorithm for compression and decompression can be find in:
https://www.gnu.org/software/gzip/

4.6.3 Delta Encoding

4.6.3.1 Delta Encoding Introduction

4.6.3.1.1 Logical System Components

The image below gives a component and interface overview of the Delta Encoded Software updates system.



Subsystem and Interface Overview

4.6.3.1.2 Requisite Documents

Reference	Source	Document
[Delta_1]	External	Kris Coppieters. A cross-platform binary diff. Dr. Dobb's Journal, May 1995
[Delta_2]	External	Brenda S. Baker, Udi Manber and Robert Muth. Compressing Differences of Executable Code, April 22, 1999
[Delta_3]	External	Nikolai Samteladze, Ken Christensen. DELTA: Delta Encoding for Less Traffic for Apps
[Delta_4]	External	Colin Percival, Naive Differences of Executable Code
[Delta_5]	Internal	Data Compression and Encryption
[Delta_6]	Internal	Versatile Binary Format Specification

[Delta_7]	Internal	Software Authentication Specification

4.6.3.1.3 Definitions & Abbreviations

Definition	Description
On-board system	Systems and logical components located in the vehicle
Off-board system	Systems and logical components located outside the vehicle
Normal operational mode	The ECU is executing the application
Program mode	The ECU is executing the primary or secondary bootloader
Full version	The complete software product in file form or installed in an ECU
Target version	The target version is the newer version between two versions.
Source/current version	The source/current version is older then the target version.
Patch, patching	Producing a target version given a source version and a delta file (difference file)
Delta encoding	Delta encoding (or data differencing) consists of producing a difference given a source and a target, with patching producing a target given a source and a difference.

Abbreviation	Description
ECU	Electronic Control Unit. In vehicle electronic control unit with a computer/microcontroller.
SW	Software
OTA	Over The Air. Refers to a software distribution channel over Wi-Fi or mobile broadband
LIN	Local Interconnect Network. In Vehicle Network type.
CAN	Controller Area Network. In Vehicle Network type.
SBL	Secondary bootloader
RAM	Random Access Memory. Associated with volatile types of memory where stored information is lost if power is removed
UDS	Unified Diagnostic Services

4.6.3.2 Delta generation

4.6.3.2.1 Requirements

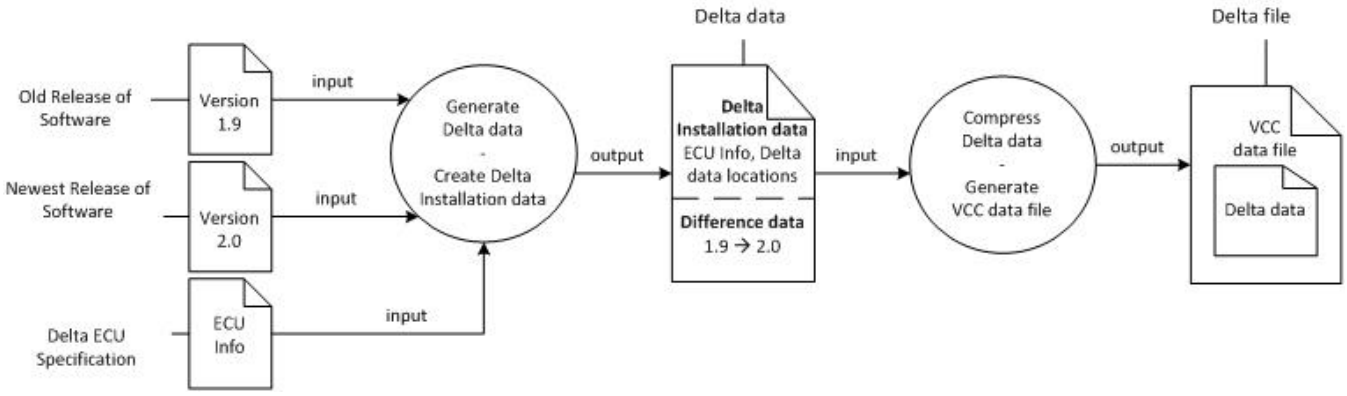
Delta encoding – or more generally, data differencing – is an approach to storing and transferring data using calculated differences between software versions rather than the complete new version of the software. A delta encoding algorithm takes an old and a new version of a file as input and computes the difference between them. The difference data can then be used to construct the new version of the file by merging it with the old. The measures of interest for delta encoding implementations are the size of the delta and the amount of memory and the time required to construct a delta and to apply it.

In order to update an existing software in an ECU by means of delta software delivery, the ECU needs information on how the delta data shall be programmed, E.g. Memory addresses to locate the differences that shall be updated. Also if there are any specific conditions that must be fulfilled before the installation can start, E.g. Software version check, to ensure that the delta is compatible a version number must be provided. This type of “ECU Info” needs to be provided to the ECU somehow and should be included in the delta file during the delta generation

process.

In addition to delta encoding the Delta Software Delivery Process can also add Software Compression in order to minimize the size of the data that is to be transmitted to the ECUs.

The generation of delta files can be described as in the image below.



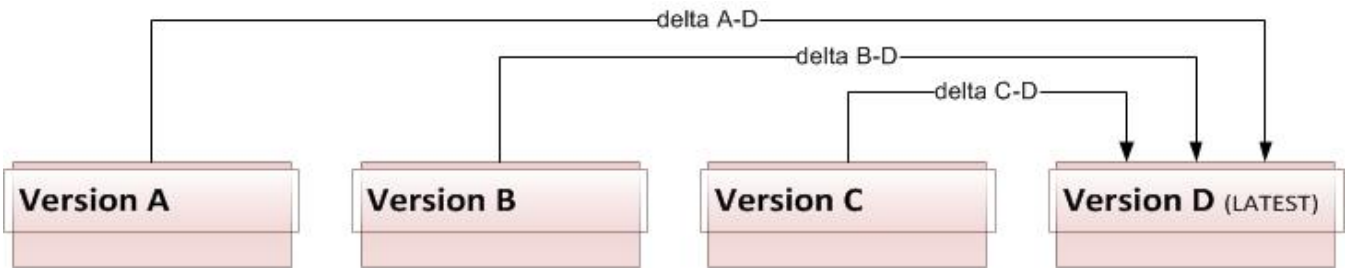
Delta generation process

6286 v3 **Compatibility between versions**

Version history:
(3):

Purpose: To guarantee compatibility between versions
Source:
Verification Method: Test
Legacy ID: 400851v2

Together with the latest software version for every new software release there shall be delta files generated for three previous versions of the of the software.



Different originating software versions

6287 v2 **Compression**

Version history:
(2):

Purpose: To decrease the size of the delta encoded data

Source:
Verification Method: Inspection
Legacy ID: 400792v1

The delta encoded data shall be compressed.

See *Data Compression and Encryption Specification and Versatile Binary Format Specification for details.*

6288 v2 **Data file type**

Version history:
(2):

Purpose: To ensure support for data file format
Source:
Verification Method: Inspection
Legacy ID: 400689v1

The delta files shall be provided within the data file format. *See [Versatile Binary Format Specification]*

6289 v1 **Delta file distribution**

Version history:
(1):

Purpose: To ensure consistent software deliveries
Source:
Verification Method: Inspection
Legacy ID: 409691v0

The delta files shall be provided to the Software Configuration System in the same software archive as the target version that they are intended to create.

6290 v4 **Delta file name**

Version history:
(4):

Purpose: To guarantee delta version traceability
Source:
Verification Method: Inspection
Legacy ID: 403939v3

The generated delta files shall have the source and target software part number and version included in the filename.

Example:

sw_part_number = "31808832"; // Target version part no
sw_version = "AB"; // Target version
sw_current_part_number = "31808831"; // Source/current version part no
sw_current_version = "A";

A VBF file that contains a delta and uses the header data above shall utilize the file name of:
<sw_current_part_number><sw_current_version>_<sw_part_number>< sw_version>.VBF

E.g: **31808831A _31808832AB.VBF**

6291 v2 **Delta file release information**

Version history:
(2):

Purpose: To guarantee installation efficiency
Source:
Verification Method: Inspection
Legacy ID: 400796v1

Together with the delta file delivery there shall be formal information about the software supplier's test results.
The figures shall be results from system tests conducted in a system testing environment.
The delta file installation time and delta file size shall be stated together with the install time and file size of a full target version.

6292 v2 **Delta file size**

Version history:
(2):

Purpose: To guarantee installation efficiency
Source:
Verification Method: Test
Legacy ID: 400795v1

The generated compressed delta file shall be at least 50% smaller, compared to a complete compressed full version *See [Data Compression and Encryption]*.

Note: If the full version file is not compressed due to decreased install time as a result of the decompression routine, then the file shall be compressed using the compression method described in *[Data Compression and Encryption]* in order to perform the comparison. The file size shall be verified by selecting the file in Windows Explorer and select properties (Or equivalent for e.g Linux).

6293 v2 **Delta file structure**

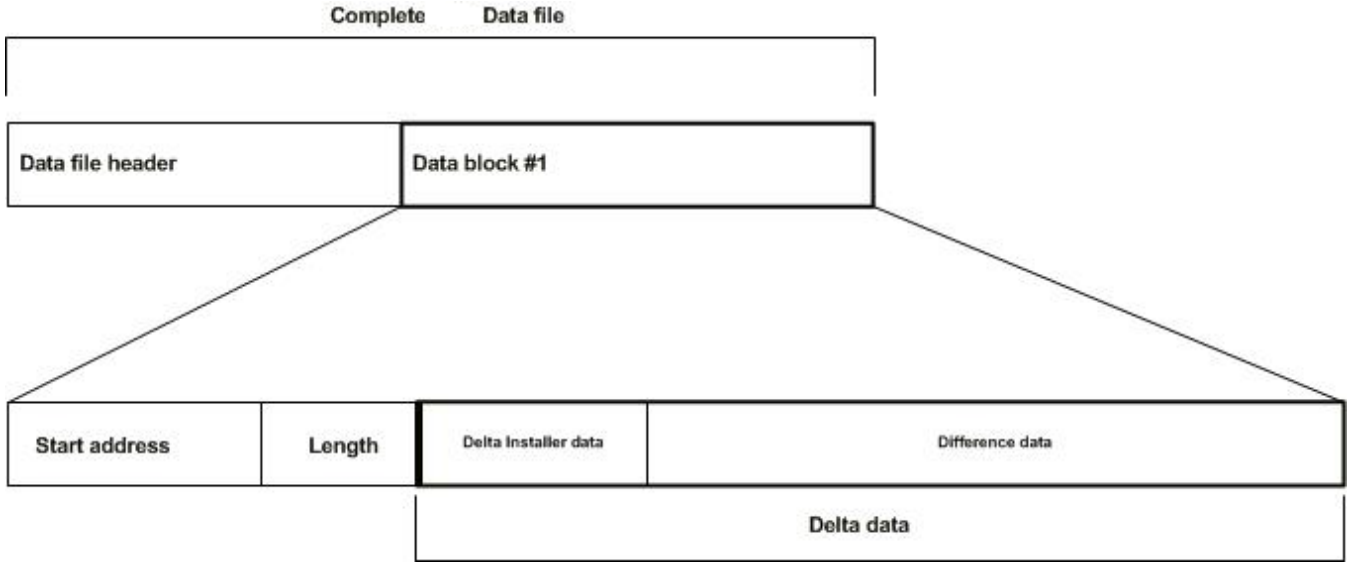
Version history:
(2):

Purpose: To ensure a consistent delta file structure
Source:
Verification Method: Inspection
Legacy ID: 407512v1

Delta files are Data files with one single data block that shall consist of delta installation parameters and the actual delta/difference data.

See also [Versatile Binary Format Specification]

The image below shows how the delta is located within a Data file



Delta file structure

6294 v3 **Delta file version info**

Version history:
(3):

Purpose: To guarantee compatibility between ECU software versions.
Source:
Verification Method: Test
Legacy ID: 400819v2

The delta installation data shall include the Software product number as well as the Software version number for the source version (see below example), so that the ECU can read this data and decide to reject the installation before start if the installed ECU software version is not compatible.

E.g
sw_current_part_number = "31808831";

sw_current_version = "A";

See [Versatile Binary Format Specification] for details

6295 v1 **Delta installation RAM buffer size**

Version history:
(1):

Purpose: To ensure that the ECU provides enough memory for the delta installation
Source:
Verification Method: Test
Legacy ID: 413074v0

The delta installation data shall include the size of the RAM needed for the delta installation, so that the ECU can read this data and decide to reject the installation before start if the memory amount can not be allocated.

6296 v2 **Delta installation time**

Version history:
(2):

Purpose: To guarantee installation efficiency
Source:
Verification Method: Inspection
Legacy ID: 400794v1

The Installation time of the generated delta shall be at least 50% shorter in average compared to a complete installation of the compressed full version.
(I.e. The time from the first RequestDownload request until the CheckMemory final response)

6297 v1 **New and obsolete data info**

Version history:
(1):

Purpose: To ensure that the bootloader can locate and apply the delta file changes as well as remove unused data.
Source:
Verification Method: Inspection
Legacy ID: 417656v0

The delta installation data shall contain instructions for the bootloader to locate where in memory the delta data shall be written.
The installation data shall also contain instructions in order to locate and erase any obsolete data from the old version that otherwise would remain unused in memory after the delta update.

4.6.3.3 Delta installation

4.6.3.3.1 Requirements

Delta encoded data is applied to the ECU software by the bootloader's delta installation routine.

6298 v2 **Data decoding**

Version history:
(2):

Purpose: To decode the delta file
Source:
Verification Method: Test
Legacy ID: 400914v1

The ECU shall support decoding of a delta-encoded file

Note: The dataFormatIdentifier in the RequestDownload request indicates if the data is delta encoded

6299 v2 **Data decompression**

Version history:
(2):

Purpose: To decompress the delta file
Source:
Verification Method: Test
Legacy ID: 400913v1

The ECU shall support decompression of a compressed delta file

Note: The dataFormatIdentifier in the RequestDownload request indicates if the data is compressed

6300 v2 **Delta installation data**

Version history:
(2):

Purpose: To ensure that the ECU installation process receives all installation data provided in the delta file
Source:
Verification Method: Test
Legacy ID: 400883v1

When the bootloader receives a delta file, it shall read and validate the delta installation data provided prior start of the installation phase.
(E.g. Software version information shall be provided and used to decide if installation can

start.)

The delta installation data shall be provided within the beginning of the streamed delta data, the complete delta file shall not be downloaded before this information is checked.

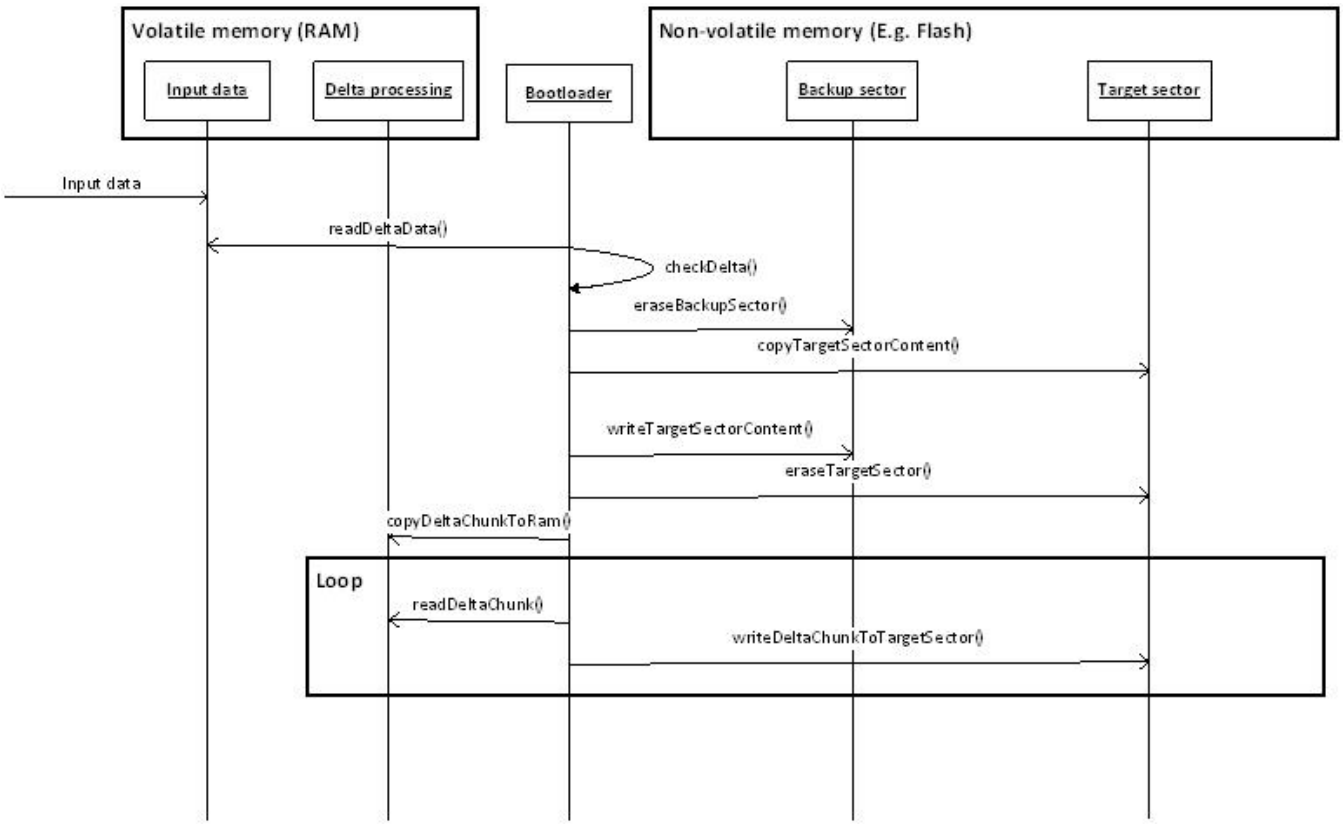
6301 v3 **Delta installation memory segment**

Version history:
(3):

Purpose: To enable delta data manipulation
Source:
Verification Method: Test
Legacy ID: 400885v2

The ECU memory map must have memory segment (e.g. flash sector) dedicated for delta installation in order to read, modify and place back data.

Note: The image below illustrates an example. It shall not be considered a solution



Delta installation - Extra sector usage example

6302 v2 **Delta installation RAM usage**

Version history:

(2):

Purpose: To ensure fast delta installation
Source:
Verification Method: Test
Legacy ID: 400890v1

The bootloader shall use volatile memory (RAM) when processing the delta data.
This refers to physically accessible RAM not occupied by the boot loader

6303 v2 Delta memory segment size

Version history:
(2):

Purpose: To enable delta data manipulation
Source:
Verification Method: Test
Legacy ID: 400889v1

The available non-volatile memory segment size shall be at least the same size compared to the largest possible segment size within the ECU. (E.g. largest flash sector)

6304 v3 Delta SWDL in normal operational mode

Version history:
(3):

Purpose: Reduce installation time
Source:
Verification Method: Test
Legacy ID: 400877v2

If the ECU supports software installation in normal operational mode then delta download and installation shall also be supported.

6305 v2 Delta SWDL in program mode

Version history:
(2):

Purpose: Reduce programming time
Source:
Verification Method: Test
Legacy ID: 400876v1

The ECU shall support delta download and installation of software in program mode.

6306 v3 Obsolete data removal

Version history:
(3):

Purpose: To ensure that data that is located in memory that is no longer a part of the installed software product is erased.
Source:
Verification Method: Test
Legacy ID: 416292v2

The bootloader shall use instruction provided in the delta installation data to locate and erase data that is no longer a part of the installed software product.

6307 v2 Software consistency check

Version history:
(2):

Purpose: To ensure that the target image is correct after installation
Source:
Verification Method: Test
Legacy ID: 400916v1

The bootloader shall verify that the applied delta resulted in the correct software image by using the method specified in *[Software Authentication Specification].*

6308 v2 Streamed installation

Version history:
(2):

Purpose: To reduce programming time
Source:
Verification Method: Test
Legacy ID: 400893v1

The ECU shall support streamed installation using UDS, meaning that the bootloader shall start to decode and install data before the complete data file is received.
For more information please see *[General Software Download REQPROD 53978]*

6309 v1 Target sector erase check

Version history:
(1):

Purpose: To ensure that a fallback is possible on each sector during installation
Source:
Verification Method: Test
Legacy ID: 420311v0

Before a target sector is erased and replaced with the delta update, a backup of the sector shall be stored and verified to be exactly the same as the target sector.

4.6.3.3.1.1 Error Handling

6310 v1 **Consistency check failure**

Version history:
(1):

Purpose: To guarantee that the correct image is installed
Source:
Verification Method: Test
Legacy ID: 413452v0

The ECU shall reject the installation and respond with a negative response code if the ECU software is not verified as consistent using the method defined in *[Software Authentication Specification]*

6311 v2 **Delta file corruption**

Version history:
(2):

Purpose: To guarantee that corrupted data is not used as installation data
Source:
Verification Method: Test
Legacy ID: 400955v1

The ECU shall reject the installation and respond with a negative response code if the bootloader detects that the delta update package is corrupted.

6312 v3 **Error code mapping**

Version history:
(3):

Purpose: To ensure negative diagnostic response code feedback for error codes during the delta installation
Source:
Verification Method: Test
Legacy ID: 400952v2

Error codes used during the delta installation shall have corresponding negative diagnostic response codes.
See [ISO 14229-1] for NRC reference

The following table shows possible error code mappings:

Delta Installation Error codes	Description	Negative response code (NRC)
NOT_ENOUGH _MEMORY	Either the non-volatile memory or the RAM buffer provided in	generalProgrammingFailure

	the init-routine is less than needed to apply the update	
FAILED_DECOMPRESSION	The bootloader failed to decompress the update package	generalProgrammingFailure
PACKAGE_CORRUPTED	The delta update package was identified as corrupted	generalProgrammingFailure
FAILED_CONSISTENCY_CHECK	The delta update was not identified as a valid software image	generalProgrammingFailure
WRONG_TARGET_VERSION	Source image version is not as expected in delta	uploadDownloadNotAccepted

6313 v2 **Full version install check**

Version history:
(2):

Purpose: To ensure that a full version installation can be installed after a failed delta installation
Source:
Verification Method: Test
Legacy ID: 400959v1

After a failed delta installation the bootloader shall not prevent or resume installation when a full version is to be installed.

Note: Full version is indicated by the dataFormatIdentifier in the RequestDownload request

See [Versatile Binary Format Specification] for reference

6314 v1 **Not enough memory**

Version history:
(1):

Purpose: To guarantee that there is enough memory to perform a delta installation
Source:
Verification Method: Test
Legacy ID: 412891v0

The ECU shall reject the installation and respond with a negative response code if the bootloader detects that either non-volatile memory or the RAM buffer provided in the init-routine is less than needed to apply the update

6315 v2 **Recover from interrupted Installation**

Version history:
(2):

Purpose: To ensure recovery after interrupted Installation
Source:
Verification Method: Test
Legacy ID: 400957v1

If the ECU has memory enough to store a copy of the source version before the delta installation starts, then it shall be possible to fall back to the source version.
If the ECU does not have enough memory available to fall back to the source version then it shall be possible to resume the update installation from the beginning or from the last successfully written code.

6316 v2 Subsequent reprogramming during recover

Version history:
(2):

Purpose: To avoid loss of data or broken ECUs
Source:
Verification Method: Test
Legacy ID: 400958v1

The ECU software shall be designed in such way that an aborted download or install, or removing ECU power at any time during the download and install operation, shall not prevent subsequent reprogramming of the same delta file.

Note: Subsequent reprogramming means that the diagnostic tester is restarting the download from the beginning.

6317 v2 Version mismatch

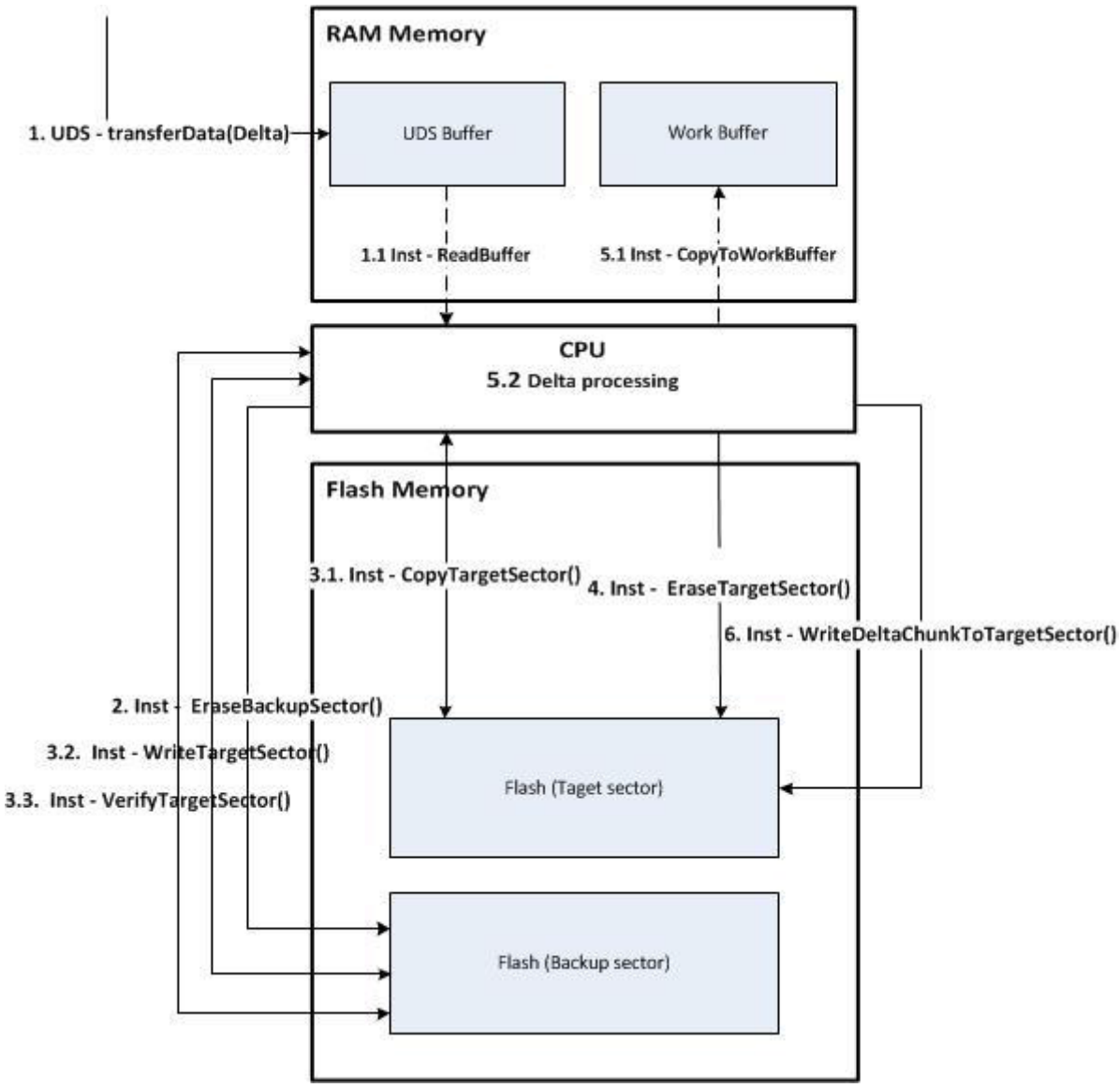
Version history:
(2):

Purpose: To guarantee compatibility between versions
Source:
Verification Method: Test
Legacy ID: 400953v1

The ECU shall reject the installation and respond with a negative response code if the currently installed ECU software version does not match *sw_current_version* provided in the delta installation data.

4.6.3.4 Extended information

4.6.3.4.1 Delta installation - Sequence and memory usage



Delta installation - Sequence and memory usage

4.6.4 Bootloader - UDSonCAN

4.6.4.1 Bootloader - UDSonCAN

4.6.4.1.1 Transport-Network Layer

The transport/network layer required by this document shall be compliant to *[BL_CAN_5] DoCAN*.

3248 v2	Transport_Network Layer

Version history:
(2):

Purpose: To define the transport/network layer used by the bootloader.
Source:
Verification Method: Test
Legacy ID: 52246v1

The CAN bootloader shall implement the transport/network layer defined in *[BL_CAN_5] DoCAN* .

4.6.4.1.2 Data Link Layer

The data link layer required by this document shall be compliant to *[BL_CAN_6] CAN Datalink Layer*.

3246 v3 **Data Link Layer**

Version history:
(3):

Purpose: To define the data link layer used by the bootloader.
Source:
Verification Method: Test
Legacy ID: 52247v2

The CAN bootloader shall implement the data link layer defined in *[BL_CAN_6] CAN Datalink Layer*.

4.6.4.1.2.1 Addressing USDT frames

Since 11 bit CAN address mode with normal addressing format (see *[BL_CAN_5] DoCAN)* is chosen, a fixed transmission ID is used. The ECU shall use the response ID range specified in the table *ECU Diagnostic Reception / Transmission CAN IDs* as N_TA (Session layer Target Address).

ECU Reception ID (Client Transmit Request)	ECU Transmission ID (Client Receive Request)
0x700–0x70F (1)	0x600–0x60F (1)
0x710	0x610
0x711	0x611
0x712	0x612
…	…
…	…
0x7FF	Functional request

Table - ECU Diagnostic Reception / Transmission CAN IDs
Note (1): The ECU reception ID from 0x700 to 0x70F is reserved due to the domain ECU always shall have network ID equal to zero(0).

Each ECU shall support these USDT CAN diagnostic IDs.

- Functionally addressed requests with CAN frame identifier 0x7FF.
- Physically addressed requests (or ECU Diagnostic Reception ID).

- Physically addressed responses or (ECU Diagnostic Transmission ID). The ECU Transmission ID is always equal to the value of the ECU Reception ID minus 0x100.

3247 v2 **Supported CAN IDs**

Version history:
(2):

Purpose: To ensure communication is possible with an ECU.
Source:
Verification Method: Test
Legacy ID: 52243v1

An ECU connected to a CAN shall have a designated USDT ECU Reception ID. The Reception ID shall be the least significant byte of an ECU address. Transmission ID shall be Reception ID – 0x100. Example: ECU address = 0x1A12: Reception ID=0x712; Transmission ID=0x612.
An ECU shall also support functional requests on 0x7FF.

4.6.4.1.3 Session Layer

The session layer required by this document shall be compliant to *[BL_CAN_2] UDS Session generic reqs* and *[BL_CAN_3] UDS Session server reqs* with the restrictions/additions as defined by the subsequent sub-sections.

3249 v2 **Diagnostic Session Layer**

Version history:
(2):

Purpose: To define the Session layer used by the bootloader.
Source:
Verification Method: Test
Legacy ID: 52245v1

The CAN bootloader shall implement the session layer defined in *[BL_CAN_2] UDS Session generic reqs* and *[BL_CAN_2] UDS Session server reqs* .

4.6.4.1.4 ECU Program Mode

Program mode means that the ECU executing the primary or secondary bootloader. This chapter describes state and flow chart diagrams unique to the CAN bootloader. Additional shall the CAN bootloader also implement the non network dependent bootloader requirements described in *[BL_CAN_1] General Software Download*.

134210 v1 **Bootloader CAN busoff handling**

Version history:

Purpose: To increase availablility of the system. Avoid that an ECU gets stuck in bus-off state, or exits programming session due to network errors.
Source:
Verification Method: Test
Legacy ID: 466900

If hardware support automatic CAN busoff recovery, that recovery mechanism shall be enabled.

If hardware doesn't support automatic bus-off recovery the bootloader shall check all active CAN controllers for bus-off status at maximum 10 ms intervals and if bus-off is detected start the bus-off recovery sequence within 10ms.
This requirement applies to Primary Boot Loader (PBL) and Secondary Boot Loader (SBL).

Note: If a permanent busoff condition is detected that can not be recovered, and a complete and compatible application exist, the S3server timer shall still run and will finally elapse, causing and exit of the programming session.

301 v3 **General bootloader requirements**

Version history:
(3):

Purpose: Non-network dependent bootloader requirements are defined in a separate document which is applicable for all bootloaders.
Source:
Verification Method: Test
Legacy ID: 52249v2

The CAN bootloader shall implement the non network dependent requirement defined in *[BL_CAN_1] General Software Download* .

4.6.4.1.4.1 Bootloader state diagram

There are two bootloaders required, the primary bootloader and the secondary bootloader. These are described in more detail in *[BL_CAN_1] General Software Download.* The CAN bootloader state diagram , the figure below is a complement to the flow chart diagram described in section *Bootloader flow chart diagram - Overview.*

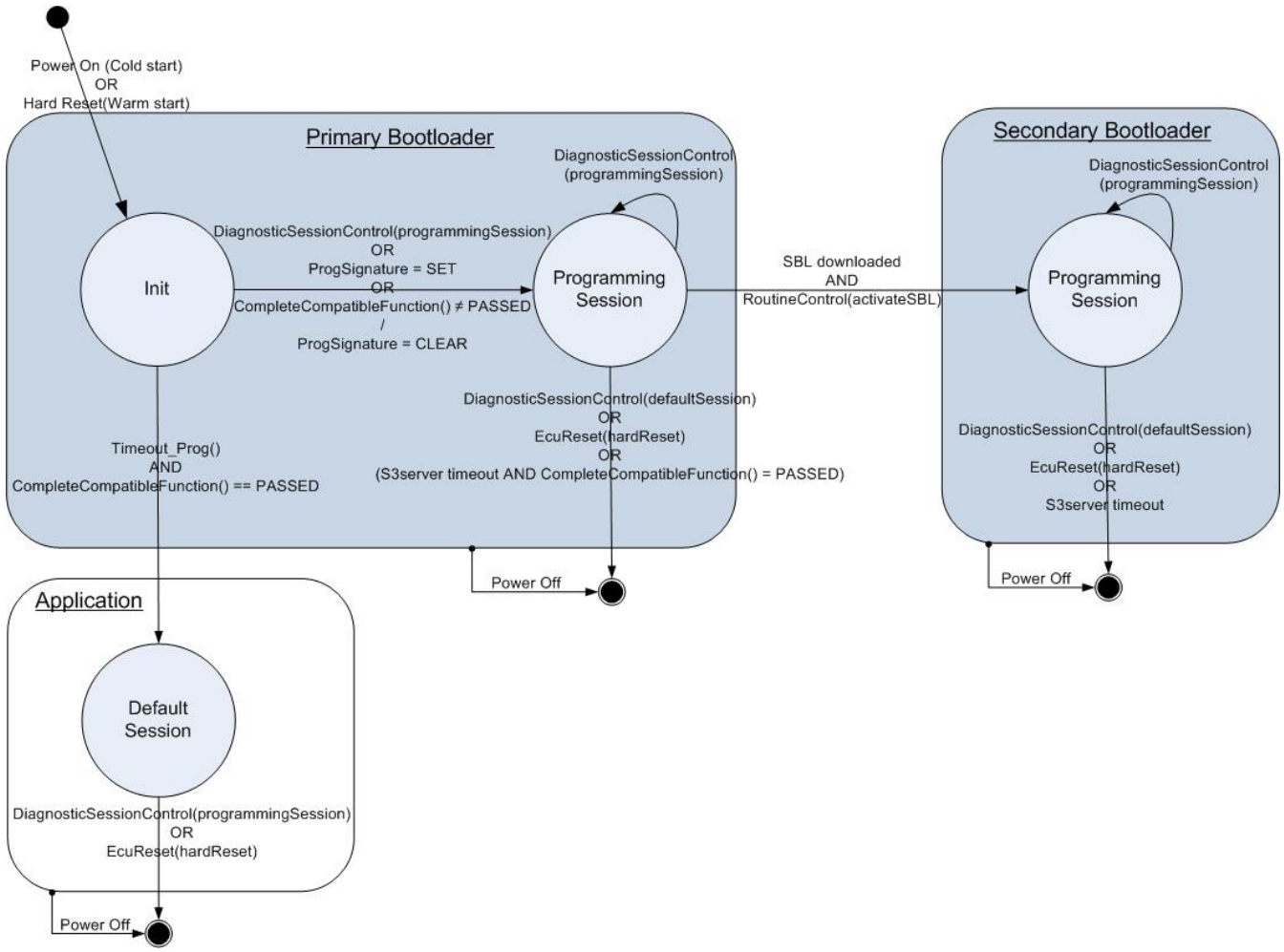


Figure – Bootloader State diagram

4.6.4.1.4.2 Bootloader flow chart diagram - Overview

The CAN bootloader flow chart diagram applies to all CAN based bootloader ECUs, requirements. The flow chart diagram is divided in states. Each state is further described in the following sections:

- Bootloader flow chart diagram (Init)
- Bootloader flow chart diagram (ProgSession)
- Flow chart diagram (Application)

Some figures contain yellow requirements tags. These requirement tags show where each requirement applies in the flow chart. To avoid problems with hardware initialization (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.), the ECU shall make a complete reset when exiting the application.

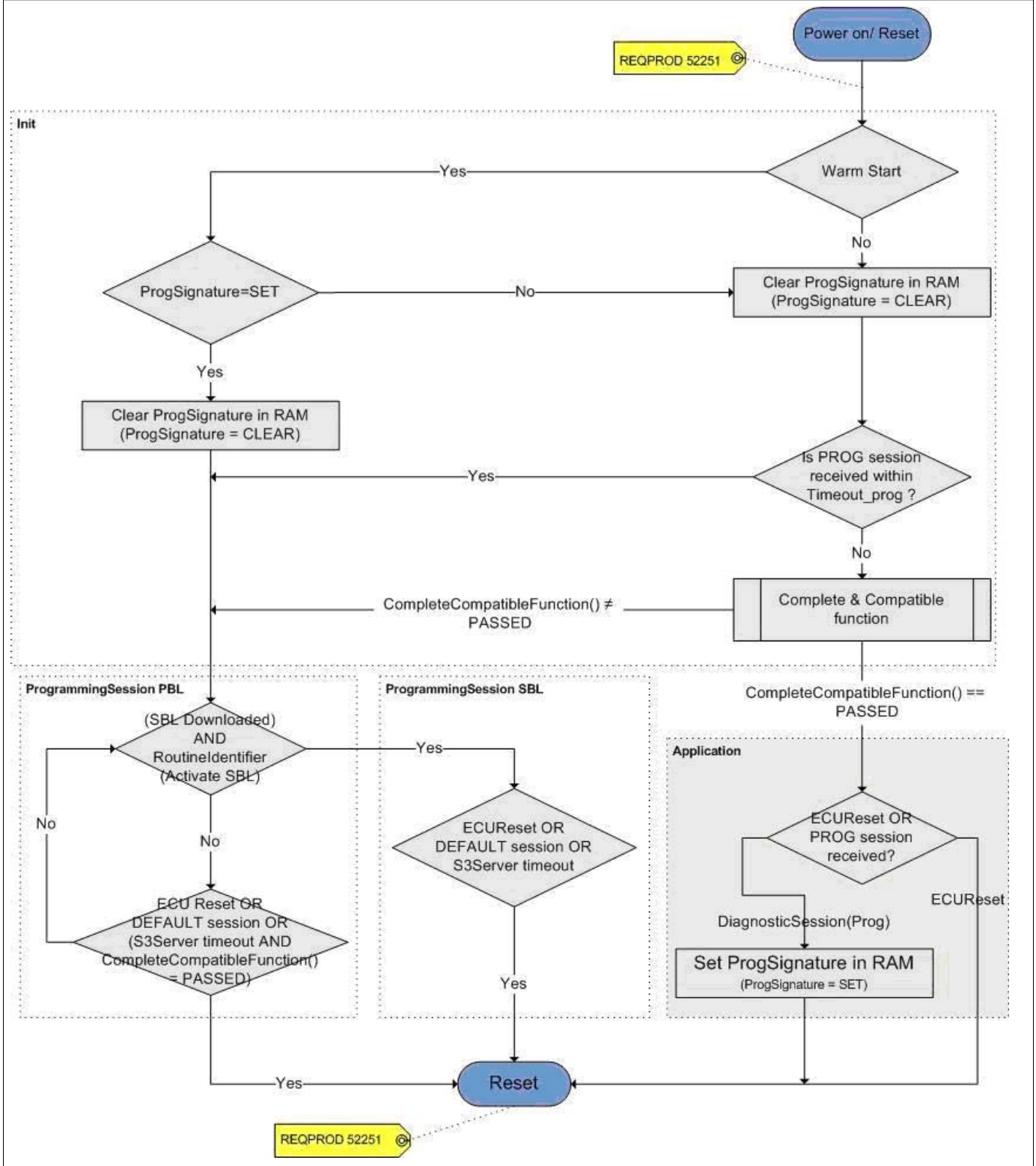


Figure - Bootloader flow chart diagram (Overview)

3251 v2 **Bootloader flow chart**

Version history:
(2):

Purpose: To define the bootloader flow chart. The flow chart diagram is applicable for all ECUs implementing CAN.
Source:
Verification Method: Test
Legacy ID: 52250v1

The CAN bootloader implementation shall follow flow chart in accordance to the figure *Bootloader flow chart diagram (Overview)* . The figure shows the order in which different sessions are activated.

3252 v2 **Complete hardware initiation when starting the ECU**

Version history:
(2):

Purpose: To avoid problems with hardware initialisation (i.e., write once registers, PLL-setup, memory re-map, interrupt vector table re-map, etc.)
Source:
Verification Method: Analysis
Legacy ID: 52251v1

The ECU shall always start from reset, regardless if the reset was initiated by a diagnostic request or by power up.

4.6.4.1.4.2.1 Bootloader flow chart diagram (Init)

The ECU shall be able to detect if start-up was caused by power-on (cold start) or initiated by the application (warm start).
Before the ECU starts the application, the ECU shall listen for the diagnostic request DiagnosticSessionControl(ProgrammingSession) during Timeout_prog. This time window may be used as a "backdoor" if the ECU is for some reason not able to communicate when the application is started. Duration of Timeout_prog is defined in *[BL_CAN_1] General Software Download*.
The Complete & Compatible function is responsible to decide whether ECUs software's are complete and compatible. The implementation of this step is specified in *[BL_CAN_1] General Software Download*.

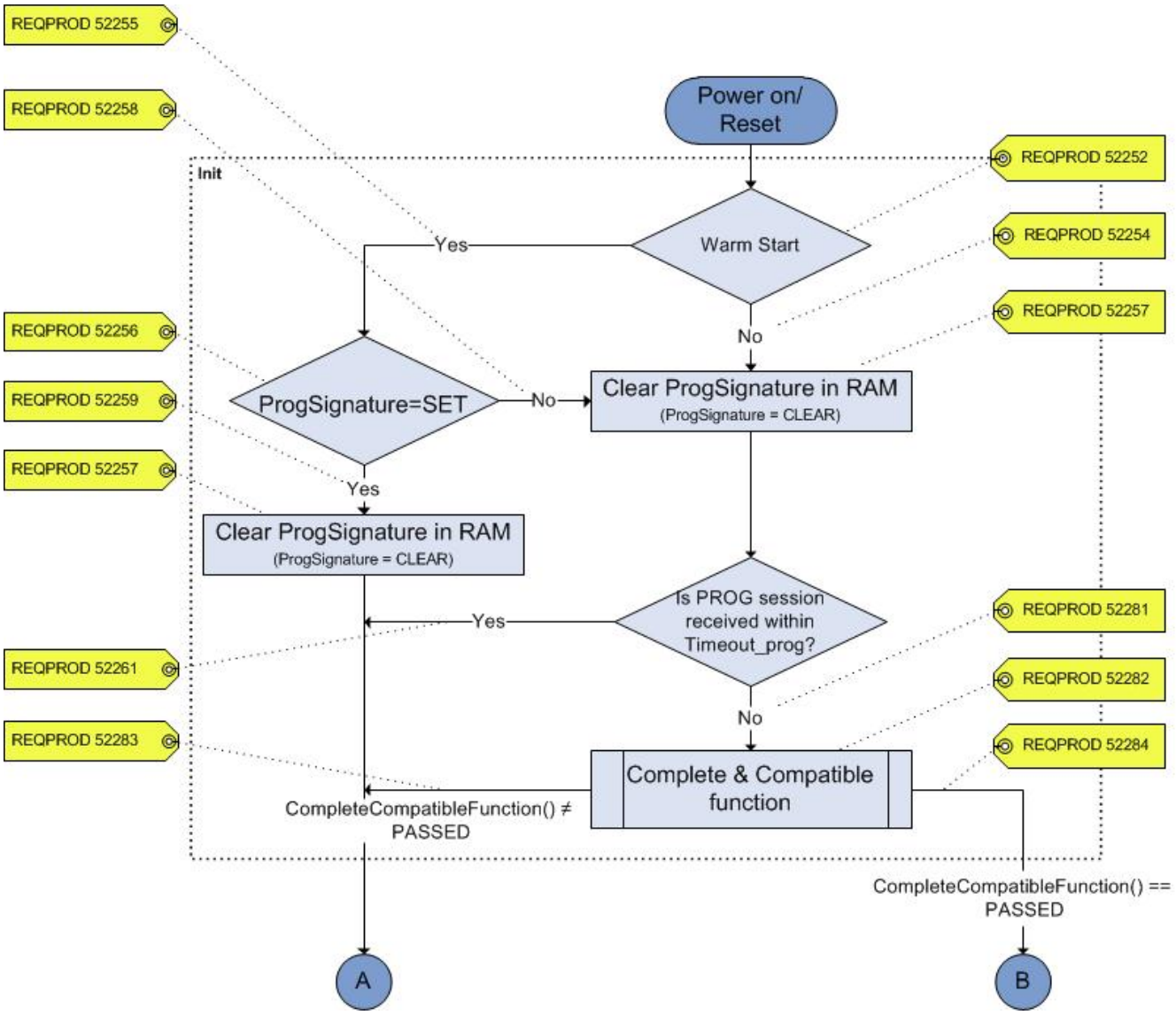


Figure - Bootloader flow chart diagram (Init)

3253 v2 **Cold or warm start detected**

Version history:
(2):

Purpose: The warm start-up state checks the cause of start-up to avoid having the bootloader unintentionally entering programming mode during cold start

Source:

Verification Method: Analysis

Legacy ID: 52252v1

The ECU shall be able to detect the type of start (cold or warm) and use that information to decide the next transition.

Note:

This information can for example be found in a transceiver, in the processor or in a voltage regulator. Where it is found is dependent on the hardware.

3254 v2 Cold start detected

Version history:
(2):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when cold start is detected
Source:
Verification Method: Analysis
Legacy ID: 52254v1

When cold start is detected next step is to clear the ProgSignature.

3255 v2 Warm start detected

Version history:
(2):

Purpose: There is a difference in actions between warm and cold start-up. This requirement defines the next transition when warm start is detected.
Source:
Verification Method: Analysis
Legacy ID: 52255v1

When warm start is detected next step is to check if the ProgSignature is SET.

302 v2 Check ProgSignature in RAM

Version history:
(2):

Purpose: To define the "Check ProgSignature in RAM" state that is used to communicate status between the application and the bootloader.
Source:
Verification Method: Analysis
Legacy ID: 52256v1

The ECU shall after a warm start check whether the ProgSignature at fixed RAM address is SET or not.
Note:
The ProgSignature in RAM shall be written by the application when a DiagnosticSessionControl(programmingSession) received, the format and length of the ProgSignature is described in *[BL_CAN_1] General Software Download*.

298 v2 Clear ProgSignature in RAM

Version history:
(2):

Purpose: To define the "Clear ProgSignature in RAM" function that provides protection against unwanted bootloader activation.
Source:

Verification Method: Analysis
Legacy ID: 52257v1

Note: Th e format and length of the ProgSignature is described in *[BL_CAN_1] General Software Download*.

3256 v2 ProgSignature is not SET

Version history:
(2):

Purpose: To define next transition when the ProgSignature not is set
Source:
Verification Method: Analysis
Legacy ID: 52258v1

When the ProgSignature is not SET the ECU shall enter the “Clear ProgSignature in RAM” state.

3257 v2 ProgSignature is SET

Version history:
(2):

Purpose: The purpose of setting the ProgSignature to a predefined value is to enter the programmingSession state
Source:
Verification Method: Analysis
Legacy ID: 52259v1

When the ProgSignature is SET the next step is to enter the “Clear ProgSignature in RAM” state.

3258 v2 PROG received within Timeout_Prog

Version history:
(2):

Purpose: To be able to enter the programmingSession state when the DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_prog.
Source:
Verification Method: Test
Legacy ID: 52261v1

When the ECU receives the diagnostic request DiagnosticSessionConrol(ProgrammingSession) within Timeout_Prog shall the ECU enter the “programmingSession” state.

3259 v2 No PROG received within Timeout_Prog

Version history:
(2):

Purpose: To define next transition when no DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_Prog.
Source:
Verification Method: Test
Legacy ID: 52281v1

If no DiagnosticSessionConrol(ProgrammingSession) request is received within Timeout_Prog shall the ECU enter the Complete & Compatible function.

299 v3 Complete & Compatible function

Version history:
(3):

Purpose: The ECU shall be able to detect if the current downloaded software is compatible or not.
Source:
Verification Method: Analysis
Legacy ID: 52282v2

The ECU shall detect if the software is compatible or not, by using Complete & Compatible function specified in *[BL_CAN_1] General Software Download*.

3260 v2 Non complete or compatible software(s)

Version history:
(2):

Purpose: To define the action when software(s) is/are neither complete nor compatible.
Source:
Verification Method: Analysis
Legacy ID: 52283v1

If the return value from the Complete & Compatible function is not equal to PASSED it means the ECU is either not complete or compatible, and therefore the ECU shall go to “programmingSession” state.

3261 v2 Complete and compatible software(s)

Version history:
(2):

Purpose: To define the action when the software(s) is/are complete and compatible.
Source:
Verification Method: Analysis
Legacy ID: 52284v1

If the return value from the Complete & Compatible function is equal to PASSED, it is safe to start the application.

4.6.4.1.4.2.2 Bootloader flow chart diagram (ProgSession)

ProgrammingSession state is entered when ProgSignature is set or when the software's is not Complete & Compatible or a ProgrammingSession was received before Timeout_Prog time ran out.
When the ECU has entered the programmingSession state the ECU shall be able to receive, process and respond to diagnostic requests supported by the bootloader.

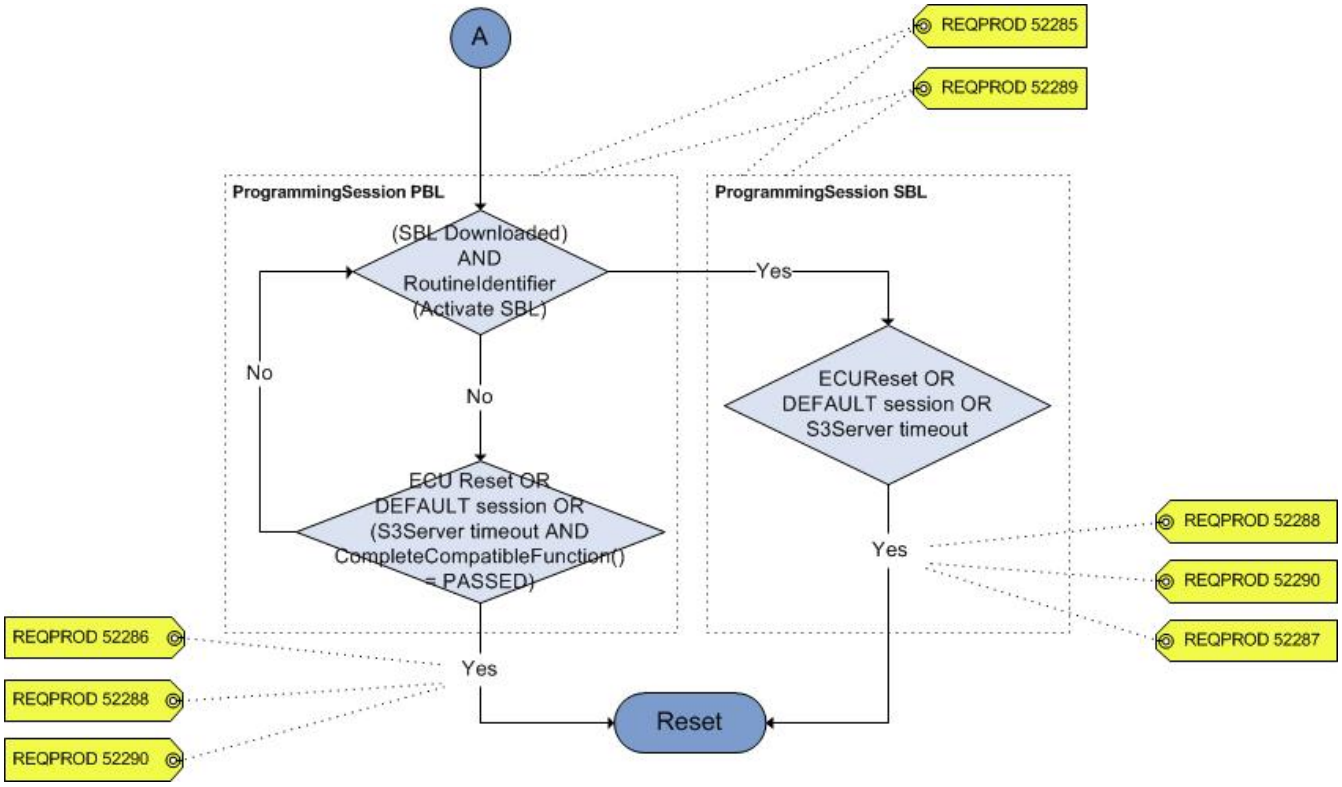


Figure - Bootloader flow chart diagram (ProgSession)

3262 v2	Receive and process diagnostic programming session requests

Version history:
(2):

Purpose: To define when the ECU shall be able to receive and process a diagnostic programming session request.

Source:

Verification Method: Test

Legacy ID: 52285v1

When the ECU is in programmingSession state, the ECU shall be able to receive and process diagnostic programming session requests.

300 v3	S3Server timer timeout in SBL

Version history:
(3):

Purpose: To define the behaviour when the S3server times out in SBL
Source:
Verification Method: Test
Legacy ID: 52287v2

When the S3 $_{server}$ times out when running SBL, the ECU shall make a reset.

Note: *The S3 $_{server}$ is defined in [BL_CAN_2] UDS Session generic reqs*.

3263 v3 S3Server timer timeout in PBL

Version history:
(3):

Purpose: To define the behaviour when S3server times out in PBL.
Source:
Verification Method: Test
Legacy ID: 52286v2

If S3 $_{server}$ times out when running PBL, the ECU shall make a test on Complete and Compatible function and if Complete and Compatible function returns PASSED the ECU shall make a reset.

Note:
The S3 $_{server}$ is defined in [BL_CAN_2] UDS Session generic reqs.

3264 v2 Reception of DiagnosticSessionControl(defaultSession)

Version history:
(2):

Purpose: To define the behaviour when the ECU receives the DiagnosticSessionControl(defaultSession) request in the programmingSession state.
Source:
Verification Method: Test
Legacy ID: 52288v1

When the ECU receives a DiagnosticSessionControl(defaultSession) request, the ECU shall make a reset, thus restarting the ECU from Init state.

3265 v2 Reception of DiagSessionControl(programmingSession)

Version history:
(2):

Purpose: To prevent a reset when the bootloader is in the programmingSession state and receive a diagnostic request DiagnosticSessionControl(programmingSession)
Source:
Verification Method: Test
Legacy ID: 52289v1

When the bootloader is in the programmingSession state (PBL or SBL) and receives a DiagnosticSessionControl(programmingSession) request, the ECU shall not reset i.e. the ECU shall stay in the programmingSession state.

3266 v2 **Reception of the ECUReset service**

Version history:
(2):

Purpose: To define the behaviour when the ECU receives the ECUReset request in the programmingSession state.
Source:
Verification Method: Test
Legacy ID: 52290v1

When the ECU receives an ECUReset request, the ECU shall make a reset, thus restarting the ECU from Init state.

4.6.4.1.4.2.3 Flow chart diagram (Application)

This is the normal operation mode of the ECU. To make the ECU enter the programmingSession, a client needs to send a DiagnosticSessionControl(programmingSession) request to the ECU. When all criteria for entering the bootloader has been met, it shall be enough sending one DiagnosticSessionControl(programmingSession) request to make the ECU set the ProgSignature and execute the reset.

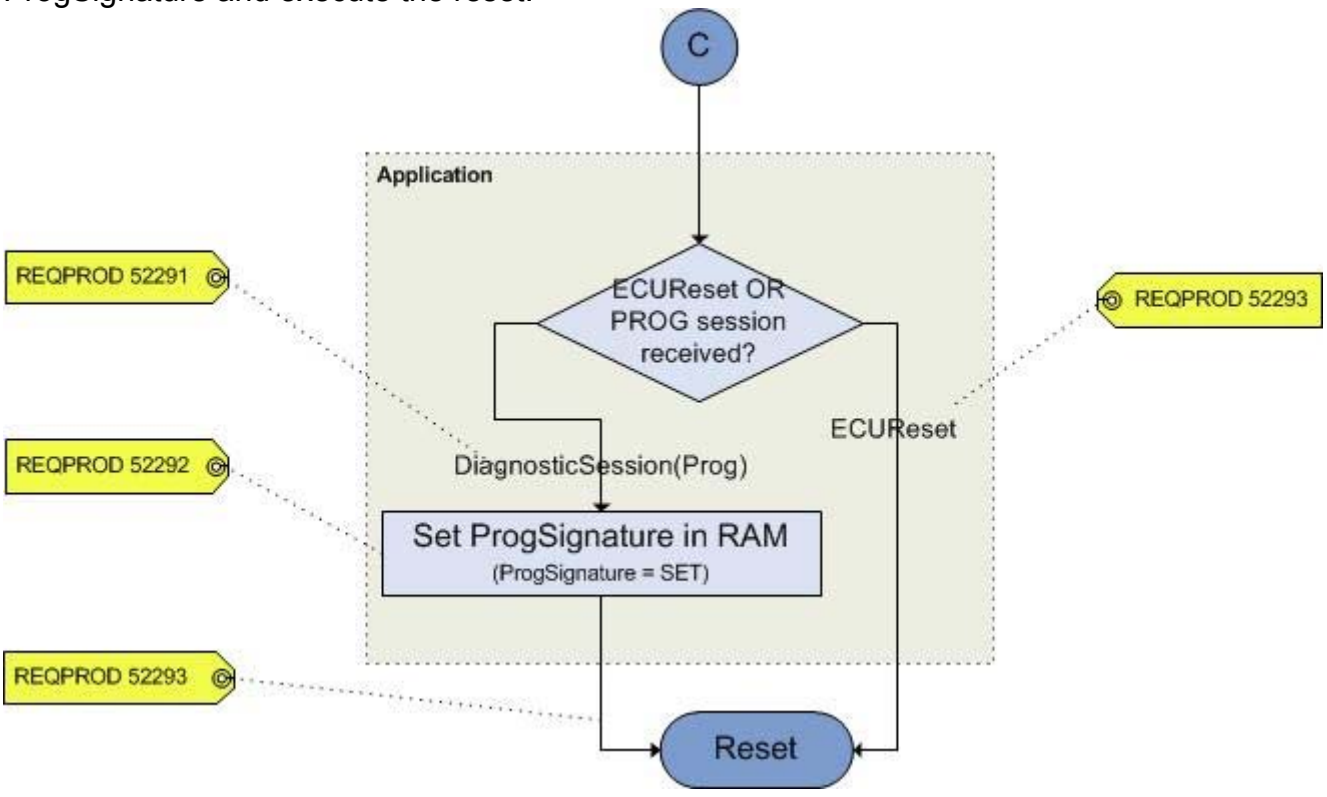


Figure - Flow chart diagram (Start Application)

3267 v2 **Enter bootloader from ECU application**

Version history:
(2):

Purpose: To define how to set the ECU to program mode when the ECU application is active.
Source:
Verification Method: Test
Legacy ID: 52291v1

When all criteria for entering the bootloader has been met, it must be possible to enter the bootloader from the application by receiving the DiagnosticSessionControl(programmingSession) service only once.

3268 v3 **Write ProgSignature**

Version history:
(3):

Purpose: The ProgSignature is used after reset to determine whether the ECU shall stay in the bootloader or start the application.
Source:
Verification Method: Test
Legacy ID: 52292v2

W hen the ECU application receives a DiagnosticSessionControl(programmingSession) request, the ProgSignature shall be written to a fixed address in RAM.

Note: *The format and length of the ProgSignature is described in [BL_CAN_1] General Software Download*.

3269 v2 **Trig a reset in ECU application mode**

Version history:
(2):

Purpose: To enter the bootloader from the application
Source:
Verification Method: Test
Legacy ID: 52293v1

The ECU application shall trig a reset on reception of a DiagnosticSessionControl(programmingSession) or an ECUReset(hardReset)

4.6.4.1.5 Physical Layer

The physical layer required by this document shall be compliant to *[BL_CAN_6] CAN Datalink Layer*.

3250 v2 **Physical Layer**

Version history:

(2):

Purpose: To define the physical layer used by the bootloader.
Source:
Verification Method: Test
Legacy ID: 83301v1

The CAN bootloader shall implement the physical layer defined in *[BL_CAN_6] CAN Datalink Layer*.

4.6.4.1.6 Application Layer

The application layer services shall be compliant to *[BL_CAN_8] ISO Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN)*.

3245 v2 Application Layer

Version history:
(2):

Purpose: To define the Application layer used by the bootloader
Source:
Verification Method: Test
Legacy ID: 52242v1

The CAN bootloader application layer shall be compliant to *Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN)* .

4.6.4.1.7 Requisite Documents

Reference number	Source	Title
[BL_CAN_1]	Internal	General Software Download
[BL_CAN_2]	Internal	Session generic reqs
[BL_CAN_3]	Internal	UDS Session server reqs
[BL_CAN_4]	Internal	UDS Services
[BL_CAN_5]	Internal	DoCAN
[BL_CAN_6]	Internal	CAN Datalink Layer
[BL_CAN_7]	Internal	Diagnostic and Bootloader Gateway Specification
[BL_CAN_8]	ISO	ISO 14229-3: Road vehicles - Unified diagnostic services (UDS) - Part 3: UDS on CAN implementation (UDSonCAN)

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.
Information obtaining ISO documents may be found on the internet: www.iso.org

4.6.4.1.8 Introduction

This document specifies OEM specific bootloaders requirements for a CAN ECU. It intends to provide guidance and requirements to architects, system designers, application developers, and suppliers who are implementing the bootloader to an ECU connected to a CAN network.

Important to understand is that this document only specifies the CAN network specific requirements and not the complete bootloader. Figure - Document structure, shows the specifications that are related for a CAN ECU when implementing the bootloader.

The main aim with software download is to fulfill the needs of replace software's at repair shop and the assembly plant.

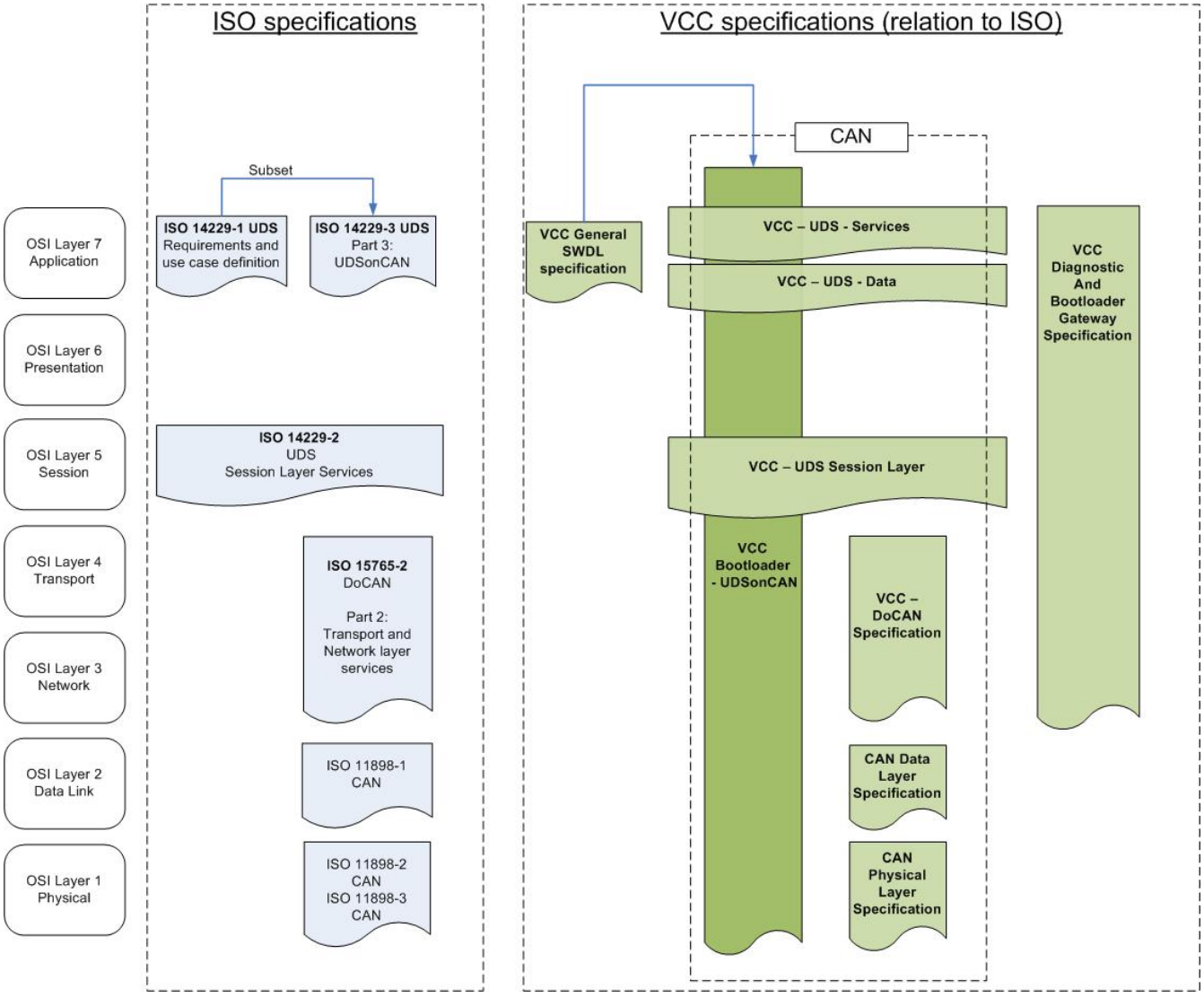


Figure - Document Structure

4.6.4.1.9 Definitions and Abbreviations

Definition	Description
Client	Software entity which uses services of a server
Client-Server Communication	A specific form of communication in a possibly distributed system in which software entities act as clients, servers or both, where 1...n clients are requesting services via a specific protocol from typically

	one server.
Cold Start	The start-up of the ECU from a powered-down or off state.
Gateway	An ECU that is connected to two or more independent communication networks that allows information to flow between the networks.
Program mode	Program mode means that the ECU executing the primary or secondary bootloader.
Sender-Receiver Communication	A communication pattern which offers asynchronous distribution of information where a sender communicates information to one or more receivers, or a receiver receives information from one or several senders.
Server	Software entity which provides services for clients
Warm start	The start-up of the ECU initiated by the application.

Abbreviation	Description
ECU	Electronic Control Unit
PBL	Primary Boot Loader
SBL	Secondary Boot Loader
UDS	Unified Diagnostics Services
USDT	Unified Segmented Data Transfer

4.6.4.1.10 Unified Diagnostic Services Implementation

This section specifies the diagnostic services and data required for a CAN bootloader.

4.6.4.1.10.1 Diagnostic Services

The required diagnostic services for the bootloader are specified in *[BL_CAN_4] UDS Services* with the restrictions/additions as defined by the subsequent sub-sections.

4.6.4.1.10.1.1 RequestDownload (34 hex) Service

The required diagnostic services for the bootloader are specified in *[BL_CAN_2] UDS Services* with the restrictions/additions as defined by the subsequent sub-sections.

3244 v2 **RequestDownload Service**

Version history:
(2):

Purpose: To define the diagnostic data used by the bootloader.
Source:
Verification Method: Test
Legacy ID: 52235v1

The maxNumberOfBlockLenght shall be dimensioned to be as big as possible. Minimum allowed maxNumberOfBlockLenght shall be 2kB.

4.6.4.1.11 Overview

This document specifies the CAN specific bootloader requirements that shall apply to a CAN based ECU. All diagnostic messages on CAN shall be sent and received according to this and referenced documents in section Requisite documents.

Additional requirements are added if the ECU shall act as a SWDL gateway to another network which is defined in *[BL_CAN_7] Diagnostic and Bootloader Gateway Specification*.

Note
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

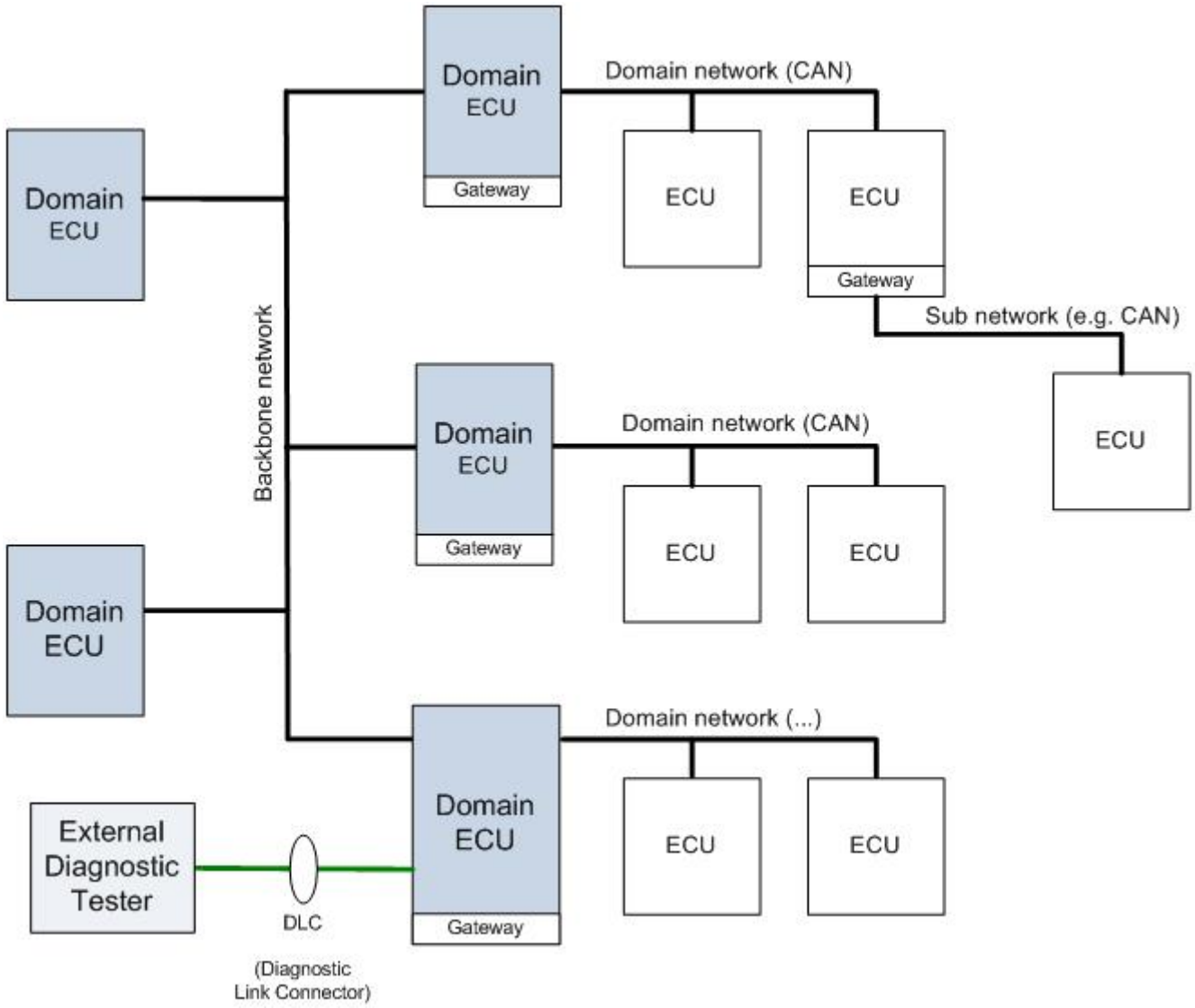


Figure - Example of the vehicle network topology.

4.6.5 Versatile Binary Format Specification

4.6.5.1 Versatile Binary Format Specification

4.6.5.1.1 Overview of the VBF file

4.6.5.1.1.1 Version Section

3505 v2 **Version section**

Version history:
(2):

Purpose: To identify the version of the VBF file.
Source: N/A
Verification Method: Test
Legacy ID: 64702v1

The first line of the VBF file shall indicate the version of the VBF file. No comments or non-white space characters are allowed before or after the expression. No white space characters are allowed before the expression. Note that this means the first byte of the VBF file shall always be 0x76 (i.e., ASCII 'v'). This specification is for version 2.6 of the VBF file, and so the first line shall always read:

vbf_version = 2.6;

Examples:

Valid Format:
1. vbf_version=2.6;
2. vbf_version = 2.6 ;

Invalid Format:
1. vbf_version=2.6;
2. /*Comment*/vbf_version/* Comment*/=/*Comment*/2.6/*Comment*/;
3. text vbf_version text= text 2.6 text; text

vbf_version is immediately followed by the header identifier with the exception of white space as described above. The header section as explained below contains all information needed to identify the VBF file and also information for the tester to perform a software download operation.

4.6.5.1.1.2 Header Section

3487 v2 **Header section**

Version history:
(2):

Purpose: To define the start of the header section.
Source: N/A
Verification Method: Test
Legacy ID: 64703v1

All the header information is contained in braces { }. The information between the braces in the header section shall consist entirely of expressions as defined in section 4.3. No comments or non-white space characters are allowed before **header** identifier or in between the **header** identifier and beginning braces ({).
Examples:

Valid Format:

```
1.      vbf_version = 2.6; header {
2.      vbf_version = 2.6;
        header{
3.      vbf_version=2.6;
        header
                {
```

Invalid Format:

```
1.      vbf_version=2.6;
        /*Comment*/
        header{
2.      header /*Comment*/ {
```

3488 v6 **Header section - Format**

Version history:
(6): Clarification

Purpose: To define the format of the header section
Source: N/A
Verification Method: Test
Legacy ID: 64705v5

The format of the header section is defined as follows (the grouping symbols [ ] indicates optional identifiers, i.e. the brackets are not to be part of the identifier defintion):

header {

[**description** = { *"text string row 1"* ,
"text string row 2" ,
"text string row 3" ,
:
"text string row n"
} ;]

sw_part_number = "part number";

sw_version = “version suffix”;

sw_part_type = *type*;

[**sw_current_part_number =** "installed part number";]
[**sw_current_version =** "installed version suffix";]

[**data_format_identifier** = *data compression and encryption method*;]

ecu_address = *ECU target address;*

[**erase** = { { *start address 1, length 1* },
{ *start address 2, length 2* },
:
{ *start address n, length n* }
} ;]

[**call** = *start address* **;**]

[**verification_block_root_hash** = *hash value of unprocessed data to be signed;*]
[**verification_block_start** = *start of the unprocessed verification block table;*]
[**verification_block_length** = *unprocessed length of the verification block table;*]
[**sw_signature_dev =** *software signature (test and development);*]
[**sw_signature** = *software signature;*]

[**parameter_settings** = { {"*parameter name 1", "value"*}, {"*parameter name n", "value"*} };]

file_checksum = *checksum*;
}

4.6.5.1.1.2.1 description

3491 v1 **description**

Version history:
(1):

Purpose: To define the format and use of the description identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64706v0

The **description** identifier is optional and can be used for a short description of the file. The **description** IdentifierValue may contain a maximum of 16 rows, with each row containing a maximum of 80 characters (bytes). Each row is contained in quotes (") and is terminated by a

comma (,) or by right brace (}) followed by semicolon (;) if the row is a last row. White space within quotes is not ignored. All of the description rows shall be contained within braces whether there is one row or multiple rows. No quotes (") are allowed within a row.

Examples:

Valid Format:

```
1.    description =    {"Created: 2002-03-14"};

2.    description =    { "SOP software for S60 AWD" ,
                       "Created: 2002-03-14"
                     };
```

Invalid Format:

```
1.    description =      " Created: 2002-03-14" ;                  // Missing braces

2.    description =      "SOP software for S60 AWD" ,               // Missing braces
                       "Created: 2002-03-14";

3.    description =    { SOP software for S60 AWD,                 // Missing quotes
                       Created: 2002-03-14
                     };
4.    description =    { "SOP software for "S60" AWD"};            // Extra quotes
```

4.6.5.1.1.2.2 sw_part_number

3497 v3 **sw_part_number**

Version history:
(3):

Purpose: To define the format and use of the sw_part_number identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64707v2

The **sw_part_number** IdentifierValue contains the vehicle manufacturer part number for identification of the software component (VBF file). The **sw_part_number** IdentifierValue shall be contained in quotes and shall consist of a maximum of 20 characters. No white space characters or comments are allowed within these quotes .

Examples:

Valid Format:

```
 1.     sw_part_number = "31808832";                                  // Part number
format
```

Invalid Format:

```
1.    sw_part_number = 31808832;                                     // Missing quotes
```

2. sw_part_number = "318088323322256789713"; // Maximum characters > 20

3. sw_part_number = " 31808832 "; // White Space within quotes

4. sw_part_number = "/*Comment*/31808832 /*Comment*/"; // Comments within quotes

4.6.5.1.1.2.3 sw_current_part_number

3495 v3 **sw_current_part_number**

Version history:
(3):

Purpose: To define the format and use of the sw_current_part_number identifier
Source:
Verification Method: Inspection
Legacy ID: 408537v2

The **sw_current_part_number** IdentifierValue contains the vehicle manufacturer part number for identification of the software component (VBF file) that is expected to be installed before a **delta** software update occurs.
This field is mandatory only for software updates with compression type **delta.**
The **sw_current_part_number** IdentifierValue shall be contained in quotes and shall consist of a maximum of 20 characters. No white space characters or comments are allowed within these quotes.

Examples:

Valid Format:
1. sw_current_part_number = "31808832"; // Part number format

Invalid Format:
1. sw_current_part_number = 31808832; // Missing quotes

2. sw_current_part_number = "318088323322256789713"; // Maximum characters > 20

3. sw_current_part_number = " 31808832 "; // White Space within quotes

4. sw_current_part_number = "/*Comment*/31808832 /*Comment*/"; // Comments within quotes

4.6.5.1.1.2.4 sw_version

3501 v1 **sw_version**

Version history:
(1):

Purpose: To define the format and use of the sw_version identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64708v0

The **sw_version** IdentifierValue contains the vehicle manufacturer version for identification of the software component (VBF file). The **sw_version** IdentifierValue shall be contained in quotes and shall consist of a maximum of 4 characters (bytes). No white space characters or comments are allowed within these quotes. **sw_version** IdentifierValue is case sensitive and shall only contain the uppercase characters A-Z.

Examples:

Valid Format:
1. sw_version = "ABC"; // Version suffix format

2. sw_version = “AA"; // Version suffix format

3. sw_version = “A”; // Version suffix format

Invalid Format:
1. sw_version = ABC; // Missing quotes

2. sw_part_number = "ABCDE"; // Maximum characters > 4

3. sw_part_number = " ABC "; // White Space within quotes

4. sw_part_number = "/*Comment*/ABC /*Comment*/"; // Comments within quotes

5. sw_version = aBC; // Lowercase character

6. sw_version = 1BC; // Numerical character

4.6.5.1.1.2.5 sw_current_version

3496 v2	sw_current_version

Version history:
(2):

Purpose: To define the format and use of the sw_current_version identifier
Source:
Verification Method: Inspection
Legacy ID: 408538v1

The **sw_current_version** IdentifierValue contains the vehicle manufacturer version for identification of the software component (VBF file) that is expected to be installed before a **delta** software update occurs.
This field is mandatory only for software updates with compression type **delta**.
The **sw_current_version** IdentifierValue shall be contained in quotes and shall consist of a maximum of 4 characters (bytes). No white space characters or comments are allowed within these quotes.
The **sw_current_version** IdentifierValue is case sensitive and shall only contain the uppercase characters A-Z.

Examples:

Valid Format:
1. sw_current_version = "ABC"; // Version suffix format

2. sw_current_version = “AA"; // Version suffix format

3. sw_current_version = “A”; // Version suffix format

Invalid Format:
1. sw_current_version = ABC; // Missing quotes

2. sw_current_version = "ABCDE"; // Maximum characters > 4

3. sw_current_version = " ABC "; // White Space within quotes

4. sw_current_version = "/*Comment*/ABC /*Comment*/"; // Comments within quotes

5. sw_current_version = aBC; // Lowercase character

6. sw_current_version = 1BC; // Numerical character

4.6.5.1.1.2.6 sw_part_type

3498 v5 **sw_part_type**

Version history:
(5): Clarification

Purpose: To define the format and use of the sw_part_type identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64709v5

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue Reserved Words	Description
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file
VARIANT_INFO	Settings file (for offboard tool settings)

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:
1. sw_part_type = SIGCFG ;

2. sw_part_type = EXE ; // or any other valid type but "SBL" or

```
"SSBL"
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             };
```

Invalid Format:

```
1.      sw_part_type = exe;                          //  Invalid case

2.      sw_part_type = SBL;                          // Invalid combination of sw_part_type and
erase
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             } ;

3.      sw_part_type = MYOWNTYPE;                    // Unknown IdentifierValue
```

4.6.5.1.1.2.7 data_format_identifier

3490 v4 **data_format_identifier**

Version history:
(4): Clarification

Purpose: To define the format and usage of the data_format_identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64712v3

The data_format_identifier is optional and when is presented with a non-zero value in the header section, it represents that a data compression/encryption method is used. Valid values for the data_format_identifier are one byte long with a range of 0x00 to 0xFF with upper nibble representing "Data Compression Method" and lower nibble representing "Data Encryption Method". Refer to REQPROD 459337: RequestDownload - dataFormatIdentifier for valid values. If the identifier is not present in the header section, it represents that neither compression nor encryption is used.

Example of Valid Format:

1. data_format_identifier = 0x00; // represents no compression and no encryption

2. data_format_identifier = 0x10; // represents compression method #1 and no encryption

3. data_format_identifier = 0x20; // represents compression method #2 and no encryption

Example of invalid Format:

1. data_format_identifier = 0x123; // Invalid format (must be from 0x00 – 0xFF)

2. data_format_identifier = {0x10}; // Extra braces

3. data_format_identifier = 23; // Invalid format (integers shall be hexadecimal)

4.6.5.1.1.2.8 ecu_address

3492 v1 **ecu_address**

Version history:
(1):

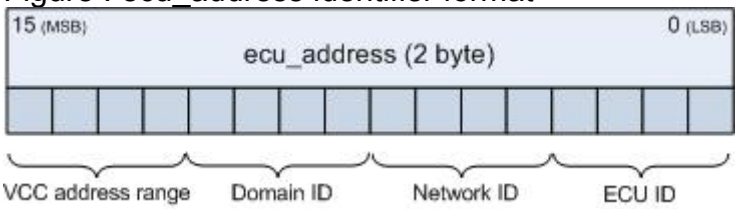
Purpose: To define the format and use of the ecu_address identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64714v0

The **ecu_address** identifier indicates the physical target address of the ECU. The **ecu_address** is divided into three parts where the combination of Domain ID, Network ID and ECU ID is utilized. The **ecu_address** depends on where in the network topology the ECU is placed.

The ecu_address length is two bytes long .

The addressing format and valid address ranges for a physical target address (TA) is defined in *[VBF_2] UDS Services*. See the figure below for an overview.

Figure : ecu_address identifier format



Examples:

Valid Format:
1. ecu_address = 0x1321; // Domain ID 0x3, Network ID 0x2, ECU ID 0x1

Invalid Format:
1. ecu_address = {0x0321}; // Extra braces

4.6.5.1.1.2.9 erase

3493 v3 **erase**

Version history:
(3):

Purpose: To define the format and use of the erase identifier
Source: N/A
Verification Method: Test
Legacy ID: 64715v2

The **erase** identifier indicates the range(s) of memory that needs to be erased before a software download operation can take place. The erase start address and length need not to be matched to start address and length of a flash sector. The erase length can span over more than one flash sector, or can be within a single flash sector. The erase start address and length can automatically be derived from a Motorola S-record file or an Intel Hex file during generation of a VBF file, but there is no guarantee that this will align with acceptable flash erase segment boundaries for a given SBL. The **erase** identifier is mandatory for memory types (i.e., flash) that need to be erased before programming. The erase start address and length are four bytes long with a range of 0x00000000 to 0xFFFFFFFF. The **erase** identifier values shall always be contained in two sets of braces whether there is one or more ranges present.

Note: If **sw_part_type = SBL or SSBL**, the VBF file shall not contain the **erase** identifier
Note: If delta encoding is used **(data_format_identifier = 0x80 or 0x90)** , the VBF file shall not contain the **erase** identifier

Examples:

Valid Format:

```
1.  erase =   {  { /*start address*/ 0x00004200, /*Length*/ 0x00003CBF }
          };

2.  erase =   {  { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
              };                          // Two blocks: 0x00004200-0x00007EBE, 0x00008010-
0x0000B002
```

Invalid Format:

```
1.  erase =   { 0x00004200, 0x00003CBF };                    // Missing outer braces

2.  erase =   { { 0xFFFFFFFE, 0x00000002 }                   // Address Overflow
            };

3.  erase =   { 0x00004200, 0x00003CBF },                    // Missing outer braces
            { 0x00008010, 0x00002FF3 };

4.  erase =   { 0x00004200, 0x00003CBF,                      // Missing inner braces
              0x00008010, 0x00002FF3
```

};

4.6.5.1.1.2.10 call

3489 v1 **call**

Version history:
(1):

Purpose: To define the format and use of the call identifier.
Source: N/A
Verification Method: Test
Legacy ID: 64716v0

The **call** entry is mandatory in VBF files with **sw_part_type** identifier values equal to SBL or SSBL and optional in VBF files with **sw_part_type** equal to TEST. For all other identifier values of **sw_part_type** , the **call** entry is not allowed. The **call** entry indicates the start address to a pre-defined executable function in ECU memory. The call identifier start address is four bytes long with a range of 0x00000000 to 0xFFFFFFFF.

Examples:

Valid Format:
1. sw_part_type = SBL;
   call = 0x00000100; // Start address for SBL

2. sw_part_type = SSBL;
   call = 0x00000100; // Start address for SSBL

3. sw_part_type = TEST;
   call = 0x00000100; // Start address for TEST

Invalid Format:
1. sw_part_type = SBL;
   With no **call** identifier

2. sw_part_type = SSBL;
   With no **call** identifier

3. sw_part_type = EXE: // or any other IdentifierValue but SBL, SSBL or TEST
   call = 0x00000100;

4.6.5.1.1.2.11 verification_block_start

3504 v1 **verification_block_start**

Version history:
(1):

Purpose: To define the format and use of the verification_block_start identifier. The purpose of the identifier is to identify the start address of the data (unprocessed) that is signed.

Source:

Verification Method: Test

Legacy ID: 417752v0

The **verification_block_start** represents the start address of the unprocessed data that is to be signed, according to the signing method *[General Software Authentication]*. The **verification_block_start** address is four bytes long integer with a range of 0x00000000 to 0xFFFFFFFF.

Example:

Valid Format:
1. verification_block_start = 0x0002FF00;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

4.6.5.1.1.2.12 verification_block_length

3502 v1 **verification_block_length**

Version history:
(1):

Purpose: To define the format and use of the verification_block_length identifier. The purpose of the identifier is to identify the length of the unprocessed data that is signed.

Source:

Verification Method: Test

Legacy ID: 417753v0

The **verification_block_length** represents the unprocessed length (prior to compression/encryption is applied) of the data that is to be signed, according to the signing method *[General Software Authentication]*. The **verification_block_length** is four bytes long integer with a range of 0x00000000 to 0xFFFFFFFF.

Example:

Valid Format:
1. verification_block_length = 0x00000054;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

4.6.5.1.1.2.13 verification_block_root_hash

3503 v2 **verification_block_root_hash**

Version history:
(2): Clarification

Purpose: To define the format and use of the verification_block_root_hash identifier. The purpose of the hash value is to provide the Trust Centre (signing entity) with required information of the unprocessed data (no compression/encryption method applied).

Source:

Verification Method: Test

Legacy ID: 417751v1

The verification_block_root_hash represents the hash value of the unprocessed data that is to be signed, according to the signing method [General Software Authentication]. The verification_block_root_hash shall be represented as an integer and the length is dependent of the hashing method used.

Example:

Valid Format:
1. verification_block_root_hash = 0xF1F62AC8EDFECD610234DBDA84F7561998ADC47A0F0EDE6CA23F0FC22974F9F4;

Note: There are use cases when a data file is not signed, e.g. prior to a file has been released, approved and signed.

4.6.5.1.1.2.14 sw_signature

3499 v3 **sw_signature**

Version history:
(3): Clarification

Purpose: To define the format and use of the sw_signature identifier. The purpose of the digital signature is to validate the authenticity of the programmed data in the ECU.

Source:

Verification Method: Test

Legacy ID: 417716v2

The sw_signature represents the digital signature according to the applied signature method [General Software Authentication]. The sw_signature shall be represented as an integer and the length is dependent of the signing method used.

Example:

Valid Format:

1. sw_signature = 0x514E3E55831E43E89D4118E2F29F4996C3A8ECBD11B1CDB8F4940AE4BF8BF866C28EA6F1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD963637203FAEA496F44C79EC11E706C3911749374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF34C7C297F6CFDAC7A436D6C0FA2B0551C032B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB47359A9D48F;

Note: There are use cases where a data file is not signed, e.g. prior to a file has been released, approved and signed.
Note: In production, e.g. at manufacturing and workshop systems, this information is to be transferred in a separate (non-vbf) file that is defined elsewhere.

4.6.5.1.1.2.15 sw_signature_dev

3500 v1 **sw_signature_dev**

Version history:
(1):

Purpose: To define the format and use of the sw_signature_dev identifier. The purpose of the digital signature is to validate the authenticity of the programmed data in the ECU for test & development objects.
Source:
Verification Method: Test
Legacy ID: 435002v0

The sw_signature_dev represents the digital signature for test and development according to the applied signature method *[General Software Authentication]*. The sw_signature_dev shall be represented as an integer and the length is dependent of the signing method used.

Example:

The software release candidate is signed with a dedicated key for development and the resulting digital signature is stored at the sw_signature_dev. A software successfully verified and ready for release (production) is signed with the dedicated key for production (append sw_signature), but the sw_signature_dev is still kept. Refer to sw_signature identifier for format example.

4.6.5.1.1.2.16 file_checksum

3494 v1 **file_checksum**

Version history:
(1):

Purpose: To define the format and use of the file_checksum identifier.
Source: N/A

Verification Method: Test
Legacy ID: 64718v0

A four-byte file checksum shall be provided in the header section with a range of 0x00000000 to 0xFFFFFFFF. The purpose of this checksum is to validate the integrity of the file itself prior to attempting a download. The checksum shall be a CRC32 of all the data contained in the data section of the file, including start address, length and checksum for all data blocks. This means that if the data bytes to download are compressed (i.e., data_format_identifier contains a non-zero value), then the file checksum is performed using the compressed data. The tester shall check this checksum against the file checksum it has calculated before the software download operation is performed with the VBF file.

The file checksum shall meet the following criteria:
· 32 bit CRC, polynomial = 0x04C11DB7
(x^32 + x^26 + x^23 + x^22 + x^16 + x^12 + x^11 + x^10 + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1)
· Initial value = 0xFFFFFFFF
· Bit reversed

For a fast CRC32 calculation a look-up table implementation is the preferred solution.
See Appendix B for "C-code" example.

Example:
file_checksum = 0x29058c73

Note: This is the same CRC32 algorithm as used by "WinZip" (among many others).

134130 v1 **file_checksum - no data blocks**

Version history:

Purpose: Define the file_checksum value for a data file with no data blocks. This is a special case when the data file is used as an information carrier to offboard tool-chain, i.e. no data downloaded to the ECU.
Source:
Verification Method: Test
Legacy ID: 465519v0

The file_checksum for a data file having no data blocks shall be 0xFFFFFFFF.

4.6.5.1.1.2.17 parameter_settings

134131 v1 **parameter_settings**

Version history:

Purpose: The file_checksum for a data file having no data blocks shall be 0xFFFFFFFF.

Source:

Verification Method: Test
Legacy ID: 445413v2

The optional **parameter_settings** identifier is used to define and carry configurations required by off-board tools. To be able to define multiple parameters using this identifier, a parameter is defined by a "name" and "value" pair.
Each "name" and "value" pair must be contained in two sets of braces and every pair is to be separated by a comma. A single element of each pair, i.e. the "name" and "value" respectively, must be quoted and separated by a comma. The overall parameter_settings definitions must be embraced by two outer braces whether there is one or more "name" and "value" pairs defined.

Example 1:
Valid format:

1. parameter_settings = {{"swAuthKeyAlgorithm", "RSA4096"}};
2. parameter_settings = {{"nameOne", "VALUE1"}, {"nameTwo", "VALUE2"}, {"nameThree", "VALUE3"}};

Invalid format:
1. parameter_settings = {"swAuthKeyAlgorithm", "RSA4096"}; //Missing outer braces
2. parameter_settings = {{"swAuthKeyAlgorithm"}}; //Incomplete pair

Example 2:

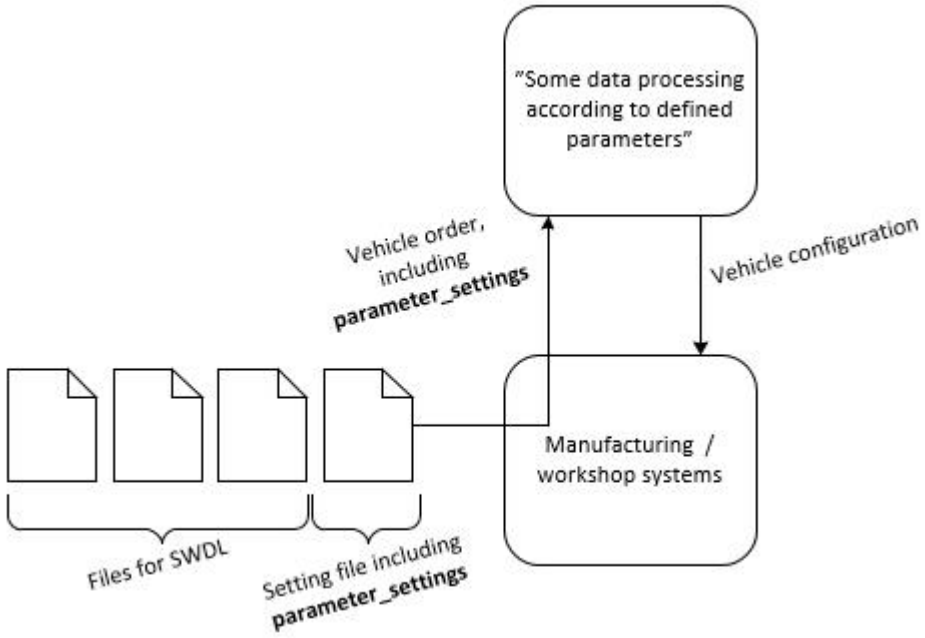


Figure - Usage of parameter_settings

The *Figure - Usage of parameter_settings* shows where off-board tools detects that some configuration is required and reacts according to the parameter_settings.
Notes: *This requirement does neither specify the applicable "name" and "value" pairs nor when the identifier is required. These are to be specified by the concept(s) requiring the parameter_settings identifier. For example, "nameOne", "VALUE1" shall only be considered as examples of the format and not valid element names. See e.g. "REQPROD 456934: Software Settings - Definition"*

4.6.5.1.1.2.18 Header section example #1

The first example is an VBF-file with **sw_part_type** = EXE.

Figure: Header section example #1:

```
vbf_version = 2.6;
header {

   description = { "Application SW DIM"};

  // Software part number
   sw_part_number = "11223344";

   // Software version
   sw_version = "AB";

   // Software type
   sw_part_type = EXE;

   // Compression and Encryption Method
   data_format_identifier = 0x10;

   // ECU address
   ecu_address = 0x1321;

   // Erase information
   // start length
   erase = {0x000200000, 0x00040000};

   //Verification block root hash value
   verification_block_root_hash =
0xFC10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x0005FF00;

   //Length of the verification block
   verification_block_length = 0x00000054;
```

```
   //Software signature for test and development purposes
   sw_signature_dev =
0x16956823884E89FBFB1AFB81055C43732E30B066DB24AD7B4D7BF8ABB4EA3C98F283
9C3443FF267A3937E49ED4E50E028C94FAD8B214930CEE37C18111D8548F6B3D706FE3
2E1CA5B4C245276A9396762233B17C9A4DE258EA7EA1DF1D8E1006E5B1FE1061441864
A9718A2F696A89615ABDC3013F1F46682A03FAAB92ADFD43858F6D159020B09632FF83
AD1F29525A43455CF21B8D6800CC88D439D87651295552B3D4414DF55D764172595B59
AF2F354D27F75661565798FABFEF6638E5BF92ABECEAC7F119E893C6BD238A464C1CC9
65E084690650D048AE51F43295857DF5B450F60C5C19D2699C385090651B94282C3FE5
1703F249DAB62644EEDC07EF;

   //Software signature
   sw_signature =
0x514E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28E
A6A1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115E
C00E37274066678AEFD54DD963637203FAEAA96F44C79EC11E706C3921F49374AED7BC
B7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A4
8B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF3467C
297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768A
D3B6CBDB222DB47359A9D48F;

   // File checksum
   file_checksum = 0x1A47EDA3;
}
<binary data section follows>
```

4.6.5.1.1.2.19 Header section example #2

The second example is a VBF-file with **sw_part_type** = SBL.

Figure: Header section example #2:

```
vbf_version = 2.6;
header {

   //This is a comment
   /*This is also a comment */

   //Short description of the VBF file
      description= {"SBL for DIM",
      "Created 2015-10-10"
   };
   //Software part number
   sw_part_number= "31888832";

   //Software version
   sw_version= "AB";

   //Software type
   sw_part_type= SBL;
```

```
   //Compression and Encryption Method
   data_format_identifier= 0x10;

   //ECU address
   ecu_address= 0x1123;

   //Start address for the SBL
   call= 0x00000100;

   //Verification block root hash value
   verification_block_root_hash =
   0xAA10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;


   //Start address of the verification block
   verification_block_start = 0x0001FF00;

   //Length of the verification block
   verification_block_length = 0x0000002C;

   //Software signature for test and development purposes
   sw_signature_dev =
   0x7EE874D0FFEBC4CA63AA3377AB3FACBC839690F903B96F30F4968D18101BDAD72
EDC343E6A0764469D47AAEFD92F416E7C44502CCD31274450DD5952E8F1C2B75BFFBC1
57C112AD1E7B4F46C99B89EED1BADB38D06C6B21036A55E0FF9B2EFFEC4DBB5856C7CD
946F432AC2E3CD41CE049D05C728F8C93EEF0AAA6703245ACD136BC3B33691781EFE67
430AEBCA99B0FBCF50CD1D30A9CC7DC4D81B35208788375727E0DD0106B46BBB80DB38
87DFD2D69DF0B4ADCD7BD69AF24C72F55E357457CB1D389A4976344417EF8A730BBB5D
4D8C22BA8D1258CEBEE9B218AF1D5D54328C47BF4D0FA8567F1AB123512BEE45828DE1
D4E3440DF182D1637737BA8C77D;

   //Software signature
   sw_signature =
   0xAC4E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C
28EA6A1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E5531
15EC00E37274066678AEFD54DD963637203FAEAA96F44C79EC11E706C3921F49374AED
7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC
0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF34
67C297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022
E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B997
68AD3B6CBDB222DB47359A9D48F;

   // File checksum
   file_checksum = 0xA1DC5A45;
}
<binary data section follows>
```

4.6.5.1.1.2.20 Header section example #3

A VBF-file containing delta encoded data, with **data_format_identifier = 0x80**. Note that the identifiers for Software Authentication, e.g. the sw_signature, are for the full target image (after delta enconding has been applied)

Figure: Header section example #3:

```
vbf_version = 2.6;
header {

   //Description
   description = {"SWLM (Delta) for SUM from Version 31429076AD to
31429077AA"
                  };
   //Software part number (New part number after this update)
   sw_part_number = "31429077";

   //Software version (New software version after this update)
   sw_version = "AA";

   //Current Software part number
      //(Expected part number, installed before this update)
   sw_current_part_number = "31429076";

   //Current Software version
      //(Expected software version, installed before this update)
   sw_current_version = "AD";

   //Software part type
   sw_part_type = DATA;

   //Data Format Identifier
   //Delta encoded data and no compression method
   data_format_identifier = 0x80;

   //ECU address
   ecu_address= 0x1123;

   //Verification block root hash value
   verification_block_root_hash =
0xAA10397BEABFBFF17519DCC6144EB4D86429CCC5682D8ED35AF8FBA984F26ED2;

   //Start address of the verification block
   verification_block_start = 0x0001FF00;

   //Length of the verification block
   verification_block_length = 0x00000054;

   //Software signature for test and development purposes
   sw_signature_dev =
0x073A43404527ABFC1C70F637FBD689A83B0C14172C6C588268F80F7021CD82DF9489
C676452B7B1BB85D538B670F4E14FA71933AD1612E75F66574955B82755B0FF1A29D4D
5A9421EED6D9E705098CCD5F4E2A71C526D8DD90AF02212EB715937A3D49311B385E1C
3B754C90F0EFE20BB22240582A8FD03BF6FF04B59C114AA7B07D8BDCCBCEC77A8E00CC
C9D1CE18EC446A8DA45756CF3CFD594134859E407C7421D50221B1D71A1976AA68E36D
5642FA762B1DAFBB9804FFD8C479CA037307D8AC522AC4CB946248064E5A789797B1B5
5760EAC40FBC7E616F848C8A4B389899FB8DEF7DDCA4933E2F11C1638D05F66144AEC3
```

```
0E8DBAF2317827DB27438560;

   //Software signature
   sw_signature =
0xAC4E3EB5831E43E89D5118E2F29F4996C3A8ECBD11B4CDB8F4940AE4BF8BC866C28E
A6A1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115E
C00E37274066678AEFD54DD963637203FAEAA96F44C79EC11E706C3921F49374AED7BC
B7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A4
8B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF3467C
297F6CFDAC7A136D6C0FA2B0551C038B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93
F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768A
D3B6CBDB222DB47359A9D48F;

   // File checksum
   file_checksum = 0xA1DC5A45;
}
<binary data section follows>
```

4.6.5.1.1.2.21 Header section example #4

The fourth example is an VBF-file with sw_part_type = SIGNED, i.e. this data file itself is not signed but used to transfer a digital signature (legacy Software Authentication concept).

Figure: Header section example #4:

```
vbf_version = 2.6;
header {

   description = { "Digital Signature"};

   // Software part number
   sw_part_number = "22223344";

   // Software version
   sw_version = "AB";

   // Software type
   sw_part_type = SIGNED;

   // Compression and Encryption Method
   data_format_identifier = 0x10;

   // ECU address
   ecu_address = 0x1321;

   // Erase information
   // start length
   erase = {0x000100000, 0x00040000};

   // File checksum
   file_checksum = 0x2A47EDA3;
}
<binary data section follows>
```

Note that there are also other use cases where a data file is not signed, for example prior to it has been released, approved and signed by the Trust Centre.

4.6.5.1.1.3 Data Section

3483 v2 **Data section - start**

Version history:
(2): Clarification

Purpose: To define the start of the data section.
Source: N/A
Verification Method: Test
Legacy ID: 64725v1

After the header section ends, the data section immediately follows. The header section ends with a } character (ASCII 0x7D). The binary data section begins with the next byte immediately following this ASCII } character. The data section might be omitted, i.e. a data file could contain zero data blocks for some special cases.

3485 v1 **Software part version in the data section**

Version history:
(1):

Purpose: To prevent resistant software part version stored in the ECU.
Source: N/A
Verification Method: Test
Legacy ID: 64726v0

The software part version containing sw_part_number and sw_version shall reside somewhere in the data section in the VBF file that is being programmed into the ECU and this downloaded part number shall be readable using service 0x22 (ReadDataByIdentifier).

3484 v2 **Data section - structure**

Version history:
(2):

Purpose: To define the structure of the data section.
Source: N/A
Verification Method: Test
Legacy ID: 64727v1

The data section may contain more than one block of data, and every block includes four parts.

1. 4-byte start address, physical address in ECU memory, with a range of 0x00000000 to 0xFFFFFFFF.
2. 4-byte length, number of data bytes in block (excluding Start address, Length and Checksum), with a range of 0x00000001 to 0xFFFFFFFF. If data compression is used, this length represents the compressed length.

3. Data
4. 2-byte checksum for the data block (excluding Start address and Length) with a range of 0x0000 to 0xFFFF.

The Start address, Length, and Checksum fields shall always be stored with the most significant byte (msb) first.

Table: Data Section

Version Section	VBF Version Number	Text	vbf_version=2.6;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	:	:	:
	Block n	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

4.6.5.1.1.3.1 Checksum calculation

3486 v3 **Data block checksum**

Version history:
(3): Clarification

Purpose: To define the format and use of the data block checksum.
Source: N/A
Verification Method: Test
Legacy ID: 64728v3

The VBF file shall include a 2-byte checksum for every data block. The 2-byte checksum shall be calculated including all data bytes (excluding Start address, Length, and Checksum) in the data block. If a data compression method is used, the data block checksum shall be calculated using unprocessed data (i.e. uncompressed and/or not encrypted). Block checksums should not be used to verify the integrity of the VBF file.

The checksum algorithm to be used shall be the CRC16-CITT:

- Polynomial = 0x1021 (x^16+x^12+x^5+1)
- Initial value = 0xFFFF

For a fast CRC16-CITT calculation a look-up table implementation is the preferred solution. See Appendix A for "C-code" example.

4.6.5.1.2 Appendix A

4.6.5.1.2.1 CRC16-CITT C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
unsigned int CalcCRC(unsigned int size,unsigned char *data);

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
};

unsigned int crctab[256] =
{
   0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7,
   0x8108, 0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF,
   0x1231, 0x0210, 0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6,
   0x9339, 0x8318, 0xB37B, 0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE,
   0x2462, 0x3443, 0x0420, 0x1401, 0x64E6, 0x74C7, 0x44A4, 0x5485,
   0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE, 0xF5CF, 0xC5AC, 0xD58D,
   0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6, 0x5695, 0x46B4,
   0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D, 0xC7BC,
   0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
   0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B,
   0x5AF5, 0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12,
   0xDBFD, 0xCBDC, 0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A,
   0x6CA6, 0x7C87, 0x4CE4, 0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41,
   0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD, 0xAD2A, 0xBD0B, 0x8D68, 0x9D49,
   0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13, 0x2E32, 0x1E51, 0x0E70,
   0xFF9F, 0xEFBE, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A, 0x9F59, 0x8F78,
   0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E, 0xE16F,
   0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
   0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E,
   0x02B1, 0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256,
   0xB5EA, 0xA5CB, 0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D,
   0x34E2, 0x24C3, 0x14A0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
   0xA7DB, 0xB7FA, 0x8799, 0x97B8, 0xE75F, 0xF77E, 0xC71D, 0xD73C,
   0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657, 0x7676, 0x4615, 0x5634,
   0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9, 0xB98A, 0xA9AB,
   0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882, 0x28A3,
   0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABBB, 0xBB9A,
   0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92,
   0xFD2E, 0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9,
```

```
   0x7C26, 0x6C07, 0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1,
   0xEF1F, 0xFF3E, 0xCF5D, 0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8,
   0x6E17, 0x7E36, 0x4E55, 0x5E74, 0x2E93, 0x3EB2, 0x0ED1, 0x1EF0
};

/* driver */
void main(void)
{
   unsigned int crc;

   crc=CalcCRC(256,data);

   while(1); /* the result of the above calculation shall be: crc=0x3FBD */
}


/* Calculate CRC */
unsigned int CalcCRC(unsigned int size, unsigned char *data)
{
   unsigned int crc=0xffff; /* initial value */
   int tmp;
   int i;

   for(i=0;i<size;i++) {
      tmp=(crc>>8)^data[i];
      crc=(crc<<8)^crctab[tmp];
   }
   return crc;
}
```

4.6.5.1.3 Appendix B

4.6.5.1.3.1 CRC32 C-code example

This example uses a look-up table with pre-calculated CRCs for fast calculation.

```
/* function declarations */
void generate_crc32_table(void);
unsigned long calc_crc32(unsigned long size, unsigned char *data);

/* variables */
unsigned long crc32;
static unsigned long crc32_table[256];

/* test data array */
unsigned char data[256] =
{  0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,
   0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f,
   0x20,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2a,0x2b,0x2c,0x2d,0x2e,0x2f,
   0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3a,0x3b,0x3c,0x3d,0x3e,0x3f,
   0x40,0x41,0x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4a,0x4b,0x4c,0x4d,0x4e,0x4f,
   0x50,0x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5a,0x5b,0x5c,0x5d,0x5e,0x5f,
   0x60,0x61,0x62,0x63,0x64,0x65,0x66,0x67,0x68,0x69,0x6a,0x6b,0x6c,0x6d,0x6e,0x6f,
   0x70,0x71,0x72,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7a,0x7b,0x7c,0x7d,0x7e,0x7f,
   0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,
   0x90,0x91,0x92,0x93,0x94,0x95,0x96,0x97,0x98,0x99,0x9a,0x9b,0x9c,0x9d,0x9e,0x9f,
   0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,
   0xb0,0xb1,0xb2,0xb3,0xb4,0xb5,0xb6,0xb7,0xb8,0xb9,0xba,0xbb,0xbc,0xbd,0xbe,0xbf,
   0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,
   0xd0,0xd1,0xd2,0xd3,0xd4,0xd5,0xd6,0xd7,0xd8,0xd9,0xda,0xdb,0xdc,0xdd,0xde,0xdf,
   0xe0,0xe1,0xe2,0xe3,0xe4,0xe5,0xe6,0xe7,0xe8,0xe9,0xea,0xeb,0xec,0xed,0xee,0xef,
   0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff
```

```
};


/* driver */
void main(void)
{
   unsigned long crc32;

   generate_crc32_table();
   crc32=calc_crc32(256,data);

   while(1); /* the result of the above calculation shall be: crc32=0x29058c73 */
}


/* calculate the crc32 value */
unsigned long calc_crc32(unsigned long size, unsigned char *data)
{

   unsigned long i, crc;

   crc=0xffffffff;       // initial value

   for(i=0;i<size;i++) {
      crc = ((crc>>8)&0x00FFFFFF)^crc32_table[(int)(crc^data[i])&0xff];
   }

   return ~crc;
}

#define CRC32_POLY 0xedb88320 // 0x04c11db7 bit reversed

/* generate the lock-up table */
void generate_crc32_table(void)

{
   int i, j;
   unsigned long crc;

   for(i=0;i<256;i++) {
      crc=i;
      for(j=8;j>0;j--) {
         if(crc&1) {
            crc=(crc>>1)^CRC32_POLY;
         }
         else{
            crc>>=1;
         }
      }
      crc32_table[i]=crc;
   }
}

// As an alternative to the "generate_crc32_table()" function the following table
can be used

/*
static unsigned long crc32_tab[256] =
{  0x00000000L, 0x77073096L, 0xee0e612cL, 0x990951baL, 0x076dc419L, 0x706af48fL,
   0xe963a535L, 0x9e6495a3L, 0x0edb8832L, 0x79dcb8a4L, 0xe0d5e91eL, 0x97d2d988L,
   0x09b64c2bL, 0x7eb17cbdL, 0xe7b82d07L, 0x90bf1d91L, 0x1db71064L, 0x6ab020f2L,
   0xf3b97148L, 0x84be41deL, 0x1adad47dL, 0x6dddde4ebL, 0xf4d4b551L, 0x83d385c7L,
```

```
    0x136c9856L, 0x646ba8c0L, 0xfd62f97aL, 0x8a65c9ecL, 0x14015c4fL, 0x63066cd9L,
    0xfa0f3d63L, 0x8d080df5L, 0x3b6e20c8L, 0x4c69105eL, 0xd56041e4L, 0xa2677172L,
    0x3c03e4d1L, 0x4b04d447L, 0xd20d85fdL, 0xa50ab56bL, 0x35b5a8faL, 0x42b2986cL,
    0xdbbbc9d6L, 0xacbcf940L, 0x32d86ce3L, 0x45df5c75L, 0xdcd60dcfL, 0xabd13d59L,
    0x26d930acL, 0x51de003aL, 0xc8d75180L, 0xbfd06116L, 0x21b4f4b5L, 0x56b3c423L,
    0xcfba9599L, 0xb8bda50fL, 0x2802b89eL, 0x5f058808L, 0xc60cd9b2L, 0xb10be924L,
    0x2f6f7c87L, 0x58684c11L, 0xc1611dabL, 0xb6662d3dL, 0x76dc4190L, 0x01db7106L,
    0x98d220bcL, 0xefd5102aL, 0x71b18589L, 0x06b6b51fL, 0x9fbfe4a5L, 0xe8b8d433L,
    0x7807c9a2L, 0x0f00f934L, 0x9609a88eL, 0xe10e9818L, 0x7f6a0dbbL, 0x086d3d2dL,
    0x91646c97L, 0xe6635c01L, 0x6b6b51f4L, 0x1c6c6162L, 0x856530d8L, 0xf262004eL,
    0x6c0695edL, 0x1b01a57bL, 0x8208f4c1L, 0xf50fc457L, 0x65b0d9c6L, 0x12b7e950L,
    0x8bbeb8eaL, 0xfcb9887cL, 0x62dd1ddfL, 0x15da2d49L, 0x8cd37cf3L, 0xfbd44c65L,
    0x4db26158L, 0x3ab551ceL, 0xa3bc0074L, 0xd4bb30e2L, 0x4adfa541L, 0x3dd895d7L,
    0xa4d1c46dL, 0xd3d6f4fbL, 0x4369e96aL, 0x346ed9fcL, 0xad678846L, 0xda60b8d0L,
    0x44042d73L, 0x33031de5L, 0xaa0a4c5fL, 0xdd0d7cc9L, 0x5005713cL, 0x270241aaL,
    0xbe0b1010L, 0xc90c2086L, 0x5768b525L, 0x206f85b3L, 0xb966d409L, 0xce61e49fL,
    0x5edef90eL, 0x29d9c998L, 0xb0d09822L, 0xc7d7a8b4L, 0x59b33d17L, 0x2eb40d81L,
    0xb7bd5c3bL, 0xc0ba6cadL, 0xedb88320L, 0x9abfb3b6L, 0x03b6e20cL, 0x74b1d29aL,
    0xead54739L, 0x9dd277afL, 0x04db2615L, 0x73dc1683L, 0xe3630b12L, 0x94643b84L,
    0x0d6d6a3eL, 0x7a6a5aa8L, 0xe40ecf0bL, 0x9309ff9dL, 0x0a00ae27L, 0x7d079eb1L,
    0xf00f9344L, 0x8708a3d2L, 0x1e01f268L, 0x6906c2feL, 0xf762575dL, 0x806567cbL,
    0x196c3671L, 0x6e6b06e7L, 0xfed41b76L, 0x89d32be0L, 0x10da7a5aL, 0x67dd4accL,
    0xf9b9df6fL, 0x8ebeeff9L, 0x17b7be43L, 0x60b08ed5L, 0xd6d6a3e8L, 0xa1d1937eL,
    0x38d8c2c4L, 0x4fdff252L, 0xd1bb67f1L, 0xa6bc5767L, 0x3fb506ddL, 0x48b2364bL,
    0xd80d2bdaL, 0xaf0a1b4cL, 0x36034af6L, 0x41047a60L, 0xdf60efc3L, 0xa867df55L,
    0x316e8eefL, 0x4669be79L, 0xcb61b38cL, 0xbc66831aL, 0x256fd2a0L, 0x5268e236L,
    0xcc0c7795L, 0xbb0b4703L, 0x220216b9L, 0x5505262fL, 0xc5ba3bbeL, 0xb2bd0b28L,
    0x2bb45a92L, 0x5cb36a04L, 0xc2d7ffa7L, 0xb5d0cf31L, 0x2cd99e8bL, 0x5bdeae1dL,
    0x9b64c2b0L, 0xec63f226L, 0x756aa39cL, 0x026d930aL, 0x9c0906a9L, 0xeb0e363fL,
    0x72076785L, 0x05005713L, 0x95bf4a82L, 0xe2b87a14L, 0x7bb12baeL, 0x0cb61b38L,
    0x92d28e9bL, 0xe5d5be0dL, 0x7cdcefb7L, 0x0bdbdf21L, 0x86d3d2d4L, 0xf1d4e242L,
    0x68ddb3f8L, 0x1fda836eL, 0x81be16cdL, 0xf6b9265bL, 0x6fb077e1L, 0x18b74777L,
    0x88085ae6L, 0xff0f6a70L, 0x66063bcaL, 0x11010b5cL, 0x8f659effL, 0xf862ae69L,
    0x616bffd3L, 0x166ccf45L, 0xa00ae278L, 0xd70dd2eeL, 0x4e048354L, 0x3903b3c2L,
    0xa7672661L, 0xd06016f7L, 0x4969474dL, 0x3e6e77dbL, 0xaed16a4aL, 0xd9d65adcL,
    0x40df0b66L, 0x37d83bf0L, 0xa9bcae53L, 0xdebb9ec5L, 0x47b2cf7fL, 0x30b5ffe9L,
    0xbdbdf21cL, 0xcabac28aL, 0x53b39330L, 0x24b4a3a6L, 0xbad03605L, 0xcdd70693L,
    0x54de5729L, 0x23d967bfL, 0xb3667a2eL, 0xc4614ab8L, 0x5d681b02L, 0x2a6f2b94L,
    0xb40bbe37L, 0xc30c8ea1L, 0x5a05df1bL, 0x2d02ef8dL
};
*/
```

4.6.5.1.4 Overview

4.6.5.1.4.1 Requisite documents

Reference number	Source	Title
[VBF_1]	Internal	Data Compression and Encryption Specification
[VBF_2]	Internal	UDS Services
[VBF_3]	Internal	General Software Authentication

Note:
Internal references are typically available as separate chapters in ECU specific Base Technology SWRS.

4.6.5.1.4.2 Definitions and Abbreviations

Definition	Description
Domain ID	Part of the ECU Source Address (SA), states which Domain the ECU is

	physical connected to.
Network ID	Part of the ECU Source Address (SA), states which Network the ECU is physical connected to.
ECU ID	Part of the ECU Source Address (SA).
Tester	A system that controls functions such as test, inspection, monitoring, or diagnosis of an in-vehicle electronic control unit and may be dedicated to a specific type of operators (e.g. a scan tool dedicated to garage mechanics or a test tool dedicated to assembly plant agents). The tester is also referenced as the client.

Abbreviation	Description
CM	CoMments
ECU	Electronic Control Unit
PBL	Primary BootLoader
SBL	Secondary BootLoader
SSBL	Special Secondary BootLoader
VBF	Versatile Binary Format
WS	White Space

4.6.5.1.5 Versatile Binary Format (VBF)

4.6.5.1.5.1 VBF version

This document specifies VBF version 2.6 of the Versatile Binary Format.

4.6.5.1.5.2 Introduction

The VBF file contains three parts: VBF version section, a header section and a data section. VBF version section specifies the version number of the VBF specification that the file is compliant with. VBF version uses ASCII representation. The header section contains general information relating to file usage, setup, etc. The header section uses ASCII representation. The data section contains the actual data that is to be transferred to the target ECU. The data section uses binary representation.

Table: An Overview of the Versatile Binary Format

Name	Description	Type	Data
Version Section	VBF Version Number	Text	vbf_version=2.6;
Header section	Header	Text	n bytes, ASCII
Data section	Block 1	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary
	Block 2	Start address	4 bytes, binary
		Length	4 bytes, binary
		Data	n bytes, binary
		Checksum	2 byte, binary

:	:	:
Block n	Start address	4 bytes, binary
	Length	4 bytes, binary
	Data	n bytes, binary
	Checksum	2 byte, binary

4.6.5.1.5.3 File Naming

3506 v3 **File Naming**

Version history:
(3):

Purpose: To identify the release of the VBF file.
Source: N/A
Verification Method: Test
Legacy ID: 64686v2

All delivered or released VBF files shall be named the same as the corresponding released software part number which consist of (sw_part_number and sw_version) with an extension of ".VBF

Examples:
A VBF file with a header entry of
sw_part_number = "31808832";
sw_version = "AB";
shall utilize the file name of 31808832AB.VBF

A VBF file with a header entry of
sw_part_number = "31808832";
sw_version = "A";
shall utilize the file name of 31808832A.VBF

Delta files shall be released with a filename reference to its source and target version by including the software part number and software version of source and target in the delta filename.
Example:
sw_part_number = "31808832"; // Target version part no
sw_version = "AB"; // Target version
sw_current_part_number = "31808831"; // Source/current version part no
sw_current_version = "A";

The format of a delta filename shall be: <sw_current_part_number><sw_current_version><separator><sw_part_number><sw_version>.VBF

A VBF file containing delta and the header data above shall have the name:
31808831A _31808832AB.VBF

4.6.5.1.5.4 VBF file generation

3507 v1 **VBF file generation**

Version history:
(1):

Purpose: To generate the VBF file.
Source: N/A
Verification Method: Test
Legacy ID: 64690v0

The VBF Convert tool shall generate VBF files from Motorola S-record or Intel Hex formatted files.

4.6.5.1.6 Notation and lexical elements

The VBF file header section uses some basic lexical elements, which are described, in this section. Also, the notation for describing the syntax is given here.

4.6.5.1.6.1 Notation

Words in bold indicate identifiers, which are special tokens.

Italicized words indicate a lexical item, and are described in text format.

The grouping symbols [ ], brackets, indicate that the contents are optional.

The grouping symbols { }, braces, group a set of items.

The grouping symbols ( ), parentheses, contain a set of items separated by |, then one of the items must appear.

4.6.5.1.6.2 General structure

3477 v2 **Header expression termination**

Version history:
(2):

Purpose:
Source: N/A
Verification Method: Test
Legacy ID: 64693v1

The header consists of sequences of expressions. An expression is made up of identifiers, reserved words, constants, integers and is always terminated with a semicolon (;).

Expressions containing multiple values are grouped into nested sections using matching brace characters “{“ and “}”. Data shall be contained in braces only if there is more than one set of

data with the exception of description [see section *Header section - description*] and erase [see section *Header section - erase*] identifiers. In these cases, a single set of data shall also be contained in braces. Refer to section *Notation and lexical elements - Identifiers, reserved words, and expressions* for general format requirements for expressions and to section *Overview of the VBF file* for detailed format requirements for each identifier.

4.6.5.1.6.3 Identifiers, reserved words and expressions

3478 v5 **Header Identifiers**

Version history:
(5): Clarification

Purpose: To define allowed header identifiers
Source: N/A
Verification Method: Test
Legacy ID: 64694v4

Identifier is the name of a variable and IdentifierValue is the data assigned to the variable. Allowed identifiers are **vbf_version**, **header**, **description**, **sw_part_number**, **sw_current_part_number**, **sw_version**, **sw_current_version**, **sw_part_type**, **data_format_identifier**, **ecu_address**, **erase**, **call**, **verification_block_start**, **verification_block_length**, **verification_block_root_hash**, **sw_signature_dev**, **sw_signature, parameter_settings** and **file_checksum**. Duplicates of identifiers are not allowed.

3479 v1 **IdentifierValue format**

Version history:
(1):

Purpose: To define the format of the IdentifierValue.
Source: N/A
Verification Method: Test
Legacy ID: 64695v0

Each element within an IdentifierValue shall either be an integer, real, string embedded within quotes, or a reserved IdentifierValue (reserved word). Each identifier has a specific format allowed for its identifier values as defined in section 5 . The lexical rules for identifiers and reserved words are identical to those for “C” programming language with upper- and lower-case always distinguished.

Unless otherwise specified, a valid expression shall always consist of the following format (where WS/CM is defined as any combination of white space characters and complete comments).

[WS/CM]identifier[WS/CM]=[WS/CM]IdentifierValue[WS/CM];[WS/CM]

When the IdentifierValue is enclosed within matching brace characters "{" and "}" as allowed by this specification, the format of the IdentifierValue (including braces) shall be as follows :

{[WS/CM]IdentifierValue[WS/CM][,[WS/CM]IdentifierValue[WS/CM]]} ;

where the optional field of [,[WS/CM]IdentifierValue[WS/CM]] may occur multiple times .

Example:
sw_part_type = CARCFG;

In this example, **sw_part_type** is an identifier, and CARCFG is the reserved word value of the identifier **sw_part_type** .

4.6.5.1.6.4 Integers

3480 v2 **Integers**

Version history:
(2):

Purpose: Clarification of the integer format definition used by the VBF file.
Source: N/A
Verification Method: Test
Legacy ID: 64696v1

An integer is a sequence of digits. It shall be represented in hexadecimal format, start with the prefix “0x” and the following characters must be chosen from the string “0123456789abcdefABCDEF”.

Examples:

Valid Format:
1. 0xFF //Hexadecimal
2 0x12345678 //Hexadecimal, 32 bits representation

Invalid Format:
1. 0b11111111 //Binary
2. 255 //Decimal
3. 0x12 0x34 0x56 0x78 //Each byte separated with prefix “0x”.

4.6.5.1.6.5 Comments

3476 v1 **Comments**

Version history:
(1):

Purpose: To define the usage and format of comments within the VBF file.
Source: N/A
Verification Method: Test
Legacy ID: 64698v0

Comments are allowed in the header section only and may appear within an expression as described in section *Identifiers, reserved words, and expressions.* Valid comments shall always

be ignored by a VBF parser. Comments can be grouped together by “/*” and “*/” and stretch over several lines (C-style), or they can start with “//” and stretch to the end of the line (C++ style). Neither bracket syntax nests. It is recommended that C++-style comments be used wherever possible, since it is much more difficult to accidentally comment out more than was intended (as compared to using the C-style syntax).

4.6.5.1.6.6 White Space Characters

3482 v1 **White space characters**

Version history:
(1):

Purpose: To define allowed white space characters.
Source: N/A
Verification Method: Test
Legacy ID: 64699v0

Blank Space, Horizontal Tab (or Tab), Line Feed, Carriage Return, Form Feed, and Vertical Tab characters are defined as white space characters for the purpose of this specification. No other white space characters are allowed. A VBF parser shall always ignore valid white space characters unless otherwise specified in this document.

Table : ASCII Representation of White Space Characters

	Description	Symbol	Hex Value	Binary Value
1	Horizontal Tab (or Tab)	HT	09	00001001
2	Line Feed	LF	0A	00001010
3	Vertical Tab	VT	0B	00001011
4	Form Feed	FF	0C	00001100
5	Carriage Return	CR	0D	00001101
6	Blank Space	SPACE	20	00100000

4.6.5.1.6.7 Non-printing Characters

3481 v1 **Non-printing characters**

Version history:
(1):

Purpose: To define the range of non-printing characters allowed.
Source: N/A
Verification Method: Test
Legacy ID: 64701v0

0x00 to 0x1F represents non-printing characters. All Non-printing characters, except white space characters, are not allowed in VBF file header. For white space non-printing (0x09 to 0x0D) character handling, refer to section *White Space Characters*

4.7 Software Platform

4.7.1 Autosar Configuration

4.7.1.1 Autosar Configuration

4.7.1.1.1 E2E PROTECTION CONCEPT

4.7.1.1.1.1 E2E Protection Introduction

The E2E-protection mechanism within AUTOSAR is a methodology in which safety-related functions need to exchange safety-related information between SWCs. Such safety-related information most often consists of a set of signals, e.g. exchange of a sensor value together with its quality factor.

In E2E-protection there is a strict 1:n relationship, i.e. there is one sending SWC and one or more receiving SWCs. The receiving SWCs can be distributed across the network i.e. E2EProtected data need to be sent over the Network. The receiving SWC shall be able to choose whether to use or ignore the E2E-protection mechanism depending on its safety-classification.

To be able to detect errors in the communication, the safety-related signals are extended by additional signals, i.e. CRC checksum and a sequence counter. All data is grouped together within a signal group which is then transmitted over the network. To be able to “unpack” the safety-related data on a different ECU in a network, the description how to calculate the checksum is needed. These descriptions are called profiles in the AUTOSAR specifications.

The AUTOSAR standard defines three different ways to invoke the EndToEnd library, via ComCallout, Wrapper and Transformer. This specification allows two of these ways, i.e. Wrapper and Transformer.

4.7.1.1.1.2 E2E Protection Requirements

3510 v1 **Simultaneous use of Profiles and Transformers**

Version history:
(1):

Purpose: E2E Protection realization inside an ECU must not prevent simultaneous use of different E2E profiles and different ways to invoke the E2E library.

Source:

Verification Method: Inspection

Legacy ID: 435535v0

An ECU shall be able to handle multiple combinations of E2E Profiles, Serializing Transformers and E2E Wrappers simultaneously.

3511 v1 **Use of Profiles and Transformers**

Version history:
(1):

Purpose: E2E Protection shall be realized using methods and modules as specified by AUTOSAR.
Source:
Verification Method: Inspection
Legacy ID: 435534v0

E2E protection shall be realized using AUTOSAR modules:

Profile 1A
ComXf (ref: "AUTOSAR Specification of ComBasedTransformer") and **E2EXf** (ref: "AUTOSAR Specification of E2ETransformer").

Note: If the chosen AUTOSAR BSW version is not supporting ComXf and E2EXf, E2E protection shall be realized using an **E2E Wrapper** (ref: "AUTOSAR Specification of E2ELibrary").

Profile 4A
SomeIpXf (ref: "AUTOSAR Specification of SOMEIPTransformer") and **E2EXf** (ref: "AUTOSAR Specification of E2ETransformer").

Profile 5
ComXf (ref: "AUTOSAR Specification of ComBasedTransformer") and **E2EXf** (ref: "AUTOSAR Specification of E2ETransformer").

Profile VCC
E2E protection shall be realized using an **E2E Wrapper** (ref: "AUTOSAR Specification of E2ELibrary" and "Profile VCC requirements").

Note: Profiles and transformers are defined per signal and communicated to supplier via the ECU Extract.

4.7.1.1.1.2.1 Profile VCC Requirements

3512 v4 **E2E CRC Calculation**

Version history:
(4):

Purpose: Defines how to calculate CRC.
Source:
Verification Method: Test
Legacy ID: 181435v3

To use the E2E_P01Protect function, the data that should be computed shall be packed in an

array of uint8s. To get the same result in both sender and receiver(s), the elements need to be packed in the same order in both sides.

The array sent to E2E_P01Protect shall have the following order:

1. Index 0: CRC (8-bit)
2. Index 1: 0x0 + 4-bit counter (4 bit counter extended to 8 bits using 0 as padding)
3. Index 2: data 1
4. Index n: data 2

The data (data 1, data 2) shall be ordered alphabetically by the name of the data-element variable.

Note: The data-element variable is not referring to system signals, i.e. data shall not be sorted using the names of the system signal. Data-element variable is referring to VariableDataPrototype -> ApplicationRecordDataType -> ApplicationRecordElement. The data shall be sorted by the names of the ApplicationRecordElements.

3513 v3 E2E Data-Type Size

Version history:
(3):

Purpose: Defines the size that shall be used when calculating checksums.
Source:
Verification Method: Test
Legacy ID: 181434v2

To be able to calculate the E2E checksum in the same way in all ECUs in the network, the same size of the E2E data-types shall be used. The size to be used is the size that is defined by the System Signal. In addition, the E2E_P01Protect function is expecting an array of uint8s and therefore all data shall be extended to the closest upper 8-bit boundary i.e. a two bit signal shall be represented with 8-bits during the CRC-calculations. Unused bits shall be set to 0 (zero).

If the data-type is larger than 8-bits, the little-endian method shall be used to pack the data into an 8-bit array.

Note: The size of the E2E data type is defined by the System Signal. For signed signals, if any, the sign bit is included in the length of the System Signal, i.e. signed bit shall not be seen as an unused bit.

3514 v2 E2E Protection Mechanisms

Version history:
(2):

Purpose: Defines the E2E Protection mechanisms.
Source:
Verification Method: Test
Legacy ID: 181433v1

The ProfileVCC shall have the following mechanisms:

1. Counter - 4bit (explicitly sent) representing numbers from 0 to 14 incremented on every send request.

2. Timeout - Timeout is determined by E2E Library by means of evaluation of the Counter, by a non-blocking read at the receiver.

3. Data ID - 16 bit, unique number, included in the CRC calculation but not transmitted (implicit transmission).

4. CRC - CRC-8-SAE J1850 - 0x1D (x8 + x4 + x3 + x2 + 1) , but with different start and XOR values (both start value and XOR value are 0x00). This CRC is provided by CRC library. Starting with R4.0, the SAE8 CRC function of the CRC library uses 0xFF as start value and XOR value. To compensate a different behavior of the CRC library, the E2E Library applies additional XOR 0xFF operations starting with R4.0, to come up with 0x00 as start value and XOR value.

3515 v2 E2E Protection Wrapper

Version history:
(2):

Purpose: Defines the usage of Autosar E2EWrapper approach.
Source:
Verification Method: Test
Legacy ID: 181453v1

Within the SWC that is sending/receiving an E2E protected signal, the calculations shall be done using the Autosar E2EWrapper approach. This E2EWrapper shall be part of the sending/receiving SWC.

On the sending side, the wrapper shall do the following:

1. Pack the data into an array.

2. Call the E2E-library with the array as an argument (E2E_P01Protect).

3. On return from the E2E-library, put the data + CRC and counter into a signal group (record).

4. Send the signal group to the RTE.

On the receiving side, the wrapper shall do the following:

1. Receive the signal group from the RTE.

2. Get the data + CRC and counter from the signal group.

3. Pack everything in an array.

4. Send the array to the E2E-library (E2E_P01Check).

5. Evaluate the result from the E2E-library and take actions if the CRC or counter is not correct.

4.7.1.1.1.3 Profiles vs Transformers and Wrappers

It is likely more than one profile needs to be supported in an ECU simultaneously. The type of network, size of the data, the need for gatewaying the data and AUTOSAR BSW version, are the main factors for deciding the profile and mechanism to use.

SOMEIPTransformer and E2ETransformer using Profile 4A is used on Ethernet for protection of data between 32byte up to 4kbyte.

ComBasedTransformer and E2ETransformer using Profile 1A is used on CAN and FlexRay for protection of data up to 30byte. This combination can also be used on Ethernet when data is optimized for CAN, e.g. gatewaying from Ethernet to CAN.

ComBasedTransformer and E2ETransformer using Profile 5 can be used for protection of data between 18byte to 64byte when data is optimized for CAN FD or FlexRay, e.g. gatewaying from Ethernet to CAN FD or FlexRay.

E2E Wrapper using Profile 1A is used on CAN and FlexRay for protection of data up to 30byte. E2E Wrapper is used when the AUTOSR BSW version does not support Transformers, i.e. prior to AUTOSAR version 4.2.1.

4.7.1.1.2 TIME SYNCHRONIZATION CONCEPT

4.7.1.1.2.1 Time Synchronization Introduction

Time Synchronization is a method that enables distribution of time information over networks such as Ethernet, CAN and FlexRay networks in order for ECUs to have a common understanding of time.

On Ethernet networks, the AUTOSAR standard has incorporated the mechanisms as specified in IEEE802.1AS-2011. For CAN and Flexray networks, a “new” Time Synchronization concept is developed by AUTOSAR.

Time Synchronization functionality shall realized with AUTOSAR BSW modules such as StbM (Synchronized Time Base Manager), EthTSyn (Time Syncronization on Ethernet), CanTSyn (Time Synchronization on CAN) and FrTSyn (Time Synchronization on Flexray).

AUTOSAR standard defines different roles w.r.t. time synchronization and these roles are Global Time Master, Time Master, Time Slave and Time Gateway. All of these roles are "time aware" which is required by 802.1AS-2011.

The expectations w.r.t. time synchronization is that the difference in perception of time in any two ECUs in a vehicle network shall be less than 500us.

4.7.1.1.2.2 Time Synchronization Requirements

3516 v2 **Hardware time stamping support**

Version history:
(2):

Purpose: Hardware time stamping enables more accurate time information than software time stamping does.
A simplified explanation is that no internal latency is affecting the time synchronization mechanism in a negative way when using hardware time stamping.
Source:
Verification Method: Analysis
Legacy ID: 435545v0

Hardware time stamping support shall be enabled in the AUTOSAR BSW module EthTSyn.

3517 v1 **Use of AUTOSAR BSW modules**

Version history:
(1):

Purpose: Time Synchronization shall be realized using methods and modules as specified by AUTOSAR.
Source:
Verification Method: Inspection
Legacy ID: 435517v0

Time Synchronization functionality shall be realized with the following AUTOSAR modules:

- **StbM**, ref: "AUTOSAR Specification of Synchronized Time-Base Manager"
- **EthTSyn**, ref: “AUTOSAR Specification of Time Synchronization over Ethernet “
- **CanTSyn**, ref: “AUTOSAR Specification of Time Synchronization over Can”
- **FrTSyn**, ref: “AUTOSAR Specification of Time Synchronization over Flexray”

All requirements, constraints and assumptions, dependencies to other modules etc. included in the AUTOSAR specifications above are applicable.

4.7.1.1.3 AUTOSAR CONFIGURATION REQUIREMENTS

In this section all requirements are numbered in order to accomplish traceability during the development process. Text written outside the requirement tags in this section shall be regarded as complementing information and not as a requirement.

3508 v16 **Autosar configuration - required configuration values**

Version history:
(16):

Purpose: Enforce similar network behaviour for all modules in a vehicle
Source:
Verification Method: Inspection

Legacy ID: 52546v14

These are mandatory values for non-programming session. Using an Autosar version 4.0.3 or higher means that the communication stack must support Partial Network Cluster (PNC).

Parameter	Value
CanControllerFdBaudrateConfig	ECU with "Classic CAN": N/A ECU with CAN FD support: Configure according to requirements in the *[AR_Conf_4] CAN Data Link Layer* and/or the provided ECU Extract.
CanSetBaudrateApi (4.1.1-)	FALSE
CanIfRxPduReadData	FALSE
CanIfTxPduType	STATIC
CanIfSetBaudrateApi (4.1.1-)	FALSE
CanNmActiveWakeupBitEnabled (4.1.1-)	TRUE
CanNmAllNmMessagesKeepAwake (4.0.3-)	ECU using ISO11898-6 selective wakeup CAN transceiver: FALSE. ECU not using ISO11898-6 selective wakeup CAN transceiver: TRUE.
CanNmBusLoadReductionActive	FALSE
CanNmBusLoadReductionEnabled	FALSE
CanNmBusSynchronizationEnabled	FALSE
CanNmCarWakeUpRxEnabled (4.0.3-)	FALSE
CanNmComUserDataSupport (4.0.2)	FALSE
CanNmComUserDataSupport (4.0.3-)	TRUE
CanNmImmediateNmCycleTime	0.02
CanNmImmediateNmTransmissions	20
CanNmImmediateRestartEnabled	FALSE
CanNmMainFunctionPeriod	0.005
CanNmMsgCycleOffset	0.1
CanNmMsgCycleTime	1.0
CanNmMsgTimeoutTime	0.5
CanNmNodeDetectionEnabled	TRUE
CanNmNodeIdEnabled (4.1.1-)	TRUE
CanNmPassiveModeEnabled	FALSE
CanNmPduCbvPosition	1
CanNmPduNidPosition	0
CanNmMsgReducedTime	0.5
CanNmRepeatMessageTime	1.0
CanNmPnEiraCalcEnabled (4.0.3-)	TRUE
CanNmPnEnabled (4.0.3-)	TRUE
CanNmPnEraCalcEnabled (4.0.3-)	For a PNC gateway: TRUE. For a non-PNC-gateway: FALSE.
CanNmPnHandleMultipleNetworkRequests	TRUE

(4.0.3-)	
CanNmPnInfoLength (4.0.3-)	6
CanNmPnInfoOffset (4.0.3-)	2
CanNmPnResetTime (4.0.3-)	2.95
CanNmRetryFirstMessageRequest (4.2.1-)	TRUE
CanNmTimeoutTime	3.0
CanNmUserDataEnabled (4.0.3-)	TRUE
CanNmUserDataLength (4.0.3-4.1.3)	6
CanNmWaitBusSleepTime	4.0
CanSMBorCounterL1ToL2	32
CanSMBorCounterL2Err (4.0.2)	31
CanSMBorTimeL1	0.01
CanSMBorTimeL2	60.0
CanSMBorTimeTxEnsured	1.0
CanSMEnableBusOffDelay (4.1.3-)	FALSE
CanSMIcomSupport (4.2.1-)	FALSE
CanSMMainFunctionPeriod	0.005
CanSMPncSupport (4.1.2-)	TRUE
CanSMSetBaudrateApi (4.1.1-)	FALSE
CanTpChannelMode	CANTP_MODE_HALF_DUPLEX
CanTpFlexibleDataRateSupport (4.2.1-)	FALSE
CanTpMainFunctionPeriod	0.005
CanTpNar	1.0
CanTpNas	1.0
CanTpNbr	0.9
CanTpNbs	1.0
CanTpNcr	1.0
CanTpNcs	0.9
CanTpRxAddressingFormat	Standard
CanTpRxPaddingActivation	CANTP_ON
CanTpRxWftMax	255
ComFilterAlgorithm	ALWAYS
ComMMainFunctionPeriod	0.005
ComMNmPassiveModeEnable (4.1.1-)	FALSE
ComMNmVariant (4.0.3-)	FULL
ComMPncNmRequest (4.0.3-)	TRUE
ComMPncEnabled (4.0.3-)	TRUE
ComMPncGatewayEnabled (4.0.3-)	Domain master ECU: TRUE Other than domain master ECU: FALSE
ComMPncPrepareSleepTimer (4.0.3-)	0.025
ComMPncSupport (4.0.3-)	TRUE
FrIfAllowDynamicLSduLength	FALSE
FrIfUnusedBitValue	0

FrNmActiveWakeupBitEnabled (4.1.2-)	TRUE
FrNmComUserDataSupport (4.0.2)	FALSE
FrNmComUserDataSupport (4.0.3)-	TRUE
FrNmControlBitVectorActive	TRUE
FrNmControlBitVectorEnabled	TRUE
FrNmCycleCounterEmulation	FALSE
FrNmDataCycle	4
FrNmMainAcrossFrCycle	FALSE
FrNmMainFunctionPeriod	0.005
FrNmMsgTimeoutTime	2.0
FrNmNodeDetectionEnabled	TRUE
FrNmNodeDetectionLock (4.0.2)	0
FrNmPassiveModeEnabled	FALSE
FrNmPduLength	8
FrNmPduScheduleVariant	FRNM_PDU_SCHEDULE_VARIANT_1
FrNmPnEiraCalcEnabled (4.0.3-)	TRUE
FrNmPnEnabled (4.0.3-)	TRUE
FrNmPnEraCalcEnabled (4.0.3-)	TRUE
FrNmPnInfoLength (4.0.3-)	6
FrNmPnInfoOffset (4.0.3-)	2
FrNmPnResetTime (4.0.3-)	2.95
FrNmReadySleepCnt (4.0.2, 4.0.3)	150
FrNmReadySleepCnt (4.1.1-)	149
FrNmRepeatMessageBitActive	FALSE
FrNmRepeatMessageTime	0.2
FrNmRepetitionCycle	4
FrNmSourceNodeIdentifierEnabled	TRUE
FrNmSyncLossTimer (-4.1.3)	0.04
FrNmUserDataEnabled (4.0.3-)	TRUE
FrNmVotingCycle	4
FrNmVotingNextToLastRepetitionCycleDisable	TRUE
FrNmVoteInhibitionEnabled	TRUE
FrSMCheckWakeupReason	TRUE
FrSMDurationT1	0.0
FrSMDurationT2	0.2
FrSMDurationT3	10.0
FrSMMainFunctionCycleTime	0.005
FrSMNumWakeupPatterns	5
FrSMStartupRepetitions	10
FrSMStartupRepetitionsWithWakeup	10
FrSMTrcvStdbyDelay (4.1.1-)	0.02
FrTpAckRt	FALSE

FrTpFullDuplexEnabled	FALSE
FrTpMainFuncCycle	0.005
FrTpMaxFCWait	255
FrTpTimeBr	0.25
FrTpTimeBuffer (-4.1.3)	0.020
FrTpTimeoutAr	1.0
FrTpTimeoutAs	1.0
FrTpTimeoutBs	1.0
FrTpTimeoutCr	4.0
NmBusSynchronizationEnabled	FALSE
NmComUserDataSupport (4.0.2)	FALSE
NmComUserDataSupport (4.0.3-)	TRUE
NmCoordinatorSupportEnabled	FALSE
NmCoordinatorSyncSupport (4.1.1-)	FALSE
NmNodeDetectionEnabled	TRUE
NmNodeIdEnabled	TRUE
NmPassiveModeEnabled	FALSE
NmUserDataEnabled (4.0.3-)	TRUE

Note:
When a configuration parameter value is mandatory for all Autosar versions, no Autosar version is denoted after the parameter name.
When different parameter values are required depending on Autosar version or the parameter exist only for one Autosar version, the applicable version is denoted, e.g. "(4.0.3)".
4.0.3- with a dash after means it is applicable for 4.0.3 and later versions.

3509 v9 **Autosar configuration - recommended values**

Version history:
(9):

Purpose: Define recommended configuration values
Source:
Verification Method: Analysis
Legacy ID: 52547v7

These are recommended values that are suitable for most ECU:s but may be deviated from if needed (but any change of a parameter value must result in that requirements are fulfilled).

Parameter	Value
CanIfPrivateSoftwareFilterType	BINARY
CanIfPublicCancelTransmitSupport	FALSE
CanIfPublicReadRxPduDataApi	FALSE
CanIfPublicReadRxPduNotifyStatusApi	FALSE
CanIfPublicReadTxPduNotifyStatusApi	FALSE
CanIfPublicSetDynamicTxIdApi	FALSE
CanIfPrivateDlcCheck	TRUE
CanIfSupportTTCAN	FALSE

CanTimeoutDuration	0.002
CanTpSTmin	0.005
ComSignalInitValue	0
ComSignalType	avoid float
ComTxIPduCancelTransmitSupport	FALSE
ComTxIPduUnusedAreasDefault	0
ComTxModeMode	do not use MIXED
ComTxModeNumberOfRepetitions	0
FrRxStringentCheck	TRUE
FrRxStringentLengthCheck	TRUE
FrBufferReconfig	FALSE
IpduMSelectorFieldLength	8
IpduMSelectorFieldPosition	0
LinChannelBaudRate	19200
LinIfCancelTransmitSupported	FALSE

4.7.1.1.4 Documents

Reference	Source	Document
[AR_Conf_1]	Internal	Autosar Diagnostic Guideline
[AR_Conf_2]	Internal	Autosar Network Management - Additional requirements
[AR_Conf_3]	Internal	FlexRay Data Link Layer
[AR_Conf_4]	Internal	CAN Data Link Layer

Note:
Internal references is typically found as a separate chapter in ECU specific Base Technology SWRS.
Referenced AUTOSAR SWS Specification can be found at http://www.autosar.org/

4.7.1.1.5 Definitions and Abbreviations

Abbreviation	Description
PC	PreCompile - A parameter that can't be changed without re-compiling the application
LT	LinkTime - A parameter that can't be changed without re-linking the application i.e. it is not necessary to re-compile but there will be a new version of the application
PB	PostBuild - A parameter that can be changed without updating the application

4.7.2 Autosar Network Management - Additional Requirements

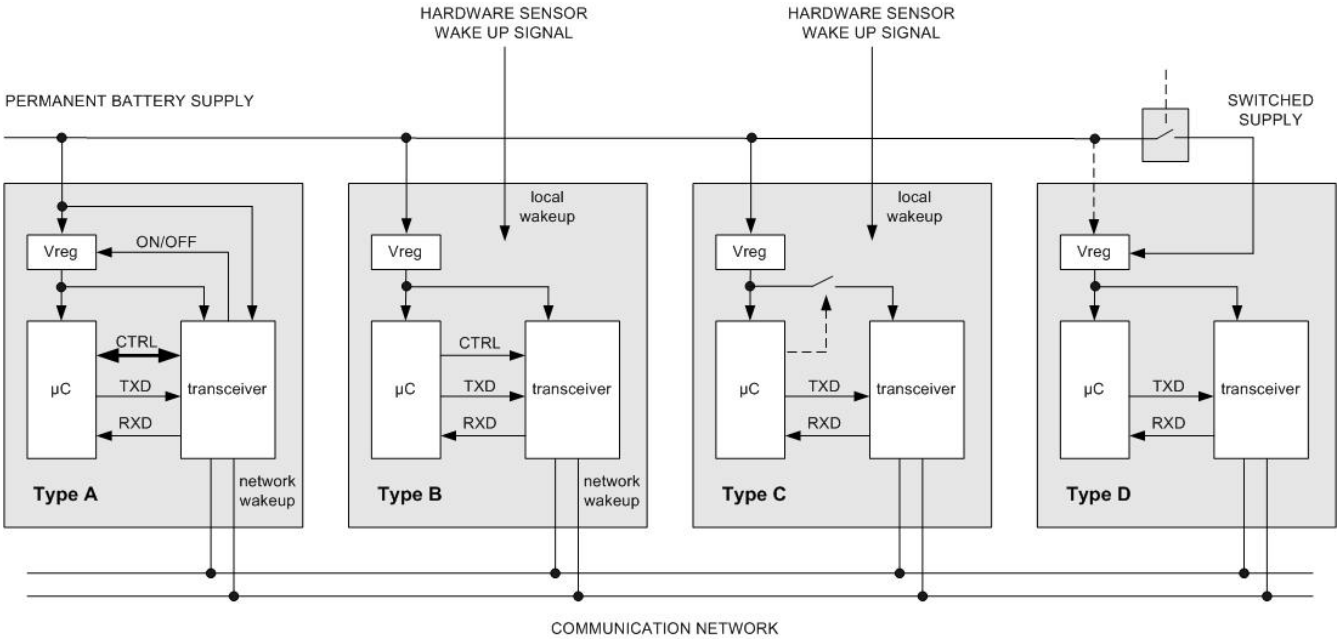
4.7.2.1 AutoSar NM - Generic reqs

4.7.2.1.1 Introduction

Software Component Requirements
In this document all requirements are numbered in order to accomplish traceability during the development process. Text written outside the requirement tags in this section shall be

regarded as complementing information and not as a requirement.

Node types
There are four basic node types; A, B ,C and D, defined based on how the node is started up, electrical current consumption performance, and resulting startup timing performance. The purpose of the node types shall be to identify the requirements based on whether network wake up exist, hardware sensor signal wake up exist, if supply voltage is permanent or switched. The selection shall only be based on how the internal power supply, microcontroller and transceiver that handle the network communication is activated by external signals, internal signals and power supply. Power supply to other parts of the node shall not influence the type selection. Note that node types A-D are the most common combinations. The timing requirements are grouped based on whether the microcontroller is actually powered or unpowered in a sleep mode, which a the major factor regarding possible startup timing performance. Autosar Network Management is expected to be used for all node types unless otherwise stated.



Node types and internal power supply and wake up.

Node Type A, permanent power supply and network wake up
Nodes which have to be functionally available all time, even when the vehicle is parked and ignition-key is off. These types of nodes are permanently supplied from a permanent battery supply line. These applications allow switching off the entire supply system of the node including the microcontroller supply while keeping the wake-up capability by the communication network. Node sleep current typically below 100 µA, while keeping the capability to support wake-up by the communication network and to restart the application.

Node Type B, permanent power supply, network and hardware wake up
Nodes which have to be available all time, even when the vehicle is parked and ignition-key is off. These types of nodes are permanently supplied from a permanent battery supply line, and using a continuously active Vcc supply to a microcontroller. In order to reduce the node power

consumption, the transceiver needs to be set into a mode with reduced supply current while the Vcc stays active. This type of node support wake-up by the communication network and by an input signal from a hardware sensor.

Node Type C, permanent power supply and hardware wake up
Nodes which have to be available all time, even when the vehicle is parked and ignition-key is off. These types of nodes are permanently supplied from a permanent battery supply line, additionally come with a microcontroller controlled transceiver voltage supply. In contrast to node Type B, electrical sleep current consumption can be further reduced, because the transceiver becomes completely un-powered by microcontroller control. It supports wake up only by an input signal from hardware sensor.

Node Type D, switched power supply
Nodes which do not need to be available with ignition-key off are simply switched off and become totally un-powered during ignition-key off. They are supplied from a switched battery supply line, for example “Clamp-15”. This supply line is only switched on with ignition-key on or by a relay. Ignition-sensing nodes that have a permanent power supply and determine when to start communication based on e.g. a clamp-15 signal is also considered to be type D. Optionally, ignition sensing nodes may communicate after ignition off (after-run). For these nodes network management is also required since network communication is not expected automatically when a node has power supply. There must be an explicit need for network communication for the node to communicate since the node may be power supplied for other reasons, e.g. standalone operation in some situations. This enables decoupling the node application operation from communication.

Detailed functional requirements
The Autosar network management BSW modules are partly configured by the parameters in *Autosar Communication Configuration* to assure common fundamental aspects e.g. communication timing between nodes, while remaining configuration is expected bye the supplier. To define additional requirements on behavior the following sections add requirements that are common for network types CAN and FlexRay and also specific for each network type.

4.7.2.1.2 Overview

Software Component Description
This document describes requirements that applies to an Autosar based node and are in addition to the behavior defined by Autosar network management BSW. It applies to CAN and FlexRay networks.

Component context
This SWRS describes Network Management for Autosar based nodes. Network management within Autosar is limited compared to the desired network management functionality. The basic scope of Autosar network management is to coordinate the network requests of all or some nodes on a single network in order to obtain a controlled, simultaneous and synchronized shutdown of communication when those nodes are ready to go to sleep. With partial networking (PNC) the scope applies to multiple networks.

The scope of network management desired is more extensive and is described in this document. It includes:

- Start-up timing of communication on a node by network wake up, hardware wake up and power on (i.e. also relates to hardware performance).

- Support of network communication state history for diagnostic purposes.

The ECU specfic decision when to request a network to be active or not active is not defined in this document. However network management is a mechanism and protocol where NM-PDUs are sent between nodes for communicating the needs for network communication. Communication needs may be defined elsewhere and determined on logical level (in application SW-Cs) by Virtual Function Cluster (VFC) activation and deactivation criteras. If BSW supports partial networking, then VFC requests are translated in BSW to Partial Network Clusters (PNC) requests which are communicated among nodes in NM-PDUs. PNCs are related to physical networks being requested active or not active. Partial networking may improve energy efficiency by only activating currently needed networks and nodes. In this specification, timing requirements for a node are only applicable when network communication has been granted to be activated, the specific communication conditions is expected to be found in node specific requirements. Other generic requirements applicable when implementing logic for requesting communication can be found in *Autosar - ECU Mode Management*. FlexRay schedule coordination is described in *FlexRay Data Link Layer*.

Note
This document covers both the older CAN standard, referred to as classic CAN, and the updated standard CAN Flexible Data-Rate, referred to as CAN FD. Unless explicitly specifying the standard of CAN in this document, any reference to CAN refers to both the classic CAN and CAN FD standards.

4.7.2.1.2.1 Definitions and Abbreviations

Definition	Description
AR	Autosar
Bus	See network
Clamp 15	A power supply/control signal that activate nodes.
Clamp 30	A permanent power supply.
Coordination clustering	A discrete set of NM Channels on which shutdown is coordinated. The NM Coordinator will keep all presently awake NM Channels of a NM Coordination Cluster awake until it is possible to coordinate network sleep on all the awake channels.
ECU	Same as node in this document, see Node.
Full Autosar	A node where at least all RTE and all BSW modules are Autosar.
HW	Hardware
Network	An interconnection of nodes for the purpose of exchanging information.
Network type	Network types are LIN, CAN and FlexRay networks.
NM	Network Management
NM User Data	A number of payload bytes in an NM-PDU in a frame that can be used for signalling arbitrary information. If PN is supported then PNC request bits are transmitted in NM User Data.
NM Vote	A bit in a FlexRay NM-PDU in a frame that indicate if the node wants communication to be enabled or not.
Node	An electronic control unit that execute software.
Node type	See *Node types, Software Component Requirements – Requirement Content*.

Partial network	System state in a CAN/FlexRay system in which some nodes are sleeping while other nodes are communicating.
Selective wakeup transceiver	A CAN transceiver according to ISO11898-6 which allows a node that supports PN to be in sleep when no PNC requests are active for the specific node. The transceiver will wakeup the node when frames with defined CAN identifiers and PNC bits match a predefined configuration.
SW	Software

Abbreviation	Description
AR	Autosar
BSW	Basic Software
DID	Diagnostic Identifier
DTC	Diagnostic Trouble Code
FIFO	First in first out
HW	Hardware
IGN	Ignition
µC	Micro controller
N/A	Not applicable
NM	Network Management
NMPDU	Network Management Protocol Data Unit
PDU	Protocol Data Unit
PN	Partial Network
PNC	Partial Network Cluster. Partial Network Cluster is a group of system signals necessary to support one or more vehicle functions that are distributed across multiple ECUs in the vehicle network. This represents the system view of mapping a group of buses to one ore more VFCs.
POC	Protocol Operation Control
RTE	Real Time Environment
StbM	Synchronized Time Base Manager
SW-C	Software Component
VFC	Virtual Function Cluster. Virtual Function Cluster groups the communication on port level between SW- components that are required to realize one or more vehicle functions. This is the logical view and allows for a reusable bus/ECU independent design.
WUP	Wake up pattern. Multiple repetitions of WUP for the purpose of waking up a FlexRay network.
WUS	Wake up symbol. A specific bit pattern used for network wake up on FlexRay.

4.7.2.1.2.2 Requisite documents

Reference	Source	Document
[AR_NM_1]	AUTOSAR	Autosar Specification of Generic Network Management Interface AUTOSAR_SWS_NetworkManagementInterface.pdf

[AR_NM_2]	AUTOSAR	Autosar Specification of LIN Network Management AUTOSAR_SWS_LINNetworkManagement.pdf
[AR_NM_3]	AUTOSAR	Autosar Specification of CAN Network Management AUTOSAR_SWS_CANNetworkManagement.pdf
[AR_NM_4]	AUTOSAR	Autosar Specification of CAN State Manager AUTOSAR_SWS_CANStateManager.pdf
[AR_NM_5]	AUTOSAR	Autosar Specification of FlexRay Network Management AUTOSAR_SWS_FlexRayNetworkManagement.pdf
[AR_NM_6]	AUTOSAR	Autosar Specification of FlexRay Driver AUTOSAR_SWS_FlexRayDriver.pdf
[AR_NM_7]	AUTOSAR	Autosar Specification of FlexRay State Manager AUTOSAR_SWS_FlexRayStateManager.pdf
[AR_NM_8]	AUTOSAR	Autosar Specification of FlexRay Interface AUTOSAR_SWS_FlexRayInterface.pdf
[AR_NM_9]	AUTOSAR	Autosar Specification of Flash EEPROM Emulation AUTOSAR_SWS_FlashEEPROMEmulation.pdf
[AR_NM_10]	FlexRay Consortium	FlexRay Communications System Protocol Specification Version 2.1 Revision A
[AR_NM_11]	LIN Consortium	LIN Specification Package Revision 1.3
[AR_NM_12]	LIN Consortium	LIN Specification Package Revision 2.1
[AR_NM_13]	Internal	Autosar Communication Configuration
[AR_NM_14]	Internal	Autosar - ECU Mode Management
[AR_NM_15]	Internal	UDS Session
[AR_NM_16]	Internal	FlexRay Data Link Layer
[AR_NM_17]	Internal	UDS Data

4.7.2.1.2.3 OEM specific concepts

Network management state history for diagnostic purposes To support verification and fault tracing of network communication activation there is a need for a FIFO history mechanism for each connected network (for each instance of NM). The state history is accessible thru a diagnostic service by a diagnostic identifier, one separate for each network in case the node has more than one network.

4.7.2.2 AutoSar NM - CAN reqs

4.7.2.2.1 Diagnostics

3522 v2 **CAN network management history algorithm**

Version history:
(2):

Purpose: To be able to trace state history of CAN NM state machine combined with added OEM specific functionality.

Source: Autosar requirement CANNM089 in Autosar Specification of CAN Network Management, AUTOSAR_SWS_CANNetworkManagement.pdf, Autosar Network Management (additional requirements) - External publications

Verification Method: Test

Legacy ID: 52305v1

The state of the Autosar CAN network management state machine and additional conditions shall continuously be evaluated and prioritized according to the tables below. A new value shall be added to the history buffer if it differs from the last value that was added to the buffer.

Algorithm inputs			Algorithm output
Bus Off CANSMB or TimeL1 recovery	**Bus Off CANSMB or TimeL2 recovery**	**CAN NM state machine state**	**Value added to history buffer**
Active	Not active	Don't care	CANSMBusOffRecoveryLevel1
Not active	Active	Don't care	CANSMBusOffRecoveryLevel2
Not active	Not active	Bus Sleep Mode	BusSleepMode
Not active	Not active	Prepare Bus Sleep Mode	PrepareBusSleepMode
Not active	Not active	RepeatMessageState	RepeatMessageState
Not active	Not active	Normal Operation State	NormalOperationState
Not active	Not active	Ready Sleep State	ReadySleepState

Priority algorithm for adding values to CAN history buffer.

Overview of all values allowed in history buffer.

Parameter	Value
Off (initial state after power on or reset)	0x01
BusSleepMode	0x02
RepeatMessageState	0x04
Normal Operation State	0x05
ReadySleepState	0x06
PrepareBusSleepMode	0x07
CANSMBusOffRecoveryLevel1	0x08
CANSMBusOffRecoveryLevel2	0x09

State history values for CAN.

3523 v5 **Diagnostic trouble code for CAN bus-off**

Version history:
(5):

Purpose: To be able to detect bus off events. The same behavior regarding setting the bus off DTC is required for all Autosar versions, but configuration differs. To clarify configuration differences between Autosar versions.

Source: Autosar Specification of CAN State Manager, AUTOSAR_SWS_CANStateManager.pdf. For Autosar 4.02 see also CANSM301. For Autosar 4.0.3 and later see also CANSM522

Verification Method: Test
Legacy ID: 52314v4

For all Autosar versions: There shall be 32 detections of CAN bus off in CANSM until a DTC for CAN bus off is set to testFailed. A diagnostic trouble code (DTC) for CAN bus off shall be set within 100 ms according to *[AR_NM_17] UDS Data*. Depending on Autosar version, a fault detection counter is implemented in CANSM or DEM. The number of CAN bus off events required until a DTC is set to testFailed shall be same regardless of Autosar version.

Specific for Autosar 4.0.2: If the bus off counter in CANSM exceeds CanSMBorCounterL2Err, and is defined in *[AR_NM_4] Autosar Specification of CAN State Manager,* a diagnostic trouble code (DTC) shall be set to testFailed (FDC=+127).

Specific for Autosar 4.0.3 and later where no bus off counter exist in CANSM but a fault detection counter FDC10 in DEM shall acts as a prescaler until DTC is set: The fault detection counter FDC10 in DEM shall be increased by (127/(CanSMBorCounterL2Err+1)) at every test call from CANSM to DEM with EventStatus := DEM_EVENT_STATUS_PRE_FAILED. This results in that 32 consecutive calls are required to make the FDC10 counter reach +127 and to set DTC to testFailed.

General for all Autosar versions: If the bus off counter becomes 0, according to Autosar Specification of CANSM, the diagnostic trouble code shall be set to testPassed (FDC=-128).

Note: Example for 4.03 and later: when busoff is detected by CANSM the fault detection counter FDC10 in DEM shall be increased by (127/32), which rounds off to 4.

4.7.2.2.2 Timing requirements

Startup timing requirements vs. Autosar
The wakeup timing requirements described, e.g. TCANPowerWakeUpToApp, do not have a direct configuration parameter in Autosar. The wakeup requirements are aimed towards the complete node consisting of hardware and software, and the interaction between them. Therefore attention must be paid on the hardware design as well even if this specification is called *Autosar Network Management - additional requirements*, and Autosar actually is a software architecture.

3518 v5 **Definition of TCANPowerWakeUpToApp**

Version history:
(5):

Purpose: To define maximum time to start communication.
Source:
Verification Method: Test
Legacy ID: 52306v4

TCANPowerWakeUpToApp shall be equal to 150 ms.
If another requirement define this time to be shorter that requirement shall override this requirement.

3519 v1 Definition of TCANResumeCom

Version history:
(1):

Purpose: To define maximum time to start communication if communication is started some time after node is started.
Source:
Verification Method: Test
Legacy ID: 339294v0

TCANResumeCom shall be equal to 25 ms.
If another requirement define this time to be shorter that requirement shall override this requirement.

3520 v4 Timing at power up or reset initialization CAN

Version history:
(4):

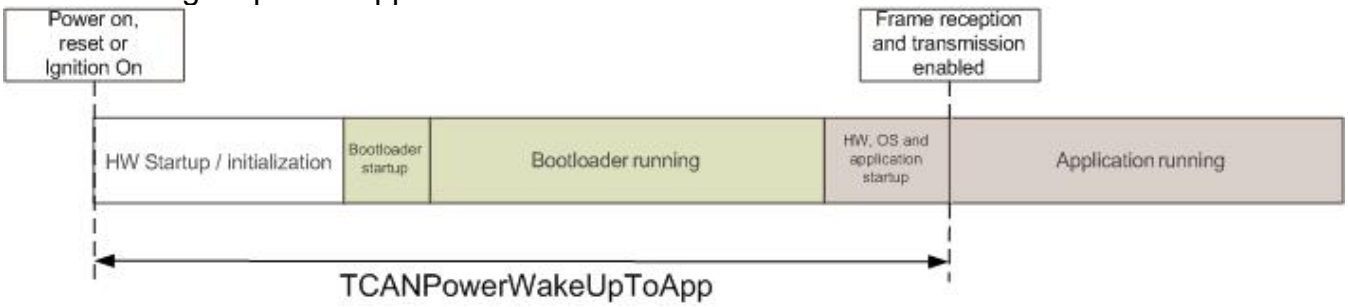
Purpose: To ensure sufficient response time on frame level.
Source: For definition of node types see Node Types, Autosar Network Management (additional requirements) - Overview
Verification Method: Test
Legacy ID: 52307v3

A node shall be ready to receive and transmit CAN frames maximum TCANPowerWakeUpToApp after power is applied, a hardware reset and a diagnostic ECUReset(hardReset).

For nodes **Type A**, **Type B** and **Type C** the following case applies:
- The permanent power supply is applied.

For nodes **Type D** the following cases applies:
- Power is supplied by a relay.
- Ignition On or Clamp-15 goes active.

The following sequence applies:



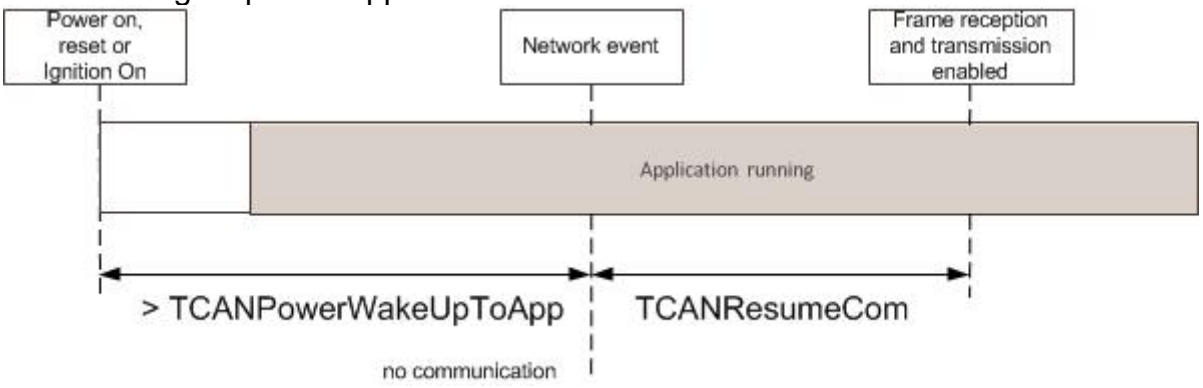
Wakeup sequence for a node power on or reset initialization.

3521 v1 **Timing at resumed communication CAN - relay powered/ignition sensing node**

Version history:
(1):

Purpose: To ensure sufficient response time on frame level.
Source: For definition of node types see Node Types, Autosar Network Management (additional requirements) - Overview
Verification Method: Test
Legacy ID: 339293v0

If more than TCANPowerWakeUpToApp has passed since power on, a hardware reset or a diagnostic ECUReset(hardReset), a node shall be ready to receive and transmit application signals maximum TCANResumeCom after communication is requested by another node. The requirement applies every time the node is requested to start communication and is currently not communicating. Communication shall not be enabled until all conditions are fulfilled.
For nodes Type D the following cases applies:
- Power is already supplied by a relay and communication stack is already initialized, communication is not currently active but becomes requested by another node.
- Ignition On or Clamp-15 is already active and communication stack is already initialized, communication is not currently active but becomes requested by another node.
The following sequence applies:



Sequence for activation of communication.
Note: For node Type D the software is expected to execute all time a relay or Clamp 15 supplies power. Since the communication stack is already initiated the node is expected to be able to start communication quicker than from power on. Therefore the node is not considered woken up in this case, as the software is already executing. The external request to start communication occur when NM-PDUs are transmitted from another node. The requirement applies not just when communication resumes from previously being active but also if the first communication activation occure some time after the node is started.

4.7.2.3 AutoSar NM - CAN & FlexRay additional reqs

4.7.2.3.1 Requirements

3537 v3 **Network management state history buffers**

Version history:

(3):

Purpose: To be able to trace state history of NM state machines and added OEM specific functionality.

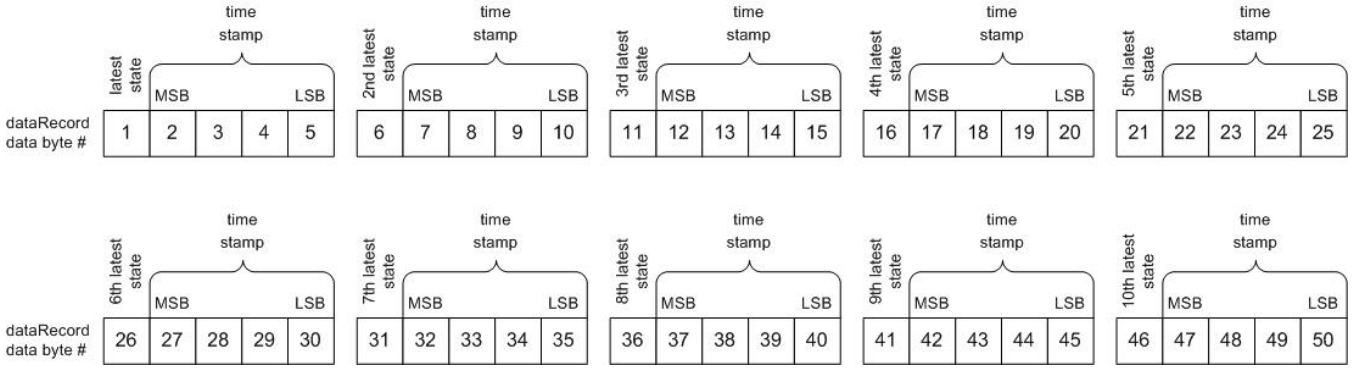
Source: Unified diagnostic services (UDS) - Part 1: Specification and requirements - Data specifies DID, Autosar Network Management (additional requirements)

Verification Method: Test

Legacy ID: 52248v2

For each network connected to the node there shall be a separate network management state history buffer. Each buffer shall be updated with the last value derived from the respective network management history algorithm. There shall be a time stamp that relates to each value in all history buffers. The source of timestamps shall be a global time received from a master node if available. This enables comparing network historic events across all networks for that node and across nodes. If a timestamp value shall be taken and there is temporarily no communication on the network it is acceptable to use the last received global time. If a global time is not available a local time source can be used, in that case resolution shall be 100ms. If more than one buffer is implemented, the same source shall always be used for all buffers. All history buffers shall operate according to FIFO, first-in-first-out. All history buffers shall hold the 10 most recent states. No changes in states or conditions may be missed.

It shall be possible to read each network history buffer by a separate diagnostic identifier (DID). The following figure show the layout of the response of each DID, with the most recent state first.



Layout of history diagnostic identifier response.

3538 v4 **Storage and clearing of network management state history buffers**

Version history:
(4):

Purpose: To be able to correctly trace state history for nodes that support deep sleep. Nodes Type D may not be noticed in advance of release of the control signal IGN, Clamp-15 which often is used to trigger write of data to persistent memory. Therefore this node type shall not be required to persistently store network management history buffer.

Source:

Verification Method: Test

Legacy ID: 52262v3

Nodes Type A, Type B and Type C shall have persistent storage, i.e. in non volatile memory, of network management state history buffers. Persistance is required also at uncontrolled ECU shutdown, e.g. sudden power loss, power supply brown out and unexpected software reset. The state history buffers may be set to the value representing an initial state (Off) and all timestamps be set to the value of 0 at diagnostic reset or at transition from programming session to non-programming sesson (this is acceptable but not preferred). When exiting sleep mode due to local wake up by a sensor signal change or network activity wake up the state history buffers shall not be cleared to initial state (Off).

Node Type D shall at power on, diagnostic reset and transition from programming session to non-programming sesson, set all positions in all history buffers to the value representing an initial state (Off) and all timestamps shall be set to the value of 0. This node type does not need to have persistent storage of history buffers but it is accepted if it has.

3539 v2 **Minimum number of write events of history buffer**

Version history:
(2):

Purpose: To ensure sufficient lifetime of the history buffer memory.
Source: FEE102, FEE103 in Autosar Specification of Flash EEPROM Emulation
Verification Method: Analysis
Legacy ID: 339314v1

Node types that are required to have persistent storage of NM history buffers shall have hardware and software that allows persistent storage of history buffer data with minimum 100000 number of event where new state data is shifted into and written in the history buffer. One event is defined as one state change in CAN NM. This requirement applies separately for each history buffer if the node has more than one history buffer (network). Wear leveling algorithms in software shall be used to fulfill requirement if hardware memory does not on its own support the number of write events.
Note: Autosar Flash EEPROM Emulation contain mechanisms using multiple segments to ensure minimum number of writes if the underlying flash device or device driver does not provide at least the configured number of erase/write cycles per physical memory cell.

4.7.2.4 AutoSar NM - CAN wake up reqs

4.7.2.4.1 Requirements

Communication startup timing for nodes with microcontroller powered and unpowered when prepared for wakeup.
For nodes that have possibility to wakeup and start communication due to hardware sensor signals and network activity will have the same timing requirements for these two wakeup sources. Different use cases result in different need for communication startup time, and also requirements for sleep current consumption affect the design choice. Therefore two implementations of the node internal hardware exist which enable shorter or longer time from the initiating event until frame reception and transmission is started. The key aspect is whether the microcontroller is kept powered or unpowered in sleep. Therefore two wakeup timing requirements exists based on whether the microcontroller has power supply when the node is in

a mode prepared for wakeup. In a given situation only one of the requirements is valid. These timing requirements do not concern transmitted application signals being updated with valid data.

Wakeup timing requirements vs. Autosar

The wakeup timing requirements described, e.g. TCANEventWakeUpToApp, do not have a direct configuration parameter in Autosar. The wakeup requirements are aimed towards the complete node consisting of hardware and software, and the interaction between them. Therefore attention must be paid on the hardware design as well even if this specification is called *Autosar Network Management - additional requirements*, and Autosar actually is a software architecture.

3524 v3 Definition of TCANEventWakeUpToApp

Version history:
(3):

Purpose: To define maximum time to start communication.
Source:
Verification Method: Test
Legacy ID: 68204v2

TCANEventWakeUpToApp shall be equal to 50 ms.
If another requirement define this time to be shorter that requirement shall override this requirement.

4.7.2.4.1.1 Microcontroller is powered in sleep

3525 v7 Timing at network and hardware wake up CAN - microcontroller powered in sleep

Version history:
(7):

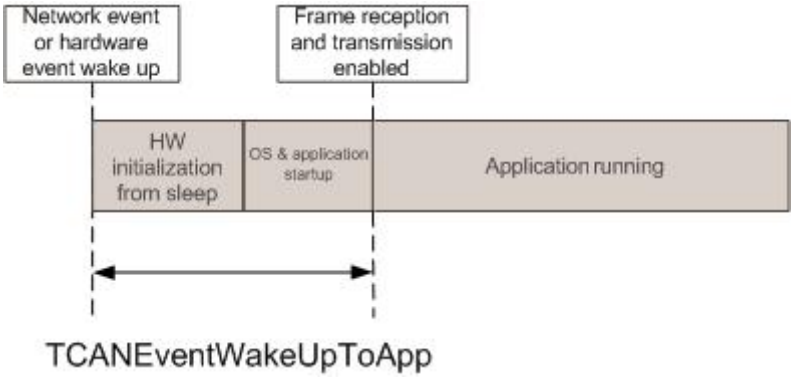
Purpose: To ensure sufficient response time on frame level.
Source:
Verification Method: Test
Legacy ID: 59045v6

Network wakeup and hardware wakeup:

A node shall be ready to receive and transmit CAN frames maximum TCANEventWakeUpToApp after network activity or a hardware sensor signal change (e.g. a door switch is detecting a door being opened) is detected.
For node **Type B** the following case apply:
- Network activity is detected by a CAN transceiver.

For node types **Type B** and **Type C** the following case apply:
- Hardware sensor signal change is detected.

The following sequence applies:



Wakeup sequence for a node wake up by network activity.
Note: Some nodes may be equipped with "selective wake up transceivers". For such a node the requirement for network wakeup applies only when the CAN transceiver has been woken up by a PNC request that matches the partial networks it is part of. Nodes not equipped with such transceiver will always wake up on any network activity and therefore the requirement will always apply to those nodes.

4.7.2.4.1.2 Microcontroller is unpowered in sleep

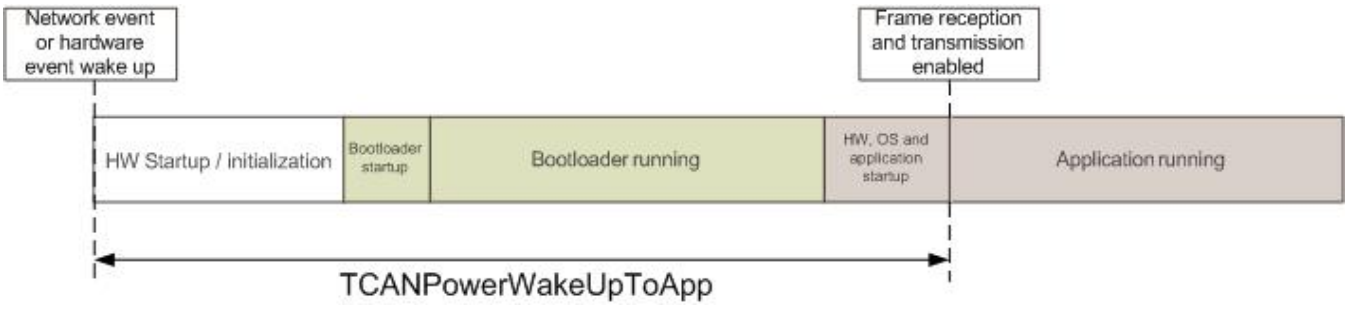
3526 v6 Timing at network and hardware wake up CAN - microcontroller unpowered in sleep

Version history:
(6):

Purpose: To ensure sufficient response time on frame level.
Source:
Verification Method: Test
Legacy ID: 58806v5

A node shall be ready to receive and transmit CAN frames maximum TCANPowerWakeUpToApp after network activity or a hardware sensor signal change (e.g. a door switch is detecting a door being opened) is detected.
For node type **Type A** the following case applies:
- Network activity is detected by CAN transceiver.

The following sequence applies:



Wakeup sequence for a node wake up by network activity or hardware sensor signal

change.
Note: Some nodes may be equipped with "selective wake up transceivers". For such a node the requirement for network wakeup applies only when the CAN transceiver has been woken up by a PNC request that matches the partial networks it is part of. Nodes not equipped with such transceiver will always wake up on any network activity and therefore the requirement will always apply to those nodes.

4.7.3 Autosar BSW - Tier1 to Tier2 Requirements

4.7.3.1 Autosar 4.0.3 BSW - Tier1 to Tier2 Requirements

4.7.3.1.1 Requirements

This section clarifies needed Tier1 (ECU supplier) requirements on a Tier2 (AUTOSAR BSW supplier)
A Tier1 (ECU supplier) will in most cases buy AUTOSAR BSW from a Tier2 (AUTOSAR BSW supplier). I in some case the ECU supplier may have the role of both Tier1 and Tier2, i.e. use their own developed AR BSW.
The AUTOSAR BSW specification for R4.0.x contains known issues/bugs that makes it impossible for a Tier1 (ECU supplier) to realize their own REQPROD requirements, i.e. a Tier1 (ECU supplier) must require bug fixes from the Tier2 (AUTOSAR BSW supplier) to be able to fulfill their own REQPROD requirements.

3558 v2 **Needed Tier1 requirements on Tier2 for AUTOSAR Diagnostic BSW**

Version history:
(2):

Purpose:
Source:
Verification Method: Analysis
Legacy ID: 362336v0

To be able to fulfill Tier1 (ECU supplier) Diagnostic requirements when using AUTOSAR 4.0.3, the Tier1 (ECU supplier) shall require a Tier2 (AUTOSAR BSW supplier) to at least fix the already known AUTOSAR BSW issues as specified in:

• SWRS_31835918-VCC AUTOSAR Dem
• SWRS_31835919-VCC AUTOSAR Dcm
• SWRS_31836996-VCC AUTOSAR FiM

Note:
The VCC AUTOSAR Diagnostic BSW specifications are available from CEVT upon request.

Note:
Which issues a Tier1 must require a Tier2 to fix in the AUTOSAR BSW is dependent on the Tier1 REQPROD requirements.
E.g. If the Tier1 does not use the BaseTech optional service 2A, it is not needed by the Tier1 to require a Tier2 to fix the issues regarding this service in the Dcm module.
E.g. If the Tier1 does not use the optional FiM module, it is not needed by the Tier1 to require a Tier2 to fix the issues regarding FiM module.

Note:
It can still be not yet found issues in the AUTOSAR BSW specifications which are not yet covered in the VCC AUTOSAR Diagnostic BSW specifications. If new issues are found, the VCC AUTOSAR BSW Diagnostic specifications will be updated. I.e. recommended to always use latest VCC AUTOSAR BSW Diagnostic specifications to get a as bug free AUTOSAR BSW platform for diagnostic as possible from the beginning.
Note:
The used version of VCC AUTOSAR BSW Diagnostic specifications shall be documented in DID F126.

5 DETAILED CHANGE INFO

1597 v3 **Software authentication security classes**

(2) => (3):
Attribute 'Legacy ID' modified: 441876v0 => 441876v1.
Description modified.

Current:Software authentication security classes(3)

The software authentication security classes as defined in the *Table - Software Authentication security classes* must be supported with respect to key management. This means that it shall be possible to configure the key usage in the off-board variant management systems:
1) Integrity check only, i.e. no cryptographic keys needed
2) ECU global keys used for a period of time. This shall be applied by default.
3) ECU unique keys

Security Class – Software Authentication	Meaning
SwAuth Class 1	Integrity check only, i.e. only using checksums/hash values, for error detection. This is not part of the General Software Authentication document, but added here for legacy reasons.
SwAuth Class 2	Authenticity (origin + integrity), using common/global keys. A dedicated signing key per ECU type shall anyhow be applied, e.g. one for all door modules and another for the climate control respectively (for segmentation) The key used shall be possible to change after a period of time (structure week) and (optional) vehicle type.
SwAuth Class 3	Authenticity + “copy between vehicles protection”. An ECU unique key shall be applied. This class is recommended for sellable features but also for risk reduction (e.g. a key is revealed).

	SwAuth Class 3 shall always be applied when there is device unique keys for diagnostic service securityAccess, for programming services.

Table - Software Authentication security classes

Previous: Software authentication security classes(2)

The software authentication security classes as defined in the *Table - Software Authentication security classes* must be supported with respect to key management. This means that it shall be possible to configure the key usage in the off-board variant management systems:
1) Integrity check only, i.e. no cryptographic keys needed
2) ECU global keys used for a period of time
3) ECU unique keys

Security Class – Software Authentication	Meaning
SwAuth Class 1	Integrity check only, i.e. only using checksums/hash values, for error detection. This is not part of the General Software Authentication document, but added here for legacy reasons.
SwAuth Class 2	Authenticity (origin + integrity), using common/global keys. A dedicated signing key per ECU type shall anyhow be applied, e.g. one for all door modules and another for the climate control respectively (for segmentation) The key used shall be possible to change after a period of time (structure week) and (optional) vehicle type.
SwAuth Class 3	Authenticity + “copy between vehicles protection”. An ECU unique key shall be applied. This class is recommended for sellable features but also for risk reduction (e.g. a key is revealed)

Table - Software Authentication security classes

1599 v4 **Signing Method - Variability control**

(3) => (4):
Attribute 'Legacy ID' modified: 404595v2 => 404595v3.
Description modified.

Current:Signing Method - Variability control(4)

To define and distribute the Signing Method, a Software Settings File (a software part) shall be used for methods where each software part is signed individually.

Note: The notation <HashAlgorithm_SigningAlgorithm_SigningObject>, e.g. SHA256_RSA2048_sw_part is used in nextcoming requirements to define algorithms to be applied. The "sw_part" is used to seperate this method from legacy concepts. See also section "Software Settings File" actual definitions.

Previous: Signing Method - Variability control(3)

To control the **Signing Method**, a variant shall be defined and used in the off-board systems. Applicable methods are:

Signing Methods	
Short name	**Comment**
No signing method applied.	To be treated as signing_method is omitted, i.e. for legacy reasons.
SHA1_RSA1024_sw_superset	These legacy methods are defined in *Software Authentication*, i.e. where a complete set of software parts (complete flash image) is signed and the signature is transferred as a dedicated software part.
SHA256_RSA2048_sw_superset	
Spare, sw_superset	
SHA256_RSA2048_sw_part	These methods are used when each software part is signed individually and the signature is transferred using the CheckMemory routine control.
Spare	*Future growth.*

Table - Signing Methods

The short names have the following properties <*HashAlgorithm_SigningAlgorithm_SigningObject*>, e.g. **SHA256_RSA2048_sw_part**, where:

- **SHA256** represents the hash algorithm
- **RSA2048** represents the signing algorithm and key size
- **sw_part** represents that each software part is signed and verified independently, while the **sw_superset** represents that the combination of the complete set of software parts is signed.

Note: *The corresponding variant family is defined in the back-end variant control systems.*

1600 v5 **Format of the Verification Block Table**

(4) => (5):
Attribute 'Legacy ID' modified: 398360v3 => 398360v4.
Description modified.

Current:Format of the Verification Block Table(5)

The verification block table, encoded with big endian, shall have the following format:

- Verification Block Table Format Identifier. The verification block will start with most significant byte of this format identifier. It defines the version of the verification block table, mainly intended for tool parsing and format checks in the ECU, i.e. it shall not be used to configure e.g. the actual algorithm to be used by the bootloader. Size 16 bits.
- Number of data blocks. This shall be identical to the number of data blocks included in the software part. The data block containing the verification block table itself shall not be included. Size 16 bits.
- StartAddress [size 32 bits] , Length [size 32 bits], Hash value. Every data blocks shall be defined, except the block table itself. The size of the hash is dependent of the hash function used and is specified elsewhere. The Length is the unprocessed length (see other requirements for more details).
- The data blocks shall be sorted according to the order the data blocks are being programmed to the ECU.

Example.

- SHA-256 hash function is used, i.e. each hash value is 256 bits.
- Number of data blocks are two, i.e. the total length of the verification block table, in octets, will be (16+16+2x(256+32+32))/8=84 octets.

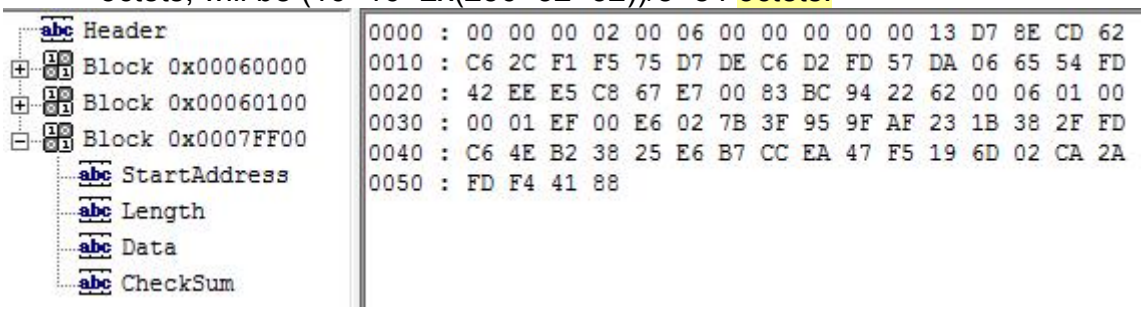


Figure - Example of a VBF file with a verification block table

Figure - Example of a VBF file with a verification block table shows a software part (VBF data file). The data shown belongs to the verification block. The Verification block starts at address 0x0007FF00, i.e. it is the third data block in the figure:
0000
0002
00060000 00000013
D78ECD62C62CF1F575D7DEC6D2FD57DA066554FD42EEE5C867E70083BC942262
00060100 0001EF00
E6027B3F959FAF231B382FFDC64EB23825E6B7CCEA47F5196D02CA2AFDF44188

- The format identifier is 0x0000
- Number of data blocks are two (0x0002)
- Data block 1: Start address 0x00060000 and length 0x00000013. Hash 0xD78ECD...
- Data block 2: Start address 0x00060100 and length 0x0001EF00. Hash 0xE6027B...

Previous: Format of the Verification Block Table(4)

The verification block table, encoded with big endian, shall have the following format:

- Verification Block Table Format Identifier. The verification block will start with most significant byte of this format identifier. It defines the version of the verification block table, mainly indented for tool parsing etc., i.e. it shall not be used to configure e.g. the actual algorithm to be used by the bootloader. Size 16 bits.
- Number of data blocks. This shall be identical to the number of data blocks included in the software part. The data block containing the verification block table itself shall not be included. Size 16 bits.
- StartAddress [size 32 bits] , Length [size 32 bits], Hash value. Every data blocks (m) shall be defined, except the block table itself. The size of the hash is dependent of the hash function used and is specified elsewhere.
- The data blocks shall be sorted according to the order the data blocks are being programmed to the ECU.

Example.

- SHA-256 hash function is used, i.e. each hash value is 256 bits.
- Number of data blocks are two, i.e. the total length of the verification block table, in octets, will be (16+16+2x(256+32+32))/8=84 octets (bytes).
- Note that unused space of the *Verification Block Table Format Identifier, Number of data blocks*, *startAddress* and *length* always must be padded to having the expected size (16

or 32 bits respectively).

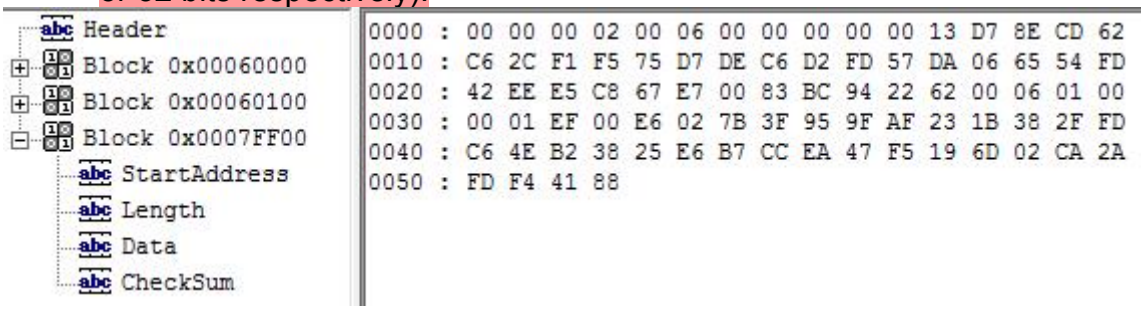


Figure - Example of a VBF file with a verification block table

Figure - Example of a VBF file with a verification block table shows a software part (VBF data file). The data shown belongs to the verification block. The Verification block starts at address 0x0007FF00, i.e. it is the third data block in the figure:
0000
0002
00060000 00000013
D78ECD62C62CF1F575D7DEC6D2FD57DA066554FD42EEE5C867E70083BC942262
00060100 0001EF00
E6027B3F959FAF231B382FFDC64EB23825E6B7CCEA47F5196D02CA2AFDF44188

- The format identifier is 0x0000
- Number of data blocks are two (0x0002)
- Data block 1: Start address 0x00060000 and length 0x00000013. Hash 0xD78ECD...
- Data block 2: Start address 0x00060100 and length 0x0001EF00. Hash 0xE6027B...

1601 v4 **Verification Block Root Hash**

(3) => (4):
Attribute 'Legacy ID' modified: 404694v1 => 404694v2.
Description modified.

Current:Verification Block Root Hash(4)

Req:
The verification_block_root_hash must be calculated on the "raw" unprocessed data in the verification block table, i.e. when no prior to any padding, encryption/compression/delta encoding method hasmethods have been applied to the VBT data block. The verification block root hash shall be calculated as Hash(startAddress||length||Verification Block Table data), where || represents the concatenation. The order shall be according to *Figure - Verification Block Root Hash calculation*. The start address (verification_block_start) and length (verification_block_length) are included to ensure the placement of the verification block and shall always be 32 bits long each. The (root) hash function shall be the same as used when calculating the hash value of the data blocks defined in the verification block table (REQPROD 404696).

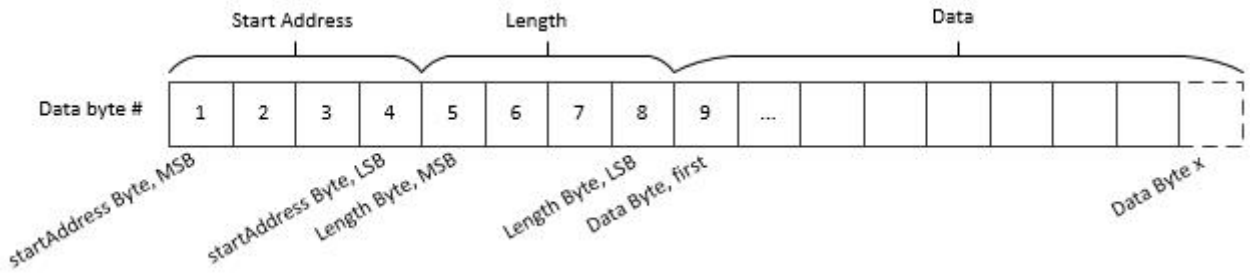


Figure - Verification Block Root Hash calculation

The root hash value shall be added by the software supplier to the header of the software part.

Example 1.

All values are in hexadecimal. SHA-256 is used as hash function.

Hash(startAddress||length||Verification Block Table data)=

Hash(*00 05 FF 00***00 00 00 54** 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1) =

9B B4 E0 92 1C 4B 84 1C 31 C3 CB 3F DD 15 D0 B9 E8 ED 70 5E D3 C2 52 DC C9 2A D5 06 E0 5E 2B AE.

Start Address: *0005FF00*

Length: **00000054**

Data: 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1

Example 2.

The length of the data block for a verification block table is 44 bytes long, prior to any padding or processing method is applied.

All data blocks shall be aligned to a multiple of 8 bytes. Hence, the VBT data block is padded to 48 bytes, but verification_block_length is still 44 (0x2C). It is this length that is used when calculating the verification_block_root_hash.

Note: Be aware of the differences of how to manage the padding of data and start/length information for data blocks that are defined in the verification block table versus the data block containing the VBT.

Previous: Verification Block Root Hash(3)

The verification_block_root_hash must be calculated on the unprocessed verification block table, i.e. when no encryption/compression/delta encoding method has been applied. The verification block root hash shall be calculated as Hash(startAddress||length||Verification Block Table data), where || represents the concatenation. The order shall be according to *Figure - Verification Block Root Hash calculation*. The start address and length are included to ensure the placement of the verification block and shall always be 32 bits long each.

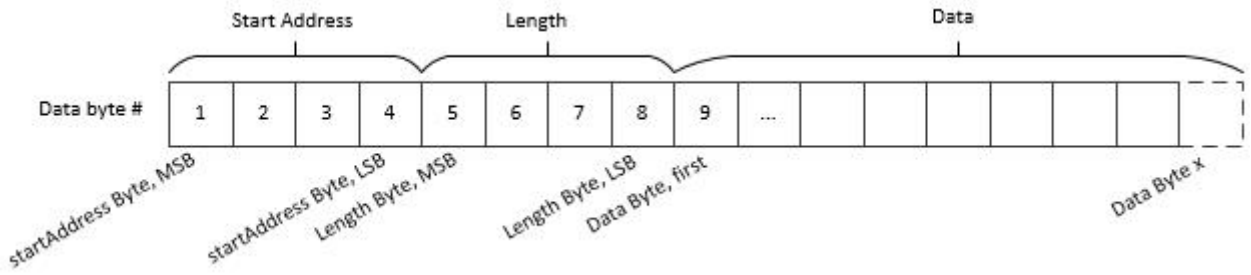


Figure - Verification Block Root Hash calculation
The hash value shall be added by the software supplier to the header of the software part.

Example.
All values are in hexadecimal. SHA-256 is used as hash function.
Hash(startAddress||length||Verification Block Table data)=
Hash(*00 05 FF 00***00 00 00 54** 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1) =
9B B4 E0 92 1C 4B 84 1C 31 C3 CB 3F DD 15 D0 B9 E8 ED 70 5E D3 C2 52 DC C9 2A D5 06 E0 5E 2B AE.

Start Address: *0005FF00*
Length: **00000054**
Data: 00 00 00 02 00 00 F0 00 00 00 00 88 B7 07 24 15 45 A3 46 26 5A AB 1F FB 32 FF 64 B5 5B F8 F8 DC 1B 56 A4 6E F3 3C E3 D1 5D B1 1D 33 00 00 F1 00 00 00 00 10 5A 2C FE 8A B9 35 91 85 25 D4 4F D6 FD 87 C7 0F C8 3B 4F 29 D1 A7 27 67 2E 1B 48 F3 80 47 3F C1

1603 v4 **Verification Block Table Format Identifier - Definition**

(3) => (4):
Attribute 'Legacy ID' modified: 404696v2 => 404696v3.
Description modified.

Current:Verification Block Table Format Identifier - Definition(4)

The Verification Block Table Format Identifier shall be according to *Table - Verification Block Format Identifier*.

Format of the Verification Block Table	Verification Block Format Identifier	Hash function	Comment
As defined in REQPROD 398360	0x0000	SHA-256	Hash(Data block)
As defined in REQPROD 398360	0x0001	SHA-384	Hash(Data block)
As defined in REQPROD 398360	0x0002	SHA-512	Hash(Data block)
-	0x0003-0xFFFF		Spare

Table - Verification Block Format Identifier

Example.
The signing method used is, as defined in REQPROD 410732, e.g. SHA256_RSA2048. This means that Verification Block Table Format Identifier shall be 0x0000.

Previous: Verification Block Table Format Identifier - Definition(3)

The Verification Block Table Format Identifier shall be set to 0 (padded with zeroes to have the size of 16 bits).

Format of the Verification Block Table	Verification Block Format Identifier	Comment
As defined in REQPROD 398360	0x0000	Hash function: SHA-256
-	0x0001-0xFFFF	Spare, future growth.

Table - Verification Block Format Identifier

1604 v4 Verification Block Table Header Identifiers

(3) => (4):
Attribute 'Legacy ID' modified: 404693v2 => 404693v3.
Description modified.

Current:Verification Block Table Header Identifiers(4)

The header section of the delivered software part must contain the following identifier with respect to the verification block table, when a verification block table is constructed according to the format in "REQPROD 398360: Format of the Verification Block Table".
Alternative 1 - verification block header identifiers (default configuration):

- **verification_block_root_hash**: The hash value of the unprocessed verification block table.
- **verification_block_start**: The start address of the unprocessed verification block table.
- **verification_block_length**: The length of the unprocessed verification block table.

Upon OEM approval, the header section can exclude the verification_block_start and verification_block_length identifiers:
Alternative 2 - verification block header identifiers:
verification_block_root_hash

This might be applied for ECUs where existing installation and verification mechanisms, using e.g. ECU specific manifests, are to be reused in the ECU to ensure authenticity that is not possible to realize using the Volvo verification block table format.
Example.

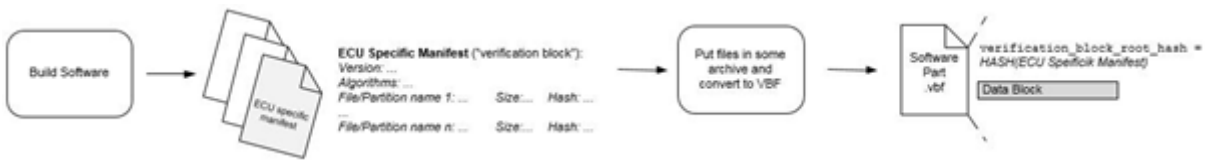


Figure - Example of ECU supplier specific verification block (manifest)

"Figure - Example of ECU supplier specific verification block (manifest)" shows a data file header where verification_block_root_hash is present only.

Previous: Verification Block Table Header Identifiers(3)

The header section of the delivered software part shall contain the following identifiers with respect to the verification block table:

- **verification_block_root_hash**: The hash value of the unprocessed verification block table.
- **verification_block_start**: The start address of the unprocessed verification block table.
- **verification_block_length**: The length of the unprocessed verification block table.

1605 v3 **Verification Block Table on unprocessed data**

(2) => (3):
Attribute 'Legacy ID' modified: 404184v1 => 404184v2.
Description modified.

Current:Verification Block Table on unprocessed data(3)

Req:
The data block definitions in the Verification Block Table shall be prior to processing methods have been applied, i.e. prior to the any compression, encryption or delta encoding is applied. If there is a need to align the data blocks to a specific size prior to a processing method is applied (e.g. to match a boundary of a flash sector and to be a multiple of the minimum programmable block size for the specific ECU), it is the padded data blocks that are to be used as inputs when defining the data blocks in the Verification Block Table.

Example.

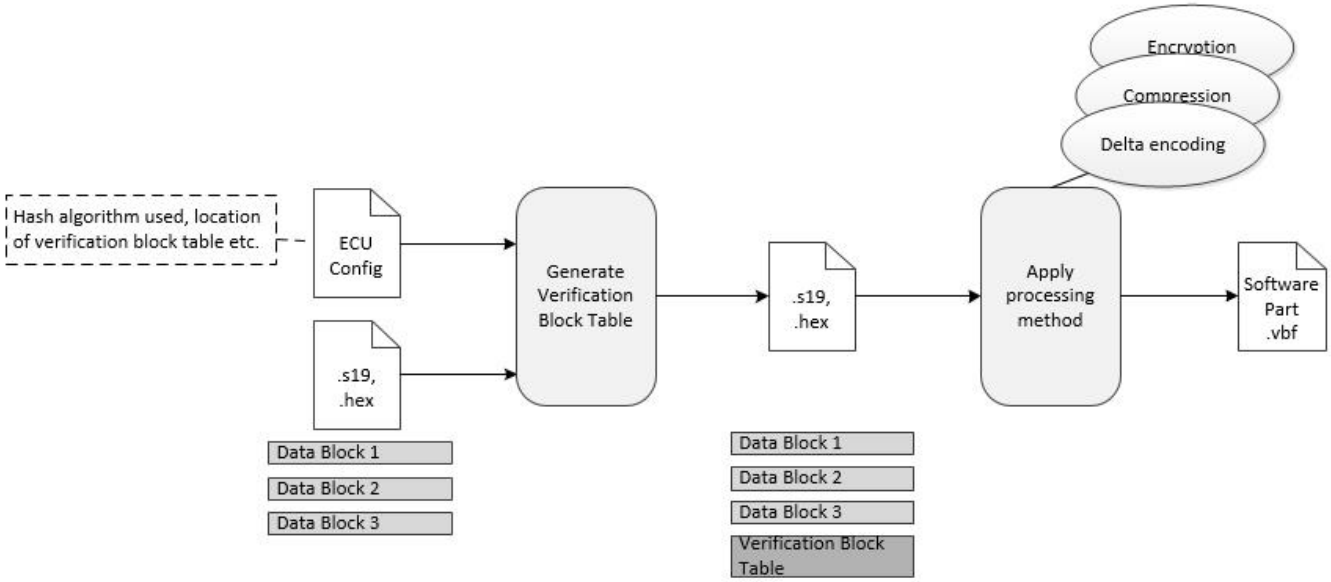


Figure - Generate verification block table on unprocessed data

Previous: Verification Block Table on unprocessed data(2)

The Verification Block Table shall be calculated on data (blocks) prior to some processing methods have been applied, i.e. prior to the any compression, encryption or delta encoding is applied.

Example.

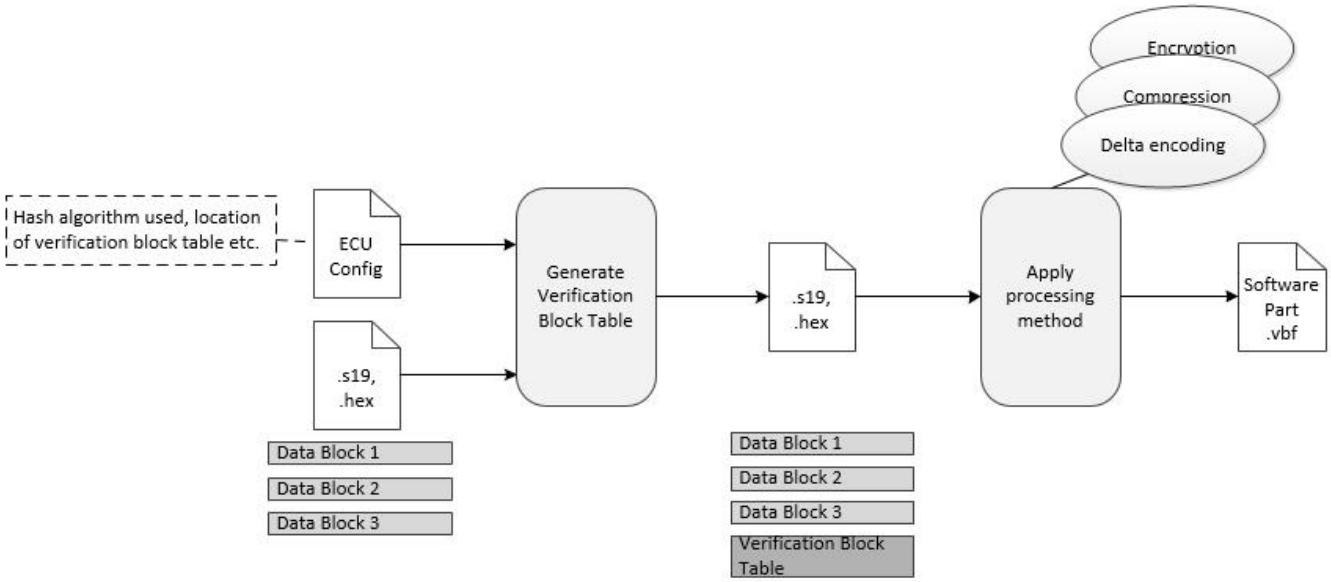


Figure - Generate verification block table on unprocessed data

1608 v3 **Public Key Types**

(2) => (3):
Attribute 'Legacy ID' modified: 442538v0 => 442538v1.
Description modified.

Current:Public Key Types(3)

Information about the public key type must be provided by the entity managing the keys. That information is optionally used to control how the diagnostic client shall program the public key, e.g. what diagnostic service(s) that shall be applied.
The Public key type (PublicKeyModulus||PublicKeyExponent||PublicKeyChecksum) shall be referred to as "RAW_KEY_WITH_CHECKSUM".

This requirement is not applicable for the ECU supplier but for off-board systems.

Previous: Public Key Types(2)

Information about the public key type must be provided by the entity managing the keys. That information is optionally used to control how the diagnostic client shall program the public key, e.g. what diagnostic service(s) that shall be applied.

This requirement is not applicable for the ECU supplier but for off-board systems.

1610 v5 Append signature to software part

(4) => (5):
Description modified.

Current:Append signature to software part(5)

The digital signature when the software part is signed with the intended private key for **development**, sw_signature_dev, shall be appended to the software part header by the software supplier when a part is delivered.. The resulting sw_signature signed by the trust centre at OEM with the intended key for **production** shall be stored in a separate file, using a standardized format separately (e.g. .xml). The signing tool must support both sign and re-sign functionality of a software part.

Example.
(1) A software part is delivered containing the sw_signature_dev. This software can only be used in an ECU equipped with the public key for development.
(2) The software part is, when successfully verified, validated and ready for production, signed with the private key for production. This means that two signatures are present (the sw_signature_dev is kept).
(3) This software part can now be used in both a development and production ECUs, but it must be ensured that the corresponding signature is transferred at SWDL.

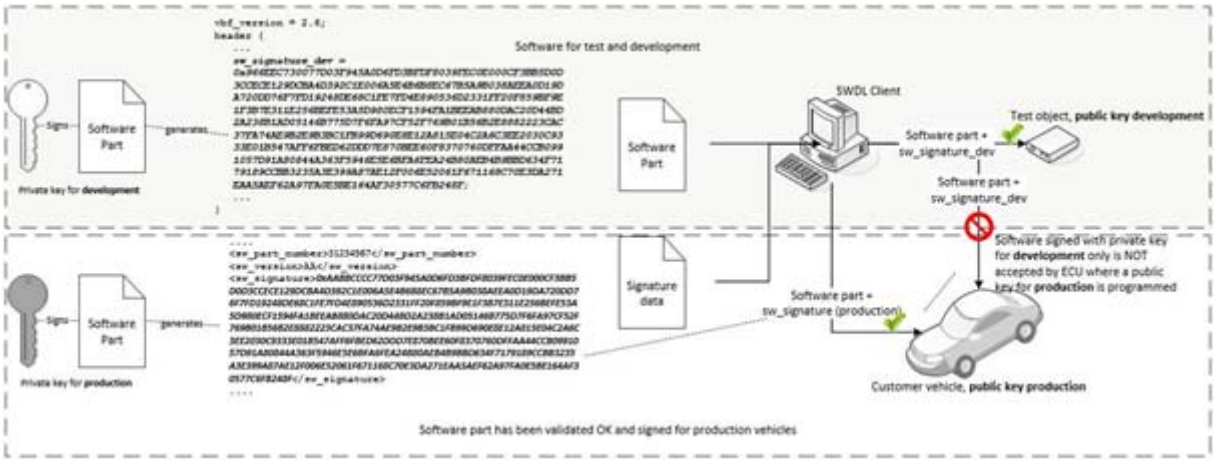


Figure - Signature for test and development and production

Previous: Append signature to software part(4)

The digital signature when the software part is signed with the intended key for development, sw_signature_dev, shall be appended to the software part header by the software supplier when a part is delivered. The sw_signature signed by the trust centre with the intended key for production shall be stored in a separate file, using a standardized format (e.g. .xml). The signing tool must support both sign and re-sign functionality of a software part.
Example.
(1) A software part is delivered containing the sw_signature_dev. This software can only be used in an ECU equipped with the public key for development.
(2) The software is, when successfully verified, signed with the private key for production. This means that two signatures are present.
(3) This software part can now be used in both a development and production ECU, but it must be ensured that the corresponding signature is transferred at SWDL.

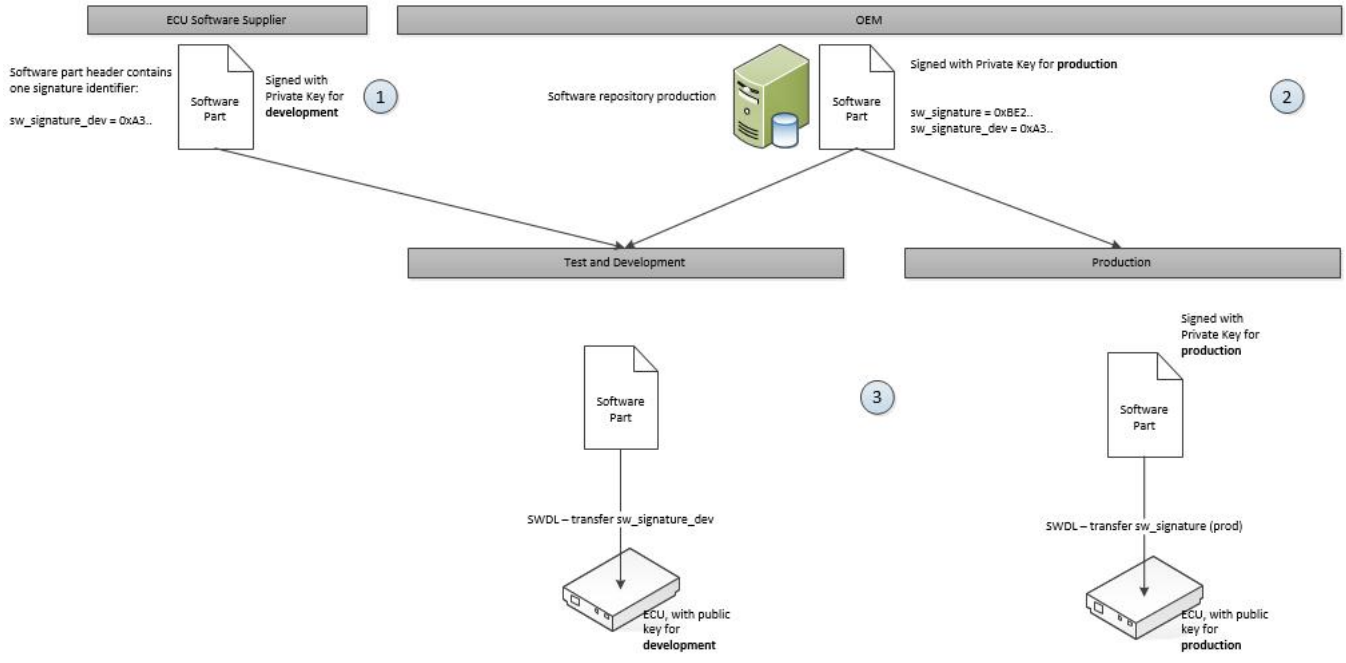


Figure - Signature for test and development and production

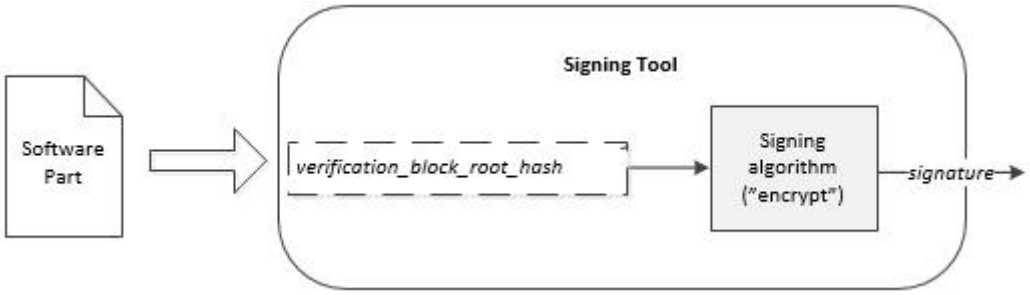
1612 v4 **Signing function - Generate signature**

(3) => (4):
Attribute 'Legacy ID' modified: 428667v1 => 428667v3.
Description modified.

Current:Signing function - Generate signature(4)

The data to be signed is defined by the verification_block_start and verification_block_length identifiers. These start address, length and data of the addressed data block shall be used as inputs when generating the hash value. The signing entity has two possibilities:
(1) always calculate the hash value over the verification block table and ensure that the content of the table is correct. This is the preferred option.
(2) use the already calculated hash value (verification_block_root_hash).

Example.
The verification_block_root_hash might be used when the processing method is not known (e.g. delta compression method) as defined by the software part data format identifier but it might also be used to optimize off-board functions (e.g. computational time), alternatively when the verification_block_start and verification_block_length are not present in data file header.

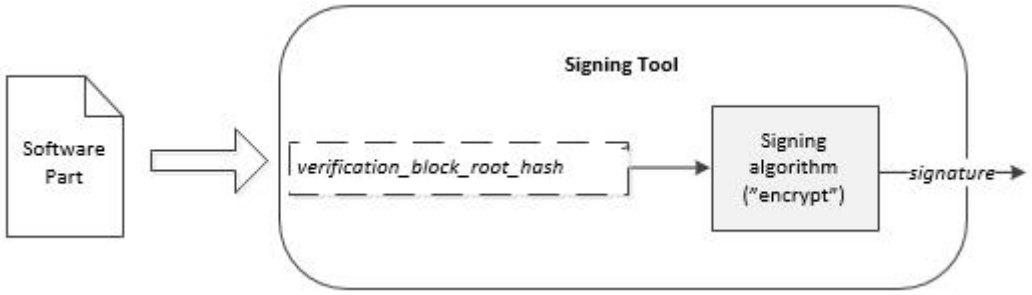


Sign - Processing method unknown

Previous: Signing function - Generate signature(3)

The data to be signed is defined by the **verification_block_start** identifier. The start address, length and data of the addressed data block shall be used as inputs when generating the hash value. The signing entity has two possibilities:
(1) calculate the hash value, even though the expected hash value (verification_block_root_hash) is present in the file header.
(2) use the already calculated hash value (verification_block_root_hash).

Example.
The verificaiton_block_root_hash is used, typically when the processing method is not known (e.g. encryption method applied) as defined by the software part data format identifier but it might also be used to optimize off-board functions (e.g. computational time).



Sign - Processing method unknown

1614 v5 **Sign a delta file**

(4) => (5):
Attribute 'Legacy ID' modified: 429082v2 => 429082v3.
Description modified.

Current:Sign a delta file(5)

A delta software part shall contain the same header identifiers as the corresponding full version, with respect to following Software Authentication identifiers: verification_block_start, verification_block_length, verification_block_root_hash and sw_signature_dev.

Previous: Sign a delta file(4)

A delta software part shall contain the same header identifiers as the corresponding full version, with respect to following Software Authentication identifiers: verification_block_start, verification_block_length, verification_block_root_hash. The signature(s) shall be calculated using the verification_block_root_hash as input.

1629 v43 **ECU Addresses**

(42) => (43):
Description modified.

Current:ECU Addresses(43)

FLEXRAY
ECUs with Flexray interface
FlexRay ECUs shall use FlexRay Frame ID's in the static segment【 to address application communication and Frame ID's in the dynamic segment to address diagnostic and SWDL communication. The Flexray Frame ID is included in the ECU extract.

CAN
ECUs with CAN interface
All CAN ECUs shall have a CAN ID according to the table below. The CAN ID is a static address calculated from the ECU address and is used to address CAN ECUs for SWDL and Diagnostics. The CAN ID is included in the ECU extract.

LIN
LIN slave ECUs, using SWDL functionality (bootloader)
All LIN ECUs using a bootloader shall use a complete ECU address according to the table below. The ECU address is a static address that is used to address LIN ECUs for SWDL and

Diagnostics. The ECU address is included in the LDF file.

LIN slave ECUs without SWDL, but using LIN diagnostic frames
A LIN slave ECU (not using a bootloader) shall use a NAD address according to the table below. Otherwise the LIN master will not be able to address diagnostic requests to the LIN slave. The NAD address is the last byte from the 2 byte ECU address and is used for diagnostics. The NAD address is included in the LDF file.

LIN slave ECUs without SWDL and without LIN diagnostic frames
If there is no need for a LIN master to address diagnostic requests to a LIN slave ECU, then a NAD address is NOT needed. In this case the LIN slave will be assigned a node address default value. The default value will be 0x7D. In case of multiple LIN Slaves on the same network the addresses will be set in decreasing order starting from 0x7D. The LIN address to be used by the LIN slave is included in the LDF file.

LIN masters
LIN masters shall use the NAD according to table ECU Addresses below in order to address diagnostic requests to LIN slaves.

MOST
ECUs with MOST150 interfaces
MOST Logical Node Address (RxTxLog) is a static address predefined as according to the table below and is used by MOST applications when using the control channel on MOST. The MOST Logical Node Address must be implemented into the MOST ECU by supplier.

IP
ECUs with IP-based interfaces
The internal MAC addresses and internal IP addresses of IP-ECUs shall be assigned as according to table ECU Addresses. The IP address must be implemented into the IP ECU by supplier. The following IP address space has been chosen: 198.18.0.0/16 for the vehicle network. From this address space, the vehicle IP network is using one subnet according to below:

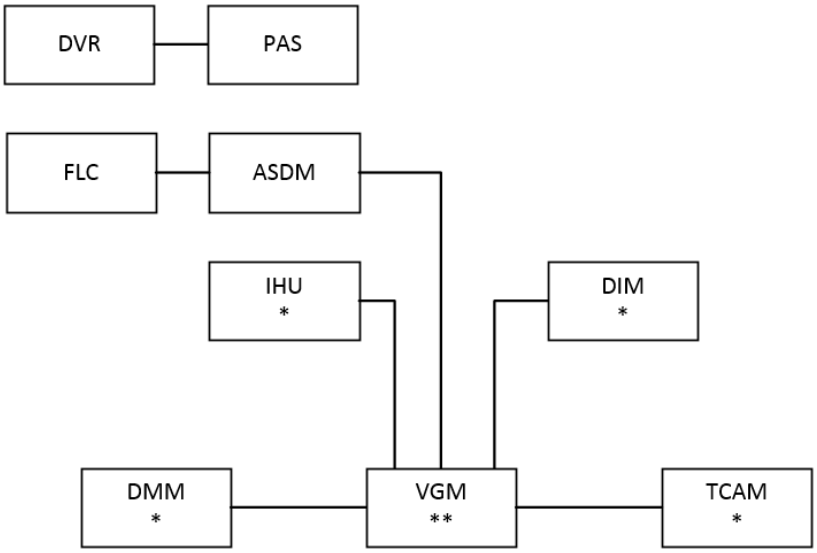


Figure:IP subnets of the GEEA 2.0 network

NOTE 1: In *Figure IP subnets of the vehicle network*, (the Ethernet capability of APIX2 link is not used for diagnostics. Instead, FlexRay is used for diagnostics.

NOTE 2: In *Figure IP subnets of the vehicle network* and in the *table "ECU Addresses"*, some IP addresses are marked with a * or a **.

* == Internal ECU-IP address used for Diagnostic (ISO13400) TCP clients (Diagnostic Server) to connect to the Edge Node (VGM).

** == Internal ECU-IP address used by Edge Node (VGM) acting as TCP Server (Diagnostic Client) offering Diagnostic Service

Overview of IP-address space :

The vehicle IP network shall use one subnet according to table below.

Network Friendly Name:	**VEHICLE IP NETWORK**
Subnet ID:	198.18.0.0 / 16
Usable IP addresses:	198.18.0.1 – 198.18.255.254
Netmask:	255.255.0.0
Broadcast address:	198.18.255.255
Default GW address:	VGM – Ethernet interface according to, ***Table:*** *ECU addresses – Showing IP-based ECU's*

IP address space allocation depending on physical network

The IP address range of 198.18.0.0 / 16 has been divided into different address ranges to be able to identify which physical communication carrier is used for a specific ECU. The tables below are used in conjunction with REQPROD 342169.

Example: ECU's which are connected to Ethernet will have an IP address range of: 198.18.32.1-198.18.47.255.

Network Friendly Name:	**MOST150**
Network ID:	198.18. **0**.0
IP addresses:	198.18.0.1 – 198.18.15.255

Network Friendly Name:	**APIX/LVDS**
Network ID:	198.18. **16**.0
IP addresses:	198.18.16.1 – 198.18.31.255

Network Friendly Name:	**Ethernet**
Network ID:	198.18. **32**.0
IP addresses:	198.18.32.1 – 198.18.47.255

Network Friendly Name:	**USB**
Network ID:	198.18. **48**.0
IP addresses:	198.18.48.1 – 198.18.63.255

Relationship between ECU addresses and IP addresses
The ECU address consist of 16 bits and are divided into 4 nibbles as follows: [Address range]. [Domain id].[Network id].[Ecu id]

NOTE: The addressing format for diagnostic request and response, source address (SA) and target address (TA), using Domain ID, Network ID and ECU ID is defined in [*UDS Services*].

The IP address v4 consist of 32 bits and are divided into 4 bytes. It is required for diagnostic purposes, that it shall be possible to calculate the ECU address from the IP address of a specific ECU. The IP addresses have been divided as follows:

198.18.{ssssdddd}.{nnnneeee}.

ssss = Consists of 4 bits, defines the start of the Network ID.
dddd = Consists of 4 bits, defines the Domain id
nnnn = Consists of 4 bits, defines the Network id
eeee = Consists of 4 bits, defines the ECU id

The ECU addresses are always represented in hexadecimal form and the IP addresses are written in decimal form. This means when calculating the ECU address from the IP address there is a need for conversion from hexadecimal to decimal.

Example:
ECU Address 0x1212, would mean IP address 198.18.2.18.

198.18.[S] [D].[N][E]

[S]=0x0 (Network Friendly Name = “MOST150”, Network ID: 198.18. **0**.0)
[D] = 0x2
[N] = 0x1
[E] = 0x2

Table: IP Address calculation

IP address calculation				
	198	**18**	**{ssssdddd}**	**{nnnneeee}**
Binary	1100 0110	0001 0010	0000 0010	0001 0010
Hex	0xC6	0x12	0x02	0x12

Decimal	198	18	2	18

Table: ECU addresses

ECU Name	ECU Alias	Diagnostic Network = Source network for SWDL	ECU Address	CAN ID	NAD (LIN)	Internal MAC Addresses	Internal IP Addresses
Air Condition Control Module	ACCM	ECM_LIN1	N/A	N/A	0x7D	N/A	N/A
Air Fragrance Unit	AFU	CCM_LIN2	N/A	N/A	0x7C	N/A	N/A
Active Grill Shutter Module	AGM	ECM_LIN4	N/A	N/A	0x7A	N/A	N/A
Ambience Lights Module	ALM1	CEM_LIN5	N/A	N/A	0x7D	N/A	N/A
Ambience Lights Module	ALM2	CEM_LIN8	N/A	N/A	0x7D	N/A	N/A
Air Particulate Matter Sensor	APMS	CCM_LIN2	N/A	N/A	0x7B	N/A	N/A
Active Safety Domain Master	ASDM	Backbone FlexRay	0x1401	0x701	N/A	Ethernet: 02:00:00:00:14:01	Ethernet: 198.18.36.1
Active Spoiler Shutter Module	ASSM	ECM_LIN4	N/A	N/A	0x7C	N/A	N/A
Adjustable Steering Wheel Module	ASWM	CEM_LIN4	N/A	N/A	0x7D	N/A	N/A
Audio Module	AUD	Private Info CAN	0x1212	0x712	N/A	N/A	Ethernet: 198.18.34.1 (Note 1)
Brake Booster Module	BBM	Backbone FlexRay	0x1602	0x702	N/A	N/A	N/A
Battery Backed-up Sounder	BBS	CEM_LIN2	N/A	N/A	0x7C	N/A	N/A
Battery Circuit Four way Valve	BCFV	ECM_LIN1	N/A	N/A	0x7B	N/A	N/A
Battery Cooling Pump Front module	BCPF	ECM_LIN3	N/A	N/A	0x7D	N/A	N/A
Battery Circuit Three way Valve	BCTV	ECM_LIN1	N/A	N/A	0x7A	N/A	N/A
Battery Energy Control Module	BECM	Propulsion CAN	0x1635	0x735	N/A	N/A	N/A
Battery Electric Expansion Valve	BEXV	CCM_LIN2	N/A	N/A	0x77	N/A	N/A
Battery Monitoring Sensor	BMS	CEM_LIN6	0x1B61	N/A	0x61	N/A	N/A
Bluetooth NFC CommunicationModule	BNCM(NFC)	TCAM_LIN1	N/A	N/A	0x7C	N/A	N/A
Bluetooth Short Range Module	BSRM	TCAM_LIN1	N/A	N/A	0x7D	N/A	N/A
Climate Control Module	CCM	Body CAN	0x1A11	0x711	N/A	N/A	N/A
Center Console Switch Module	CCSM	IHU_LIN2	N/A	N/A	0x7D	N/A	N/A
Charger and DCDC Module	CDD	Propulsion CAN	0x1650	0x750	N/A	N/A	N/A
Carbon Dioxide Sensor	CDS	CCM_LIN2	N/A	N/A	0x7D	N/A	N/A
Central Electronic Module	CEM	Backbone FlexRay	0x1A01	0x701	N/A	N/A	N/A
Condenser Electric Expansion Valve	CEXV	CCM_LIN2	N/A	N/A	0x73	N/A	N/A
Centre Stack Display	CSD	GMSL	0x1241	N/A	0x41	N/A	Ethernet: 198.18.34.1 (Note 1)
Cloud Service Provider	CSP	N/A	N/A	N/A	N/A	N/A	N/A
Drivetrain Coolant Pump	DCP	ECM_LIN3	N/A	N/A	0x7B	N/A	N/A

Driver Door Module	DDM	Body CAN	0x1A12	0x712	N/A	N/A	N/A
Driver Door Switches	DDS	DDM_LIN1	N/A	N/A	0x7D	N/A	N/A
Differential Electronic Module	DEM	Propulsion CAN	0x1638	0x738	N/A	N/A	N/A
Display Head Unit	DHU	Ethernet	0x1201	0x701	N/A	Ethernet: 02:00:00:02:12:01	Ethernet: 198.18.34.1* (Note 4)
Display Head Unit	DHU	Ethernet	0x120F	N/A	N/A	Ethernet: 02:00:00:02:12:15	Ethernet: 198.18.34.15 (Note 4)
Driver Information Module	DIM	Backbone FlexRay	0x1801	N/A	N/A	GPU SWDL via Ethernet (Note 2)	GPU SWDL via Ethernet (Note 2)
Driver Information Module	DIM	DIM Ethernet	0x1013	N/A	N/A	Ethernet: 02:00:00:00:10:13	Ethernet: 198.18.32.19
Driver Information Screen	DIS	DIM_LIN1	0x1253	N/A	DIM_LIN1: 0x53	N/A	Ethernet: 198.18.34.1 (Note 1)
Driver Information Screen	DIS	DIM_LIN1	0x1853	N/A	DIM_LIN1: 0x53	N/A	N/A
Driver Monitor Module	DMM	DMM Ethernet	0x1012	0x712	N/A	Ethernet: 02:00:00:00:10:12	Ethernet: 198.18.32.18 *
Drive Mode Switch Module	DMSM	IHU_LIN2	N/A	N/A	0x7C	N/A	N/A
Passenger Door Module	PDM	Body CAN	0x1A13	0x713	N/A	N/A	N/A
Power Operated Tailgate	POT	Body CAN	0x1A15	0x715	N/A	N/A	N/A
Dedicated Short Range Communication	DSRC	Connectivity CAN HS	0x1021	0x721	N/A	N/A	N/A
Digital Video Recorder	DVR	Passive Safety CAN	0x1514	0x714	N/A	Ethernet: 02:00:00:00:15:14	198.18.37.20
Engine Control Module	ECM	Propulsion CAN	0x1630	0x730	0x30	N/A	N/A
Engine Coolant Pump Module	ECPM	ECM_LIN3	N/A	N/A	0x06	N/A	N/A
Electrical motor Cooling Regulator Valve module	ECRV	ECM_LIN1	N/A	N/A	0x10	N/A	N/A
Electric Drivetrain Coolant Pump	EDCP	ECM_LIN3	N/A	N/A	0x5A	N/A	N/A
Evaporator Electric Expansion Valve	EEXV	CCM_LIN2	N/A	N/A	0x78	N/A	N/A
Electronic Gear Selector Module	EGSM	Propulsion CAN	0x1633	0x733	0x33	N/A	N/A
Exterior Sound Module	ESM	Propulsion CAN	0x163A	0x73A	N/A	N/A	N/A
Electric Vehicle Communication Controller	EVCC	Private CAN	N/A	N/A	N/A	N/A	N/A
Fast Charger Socket Indicator	FCSI	CEM_LIN7	N/A	N/A	0x7D	N/A	N/A
Forward Loooking Camera	FLC	FlexRay	0x1411	0x711	N/A	N/A	N/A
Forward Looking Radar	FLR	CAN FD 2	0x1444	0x744	0x7B	N/A	N/A
Foot Movement Detection Module	FMDM	CEM_LIN6	N/A	N/A	0x62	N/A	N/A
Front Side Radar Left	FSRL	CAN FD 1	0x1431	0x731	0x78	N/A	N/A
Front Side Radar Right	FSRR	CAN FD 1	0x1430	0x730	N/A	N/A	N/A

HVAC Blower Module Front	HBMF	CCM_LIN1	N/A	N/A	0x7D	N/A	N/A
Headlamp Control Module Left	HCML	Body Exposed CAN	0x1BB3	0x7B3	N/A	N/A	N/A
Headlamp Control Module Right	HCMR	Body Exposed CAN	0x1BB4	0x7B4	N/A	N/A	N/A
Heater Circuit Three way Valve	HCTV	ECM_LIN1	N/A	N/A	0x11	N/A	N/A
HVAC Flap Actuators	HFA	CCM_LIN4	N/A	N/A	0x7D	N/A	N/A
Hands Off Detection	HOD	DIM_LIN1	N/A	N/A	0x7C	N/A	N/A
Head Up Display	HUD	DIM_LIN1	0x1251	N/A	DIM_LIN1: 0x51	N/A	Ethernet: 198.18.34.1 (Note 1)
Head Up Display	HUD	DIM_LIN1	0x1851	N/A	DIM_LIN1: 0x51	N/A	N/A
High Voltage Coolant Heater	HVCH	CCM_LIN2	N/A	N/A	0x7A	N/A	N/A
High Voltage Secondary Conversionmodule	HVSC	Chassis CAN 1	0x1673	0x773	N/A	N/A	N/A
Inverter ERAD Module	IEM	Propulsion CAN	0x1637	0x737	N/A	N/A	N/A
Inverter Generator Module	IGM	Propulsion CAN	0x1636	0x736	N/A	N/A	N/A
Infotainment Head Unit	IHU	Ethernet	0x1201	0x701	N/A	Ethernet: 02:00:00:02:12:01	Ethernet: 198.18.34.1* (Note3)
Infotainment Head Unit	IHU	Ethernet	0x120F	N/A	N/A	Ethernet: 02:00:00:02:12:15	Ethernet: 198.18.34.15 (Note3)
Trailer Module	TRM	Body CAN	0x1A17	0x717	N/A	N/A	N/A
Interior Movement Sensor	IMS	CEM_LIN2	N/A	N/A	0x7B	N/A	N/A
Interia Monitor Unit	IMU	Private CAN	N/A	N/A	N/A	N/A	N/A
Interior Rear view Mirror Module	IRMM	CEM_LIN1	N/A	N/A	0x13	N/A	N/A
Integrated Start Generate Module	ISGM	Propulsion CAN	0x163E	0x73E	N/A	N/A	N/A
Rear Left Door Module	RLDM	Body CAN	0x1A21	0x721	N/A	N/A	N/A
Motor Generator Module	MGM	Propulsion CAN	0x1631	0x731	N/A	N/A	N/A
Driver Massage Module	MMD	SMD_LIN1	0x1A32	0x732	0x32	N/A	N/A
Massage Module Passenger	MMP	SMP_LIN1	0x1A33	0x733	0x33	N/A	N/A
Medium Voltage Battery Module	MVBM	Propulsion CAN	0x1652	0x752	N/A	N/A	N/A
Middle Voltage Converter Module	MVCM	Propulsion CAN	0x163D	0x73D	N/A	N/A	N/A
Middle Voltage Electrical Motor	MVEM	Propulsion CAN	0x1651	0x751	N/A	N/A	N/A
On Board Charger	OBC	Propulsion CAN	0x1634	0x734	N/A	N/A	N/A
Overhead Console	OHC	CEM_LIN3	N/A	N/A	0x7D	N/A	N/A
Occupant Weight Sensor	OWS	SRS_LIN1	N/A	N/A	0x42	N/A	N/A
Parking Assist System	PAS	CAN FD 4	0x1460	0x760	N/A	02:00:00:00:14:60	198.18.36.96
Rear Right Door Module	RRDM	Body CAN	0x1A22	0x722	N/A	N/A	N/A
Air Particulate Matter Sensor Indoor	PMSI	CCM_LIN2	N/A	N/A	0x76	N/A	N/A
Air Particulate Matter Sensor Outdoor	PMSO	CCM_LIN2	N/A	N/A	0x75	N/A	N/A
Driver Power Operated	DPOD	DRM_LIN1	N/A	N/A	0x7D	N/A	N/A

Door							
Passenger Power Operated Door	PPOD	DRM_LIN1	N/A	N/A	0x7C	N/A	N/A
Power Steering Control Module 1	PSCM1	Chassis CAN 1	0x1670	0x770	N/A	N/A	N/A
Power Steering Control Module 2	PSCM2	Chassis CAN 1	0x1671	0x771	N/A	N/A	N/A
Passenger Smart Surface Module	PSSM	IHU_LIN2	N/A	N/A	0x7B	N/A	N/A
Rear Light Control Module Left	RCML	Body Exposed CAN	0x1BB5	0x7B5	N/A	N/A	N/A
Rear lamp Control Module Middle	RCMM	Body Exposed CAN	0x1BB7	0x7B7	N/A	N/A	N/A
Rear Light Control Module Right	RCMR	Body Exposed CAN	0x1BB6	0x7B6	N/A	N/A	N/A
Rear Console Switch Module	RCSM	IHU_LIN2	N/A	N/A	0x7A	N/A	N/A
Left Power Operated Door	LPOD	DRM_LIN1	N/A	N/A	0x7B	N/A	N/A
Rain and Light Sensor Module	RLSM	CEM_LIN1	N/A	N/A	0x7B	N/A	N/A
Rear View Mirror Display	RMD	Passive Safety CAN	0x1513	0x713	N/A	N/A	N/A
Restraint Module Left	RML	Passive Safety CAN	0x1510	0x710	N/A	N/A	N/A
Restraint Module Right	RMR	Passive Safety CAN	0x1511	0x711	N/A	N/A	N/A
Rear Power Operated Door	RPOD	DRM_LIN1	N/A	N/A	0x7A	N/A	N/A
Seat Module Driver	SMD	Body CAN	0x1A27	0x727	N/A	N/A	N/A
Rear Seat Display Module	RSDM	IHU_LIN2	N/A	N/A	0x78	N/A	N/A
Rear Seat Entertainment Module	RSEM	GMSL	N/A	N/A	0x7D	N/A	N/A
Steering Angle Sensor	SAS	Chassis CAN 1	0x1672	0x772	N/A	N/A	N/A
Second Battery Monitoring Sensor	SBMS	ECM_LIN3	N/A	N/A	0x76	N/A	N/A
Steering Column Lock	SCL	Chassis CAN 2	0x1615	0x715	N/A	N/A	N/A
Sun Curtain Module	SCM	CEM_LIN3	N/A	N/A	0x7B	N/A	N/A
Sunroof Intelligent Change Colour Glass Module	SIG	CEM_LIN3	N/A	N/A	0x79	N/A	N/A
Seat Module Passenger	SMP	Body CAN	0x1A28	0x728	N/A	N/A	N/A
Seat Module Back	SMB	Body CAN	0x1A29	0x729	N/A	N/A	N/A
Interior Lighting Control Module	ILCM	Body CAN	0x1A30	0x730	N/A	N/A	N/A
Side Obstacle Detection Left	SODL	CAN FD 3	0x1450	0x750	N/A	N/A	N/A
Side Obstacle Detection Right	SODR	CAN FD 3	0x1451	0x751	N/A	N/A	N/A
Sun Roof Module	SRM	CEM_LIN3	N/A	N/A	0x78	N/A	N/A
Supplementary Restraint System	SRS	Backbone FlexRay	0x1C01	0x701	N/A	N/A	N/A
Suspension Module	SUM	Chassis CAN 2	0x1614	0x714	N/A	N/A	N/A
Steering Wheel Module	SWM	CEM_LIN4	0x1B72	N/A	0x72	N/A	N/A
Steering Wheel Switch Module	SWSM	DIM_LIN1	N/A	N/A	0x7D	N/A	N/A
TBD	SWTL	Private DHU CAN	N/A	N/A	N/A	N/A	N/A
TBD	SWTR	Private DHU CAN	N/A	N/A	N/A	N/A	N/A
Transmission Actuator Control Module	TACM	Propulsion CAN	0x163C	0x73C	0x3C	N/A	N/A

Telematics & Connectivity Module	TCAM	TCAM Ethernet	0x1011	0x711	N/A	Ethernet: 02:00:00:00:10:11	Ethernet: 198.18.32.17 *
Thermal Cup Holder /	TCH	IHU_LIN2	N/A	N/A	0x79	N/A	N/A
Transmission Control Module	TCM	Propulsion CAN	0x1632	0x732	N/A	N/A	N/A
Door Radar Module	DRM	Body CAN	0x1A31	0x731	0x31	N/A	N/A
VAC Blower Module Rear	VBMR	CCM_LIN2	N/A	N/A	0x74	N/A	N/A
Vehicle Dynamics Domain Master	VDDM	Backbone FlexRay	0x1601	0x701	N/A	N/A	N/A
Vent Grill Actuator	VGA	CCM_LIN3	N/A	N/A	0x7D	N/A	N/A
Vehicle Gateway Module	VGM	Connectivity Ethernet	0x1001	0x701	N/A	Ethernet: 02:00:00:00:10:01	Ethernet: 198.18.32.1 **
Wiper Motor Module	WMM	CEM_LIN1	N/A	N/A	0x12	N/A	N/A
Wireless Phone Charger/Near-field Communication	WPC/NFC	IHU_LIN1	N/A	N/A	0x7D	N/A	N/A
Wireless Power Transfer	WPT	Propulsion CAN	0x163B	0x73B	N/A	N/A	N/A

Note: This requirement is valid for all ECU's.
Note 1: The VCM/VGM shall route diagnostic traffic to infotainment subnodes via the IHU. Therefore, the subnodes need to have the same IP address as the IHU on the Ethernet interface.
Note 2: The VCM/VGM shall route diagnostic traffic with destination to DIM GPU (0x1802) via APIX2 using the stated IP address.
Note 3: The IHU is using 198.18.34.1 for the IHU VP and 198.18.34.15 for the IHU AP.
Note 4: The DHU is using 198.18.34.1 for the DHU QNX and 198.18.34.15 for the DHU Android.

Table: OBD addresses

ECU name according to SAE J1979	ECU name in GEEA2.0	29 bit address Req/Resp	11 bit address for Request/Response
ECM EngineControl	ECM	0x18DA10F1/0x18DAF110	7E0/7E8
TCM TransmisCtrl	TCM	0x18DA18F1/0x18DAF118	7E1/7E9
BECM B+EnergyCtrl	BECM	0x18DACBF1/0x18DAF1CB	7E2/7EA
BECM B+EnergyCtrl	MVBM	0x18DACBF1/0x18DAF1CB	7E2/7EA
DMCM DriveMotorCtrl	IEM	0x18DA11F1/0x18DAF111	7E3/7EB
DMCM DriveMotorCtrl	MVEM	0x18DA11F1/0x18DAF111	7E3/7EB
SGCM Start/GenCtrl	IGM	0x18DA14F1/0x18DAF114	7E4/7EC
SGCM Start/GenCtrl	MGM	0x18DA14F1/0x18DAF114	7E4/7EC
SGCM Start/GenCtrl	ISGM	0x18DA14F1/0x18DAF114	7E4/7EC

BCCM B+ChargerCtrl	OBC	0x18DACAF1/0x18DAF1CA	7E5/7ED
BCCM B+ChargerCtrl	CDD	0x18DACAF1/0x18DAF1CA	7E5/7ED
BSCM BrakeSystem	VDDM	0x18DA28F1/0x18DAF128	7E6/7EE
EBBC ElecBrkBstCtrl	BBM	0x18DA29F1/0x18DAF129	7E7/7EF

Previous: ECU Addresses(40)

FLEXRAY

ECUs with Flexray interface

FlexRay ECUs shall use FlexRay Frame ID's in the static segment【 to address application communication and Frame ID's in the dynamic segment to address diagnostic and SWDL communication. The Flexray Frame ID is included in the ECU extract.

CAN

ECUs with CAN interface

All CAN ECUs shall have a CAN ID according to the table below. The CAN ID is a static address calculated from the ECU address and is used to address CAN ECUs for SWDL and Diagnostics. The CAN ID is included in the ECU extract.

LIN

LIN slave ECUs, using SWDL functionality (bootloader)

All LIN ECUs using a bootloader shall use a complete ECU address according to the table below. The ECU address is a static address that is used to address LIN ECUs for SWDL and Diagnostics. The ECU address is included in the LDF file.

LIN slave ECUs without SWDL, but using LIN diagnostic frames

A LIN slave ECU (not using a bootloader) shall use a NAD address according to the table below. Otherwise the LIN master will not be able to address diagnostic requests to the LIN slave. The NAD address is the last byte from the 2 byte ECU address and is used for diagnostics. The NAD address is included in the LDF file.

LIN slave ECUs without SWDL and without LIN diagnostic frames

If there is no need for a LIN master to address diagnostic requests to a LIN slave ECU, then a NAD address is NOT needed. In this case the LIN slave will be assigned a node address default value: 0x7F. The LIN address to be used by the LIN slave is included in the LDF file.

LIN masters

LIN masters shall use the NAD according to table ECU Addresses below in order to address diagnostic requests to LIN slaves.

MOST

ECUs with MOST150 interfaces

MOST Logical Node Address (RxTxLog) is a static address predefined as according to the table below and is used by MOST applications when using the control channel on MOST. The MOST

Logical Node Address must be implemented into the MOST ECU by supplier.

IP

ECUs with IP-based interfaces

The internal MAC addresses and internal IP addresses of IP-ECUs shall be assigned as according to table ECU Addresses. The IP address must be implemented into the IP ECU by supplier. The following IP address space has been chosen: 198.18.0.0/16 for the vehicle network. From this address space, the vehicle IP network is using one subnet according to below:

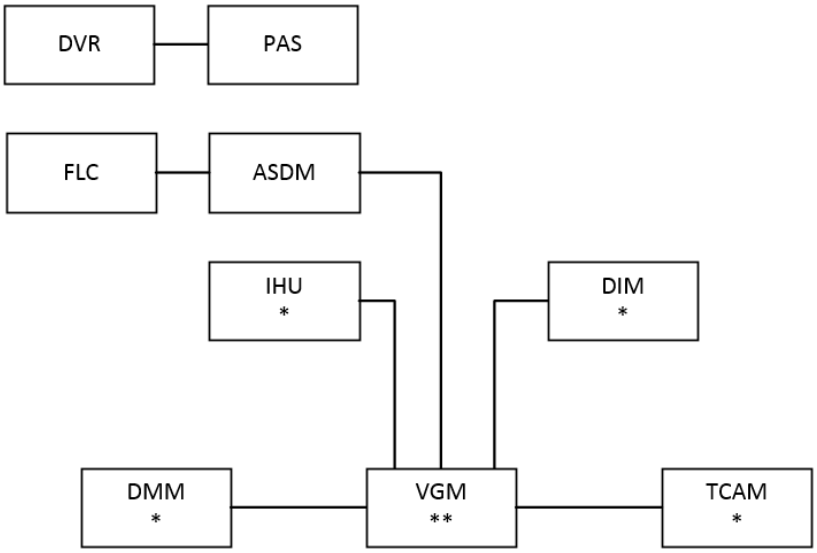


Figure:IP subnets of the GEEA 2.0 network

NOTE 1: In *Figure IP subnets of the vehicle network*, (the Ethernet capability of APIX2 link is not used for diagnostics. Instead, FlexRay is used for diagnostics.

NOTE 2: In *Figure IP subnets of the vehicle network* and in the *table "ECU Addresses"*, some IP addresses are marked with a * or a **.

* == Internal ECU-IP address used for Diagnostic (ISO13400) TCP clients (Diagnostic Server) to connect to the Edge Node (VGM).

** == Internal ECU-IP address used by Edge Node (VGM) acting as TCP Server (Diagnostic Client) offering Diagnostic Service

Overview of IP-address space :

The vehicle IP network shall use one subnet according to table below.

Network Friendly Name:	**VEHICLE IP NETWORK**
Subnet ID:	198.18.0.0 / 16

Usable IP addresses:	198.18.0.1 – 198.18.255.254
Netmask:	255.255.0.0
Broadcast address:	198.18.255.255
Default GW address:	VGM – Ethernet interface according to, ***Table:*** *ECU addresses – Showing IP-based ECU's*

IP address space allocation depending on physical network

The IP address range of 198.18.0.0 / 16 has been divided into different address ranges to be able to identify which physical communication carrier is used for a specific ECU. The tables below are used in conjunction with REQPROD 342169.

Example: ECU's which are connected to Ethernet will have an IP address range of: 198.18.32.1-198.18.47.255.

Network Friendly Name:	**MOST150**
Network ID:	198.18. **0**.0
IP addresses:	198.18.0.1 – 198.18.15.255

Network Friendly Name:	**APIX/LVDS**
Network ID:	198.18. **16**.0
IP addresses:	198.18.16.1 – 198.18.31.255

Network Friendly Name:	**Ethernet**
Network ID:	198.18. **32**.0
IP addresses:	198.18.32.1 – 198.18.47.255

Network Friendly Name:	**USB**
Network ID:	198.18. **48**.0
IP addresses:	198.18.48.1 – 198.18.63.255

Relationship between ECU addresses and IP addresses
The ECU address consist of 16 bits and are divided into 4 nibbles as follows: [Address range]. [Domain id].[Network id].[Ecu id]

NOTE: The addressing format for diagnostic request and response, source address (SA) and target address (TA), using Domain ID, Network ID and ECU ID is defined in [*UDS Services*].

The IP address v4 consist of 32 bits and are divided into 4 bytes. It is required for diagnostic purposes, that it shall be possible to calculate the ECU address from the IP address of a specific ECU. The IP addresses have been divided as follows:

198.18.{ssssdddd}.{nnnneeee}.

ssss = Consists of 4 bits, defines the start of the Network ID.
dddd = Consists of 4 bits, defines the Domain id
nnnn = Consists of 4 bits, defines the Network id
eeee = Consists of 4 bits, defines the ECU id

The ECU addresses are always represented in hexadecimal form and the IP addresses are written in decimal form. This means when calculating the ECU address from the IP address there is a need for conversion from hexadecimal to decimal.

Example:
ECU Address 0x1212, would mean IP address 198.18.2.18.

198.18.[S] [D].[N][E]

[S]=0x0 (Network Friendly Name = “MOST150”, Network ID: 198.18. **0**.0)
[D] = 0x2
[N] = 0x1
[E] = 0x2

Table: IP Address calculation

IP address calculation				
	198	**18**	**{ssssdddd}**	**{nnnneeee}**
Binary	1100 0110	0001 0010	0000 0010	0001 0010
Hex	0xC6	0x12	0x02	0x12
Decimal	198	18	2	18

Table: ECU addresses

ECU Name	ECU Alias	Diagnostic Network = Source network for SWDL	ECU Address	CAN ID	NAD (LIN)	Internal MAC Addresses	Internal IP Addresses
Air Condition Control Module	ACCM	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
Alternator Control Module	ACM	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
Air Fragrance Unit	AFU	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Active Grill Shutter Module	AGM	ECM_LIN4	N/A	N/A	0x7F	N/A	N/A
Air ionizing unit	AIU	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Ambience Lights Module	ALM	CEM_LIN5	N/A	N/A	0x7F	N/A	N/A
Ambience Lights Module	ALM	CEM_LIN8	N/A	N/A	0x7F	N/A	N/A
Ambience Lights Module	ALM	Private ILCM_LIN1	N/A	N/A	0x7F	N/A	N/A
Air Particulate Matter Sensor	APMS	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Active Safety Domain Master	ASDM	Backbone FlexRay	0x1401	0x701	N/A	Ethernet: 02:00:00:00:14:01	Ethernet: 198.18.36.1

Active Spoiler Shutter Module	ASSM	ECM_LIN4	N/A	N/A	0x7F	N/A	N/A
Adjustable Steering Wheel Module	ASWM	CEM_LIN4	0x1B71	N/A	0x71	N/A	N/A
Audio Module	AUD	Private CAN	0x1212	0x712	N/A	N/A	Ethernet: 198.18.34.1 (Note 1)
Active Wing Module	AWM	CEM_LIN6	N/A	N/A	0x7F	N/A	N/A
Brake Booster Module	BBM	Backbone FlexRay	0x1602	0x702	N/A	N/A	N/A
Battery Backed-up Sounder	BBS	CEM_LIN2	0x1B51	N/A	0x51	N/A	N/A
Battery Circuit Four way Valve	BCFV	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
Battery Cooling Pump Front module	BCPF	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
Battery Cooling Pump Rear module	BCPR	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
Battery Circuit Three way Valve	BCTV	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
Battery Energy Control Module	BECM	Propulsion CAN	0x1635	0x735	N/A	N/A	N/A
Battery Electric Expansion Valve	BEXV	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Battery Monitoring Sensor	BMS	CEM_LIN6	0x1B61	N/A	0x61	N/A	N/A
Bluetooth Short Range Module	BSRM	Private TCAM_LIN1	N/A	N/A	0x7F	N/A	N/A
Climate Control Module	CCM	Body CAN	0x1A11	0x711	N/A	N/A	N/A
Center Console Switch Module	CCSM	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A
	CDD	Propulsion CAN	0x1650	0x750	N/A	N/A	N/A
Central Electronic Module	CEM	Backbone FlexRay	0x1A01	0x701	0x01	N/A	N/A
Condenser Electric Expansion Valve	CEXV	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Centre Stack Display	CSD	GMSL	0x1241	N/A	0x41	N/A	Ethernet: 198.18.34.1 (Note 1)
Console Switch Display Module	CSDM	GMSL	0x1231	N/A	0x31	N/A	Ethernet: 198.18.34.1 (Note 1)
Cloud Service Provider	CSP	N/A	N/A	N/A	N/A	N/A	N/A
Drivetrain Coolant Pump	DCP	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
Driver Door Module	DDM	Body CAN	0x1A12	0x712	0x12	N/A	N/A
Driver Door Switches	DDS	DDM_LIN1	N/A	N/A	0x7F	N/A	N/A
Differential Electronic Module	DEM	Propulsion CAN	0x1638	0x738	N/A	N/A	N/A
Driver Intelligent Change Colour Glass Module	DIG	DDM_LIN1	N/A	N/A	0x7F	N/A	N/A
Driver Information Module	DIM	Backbone FlexRay	0x1801	N/A	0x01	GPU SWDL via Ethernet (Note 2)	GPU SWDL via Ethernet (Note 2)

Driver Information Module	DIM	DIM Ethernet	0x1013	N/A	N/A	Ethernet: 02:00:00:00:10:13	Ethernet: 198.18.32.19
Driver Information Screen	DIS	DHU_LIN1	0x1213	N/A	DHU_LIN1: 0x12	N/A	N/A
Driver Information Screen	DIS	DIM_LIN1	0x1813	N/A	DIM_LIN1: 0x12	N/A	N/A
Driver Monitor Module	DMM	DMM Ethernet	0x1012	0x712	N/A	Ethernet: 02:00:00:00:10:12	Ethernet: 198.18.32.18 *
Drive Mode Switch Module	DMSM	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A
Driver Power Operated Door	DPOD	Body CAN	0x1A23	0x723	0x23	N/A	N/A
Driver Radar Module	DRM	DPOD_LIN1	0x1A31	0x731	0x31	N/A	N/A
Dedicated Short Range Communication	DSRC	Connectivity CAN HS	0x1021	0x721	N/A	N/A	N/A
Digital Video Recorder	DVR	Passive Safety CAN	0x1514	0x714	N/A	Ethernet: 02:00:00:00:15:14	198.18.37.20
Exhaust Acoustic Valve	EAV	ECM_LIN4	N/A	N/A	0x7F	N/A	N/A
Electrical motor Cooling Distribution Valve module	ECDV	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
Engine Control Module	ECM	Propulsion CAN	0x1630	0x730	0x30	N/A	N/A
Engine Coolant Pump Module	ECPM	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
Electrical motor Cooling Regulator Valve module	ECRV	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
E-clutch System	ECS	Propulsion CAN	0x1639	0x739	N/A	N/A	N/A
Electric Drivetrain Coolant Pump	EDCP	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
Evaporator Electric Expansion Valve	EEXV	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Electronic Gear Selector Module	EGSM	Propulsion CAN	0x1633	0x733	0x33	N/A	N/A
Exterior Sound Module	ESM	Propulsion CAN	0x163A	0x73A	N/A	N/A	N/A
Electric Vehicle Inlet Module	EVIM	Private CAN	N/A	N/A	N/A	N/A	N/A
Fast Charger Socket Indicator	FCSI	CEM_LIN7	N/A	N/A	0x7F	N/A	N/A
Forward Loooking Camera	FLC	CAN FD 2	0x1442	0x742	N/A	Ethernet: 02:00:00:00:14:42	Ethernet: 198.18.36.66
Forward Looking Radar	FLR	CAN FD 2	0x1444	0x744	N/A	N/A	N/A
Foot Movement Detection Module	FMDM	CEM_LIN6	N/A	N/A	0x7F	N/A	N/A

FingerPrint Module	FPM	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A
Front Side Radar Left	FSRL	CAN FD 1	0x1431	0x731	N/A	N/A	N/A
Front Side Radar Right	FSRR	CAN FD 1	0x1430	0x730	N/A	N/A	N/A
HVAC Blower Module Front	HBMF	CCM_LIN1	N/A	N/A	0x7F	N/A	N/A
Headlamp Control Module Left	HCML	Body Exposed CAN	0x1BB3	0x7B3	N/A	N/A	N/A
Headlamp Control Module Right	HCMR	Body Exposed CAN	0x1BB4	0x7B4	N/A	N/A	N/A
Heater Circuit Three way Valve	HCTV	ECM_LIN1	N/A	N/A	0x7F	N/A	N/A
HVAC Flap Actuators Front	HFAF	CCM_LIN4	N/A	N/A	0x7F	N/A	N/A
Haptic Node Module	HNM	GMSL	0x1232	N/A	0x32	N/A	Ethernet: 198.18.34.1 (Note 1)
Head Up Display	HUD	DHU_LIN1	0x1211	N/A	DHU_LIN1: 0x11	N/A	TBD (via DIM MCU or via DIM GPU?)
Head Up Display	HUD	DIM_LIN1	0x1811	N/A	DIM_LIN1: 0x11	N/A	TBD (via DIM MCU or via DIM GPU?)
High Voltage Coolant Heater	HVCH	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Inverter ERAD Module	IEM	Propulsion CAN	0x1637	0x737	N/A	N/A	N/A
Inverter Generator Module	IGM	Propulsion CAN	0x1636	0x736	N/A	N/A	N/A
Display Head Unit	DHU	Ethernet	0x1201	0x701	0x01	Ethernet: 02:00:00:02:12: 01	Ethernet: 198.18.34.1 *
Infotainment Head Unit	IHU	Ethernet	0x1201	0x701	0x01	Ethernet: 02:00:00:02:12: 01	Ethernet: 198.18.34.1 *
Interior Lighting Control Module	ILCM	Body CAN	0x1A30	0x730	0x30	N/A	N/A
Interior Movement Sensor	IMS	CEM_LIN2	0x1B52	N/A	0x52	N/A	N/A
Interia Monitor Unit	IMU	Private CAN	N/A	N/A	N/A	N/A	N/A
Interior Rear view Mirror Module	IRMM	CEM_LIN1	N/A	N/A	0x7F	N/A	N/A
Integrated Start Generate Module	ISGM	Propulsion CAN	0x163E	0x73E	N/A	N/A	N/A
Left Power Operated Door	LPOD	Body CAN	0x1A25	0x725	0x25	N/A	N/A
Motor Generator Module	MGM	Propulsion CAN	0x1631	0x731	N/A	N/A	N/A
Driver Massage Module	MMD	SMD_LIN1	0x1A32	0x732	0x32	N/A	N/A
Massage Module Passenger	MMP	SMP_LIN1	0x1A33	0x733	0x33	N/A	N/A
Massage Module Rear Left	MMRL	SMB_LIN1	0x1A34	0x734	0x34	N/A	N/A
Massage Module Rear Right	MMRR	SMB_LIN1	0x1A35	0x735	0x35	N/A	N/A
Medium Voltage	MVBM	Propulsion	0x163F	0x73F	N/A	N/A	N/A

Battery Module		CAN					
Medium Voltage Converter Module	MVCM	Propulsion CAN	0x163D	0x73D	N/A	N/A	N/A
Night Vision Module	NVM	Passive Safety CAN	0x1515	0x715	N/A	N/A	N/A
On Board Charger	OBC	Propulsion CAN	0x1634	0x734	N/A	N/A	N/A
Overhead Console	OHC	CEM_LIN3	N/A	N/A	0x7F	N/A	N/A
Occupant Weight Sensor	OWS	SRS_LIN1	N/A	N/A	0x7F	N/A	N/A
Parking Assist System	PAS	CAN FD 4	0x1460	0x760	N/A	02:00:00:00:14:60	198.18.36.96
Passenger Door Module	PDM	Body CAN	0x1A13	0x713	0x13	N/A	N/A
Passenger Intelligent Change Colour Glass Module	PIG	PDM_LIN1	N/A	N/A	0x7F	N/A	N/A
Air Particulate Matter Sensor Indoor	PMSI	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Air Particulate Matter Sensor Outdoor	PMSO	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Power Operated Tailgate	POT	Body CAN	0x1A15	0x715	N/A	N/A	N/A
Passenger Power Operated Door	PPOD	Body CAN	0x1A24	0x724	0x24	N/A	N/A
Passenger Radar Module	PRM	PPOD_LIN1	0x1A37	0x737	0x37	N/A	N/A
Power Steering Control Module	PSCM	Chassis CAN 1	0x1611	0x711	N/A	N/A	N/A
Running Board Module	RBM	CEM_LIN1	N/A	N/A	0x7F	N/A	N/A
Rear Light Control Module Left	RCML	Body Exposed CAN	0x1BB5	0x7B5	N/A	N/A	N/A
Rear Light Control Module Right	RCMR	Body Exposed CAN	0x1BB6	0x7B6	N/A	N/A	N/A
Rear Console Switch Module	RCSM	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A
Rear Driver Radar Module	RDRM	LPOD_LIN1	0x1A38	0x738	0x38	N/A	N/A
Rear Left Door Module	RLDM	Body CAN	0x1A21	0x721	0x21	N/A	N/A
Rear Left Intelligent Glass	RLIG	RLDM_LIN1	0x1A40	0x740	0x40	N/A	N/A
Rain and Light Sensor Module	RLSM	CEM_LIN1	N/A	N/A	0x7F	N/A	N/A
Rear View Mirror Display	RMD	Passive Safety CAN	0x1513	0x713	N/A	N/A	N/A
Restraint Module Left	RML	Passive Safety CAN	0x1510	0x710	N/A	N/A	N/A
Restraint Module Right	RMR	Passive Safety CAN	0x1511	0x711	N/A	N/A	N/A
Rear Power Operated Door	RPOD	Body CAN	0x1A26	0x726	0x26	N/A	N/A
Rear Passenger Radar Module	RPRM	RPOD_LIN1	0x1A39	0x739	0x39	N/A	N/A
Rear Right Door Module	RRDM	Body CAN	0x1A22	0x722	0x22	N/A	N/A

Rear Right Intelligent Glass	RRIG	RRDM_LIN1	0x1A41	0x741	0x41	N/A	N/A
Steering Angle Sensor	SAS	Chassis CAN 1	0x1616	0x716	N/A	N/A	N/A
Sun Blind Module Rear	SBMR	CEM_LIN3	N/A	N/A	0x7F	N/A	N/A
Steering Column Lock	SCL	Chassis CAN 2	0x1671	0x771	N/A	N/A	N/A
Sun Curtain Module	SCM	CEM_LIN3	N/A	N/A	0x7F	N/A	N/A
Slow Charger Socket Indicator	SCSI	CEM_LIN7	N/A	N/A	0x7F	N/A	N/A
Skylight Glass Top	SGT	CEM_LIN3	N/A	N/A	0x7F	N/A	N/A
Sunroof Intelligent Change Colour Glass Module	SIG	CEM_LIN3	N/A	N/A	0x7F	N/A	N/A
Seat Module Back	SMB	Body CAN	0x1A29	0x729	0x29	N/A	N/A
Seat Module Driver	SMD	Body CAN	0x1A27	0x727	0x27	N/A	N/A
Seat Module Passenger	SMP	Body CAN	0x1A28	0x728	0x28	N/A	N/A
Side Obstacle Detection Left	SODL	CAN FD 3	0x1450	0x750	N/A	N/A	N/A
Side Obstacle Detection Right	SODR	CAN FD 3	0x1451	0x751	N/A	N/A	N/A
Sun Roof Module	SRM	CEM_LIN3	N/A	N/A	0x7F	N/A	N/A
Supplementary Restraint System	SRS	Backbone FlexRay	0x1C01	0x701	0x01	N/A	N/A
Suspension Module	SUM	Chassis CAN 2	0x1670	0x770	N/A	N/A	N/A
Steering Wheel Module	SWM	CEM_LIN4	0x1B72	N/A	0x72	N/A	N/A
Steering Wheel Switch Module	SWSM	DIM_LIN1	N/A	N/A	0x7F	N/A	N/A
Transmission Actuator Control Module	TACM	Propulsion CAN	0x163C	0x73C	0x3C	N/A	N/A
Telematics & Connectivity Module	TCAM	TCAM Ethernet	0x1011	0x711	N/A	Ethernet: 02:00:00:00:10:11	Ethernet: 198.18.32.17 *
Thermal Cup Holder /	TCH	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A
Transmission Control Module	TCM	Propulsion CAN	0x1632	0x732	0x32	N/A	N/A
Third Row Console Switch Module	TCSM	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A
TOC Circuit Water Pump	TCWP	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
Trailer Module	TRM	Body CAN	0x1A17	0x717	N/A	N/A	N/A
Two Way Valve	TWV	ECM_LIN3	N/A	N/A	0x7F	N/A	N/A
VAC Blower Module Rear	VBMR	CCM_LIN2	N/A	N/A	0x7F	N/A	N/A
Vehicle Dynamics Domain Master	VDDM	Backbone FlexRay	0x1601	0x701	N/A	N/A	N/A
Vent Grill Actuator	VGA	CCM_LIN3	N/A	N/A	0x7F	N/A	N/A
Vehicle Gateway Module	VGM	Connectivity Ethernet	0x1001	0x701	N/A	Ethernet: 02:00:00:00:10:01	Ethernet: 198.18.32.1 **
Wiper Motor Module	WMM	CEM_LIN1	N/A	N/A	0x7F	N/A	N/A

Wireless Phone Charger/Near-field Communication	WPC/ NFC	IHU_LIN1	0x1221	N/A	0x21	N/A	Ethernet: 198.18.34.1 (Note 1)
Wireless Power Transfer	WPT	Propulsion CAN	0x163B	0x73B	N/A	N/A	N/A
Vehicle Refrigerator Module	VRM	IHU_LIN2	N/A	N/A	0x7F	N/A	N/A

Note: This requirement is valid for all ECU's.
Note 1: The VCM/VGM shall route diagnostic traffic to infotainment subnodes via the IHU. Therefore, the subnodes need to have the same IP address as the IHU on the Ethernet interface.
Note 2: The VCM/VGM shall route diagnostic traffic with destination to DIM GPU (0x1802) via APIX2 using the stated IP address.

Table: OBD addresses

ECU name according to SAE J1979	ECU name in GEEA2.0	11 bit address for Request/Response
ECM EngineControl	ECM	7E0/7E8
TCM TransmisCtrl	TCM	7E1/7E9
BECM B+EnergyCtrl	BECM	7E2/7EA
TBD	MVBM	7E2/7EA
DMCM DriveMotorCtrl	IEM	7E3/7EB
SGCM Start/GenCtrl	IGM	7E4/7EC
TBD	ECS	7E5/7ED
BCCM B+ChargerCtrl	OBC	7E5/7ED
BSCM BrakeSystem	VDDM	7E6/7EE
EBBC ElecBrkBstCtrl	BBM	7E7/7EF

1657 v37 **Network management CAN frame identifiers**

(36) => (37):
Description modified.

Current:Network management CAN frame identifiers(37)

An ECU using AUTOSAR CAN Network Management shall be able to receive network management frames with CAN identifiers ranging from 0x500 to 0x53F.

The CAN frame identifiers in the table below shall be used for transmission of network management frames containing the NM-PDU.

ECU Alias	NM CAN Network	NM CAN Frame ID	VFCVector ID

PSCM	AD Redundancy CAN	0x501	0x540
IMU	AD Redundancy CAN	0x502	0x541
FLC	AD Redundancy CAN	0x503	0x542
BBM	AD Redundancy CAN	0x504	0x543
CEM	Body CAN	0x501	0x541
CCM	Body CAN	0x502	0x542
DDM	Body CAN	0x503	0x543
PDM	Body CAN	0x504	0x544
DRM	Body CAN	0x505	0x545
POT	Body CAN	0x507	0x547
RLDM	Body CAN	0x509	0x549
TRM	Body CAN	0x50A	0x555
RRDM	Body CAN	0x510	0x550
SMD	Body CAN	0x512	0x552
SMP	Body CAN	0x513	0x553
SMB	Body CAN	0x514	0x554
ILCM	Body CAN	0x515	0x555
CEM	Body Exposed CAN	0x52A	0x540
HCML	Body Exposed CAN	0x531	0x541
HCMR	Body Exposed CAN	0x532	0x542
RCML	Body Exposed CAN	0x533	0x543
RCMR	Body Exposed CAN	0x534	0x544
RCMM	Body Exposed CAN	0x535	0x545
ASDM	CAN FD 1	0x501	0x540
FSRR	CAN FD 1	0x502	0x541
FSRL	CAN FD 1	0x503	0x542
ASDM	CAN FD 3	0x501	0x540
SODL	CAN FD 3	0x502	0x541
SODR	CAN FD 3	0x503	0x542
ASDM	CAN FD 4	0x501	0x540
PAS	CAN FD 4	0x502	0x541
ASDM	CAN FD 2	0x501	0x540
FLR	CAN FD 2	0x502	0x541
FLC	CAN FD 2	0x503	0x542
ECM	Chassis CAN 1	0x510	0x540
VDDM	Chassis CAN 1	0x521	0x545
PSCM1	Chassis CAN 1	0x523	0x541
SAS	Chassis CAN 1	0x527	0x544
PAS	Chassis CAN 1	0x528	0x546
ASDM	Chassis CAN 1	0x529	0x547
TCM	Chassis CAN 1	0x530	0x548
HVSC	Chassis CAN 1	0x531	0x549
MVEM	Chassis CAN 1	0x532	0x550
ECM	Chassis CAN 2	0x51F	0x546

VDDM	Chassis CAN 2	0x522	0x549
SUM	Chassis CAN 2	0x525	0x548
SCL	Chassis CAN 2	0x526	0x547
BBM	Chassis CAN 2	0x527	0x550
DSRC	Connectivity CAN	0x508	0x548
TCAM	Connectivity CAN	0x509	0x549
VGM	Connectivity CAN	0x533	0x543
AUD	Infotainment CAN	0x53A	0x54A
IHU	Infotainment CAN	0x53B	0x54B
SRS	Passive Safety CAN	0x50B	0x54B
RMR	Passive Safety CAN	0x50D	0x54D
RML	Passive Safety CAN	0x50E	0x54E
ASDM	Passive Safety CAN	0x50F	0x54F
DVR	Passive Safety CAN	0x511	0x551
DMM	Passive Safety CAN	0x512	0x552
RMD	Passive Safety CAN	0x513	0x553
EGSM	Propulsion CAN	0x516	0x545
TCM	Propulsion CAN	0x517	0x555
OBC	Propulsion CAN	0x518	0x552
BECM	Propulsion CAN	0x519	0x541
SRS	Propulsion CAN	0x51A	0x544
IGM	Propulsion CAN	0x51B	0x548
IEM	Propulsion CAN	0x51C	0x547
DEM	Propulsion CAN	0x51E	0x542
ECM	Propulsion CAN	0x520	0x543
ECS	Propulsion CAN	0x521	0x544
MVBM	Propulsion CAN	0x522	0x550
MVCM	Propulsion CAN	0x523	0x551
ISGM	Propulsion CAN	0x524	0x549
WPT	Propulsion CAN	0x525	0x556
VDDM	Propulsion CAN	0x526	0x554
ESM	Propulsion CAN	0x528	0x546
TACM	Propulsion CAN	0x52B	0x553
VGM	Propulsion CAN	0x52C	0x557
MGM	Propulsion CAN	0x52D	0x558
CDD	Propulsion CAN	0x52F	0x559

Note 1:
NM CAN Frames use IDs in range 0x501-0x53F, all unlisted in this range shall be considered

spare IDs.

Note 2:
This requirement applies to all CAN ECUs where AUTOSAR Network Management is used.

Previous: Network management CAN frame identifiers(36)

An ECU using AUTOSAR CAN Network Management shall be able to receive network management frames with CAN identifiers ranging from 0x500 to 0x53F.

The CAN frame identifiers in the table below shall be used for transmission of network management frames containing the NM-PDU.

ECU Alias	NM CAN Network	NM CAN Frame ID	VFCVector ID
PSCM	AD Redundancy CAN	0x501	0x540
IMU	AD Redundancy CAN	0x502	0x541
FLC	AD Redundancy CAN	0x503	0x542
BBM	AD Redundancy CAN	0x504	0x543
CEM	Body CAN	0x501	0x541
CCM	Body CAN	0x502	0x542
DDM	Body CAN	0x503	0x543
PDM	Body CAN	0x504	0x544
DPOD	Body CAN	0x505	0x545
LPOD	Body CAN	0x506	0x546
POT	Body CAN	0x507	0x547
PPOD	Body CAN	0x508	0x548
RLDM	Body CAN	0x509	0x549
TRM	Body CAN	0x50A	0x555
RRDM	Body CAN	0x510	0x550
RPOD	Body CAN	0x511	0x551
SMD	Body CAN	0x512	0x552
SMP	Body CAN	0x513	0x553
SMB	Body CAN	0x514	0x554
ILCM	Body CAN	0x515	0x555
CEM	Body Exposed CAN	0x52A	0x540
HCML	Body Exposed CAN	0x531	0x541
HCMR	Body Exposed CAN	0x532	0x542
RCML	Body Exposed CAN	0x533	0x543
RCMR	Body Exposed CAN	0x534	0x544
ASDM	CAN FD 1	0x501	0x540
FSRR	CAN FD 1	0x502	0x541
FSRL	CAN FD 1	0x503	0x542
ASDM	CAN FD 3	0x501	0x540
SODL	CAN FD 3	0x502	0x541
SODR	CAN FD 3	0x503	0x542
ASDM	CAN FD 4	0x501	0x540

PAS	CAN FD 4	0x502	0x541
ASDM	CAN FD 2	0x501	0x540
FLR	CAN FD 2	0x502	0x541
FLC	CAN FD 2	0x503	0x542
ECM	Chassis CAN 1	0x510	0x540
VDDM	Chassis CAN 1	0x521	0x545
PSCM1	Chassis CAN 1	0x523	0x541
SAS	Chassis CAN 1	0x527	0x544
PAS	Chassis CAN 1	0x528	0x546
ASDM	Chassis CAN 1	0x529	0x547
TCM	Chassis CAN 1	0x530	0x548
ECM	Chassis CAN 2	0x51F	0x546
VDDM	Chassis CAN 2	0x522	0x549
SUM	Chassis CAN 2	0x525	0x548
SCL	Chassis CAN 2	0x526	0x547
BBM	Chassis CAN 2	0x527	0x550
DSRC	Connectivity CAN	0x508	0x548
TCAM	Connectivity CAN	0x509	0x549
VGM	Connectivity CAN	0x533	0x543
AUD	Infotainment CAN	0x53A	0x54A
IHU	Infotainment CAN	0x53B	0x54B
SRS	Passive Safety CAN	0x50B	0x54B
RMR	Passive Safety CAN	0x50D	0x54D
RML	Passive Safety CAN	0x50E	0x54E
ASDM	Passive Safety CAN	0x50F	0x54F
NVM	Passive Safety CAN	0x510	0x550
DVR	Passive Safety CAN	0x511	0x551
DMM	Passive Safety CAN	0x512	0x552
RMD	Passive Safety CAN	0x513	0x553
EGSM	Propulsion CAN	0x516	0x545
TCM	Propulsion CAN	0x517	0x555
OBC	Propulsion CAN	0x518	0x552
BECM	Propulsion CAN	0x519	0x541
SRS	Propulsion CAN	0x51A	0x544
IGM	Propulsion CAN	0x51B	0x548
IEM	Propulsion CAN	0x51C	0x547
DEM	Propulsion CAN	0x51E	0x542
ECM	Propulsion CAN	0x520	0x543
ECS	Propulsion CAN	0x521	0x544
MVBM	Propulsion CAN	0x522	0x550
MVCM	Propulsion CAN	0x523	0x551
ISGM	Propulsion CAN	0x524	0x549
WPT	Propulsion CAN	0x525	0x556
VDDM	Propulsion CAN	0x526	0x554

ESM	Propulsion CAN	0x528	0x546
TACM	Propulsion CAN	0x52B	0x553
VGM	Propulsion CAN	0x52C	0x557
MGM	Propulsion CAN	0x52D	0x558
CDD	Propulsion CAN	0x52F	0x559

Note 1:
NM CAN Frames use IDs in range 0x501-0x53F, all unlisted in this range shall be considered spare IDs.

Note 2:
This requirement applies to all CAN ECUs where AUTOSAR Network Management is used.

1663 v7 **Bootloader routing - Spare networks**

(6) => (7):
Description modified.

Current:Bootloader routing - Spare networks(7)

The ECU shall prepare the bootloader for adding new networks according to below.

```
1 : D : N : E
|   |   |   |
|   |   |   --- ECU ID
|   |   --- Network ID
|   --- Domain ID
--- Always 1
```

VGM
0x1010-0x101F--> Connectivity Ethernet
0x1100-0x11FF--> Connectivity Ethernet
0x1200-0x12FF--> Connectivity Ethernet
0x1300-0x13FF--> Connectivity Ethernet
0x1400-0x14FF--> Flexray
0x1500-0x15FF-> Flexray
0x1600-0x16FF--> Flexray
0x1800-0x18FF--> Flexray
0x1A00-0x1AFF--> Flexray
0x1B00-0x1BFF--> Flexray
0x1C00-0x1CFF--> Flexray
0x1D00-0x1DFF--> Flexray
0x1E00-0x1EFF--> Flexray

VDDM

0x161x -> Chassis CAN 2
0x162x -> Chassis CAN 2 (Spare)
0x163x -> Propulsion CAN
0x164x -> Propulsion CAN (Spare)
0x165x -> Propulsion CAN
0x166x -> Propulsion CAN
0x167x -> Chassis CAN 1
0x168x -> Chassis CAN 1 (Spare)

CEM
0x1A02-0x1AFF--> Body CAN
0x1B10-0x1B1F--> LIN1
0x1B20-0x1B2F--> LIN3
0x1B30-0x1B3F--> LIN0/K-Line
0x1B40-0x1B4F--> LIN5
0x1B50-0x1B5F--> LIN2
0x1B60-0x1B6F--> LIN6
0x1B70-0x1B73--> LIN4
0x1B74-0x1B79--> LIN7
0x1B7A-0x1B7F--> LIN8
0x1BB0-0x1BBF--> Body CAN Exposed

ASDM
0x1410-0x141F--> Flexray
0x1420-0x142F--> Safety CAN FD (Spare)
0x1430-0x143F--> Safety CAN FD 1
0x1440-0x144F--> Safety CAN FD 2
0x1450-0x145F--> Safety CAN FD 3
0x1460-0x146F--> Safety CAN FD 4
0x1470-0x147F--> Safety CAN FD (Spare)
0x1510-0x151F--> Passive Safety CAN

Note: This requirement is applicable for ASDM, CEM, VGM and VDDM.

Previous: Bootloader routing - Spare networks(5)

The ECU shall prepare the bootloader for adding new networks according to below.

```
1 : D : N : E
|   |   |   |
|   |   |   --- ECU ID
|   |   --- Network ID
|   --- Domain ID
--- Always 1
```

VGM
0x1010-0x101F--> Connectivity Ethernet
0x1100-0x11FF--> Connectivity Ethernet
0x1200-0x12FF--> Connectivity Ethernet

0x1300-0x13FF--> Connectivity Ethernet
0x1400-0x14FF--> Flexray
0x1600-0x16FF--> Flexray
0x1800-0x18FF--> Flexray
0x1A00-0x1AFF--> Flexray
0x1B00-0x1BFF--> Flexray
0x1C00-0x1CFF--> Flexray
0x1D00-0x1DFF--> Flexray
0x1E00-0x1EFF--> Flexray

VDDM
0x161x -> Chassis CAN 1
0x162x -> Chassis CAN 1 (Spare)
0x163x -> Propulsion CAN
0x164x -> Propulsion CAN (Spare)
0x165x -> Propulsion CAN
0x166x -> Propulsion CAN
0x167x -> Chassis CAN 2
0x168x -> Chassis CAN 2 (Spare)

CEM
0x1A02-0x1AFF--> Body CAN
0x1B10-0x1B1F--> LIN1
0x1B20-0x1B2F--> LIN3
0x1B30-0x1B3F--> LIN0/K-Line
0x1B40-0x1B4F--> LIN5
0x1B50-0x1B5F--> LIN2
0x1B60-0x1B6F--> LIN6
0x1B70-0x1B73--> LIN4
0x1B74-0x1B79--> LIN7
0x1B7A-0x1B7F--> LIN8
0x1BB0-0xBBFF--> Body CAN Exposed

ASDM
0x1410-0x141F--> Flexray
0x1420-0x142F--> Safety CAN FD
0x1430-0x143F--> Safety CAN FD 1
0x1440-0x144F--> Safety CAN FD 2
0x1450-0x145F--> Safety CAN FD 3
0x1460-0x146F--> Safety CAN FD 4
0x1470-0x147F--> Safety CAN FD

Note: This requirement is applicable for ASDM, CEM, VGM and VDDM.

1664 v7 **Deployment of the J2534 requirements**

(6) => (7):
Description modified.

Current:Deployment of the J2534 requirements(7)

The VGM shall implement the J2534 requirements according to *Diagnostic and Bootloader Gateway Specification* for reference see Diagnostics & ECU Platform - Requisite documents.

Target ECU	ECU addr	J2534 Request	J2534 Response
VDDM	0x1601	0x1DD01601	0x1EC02E80
BBM	0x1602	0x1DD01602	0x1EC04E80
ECM	0x1630	0x1DD01630	0x1EC60E80
MGM	0x1631	0x1DD01631	0x1EC62E80
TCM	0x1632	0x1DD01632	0x1EC64E80
OBC	0x1634	0x1DD01634	0x1EC68E80
BECM	0x1635	0x1DD01635	0x1EC6AE80
IGM	0x1636	0x1DD01636	0x1EC6CE80
IEM	0x1637	0x1DD01637	0x1EC6EE80
ISGM	0x163E	0x1DD0163E	0x1EC7CE80
CDD	0x1650	0x1DD01650	0x1ECA0E80
MVEM	0x1651	0x1DD01651	0x1ECA2E80
MVBM	0x1652	0x1DD01652	0x1ECA4E80
Functional addr.	0x1FFF	0x1DD01FFF	N/A

Note: This requirement is valid for the VGM.

Previous: Deployment of the J2534 requirements(6)

The VGM shall implement the J2534 requirements according to *Diagnostic and Bootloader Gateway Specification* for reference see Diagnostics & ECU Platform - Requisite documents.

Target ECU	ECU addr	J2534 Request	J2534 Response
VGM	0x1001	0x1DD01001	0x1E002E80
TCAM	0x1011	0x1DD01011	0x1E022E80
DMM	0x1012	0x1DD01012	0x1E024E80
DIM	0x1013	0x1DD01013	0x1E026E80
DSRC	0x1021	0x1DD01021	0x1E042E80
IHU	0x1201	0x1DD01201	0x1E402E80
AUD	0x1212	0x1DD01212	0x1E424E80
WPC/NFC	0x1221	0x1DD01221	0x1E442E80
CSDM	0x1231	0x1DD01231	0x1E462E80
HNM	0x1232	0x1DD01232	0x1E464E80
CSD	0x1241	0x1DD01241	0x1E482E80
ASDM	0x1401	0x1DD01401	0x1E802E80
FSRR	0x1430	0x1DD01430	0x1E860E80
FSRL	0x1431	0x1DD01431	0x1E862E80
FLC	0x1442	0x1DD01442	0x1E884E80
FLR	0x1444	0x1DD01444	0x1E888E80
SODL	0x1450	0x1DD01450	0x1E8A0E80

SODR	0x1451	0x1DD01451	0x1E8A2E80
PAS	0x1460	0x1DD01460	0x1E8C0E80
SRS	0x1501	0x1DD01501	0x1EA02E80
RML	0x1510	0x1DD01510	0x1EA20E80
RMR	0x1511	0x1DD01511	0x1EA22E80
RMD	0x1513	0x1DD01513	0x1EA26E80
DVR	0x1514	0x1DD01514	0x1EA28E80
NVM	0x1515	0x1DD01515	0x1EA30E80
VDDM	0x1601	0x1DD01601	0x1EC02E80
PSCM	0x1611	0x1DD01611	0x1EC22E80
SAS	0x1616	0x1DD01616	0x1EC2CE80
ECM	0x1630	0x1DD01630	0x1EC60E80
MGM	0x1631	0x1DD01631	0x1EC62E80
TCM	0x1632	0x1DD01632	0x1EC64E80
EGSM	0x1633	0x1DD01633	0x1EC66E80
OBC	0x1634	0x1DD01634	0x1EC68E80
BECM	0x1635	0x1DD01635	0x1EC6AE80
IGM	0x1636	0x1DD01636	0x1EC6CE80
IEM	0x1637	0x1DD01637	0x1EC6EE80
DEM	0x1638	0x1DD01638	0x1EC70E80
ECS	0x1639	0x1DD01639	0x1EC72E80
ESM	0x163A	0x1DD0163A	0x1EC74E80
WPT	0x163B	0x1DD0163B	0x1EC76E80
TACM	0x163C	0x1DD0163C	0x1EC78E80
MVCM	0x163D	0x1DD0163D	0x1EC7AE80
ISGM	0x163E	0x1DD0163E	0x1EC7CE80
MVBM	0x163F	0x1DD0163F	0x1EC7EE80
CDD	0x1650	0x1DD01650	0x1ECA0E80
SUM	0x1670	0x1DD01670	0x1ECE0E80
SCL	0x1671	0x1DD01671	0x1ECE2E80
BBM	0x1701	0x1DD01701	0x1EE02E80
DIM	0x1801	0x1DD01801	0x1F002E80
HUD	0x1811	0x1DD01811	0x1F022E80
DIS	0x1812	0x1DD01812	0x1F024E80
CEM	0x1A01	0x1DD01A01	0x1F402E80
CCM	0x1A11	0x1DD01A11	0x1F422E80
DDM	0x1A12	0x1DD01A12	0x1F424E80
PDM	0x1A13	0x1DD01A13	0x1F426E80
POT	0x1A15	0x1DD01A15	0x1F42AE80
TRM	0x1A17	0x1DD01A17	0x1F42EE80
RLDM	0x1A21	0x1DD01A21	0x1F442E80
RRDM	0x1A22	0x1DD01A22	0x1F444E80
DPOD	0x1A23	0x1DD01A23	0x1F446E80
PPOD	0x1A24	0x1DD01A24	0x1F448E80

LPOD	0x1A25	0x1DD01A25	0x1F44AE80
RPOD	0x1A26	0x1DD01A26	0x1F44CE80
SMD	0x1A27	0x1DD01A27	0x1F44EE80
SMP	0x1A28	0x1DD01A28	0x1F450E80
SMB	0x1A29	0x1DD01A29	0x1F452E80
ILCM	0x1A30	0x1DD01A30	0x1F460E80
DRM	0x1A31	0x1DD01A31	0x1F456E80
MMD	0x1A32	0x1DD01A32	0x1F458E80
MMP	0x1A33	0x1DD01A33	0x1F466E80
MMRL	0x1A34	0x1DD01A34	0x1F462E80
MMRR	0x1A35	0x1DD01A35	0x1F464E80
PRM	0x1A37	0x1DD01A37	0x1F468E80
RDRM	0x1A38	0x1DD01A38	0x1F470E80
RPRM	0x1A39	0x1DD01A39	0x1F472E80
RLIG	0x1A40	0x1DD01A40	0x1F480E80
RRIG	0x1A41	0x1DD01A41	0x1F482E80
BBS	0x1B51	0x1DD01B51	0x1F6A2E80
IMS	0x1B52	0x1DD01B52	0x1F6A4E80
BMS	0x1B61	0x1DD01B61	0x1F6C2E80
SWM	0x1B91	0x1DD01B91	0x1F722E80
ASWM	0x1B92	0x1DD01B92	0x1F724E80
HCML	0x1BB3	0x1DD01BB3	0x1F766E80
HCMR	0x1BB4	0x1DD01BB4	0x1F768E80
RCML	0x1BB5	0x1DD01BB5	0x1F76AE80
RCMR	0x1BB6	0x1DD01BB6	0x1F76CE80
Functional addr.	0x1FFF	0x1DD01FFF	N/A

Note: This requirement is valid for the VGM.

1665 v30 **SWDL time in aftermarket**

(29) => (30):
Description modified.

Current:SWDL time in aftermarket(30)

The figures below includes maximum time per ECU for erase software and download software. This requirement is valid on ECU level, assuming an ideal downloading tool. Supplier should do their best to make the SWDL time as short as possible. ECU's that do not appear in the list have no SWDL function until a new agreement is made with the base technology team.

Table: SWDL Time in aftermarket

ECU Name	ECU Alias	SWDL time in aftermarket
Active Safety Domain Master 1	ASDM1	4.5min
Active Safety Domain Master 2	ASDM2	18min
Active Safety Domain Master 3	ASDM3	26min
Audio Module	AUD	TBD

Brake Booster Module	BBM	80s
Battery Energy Control Module	BECM	TBD
Battery Monitoring Sensor	BMS	3-5min
Climate Control Module	CCM	150s
Charger and DCDC module	CDD	TBD
Central Electronic Module	CEM	55s
Centre Stack Display	CSD	TBD
Driver Door Module	DDM	Software file: 30s Parameter file: 10s
Differential Electronic Module	DEM	30s
Driver Information Module	DIM(GC)	APP:250s Calibration: 1800s
Driver Information Module	DIM(AC)	APP:60s Calibration: 20s
Driver Information Screen	DIS	TBD
Driver Monitor Module	DMM1	4min
Driver Power Operated Door	DPOD	TBD
Driver Radar Module	DRM	TBD
Dedicated Short Range Communication	DSRC	TBD
Digital Video Recorder	DVR	9min
Engine Control Module1	ECM1	195s
Engine Control Module2	ECM2	200s
Engine Control Module3	ECM3	247s
Electronic Gear Selector Module	EGSM	TBD
Exterior Sound Module	ESM	60s
Forward Loooking Camera1	FLC1	7-8.5min
Forward Loooking Camera2	FLC2	7-8.5min
Forward Looking Radar	FLR	80s
Front Side Radar Left	FSRL	120s
Front Side Radar Right	FSRR	120s
Headlamp Control Module Left1	HCML1	TBD
Headlamp Control Module Left2	HCML2	TBD
Headlamp Control Module Right1	HCMR1	TBD
Headlamp Control Module Right2	HCMR2	TBD
Head Up Display	HUD	App:650s Calibration:25s
High Voltage Secondary Conversion Module	HVSC	TBD
Inverter ERAD Module	IEM	3min
Inverter Generator Module	IGM	3min
Infotainment Head Unit	IHU	20min

Display Head Unit	DHU	<45min
Interior Lighting Control Module	ILCM	TBD
Integrated Start Generate Module	ISGM	5min
Left Power Operated Door	LPOD	TBD
Motor Generator Module	MGM	3min
Driver Massage Module	MMD	TBD
Massage Module Passenger	MMP	TBD
Medium Voltage Battery Module	MVBM	3-5min
Medium Voltage Converter Module	MVCM	3min
Medium Voltage Converter Module2	MVCM2	TBD
Middle Voltage Electrical Motor	MVEM	3min
On Board Charger	OBC	TBD
Parking Assist System1	PAS1	2-3min
Parking Assist System2	PAS2	2-3min
Parking Assist System3	PAS3	20min
Passenger Door Module	PDM	Software file: 30s Parameter file: 10s
Power Operated Tailgate	POT	30s
Passenger Power Operated Door	PPOD	TBD
Power Steering Control Module 1	PSCM1	60s
Power Steering Control Module 2	PSCM2	60s
Rear Light Control Module Left	RCML	TBD
Rear light Control Middle Module	RCMM	TBD
Rear Light Control Module Right	RCMR	TBD
Rear Left Door Module	RLDM	Software file: 30s Parameter file: 10s
Rear View Mirror Display	RMD	TBD
Restraint Module Left	RML	TBD
Restraint Module Right	RMR	TBD
Rear Power Operated Door	RPOD	TBD
Rear Right Door Module	RRDM	Software file: 30s Parameter file: 10s
Steering Angle Sensor	SAS	TBD
Steering Column Lock	SCL	20s
Seat Module Back	SMB	120s
Seat Module Driver	SMD	120s
Seat Module Passenger	SMP	120s
Side Obstacle Detection Left	SODL	120s

Side Obstacle Detection Right	SODR	120s
Supplementary Restraint System	SRS	50s
Suspension Module	SUM2	120s
Steering Wheel Module	SWM	180s
Transmission Actuator Control Module	TACM	60s
Telematics & Connectivity Module	TCAM	10min
Transmission Control Module1	TCM1	180s
Transmission Control Module2	TCM2	160s
Trailer Module	TRM	TBD
Vehicle Dynamics Domain Master	VDDM	4min
Vehicle Gateway Module	VGM	30s
Wireless Power Transfer	WPT	TBD

Notes:
If project or cluster is not specifically mentioned for an ECU then the given download time is valid for all project&/clusters that ECU is defined in.

Previous: SWDL time in aftermarket(28)

The figures below includes maximum time per ECU for erase software and download software. This requirement is valid on ECU level, assuming an ideal downloading tool. ECU's that do not appear in the list have zero second download time until a new agreement is made with the base technology team.

Table: SWDL Time in aftermarket

ECU Name	ECU Alias	SWDL time in aftermarket [Seconds]
Active Safety Domain Master	ASDM	30
Audio Module	AUD	56
Brake Booster Module	BBM	40
Battery Backed-up Sounder	BBS	50
Battery Energy Control Module	BECM	35
Battery Monitoring Sensor	BMS	78
Climate Control Module	CCM	34
Central Electronic Module	CEM	55
Centre Stack Display	CSD	100

Console Switch Display Module	CSDM	50
Driver Door Module	DDM	125
Differential Electronic Module	DEM	12
Driver Information Module	DIM	870
Driver Information Module	DIM	870
Driver Information Screen	DIS	870
Driver Monitor Module	DMM	30
Driver Power Operated Door	DPOD	30
Driver Radar Module	DRM	30
Dedicated Short Range Communication	DSRC	30
Digital Video Recorder	DVR	30
Engine Control Module	ECM	150
E-clutch System	ECS	30
Electronic Gear Selector Module	EGSM	14
Exterior Sound Module	ESM	35
Forward Loooking Camera	FLC	120
Forward Looking Radar	FLR	40
Front Side Radar Left	FSRL	30
Front Side Radar Right	FSRR	30
Headlamp Control Module Left	HCML	25
Headlamp Control Module Right	HCMR	25
Haptic Node Module	HNM	30
Head Up Display	HUD	240
Inverter ERAD Module	IEM	30
Inverter Generator Module	IGM	31
Infotainment Head Unit	IHU	1261
Interior Lighting Control Module	ILCM	30
Interior Movement Sensor	IMS	84
Integrated Start Generate Module	ISGM	30
Left Power Operated Door	LPOD	30
Motor Generator Module	MGM	30
Driver Massage Module	MMD	15
Massage Module Passenger	MMP	15
Massage Module Rear Left	MMRL	15
Massage Module Rear Right	MMRR	15
Medium Voltage Battery Module	MVBM	30
Medium Voltage Converter Module	MVCM	30
Night Vision Module	NVM	30

On Board Charger	OBC	75
Overhead Console	OHC	15
Parking Assist System	PAS	30
Passenger Door Module	PDM	125
Power Operated Tailgate	POT	30
Passenger Power Operated Door	PPOD	30
Passenger Radar Module	PRM	30
Power Steering Control Module	PSCM	40
Rear Light Control Module Left	RCML	30
Rear Light Control Module Right	RCMR	30
Rear Driver Radar Module	RDRM	30
Rear Left Door Module	RLDM	30
Restraint Module Left	RML	20
Restraint Module Right	RMR	20
Rear Power Operated Door	RPOD	30
Rear Passenger Radar Module	RPRM	20
Rear Right Door Module	RRDM	30
Steering Angle Sensor	SAS	20
Steering Column Lock	SCL	10
Seat Module Back	SMB	30
Seat Module Driver	SMD	30
Seat Module Passenger	SMP	30
Side Obstacle Detection Left	SODL	30
Side Obstacle Detection Right	SODR	30
Supplementary Restraint System	SRS	12
Suspension Module	SUM	30
Steering Wheel Module	SWM	57
Transmission Actuator Control Module	TACM	20
Telematics & Connectivity Module	TCAM	260
Transmission Control Module	TCM	80
Trailer Module	TRM	70
Vehicle Dynamics Domain Master	VDDM	45
Vehicle Gateway Module	VGM	36
Wireless Phone Charger/Near-field Communication	WPC/NFC	15
Wireless Power Transfer	WPT	30

Notes:
If project or cluster is not specifically mentioned for an ECU then the given download time is valid for all project&/clusters that ECU is defined in.

1666 v33 **SWDL time in factory**

(32) => (33):
Description modified.

Current:SWDL time in factory(33)

The figures below includes maximum time per ECU for erase software and download software. This requirement is valid on ECU level, assuming an ideal downloading tool.Supplier should do their best to make the SWDL time as short as possible. ECU's that do not appear in the list have no SWDL function until a new agreement is made with the base technology team.

Table: SWDL Time in factory

ECU Name	ECU Alias	SWDL time in factory
Active Safety Domain Master 1	ASDM1	0
Active Safety Domain Master 2	ASDM2	30s
Active Safety Domain Master 3	ASDM3	30s
Audio Module	AUD	0
Brake Booster Module	BBM	80s
Battery Energy Control Module	BECM	0
Battery Monitoring Sensor	BMS	0
Climate Control Module	CCM	0
Charger and DCDC module	CDD	0
Central Electronic Module	CEM	55s
Centre Stack Display	CSD	0
Driver Door Module	DDM	10s
Differential Electronic Module	DEM	0
Driver Information Module	DIM(GC)	0
Driver Information Module	DIM(AC)	30s
Driver Information Screen	DIS	0
Driver Monitor Module1	DMM1	120s
Driver Power Operated Door	DPOD	0
Driver Radar Module	DRM	0
Dedicated Short Range Communication	DSRC	120s
Digital Video Recorder	DVR	0
Engine Control Module1	ECM1	195s
Engine Control Module2	ECM2	200s
Engine Control Module3	ECM3	247s
Electronic Gear Selector Module	EGSM	0
Exterior Sound Module	ESM	0
Forward Loooking Camera1	FLC1	0
Forward Loooking Camera2	FLC2	0
Forward Looking Radar	FLR	0
Front Side Radar Left	FSRL	0
Front Side Radar Right	FSRR	0

Headlamp Control Module Left1	HCML1	0
Headlamp Control Module Left2	HCML2	0
Headlamp Control Module Right1	HCMR1	0
Headlamp Control Module Right2	HCMR2	0
Head Up Display	HUD	0
High Voltage Secondary Conversion Module	HVSC	0
Inverter ERAD Module	IEM	0
Inverter Generator Module	IGM	0
Infotainment Head Unit	IHU	1s
Display Head Unit	DHU	90s
Display Head Unit	DHU	90s
Interior Lighting Control Module	ILCM	0
Integrated Start Generate Module	ISGM	0
Left Power Operated Door	LPOD	0
Motor Generator Module	MGM	0
Driver Massage Module	MMD	0
Massage Module Passenger	MMP	0
Medium Voltage Battery Module	MVBM	0
Medium Voltage Converter Module	MVCM	0
Medium Voltage Converter Module2	MVCM2	0
Middle Voltage Electrical Motor	MVEM	0
On Board Charger	OBC	0
Parking Assist System1	PAS1	0
Parking Assist System2	PAS2	0
Parking Assist System3	PAS3	0
Passenger Door Module	PDM	10s
Power Operated Tailgate	POT	20s
Passenger Power Operated Door	PPOD	0
Power Steering Control Module 1	PSCM1	60s
Power Steering Control Module 2	PSCM2	60s
Rear Light Control Module Left	RCML	0
Rear light Control Middle Module	RCMM	0
Rear Light Control Module Right	RCMR	0
Rear Left Door Module	RLDM	10s
Rear View Mirror Display	RMD	0
Restraint Module Left	RML	0
Restraint Module Right	RMR	0
Rear Power Operated Door	RPOD	0
Rear Right Door Module	RRDM	10s
Steering Angle Sensor	SAS	0

Steering Column Lock	SCL	5s
Seat Module Back	SMB	15s
Seat Module Driver	SMD	15s
Seat Module Passenger	SMP	15s
Side Obstacle Detection Left	SODL	0
Side Obstacle Detection Right	SODR	0
Supplementary Restraint System	SRS	6s
Suspension Module	SUM2	60s
Suspension Module	SUM2	60s
Steering Wheel Module	SWM	0
Transmission Actuator Control Module	TACM	30s
Telematics & Connectivity Module	TCAM	20s
Transmission Control Module1	TCM1	88s
Transmission Control Module2	TCM2	60s
Trailer Module	TRM	0
Vehicle Dynamics Domain Master	VDDM	240s
Vehicle Gateway Module	VGM	0
Wireless Power Transfer	WPT	0

Notes:
SWDL time equal to "0" second means that the ECU is not allowed to download data in the factory. In this case the software shall be pre-loaded before delivery to the factory.
If project or cluster is not specifically mentioned for an ECU then the given download time is valid for all project&/clusters that ECU is defined in.

Previous: SWDL time in factory(31)

The figures below includes maximum time per ECU for erase software and download software. This requirement is valid on ECU level, assuming an ideal downloading tool. ECU's that do not appear in the list have zero second download time until a new agreement is made with the base technology team.

Table: SWDL Time in factory

ECU Name	ECU Alias	SWDL time in factory [Seconds]
Active Safety Domain Master	ASDM	15
Audio Module	AUD	15
Brake Booster Module	BBM	0
Battery Backed-up Sounder	BBS	0
Battery Energy Control Module	BECM	30

Battery Monitoring Sensor	BMS	0
Climate Control Module	CCM	34
Central Electronic Module	CEM	55
Centre Stack Display	CSD	11
Console Switch Display Module	CSDM	0
Driver Door Module	DDM	125
Differential Electronic Module	DEM	12
Driver Information Module	DIM	30
Driver Information Module	DIM	30
Driver Information Screen	DIS	0
Driver Monitor Module	DMM	0
Driver Power Operated Door	DPOD	0
Driver Radar Module	DRM	0
Dedicated Short Range Communication	DSRC	0
Digital Video Recorder	DVR	0
Engine Control Module	ECM	150
E-clutch System	ECS	0
Electronic Gear Selector Module	EGSM	14
Exterior Sound Module	ESM	0
Forward Loooking Camera	FLC	0
Forward Looking Radar	FLR	3
Front Side Radar Left	FSRL	0
Front Side Radar Right	FSRR	0
Headlamp Control Module Left	HCML	25
Headlamp Control Module Right	HCMR	25
Haptic Node Module	HNM	0
Head Up Display	HUD	0
Inverter ERAD Module	IEM	0
Inverter Generator Module	IGM	10
Infotainment Head Unit	IHU	240
Interior Lighting Control Module	ILCM	0
Interior Movement Sensor	IMS	0
Integrated Start Generate Module	ISGM	0
Left Power Operated Door	LPOD	0
Motor Generator Module	MGM	0
Driver Massage Module	MMD	0

Massage Module Passenger	MMP	0
Massage Module Rear Left	MMRL	0
Massage Module Rear Right	MMRR	0
Medium Voltage Battery Module	MVBM	0
Medium Voltage Converter Module	MVCM	0
Night Vision Module	NVM	0
On Board Charger	OBC	0
Overhead Console	OHC	0
Parking Assist System	PAS	0
Passenger Door Module	PDM	125
Power Operated Tailgate	POT	0
Passenger Power Operated Door	PPOD	0
Passenger Radar Module	PRM	0
Power Steering Control Module	PSCM	54
Rear Light Control Module Left	RCML	0
Rear Light Control Module Right	RCMR	0
Rear Driver Radar Module	RDRM	0
Rear Left Door Module	RLDM	0
Restraint Module Left	RML	5
Restraint Module Right	RMR	5
Rear Power Operated Door	RPOD	0
Rear Passenger Radar Module	RPRM	0
Rear Right Door Module	RRDM	0
Steering Angle Sensor	SAS	0
Steering Column Lock	SCL	5
Seat Module Back	SMB	0
Seat Module Driver	SMD	0
Seat Module Passenger	SMP	0
Side Obstacle Detection Left	SODL	0
Side Obstacle Detection Right	SODR	0
Supplementary Restraint System	SRS	6
Suspension Module	SUM	0
Steering Wheel Module	SWM	0
Transmission Actuator Control Module	TACM	20
Telematics & Connectivity Module	TCAM	5
Transmission Control Module	TCM	80
Trailer Module	TRM	0
Vehicle Dynamics Domain Master	VDDM	45
Vehicle Gateway Module	VGM	5
Wireless Phone Charger/Near-field Communication	WPC/NFC	0
Wireless Power Transfer	WPT	0

Notes:
SWDL time equal to "0" second means that the ECU is not allowed to download data in the

factory. In this case the software shall be pre-loaded before delivery to the factory.
If project or cluster is not specifically mentioned for an ECU then the given download time is valid for all project&/clusters that ECU is defined in.

3112 v4 Support for (un)compressed data

(3) => (4):
Attribute 'Legacy ID' modified: 53923v2 => 53923v3.
Description modified.

Current:Support for (un)compressed data(4)

To decrease the software download time of an ECU,compression of data stream to an ECU shall be used, while the ECU shall still support non-compressed data files.The ECU shall implement the decompression algorithm specified in the reference [Data Compression and Encryption].

Only the tool that makes the data file and the ECU that is receiving the content of the compressed data file need to implement the compression and decopression algorithms respectively.

The tester informs the ECU whether the data is compressed or not.

Previous: Support for (un)compressed data(3)

The ECU shall implement the decompression algorithm specified in the reference *[SWDL_3] Data Compression and Encryption*. The ECU shall still support non-compressed data.

3113 v3 Delta encoding

(2) => (3):
Attribute 'Legacy ID' modified: 396753v1 => 396753v2.
Description modified.

Current:Delta encoding(3)

Delta encoding and decoding method is required when a data file that is contained within a software delivery exceeds 10MB in size.

Previous: Delta encoding(2)

Delta encoding and decoding method is required when a data file that is contained within a software delivery exceeds 10MB in size or exceeds 10min in programming time.

Note: The programming time is calculated from the first EraseMemory/RequestDownload request to the last positively confirmed RequestTransferExit/CheckMemory response (SBL is excluded from this time calculation).

3135 v5 Request Download - restrictions on number of addressed logical blocks

(4) => (5):
Description modified.

Current:Request Download - restrictions on number of addressed logical blocks(5)

The ECU shall check that only one logical block is being updated for a single downloaded software part, i.e. one logical block is allowed to be addressed from a valid RequestDownload to a CheckMemory request is detected. If any memory outside that first logical block is addressed by a RequestDownload request(s), e.g. a second logical block or some location not defined by the ESS at all, the ECU shall abort the RequestDownload and inform the client via a NRC.

As the processed length (compressed/encrypted) length is used as memorySize parameter of the RequestDownload request, there are scenarios when it is not possible to detect this at the RequestDownload stage but identified later when the data is uncompressed/decrypted and about to be written to the memory. Anyhow, the download shall be aborted when it is detected.

To be able to recover when download sequence is aborted, for example trying download the same software part once again or continue downloading the next software part (another logical block), the ECU shall upon a rejected request enter a state where no logical block is active and wait for next request to erase or program the memory.

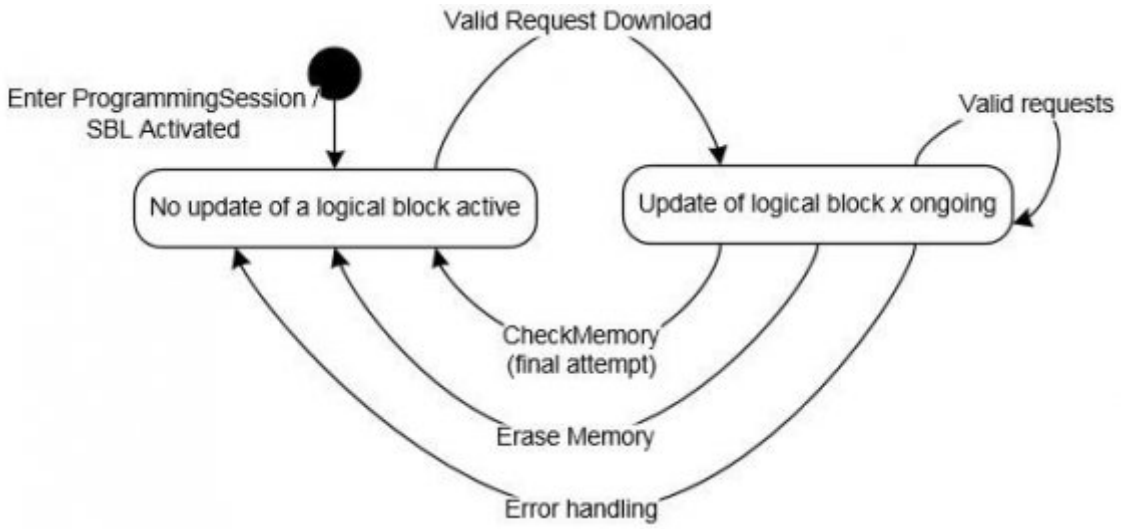


Figure - Simplified logical block active states overview

Notes:
- "Figure Simplified logical block active states overview" shows the principle only, details are omitted. 'Valid requests' are e.g. ReadDataByIdentifer requests, TransferData, RequestsTransferExit. "Error handling" includes the previously described scenario, where e.g. a RequestDownload addresses a logical block that is the 'active' block.
- The figure does not show the actual verification status of each logical block.

Example.
- the ECU is having a file based memory structure, not requiring an explicit erase memory request prior to request download. SBL is already verified and activated.

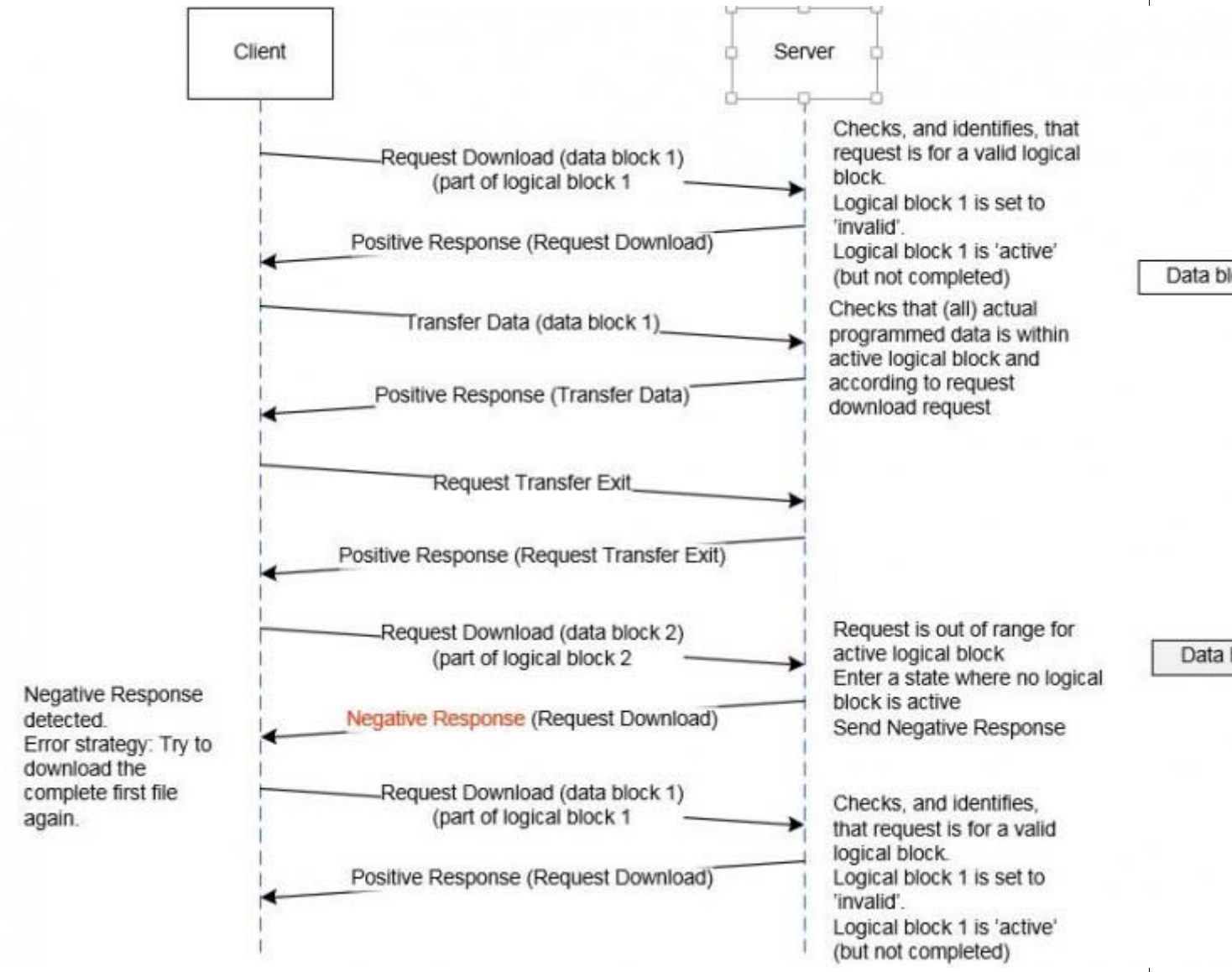


Figure - Example of ECU reaction after failed request (out of active logical block)

Previous: Request Download - restrictions on number of addressed logical blocks(4)

The ECU shall check that only one logical block is being updated for a single downloaded software part, i.e. one logical block is allowed to be addressed from the first RequestDownload to a CheckMemory request is detected. If any memory outside that first logical block is addressed by a RequestDownload request(s), e.g. a second logical block or some location not defined by the ESS at all, the ECU shall abort the RequestDownload.
Note: *As the processed length (compressed/encrypted) length is used as memorySize*

parameter of the RequestDownload, there are scenarios when it is not possible to detect this at the RequestDownload stage but identified later when the data is uncompressed/decrypted and about to be written to the memory. Anyhow, the download shall be aborted when it is detected.

3136 v2 RequestUpload - Restricted use

(1) => (2):
Attribute 'Legacy ID' modified: 429415v0 => 429415v1.
Description modified.

Current:RequestUpload - Restricted use(2)

If the RequestUpload service is supported by the ECU, the following areas shall be blocked and not possible to upload data from:
(i) Boot area and where the verification function is stored
(ii) Area where the public key is stored
(iii) Area where validity status information is stored
(iv) Area where any confidential key is stored

Note: For intellectual properties rights (e.g. when the software part is encrypted at download), the service shall be used with care.

Previous: RequestUpload - Restricted use(1)

If the RequestUpload service is supported by the ECU, the following areas shall be blocked and not possible to upload data from:
(i) Boot area and where the verification function is stored
(ii) Area where the public key is stored
(iii) Area where validity status information is stored

Note: For intellectual properties rights (e.g. when the software part is encrypted), the service shall be used with care.

3139 v3 Authenticity verification at every start-up

(2) => (3):
Description modified.

Current:Authenticity verification at every start-up(3)

If the ECU is able to fulfill the application start-up conditions, the authenticity verification shall be started every time the (main) application is to be started (via bootloader), typically at every ECU Reset or Power-Up. Hence, the validity status information shall not be permanently stored in the ECU.

Note: *Design consideration - if the verification using digital signatures is not feasible every start, the ECU could after a successful signature verification at software download calculate a message authentication code (MAC) over the verified logical block. During start, that MAC is verified. ECU internal secret keys shall be applied in that case, both the keys and MAC shall be stored in protected memory. This shall be considered if the ECU has the capabilities to calculate and store the MAC and still fulfill the start-up timing* If the verification cannot be performed at start (boot), it must be proven by the ECU supplier by providing e.g. hardware- or performance figures why it is not possible and how the system is hardened against any updates

except the defined software download sequence. If any software fails to be authenticated during boot, that sequence shall be aborted.

Design considerations: If the signature verification (as part of CheckMemory) could not be applied at start-up at any stage, the boot and application software(s) are to be signed (and verified) explicitly for verification at boot. These methods relies on solutions using either symmetric or asymmetric keys as provided by the ECU hardware. It relies on HW support to provide the initial root-of-trust (read only) code and a one-time-programmable verification key. Unless others are explicitly required, any others keys but the public key used at CheckMemory are supplier responsibility. Symmetric key solutions (e.g. AES-CMAC calculated over a logical block) requires that the confidential key is in a secured memory, protected from being altered and read as well as ensuring that a verification request origins from a trustworthy component in a secure environment.

Note 1: *Adaptation data that is changed when application is running is not part of start-up check.*
Example.

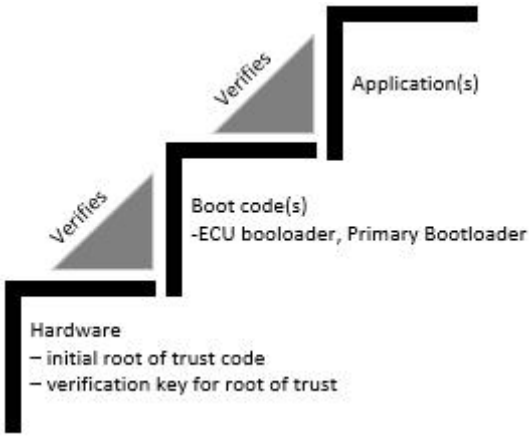


Figure - Chain-of-trust for authenticity verification at start

Note 2: *If the ECU calculates a cryptographic message authentication code (MAC, e.g. AES-CMAC) upon a valid signature verification at software download and verifies that MAC at every start-up, device unique keys must be used.*

Previous: Authenticity verification at every start-up(2)

If the ECU is able to fulfill the application start-up conditions, the authenticity verification shall be started every time the (main) application is to be started (via bootloader), typically at every ECU Reset or Power-Up. Hence, the validity status information shall not be permanently stored in the ECU.

Note: *Design consideration - if the verification using digital signatures is not feasible every start, the ECU could after a successful signature verification at software download calculate a message authentication code (MAC) over the verified logical block. During start, that MAC is verified. ECU internal secret keys shall be applied in that case, both the keys and MAC shall be stored in protected memory. This shall be considered if the ECU has the capabilities to calculate and store the MAC and still fulfill the start-up timing.*

3156 v3 **Public Key storage**

(2) => (3):
Attribute 'Legacy ID' modified: 366329v1 => 366329v2.
Description modified.

Current:Public Key storage(3)

The trust anchor, i.e. the public key, shall be stored in a protected (read-only) area, typically in a hardware secure module or a protected boot area. It shall be one-time-programmable.

Previous: Public Key storage(2)

The trust anchor, i.e. the public key or the root certificate, shall be stored in a protected area, typically in a hardware secure module or a protected boot area. If a hardware security module is deployed to the ECU this is required to be used for secure storage.

3157 v3 **Re-programmable initial development key**

(2) => (3):
Attribute 'Verification Method' modified: Test => Analysis AND Test.

Current:Re-programmable initial development key(3)

The ECU shall use an initial public key for development purposes. It is stored in the ECU by the ECU supplier (e.g. "virgin" ECU). When the initial development key is stored in the ECU, the public key for production shall always be possible to program. When a valid public key has been programmed, the initial development key must no longer be used by the ECU. The development key defined in *[General Software Authentication]* shall be applied unless else is explicitly specified.

Previous: Re-programmable initial development key(2)

The ECU shall use an initial public key for development purposes. It is stored in the ECU by the ECU supplier (e.g. "virgin" ECU). When the initial development key is stored in the ECU, the public key for production shall always be possible to program. When a valid public key has been programmed, the initial development key must no longer be used by the ECU. The development key defined in *[General Software Authentication]* shall be applied unless else is explicitly specified.

3160 v6 **Validity Status Information - Properties**

(5) => (6):
Attribute 'Legacy ID' modified: 400961v4 => 400961v6.
Description modified.

Current:Validity Status Information - Properties(6)

The validity status information for each logical block, i.e. if the block has passed- or failed the authenticity verification at software download, shall have a minimum length of 8 octets. One single value shall represent ‘Valid’ and all other values shall be interpreted as ‘Invalid’. ‘Valid’ must not be represented by setting all bytes to the same value, e.g. 0xFF, or to the value of erased memory.
If the authenticity verification is not performed at every boot sequence, the validity information must be stored in a non-volatile protected (secure) memory. The best available solution provided by the ECU shall be used (hardware security module or equivalent, EEPROM, a pattern in flash etc.). The validity status must not be altered by other means than by the trustworthy verifying component, i.e. it must not be altered by the application or by a tester tool or any other unauthorized entity addressing that area. It must also not be possible to read, or upload, using a tester or debugger tool.

Note: *An ECU unique value for 'Valid' shall be applied.*

Previous: Validity Status Information - Properties(5)

The validity status information for each logical block, i.e. if the block has passed- or failed the authenticity verification, shall have a minimum length of 8 octets. One single value shall represent ‘Valid’ and all other values shall be interpreted as ‘Invalid’. ‘Valid’ must not be represented by setting all bytes to the same value, e.g. 0xFF, or to the value of erased memory.
If the authenticity verification using digital signatures is not performed at every time the application is started, the validity information must be stored in a non-volatile protected (secure) memory. The best available solution provided by the ECU shall be used (hardware security module or equivalent, EEPROM, a pattern in flash etc.). The validity status must not be altered by other means than by the trustworthy verifying component, i.e. it must not be altered by the application or by a tester tool or any other unauthorized entity addressing that area. It must also not be possible to read, or upload, using a tester or debugger tool.

If the signature verification could not be applied at start-up, the ECU could for example calculate a message authentication (MAC) upon a valid signature verification at software download and verify that MAC at every start-up. That MAC must be in a protected memory.

Note *: It is recommended to use an ECU unique value for 'Valid'.*

3161 v4 Allocation of the Verification Block Table

(3) => (4):
Attribute 'Legacy ID' modified: 398372v1 => 398372v2.
Description modified.

Current:Allocation of the Verification Block Table(4)

A verification block table as defined Req ID1600: *Format of the Verification Block* Table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The

length (unprocessed) must be calculated by the ECU when the data is programmed.

Design consideration: When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

Previous: Allocation of the Verification Block Table(2)

The verification block table shall be stored in a separate data block. The start address shall be defined by the logical block definition, i.e. by the ECU Software Structure. The correct location of the verification block is ensured by including its address information (start and length) when it is being signed. The length (unprocessed) must be calculated by the ECU when the data is programmed. This data block that is transferred to the ECU must only contain the verification block table. When delta encoding is applied, the length of the Verification Block Table could be derived by using the start address and Number of data blocks.

3240 v3 Data compression Method 1 parameters

(2) => (3):
Attribute 'Legacy ID' modified: 52683v1 => 52683v2.
Attribute 'Purpose' modified: □□ÀÙ□鱸俏ฉヂ尐□ □□ㅅ 天ゖ□Ṇ□殰袾ᦓ □□寐鋮□ □ □燲瘢鰌□踵述□漫ᾰᵾ殞敍鎘ǫ□ㄱ□ᵢ□□姥鼌稓鋼□□□襮諗誂□鳩张□□锤:寸□□ 凡焆ᛘ鼻驀□ 遝□僈ᦷ腠□□綏□□Ť□□ 給逶□: 睄□虱胹蕊⊕๐瞅| □紜 => □□Àé□鱸佝檫碨鴐涿□毘□+□□ 鑽ᧇ□埕橝□雝糶ᧈ□□鯃臍曳越阒□蔂 徯ᦉ□熚躝□暯 □□□郾揌□帚龑□裖例韲□□ﻉ□螃坛諷□□□□□筌□暴订錣□□ 攫□□□ ᦋ 轗ᵋ Ȼ ᾳ□□ 鮂蔂₀◌ↄ略□□□ճ撜檫ᔕᦋ聞贻ᷬ□□栖□鼇篌。餞.

Current:Data compression Method 1 parameters(3)

The size of the sliding window in the LZSS compression method shall be 1024 bytes.

Previous: Data compression parameters(2)

The size of the sliding window in the LZSS compression method shall be 1024 bytes.

3241 v3 Data compression Method 1 parameters

(2) => (3):
Attribute 'Legacy ID' modified: 52687v1 => 52687v2.
Attribute 'Purpose' modified: □□ÀÙ□鱸俏ฉヂ尐□ □□ㅅ 天ゖ□Ṇ□殰袾ᦓ □□寐鋮□ □ □燲瘢鰌□踵述□漫ᾰᵾ殞敍鎘ǫ□ㄱ□ᵢ□□姥鼌稓鋼□□□襮諗誂□鳩张□□锤:寸□□ 凡焆ᛘ鼻驀□ 遝□僈ᦷ腠□□綏□□Ť□□ 給逶□: 睄□虱胹蕊⊕๐瞅| □紜 => □□Àé□鱸佝檫碨鴐涿□毘□+□□ 鑽ᧇ□埕橝□雝糶ᧈ□□鯃臍曳越阒□蔂 徯ᦉ□熚躝□暯 □□□郾揌□帚龑□裖例韲□□ﻉ□螃坛諷□□□□□筌□暴订錣□□ 攫□□□ ᦋ 轗ᵋ Ȼ ᾳ□□ 鮂蔂₀◌ↄ略□□□ճ撜檫ᔕᦋ聞贻ᷬ□□栖□鼇篌。餞.

Current:Data compression Method 1 parameters(3)

The size of the look-ahead buffer in the LZSS compression method shall be 16 bytes.

Previous: Data compression parameters(2)

The size of the look-ahead buffer in the LZSS compression method shall be 16 bytes.

3242 v3 Data compression Method 1 parameters

(2) => (3):
Description modified.

Current:Data compression Method 1 parameters(3)

First bit in a compressed data segment shall indicate if the following data in the segment is an uncompressed byte or a position/length-pair. ‘1’ indicates an uncompressed byte and ‘0’ a position/length-pair.

Previous: Data compression parameters(2)

The first bit in a compressed data segment shall indicate if the following data in the segment is an uncompressed byte or a position/length-pair. ‘1’ indicates an uncompressed byte and ‘0’ a position/length-pair.

3243 v3 Data compression method 1

(2) => (3):
Attribute 'Legacy ID' modified: 52677v1 => 52677v2.
Description modified.

Current:Data compression method 1(3)

The data compression method #1 shall be LZSS.

Previous: Data compression-decompression method(2)

The data compression/decompression method shall be LZSS.

3478 v5 Header Identifiers

(4) => (5):
Attribute 'Legacy ID' modified: 64694v3 => 64694v4.
Description modified.

Current:Header Identifiers(5)

Identifier is the name of a variable and IdentifierValue is the data assigned to the variable. Allowed identifiers are **vbf_version**, **header**, **description**, **sw_part_number**, **sw_current_part_number**, **sw_version**, **sw_current_version**, **sw_part_type**, **data_format_identifier**, **ecu_address**, **erase**, **call**, **verification_block_start**, **verification_block_length**, **verification_block_root_hash**, **sw_signature_dev**, **sw_signature,** **parameter_settings** and **file_checksum**. Duplicates of identifiers are not allowed.

Previous: Header Identifiers(4)

Identifier is the name of a variable and IdentifierValue is the data assigned to the variable. Allowed identifiers are **vbf_version**, **header**, **description**, **sw_part_number**, **sw_current_part_number**, **sw_version**, **sw_current_version**, **sw_part_type**, **data_format_identifier**, **ecu_address**, **erase**, **call**, **verification_block_start**, **verification_block_length**, **verification_block_root_hash**, **sw_signature_dev**, **sw_signature** and **file_checksum**. Duplicates of identifiers are not allowed.

3483 v2 Data section - start

(1) => (2):
Attribute 'Legacy ID' modified: 64725v0 => 64725v1.
Description modified.

Current:Data section - start(2)

After the header section ends, the data section immediately follows. The header section ends with a } character (ASCII 0x7D). The binary data section begins with the next byte immediately following this ASCII } character. The data section might be omitted, i.e. a data file could contain zero data blocks for some special cases.

Previous: Data section - start(1)

After the header section ends, the data section immediately follows. The header section ends with a } character (ASCII 0x7D). The binary data section begins with the next byte immediately following this ASCII } character.

3486 v3 Data block checksum

(2) => (3):
Attribute 'Legacy ID' modified: 64728v1 => 64728v3.
Description modified.

Current:Data block checksum(3)

The VBF file shall include a 2-byte checksum for every data block. The 2-byte checksum shall be calculated including all data bytes (excluding Start address, Length, and Checksum) in the data block. If a data compression method is used, the data block checksum shall be calculated using unprocessed data (i.e. uncompressed and/or not encrypted). Block checksums should not be used to verify the integrity of the VBF file.

The checksum algorithm to be used shall be the CRC16-CITT:

- Polynomial = 0x1021 (x^16+x^12+x^5+1)
- Initial value = 0xFFFF

For a fast CRC16-CITT calculation a look-up table implementation is the preferred solution. See Appendix A for "C-code" example.

Previous: Data block checksum(2)

The VBF file shall include a 2-byte checksum for every data block. The 2-byte checksum shall be calculated including all data bytes (excluding Start address, Length, and Checksum) in the data block. If data compression is used, the data block checksum shall be calculated using unprocessed data (i.e. uncompressed and/or not encrypted). The test tool shall not use these embedded checksums to validate the integrity of the file before the software download operation is performed with the VBF file.

The checksum algorithm to be used shall be the CRC16-CITT:

- Polynomial = 0x1021 (x^16+x^12+x^5+1)
- Initial value = 0xFFFF

For a fast CRC16-CITT calculation a look-up table implementation is the preferred solution. See Appendix A for "C-code" example.

3488 v6 **Header section - Format**

(5) => (6):
Attribute 'Legacy ID' modified: 64705v4 => 64705v5.
Description modified.

Current:Header section - Format(6)

The format of the header section is defined as follows (the grouping symbols [ ] indicates optional identifiers, i.e. the brackets are not to be part of the identifier defintion):

header {

[**description =** { *"text string row 1"* ,
"text string row 2" ,
"text string row 3" ,
:
"text string row n"
} ;]

sw_part_number = "part number";

sw_version = “version suffix”;

sw_part_type = *type*;

[**sw_current_part_number** **=** "installed part number";]
[**sw_current_version** **=** "installed version suffix";]

[**data_format_identifier** = *data compression and encryption method*;]

ecu_address = *ECU target address;*

[**erase** = { { *start address 1, length 1* },
{ *start address 2, length 2* },
:
{ *start address n, length n* }
} ;]

[**call** = *start address* **;**]

[**verification_block_root_hash** = *hash value of unprocessed data to be signed;*]

[**verification_block_start** = *start of the unprocessed verification block table;*]
[**verification_block_length** = *unprocessed length of the verification block table;*]
[**sw_signature_dev** = *software signature (test and development);*]
[**sw_signature** = *software signature;*]

[**parameter_settings** = { {*"parameter name 1", "value"*}, {*"parameter name n", "value"*} };]

file_checksum = *checksum*;
}

Previous: Header section - Format(5)

The format of the header section is defined as follows:

header {
[**description** = { *"text string row 1"* ,
"text string row 2" ,
"text string row 3" ,
:
"text string row n"
} ;]

sw_part_number = "part number";

sw_version = “version suffix”;

sw_part_type = *type*;

sw_current_part_number = "installed part number";

sw_current_version = "installed version suffix";

[**data_format_identifier** = *data compression and encryption method*;]

ecu_address = *ECU target address;*

[**erase** = { { *start address 1, length 1* },
{ *start address 2, length 2* },
:
{ *start address n, length n* }
} ;]

[**call** = *start address* ;]

verification_block_root_hash = *hash value of unprocessed data to be signed;*

verification_block_start = *start of the unprocessed verification block table;*

verification_block_length = *unprocessed length of the verification block table;*

sw_signature_dev = *software signature (test and development);*

sw_signature = *software signature;*

file_checksum = *checksum*;
}

3490 v4 **data_format_identifier**

(3) => (4):
Attribute 'Legacy ID' modified: 64712v2 => 64712v3.
Attribute 'Purpose' modified: □□Àõ□鱸偭饁碉紐嚆拭嗉　　醱□□稠ℵ崆暰□跗　鋼□□□□鯇□詖┙ 劍□　□禽Ⓠ犖я□□□□□錞綽勀谁覶　亢稷蹻爰□□□　ʙ□Ч賣□□礼　□棌□ꭒ飧□□繯□□　□懠□□湄鷭□监哱□　□　佢妣ᵀᵁ□. □薆鷲□桐□嬚襬ǀǀ𝔰轡穱崒瀠□壏呶　演□□□忨□　鬆低 => □□Àô□鱸佝位硙紐ᦓ□妄磛　罕糋鈹□□门瘬鼛ɮ　□□ϕ□　ᄇ抔:˙蓮□菄蜱燘ᵻ殰□咬ᦒ□□□□ᩣ□稀崗□鶊:ㄴ歈　颽快拸□ᵔᵒ　□□劇□□狛苴□ഌ蛟鯨嫂□鮐　猢歃玊ẇ□　脣□鍊葐□　□一|　瞁訊□□□バ□□坑ڀᵛ椆讖　□挬　趎□靸　□殆熹　□髒浒.
Description modified.

Current:data_format_identifier(4)

The data_format_identifier is optional and when is presented with a non-zero value in the header section, it represents that a data compression/encryption method is used. Valid values for the data_format_identifier are one byte long with a range of 0x00 to 0xFF with upper nibble representing "Data Compression Method" and lower nibble representing "Data Encryption Method". Refer to REQPROD 459337: RequestDownload - dataFormatIdentifier for valid values. If the identifier is not present in the header section, it represents that neither compression nor encryption is used.

Example of Valid Format:

1. data_format_identifier = 0x00; // represents no compression and no encryption

2. data_format_identifier = 0x10; // represents compression method #1 and no encryption

3. data_format_identifier = 0x20; // represents compression method #2 and no encryption

Example of invalid Format:

1. data_format_identifier = 0x123; // Invalid format (must be from 0x00 – 0xFF)

2. data_format_identifier = {0x10}; // Extra braces

3. data_format_identifier = 23; // Invalid format (integers shall be hexadecimal)

Previous: data_format_identifier(3)

The data_format_identifier is optional and when present with a non-zero value in the header section it represents that a data compression/encryption method is used. Valid values for the data_format_identifier are one byte long with a range of 0x00 to 0xFF with upper nibble representing "Data Compression Method" and lower nibble representing "Data Encryption Method". For the values, *see Data Compression and Encryption Specification, Delta Encoding Specification, Versatile Binary Format Specification - Requisite documents.* If the identifier is not present in the header section, it represents that neither compression nor encryption is used.

Note: Delta encoding shall not be used together with the **erase** identifier.
Note: Delta encoding shall not be used together with sw_part_type = **SBL or SSBL**.

Examples:

Valid Format:

1. data_format_identifier = 0x00; // represents no compression and no encryption

2. data_format_identifier = 0x10; // represents compression method #1 and no encryption

3. data_format_identifier = 0x17; // represents compression method #1 and encryption method #7

Delta encoded data examples:

4. data_format_identifier = 0x80 // represents Delta encoding and no compression method (e.g suppliers custom compression)

5. data_format_identifier = 0x90; // represents Delta encoding with compression method #1 and no encryption

Invalid Format:

1. data_format_identifier = 0x123; // Invalid format (must be from 0x00 – 0xFF)

2. data_format_identifier = {0x10}; // Extra braces

3. data_format_identifier = 23; // Invalid format (integers shall be hexadecimal)

Note: Requisite documents: *Data Compression and Encryption Specification, VBF*

Specification, Delta Encoding Specification

3498 v5 **sw_part_type**

(4) => (5):
Attribute 'Legacy ID' modified: 64709v3 => 64709v5.
Description modified.

Current:sw_part_type(5)

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue Reserved Words	Description
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file
VARIANT_INFO	Settings file (for offboard tool settings)

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:
1. sw_part_type = SIGCFG ;

2. sw_part_type = EXE ; // or any other valid type but "SBL" or "SSBL"
erase = { { 0x00004200, 0x00003CBF },

```
                { 0x00008010, 0x00002FF3 }
            };
```

Invalid Format:

```
1.      sw_part_type = exe;                         //  Invalid case

2.      sw_part_type = SBL;                         // Invalid combination of sw_part_type and
erase
    erase =  {   { 0x00004200, 0x00003CBF },
                 { 0x00008010, 0x00002FF3 }
             } ;

3.      sw_part_type = MYOWNTYPE;                    // Unknown IdentifierValue
```

Previous: sw_part_type(4)

The **sw_part_type** identifier indicates different types of a software parts in an ECU. Only the reserved word IdentifierValues defined in the table below shall be considered valid for the **sw_part_type** identifier.

Table: Software Part Types and Allowed Reserved Words

sw_part_type IdentifierValue Reserved Words	Description
CARCFG	Car configuration
CUSTOM	Customer parameters
DATA	Data or parameters (i.e., calibrations)
EXE	Executable
SBL	Secondary Bootloader
SIGCFG	Signal configuration
PBL	Primary Bootloader
SSBL	Special SBL which has the possibility to replace the PBL
TEST	Test program, (i.e. production test, diagnostics)
FBL	Flashable Bootloader
CERT	Certificate
PWD	Password for certificate
APP	Application file
MAP	Map file used for navigation
ESS	ECU Software Structure
SIGNED	Digital Signature file

The identifier **sw_part_type** in a single VBF file can only be assigned one of the reserved word values in the table above.

A VBF file with **sw_part_type = SBL or SSBL** is not allowed to use the identifier **erase**. These software types are downloaded to RAM and no erase operation shall be performed in this case.

Examples:

Valid Format:

1. sw_part_type = SIGCFG ;

2. sw_part_type = EXE ; // or any other valid type but "SBL" or "SSBL"
erase = { { 0x00004200, 0x00003CBF },
{ 0x00008010, 0x00002FF3 }
};

Invalid Format:
1. sw_part_type = exe; // Invalid case

2. sw_part_type = SBL; // Invalid combination of sw_part_type and erase
erase = { { 0x00004200, 0x00003CBF },
{ 0x00008010, 0x00002FF3 }
} ;

3. sw_part_type = MYOWNTYPE; // Unknown IdentifierValue

3499 v3 **sw_signature**

(2) => (3):
Attribute 'Legacy ID' modified: 417716v1 => 417716v2.
Description modified.

Current:sw_signature(3)

The sw_signature represents the digital signature according to the applied signature method [General Software Authentication]. The sw_signature shall be represented as an integer and the length is dependent of the signing method used.

Example:

Valid Format:
1. sw_signature =
0x514E3E55831E43E89D4118E2F29F4996C3A8ECBD11B1CDB8F4940AE4BF8BF866C28EA6F1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD963637203FAEA496F44C79EC11E706C3911749374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF34C7C297F6CFDAC7A436D6C0FA2B0551C032B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB47359A9D48F;

Note: There are use cases where a data file is not signed, e.g. prior to a file has been released, approved and signed.
Note: In production, e.g. at manufacturing and workshop systems, this information is to be transferred in a separate (non-vbf) file that is defined elsewhere.

Previous: sw_signature(2)

The **sw_signature** represents the digital signature according to the applied signature method *[General Software Authentication]*. The **sw_signature** shall be represented as an integer and the length is dependent of the signing method used.

Example:

Valid Format:
1. sw_signature =
0x514E3E55831E43E89D4118E2F29F4996C3A8ECBD11B1CDB8F4940AE4BF8BF866C28EA6F1B4B1458A7434788F0460434368ACE9863FDFF7F61E90AF2A95421A6757E553115EC00E37274066678AEFD54DD963637203FAEA496F44C79EC11E706C3911749374AED7BCB7C1768E2EB09400D6146D8771BC6A5C0B10D420E785BCAD91ED553537A43CFE9EC0A48B66179416D9615A1108588005568F46331E2F5BD13F10005A7F0CE2B1AA820CF34C7C297F6CFDAC7A436D6C0FA2B0551C032B63FAB04DD5A2E8F54B78BAD9CE609C0B022E93F716D857616D2FAAD2BBEEBDFB162A6330123FCB23FB52D56ECBD449DB89594B99768AD3B6CBDB222DB47359A9D48F;

Note: There are use cases where a data file is not signed, e.g. prior to a file has been released, approved and signed.

3503 v2 **verification_block_root_hash**

(1) => (2):
Attribute 'Legacy ID' modified: 417751v0 => 417751v1.
Description modified.

Current:verification_block_root_hash(2)

The verification_block_root_hash represents the hash value of the unprocessed data that is to be signed, according to the signing method [General Software Authentication]. The verification_block_root_hash shall be represented as an integer and the length is dependent of the hashing method used.

Example:

Valid Format:
1. verification_block_root_hash =
0xF1F62AC8EDFECD610234DBDA84F7561998ADC47A0F0EDE6CA23F0FC22974F9F4;

Note: There are use cases when a data file is not signed, e.g. prior to a file has been released, approved and signed.

Previous: verification_block_root_hash(1)

The **verification_block_root_hash** represents the hash value of the unprocessed data that is to be signed, according to the signing method *[General Software Authentication]*. The **verification_block_root_hash** shall be represented as an integer and the length is dependent of the signing method used.

Example:

Valid Format:
1. verification_block_root_hash = 0xF1F62AC8EDFECD610234DBDA84F7561998ADC47A0F0EDE6CA23F0FC22974F9F4;

Note: There are use cases where a data file is not signed, e.g. sw_part_type SIGNED or prior to a file has been released, approved and signed.

45485 v4 **Network management FR frame identifiers**

(3) => (4):
Description modified.

Current:Network management FR frame identifiers(4)

An ECU using AUTOSAR FR Network Management shall be able to receive network management frames with FR identifiers.

The FR frame identifiers in the table below shall be used for transmission of network management frames containing the NM-PDU.

Node on Backbone FlexRay	FR NM Node ID
DIM	0x51
VDDM	0x52
CEM	0x53
ASDM	0x54
SRS	0x55
IHU	0x56
VGM	0x57
BBM	0x59

Node on FLC FlexRay	FR NM Node ID
ASDM	0x54
FLC	0x55

Previous: Network management FR frame identifiers(3)

An ECU using AUTOSAR FR Network Management shall be able to receive network management frames with FR identifiers.

The FR frame identifiers in the table below shall be used for transmission of network management frames containing the NM-PDU.

Node on Backbone FlexRay	FR NM Node ID
DIM	0x51
VDDM	0x52
CEM	0x53
ASDM	0x54

SRS	0x55
IHU	0x56
VGM	0x57
BBM	0x59

96055 v2 **Terminating ECUs on CAN**

(1) => (2):
Description modified.

Current:Terminating ECUs on CAN(2)

List of terminating ECUs for each CAN network

Table: Terminating ECUs

CAN Network	Terminating ECU1	Terminating ECU2
Propulsion CAN	ECM	VGM
Chassis CAN1	ECM	SAS
Chassis CAN2	ECM	VDDM
Passive Safety CAN	ASDM	SRS
AD Redundancy CAN	PSCM	FLC
Safety CAN FD1	FSRL	FSRR
Safety CAN FD2	ASDM	FLR
Safety CAN FD3	SODL	SODR
Safety CAN FD4	ASDM	PAS
Body Exposed CAN	HCML	HCMR
Body CAN	CEM	CCM
DHU Private CAN1	DHU	SWTL
Connectivity CAN HS	VGM	TCAM
Private CAN1	FLC	IMU
Private CAN2	BECM	EVCC
Private Info CAN	IHU/DHU	AUD
Diagnostics CAN	VGM	Tester

Previous: Terminating ECUs on CAN(1)

List of terminating ECUs for each CAN network

Table: Terminating ECUs

CAN Network	Terminating ECU1	Terminating ECU2
Propulsion CAN	ECM	VGM

Chassis CAN1	ECM	SAS
Chassis CAN2	ECM	VDDM
Passive Safety CAN	ASDM	SRS
AD Redundancy CAN	PSCM	FLC
Safety CAN FD1	FSRL	FSRR
Safety CAN FD2	ASDM	FLR
Safety CAN FD3	SODL	SODR
Safety CAN FD4	ASDM	PAS
Body Exposed CAN	HCML	HCMR
Body CAN	CEM	CCM
Connectivity CAN HS	VGM	TCAM
Private CAN	FLC	IMU
Private CAN	IHU	AUD
Diagnostics CAN	VGM	Tester